

DATA STRUCTURES · ALGORITHMS · SYSTEM DESIGN

क्रम

KRAMA

The Ordered Path

180 Days from First Principles to the Product-Company Interview

NISHANT GUPTA - GRANTH SKILLS

VOLUME II OF EIGHT

Scanning and Searching

DAYS 027 - 050

DAY 001

THE ORDERED PATH

DAY 180



TWO TRACKS

24 OF 180 DAYS IN THIS VOLUME

180 STEPS

K R A M A

THE ORDERED PATH

क्रम

K R A M A

The Ordered Path

180 Days from First Principles to the Product-Company Interview

Nishant Gupta - Granth Skills

VOLUME II · SCANNING AND SEARCHING

DAYS 27 TO 50

KRAMA — THE ORDERED PATH

180 Days from First Principles to the Product-Company Interview

Volume II of eight: **Scanning and Searching**. Days 27 to 50.

Copyright © 2026 Nishant Gupta - Granth Skills. All rights reserved.

ABOUT THE TEXT

This edition is generated from the Krama course sources. The lessons, stories, code, diagrams and interview material are original work written for this course.

Practice problems are named, with their source and one line on what they test. No problem statement is reproduced. Trade marks and problem titles belong to their respective owners and are used here only to identify where a problem can be found.

ABOUT THE MAKING OF IT

Typeset from Markdown by rendering to a browser engine and printing to PDF. Every tool in that chain is free software and every typeface is published under the SIL Open Font Licence. Appendix F lists all of them with their licences.

Text face: Source Serif 4 — SIL Open Font Licence 1.1

Display face: Fraunces — SIL Open Font Licence 1.1

Sans face: Inter — SIL Open Font Licence 1.1

Monospace face: JetBrains Mono — SIL Open Font Licence 1.1

Devanagari: Noto Serif Devanagari — SIL Open Font Licence 1.1

Diagrams: Mermaid — MIT Licence

Code colouring: Pygments — BSD 2-Clause

EDITION

First edition, 2026. Set in Source Serif 4 on a 6.75 by 9.5 inch page.

क्रम

krama, Sanskrit: order; sequence; the step that follows the step before it.

YOU CANNOT SKIP A STEP. THAT IS THE WHOLE METHOD.

Why this book teaches two things at once

This book teaches two subjects at the same time, on purpose.

Most preparation separates them. You spend three months on data structures and algorithms, then start again from nothing on system design, and by the time the second subject makes sense the first has gone quiet. Interviews do not work that way. A single loop at a product company will ask you to reverse a linked list in one room and design a rate limiter in the next, on the same day, from the same head.

So every chapter here teaches one topic from each track, side by side. Day 42 gives you binary search and the object-oriented design of a parking lot. Day 130 gives you graph colouring and message queues. You will not finish this book with one strong subject and one weak one.

Who this is written for

Someone who has never studied this before. Not a computer science graduate. Not someone brushing up after a few years in the job.

If you need to be told what a server is, you are the reader. The first chapter explains what happens when a program runs. The fourteenth explains what happens when you type an address into a browser. Nothing before that is assumed.

The other half of that promise matters just as much: you are someone who will sit in a real interview. So the writing never stops at understanding. Every chapter ends by putting the idea in an interviewer's mouth, giving you the first ninety seconds of an answer, and writing out a full model response you can compare yours against.

The one question every chapter answers

Every lesson in this book was written to satisfy a single test:

Does this leave the reader able to answer one real interview question out loud, from memory, to a stranger?

That test decided what went in and, more usefully, what stayed out. There are no formal proofs here. No potential functions, no decision-tree lower bounds, no interpreter internals for their own sake. If a thing will not be asked, and does not make an asked thing clearer, it is not in this book.

What is different about how it is written

Every lesson opens with a story about a person. Not an analogy dressed up as a story: an actual scene, with a name in it, told in ordinary words with no technical vocabulary at all. Someone sorting socks. Someone in a canteen queue. Someone laying tables for a wedding. The story comes before the definition because a beginner needs somewhere to put the idea before they have the word for it.

Every solution is shown in full. This book does not hide answers. Code is built up in fragments of a few lines, each one explained, and then given again complete and ready to run. You are never told to work it out yourself as a test of character.

The arithmetic is always done out loud. Not “this is slow” but “this line runs about twenty-five million times”. Not “that is a lot of data” but `50M x 20 x 2KB = 2TB`. Numbers are worked through where you can see them.

Errors are printed exactly as they appear. When something breaks, the real message is on the page - `IndexError: list index out of range` - not a description of it. You will recognise it when your own screen shows it.

How this edition was made

The text is generated from the course repository, so the book and the working material cannot drift apart. The typesetting was done by rendering the pages in a browser engine and printing them to PDF.

Every part of the toolchain is free software, and every typeface is published under the SIL Open Font Licence. Appendix F lists all of it, with licences. No proprietary tool, font, or copyrighted third-party text was used to produce this book.

That extends to the practice problems. Problems are named, with their source and one line on what they are really testing. Their statements are never reproduced. You are meant to read them where they live, and solve them there.

How to use this book

The book is one hundred and eighty chapters long. One chapter is one day's work. That is the only pacing instruction in it, and it is deliberate: a day here is a unit of subject, not a unit of hours. Some days will take you longer than others. That is information about the topic, not about you.

The shape of every chapter

Each chapter holds three pieces, in this order.

1. **The DSA lesson.** Nine sections, always the same nine, always in the same order. Appendix A sets out what each one is for.
2. **The system design lesson.** The same nine sections, on a different subject.
3. **The practice sheet.** Named problems to solve, and questions to answer out loud.

The repetition is the point. After a week you stop navigating and start reading. You know that section 5 has the complete code, that section 7 has the mistakes you are about to make, and that section 8 is the interview itself.

How to read a lesson

Read the story first, and do not skip it. It is section 2, it has no technical words in it, and it is the part that makes the idea stick. People who skip it can define the term and cannot use it.

Type the code. Do not read it. Section 5 builds the solution in fragments and then gives it complete. Type the fragments as they come. Run the finished program. Change a number and run it again.

Say section 9 out loud. Every lesson ends with a recall card - the two or three sentences that are the whole lesson. Say them from memory, aloud, in full sentences. Not in your head. Aloud. An answer you have only ever thought is an answer you have never tested.

Treat section 8 as a rehearsal, not a reading. It gives you real interviewer phrasings, a script for the opening ninety seconds, the three follow-ups that usually come next, and a written-out model answer. Deliver yours before you read theirs.

How to use the practice sheet

Solve the problems where they live. The sheet names each one, tells you where to find it, and says in one line what it is really testing - which is usually not what it appears to be about.

Solve it, then say what you did out loud, as if someone had asked. If you cannot narrate your own solution, you have not finished the problem.

When you get stuck

Go back one chapter, not forward. This course is built so that each day rests on the one before it. A chapter that will not open is almost always a chapter whose foundation is thin, and the foundation is behind you.

If you want the vocabulary, Appendix D is a glossary of every term the book defines, with the day it first appears. If you want to see the whole route, Appendix C prints all one hundred and eighty days on two facing pages.

Revision

Certain chapters are revision days, and they are marked as such in the contents. Do not read past them. They exist because recall fades on a schedule, and the schedule is not negotiable.

Appendix B collects the recall cards for every part in this volume. Cover the answer, say it, then check. That is the whole drill.

Where this volume sits

This volume is marked in colour below.

- **Volume I — Foundations**
Days 1 to 26. How Code Costs, and How the Internet Works
- **Volume II — Scanning and Searching**
Days 27 to 50. Pointers, Windows and Binary Search · Databases from Zero
- **Volume III — Order and Access**
Days 51 to 77. Sorting, Hashing, Stacks and Queues · Objects, SOLID and Patterns
- **Volume IV — Links and Recursion**
Days 78 to 97. Linked Lists, Recursion and Backtracking · Low-Level Design
- **Volume V — Trees and Priorities**
Days 98 to 124. Trees, Heaps and Tries · Scaling Fundamentals
- **Volume VI — Graphs**
Days 125 to 142. Graph Algorithms · Distributed Systems Core
- **Volume VII — Optimal Choices**
Days 143 to 170. Dynamic Programming and Greedy · High-Level Design Case Studies
- **Volume VIII — The Last Mile**
Days 171 to 180. Bits, Maths and Mock Interviews · Reliability and Security

The nineteen parts

PART I	Foundations: how code costs	Days 1–8
PART II	Arrays	Days 9–18
PART III	Strings	Days 19–26
Part IV	Two pointers and sliding window	Days 27–36
Part V	Prefix sums	Days 37–41
Part VI	Binary search	Days 42–50
PART VII	Sorting	Days 51–59
PART VIII	Hashing: maps and sets	Days 60–67

PART IX	Stacks and queues	Days 68–77
PART X	Linked lists	Days 78–86
PART XI	Recursion and backtracking	Days 87–97
PART XII	Trees and binary search trees	Days 98–112
PART XIII	Heaps and priority queues	Days 113–119
PART XIV	Tries	Days 120–124
PART XV	Graphs	Days 125–142
PART XVI	Dynamic programming	Days 143–163
PART XVII	Greedy and intervals	Days 164–170
PART XVIII	Bits and maths	Days 171–176
PART XIX	Final mocks and revision	Days 177–180

What is in this volume

PART IV · TWO POINTERS AND SLIDING WINDOW	1
027 Two pointers: the idea	2
DSA Two pointers: the idea	3
DESIGN SQL you must know for interviews	19
PRACTICPractice — Two pointers: the idea	33
028 Opposite ends: pair sums on a sorted array	39
DSA Opposite ends: pair sums on a sorted array	40
DESIGN Joins, drawn	58
PRACTICPractice — Opposite ends: pair sums on a sorted array	73
029 Same direction: the read pointer and the write pointer	78
DSA Same direction: the read pointer and the write pointer	79
DESIGN Normalisation and when to break it	98
PRACTICPractice — Same direction: the read pointer and the write pointer	115
030 Fast and slow pointers	121
DSA Fast and slow pointers	122
DESIGN Indexes: how a database finds a row fast	140
PRACTICPractice — Fast and slow pointers	155
031 Fixed-size sliding window	161
DSA Fixed-size sliding window	162
DESIGN B-trees and why indexes are shaped that way	179
PRACTICPractice — Fixed-size sliding window	193
032 Variable-size sliding window	199
DSA Variable-size sliding window	200
DESIGN Query plans and the slow query	219
PRACTICPractice — Variable-size sliding window	234
033 Window plus hash map: the longest-substring family	240
DSA Window plus hash map: the longest-substring family	241
DESIGN Transactions and ACID	260
PRACTICPractice — Window plus hash map: the longest-substring family	275
034 At-most-K, and the exactly-K trick	279
DSA At-most-K, and the exactly-K trick	280
DESIGN Isolation levels and the anomalies they allow	295
PRACTICPractice — At-most-K, and the exactly-K trick	307
035 Choosing between two pointers and a window, under pressure	311
DSA Choosing between two pointers and a window, under pressure	312
DESIGN Locking and deadlocks	326

	PRACTICPractice — Choosing between two pointers and a window, under pressure	337
036	Two pointers revision and mock round	341
	DSA Two pointers revision and mock round	342
	DESIGN NoSQL: what it actually means	354
	PRACTICPractice — Two pointers revision and mock round	364
PART V · PREFIX SUMS		368
037	Prefix sums: answering range queries instantly	369
	DSA Prefix sums: answering range queries instantly	370
	DESIGN Key-value stores	381
	PRACTICPractice — Prefix sums: answering range queries instantly	391
038	Subarray sum equals K: prefix plus hash map	395
	DSA Subarray sum equals K: prefix plus hash map	396
	DESIGN Document databases	408
	PRACTICPractice — Subarray sum equals K: prefix plus hash map	419
039	Difference arrays: range updates, cheaply	423
	DSA Difference arrays: range updates, cheaply	424
	DESIGN Wide-column and time-series stores	436
	PRACTICPractice — Difference arrays: range updates, cheaply	447
040	2D prefix sums and inclusion-exclusion	451
	DSA 2D prefix sums and inclusion-exclusion	452
	DESIGN Choosing SQL or NoSQL in an interview	464
	PRACTICPractice — 2D prefix sums and inclusion-exclusion	475
041	Prefix sums revision and mock round	479
	DSA Prefix sums revision and mock round	480
	DESIGN Connection pools, ORMs, and the N + 1 query	491
	PRACTICPractice — Prefix sums revision and mock round	501
PART VI · BINARY SEARCH		505
042	Binary search: the idea and the invariant	506
	DSA Binary search: the idea and the invariant	507
	DESIGN Database revision and interview questions	520
	PRACTICPractice — Binary search: the idea and the invariant	532
043	Writing binary search without off-by-one bugs	535
	DSA Writing binary search without off-by-one bugs	536
	DESIGN Why interviews ask object-oriented design at all	550
	PRACTICPractice — Writing binary search without off-by-one bugs	562
044	First and last occurrence	566
	DSA First and last occurrence	567
	DESIGN Classes and objects	581
	PRACTICPractice — First and last occurrence	595

045	Search in a rotated sorted array	599
DSA	Search in a rotated sorted array	600
DESIGN	Encapsulation	615
PRACTIC	Practice — Search in a rotated sorted array	629
046	Binary search on the answer	633
DSA	Binary search on the answer	634
DESIGN	Inheritance and its costs	650
PRACTIC	Practice — Binary search on the answer	664
047	Minimise the maximum: the capacity family	668
DSA	Minimise the maximum: the capacity family	669
DESIGN	Polymorphism	687
PRACTIC	Practice — Minimise the maximum: the capacity family	700
048	Binary search on floats, and the epsilon question	704
DSA	Binary search on floats, and the epsilon question	705
DESIGN	Abstraction and interfaces	721
PRACTIC	Practice — Binary search on floats, and the epsilon question	735
049	Peak finding, and searching data that is structured but not sorted	739
DSA	Peak finding, and searching data that is structured but not sorted	740
DESIGN	Composition over inheritance	755
PRACTIC	Practice — Peak finding, and searching data that is structured but not sorted	769
050	Binary search revision and mock round	773
DSA	Binary search revision and mock round	774
DESIGN	Class diagrams and the UML you will actually draw	789
PRACTIC	Practice — Binary search revision and mock round	802

BACK MATTER

A	Appendix A - The nine sections, and what each is for	807
B	Appendix B - Recall cards	809
C	Appendix C - The complete 180-day map	828
D	Appendix D - Glossary	831
E	Appendix E - References and further reading, by section	850
F	Appendix F - Colophon: how this book was made	852
G	Appendix G - Problem index	853
H	Index	860

List of figures

29.1	System design - Normalisation and when to break it	78
35.1	System design - Locking and deadlocks	311
37.1	System design - Key-value stores	369
40.1	System design - Choosing SQL or NoSQL in an interview	451
42.1	System design - Database revision and interview questions	506
43.1	System design - Why interviews ask object-oriented design at all	535
44.1	System design - Classes and objects	566
46.1	Data structures and algorithms - Binary search on the answer	633
46.2	System design - Inheritance and its costs	633
47.1	Data structures and algorithms - Minimise the maximum: the capacity family	668
49.1	System design - Composition over inheritance	739
50.1	Data structures and algorithms - Binary search revision and mock round	773
50.2	System design - Class diagrams and the UML you will actually draw	773

PART IV

Two pointers and sliding window

Days 27 to 36 · 10 chapters

Alongside, on the system design track:

Databases from zero

027 Two pointers: the idea

028 Opposite ends: pair sums on a sorted array

029 Same direction: the read pointer and the write pointer

030 Fast and slow pointers

031 Fixed-size sliding window

032 Variable-size sliding window

033 Window plus hash map: the longest-substring family

034 At-most-K, and the exactly-K trick

035 Choosing between two pointers and a window, under pressure

036 Two pointers revision and mock round

Two pointers: the idea

DATA STRUCTURES AND ALGORITHMS

Two pointers: the idea

You can spot the shape of a problem that two pointers turns from $O(n^2)$ into $O(n)$.

SYSTEM DESIGN

SQL you must know for interviews

You can write SELECT, WHERE, GROUP BY, HAVING and ORDER BY without looking anything up.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Can you do better than the nested loop?*
- *Write a query for the top five customers by total spend.*

Two pointers: the idea

1 What this is, and why they ask it

Two pointers means keeping two indices into the same sequence and moving them according to a rule, instead of looping over every pair. It turns a great many $O(n^2)$ problems into $O(n)$, and it is probably the highest-value pattern in the whole of interview preparation — the next ten days are built on it, and it reappears in linked lists, sorting, and dynamic programming.

You have already used it three times without it being named. The write pointer on [day 015](#). The two ends walking inwards on [day 023](#). The two indices comparing two strings on [day 026](#). Today names the family and — much more importantly — explains **why it is allowed to work**, because that is the part interviewers push on and the part most candidates cannot supply.

The reason is always the same and it is worth learning as a sentence: **each move eliminates a whole set of possibilities at once, and you can prove none of them could have been the answer**. A nested loop checks $n^2/2$ pairs. Two pointers checks about n of them and discards the rest with an argument. If you cannot state the argument, you have memorised a solution rather than understood one — and “*why is it safe to move the left pointer there?*” is the follow-up that finds out.

2 The story

Sathish is eleven and he spends most of the summer at his grandfather’s provision shop, and the job he likes is the weighing.

There is a two-pan balance on the counter, the old kind, and beside it a wooden block with the brass weights standing in a row in their holes — the smallest at one end, the biggest at the other, in order, because his grandfather has kept them that way for forty years and gets annoyed if they are not.

The game his grandfather sets him is this. A woman wants exactly two and a half kilos of rice in one go. Find two weights from the row that come to exactly two and a half, or say there are none.

Sathish used to do it the obvious way. Take the first weight, try it against every other weight in turn. Then take the second weight and try it against every other one. It works and it takes a long time and he lost his place constantly.

His grandfather showed him the other way, and it is the only thing Sathish has ever learnt from him that felt like a trick.

Put one finger on the smallest weight and one finger on the biggest. Lift both onto the pan and look.

Too heavy. Now here is the thing — that biggest weight is finished. Not just with this small one. With *everything*. Because it is already too heavy paired with the very lightest weight in the row, and every other weight is heavier than that one, so it can only get worse. So take your finger off it and move to the next biggest, and never think about it again.

Too light works the same way in the other direction. The smallest weight is finished, because even paired with the biggest thing on the block it did not reach, and everything else is smaller than the biggest. Move that finger up one.

Exactly right, and you stop.

What Sathish noticed after a week is that his fingers only ever move towards each other, and each one only moves forward, so the whole thing is over by the time they meet. On the block with fourteen weights, the old way was ninety-one lifts. This way is never more than thirteen.

3 The idea in plain English

Sathish's two fingers are two indices. His grandfather's "*that weight is finished*" is the elimination argument, and it is the whole subject.

The canonical problem

Given a sorted array and a target, find two numbers that add to the target.

```
left, right = 0, len(nums) - 1
while left < right:
    total = nums[left] + nums[right]
    if total == target:
        return [left, right]
    if total < target:
        left += 1          # the smallest value is too small to ever work
    else:
        right -= 1         # the largest value is too large to ever work
```

Nine lines. $O(n)$ time, $O(1)$ space, where the nested loop is $O(n^2)$.

Why each move is safe — say this out loud

This is the part that matters, and it needs the array to be **sorted**.

When the total is too small, `nums[left]` is the smallest value still in play, and it is currently paired with `nums[right]`, the **largest** value still in play. That is the best this element will ever do. If even its best partner does not reach the target, then no partner can, so `nums[left]` cannot be part of any answer. Discard it — that is one move that eliminates every remaining pair containing it.

When the total is too large, the mirror image. `nums[right]` is already paired with the smallest remaining value and is still too big, so no smaller partner will save it. Discard it.

Each step throws away a whole row of the pair table — up to `n` pairs — with one comparison. That is where the factor of `n` goes, and it is the answer to “*why is this safe?*”

Notice what the argument depends on: **sortedness**. On an unsorted array, “the largest value still in play” is not `nums[right]`, and the whole argument collapses. That is why “*is it sorted?*” is the first question you ask.

Termination, and why `left < right`

Every turn moves exactly one of the two indices, and `left` only increases while `right` only decreases. So the gap between them shrinks by at least one every turn, and the loop must end after at most `n` turns.

`left < right` and not `≤`, because the two values must be different elements. With `≤` you would allow `nums[i] + nums[i]`, using the same element twice — which the problem almost certainly forbids. **Ask**. Some versions allow it and the condition changes.

The three shapes of two pointers

Not every two-pointer problem has the pointers at opposite ends. There are three arrangements, and recognising which you need is most of the skill.

SHAPE	WHERE THEY START	HOW THEY MOVE	USED FOR
Opposite ends	<code>0</code> and <code>n-1</code>	towards each other	pair sums, palindromes, container with most water. Needs sorted or symmetric input. Day 028
Same direction	both at <code>0</code>	one leads, one follows	write pointer, compaction, dedupe. Day 029
Fast and slow	both at head	one moves twice as fast	cycle detection, middle of a list. Day 030

And the **sliding window** from [day 031](#) onwards is the same-direction shape with the pair of indices treated as the ends of a range rather than as two separate things.

How to recognise the opposite-ends shape

The tells, in order of reliability:

1. The array is sorted, or you are allowed to sort it. The single strongest signal.
2. The answer is a pair — two numbers, two positions, two lines.
3. The brute force is a nested loop over pairs, and the problem says “can you do better?”
4. The input is symmetric in some way — a palindrome, or a shape read from both ends.
5. You are asked for $O(1)$ space, which rules out the hash-map answer.

The other answer, and when to prefer it

For pair-sum on an **unsorted** array, there is a different $O(n)$ solution: walk once, and for each value ask a hash set whether `target - value` has already been seen.

```
seen = {}
for i, x in enumerate(nums):
    if target - x in seen:
        return [seen[target - x], i]
    seen[x] = i
```

$O(n)$ time, $O(n)$ space, and it **does not need the array sorted**. That is LeetCode 1.

So which? The honest comparison:

	TWO POINTERS	HASH MAP
needs sorted input	yes	no
time on sorted input	$O(n)$	$O(n)$
time if you must sort first	$O(n \log n)$	$O(n)$
space	$O(1)$	$O(n)$
returns original indices	not after sorting	yes

If it arrives sorted, use two pointers — same time, less space. If it is unsorted and you need original indices, use the hash map, because sorting destroys them. Saying that comparison, rather than picking one, is the complete answer.

4 The picture

```
nums = [2, 7, 11, 15], target = 18:
```

index	0	1	2	3
value	2	7	11	15

$2 + 15 = 17 < 18 \rightarrow \text{left}++$
 (2 paired with the biggest and still short: 2 can never work. Discard it.)

index	0	1	2	3
value	2	7	11	15

$7 + 15 = 22 > 18 \rightarrow \text{right}--$
 (15 paired with the smallest left and still too big: 15 can never work. Discard it.)

index	0	1	2	3
value	2	7	11	15

$7 + 11 = 18 \rightarrow \text{found, return } [1, 2]$

What to notice: three comparisons for four elements. The nested loop would do six. The gap widens fast — at $n = 10,000$ it is 10,000 against 50 million.

What each move actually discards, drawn as the table of pairs:

	j=0	j=1	j=2	j=3
i=0	-	2+7	2+11	2+15
i=1	-	-	7+11	7+15
i=2	-	-	-	11+15
i=3	-	-	-	-

← one move of 'left' deletes this whole row
 one move of 'right' deletes this whole column

What to notice: the nested loop visits every cell — $n(n-1)/2$ of them. Two pointers deletes an entire row or an entire column per step, and there are only n rows and columns, so it finishes in about n steps. **That is the whole reason it is linear**, and it is the picture to have in mind when somebody asks why it works.

The three shapes, side by side:

OPPOSITE ENDS
(sorted input)

```
[ . . . . . ]
  ^         ^
  L         R
  →         ←
```

they meet in the middle; each move eliminates a row or a column

SAME DIRECTION
(compaction)

```
[ . . . . . ]
  ^ ^
  W R
  → →
```

write lags behind read; the gap is what was discarded

FAST AND SLOW
(cycles, middle)

```
[ . . . . . ]
  ^ ^
  S F
  → ---->
```

fast moves 2 per step of slow; they meet iff there is a cycle

5 The code, built step by step

The two indices, and the loop

```
left, right = 0, len(nums) - 1
while left < right:
```

Opposite ends. `left < right` because the two must be different elements, and because it guarantees termination — every turn moves one of them inwards.

The three-way decision

```
total = nums[left] + nums[right]
if total == target:
    return [left, right]
if total < target:
    left += 1
else:
    right -= 1
```

Three cases and each has a reason you can state. Equal: done. Too small: the left value is the smallest in play and has already been given the best partner available, so discard it. Too big: mirror image.

Write `total` into a variable rather than computing it twice. It reads better and, more usefully, it gives you something to point at while explaining.

The whole thing

```
def two_sum_sorted(nums: list[int], target: int) → list[int]:
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

`O(n)` time, `O(1)` space, and it handles the empty array and the one-element array without a special case — `left < right` is false immediately.

The same shape, a different rule: container with most water

LeetCode 11, and it is the problem that proves you understood the elimination argument rather than memorising the sum version.

Given heights of vertical lines, pick two so that the container they form holds the most water.

The area is `min(height[left], height[right]) * (right - left)` — the shorter line decides the depth, and the distance decides the width.

```
left, right = 0, len(height) - 1
best = 0
while left < right:
    best = max(best, min(height[left], height[right]) * (right - left))
    if height[left] < height[right]:
        left += 1
    else:
        right -= 1
```

Why move the shorter one? This is the argument, and it is the whole question. Whatever you do next, the width can only shrink. So the only way to beat the current area is to increase the depth. Moving the **taller** line inwards cannot increase the depth, because the depth is set by the *shorter* one — so every container you could form that way is worse than the one you just measured. Moving the **shorter** one is the only move that can possibly help.

That is a genuinely different rule from the sum problem, arrived at by the same kind of reasoning. **Two pointers is not one algorithm; it is a family with a shared proof obligation.**

Same direction: dedupe a sorted array

The write pointer from [day 015](#), now named:

```
def remove_duplicates(nums: list[int]) → int:
    if not nums:
        return 0
    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1
    return write
```

Both indices start near the front and move the same way; `write` lags. Same family, different shape.

The complete solutions

```
def two_sum_sorted(nums: list[int], target: int) → list[int]:
    """Sorted input. O(n) time, O(1) space.

    Too small → nums[left] is already paired with the largest value available,
        so it can never reach the target. Discard it.
    Too large → mirror image for nums[right].
    """
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1
    return []

def two_sum_hash(nums: list[int], target: int) → list[int]:
    """LeetCode 1. Unsorted input, original indices. O(n) time, O(n) space."""
    seen: dict[int, int] = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []

def max_area(height: list[int]) → int:
    """LeetCode 11. Move the SHORTER line: width only shrinks, so only depth can help."""
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        best = max(best, min(height[left], height[right]) * (right - left))
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best

def remove_duplicates(nums: list[int]) → int:
    """Same-direction two pointers. Sorted input, in place, returns the new length."""
    if not nums:
        return 0
    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1
    return write

def is_palindrome_list(values: list[int]) → bool:
    """Opposite ends on a symmetric input – no sorting needed here."""
    left, right = 0, len(values) - 1
    while left < right:
        if values[left] != values[right]:
            return False
        left += 1
        right -= 1
    return True
```

```

if __name__ == "__main__":
    print([two_sum_sorted(a, t) for a, t in
          ([2, 7, 11, 15], 9), ([2, 3, 4], 6), ([-1, 0], -1),
          ([1, 2, 3, 4], 100), ([], 5), ([1], 1))])
    # [[0, 1], [0, 2], [0, 1], [], [], []]

    print(two_sum_hash([3, 2, 4], 6))          # [1, 2] – unsorted, original indices

    print([max_area(x) for x in
          ([1, 8, 6, 2, 5, 4, 8, 3, 7], [1, 1], [1, 2], [4, 3, 2, 1, 4])])
    # [49, 1, 1, 16]

    a = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]
    k = remove_duplicates(a)
    print(k, a[:k])                          # 5 [0, 1, 2, 3, 4]

    print(is_palindrome_list([1, 2, 3, 2, 1]), is_palindrome_list([1, 2, 3]))
    # True False

```

6 What it costs

two_sum_sorted

Every turn of the loop moves exactly one index, `left` only forward and `right` only backward, and the loop stops when they meet. So the total number of turns is at most $n - 1$, and each turn does one addition and at most two comparisons — constant work.

$O(n)$ time. $O(1)$ extra space — two integers, whatever the size of the array.

Against the nested loop

The nested loop examines every pair: $n(n-1)/2$. Measured, on an input where no pair exists so both versions do their full work:

<code>n = 1,000</code>	two pointers 0.000067 s	nested loop 0.0225 s	334x	1,000 vs 499,500 pairs
<code>n = 5,000</code>	two pointers 0.000332 s	nested loop 0.5923 s	1,782x	5,000 vs 12,497,500
<code>n = 10,000</code>	two pointers 0.000679 s	nested loop 2.3448 s	3,454x	10,000 vs 49,995,000

Look at the ratio column: it roughly **doubles when n doubles** — 334, then 1,782, then 3,454. That is the signature of $O(n)$ against $O(n^2)$, and it means the gap keeps widening. At `n = 100,000` the nested loop would take about four minutes and two pointers about seven milliseconds.

If you have to sort first

If the input is unsorted, two pointers needs $O(n \log n)$ for the sort, so the total is $O(n \log n)$ time and $O(1)$ extra space — worse in time than the hash map's $O(n)$, better in space than its $O(n)$. And sorting **destroys the original indices**, so if the answer must be positions in the original array, either use the hash map or sort `(value, index)` pairs and pay $O(n)$ space anyway.

Say the whole comparison, not just your choice:

sorted input, positions in the sorted array	: two pointers, $O(n)$ time, $O(1)$ space	← best
unsorted input, need original indices	: hash map, $O(n)$ time, $O(n)$ space	← best
unsorted input, memory is tight	: sort + two pointers, $O(n \log n)$, $O(1)$	

max_area

Same structure, so **$O(n)$ time and $O(1)$ space**. The brute force over all pairs is $O(n^2)$; at $n = 10,000$ that is 50 million area computations against 10,000.

remove_duplicates

One pass, n turns of constant work: **$O(n)$ time, $O(1)$ extra space**.

The number to have ready

Two pointers turns $n(n-1)/2$ pair checks into at most n steps, because each move eliminates an entire row or column of the pair table. At $n = 10,000$ that is 10,000 comparisons against 50 million — measured at about 3,500 times faster, and the ratio doubles every time n doubles.

7 The traps

The near-miss: forgetting that it needs sorted input

```
print(two_sum_sorted([3, 2, 4], 6))
```

```
[]
```

The answer is `[1, 2]` — `2 + 4`. The function returns nothing, with no error, because the elimination argument is false on unsorted data: `nums[right]` is not the largest remaining value, so “discard the largest” discards something that might have been the answer.

Sortedness is a precondition, not a convenience. If it is not sorted you either sort it — $O(n \log n)$, and you lose the original indices — or you use the hash map. Ask which the interviewer wants.

The near-miss: `left <= right`

```
while left ≤ right:
```


Now `left` and `right` can land on the same element, so `nums[i] + nums[i]` counts as a pair. On `[1, 2, 3]` with target `4`, it returns `[1, 1]` — the single value 2 used twice. Almost every version of this problem forbids that. **`left < right` for pair problems; `left ≤ right` is for binary search, where a single remaining element is still a valid candidate.**

The near-miss: moving the wrong pointer in `max_area`

```
if height[left] > height[right]:
    left += 1                # moving the TALLER one
else:
    right -= 1
```

On `[1, 8, 6, 2, 5, 4, 8, 3, 7]` the correct answer is 49; this returns 8. Moving the taller line cannot increase the depth — the depth is fixed by the shorter one — and the width has shrunk, so every container reachable that way is strictly worse. **Whenever you write a two-pointer move rule, say the one-sentence reason out loud as you write it.** If you cannot, the rule is a guess.

The near-miss: forgetting the answer is a pair of *values*, not the pair you were asked for

Some versions want the values, some want indices, some want them 1-indexed — LeetCode 167 wants `[left + 1, right + 1]`. This is not a bug in your reasoning, it is a bug in your reading, and it costs a submission. **Read the return type before writing the return statement.**

The near-miss: assuming distinct values

```
two_sum_sorted([1, 1, 2, 3], 2)
```

Returns `[0, 1]`, which is correct — but only because `left < right` allowed two *different positions* holding the same value. Problems that say “you may not use the same element twice” mean the same *position*, not the same value. If a problem genuinely forbids equal values, that is a different condition and you must write it explicitly.

The trap in the follow-up: three-sum

“Now find three numbers that sum to zero” — LeetCode 15 — is not a three-pointer problem. It is a loop over the first element with **two pointers inside**:

```
for i in range(len(nums) - 2):
    # then two pointers over nums[i+1:] looking for -nums[i]
```

$O(n^2)$ overall, which is the expected answer. Candidates who try to move three pointers simultaneously get stuck, because there is no elimination argument that works for three at once. The right instinct is: **fix one, two-pointer the rest**. The other detail is skipping duplicate values of `nums[i]`, or you emit the same triple repeatedly.

8 In the interview

How it gets asked

- “Can you do better than the nested loop?” — the direct version, and it almost always means two pointers or a hash map.
- “The array is sorted. Find two numbers that add to the target.” — LeetCode 167. The word *sorted* is doing all the work.
- “Container with most water.” — LeetCode 11, the one that tests whether you can construct a new elimination argument.
- “Three sum.” — LeetCode 15, where the answer is “fix one and two-pointer the rest”.
- “Why is it safe to move that pointer?” — the follow-up that separates understanding from recall.

What to say out loud, in the first ninety seconds

1. **Ask whether it is sorted.** “Is the array sorted? That decides whether two pointers is even available.” This is the single most valuable question in this topic.
2. **State the brute force and its cost.** “The obvious approach is every pair, which is $O(n^2)$.”
3. **Name the pattern and the shape.** “Since it’s sorted, I’ll use two pointers from opposite ends moving inwards.”
4. **Give the elimination argument before writing the code.** “If the sum is too small, the left value is the smallest still available and it’s already paired with the largest — so it can never reach the target with anything, and I can discard it entirely. Too big is the mirror image.” This is the sentence the question exists for.
5. **Say what that buys.** “Each move eliminates a whole row or column of the pair table, and there are only n of each, so it’s $O(n)$ instead of $O(n^2)$.”
6. **Say why the loop terminates.** “Every turn moves one pointer inwards and they only ever approach, so it ends in at most n steps.”
7. **Give the alternative and when you would prefer it.** “If it weren’t sorted, a hash map is $O(n)$ time and $O(n)$ space and keeps the original indices, which sorting would destroy.”

The follow-ups

“Why is it safe to move the left pointer when the sum is too small?” Because of what the sortedness guarantees about what is left. `nums[left]` is the smallest value still in play, and right now it is paired with `nums[right]`, which is the **largest** value still in play. So that pairing is the best `nums[left]` will ever achieve — every other remaining partner is smaller. If even the best case falls short of the target, then no pairing involving `nums[left]` can reach the target, so I can discard it and every pair containing it in one move. That is the entire reason this is linear: one comparison removes up to `n` pairs from consideration, and the nested loop removes one. The argument depends completely on the array being sorted — on unsorted data `nums[right]` is not the largest remaining value and the whole thing is false.

“What if the array isn’t sorted?” Two options and I would name both. Sort it first, which makes the total $O(n \log n)$ time and keeps $O(1)$ extra space — but sorting destroys the original positions, so if the answer must be indices into the original array I would have to sort `(value, index)` pairs, which costs $O(n)$ space and removes the space advantage. Or use a hash map: one pass, and for each value ask whether `target - value` has already been seen. That is $O(n)$ time and $O(n)$ space and keeps the original indices naturally. For LeetCode 1, which is unsorted and wants indices, the hash map is simply the right answer. Two pointers wins specifically when the input arrives sorted, or when memory is tight enough that $O(1)$ space is worth an $O(n \log n)$ sort.

“Now do three-sum.” Not three pointers — there is no elimination argument that works on three moving indices at once. Instead I fix the first element with an outer loop and run two pointers on the remainder looking for its negation. Sort first, so the two-pointer step is valid, and the total is $O(n \log n) + O(n^2)$, which is $O(n^2)$ — and that is the expected answer; there is no known better bound for this problem. The detail that catches people is duplicates: since the problem wants distinct triples, I skip over repeated values of the outer element, and skip repeats of `left` and `right` after recording a match. Getting the deduplication right is most of the difficulty, and I would say so before writing it.

“Container with most water — why move the shorter line?” Because the width can only shrink from here, so the only way to find a bigger area is to increase the depth, and the depth is determined by the **shorter** of the two lines. If I move the taller line inwards, the depth is still capped by the shorter one — which has not changed — while the width has gone down, so every container I could form is strictly worse than the one I just measured. Moving the shorter line is the only move that can possibly increase the depth. It is the same style of reasoning as the sum problem but a completely different rule, which is why I would always state the argument rather than reach for a remembered move — two-pointer problems share a proof obligation, not an algorithm.

A model answer

“First: is the array sorted? That’s the question that decides whether this technique is available at all.

...Sorted, good.

The brute force is to check every pair, which is n choose 2, so $O(n^2)$. At ten thousand elements that’s about fifty million comparisons.

Because it’s sorted I can do it in one pass with two indices, one at each end, moving inwards.

```
def two_sum_sorted(nums: list[int], target: int) → list[int]:
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

The important part is why each move is safe, and it comes entirely from sortedness. If the total is too small, then `nums[left]` is the smallest value still available and it is currently paired with `nums[right]`, the largest one still available. That’s the best that element will ever do — every other remaining partner is smaller. So if its best case falls short, no pair containing it can reach the target, and I can discard it completely. Too large is the mirror image: `nums[right]` is already paired with the smallest remaining value and is still too big, so nothing smaller will help.

That’s where the speed comes from. Think of the table of all pairs — the nested loop visits every cell, and each move here deletes an entire row or an entire column. There are only n rows and n columns, so I finish in about n steps rather than $n^2/2$. I measured it: at ten thousand elements it was about three and a half thousand times faster, and the ratio doubles every time n doubles.

The loop terminates because every turn moves exactly one index and they only ever approach each other, so it ends within n steps. And it’s `left < right` rather than `≤`, because the two must be different positions — with `≤` I’d allow an element to pair with itself.

$O(n)$ time, $O(1)$ extra space.

If it weren't sorted, I'd change approach rather than sort. A hash map does it in one pass — for each value, ask whether target minus it has already been seen — which is $O(n)$ time and $O(n)$ space, and it keeps the original indices, which sorting would destroy. Two pointers wins when the input is already sorted, or when $O(1)$ space matters enough to pay for an $O(n \log n)$ sort.”

9 Recall card

- **Two indices, one rule, and an argument for why each move is safe.** That argument is the question.
- **Opposite ends needs sorted input.** Too small $\rightarrow$ the left value can never work; too big $\rightarrow$ the right value can never work.
- **Each move deletes a whole row or column of the pair table** — that is why $O(n^2)$ becomes $O(n)$.
- **Three shapes:** opposite ends (sorted, pairs), same direction (compaction), fast and slow (cycles).
- **Unsorted and need original indices $\rightarrow$ hash map,** $O(n)$ time, $O(n)$ space. Sorted $\rightarrow$ two pointers, $O(1)$ space.

1 What this is, and why they ask it

SQL — Structured Query Language — is how you ask a relational database for data. Its defining property is that it is **declarative**: you describe the result you want, and the database works out how to produce it. You never write a loop, never say which index to use, never decide the order of operations. You state conditions and it finds a way.

That is unusual and it takes getting used to. In Python you say *how*: iterate, accumulate, sort. In SQL you say *what*: these rows, grouped this way, filtered by that, in this order. The database's **query planner** then chooses the actual steps — and it usually chooses better than you would, because it knows how many rows are in each table and which indexes exist.

You need six clauses. `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, plus `LIMIT`. That is genuinely most of the SQL asked in interviews, and “*write me a query for the top five customers by total spend*” uses five of them in eight lines.

Interviewers ask because SQL is unfakeable and fast to assess. A candidate either produces the query in two minutes or does not. It is also the most common way people reveal that they have only ever used an ORM — they know `User.objects.filter(...)` and cannot say what it generates, which matters the first time a query is slow.

2 The story

Nagaraj cooks for a small hotel near the bus stand — sixty or seventy meals at lunch, a bit less in the evening — and three mornings a week he is at the wholesale market by five.

The first year he did it himself. He walked every row of the shed, looked at everything, remembered what he had seen, compared prices in his head, and came out two hours later fairly sure he had missed something. Usually he had.

Now he goes to Selvaraj, who sits at a desk near the gate and has run that shed for thirty years, and he asks. He has got very good at asking, and it is always the same shape.

Tomatoes. Only the ones that came in from Kolar. Only lots over fifty kilos. Cheapest first.

Twenty seconds, and Selvaraj tells him which four suppliers and what they want per kilo. Nagaraj never tells him how to find it. He never says which row to walk down, or which end to start from. He says what he wants and what counts, and Selvaraj — who knows which rows were restocked at four this morning, and which supplier's man is standing right there — works out the route.

Sometimes he asks a different shape of question, and this one took him longer to learn.

How many suppliers have each vegetable today? Only tell me the ones with more than three.

That is not a question about individual sacks at all. It is a question about groups of them, and the “more than three” is about the groups rather than about any one sack, so it has to come after the counting rather than before it. It took Nagaraj a while to see that those two kinds of condition are genuinely different, and he got wrong answers for a month before he did.

What he says about the whole arrangement is that Selvaraj is not cleverer than him. He simply knows the shed, and Nagaraj does not, and the fastest way through a place you do not know is to say clearly what you want and let somebody who does know decide the route.

3 The idea in plain English

Nagaraj's question is a `SELECT`. Selvaraj working out the route is the **query planner**. And the two kinds of condition — “only lots over fifty kilos” versus “only the vegetables with more than three suppliers” — are `WHERE` and `HAVING`, which is the distinction this lesson exists to fix.

The six clauses

```
SELECT  customer_id, SUM(amount) AS total    -- what to show
FROM    orders                             -- where to get it
WHERE   status = 'completed'               -- which ROWS count
GROUP BY customer_id                       -- collapse rows into groups
HAVING  SUM(amount) > 10000                 -- which GROUPS count
ORDER BY total DESC                        -- what order
LIMIT   5;                                -- how many
```

That is the whole shape, and almost every interview query is a subset of it.

The order they actually run in

This is the single most useful thing on this page, because it explains three things that otherwise look like arbitrary rules.

```

written:  SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT
executed: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
           1         2         3         4         5         6         7

```

Three consequences fall straight out of that:

WHERE cannot see an alias defined in SELECT, because **SELECT** has not run yet. So `WHERE total > 100` fails when `total` was defined as `SUM(amount) AS total`.

ORDER BY can see it, because **ORDER BY** runs after **SELECT**. That is why `ORDER BY total DESC` works in the query above and `WHERE total > 100` would not.

WHERE cannot use an aggregate like `SUM(...)`, because the grouping has not happened yet. That is what **HAVING** is for.

WHERE versus HAVING, which is the whole point

Nagaraj's two conditions:

- “only lots over fifty kilos” — a condition about **one row**. It applies before anything is grouped, so: `WHERE`.
- “only vegetables with more than three suppliers” — a condition about **a group**. It cannot be decided until the counting has happened, so: `HAVING`.

```

SELECT  vegetable, COUNT(*) AS supplier_count
FROM    lots
WHERE   quantity_kg > 50           -- discard individual small lots FIRST
GROUP BY vegetable
HAVING  COUNT(*) > 3;             -- then discard groups that are too small

```

The rule to say out loud: **WHERE filters rows**, **HAVING filters groups**. And a corollary that is worth marks: **put every condition you can into WHERE**, because **WHERE** runs first and reduces how much there is to group. Filtering in **HAVING** what could have been filtered in **WHERE** is correct and slower.

Aggregates

Five functions do most of the work: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

```

SELECT COUNT(*)           FROM orders; -- number of rows
SELECT COUNT(discount_code) FROM orders; -- rows where discount_code IS NOT NULL
SELECT COUNT(DISTINCT user_id) FROM orders; -- how many different users
SELECT AVG(amount)        FROM orders; -- NULLs are skipped, not treated as 0

```

Two behaviours that get asked directly:

COUNT(*) counts rows; **COUNT(column) counts non-null values in that column**. If 300 of 1,000 orders have no discount code, `COUNT(*)` is 1,000 and `COUNT(discount_code)` is 700.

Aggregates ignore NULL. `AVG` of `[10, 20, NULL]` is 15, not 10 — the null is skipped, not counted as zero. That is usually what you want and it is occasionally a bug: if a null *means* zero in your data, `AVG` will be wrong, and `COALESCE(amount, 0)` is the fix.

Grouping, precisely

`GROUP BY` collapses many rows into one row per distinct value. After it runs, **every column in `SELECT` must either be in the `GROUP BY` or be inside an aggregate** — because for anything else there would be many values and only one row to put them in.

```
-- error: name is neither grouped nor aggregated
SELECT customer_id, name, SUM(amount) FROM orders GROUP BY customer_id;

-- fine
SELECT customer_id, SUM(amount) FROM orders GROUP BY customer_id;
```

Postgres raises `column "name" must appear in the GROUP BY clause or be used in an aggregate function`. MySQL historically allowed it and returned an arbitrary value, which is worse than an error and is a real source of wrong reports.

NULL in conditions

From [day 026](#): `NULL` is *unknown*, so comparing with it gives unknown, not true.

```
WHERE discount_code = NULL      -- matches NOTHING, ever
WHERE discount_code IS NULL     -- correct
WHERE discount_code <> 'SALE'    -- excludes NULLs too! unknown is not true
```

That third line is the one that catches people. If you want the nulls as well you must say so: `WHERE discount_code <> 'SALE' OR discount_code IS NULL`.

ORDER BY and LIMIT

```
ORDER BY total DESC, customer_id ASC    -- second column breaks ties
LIMIT 5 OFFSET 10                       -- rows 11-15
```

Two things worth saying: **without `ORDER BY`, row order is undefined**, so `LIMIT 5` on its own gives you five arbitrary rows and may give different ones each run. And **add a tie-breaker** — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

4 The picture

The two orders — written and executed — with what each stage does:

WRITTEN		EXECUTED	what it holds afterwards
SELECT cust, SUM(amt)	5	1 FROM orders	every row in the table
FROM orders	2	2 WHERE status=...	only the rows that pass
WHERE status = '...'	3	3 GROUP BY cust	one bucket per customer
GROUP BY cust	4	4 HAVING SUM(..)>...	only the buckets that pass
HAVING SUM(amt) > ...	5	5 SELECT cust, SUM	one row per bucket, chosen columns
ORDER BY total DESC	6	6 ORDER BY total	the same rows, sorted
LIMIT 5	7	7 LIMIT 5	the first five

What to notice: `SELECT` is fifth, not first. That single fact explains why `WHERE` cannot use an alias, why `ORDER BY` can, and why aggregate conditions need `HAVING`.

The same query on real rows:

orders		after WHERE status='completed'
<pre> +-----+-----+-----+-----+ id cust amount status +-----+-----+-----+-----+ 1 7 1200 completed 2 7 800 completed 3 9 5000 cancelled 4 9 300 completed 5 4 9000 completed +-----+-----+-----+-----+ </pre>	--->	<pre> +-----+-----+-----+ id cust amount +-----+-----+-----+ 1 7 1200 2 7 800 4 9 300 5 4 9000 +-----+-----+-----+ </pre>
		after GROUP BY cust
		<pre> +-----+-----+ cust SUM(amount) +-----+-----+ 7 2000 9 300 4 9000 +-----+-----+ </pre>
		after HAVING SUM(amount) > 1000
		<pre> +-----+-----+ 7 2000 4 9000 +-----+-----+ </pre>
		after ORDER BY total DESC
		<pre> +-----+-----+ 4 9000 7 2000 +-----+-----+ </pre>

What to notice: the cancelled ₹5,000 order disappears at the `WHERE` stage, so customer 9's total is 300 and not 5,300. Filtering **before** grouping changes the answer, not just the speed — which is why deciding whether a condition belongs in `WHERE` or `HAVING` is a correctness question, not a style one.

5 How it actually works

Building the interview question, one clause at a time

“Write a query for the top five customers by total spend.”

Start with the raw rows.

```
SELECT customer_id, amount FROM orders;
```

Collapse to one row per customer.

```
SELECT  customer_id, SUM(amount) AS total_spend
FROM    orders
GROUP BY customer_id;
```

Ask the clarifying question out loud — this is the part interviewers are actually watching for: “Should cancelled and refunded orders count? And do you want the customer’s name, or is the id enough?” Say completed only, and the name.

Filter the rows before grouping.

```
SELECT  customer_id, SUM(amount) AS total_spend
FROM    orders
WHERE    status = 'completed'
GROUP BY customer_id;
```

Get the name, which needs the other table.

```
SELECT  c.id, c.name, SUM(o.amount) AS total_spend
FROM    orders o
JOIN    customers c ON c.id = o.customer_id
WHERE    o.status = 'completed'
GROUP BY c.id, c.name
ORDER BY total_spend DESC
LIMIT   5;
```

Joins are [day 028](#) in full; for today, `JOIN ... ON` means *match each order to its customer by id*.

Note `GROUP BY c.id, c.name` — every non-aggregated column in `SELECT` must be grouped. Grouping by `c.id` alone is enough in Postgres when `c.id` is the primary key, because the name is then functionally determined, but writing both is portable and clearer.

Two details worth mentioning unprompted:

“I’d add a tie-breaker to the `ORDER BY` — `total_spend DESC, c.id` — so the result is stable when two customers have identical totals.”

“And this excludes customers with no completed orders entirely. If you want them shown with a total of zero, that’s a `LEFT JOIN` from customers to orders with `COALESCE(SUM(o.amount), 0)`.”

Reading a query plan

`EXPLAIN ANALYZE` in front of any query shows what the planner chose and what it actually cost:

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 4217;
```

```
Seq Scan on orders  (cost=0.00..18334.00 rows=1 width=64)
    (actual time=0.021..112.4 rows=3 loops=1)
  Filter: (customer_id = 4217)
  Rows Removed by Filter: 999997
```

`Seq Scan` **plus** `Rows Removed by Filter: 999997` means it read a million rows to return three. Add the index and it becomes:

```
Index Scan using orders_customer_id_idx on orders  (cost=0.42..8.45 rows=3 width=64)
    (actual time=0.018..0.024 rows=3 loops=1)
```

112 ms to 0.024 ms. **Recognising `Seq Scan` on a large table as the thing to fix** is most of what “can you read a query plan?” is asking.

The patterns that come up

Top N per group — “the three most recent orders for each customer” — is the one that needs a window function, and it is worth knowing the shape even if you cannot write it from memory:

```
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY created_at DESC) AS rn
  FROM   orders o
) t
WHERE rn ≤ 3;
```

`PARTITION BY` is “restart the numbering for each customer”. Window functions are the single biggest gap in most candidates’ SQL, and naming `ROW_NUMBER`, `RANK` and `PARTITION BY` is a strong signal.

Counting things that may be zero needs a `LEFT JOIN`:

```
SELECT  c.id, c.name, COUNT(o.id) AS order_count
FROM    customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name;
```

`COUNT(o.id)` and not `COUNT(*)` — with a `LEFT JOIN`, a customer with no orders still produces one row with nulls, so `COUNT(*)` would say 1 and `COUNT(o.id)` correctly says 0. That is a genuinely common bug and a good thing to mention.

Existence is usually better as `EXISTS` than as a join, because it can stop at the first match:

```
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id AND o.amount > 10000);
```

Dialects

The core is standard and portable. The differences that bite: `LIMIT 5` in Postgres, MySQL and SQLite; `TOP 5` in SQL Server; `FETCH FIRST 5 ROWS ONLY` in the standard and Oracle. String concatenation is `||` in Postgres and `CONCAT()` in MySQL. Postgres is strict about `GROUP BY`; MySQL historically was not. Write standard SQL and mention that you would check the dialect.

6 The numbers

The index, again

1,000,000 orders, about 200 bytes each:

```
table size = 1,000,000 × 200 B = 200 MB

WHERE customer_id = 4217:
  sequential scan : read 200 MB, discard 999,997 rows ≈ 112 ms
  index scan      : ~3 B-tree page reads + 3 row reads ≈ 0.02 ms
                  about 5,000x
```

And it gets worse linearly: at 10 million orders the scan is 1.1 seconds and the index scan is still about 0.02 ms, because a B-tree gains a level roughly every $500\times$ growth.

`SELECT *` versus naming columns

An orders row is 200 bytes, of which `id`, `customer_id` and `amount` are 24.

```
SELECT *           : 1,000,000 × 200 B = 200 MB read and sent
SELECT id, cust, amt : the same rows, but a covering index can serve it
                    from ~24 MB of index without touching the table at all
```

A **covering index** — one that contains every column the query needs — lets the database answer without reading the table rows, which on a wide table is often $5\text{--}10\times$ less I/O. `SELECT *` makes that impossible, which is the real argument against it, over and above sending bytes nobody uses.

`GROUP BY` memory

Grouping one million orders by customer, with 50,000 distinct customers:

```
hash table: 50,000 groups × ~50 B = 2.5 MB → fits in memory, fast
```

Grouping by `order_id` instead — a million distinct values:

```
hash table: 1,000,000 × 50 B = 50 MB → may exceed work_mem and spill to disk
```

Postgres's default `work_mem` is 4 MB. Exceeding it turns an in-memory hash aggregate into a sort that spills to disk, and that is a common cause of a query that was fast in testing and slow in production. **The number of distinct groups is what decides it**, not the number of rows.

`OFFSET` pagination

From [day 017](#), now with the reason:

```
LIMIT 20 OFFSET 0      : find 20 rows ≈ 1 ms
LIMIT 20 OFFSET 1000000 : find and DISCARD 1,000,000 rows ≈ 2,000 ms
```

`OFFSET` does not skip cheaply — it generates and throws away every skipped row. That is why deep pagination gets slower and why cursor pagination — `WHERE id > :last_seen ORDER BY id LIMIT 20` — stays at about 1 ms at any depth.

`COUNT(*)` on a big table

```
SELECT COUNT(*) FROM orders; -- 10,000,000 rows
Postgres: full scan ≈ 1-3 seconds (MVCC means it must check row visibility)
```

Which is why a live “12,483,221 orders” counter on a dashboard is usually a maintained counter or an approximation from `pg_class.reltuples`, not a real `COUNT(*)`.

7 The trade-offs

Do the work in SQL, or in the application?

In SQL the database does the filtering, grouping and sorting close to the data, sends only the result, and can use indexes. Summing a million rows in the database sends back one number; doing it in your application sends back a million rows and 200 MB.

In the application the logic is testable, versioned and debuggable in a language you know well, and it does not consume database CPU — which is the hardest resource to scale, because you usually have one primary and many application servers.

The rule: filter and aggregate in SQL, put business rules in the application. Reducing a million rows to fifty in the database is obviously right. Encoding your pricing policy in a 200-line query is obviously wrong.

`SELECT *` or named columns

`SELECT *` is fine in an interactive session and a liability in code: it sends columns you do not need, it breaks the moment someone adds or reorders a column, and it prevents covering-index optimisations. **Name your columns in anything that ships.**

ORM or raw SQL

An ORM removes boilerplate, prevents SQL injection by default, and handles migrations. It also hides what is being executed, which is how the N+1 query problem happens — looping over 100 users and touching `user.orders` issues 101 queries, exactly the shape from [day 021](#). **Use the ORM for straightforward access, drop to SQL for reports and anything with a `GROUP BY`, and be able to see the generated query** — every ORM has a way to print it, and knowing that is a strong signal.

Normalised joins or denormalised columns

A join is the correct way to combine tables and it costs work at read time. Copying a value to avoid the join is faster to read and can go stale. Same trade as [day 026](#): normalise first, denormalise when a measured read-to-write ratio justifies it.

The sentence that separates candidates

I would not compute this in application code. Pulling a million order rows across the network to sum them in Python costs 200 MB of transfer, several seconds, and gigabytes of memory, to produce one number the database could have produced in 200 milliseconds using an index it already has. What I *would* keep out of SQL is business logic — a 200-line query encoding pricing rules is untestable and unreviewable, and it runs on the one machine I cannot easily add more of. Filter and aggregate in SQL; decide in the application.

8 In the interview

How it gets asked

- “Write a query for the top five customers by total spend.” — the canonical version, using five clauses.
- “What’s the difference between `WHERE` and `HAVING`?” — the definition question, and the one most often fumbled.

- “This query is slow. What would you do?” — where `EXPLAIN`, indexes and `Seq Scan` are the answer.
- “Find the customers who have never placed an order.” — the `LEFT JOIN ... IS NULL` or `NOT EXISTS` question.
- “What does `COUNT(*)` do differently from `COUNT(column) ?`” — the small precise one.

What to say out loud, in the first ninety seconds

1. **Ask two clarifying questions before writing.** “Which order statuses count — do cancelled and refunded orders get excluded? And do you want the customer’s name or just the id?”
2. **Say the shape before the syntax.** “So: filter to completed orders, group by customer, sum the amounts, sort descending, take five.”
3. **Write it in execution order in your head, not written order.** `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`.
4. **Say why the filter is in `WHERE` and not `HAVING`.** “Status is a property of an individual order, so it belongs in `WHERE` — that runs before the grouping, so it reduces what has to be grouped, and it also changes the answer rather than just the speed.”
5. **Mention the tie-breaker unprompted.** “I’d add a second `ORDER BY` column so the result is stable when two customers tie.”
6. **Say what the query excludes.** “This drops customers with no completed orders. If you want them at zero, that’s a `LEFT JOIN` with `COALESCE`.”
7. **Offer the index.** “For this to be fast I’d want an index on `orders(customer_id)` and probably on `status`.”

The follow-ups

“What’s the difference between `WHERE` and `HAVING` ?” `WHERE` filters individual rows and runs **before** grouping; `HAVING` filters groups and runs **after**. So a condition on a column of one row — `status = 'completed'` — goes in `WHERE`, and a condition on an aggregate — `SUM(amount) > 10000` — must go in `HAVING`, because the sum does not exist until the grouping has happened. It is not only a syntax rule: filtering before grouping changes the answer, because a cancelled order excluded in `WHERE` never contributes to any customer’s total, while the same condition applied afterwards could not remove it. And where a condition *could* go in either — which is rare — it belongs in `WHERE`, because reducing the rows before grouping is strictly less work.

“This query takes eight seconds. What do you do?” Run `EXPLAIN ANALYZE` on it first, before changing anything, because guessing is how people add indexes that do not get used. What I would look for, in order: a `Seq Scan` on a large table with a high “Rows Removed by Filter”, which means a missing index on the filtered column; a nested

loop join over a large outer table, which usually means a missing index on the join column; and a sort or hash aggregate that has spilled to disk, which means the grouping exceeded `work_mem`. Then I would check the estimated versus actual row counts — if they differ wildly, the statistics are stale and `ANALYZE` may fix the plan by itself. Beyond that: is the query selecting columns it does not need, which prevents a covering index; and is it doing deep `OFFSET` pagination, which discards every skipped row and should be a keyset cursor instead.

“Find customers who have never placed an order.” Three ways, and they are not equivalent in performance. `LEFT JOIN orders ON ... WHERE orders.id IS NULL` — join everything and keep the rows where the join found nothing. `NOT EXISTS (SELECT 1 FROM orders WHERE customer_id = c.id)` — usually the best, because it can stop at the first matching row rather than materialising all of them. And `NOT IN (SELECT customer_id FROM orders)`, which I would avoid: if any `customer_id` in that subquery is `NULL`, `NOT IN` returns no rows at all, because comparing with unknown gives unknown. That `NULL` trap in `NOT IN` is a genuinely nasty bug and worth naming.

“How would you get the three most recent orders for each customer?” That is top-N-per-group, and it needs a window function rather than a plain `GROUP BY`, because `GROUP BY` collapses each customer to one row and I need three. I would use `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY created_at DESC)` in a subquery, then filter the outer query to `rn ≤ 3`. `PARTITION BY` restarts the numbering for each customer, which is exactly the “per group” part. The alternative with a correlated subquery — for each customer, select from orders with its own `LIMIT 3` — is easier to read and much slower, because it runs once per customer. And I would want an index on `(customer_id, created_at DESC)` so the window function does not have to sort.

A model answer

“Two clarifications first. Which statuses count as spend — should I exclude cancelled and refunded orders? And do you want the customer’s name, or is the id enough?

...Completed only, and the name.

So the shape is: take orders, keep the completed ones, group by customer, sum the amounts, join to get the name, sort descending, take five.

```
SELECT  c.id, c.name, SUM(o.amount) AS total_spend
FROM    orders o
JOIN    customers c ON c.id = o.customer_id
WHERE   o.status = 'completed'
GROUP BY c.id, c.name
ORDER BY total_spend DESC, c.id
LIMIT   5;
```

The status filter is in `WHERE` rather than `HAVING`, and that’s deliberate: status is a property of an individual order, so it can be decided before any grouping happens. That matters twice over — it reduces how many rows have to be grouped, and it changes the answer, because a cancelled order filtered out in `WHERE` never contributes to anyone’s total. `HAVING` is for conditions on the aggregate itself, like `SUM(o.amount) > 10000`, which cannot be evaluated until the group exists.

`GROUP BY c.id, c.name` because every column in `SELECT` that isn’t inside an aggregate has to be grouped. Postgres would accept just `c.id` since it’s the primary key and the name is functionally determined by it, but writing both is portable.

I’ve added `c.id` as a second `ORDER BY` column so the result is stable — without a tie-breaker, two customers with identical totals can come back in either order, which makes paginated results duplicate and skip rows between pages.

One thing worth flagging: this excludes customers who have no completed orders at all, because an inner join drops them. If you want them shown with a total of zero, it becomes a `LEFT JOIN` from customers to orders with `COALESCE(SUM(o.amount), 0)`.

And for this to be fast on a million orders I’d want an index on `orders(customer_id)` for the join and something on `status` — or better, a partial index on `orders(customer_id) WHERE status = 'completed'`, since that’s the only status this query ever looks at. Without an index the filter is a sequential scan: about 200 MB and 112 milliseconds on a million rows, against about 0.02 with the index.”

9 Recall card

- **SQL is declarative:** say what you want, the planner picks the route. Six clauses cover almost everything.
- **Execution order is** `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT` — which is why `WHERE` cannot see an alias and `ORDER BY` can.
- `WHERE` filters rows, `HAVING` filters groups. Put every condition you can in `WHERE`.
- `COUNT(*)` counts rows, `COUNT(col)` counts non-nulls, and aggregates skip `NULL`. Use `IS NULL`, never `= NULL`.
- Always `ORDER BY` before `LIMIT`, and add a tie-breaker. Run `EXPLAIN ANALYZE` before optimising anything.

DSA topic: Two pointers: the idea **System design topic:** SQL you must know for interviews

Code these, in this order

Four problems where two indices replace a nested loop. **For every one, state the elimination argument out loud before writing code** — the sentence that says why discarding a value is safe. If you cannot produce it, you have a remembered move rather than an understood one.

Before each one, ask:

1. Is the input sorted? If not, can I sort it, and what does that cost me?
2. Which of the three shapes — opposite ends, same direction, fast and slow?
3. What does each move eliminate, and why is that safe?
4. Is it `left < right` or `left ≤ right`, and why?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Two Sum II — Input Array Is Sorted	LeetCode 167 (Medium)	The base case, and whether you notice it wants 1-indexed positions.
2	Container With Most Water	LeetCode 11 (Medium)	A <i>different</i> elimination argument. Moving the taller line is the wrong move, and you should be able to say why in one sentence.
3	3Sum	LeetCode 15 (Medium)	That it is not three pointers. Fix one, two-pointer the rest — and the duplicate-skipping is most of the work.
4	Trapping Rain Water	LeetCode 42 (Hard)	The two-pointer version, after you have written the prefix-max one. The argument here is genuinely subtle and worth the hour.

On problem 1, prove it to yourself

Before coding, take `[2, 7, 11, 15]` with target 18 and say each step out loud including the reason:

```
2 + 15 = 17 < 18 → move left, because ...
7 + 15 = 22 > 18 → move right, because ...
7 + 11 = 18      → found
```

Fill in both “because” clauses without looking. Then run your solution on `[3, 2, 4]` with target 6 and explain why it returns nothing.

On problem 2, get the rule wrong on purpose

Write the version that moves the **taller** line and run it on `[1, 8, 6, 2, 5, 4, 8, 3, 7]`. You should get 8 instead of 49. Then say, in one sentence, why moving the taller line can never help.

The sentence you want is about which of the two lines determines the depth, and what happens to the width.

On problem 3, do the duplicates properly

Solve it, then test on `[0, 0, 0, 0]` and on `[-2, 0, 0, 2, 2]`. Both are duplicate-heavy on purpose.

Then answer:

1. Why must you sort first, given the problem does not ask for sorted output?
2. Where exactly do the three duplicate-skips go — the outer element, `left`, and `right`?
3. What is the total cost, and why is it not $O(n^3)$?
4. Why does no three-pointer version exist?

The three-shapes drill

For each problem, name the shape — **opposite ends**, **same direction**, or **fast and slow** — in under five seconds:

1. Check whether a string is a palindrome.
2. Move all zeros to the end.
3. Find whether a linked list has a cycle.
4. Find two numbers in a sorted array that sum to a target.
5. Remove duplicates from a sorted array in place.
6. Find the middle node of a linked list in one pass.
7. Merge two sorted arrays into the first, in place.
8. Longest substring without repeating characters.

Number 7 is opposite-ends in a disguise — say which end and why. Number 8 is a window, which is the same-direction shape with the pair treated as a range.

The elimination drill

This is the exercise that matters most today. For each, write the one sentence that justifies the move:

1. Two-sum sorted, total too small → move `left`. Why is `nums[left]` finished?
2. Two-sum sorted, total too large → move `right`. Why?
3. Container with most water → move the shorter line. Why can the taller one never help?
4. Trapping rain water → move whichever side has the smaller wall. Why is that side's answer already determined?

Number 4 is the hardest and it is worth the effort. The answer involves knowing that the *other* side's wall is at least as tall, so it cannot be the limiting factor.

The measurement drill

Run this and read the ratio column.

```
import time

def two_pointer(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target: return [left, right]
        if s < target: left += 1
        else: right -= 1
    return []

def brute(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target: return [i, j]
    return []

for n in (1000, 5000, 10000):
    arr = [2 * i for i in range(n)]      # all even
    target = 1                          # odd, so no pair exists
    s = time.perf_counter(); two_pointer(arr, target); t1 = time.perf_counter() - s
    s = time.perf_counter(); brute(arr, target); t2 = time.perf_counter() - s
    print(f"n={n:>6}  {t1:.6f}s  {t2:.4f}s  ratio {t2/t1:>7.0f}x")
```

Then answer:

1. Why is the target chosen to be impossible?
2. The ratio roughly doubles each time `n` doubles. What does that tell you, and why is it not constant?
3. Predict the two times at `n = 100,000` before running it.

The sorted-or-not drill

For each, say whether you would use two pointers or a hash map, and why:

1. Sorted array, return positions in that array.
2. Unsorted array, return original positions.
3. Unsorted array, return the two values, memory is very tight.
4. Sorted array, but you must not modify it and cannot allocate.
5. A stream of numbers arriving one at a time.

Number 5 is the one that rules out two pointers entirely — say why.

The SQL drill

Write each query without looking anything up. Tables: `customers(id, name, city)`, `orders(id, customer_id, amount, status, created_at)`.

1. All orders over ₹5,000, newest first.
2. The number of orders per customer.
3. The number of *completed* orders per customer, only for customers with more than three.
4. The top five customers by total completed spend, with their names.
5. Customers who have never placed an order.
6. The average order value per city.
7. How many distinct customers ordered in the last 30 days.
8. The three most recent orders for each customer.

Number 8 needs a window function. Numbers 3 and 6 need you to decide `WHERE` versus `HAVING` correctly.

The execution-order drill

Answer without running anything:

1. Why does `SELECT SUM(amount) AS total FROM orders WHERE total > 100` fail?
2. Why does `ORDER BY total DESC` work in the same query?
3. Why can `WHERE` not contain `SUM(amount)`?
4. What is the difference between `COUNT(*)` and `COUNT(discount_code)`?
5. `WHERE status <> 'cancelled'` — does it return rows where status is `NULL`? Why?
6. Two customers have the same total. Why might `LIMIT 5` return different rows on different runs?

The slow-query drill

You run `EXPLAIN ANALYZE` and see:

```
Seq Scan on orders (cost=0.00..18334.00 rows=1 width=64)
    (actual time=0.021..112.4 rows=3 loops=1)
  Filter: (customer_id = 4217)
  Rows Removed by Filter: 999997
```

Say, in order: what this is telling you, what you would do, what you would expect to see afterwards, and roughly what the new time would be.

Then say two other things you look for in a plan besides `Seq Scan`.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Can you do better than the nested loop?* Ask whether it is sorted. State the brute force and its cost. Name the shape. **Give the elimination argument before the code.** Then the pair-table picture — each move deletes a row or a column, and there are only `n` of each. Finish with the hash-map alternative and when you would prefer it.
 2. *Write a query for the top five customers by total spend.* Two clarifying questions first. Say the shape before the syntax. Explain why the status filter is in `WHERE` and not `HAVING`, and that it changes the answer rather than just the speed. Mention the tie-breaker and the excluded-customers point unprompted. Offer the index.
 3. *Why is it safe to move the left pointer when the sum is too small?* This is the whole day. Smallest value still in play, already paired with the largest available, so that is its best case — if the best case falls short, nothing works, so discard it and every pair containing it. And say what the argument depends on: sortedness.
-

Before you move on

- [] I ask “is it sorted?” before considering two pointers.
- [] I state the elimination argument out loud before writing the move.
- [] I can draw the pair table and show what one move deletes.
- [] I know the three shapes and can classify a problem into one in five seconds.
- [] I write `left < right` for pair problems and can say why not `≤`.
- [] I can give the two-pointers-versus-hash-map comparison as a table, not a preference.
- [] I can write the top-five query in two minutes without looking anything up.
- [] I can state the execution order and the three things it explains.

- [] I know `WHERE` filters rows and `HAVING` filters groups, and that the choice changes the answer.
- [] I can redraw the pair-table diagram and the clause-order diagram from memory, in whatever tool I like.

Opposite ends: pair sums on a sorted array

DATA STRUCTURES AND ALGORITHMS

Opposite ends: pair sums on a sorted array

You can find a pair summing to a target in $O(n)$ and argue that you miss nothing.

SYSTEM DESIGN

Joins, drawn

You can draw inner, left, right and full joins and predict the row count of each.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- Find two numbers in a sorted array that add up to the target.
- What is the difference between an inner join and a left join?

Opposite ends: pair sums on a sorted array

1 What this is, and why they ask it

Yesterday introduced two pointers and the elimination argument. Today is the same technique used properly, and the emphasis moves to the harder half: **proving you have not missed anything**.

That distinction matters more than it sounds. Yesterday's argument said *why each individual move is safe*. Today's says *why, when the two indices meet and you have found nothing, there is genuinely nothing to find* — even though you examined about n pairs out of $n(n-1)/2$. Those are different claims, and the second is the one an interviewer probes with “*how do you know you didn't skip the answer?*”

The technique also stops being one problem today. The same two-indices-from-opposite-ends shape solves *find one pair*, *find all pairs*, *find the closest pair*, *count the pairs*, *three-sum*, and *sorted squares* — six recognisably different questions with the same skeleton and different move rules. Being able to adapt the skeleton rather than recall six solutions is what the day is for.

3Sum (LeetCode 15) in particular is one of the most-asked medium problems anywhere, and its real difficulty is not the algorithm — it is producing distinct triples without duplicates, which is §7.

2 The story

Bashir has been painting houses for twenty-six years and the part he is most careful about is not the painting.

A flat in the new block near the water tank needs its front room done, and he has worked out that it will take twelve litres — two coats, and he has measured the walls twice because he always does. The shop at the corner sells the same paint in tins of one, two, four, five, ten and twenty litres, and they stand on the shelf in that order, smallest on the left.

He wants exactly twelve, and he wants it in exactly two tins. Not three, because the third opening is where colour differences show, and not one big tin with waste left over, because the owner is paying for what is opened.

So he stands in front of the shelf and puts one hand on the one-litre and the other on the twenty.

Twenty-one. Too much. And here he does not just move on — he says the thing he always says, which is that the twenty is finished. It is already sitting with the smallest tin on the shelf and it is still over, so there is nothing on that shelf that will bring it down to twelve. Never mind about pairing it with the four or the five; those are bigger than the one, so they are worse.

Hand off the twenty, on to the ten.

One and ten is eleven. Not enough. So now the one-litre tin is finished, for the mirror reason — it is sitting with the biggest thing left and still falling short, and everything else on that shelf is smaller than the ten.

Hand off the one, on to the two. Two and ten is twelve, and he is done.

The bit that took him years to be comfortable with is the other outcome. Some days the number does not come out — the owner wants exactly thirteen and there is no pair on that shelf that makes thirteen. Bashir's hands meet in the middle and he turns round and tells the man, flatly, that it cannot be done in two tins. He does not go back and check the combinations he skipped, and he is not being lazy. Every pair he did not try had one tin in it that he had already proved could not work with anything.

3 The idea in plain English

Bashir's two hands are `left` and `right`. Him being willing to say “*it cannot be done*” without going back is the completeness argument, and it is what today exists to install.

The skeleton

```
left, right = 0, len(nums) - 1
while left < right:
    total = nums[left] + nums[right]
    if total == target:
        return [left, right]
    if total < target:
        left += 1
    else:
        right -= 1
return []
```

Nine lines, and every variant today changes only what happens in those three branches.

The property that holds all the way through

Say this sentence and the rest follows:

At every moment, if a valid pair exists anywhere in the array, both of its elements lie between `left` and `right` inclusive.

That is called an **invariant** — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

True at the start, because `left` and `right` are the two ends and everything is between them.

Still true after a move. Suppose the total is too small and we advance `left`. The only pairs we lose are the ones containing `nums[left]`. Every one of those pairs is `nums[left] + something`, where `something` is at most `nums[right]` — because the array is sorted and `nums[right]` is the largest value still in range. So every lost pair sums to at most `nums[left] + nums[right]`, which we just measured and found **less than the target**. None of them could have been the answer. The same argument mirrored covers advancing `right`.

Therefore, when the loop ends with `left` and `right` having met, the range between them is empty, so by the invariant no valid pair exists anywhere. Returning “not found” is not a guess.

That is the whole proof, and it is three sentences. **Interviewers ask for it, and almost nobody has it ready.**

Why sortedness is load-bearing

The step “every `something` is at most `nums[right]`” is the only place sortedness is used — and it is used to eliminate an entire row of pairs at once. On an unsorted array it is false, so nothing about the argument survives. That is why “*is it sorted?*” is the first question, every time.

The six variants, and what changes

The skeleton stays. Only the branches move.

PROBLEM	ON ==	ON <	ON >
find one pair	return	<code>left += 1</code>	<code>right -= 1</code>
find all pairs	record, then move both , then skip duplicates	<code>left += 1</code>	<code>right -= 1</code>
count pairs $\leq$ target	—	add <code>right - left</code> , then <code>left += 1</code>	<code>right -= 1</code>
closest pair	return	update best, <code>left += 1</code>	update best, <code>right -= 1</code>
3Sum	record, move both, skip duplicates	<code>left += 1</code>	<code>right -= 1</code>
sorted squares	—	compare magnitudes, write from the back	

Two of those deserve unpacking.

Finding *all* pairs, and the duplicate problem

On a hit you must move **both** indices. Moving only one gives you the same pair again — `nums[left]` is unchanged and its partner requirement is unchanged, so nothing has progressed.

Then you must skip repeats:

```
if total == target:
    out.append((nums[left], nums[right]))
    left += 1
    right -= 1
    while left < right and nums[left] == nums[left - 1]:
        left += 1
    while left < right and nums[right] == nums[right + 1]:
        right -= 1
```

On `[1, 1, 2, 2, 3, 3, 4, 4]` with target 5, the distinct pairs are `(1,4)` and `(2,3)`. Without the two skip loops you get `(1,4)` twice and `(2,3)` twice. **Ask whether the interviewer wants distinct pairs or all index pairs** — they are different questions and the second is a counting problem.

Counting pairs, which is the neat one

“How many pairs sum to at most the target?” When `nums[left] + nums[right] ≤ target`, then **every** element between `left` and `right` also works with `nums[left]`, because they are all at most `nums[right]`. So that single comparison contributes `right - left` pairs at once:

```

if nums[left] + nums[right] ≤ target:
    count += right - left
    left += 1
else:
    right -= 1

```

That is the same “one move settles a whole row” idea, used to *count* rather than to *discard*. It is the seed of the `right - left + 1` counting trick that runs through the window problems on [day 034](#).

3Sum: fix one, two-pointer the rest

There is no three-pointer version, because no elimination argument works on three moving indices at once. So:

1. **Sort** the array.
2. **Loop** over the first element, `i`.
3. **Two-pointer** the range `i+1 .. n-1` looking for `-nums[i]`.

$O(n \log n) + O(n^2) = O(n^2)$, which is the expected answer. The difficulty is entirely duplicates, and there are three separate places to handle them — §7.

Sorted squares: opposite ends, writing backwards

“Given a sorted array that may contain negatives, return the squares in sorted order.”

`[-4, -1, 0, 3, 10]` squares to `[16, 1, 0, 9, 100]`, which is not sorted. But the **largest square is always at one end or the other**, because the largest magnitude is. So compare magnitudes at the two ends, take the bigger, and **write it into the output from the back**.

That is opposite-ends two pointers combined with the write-from-the-back idea from [day 018](#), and it is $O(n)$ where sorting the squares would be $O(n \log n)$.

Note `while left ≤ right` here, not `<`. The single middle element still has to be written, because this variant is about covering every position rather than about forming pairs of distinct elements. **The loop condition follows from what the problem is counting**, not from habit.

4 The picture

The invariant, drawn. The shaded region is where a valid pair could still be:

start	[1 2 4 5 10 20]	target 12
	^ L ^ R	1 + 20 = 21 > 12
everything eliminated by moving R:	[1 2 4 5 10][20]	every pair containing 20 sums to at least 1 + 20 = 21 > 12
	xxxx	
after R--	[1 2 4 5 10]	
	^ L ^ R	1 + 10 = 11 < 12
everything eliminated by moving L:	[1][2 4 5 10]	every pair containing 1 sums to at most 1 + 10 = 11 < 12
	xx	
after L++	[2 4 5 10]	
	^ L ^ R	2 + 10 = 12 → found

What to notice: each `xxxx` block is a whole set of pairs discarded by one comparison, and the reason is written next to it. When the two hands meet with nothing found, the un-eliminated region is empty — which is the completeness argument made visual.

The pair table, showing coverage:

	1	2	4	5	10	20	
1	-	1+2	1+4	1+5	1+10	1+20	← one L++ deletes this entire row
2	-	-	2+4	2+5	2+10	2+20	
4	-	-	-	4+5	4+10	4+20	
5	-	-	-	-	5+10	5+20	
10	-	-	-	-	-	10+20	
20	-	-	-	-	-	-	
						^	
							one R-- deletes this entire column

15 pairs in the table. The two-pointer walk touched 3 of them and eliminated the other 12 with an argument.

What to notice: 3 examined, 12 eliminated, 0 missed. At `n = 10,000` it is 10,000 examined and about 50 million eliminated.

Sorted squares, where the writing goes backwards:															
input	[	-4	-1	0	3	10	]	output	[	_	_	_	_	_	]
			^			^								^	
			L			R								write	
-4 = 4,	10 = 10	→	10 wins	output	[	_	_	_	_	100	]	R--,	write--		
-4 = 4,	3 = 3	→	4 wins	output	[	_	_	_	16	100	]	L++,	write--		
-1 = 1,	3 = 3	→	3 wins	output	[	_	_	9	16	100	]	R--,	write--		
-1 = 1,	0 = 0	→	1 wins	output	[	_	1	9	16	100	]	L++,	write--		
L = R on the 0				output	[	0	1	9	16	100	]				

What to notice: the write index moves backwards from the end, because the **largest** value is the one you can identify first. Filling forwards would need the smallest, which is in the middle and is exactly the thing you do not know.

5 The code, built step by step

The base case, with the 1-indexing trap

LeetCode 167 wants positions numbered from 1, not 0.

```
def two_sum_ii(nums: list[int], target: int) → list[int]:
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left + 1, right + 1]      # the problem is 1-indexed
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

The `+ 1`s are the only difference from yesterday's version and they are worth a sentence in the interview — “*this one is 1-indexed, so I'll adjust on the way out.*” Reading the return format before writing the return statement is a habit worth having.

All distinct pairs

The move on a hit is the interesting part.

```
if total == target:
    out.append((nums[left], nums[right]))
    left += 1
    right -= 1
```

Both, because moving one leaves the other looking for the same partner value and you emit the same pair again.

```
while left < right and nums[left] == nums[left - 1]:
    left += 1
while left < right and nums[right] == nums[right + 1]:
    right -= 1
```

Then skip past repeats of the two values you just used. `nums[left - 1]` is the value you just consumed, so this walks forward while the value is unchanged. Both loops keep the `left < right` guard, for the same reason the palindrome skip loops did on [day 023](#) — a

block of identical values could otherwise run an index off the end.

Counting pairs at or under a target

```
count = 0
left, right = 0, len(nums) - 1
while left < right:
    if nums[left] + nums[right] ≤ target:
        count += right - left          # all of nums[left+1..right] also work with nums[left]
        left += 1
    else:
        right -= 1
```

`right - left` and not `right - left + 1`, because `nums[left]` cannot pair with itself. Say the justification while you write it: *“if the largest partner works, every smaller partner works too, so I can count them all at once.”*

3Sum, and the three duplicate guards

```
nums = sorted(nums)
for i in range(len(nums) - 2):
    if i > 0 and nums[i] == nums[i - 1]:
        continue                    # guard 1: same first element as last time
    if nums[i] > 0:
        break                       # optimisation: sorted, so no three positives sum to 0
```

Guard 1 is the one people forget. Without it, `[-1, -1, 0, 1]` produces the triple `[-1, 0, 1]` twice.

```
left, right = i + 1, len(nums) - 1
while left < right:
    total = nums[i] + nums[left] + nums[right]
    if total == 0:
        out.append([nums[i], nums[left], nums[right]])
        left += 1
        right -= 1
        while left < right and nums[left] == nums[left - 1]:
            left += 1                # guard 2
        while left < right and nums[right] == nums[right + 1]:
            right -= 1              # guard 3
    elif total < 0:
        left += 1
    else:
        right -= 1
```

Three guards in three different places, for three different repetitions. **Being able to say what each one prevents, with the input it prevents it on, is the whole of this question.**

Sorted squares

```
n = len(nums)
out = [0] * n
left, right, write = 0, n - 1, n - 1
while left ≤ right:
    a, b = abs(nums[left]), abs(nums[right])
    if a > b:
        out[write] = a * a
        left += 1
    else:
        out[write] = b * b
        right -= 1
    write -= 1
```

`left ≤ right`, because every position must be filled including the middle one. And the write index starts at the end, because the value you can identify with certainty is the **largest**.

The complete solutions

```
def two_sum_ii(nums: list[int], target: int) → list[int]:
    """LeetCode 167. Sorted input, 1-INDEXED output. O(n) time, O(1) space."""
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
    return []

def all_pairs(nums: list[int], target: int) → list[tuple[int, int]]:
    """Every DISTINCT pair of values summing to target. Sorted input."""
    out: list[tuple[int, int]] = []
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            out.append((nums[left], nums[right]))
            left += 1
            right -= 1
            # move BOTH, or you re-emit
            while left < right and nums[left] == nums[left - 1]:
                left += 1
            while left < right and nums[right] == nums[right + 1]:
                right -= 1
        elif total < target:
            left += 1
        else:
            right -= 1
    return out

def count_pairs_at_most(nums: list[int], target: int) → int:
    """How many index pairs (i < j) have nums[i] + nums[j] ≤ target. Sorted input."""
    count = 0
    left, right = 0, len(nums) - 1
    while left < right:
        if nums[left] + nums[right] ≤ target:
            count += right - left
            # every partner between left+1 and right also works
            left += 1
        else:
            right -= 1
    return count

def three_sum(nums: list[int]) → list[list[int]]:
    """LeetCode 15. Distinct triples summing to zero. O(n^2) after an O(n log n) sort."""
    nums = sorted(nums)
    out: list[list[int]] = []
    n = len(nums)
    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        if nums[i] > 0:
            break
        left, right = i + 1, n - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total == 0:
                out.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1
            elif total < 0:
                left += 1
            else:
                right -= 1
```

```

        elif total < 0:
            left += 1
        else:
            right -= 1
    return out

def three_sum_closest(nums: list[int], target: int) → int:
    """LeetCode 16. The triple sum nearest to target. Same skeleton, tracking a best."""
    nums = sorted(nums)
    best = nums[0] + nums[1] + nums[2]
    for i in range(len(nums) - 2):
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if abs(total - target) < abs(best - target):
                best = total
            if total == target:
                return total
            if total < target:
                left += 1
            else:
                right -= 1
    return best

def sorted_squares(nums: list[int]) → list[int]:
    """LeetCode 977. Sorted input with negatives. O(n) by writing from the BACK."""
    n = len(nums)
    out = [0] * n
    left, right, write = 0, n - 1, n - 1
    while left ≤ right:
        # ≤, because every position must be filled
        a, b = abs(nums[left]), abs(nums[right])
        if a > b:
            out[write] = a * a
            left += 1
        else:
            out[write] = b * b
            right -= 1
        write -= 1
    return out

if __name__ == "__main__":
    print([two_sum_ii(a, t) for a, t in (([2, 7, 11, 15], 9), ([2, 3, 4], 6), ([-1, 0], -1))])
    # [[1, 2], [1, 3], [1, 2]]

    print(all_pairs([1, 1, 2, 2, 3, 3, 4, 4], 5)) # [(1, 4), (2, 3)]
    print(all_pairs([0, 0, 0, 0], 0)) # [(0, 0)]
    print(all_pairs([1, 2, 3, 4, 5], 6)) # [(1, 5), (2, 4)]

    print(count_pairs_at_most([1, 2, 3, 4], 5)) # 4 → (1,2)(1,3)(1,4)(2,3)

    print(three_sum([-1, 0, 1, 2, -1, -4])) # [[-1, -1, 2], [-1, 0, 1]]
    print(three_sum([0, 0, 0, 0])) # [[0, 0, 0]]
    print(three_sum([0, 1, 1])) # []

    print(three_sum_closest([-1, 2, 1, -4], 1)) # 2

    print(sorted_squares([-4, -1, 0, 3, 10])) # [0, 1, 9, 16, 100]
    print(sorted_squares([-7, -3, 2, 3, 11])) # [4, 9, 9, 49, 121]

```

6 What it costs

The base case

Every turn moves exactly one index, `left` only forward and `right` only backward, and they start `n-1` apart. So the loop runs at most `n - 1` times, each turn doing one addition and at most two comparisons.

$O(n)$ time. $O(1)$ extra space.

Against the nested loop's `n(n-1)/2` pair checks, measured on inputs where no pair exists:

<code>n = 1,000</code>	two pointers	0.000067 s	nested loop	0.0225 s	334x
<code>n = 5,000</code>	two pointers	0.000332 s	nested loop	0.5923 s	1,782x
<code>n = 10,000</code>	two pointers	0.000679 s	nested loop	2.3448 s	3,454x

The ratio doubling with `n` is the $O(n)$ -versus- $O(n^2)$ signature.

`all_pairs` and `count_pairs_at_most`

Same structure, so still **$O(n)$ time**. The duplicate-skip loops do not change that: they only ever move `left` forward and `right` backward, so across the whole run the two indices still travel at most `n` positions in total. **Nested `while` loops do not imply $O(n^2)$ — count the travel, not the nesting.**

`all_pairs` is $O(k)$ extra space for `k` output pairs; `count_pairs_at_most` is $O(1)$.

`three_sum`

Sorting is $O(n \log n)$. The outer loop runs `n - 2` times and each inner two-pointer scan is $O(n)$, so the scanning is $O(n^2)$, which dominates.

$O(n^2)$ time. Space is $O(1)$ beyond the output if you may sort in place, or $O(n)$ if you must copy first — and $O(k)$ for the `k` triples returned.

Concretely, at `n = 3,000`:

brute force, all triples	: 3,000 choose 3 $\approx$ 4.5 billion	– not runnable
sort + two pointers	: $\sim$ 9 million inner steps	– a few seconds in Python

There is no known sub-quadratic algorithm for 3Sum, and saying so is a better answer than searching for one. It is a well-studied problem, and $O(n^2)$ is the expected bound.

`sorted_squares`

One pass, `n` writes: **$O(n)$ time**, $O(n)$ for the output and $O(1)$ beyond it.

The obvious alternative — square everything, then sort — is $O(n \log n)$. At $n = 1,000,000$ that is about 20 million comparisons against 1 million steps, so roughly $20\times$. Worth mentioning as the one-liner you are choosing not to use.

The number to have ready

Two pointers examines about n pairs and eliminates the other $n(n-1)/2 - n$ with an argument. On $[1, 2, 4, 5, 10, 20]$ that is 3 pairs examined out of 15. At $n = 10,000$ it is 10,000 out of 50 million, measured at about $3,500\times$ faster.

7 The traps

The near-miss: moving only one index on a hit

```
if total == target:
    out.append((nums[left], nums[right]))
    left += 1                # right not moved
```

On $[1, 2, 3, 4]$ with target 5 this finds $(1, 4)$, then advances to $2 + 4 = 6$, which is too big, so `right` drops to 3 and it finds $(2, 3)$. It happens to work here. Now try $[1, 1, 4, 4]$ with target 5: you get $(1, 4)$, then `left` moves to the second 1, and $1 + 4 = 5$ again — the same pair of *values* emitted twice.

On a hit, both indices must move, because the pair is consumed. Then the skip loops handle repeats.

The near-miss: forgetting duplicate skipping in 3Sum

```
for i in range(len(nums) - 2):
    # no guard on nums[i] == nums[i-1]
    ...
print(three_sum([-1, 0, 1, 2, -1, -4]))
```

```
[[-1, -1, 2], [-1, 0, 1], [-1, 0, 1]]
```

$[-1, 0, 1]$ appears twice, because after sorting there are two `-1`s and the outer loop runs the same two-pointer scan for each. The problem asks for **distinct** triples.

Three separate guards are needed and they are not interchangeable:

- **Guard 1**, on the outer element: `if i > 0 and nums[i] == nums[i-1]: continue`.
- **Guards 2 and 3**, after a hit, skipping repeats of `left` and `right`.

The input that finds a missing guard 1 is `[-1, 0, 1, 2, -1, -4]`. The input for guards 2 and 3 is `[-2, 0, 0, 2, 2]`. Test with both.

The near-miss: not sorting before 3Sum

The two-pointer scan inside 3Sum requires the sub-array to be sorted, for exactly the reason from §3. Skip the sort and it silently returns incomplete results — no error, just missing triples. And the duplicate guards also depend on sortedness, because they assume equal values are adjacent.

The near-miss: 0-indexed output on a 1-indexed problem

LeetCode 167 wants `[1, 2]` for the first two elements. Returning `[0, 1]` is a wrong answer for completely correct code, and it costs a submission. **Read the return format before writing the return.**

The near-miss: `left < right` in sorted squares

```
while left < right:                                # should be ≤
```

The middle element is never written, so `sorted_squares([-1, 0, 1])` returns `[1, 0, 1]` — position 1 keeps its initial 0 by accident, which happens to look plausible. On `[-4, -1, 0, 3, 10]` you get `[0, 1, 9, 16, 100]`, still correct by luck because the middle value is 0.

The loop condition depends on what you are covering. Pair problems want `<` because the two must be distinct elements. Position-filling problems want `≤` because every position needs a value. Say which one you are doing.

The near-miss: assuming the input is sorted when it is not

```
print(all_pairs([3, 1, 4, 2], 5))
```

```
[]
```

There are two pairs summing to 5. Nothing is found and nothing errors, because the elimination argument is false on unsorted input. **Sortedness is a precondition.** If the problem does not guarantee it, sort — and then say what that costs, and that it destroys original indices.

The contract corner: distinct values or distinct positions?

“Find all pairs summing to the target” is ambiguous. On `[1, 1, 4, 4]` with target 5:

- **Distinct value pairs:** one answer, `(1, 4)`.

- **Distinct index pairs:** four answers — each of the two 1s with each of the two 4s.

The second is a **counting** problem and two pointers with skip loops gives the wrong answer for it; you multiply the run lengths instead. Ask which one is wanted.

8 In the interview

How it gets asked

- “Find two numbers in a sorted array that add up to the target.” — LeetCode 167. Check the indexing.
- “How do you know you haven’t missed a pair?” — the completeness question, and the reason today exists.
- “Now find all such pairs.” — where duplicate handling appears.
- “3Sum.” — LeetCode 15, and the duplicates are the question.
- “Square a sorted array containing negatives, keeping it sorted.” — LeetCode 977, the write-from-the-back variant.

What to say out loud, in the first ninety seconds

1. **Confirm sortedness and the return format.** “It’s sorted — good. And do you want indices or values, and are indices 0-based or 1-based?”
2. **State the brute force and its cost.** “Every pair is $O(n^2)$.”
3. **Name the shape.** “Two indices at opposite ends, moving inwards.”
4. **Give the move rules with the reason attached.** “Too small, advance left: that value is paired with the largest available and still short, so it can’t work with anything. Too large, retreat right, by the mirror argument.”
5. **Give the completeness argument, unprompted.** “The property that holds throughout is that any valid pair lies between the two indices. Each move only discards pairs I’ve just proved can’t reach the target. So when they meet with nothing found, there genuinely is nothing.”
6. **Give the cost and the space.** “ $O(n)$ time, $O(1)$ space.”
7. **Say what would change your approach.** “If it weren’t sorted I’d use a hash map — $O(n)$ time, $O(n)$ space, and it keeps the original indices.”

The follow-ups

“How do you know you haven’t skipped the answer?” Because of an invariant I can state precisely: at every point in the loop, if a valid pair exists anywhere in the array, both of its elements are between `left` and `right`. It is true at the start because those are the two ends. It stays true across a move: when I advance `left` because the sum was

too small, the only pairs I lose are those containing `nums[left]`, and every one of them sums to at most `nums[left] + nums[right]` — since the array is sorted, `nums[right]` is the largest partner available — which I just measured as below the target. So every discarded pair was already known to fall short. The mirror argument covers retreating `right`. Therefore when the two meet, the range between them is empty, and by the invariant no valid pair exists. That is a proof rather than a hope, and the one place it uses sortedness is the claim that `nums[right]` is the largest remaining partner.

“Now return all the pairs, not just one.” Two changes. On a hit I move **both** indices rather than one, because the pair is consumed — moving only `left` leaves `right` on the same value and I would emit the same pair again on the next equal element. Then I skip forward over repeats of both values I just used, so `[1,1,2,2,3,3,4,4]` with target 5 gives `(1,4)` and `(2,3)` once each rather than twice. Before writing any of that I would ask what “all pairs” means: distinct pairs of **values**, or distinct pairs of **positions**. On `[1,1,4,4]` the first has one answer and the second has four, and the second is really a counting problem — I would multiply the lengths of the two runs rather than emitting pairs. It is a genuine ambiguity and worth three seconds.

“3Sum. And why can’t you use three pointers?” Because there is no elimination argument for three simultaneously moving indices — with two, “too small” identifies exactly one element that can be discarded, and with three, a sum being too small tells you nothing about which of the three to move. So the structure is: sort, then fix the first element with an outer loop, then run two pointers on the remainder looking for its negation. That is $O(n \log n)$ for the sort plus $O(n^2)$ for the scanning, so $O(n^2)$ overall, and there is no known sub-quadratic algorithm for this problem. The real difficulty is duplicates, and there are three separate guards in three places: skip a repeated outer element, and after recording a triple, skip repeated values at both `left` and `right`. Each one prevents a different repetition, and I would name the input that catches each.

“Square a sorted array with negatives, keeping it sorted, in $O(n)$.” Squaring destroys the order because a large negative squares to a large positive. But the **largest** square is always at one end or the other, since the largest magnitude is at one end. So I use two indices at the ends, compare the absolute values, take the bigger, square it, and write it into the output starting from the **back**. Then move whichever index I consumed. Writing backwards is the key move — I can identify the largest remaining value with certainty at each step, and the smallest is somewhere in the middle where the sign changes, which is exactly the thing I do not know. $O(n)$ time against $O(n \log n)$ for square-then-sort, which at a million elements is about 20 times fewer operations. And the loop is `left ≤ right` here, not `<`, because every output position must be filled including the middle one.

A model answer

“First: it’s sorted, and do you want the indices or the values? And if indices, 0-based or 1-based?

...Indices, 1-based. Noted — I’ll adjust on the way out.

The brute force checks every pair, which is n choose 2, so $O(n^2)$. Because it’s sorted I can do it in one pass with two indices, one at each end, moving inwards.

```
def two_sum(nums: list[int], target: int) → list[int]:
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

The move rules each have a reason. If the sum is too small, `nums[left]` is the smallest value still available and it’s currently paired with the largest one available — that’s its best case, so if it falls short, no pair containing it can reach the target, and I discard it. Too large is the mirror argument on `nums[right]`.

The part I’d want to be explicit about is why this doesn’t miss anything, because that’s the interesting claim. The invariant is: at every moment, if a valid pair exists anywhere, both of its elements are between `left` and `right`. True at the start, since those are the ends. Preserved by each move, because the pairs I discard are exactly the ones I’ve just proved fall on the wrong side of the target. So when the indices meet with nothing found, the remaining range is empty and there genuinely is no such pair — that’s a proof, not an assumption.

Sortedness is used in exactly one place: the claim that `nums[right]` is the largest partner still available. That’s why the whole thing collapses on unsorted input — it returns nothing, with no error, which is the dangerous kind of wrong.

Cost: at most $n-1$ turns, each doing constant work, so $O(n)$ time and $O(1)$ extra space. Against the nested loop that’s about 3,500 times faster at ten thousand elements, and the gap doubles every time n doubles.

Edge cases: fewer than two elements, where the loop never runs and I return empty; and equal values, where `left < right` correctly allows two different positions holding the same number. If the array weren’t sorted I’d switch to a hash map —

same $O(n)$ time, $O(n)$ space, and it preserves the original indices, which sorting would destroy.”

9 Recall card

- **The invariant:** any valid pair lies between `left` and `right`. Each move discards only pairs already proved impossible — so meeting with nothing found is a **proof**, not a guess.
- **Sortedness is used once:** `nums[right]` is the largest remaining partner. That is the whole dependency.
- **On a hit, move both indices**, then skip repeats of both values. Otherwise you emit the same pair twice.
- **3Sum = sort, fix one, two-pointer the rest.** $O(n^2)$, three duplicate guards in three places.
- `left < right` for pairs; `left ≤ right` when filling positions, as in sorted squares, where you write from the back.

1 What this is, and why they ask it

A **join** combines rows from two tables by matching them on a shared value. On [day 026](#) you put `author_id` on the posts table so a post could point at its author; a join is how you follow that pointer and get the author's name back alongside the post.

There are four kinds, and the only thing that separates them is **what happens to rows that find no match**:

- **Inner** — drop them. Only matched pairs survive.
- **Left** — keep everything from the left table, filling the right side with `NULL` where there was no match.
- **Right** — the mirror image.
- **Full** — keep unmatched rows from both sides.

That is the whole concept, and it takes ten seconds to say. The reason it gets a whole day is the second half of the question: **how many rows come out?** Almost everyone can define an inner join and almost nobody can predict, without thinking, that joining three rows to two rows on the same key gives six. Row multiplication is where real queries go wrong — a report that double-counts revenue is usually a join fan-out, not a bug in the arithmetic.

Interviewers ask it constantly because it is fast, unfakeable, and the follow-ups have real depth: *why did my `COUNT` go up?, why did `LEFT JOIN ... WHERE right.col = 'x'` silently become an inner join?, and what does the database actually do to execute this?*

2 The story

Sarita's brother got married in December and the reception was at a hall on the bypass, and the thing she remembers most clearly is the two lists.

Her father had been keeping one for four months — everybody invited, about six hundred names, typed into his phone a few at a time as he thought of people. Her cousin Anwar stood at the gate on the night with his own phone and typed in everyone who actually walked in.

The week after, sitting on the floor with both phones, they went through it.

The first thing they wanted was straightforward: who was invited **and** came. Six hundred on one side, five hundred and ten on the other, and the names that appeared on both lists came to four hundred and sixty. Those were the people they had to thank.

Then Sarita's mother wanted something different. She wanted the whole invitation list, all six hundred, with a mark against the ones who had not turned up — because she wanted to know who to telephone. That is a different question and it gives a different-sized answer: six hundred rows, with a hundred and forty of them blank on the right-hand side.

Anwar wanted the opposite. He had fifty names on his list that were not on his uncle's at all — neighbours, somebody's driver, a whole family nobody could identify — and he wanted those, because he thought it was funny.

And Sarita's father, who is thorough, wanted one list with everybody on it from either side, marked one way or the other. Six hundred and fifty.

The part that caused an actual argument was smaller and stranger. One man — a supplier of her father's — had been invited twice, because her father had entered him once under his name and once under his firm's name. And he had come twice, in a manner of speaking: he signed in at the gate, went out to move his car, and Anwar typed him in again.

So when they matched the two lists, that one man produced four lines. Two invitations against two arrivals. Sarita said that could not be right and her father showed her, and it was right — the matching does what it says, and if a name is on one side twice and the other side twice, it comes out four times.

3 The idea in plain English

The invitation list is the **left table**. The arrivals list is the **right table**. The four questions Sarita's family asked are the four joins. And the man who appeared four times is **row multiplication**, which is the thing you must be able to predict.

The four joins, on one example

Two small tables:

customers

id	name
1	Asha
2	Ravi
3	Nita
4	Sam

orders

id	customer_id	amount
10	1	500
11	1	300
12	2	900
13	NULL	150

Asha has two orders. Ravi has one. Nita and Sam have none. Order 13 belongs to nobody.

JOIN	ROWS OUT	WHAT COMES BACK
INNER JOIN	3	Asha × 2, Ravi × 1. Nita, Sam and order 13 all vanish.
LEFT JOIN	5	those 3, plus Nita and Sam with NULL order columns.
RIGHT JOIN	4	those 3, plus order 13 with NULL customer columns.
FULL OUTER JOIN	6	those 3, plus Nita, Sam, and order 13.

Those four numbers — 3, 5, 4, 6 — are worth being able to produce from that table in your head, because that is precisely what “predict the row count” means.

The syntax

```
SELECT c.name, o.amount
FROM   customers c
JOIN   orders o ON o.customer_id = c.id;           -- INNER is the default

SELECT c.name, o.amount
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.id;       -- LEFT OUTER JOIN, "OUTER" is optional
```

`JOIN` on its own means `INNER JOIN`. `LEFT JOIN` means `LEFT OUTER JOIN`. The word `OUTER` is noise and nobody writes it.

`ON` is not `WHERE`. `ON` says how the rows are matched; `WHERE` filters the result afterwards. For an inner join the distinction rarely matters. For an outer join it changes the answer completely, and that is §7.

Row multiplication — the part that matters

A join is not a lookup. It is a matching, and matching can multiply.

For each row on the left, the output contains **one row per match on the right**. So:

RELATIONSHIP	LEFT ROWS	MATCHES EACH	ROWS OUT
one-to-one	100	1	100
one-to-many	100 customers	avg 5 orders	500
many-to-many	3 rows with key 1	2 rows with key 1	6

That last line is Sarita's supplier. Three on one side and two on the other, on the same key, gives $3 \times 2 = 6$. Not 3, not 2, not 5.

The consequence that bites in real work: joining an orders table to an order-items table and then summing `orders.total` double-counts, because each order's total now appears once per item.

```
-- WRONG: an order with 4 items contributes its total 4 times
SELECT SUM(o.total)
FROM   orders o
JOIN   order_items i ON i.order_id = o.id;
```

The fixes are to aggregate before joining, or to use `SUM(DISTINCT ...)` carefully, or to not join at all when you only need one side. **When a total looks too big after adding a join, this is why.**

LEFT JOIN and counting

The most common real use of a left join is “everyone, with their count, including the zeroes”:

```
SELECT  c.id, c.name, COUNT(o.id) AS order_count
FROM    customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name;
```

`COUNT(o.id)`, not `COUNT(*)`. A customer with no orders still produces one output row, with all the order columns `NULL`. `COUNT(*)` counts that row and reports 1. `COUNT(o.id)` counts non-null values and correctly reports 0.

On the example above: Asha 2, Ravi 1, Nita 0, Sam 0 — with `COUNT(o.id)`. With `COUNT(*)` you get Asha 2, Ravi 1, Nita 1, Sam 1, which is wrong and looks entirely plausible.

Anti-joins: “the ones with none”

“Which customers have never ordered?” Three ways:


```

-- 1. LEFT JOIN ... IS NULL
SELECT c.* FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;

-- 2. NOT EXISTS ← usually the best
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- 3. NOT IN ← dangerous
SELECT c.* FROM customers c
WHERE c.id NOT IN (SELECT customer_id FROM orders);

```

Prefer NOT EXISTS. It can stop at the first match rather than building all of them, and — the important part — **NOT IN** returns no rows at all if the subquery contains a single NULL. In the example above, order 13 has customer_id NULL, so version 3 returns an empty result and version 2 returns Nita and Sam. That is a silent, complete wrong answer, and it is one of the nastiest gotchas in SQL.

Self-joins

A table can be joined to itself, which is how you compare rows within one table — an employee to their manager, a comment to its parent:

```

SELECT e.name AS employee, m.name AS manager
FROM   employees e
LEFT JOIN employees m ON m.id = e.manager_id;

```

LEFT, so the chief executive — who has no manager — still appears. The aliases e and m are what make it work; without them the database cannot tell which copy you mean.

CROSS JOIN

Every row with every row, with no condition: $m \times n$ rows. Rarely what you want, occasionally exactly what you want — generating every date-and-product combination for a report so that missing days show as zero. And **an accidental cross join is what you get when you forget the ON clause**, which on two tables of a million rows is a trillion rows and a very unhappy afternoon.

4 The picture

The four joins, drawn as what survives:

customers (left)

```
+-----+
|  1 2 |
|  3 4 |
+-----+
```

orders (right)

```
+-----+
|  1 2 |
| 13   |
+-----+
```

1 and 2 match; 3, 4 and 13 do not

INNER

```
+-----+
|  #### |
|  #### |
+-----+
```

matched only
3 rows

LEFT

```
+-----+
|#####|
|#####|
+-----+
```

all of left
5 rows
(3 + Nita + Sam)

RIGHT

```
+-----+
|#####|
|  ##   |
+-----+
```

all of right
4 rows
(3 + order 13)

FULL

```
+-----+
|#####|
|#####|
+-----+
```

everything
6 rows
(3 + 2 + 1)

What to notice: the row counts are not 3, 4, 4, 5. **LEFT** is 5 because Asha's *two* orders already made two rows before the unmatched customers were added. **Row counts come from matches, not from table sizes**, and that is the thing to internalise.

Row multiplication, drawn:

left rows with key 1

```
+-----+
|  x   |
|  y   |
|  z   |
+-----+
```

3

JOIN
on key

×

right rows with key 1

```
+-----+
|  p   |
|  q   |
+-----+
```

2

=

output

```
+-----+
| x p   |
| x q   |
| y p   |
| y q   |
| z p   |
| z q   |
+-----+
```

6

What to notice: every left row is paired with every matching right row. This is why an order with four items makes the order's total appear four times, and why a **SUM** over that join is wrong.

The **ON**-versus-**WHERE** trap, which is the highest-value picture on this page:

```
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'completed'
```

1. the LEFT JOIN produces: Asha|500 Asha|300 Ravi|900 Nita|NULL Sam|NULL
2. then WHERE runs on that: NULL = 'completed' is UNKNOWN, so Nita and Sam are DROPPED

the LEFT JOIN has silently become an INNER JOIN

```
LEFT JOIN orders o ON o.customer_id = c.id AND o.status = 'completed'
```

the condition is part of the MATCHING, so Nita and Sam still appear with NULLs

What to notice: the two queries differ by moving one condition from `WHERE` to `ON`, and they return different numbers of rows. This is the single most common outer-join bug, and interviewers ask about it directly.

5 How it actually works

The three join algorithms

The planner picks one; you should be able to name all three and say when each wins.

Nested loop join. For each row on the left, scan the right table for matches. $O(n \times m)$ unless there is an index on the right side's join column, in which case each lookup is a B-tree probe and it becomes $O(n \log m)$. **Best when the left side is small and the right side is indexed** — which is why an unindexed foreign key turns this from fast into catastrophic.

Hash join. Build a hash table from the smaller table's join column, then scan the larger table probing it. $O(n + m)$. **The workhorse for large unindexed joins**, and it needs the smaller side to fit in memory — if it does not, it spills to disk in batches and gets much slower.

Merge join. Sort both sides by the join key, then walk them together with two pointers — literally today's DSA lesson, applied to two sequences instead of one. $O(n \log n + m \log m)$ if sorting is needed, or $O(n + m)$ if both sides already arrive sorted, which they do when you join on indexed columns. **Best for large, already-sorted inputs.**

That merge join is worth pausing on: **the database is running the two-pointer walk from the DSA side of today**, with one index into each sorted stream, advancing whichever is behind. It is the same idea in a different costume.

Reading it in a plan

```
EXPLAIN ANALYZE
SELECT c.name, o.amount
FROM customers c JOIN orders o ON o.customer_id = c.id;
```

```
Hash Join  (cost=1.09..21.03 rows=500 width=68) (actual time=0.03..2.4 rows=500 loops=1)
  Hash Cond: (o.customer_id = c.id)
    → Seq Scan on orders o  (cost=0.00..16.00 rows=600 width=12)
    → Hash  (cost=1.04..1.04 rows=4 width=64)
       → Seq Scan on customers c  (cost=0.00..1.04 rows=4 width=64)
```

Read it inside out: scan `customers`, build a hash of it, then scan `orders` probing that hash. The thing to check is **estimated versus actual rows** — `rows=500` estimated against `rows=500` actual is healthy; a large gap means stale statistics and probably a bad

plan choice.

Indexes and joins

Index the join columns. From [day 026](#), primary keys are indexed automatically and foreign keys are not, so `orders.customer_id` needs an index you create yourself. Without it, a nested loop join scans the whole orders table once per customer.

```
1,000 customers × 1,000,000 orders, nested loop, no index:  
  1,000 × 1,000,000 = 1,000,000,000 row comparisons  
  
with an index on orders(customer_id):  
  1,000 × ~4 page reads = 4,000 reads
```

Joining more than two tables

```
SELECT c.name, o.id, i.product_name, i.quantity  
FROM   customers c  
JOIN   orders    o ON o.customer_id = c.id  
JOIN   order_items i ON i.order_id = o.id  
WHERE  o.status = 'completed';
```

The joins apply left to right, each one to the accumulated result. **Every join is another chance to multiply rows**, so a customer with 5 orders of 4 items each contributes 20 rows, not 5 and not 4. Three or four joins is normal; when you are past six, either the schema is over-normalised for this access pattern or the query is doing two unrelated jobs.

Doing it without a join

Two alternatives worth knowing, because interviewers ask “*could you avoid the join?*”:

- **A subquery in SELECT** — `(SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.id) AS n` — runs once per outer row, which is the N+1 problem in SQL form. Readable, and slower.
- **Two queries plus application-side stitching** — fetch customers, collect the ids, fetch all their orders with `WHERE customer_id IN (...)`, and match them up in code. This is exactly what DataLoader does for GraphQL on [day 021](#), and it is genuinely the right answer when the two tables live in different databases or different services.

Products

Every relational database does all four joins. **MySQL has no `FULL OUTER JOIN`** — you emulate it with a `LEFT JOIN` unioned with a `RIGHT JOIN` — which is a small, real difference worth knowing. **Vitess and PlanetScale** restrict cross-shard joins, and **Cas-**

sandra and DynamoDB have no joins at all, which is why data modelling in those stores means pre-joining at write time and duplicating data on purpose.

6 The numbers

Row counts, which is the whole skill

```
customers: 4 rows      orders: 4 rows (one with a NULL customer_id)
matches:  Asha 2, Ravi 1, Nita 0, Sam 0, orphan order 1
```

```
INNER : 2 + 1          = 3
LEFT  : 2 + 1 + 1(Nita) + 1(Sam) = 5
RIGHT : 2 + 1 + 1(orphan)      = 4
FULL  : 2 + 1 + 1 + 1 + 1      = 6
CROSS : 4 × 4              = 16
```

At realistic sizes:

```
10,000 customers, 50,000 orders, average 5 orders each, 2,000 customers with none:
```

```
INNER : 50,000          (every order matches exactly one customer)
LEFT  : 50,000 + 2,000 = 52,000 (the ones with none appear once each)
CROSS : 10,000 × 50,000 = 500,000,000 - half a billion rows from a missing ON clause
```

The double-counting bug, quantified

10,000 orders, average 4 items each, average order total ₹2,000:

```
true revenue          = 10,000 × 2,000 = ₹20,000,000
SUM(o.total) after joining to items = 40,000 rows × 2,000 = ₹80,000,000
```

Four times the real figure, and the query is syntactically perfect. A report showing four times the revenue usually gets noticed; one showing $1.3\times$ because the average is 1.3 items does not.

Join algorithms, compared

Joining 10,000 customers to 1,000,000 orders:

```
nested loop, no index      : 10,000 × 1,000,000 = 1010 comparisons      - minutes to hours
nested loop, indexed right : 10,000 × ~4 page reads = 40,000 reads        - ~50 ms
hash join                  : build 10,000-entry hash + scan 1,000,000    - ~300 ms
merge join, both sorted    : 1,010,000 sequential steps                - ~200 ms
```

The unindexed nested loop is five orders of magnitude worse than everything else, and that single comparison is the reason “index your foreign keys” is a rule rather than a suggestion.

Hash join memory

```
build side: 10,000 rows × ~64 bytes = 640 KB    → fits in work_mem (4 MB default), fast
build side: 1,000,000 rows × 64 B   = 64 MB      → spills to disk in batches
```

Postgres will choose the smaller table as the build side automatically. When a join suddenly gets slow after data growth, a hash join spilling to disk is a common cause — and `work_mem` is the knob.

`COUNT(*)` versus `COUNT(col)` on a left join

4 customers, 3 matched order rows, 2 customers with no orders

<code>COUNT(*)</code>	→ Asha 2, Ravi 1, Nita 1, Sam 1	WRONG (the NULL row is still a row)
<code>COUNT(o.id)</code>	→ Asha 2, Ravi 1, Nita 0, Sam 0	RIGHT

7 The trade-offs

`ON` or `WHERE` for an outer join

Not a preference — they mean different things, and choosing wrongly changes the result.

A condition in `ON` is part of the **matching**: rows that fail it simply do not match, and the left row survives with `NULL`s. A condition in `WHERE` runs **after** the join, and since `NULL` fails every comparison, it removes the unmatched rows entirely — turning your left join into an inner join silently.

Rule: conditions on the right-hand table of an outer join go in `ON`. Conditions on the left-hand table go in `WHERE`.

Join, or aggregate first

Joining then aggregating multiplies rows before you sum them. Aggregating first — in a subquery or CTE — keeps the join one-to-one:

```
-- safer: one row per order before joining anything else
SELECT c.name, SUM(o.total)
FROM   customers c
JOIN   (SELECT customer_id, total FROM orders WHERE status = 'completed') o
      ON o.customer_id = c.id
GROUP BY c.name;
```

Aggregate before joining whenever you are summing something from the “one” side, because that is exactly when fan-out corrupts the total.

Join in the database, or stitch in the application

The database join happens next to the data, uses indexes, and sends only the result. Application-side stitching — two queries and a dictionary — costs an extra round trip and moves work off the database, which is the hardest thing to scale because there is usually one primary and many application servers.

Join in the database by default. Stitch in the application when the two tables are in different databases or owned by different services, where a join is not available at all.

Normalised with joins, or denormalised without

A join is the price of storing each fact once. Copying the customer's name onto every order removes the join and creates a second copy to keep in step. Same trade as [day 026](#), and the deciding number is the read-to-write ratio.

`NOT IN` , `NOT EXISTS` , or `LEFT JOIN ... IS NULL`

All three express “the ones with none”. `NOT EXISTS` is usually fastest, because it can stop at the first match. `LEFT JOIN ... IS NULL` is fine and reads well. `NOT IN` is the one to avoid, because a single `NULL` in the subquery makes it return nothing at all — silently.

The sentence that separates candidates

I would not join and then aggregate here. The moment I join orders to order-items, every order's total appears once per item, so summing it gives four times the real revenue on an average of four items — and the query is syntactically perfect, so nothing warns me. I would aggregate the items to one row per order first, then join. More generally, before I write any join I want to be able to say how many rows it will produce, because row counts come from the number of matches, not from the size of the tables.

8 In the interview

How it gets asked

- “What’s the difference between an inner join and a left join?” — the definition, in ten seconds, followed immediately by a row-count question.
- “How many rows does this return?” — with two small tables on the board. This is the real test.
- “Find the customers who have never ordered.” — the anti-join question, and the `NOT IN` trap.

- “Why did my total go up when I added a join?” — fan-out, and it is a genuinely common real bug.
- “How does the database actually execute a join?” — nested loop, hash, merge, and when each wins.

What to say out loud, in the first ninety seconds

1. **Define by what happens to unmatched rows.** *“The only difference between the join types is what happens to rows with no match. Inner drops them; left keeps everything from the left table with NULLs on the right; right is the mirror; full keeps unmatched rows from both.”*
2. **Give a concrete row count immediately.** *“With four customers where two have orders and two don’t, and one orphan order — inner gives 3, left 5, right 4, full 6.”*
3. **Raise fan-out before being asked.** *“The thing worth being careful about is that a join can multiply rows: three rows on one side matching two on the other gives six, not five.”*
4. **Name the counting trap.** *“And with a left join, use `COUNT(o.id)` rather than `COUNT(*)` — unmatched customers still produce a row, so `COUNT(*)` reports 1 where the answer is 0.”*
5. **Name the `ON`-versus-`WHERE` trap.** *“A condition on the right-hand table of a left join has to go in `ON`, not `WHERE`, or the NULLs get filtered out and it becomes an inner join.”*
6. **Mention indexes.** *“And the join column on the right side needs an index, or a nested loop join scans the whole table once per row on the left.”*

The follow-ups

“How many rows does this return?” It depends entirely on the number of **matches**, not on the sizes of the tables, and that is the whole point of the question. For an inner join it is the sum over each left row of how many right rows match it — so if one customer has two orders and another has one, that is three rows from two customers. For a left join, add one row for each left row with no match at all. For a right join, mirror it. For a full join, take the inner join and add the unmatched rows from both sides. The case people get wrong is many-to-many: three rows with key `1` on the left and two with key `1` on the right give six output rows, because every left row pairs with every matching right row. And a missing `ON` clause gives a cross join, which is the product of the two table sizes — on two tables of a million rows, a trillion rows.

“Why did my revenue total go up when I added a join to order items?” Fan-out. Before the join there was one row per order carrying its total. After joining to items, an order with four items becomes four rows, each still carrying the full order total — so `SUM(o.total)` adds it four times. On an average of four items that is four times the real revenue, and nothing about the query is syntactically wrong, which is what makes it dangerous. Two fixes. Aggregate the items to one row per order in a subquery or CTE first,

then join, so the join stays one-to-one. Or, if I only needed the order totals, do not join to items at all — the join was added to get a column I could have got separately. The general habit is to be able to state the expected row count before running any query that joins and aggregates.

“Find the customers who have never ordered.” `NOT EXISTS` is what I would write: `SELECT * FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id)`. It reads as the question, and it can stop at the first matching order rather than materialising all of them. `LEFT JOIN orders ... WHERE orders.id IS NULL` is equivalent and fine — join everything, keep the rows where the join found nothing. The one I would avoid is `NOT IN` against a subquery, because if that subquery yields even a single `NULL`, the whole `NOT IN` evaluates to unknown for every row and the query returns **nothing at all**. In a table where `customer_id` is nullable, that is a silent, complete wrong answer, and it is the sort of bug that survives review.

“How does the database actually run a join?” Three algorithms and the planner chooses. A **nested loop** takes each row on the left and looks up matches on the right — fine when the left side is small and the right side’s join column is indexed, because each lookup is a B-tree probe, and disastrous without the index, since it becomes a full scan per row. A **hash join** builds a hash table from the smaller side’s join column and probes it while scanning the larger side, which is $O(n + m)$ and is the usual choice for large unindexed joins, provided the build side fits in `work_mem` — if it does not, it spills to disk and slows down sharply. A **merge join** sorts both sides by the join key and walks them together with one index into each, advancing whichever is behind — which is exactly the two-pointer walk, and it is the best option when both inputs already arrive sorted, as they do when joining on indexed columns. `EXPLAIN` tells you which was chosen, and a large gap between estimated and actual rows usually means stale statistics rather than a bad query.

“The only thing separating the join types is what happens to rows that find no match on the other side.

An **inner join** keeps only matched pairs. A **left join** keeps every row from the left table, filling the right-hand columns with NULLs where nothing matched. A **right join** is the mirror. A **full outer join** keeps unmatched rows from both sides.

Concretely — four customers, where Asha has two orders, Ravi has one, Nita and Sam have none, plus one order with no customer at all. Inner gives 3 rows. Left gives 5 — the same 3 plus Nita and Sam. Right gives 4 — the same 3 plus the orphan order. Full gives 6.

The thing I'd flag before being asked is that a join can multiply rows. It isn't a lookup — it's a matching, and for each left row you get one output row per match. Asha's two orders already produce two rows before any unmatched customers are added, which is why left is 5 and not 4. And if a key appears three times on one side and twice on the other, you get six rows, not five.

That matters in practice because of double counting. If I join orders to order items and then sum the order total, each order's total appears once per item — on an average of four items that's four times the real revenue, from a query that's syntactically perfect. The fix is to aggregate the items down to one row per order first, then join.

Two other things I'd watch on a left join specifically. First, use `COUNT(o.id)` rather than `COUNT(*)` when counting per customer — an unmatched customer still produces a row with NULLs, so `COUNT(*)` says 1 where the right answer is 0. Second, a condition on the right-hand table has to go in the `ON` clause, not `WHERE`. If it goes in `WHERE`, it runs after the join, and since NULL fails every comparison it removes exactly the unmatched rows — your left join has silently become an inner join.

And for it to be fast, the join column on the right needs an index. Without one, a nested loop join scans the whole right table once per left row — ten thousand customers against a million orders is ten billion comparisons, against about forty thousand page reads with the index.”

9 Recall card

- **The only difference is what happens to unmatched rows.** Inner drops them; left keeps the left side; right mirrors; full keeps both.

- **Row counts come from matches, not table sizes.** 3 left rows $\times$ 2 right rows on the same key = 6.
- `COUNT(o.id)` , not `COUNT(*)` , **after a left join** — the unmatched row is still a row.
- **Right-table conditions go in `ON` , not `WHERE`** — in `WHERE` they turn a left join into an inner join.
- **`NOT EXISTS` over `NOT IN`** — one `NULL` in the subquery makes `NOT IN` return nothing. And index every join column.

PRACTICE

Practice — Opposite ends: pair sums on a sorted array

DSA topic: Opposite ends: pair sums on a sorted array **System design topic:** Joins, drawn

Code these, in this order

Four problems on one skeleton. **For every one, state the completeness argument before writing code** — not just why a move is safe, but why nothing is missed when the indices meet.

Before each one, ask:

1. Is it sorted? If not, what does sorting cost, and does it destroy anything I need?
2. `left < right` or `left ≤ right` — am I forming pairs, or filling positions?
3. What do I do on a hit: return, or record and move **both**?
4. Are duplicates possible, and does the problem want distinct values or distinct positions?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Two Sum II — Input Array Is Sorted	LeetCode 167 (Medium)	The skeleton, the completeness argument, and that the output is 1-indexed .
2	Squares of a Sorted Array	LeetCode 977 (Easy)	Writing from the back , and <code>left ≤ right</code> rather than <code><</code> . The <code>O(n)</code> answer versus the <code>O(n log n)</code> one-liner.
3	3Sum	LeetCode 15 (Medium)	Fix one, two-pointer the rest — and three duplicate guards in three different places.
4	3Sum Closest	LeetCode 16 (Medium)	The same skeleton tracking a best-so-far. Note that you can no longer stop early on a mismatch.

On problem 1, produce the proof

Before coding, write out the three sentences of the invariant argument:

1. What is true at the start?
2. Why is it still true after advancing `left`? (One sentence, and it must mention sortedness.)
3. Therefore, what does it mean when the indices meet with nothing found?

Then run your solution on `[3, 2, 4]` with target 6 and explain the empty result in terms of which sentence stopped being true.

On problem 2, do it both ways and time it

Write `sorted(x*x for x in nums)` first. Say its cost. Then write the two-pointer version.

Then answer:

1. Why can you not fill the output from the front?
2. Why is the loop `left ≤ right` here when it was `left < right` yesterday?
3. On `[-1, 0, 1]`, what does the `<` version return, and why does it look almost right?
4. At `n = 1,000,000`, roughly how many operations does each version do?

On problem 3, break it three ways

Solve it, then delete each duplicate guard in turn and find the input that exposes it.

```
three_sum([-1, 0, 1, 2, -1, -4])    # guard 1 missing → which triple repeats?
three_sum([-2, 0, 0, 2, 2])        # guards 2 and 3 → which triple repeats?
three_sum([0, 0, 0, 0])            # all three at once
```

Then say, for each guard, in one sentence: what repetition does it prevent, and why can the other two guards not do its job?

On problem 4, notice what changed

3Sum Closest looks like 3Sum and one thing is different: you cannot stop scanning when the sum is wrong, because a wrong sum might still be the closest one. Say what that means for:

1. Where the `best` update goes relative to the comparison.
2. Whether you can still `break` out early, and on what condition.
3. Whether the duplicate guards are needed at all here. (They are not — say why.)

The completeness drill

For each, give the one sentence that justifies discarding, and name what would break if the input were unsorted:

1. Two-sum, total too small → advance `left`.
2. Two-sum, total too large → retreat `right`.
3. Count pairs at most `k` → `count += right - left`, then advance `left`.
4. Sorted squares → take the larger absolute value and write it at the back.

Number 3 is the one worth extra thought: why is it `right - left` and not `right - left + 1`?

The variant drill

Take the nine-line skeleton and adapt it, without looking at the lesson, for each of these:

1. Return **all** distinct pairs summing to the target.
2. Return **how many** index pairs sum to at most the target.
3. Return the pair whose sum is **closest** to the target.
4. Return **true/false** only — does any pair sum to the target?
5. Given two sorted arrays, find one element from each summing to the target.

Number 5 is the one that changes shape: say where the two indices start and which way each moves, and why the argument still holds.

The row-count drill

Two tables:

customers	orders
+-----+-----+	+-----+-----+-----+
id name	id customer_id
+-----+-----+	+-----+-----+-----+
1 Asha	10 1
2 Ravi	11 1
3 Nita	12 2
4 Sam	13 NULL
+-----+-----+	+-----+-----+-----+

Give the row count for each, from memory, in under thirty seconds total:

1. `INNER JOIN`
2. `LEFT JOIN`
3. `RIGHT JOIN`
4. `FULL OUTER JOIN`
5. `CROSS JOIN`
6. `LEFT JOIN`, then `WHERE o.id IS NOT NULL`
7. `LEFT JOIN`, then `GROUP BY c.id` with `COUNT(*)` — what does Nita get?
8. The same with `COUNT(o.id)` — what does Nita get?

Numbers 7 and 8 must differ. If they do not, re-read §3.

The fan-out drill

`orders` has 10,000 rows with an average total of ₹2,000. `order_items` has 40,000 rows — four per order on average.

1. What does `SELECT SUM(o.total) FROM orders o` give?
2. What does `SELECT SUM(o.total) FROM orders o JOIN order_items i ON i.order_id = o.id` give?
3. Why, in one sentence?
4. Write the corrected version.
5. If the average were 1.3 items per order instead of 4, why would this bug be *more* dangerous?

The ON-versus-WHERE drill

Predict the row count of each against the four-customer example, then say which is the bug:

```
-- A
SELECT c.name, o.id FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NOT NULL;

-- B
SELECT c.name, o.id FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id AND o.id > 11;

-- C
SELECT c.name, o.id FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id > 11;
```

B and C differ by moving one condition. Say what each returns and which one silently stopped being a left join.

The join-algorithm drill

Answer each in one or two sentences:

1. Name the three join algorithms and say when each is chosen.
2. Which one is the two-pointer walk in disguise?
3. What happens to a nested loop join when the right side's join column is not indexed?
4. What happens to a hash join when the build side does not fit in `work_mem`?
5. 10,000 customers against 1,000,000 orders — comparisons with and without an index?
6. In `EXPLAIN`, what does a large gap between estimated and actual rows usually mean?

The anti-join drill

Write “customers who have never ordered” three ways. Then say:

1. Which is usually fastest, and why?
 2. Which one returns **nothing at all** if `orders.customer_id` contains a single `NULL`, and why?
 3. Why is that the nastiest of the three bugs?
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find two numbers in a sorted array that add up to the target.* Confirm sortedness and the index base. Brute force and its cost. The move rules **with their reasons**. Then the invariant, in three sentences, unprompted — that is what today is for. Then `O(n)` / `O(1)` and the unsorted alternative.
 2. *What is the difference between an inner join and a left join?* Define by what happens to unmatched rows, then give the four row counts from a concrete example. Raise fan-out before being asked. Name the `COUNT(*)` trap and the `ON`-versus-`WHERE` trap.
 3. *How do you know your two-pointer scan didn't miss a pair?* The invariant, stated precisely, and the single place sortedness is used. Finish with what happens to the argument on unsorted input — it returns nothing, with no error, which is the dangerous kind of wrong.
-

Before you move on

- ☐ I can state the invariant in three sentences without preparation.
- ☐ I know the exactly one place sortedness is used in the argument.
- ☐ On a hit I move **both** indices, then skip repeats of both values.
- ☐ I can say what each of 3Sum's three duplicate guards prevents, with its input.
- ☐ I check the index base before writing the return statement.
- ☐ I choose `<` or `≤` from what the problem is covering, not from habit.
- ☐ I can give the four join row counts from a small example in under thirty seconds.
- ☐ I raise fan-out before being asked, and can quantify the double-counting bug.
- ☐ I know `COUNT(o.id)` from `COUNT(*)`, and `ON` from `WHERE`, on an outer join.
- ☐ I can redraw the elimination diagram and the four-join diagram from memory, in whatever tool I like.

Same direction: the read pointer and the write pointer

DATA STRUCTURES AND ALGORITHMS

Same direction: the read pointer and the write pointer

You can remove duplicates from a sorted array in place, in one pass.

SYSTEM DESIGN

Normalisation and when to break it

You can normalise a schema to third normal form, then argue for denormalising part of it.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Remove duplicates in place and return the new length.*
- *When would you deliberately duplicate data across tables?*

Same direction: the read pointer and the write pointer

1 What this is, and why they ask it

The second of the three two-pointer shapes. Yesterday both indices started at opposite ends and moved towards each other. Today **both start at the front and move the same way**, one leading and one lagging, and the gap between them is what has been thrown away.

You met the basic version on [day 015](#) as in-place compaction. Today it becomes a family, and the family is larger than most people realise:

- **Filter** — keep some elements, drop others. Move zeros, remove a value, remove duplicates.
- **Compare against what you kept** — dedupe, keep-at-most- k . The lagging index is not just a destination, it is a *record of your decisions*, and you read back from it.
- **Partition by swapping** — separate a list into groups in one pass. Sort Colors, quickselect's partition step, and the partition inside quicksort.
- **Walking from the back with a skip count** — backspace string compare, and merging into the end of an array.

Interviewers ask the dedupe version constantly because it is a five-minute problem with two real follow-ups: “*now allow each value at most twice*”, which tests whether you compared against the right thing, and “*does your solution preserve the order of the survivors?*”, which tests whether you know the difference between the copy version and the swap version. Most candidates write one of them and do not know the other exists.

This shape is also the engine inside quicksort on [day 054](#) and quickselect on [day 055](#), so getting it into your hands now pays twice.

2 The story

The godown near the railway siding at Sanathnagar takes in rice by the lorry-load, and Karim has been the man who checks it for nine years.

A lorry came in on Tuesday with sixty sacks, loaded in one long row down the bed, and eleven of them had got wet somewhere between there and here — you can tell by the colour along the bottom seam and by the weight, and Karim can tell without picking

a sack up.

The wet ones have to come out and the dry ones have to end up stacked from the front of the bed, because the lorry is going on to a second godown and the load has to sit forward or it rides badly.

What Karim does not do is unload all sixty and put them back. He works inside the lorry, and he keeps two things in his head.

The first is where he has got to — which sack he is looking at now. That starts at the front and moves down the row, one sack at a time, and it never goes back.

The second is where the next good sack has to go. That also starts at the front. And here is the whole thing: it only moves when he actually puts a sack down.

The first three sacks are dry. He looks at each one, puts it where it already is, and both of the things in his head move along together — no gap. The fourth is wet. He steps past it, and now the two have come apart: the place-where-the-next-good-sack-goes is standing on the wet fourth sack, and he is looking at the fifth.

Fifth is dry, so he drags it back onto the fourth's position, and the placing spot moves to five while he moves to six.

From then on the gap is exactly the number of wet sacks he has walked past, and it only ever grows. The placing spot never gets ahead of him, which is the reason he can drag a sack backwards without worrying — whatever was sitting there, he has already looked at it and already decided.

By the end of the row he is standing at the tailboard and the placing spot is eleven sacks behind him, and that number is the answer to the only question the office actually asks: forty-nine good, eleven wet.

3 The idea in plain English

Karim's *where I have got to* is `read`. His *where the next good one goes* is `write`. And the rule that makes it safe — the placing spot never gets ahead of him — is the invariant.

The skeleton

```
write = 0
for read in range(len(items)):
    if keep(items[read]):
        items[write] = items[read]
        write += 1
return write
```

Five lines, and every variant today is this with a different `keep`.

The invariant, stated properly

`write ≤ read` at every moment, and everything in `items[0:write]` is finished and correct.

Both start at 0, so it holds initially. `write` only increases in the same step where `read` increases, and `read` also increases in steps where `write` does not — so `write` can never overtake. That is why `items[write] = items[read]` is safe: position `write` is either the same position you are reading, or one that `read` has already passed and already decided about.

Say that sentence in the interview. It is what makes the whole family obviously correct rather than apparently reckless.

What `write` means when the loop ends

Three useful things, and which one you return depends on the question:

- `write` is the count of kept elements.
- `items[0:write]` is the answer, in the original relative order.
- `items[write:]` is leftovers — stale values `read` has already passed.

You cannot shrink a fixed-size array, so the convention is to **return `write` as the new length** and let the caller ignore the rest. That is what LeetCode 26, 27 and 80 all want, and it confuses people the first time: nothing is deleted, the count is the answer.

The three sub-shapes

Shape one: filter. `keep` looks only at the current element.

```
if items[read] ≠ 0:           # move zeros
if items[read] ≠ target:      # remove a value
```

Shape two: compare against what you kept. `keep` looks back at `items[write - 1]`.

This is the one that matters, and it is what makes dedupe work.

```
if items[read] ≠ items[write - 1]:      # remove duplicates from a sorted array
if write < 2 or items[read] ≠ items[write - 2]:  # allow each value at most twice
```

`items[write - 1]`, not `items[read - 1]`. `read - 1` is the element you last *looked at*; `write - 1` is the element you last *kept*. On a sorted array with no gaps they coincide, so the wrong version passes — and then the “at most twice” follow-up needs a rewrite

instead of a one-character change. Comparing against what you kept is the habit that generalises.

The generalisation is worth seeing, because it is the follow-up:

```
def keep_at_most_k(nums: list[int], k: int) → int:
    write = 0
    for x in nums:
        if write < k or x ≠ nums[write - k]:
            nums[write] = x
            write += 1
    return write
```

Is this value different from the one I kept `k` places ago? If it is, then fewer than `k` copies of it have been kept, so this one may stay. `write < k` lets the first `k` through unconditionally. One function, and `k = 1` and `k = 2` are LeetCode 26 and 80.

Shape three: partition by swapping. Instead of copying forward and leaving stale values behind, **exchange**:

```
if items[read] ≠ 0:
    items[write], items[read] = items[read], items[write]
    write += 1
```

Now the discarded elements are pushed to the back rather than overwritten, so nothing is lost. That matters when you need both groups, which is what partitioning is.

Stable or not — the follow-up people miss

The **copy** version preserves the relative order of the survivors. The **swap** version preserves the order of the *kept* group but scrambles the discarded group:

```
partition_even_odd([3, 1, 2, 4])    # → [2, 4, 3, 1]
```

The evens came out `2, 4` — original order. The odds came out `3, 1` — reversed, because `3` was swapped to the back early. An algorithm that keeps relative order is **stable**, from [day 015](#).

Ask which the problem wants. If it only cares about the kept group, either works. If it needs both groups in order, the swap version is wrong and you need extra space.

Three pointers: the Dutch national flag

Sort an array of 0s, 1s and 2s in one pass — LeetCode 75 — is the same idea with three indices:

```

low, mid, high = 0, 0, len(nums) - 1
while mid ≤ high:
    if nums[mid] == 0:
        nums[low], nums[mid] = nums[mid], nums[low]
        low += 1
        mid += 1
    elif nums[mid] == 2:
        nums[mid], nums[high] = nums[high], nums[mid]
        high -= 1
        # note: mid does NOT advance
    else:
        mid += 1

```

The array is kept in four regions: `[0, low)` all 0s, `[low, mid)` all 1s, `[mid, high]` unexamined, `[high, end]` all 2s.

The detail that catches everyone: on a 2, `mid` does not advance. The value swapped in from `high` has never been looked at, so it must be examined. On a 0 it is safe to advance both, because whatever comes back from `low` is already known to be a 1 — `low` never passes anything but 1s.

`while mid ≤ high`, not `<`, because the element at `high` is still unexamined.

Walking from the back

Some problems in this family run backwards, and the tell is that **information is destroyed as you go forward but preserved going back**. Backspace string compare is the standard one: `"ab#c"` and `"ad#c"` are equal, because `#` deletes the character before it.

Going forwards you cannot tell whether a character will survive — a `#` might arrive later. Going backwards you can, because you have already seen all the deletions that could affect it:

```

skip = 0
for i in range(len(s) - 1, -1, -1):
    if s[i] == "#":
        skip += 1
    elif skip:
        skip -= 1
        # this character is deleted
    else:
        yield s[i]
        # this character survives

```

Same idea as [day 018](#)'s merge-from-the-back: **go in the direction where the answer is already determined**.

4 The picture

`[0, 1, 0, 3, 12]`, removing zeros, with `w` and `r`:

```

start      0    1    2    3    4
+---+---+---+---+---+
|  0 |  1 |  0 |  3 | 12 |
+---+---+---+---+
      ^
    w,r      0 is dropped. Only r moves. The gap opens.

+---+---+---+---+---+
|  0 |  1 |  0 |  3 | 12 |
+---+---+---+---+
      ^      ^
      w      r      1 is kept: copy to w, then both move.

+---+---+---+---+---+
|  1 |  1 |  0 |  3 | 12 |
+---+---+---+---+
          ^      ^
          w      r      position 1 still says 1 – a stale leftover,
                        already read, never read again. This is fine.

end
+---+---+---+---+---+
|  1 |  3 | 12 |  3 | 12 |
+---+---+---+---+
|-----| |-----|
      kept      leftovers
                ^
            w = 3 kept, r - w = 2 discarded

```

What to notice: the gap only grows, and it grows by exactly one per discard. `write` is the count of keepers and `read - write` is the count of discards — both facts fall out for free.

Dedupe, showing why you compare against `write - 1`:

```

nums      0    0    1    1    1    2
w=1
r=1: nums[1]=0 == nums[w-1]=0 → duplicate, skip
r=2: nums[2]=1 ≠  nums[w-1]=0 → keep.  nums[1]=1, w=2
r=3: nums[3]=1 == nums[w-1]=1 → duplicate, skip
r=4: nums[4]=1 == nums[w-1]=1 → duplicate, skip
r=5: nums[5]=2 ≠  nums[w-1]=1 → keep.  nums[2]=2, w=3

result [0, 1, 2, ...] length 3
      ^
    nums[w-1] is always "the last thing I decided to keep",
    which is exactly the question a dedupe needs to ask.

```

The Dutch national flag, as four regions:

```
[ 0 0 0 | 1 1 1 | ? ? ? ? | 2 2 2 ]
      ^      ^      ^
      low   mid   high
```

all 0s all 1s unexamined all 2s
 [0,low) [low,mid) [mid,high) (high,end]

```
nums[mid] = 0 : swap with low, low++, mid++    (the swapped-in value is a known 1)
nums[mid] = 1 : mid++
nums[mid] = 2 : swap with high, high--        (mid does NOT move – unexamined value)
```

What to notice: the `?` region shrinks by one every turn, from one end or the other, so the loop runs at most `n` times even though `mid` sometimes stands still. That is the termination argument.

5 The code, built step by step

Dedupe a sorted array

```
if not nums:
    return 0
write = 1
```

`write` starts at 1 because position 0 is always kept — there is nothing before it to duplicate. That also means `nums[write - 1]` is valid from the first comparison. The empty guard exists because `write = 1` on an empty array would report a length of 1.

```
for read in range(1, len(nums)):
    if nums[read] != nums[write - 1]:
        nums[write] = nums[read]
        write += 1
return write
```

`read` starts at 1 too, for the same reason. Say the comparison out loud as you write it: *“is this different from the last value I kept?”*

At most `k` copies

```
def keep_at_most_k(nums: list[int], k: int) → int:
    write = 0
    for x in nums:
        if write < k or x != nums[write - k]:
            nums[write] = x
            write += 1
    return write
```


`write < k` waves the first `k` through. Then `x != nums[write - k]` asks *is this different from the one I kept `k` places ago?* — if it is, at most `k - 1` copies of it are already kept.

Note this iterates the values directly rather than by index, which is safe because the write position is always at or behind the read position, so `x` was captured before any overwrite could reach it.

The swap version, and why you might want it

```
write = 0
for read in range(len(nums)):
    if nums[read] != 0:
        nums[write], nums[read] = nums[read], nums[write]
        write += 1
```

Exchange instead of copy, so the zeros are pushed to the back rather than overwritten and no second pass is needed. Costs two writes per keeper instead of one, so on a mostly-non-zero array the copy version does about half the writes — the comparison from [day 015](#).

The reason to reach for it here is **partitioning**: when you want both groups, not just one.

Sort Colors

```
low, mid, high = 0, 0, len(nums) - 1
while mid ≤ high:
```

`mid ≤ high` because `nums[high]` is still unexamined.

```
    if nums[mid] == 0:
        nums[low], nums[mid] = nums[mid], nums[low]
        low += 1
        mid += 1
```

Swap the 0 down to the `low` boundary. **Both advance**, and the reason is worth saying: whatever comes back from `low` is a value the `low` boundary had already classified, and everything in `[low, mid)` is a 1 — so the swapped-in value is a known 1 and needs no re-examination.

```
    elif nums[mid] == 2:
        nums[mid], nums[high] = nums[high], nums[mid]
        high -= 1
```

`mid` **does not move**. The value that came back from `high` is from the unexamined region and has never been looked at. Advancing `mid` here is the single most common bug in this problem, and it fails on `[1, 2, 0]` and on `[2, 1, 2]` — try both.

```
else:
    mid += 1
```

Backspace compare, from the back

```
def gen(x: str):
    skip = 0
    for i in range(len(x) - 1, -1, -1):
        if x[i] == "#":
            skip += 1
        elif skip:
            skip -= 1
        else:
            yield x[i]
```

A generator yielding the surviving characters in reverse. Then compare two of them lazily:

```
from itertools import zip_longest
return all(a == b for a, b in zip_longest(gen(s), gen(t)))
```

`zip_longest` pads the shorter one with `None`, so different lengths correctly compare unequal — `zip` would stop at the shorter and wrongly report equal.

$O(n + m)$ time and $O(1)$ extra space, against the obvious approach of building both strings, which is $O(n + m)$ space. **That space difference is the whole reason the problem is posed.**

The complete solutions

```
from itertools import zip_longest

def remove_duplicates(nums: list[int]) → int:
    """LeetCode 26. Sorted input, in place. Returns the count of distinct values."""
    if not nums:
        return 0
    write = 1 # position 0 is always kept
    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]: # compare with what was KEPT, not read
            nums[write] = nums[read]
            write += 1
    return write

def keep_at_most_k(nums: list[int], k: int) → int:
    """LeetCode 80 generalised. Sorted input. k=1 is LeetCode 26, k=2 is LeetCode 80."""
    write = 0
    for x in nums:
        if write < k or x != nums[write - k]: # differs from the one kept k places back
            nums[write] = x
            write += 1
    return write

def move_zeros(nums: list[int]) → None:
    """Swap version: one pass, no tail fill, and the zeros end up at the back."""
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1

def partition_even_odd(nums: list[int]) → int:
    """Evens to the front. Returns where the odds start.

    NOT stable for the discarded group: [3,1,2,4] → [2,4,3,1], odds reversed.
    """
    write = 0
    for read in range(len(nums)):
        if nums[read] % 2 == 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1
    return write

def sort_colors(nums: list[int]) → None:
    """LeetCode 75. Dutch national flag, one pass, O(1) space.

    [0,low) = 0s, [low,mid) = 1s, [mid,high) = unexamined, (high,end] = 2s
    """
    low, mid, high = 0, 0, len(nums) - 1
    while mid ≤ high: # ≤, because nums[high] is unexamined
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1 # swapped-in value is a known 1
        elif nums[mid] == 2:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1 # mid does NOT move: unexamined value
        else:
            mid += 1

def backspace_compare(s: str, t: str) → bool:
    """LeetCode 844. O(1) extra space by walking backwards with a skip count."""
```

```

def surviving(x: str):
    skip = 0
    for i in range(len(x) - 1, -1, -1):
        if x[i] == "#":
            skip += 1
        elif skip:
            skip -= 1
        else:
            yield x[i]

    return all(a == b for a, b in zip_longest(surviving(s), surviving(t)))

if __name__ == "__main__":
    for case in ([0, 0, 1, 1, 1, 2, 2, 3, 3, 4], [1, 1, 2], [1], [], [1, 2, 3]):
        a = list(case)
        k = remove_duplicates(a)
        print(case, "→", k, a[:k])
    # [0,0,1,1,1,2,2,3,3,4] → 5 [0, 1, 2, 3, 4]

    a = [1, 1, 1, 2, 2, 3]
    k = keep_at_most_k(a, 2)
    print(k, a[:k]) # 5 [1, 1, 2, 2, 3]

    b = [0, 1, 0, 3, 12]
    move zeroes(b)
    print(b) # [1, 3, 12, 0, 0]

    c = [3, 1, 2, 4]
    print(partition_even_odd(c), c) # 2 [2, 4, 3, 1] ← odds reversed

    for case in ([2, 0, 2, 1, 1, 0], [2, 0, 1], [0], []):
        d = list(case)
        sort_colors(d)
        print(case, "→", d)
    # [2,0,2,1,1,0] → [0, 0, 1, 1, 2, 2]

    print([backspace_compare(x, y) for x, y in
          (("ab#c", "ad#c"), ("ab##", "c#d#"), ("a#c", "b"), ("", ""), ("a##c", "#a#c"))])
    # [True, True, False, True, True]

```

6 What it costs

`remove_duplicates` and the filter family

The loop runs `len(nums) - 1` times — once per position — and each turn does one comparison and at most one assignment and one increment. Constant work per turn.

$O(n)$ time. $O(1)$ extra space — `read` and `write` are two integers whatever the array size.

Against the obvious alternative of building a new list: also $O(n)$ time, but $O(n)$ extra space. And against deleting in place with `del nums[i]`, which is $O(n^2)$, because each delete shifts everything after it — the shifting cost from [day 011](#).

At `n = 100,000`:

```
write pointer : 100,000 comparisons
delete-in-place : ~5,000,000,000 element moves
```

keep_at_most_k

Identical structure: one pass, constant work per element. $O(n)$ time, $O(1)$ space, for any k .

sort_colors

Each turn of the loop either advances `mid` or decreases `high`, and the unexamined region `[mid, high]` therefore shrinks by exactly one every turn. Starting at size `n`, the loop runs at most `n` times.

$O(n)$ time, $O(1)$ space — and one pass. The obvious alternative is counting sort: one pass to count the 0s, 1s and 2s, a second pass to write them back. Also $O(n)$ and $O(1)$, and perfectly acceptable — **but the problem explicitly asks for one pass**, and the reason is that a real version of this may be sorting objects by a key rather than integers, where you cannot regenerate the values from a count. Mention the counting solution, then give the one-pass one.

backspace_compare

Each generator visits each character exactly once, so $O(n + m)$ time. The generators hold one integer each and yield lazily, so $O(1)$ extra space beyond the inputs.

The straightforward version — build both strings with a list and `join`, then compare — is the same $O(n + m)$ time but $O(n + m)$ space. On a very large input that is the difference between constant memory and a full second copy, and it is the entire point of the question. **Write the simple one first, then offer this.**

Copy versus swap, counted

On `[1, 2, 3, 4]` with no zeros to remove:

```
copy version : 4 self-copies, then a 0-length tail loop    = 4 writes
swap version  : 4 self-swaps                                = 8 writes
```

On `[0, 0, 0, 1]`:

```
copy version : 1 copy + 3 tail fills                        = 4 writes
swap version  : 1 swap                                       = 2 writes
```

Neither dominates. **Mostly-discarded favours swap; mostly-kept favours copy**, and both are $O(n)$.

The number to have ready

The write pointer is one pass, $O(n)$ time and $O(1)$ extra space, and it replaces either an $O(n)$ allocation or an $O(n^2)$ sequence of deletes. At a hundred thousand elements that is 100,000 operations against five billion.

7 The traps

The near-miss: comparing against `read - 1`

```
if nums[read] != nums[read - 1]:    # works on LeetCode 26, wrong in general
```

On a plain sorted dedupe this gives the right answer, because you never skip a value that could come back. It breaks the moment the rule involves *how many you kept*:

```
def keep_at_most_two(nums):
    write = 0
    for read in range(len(nums)):
        if write < 2 or nums[read] != nums[read - 2]:    # read - 2, not write - 2
            nums[write] = nums[read]
            write += 1
    return write

a = [1, 1, 1, 2, 2, 3]
k = keep_at_most_two(a)
print(k, a[:k])
```

```
4 [1, 1, 2, 3]
```

The answer is `5` and `[1, 1, 2, 2, 3]` — one of the 2s has been **dropped**. `nums[read - 2]` looks at the *input* two positions back, but by this point the write pointer has already overwritten that position, so the comparison is against a value that is no longer what it was. `nums[write - 2]` reads back from the output, which is the only thing that reflects your actual decisions.

Note that it gives the right answer on `[1, 1, 1, 1, 2, 3]`, which is exactly the kind of input people test with. **Compare against what you kept.**

The real error: forgetting the empty guard

```
def remove_duplicates(nums):
    write = 1                # no empty check
    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1
    return write

print(remove_duplicates([]))
```

```
1
```

No exception — it reports a length of 1 for an empty array, which is a silent wrong answer rather than a crash. The loop never runs, so `nums[write - 1]` is never evaluated and the bug hides. **Any solution that pre-keeps element 0 needs an empty guard.**

The near-miss: advancing `mid` after swapping a 2 in Sort Colors

```
elif nums[mid] == 2:
    nums[mid], nums[high] = nums[high], nums[mid]
    high -= 1
    mid += 1                # WRONG
```

```
nums = [1, 2, 0]
sort_colors(nums)
print(nums)
```

```
[1, 0, 2]
```

Not sorted. The value swapped in from `high` came from the unexamined region and was never looked at — here the `0` arrived at `mid` and was walked straight past. `[2, 1, 2]` is worse still: it comes back completely unchanged. Contrast with the `0` branch, where advancing `mid` is correct, because everything in `[low, mid)` is a 1, so the value coming back from `low` is a known 1.

Two swaps, two different rules, and the reason is what is known about the region you swapped with.

The near-miss: assuming the swap version is stable

```
partition_even_odd([3, 1, 2, 4])    # → [2, 4, 3, 1]
```

The evens keep their order; the odds come out reversed. If the problem says “preserve the relative order of both groups”, this is wrong and no in-place $O(1)$ -space solution exists — you need $O(n)$ extra space, or a stable partition algorithm that is considerably more work. **Ask whether order matters, and for which group.**

The near-miss: `zip` instead of `zip_longest`

```
return all(a == b for a, b in zip(surviving(s), surviving(t)))
print(backspace_compare("a#c", "b"))
```

`zip` stops at the shorter sequence, so a string that is a prefix of the other compares **equal**. Here `"a#c"` reduces to `"c"` and `"b"` stays `"b"`, so it happens to be caught — but `"ac"` against `"c"` would wrongly return `True`. `zip_longest` pads with `None`, which never equals a character.

The near-miss: forgetting the tail

The copy version leaves stale values after `write`. If the problem wants zeros at the end — LeetCode 283 — you must fill them:

```
for i in range(write, len(nums)):
    nums[i] = 0
```

Skip it and the front is right and the back is rubbish, and a test that checks only `result[:3]` passes. **Always compare the whole array.**

8 In the interview

How it gets asked

- “Remove duplicates from a sorted array in place and return the new length.” — LeetCode 26. Then “now allow each value at most twice”, which is LeetCode 80 and the real question.
- “Move all zeros to the end.” — LeetCode 283, and the stability follow-up.
- “Sort an array of 0s, 1s and 2s in a single pass.” — LeetCode 75. The counting solution is two passes, so they want the flag.
- “Compare two strings with backspaces, in $O(1)$ space.” — LeetCode 844, where the space constraint forces the backwards walk.

What to say out loud, in the first ninety seconds

1. **Pin the contract.** *“In place, so I cannot allocate a second array? And do I return the count, or is the array itself checked? And does the relative order of the survivors have to be preserved?”*
2. **Name the shape.** *“Two indices moving the same way — a read index that visits every position, and a write index that says where the next kept element goes.”*
3. **State the invariant, because it is why it is safe.** *“The write index never overtakes the read index, so anything I overwrite has already been read and decided. That is what makes writing into the array I am reading from safe.”*
4. **Say what the comparison is against.** *“For dedupe I compare against `nums[write - 1]` — the last value I kept — rather than `nums[read - 1]`, which is the last value I looked at. On this problem they coincide, but the habit is what makes the at-most-twice follow-up a one-character change.”*
5. **Say what `write` means at the end.** *“When the loop ends, `write` is the number of elements kept, and everything from there on is stale leftovers.”*
6. **Give the cost.** *“ $O(n)$ time, one pass, $O(1)$ extra space.”*
7. **Name what you are not doing.** *“I’m not deleting in place — each delete shifts everything after it, so that would be $O(n^2)$.”*

The follow-ups

“Now allow each value to appear at most twice.” One character, if I compared against the right thing. The condition becomes `nums[read]  $\neq$  nums[write - 2]` with `write < 2` letting the first two through unconditionally — *is this value different from the one I kept two places ago?* If it is, then at most one copy is already kept and this one may stay. Generalise the 2 to a `k` and the same five lines solve “at most k times”, which is the version I would actually write since it costs nothing. This is only a one-character change because the original compared against `nums[write - 1]`; a solution built on `nums[read - 1]` needs restructuring, because `read - 2` looks at the input two positions back, and by that point the input has been overwritten by earlier writes.

“Does your solution preserve the order of the elements?” The copy version does — survivors come out in their original relative order, so it is stable. The swap version preserves the order of the kept group but reverses or scrambles the discarded group, because elements get swapped backwards from wherever the write index happens to be. `[3,1,2,4]` partitioned by evens gives `[2,4,3,1]`: evens in order, odds reversed. So which one I use depends on whether anything downstream cares about the second group. If both groups must keep their order and I am not allowed extra space, there is no simple `O(n)` `O(1)` answer — stable in-place partition exists but is considerably more involved, and I would say so rather than pretend the swap version does it.

“Sort colours in one pass — why does `mid` not advance when you swap a 2?”

Because the value that comes back from `high` is from the unexamined region and I have never looked at it. If I advance `mid`, I walk past a value I have not classified — on `[2,0,1]` a `0` gets swapped to `mid` and skipped, and the result is `[1,0,2]`, unsorted. The contrast with the `0` branch is the interesting part: there, advancing `mid` is correct, because everything between `low` and `mid` is known to be a 1, so the value swapped back from `low` is a known 1 and needs no re-examination. Two swaps, two rules, and the difference is entirely about what is already known about the region on the other end of the swap. Termination is easy either way: every turn either advances `mid` or lowers `high`, so the unexamined region shrinks by one each time.

“Do the backspace comparison in $O(1)$ space.” Walk both strings backwards with a skip counter. Forwards you cannot decide whether a character survives, because a `#` might arrive later; backwards you can, because you have already seen every deletion that could affect it. So: scan right to left, increment `skip` on a `#`, decrement it and discard the character when `skip` is positive, otherwise the character survives. Comparing two such walks lazily — as generators — gives $O(n + m)$ time and $O(1)$ extra space, against the obvious build-both-strings approach which is the same time but $O(n + m)$ space. The detail to be careful about is the end: I need `zip_longest` rather than `zip`, or a string that is a suffix of the other compares equal because `zip` stops at the shorter one.

A model answer

“Three clarifications. In place, so no second array? Do I return the new length, or will you check the array? And does the order of the survivors matter?

...In place, return the length, order preserved. Good.

I'll use two indices moving in the same direction. A read index visits every position from left to right. A write index says where the next element I keep should go. Because the array is sorted, duplicates are adjacent, so 'have I seen this before?' reduces to 'is it the same as the last thing I kept?'

```
def remove_duplicates(nums: list[int]) → int:
    if not nums:
        return 0
    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1
    return write
```

Three things worth calling out.

`write` starts at 1 because position 0 is always kept — nothing precedes it — which also makes `nums[write - 1]` valid on the first comparison. And that is why I need the empty guard: without it, an empty array returns a length of 1, silently, because the loop never runs and the bug never surfaces.

I compare against `nums[write - 1]`, not `nums[read - 1]`. On this problem they happen to agree, but `write - 1` is the last value I *kept* and `read - 1` is the last value I *looked at* — and that distinction is what makes the at-most-twice follow-up a one-character change to `write - 2` rather than a rewrite.

The property that makes overwriting safe is that the write index never overtakes the read index. Both start at 0; `write` only increases in steps where `read` also increases, and `read` sometimes increases alone. So every position I write to is either the one I am reading or one I have already passed and already decided about.

That is one pass, $O(n)$ time and $O(1)$ extra space. When the loop ends, `write` is the count of distinct values, and everything from `write` onwards is stale leftovers, which the caller ignores.

The version I deliberately avoided is deleting duplicates in place — each `del` shifts everything after it, so that is $O(n^2)$: about five billion element moves on a hundred thousand elements, against a hundred thousand comparisons here.”

9 Recall card

- **Both indices move forward.** `read` visits everything; `write` advances only on a keep. The gap is what you discarded.
- **The invariant:** `write ≤ read`, so overwriting is always safe — and `items[0:write]` is finished and correct.
- **Compare against** `items[write - 1]`, **not** `items[read - 1]` — what you kept, not what you saw. That makes `k`-duplicates a one-character change.
- **Copy is stable; swap is not** for the discarded group. `[3,1,2,4] → [2,4,3,1]`.
- **Sort Colors: on a 2, `mid` does not advance** — the value from `high` is unexamined. On a 0 it does, because the value from `low` is a known 1.

1 What this is, and why they ask it

Normalisation is organising a schema so that **each fact is stored in exactly one place**. It has formal levels — first, second and third normal form — but the whole of it collapses into that one sentence, and the reason is equally short: a fact stored twice can end up disagreeing with itself, and then nobody knows which copy is right.

Denormalisation is deliberately breaking that rule for speed, and it is not a failure. Copying a comment count onto a post so you do not have to count rows on every read is a considered engineering decision. Copying it by accident, with nothing to keep it in step, is a bug.

Interviewers ask this because it is one of the few database questions where **both answers are wrong on their own**. “Always normalise” produces schemas that need eight joins to render a screen. “Denormalise for performance” produces data that quietly disagrees with itself. The candidate they want says: *here is the normalised design, here is the specific read-to-write ratio that justifies denormalising this one field, and here is what keeps it honest*. That third clause is the one almost nobody supplies.

You already have the pieces. [Day 026](#) gave you keys and the one-fact-one-place rule; [day 028](#) gave you joins, which are the cost of normalising. Today puts a name and a decision procedure on the trade.

2 The story

Vasanthi has run the provision shop on the corner since her husband died, and her son does the counter in the evenings when he gets back from college.

Every price in that shop exists in three places, and she knows it.

There is the board behind her, where the rates are written up — that is the one she treats as the truth, and it is the one she changes when the wholesaler’s rates move. There are the little labels stuck on the shelf edges, so that customers can see without asking. And there is her son’s head, because a boy who has to turn round and read the board for every packet of biscuits serves four customers an hour instead of thirty.

The copies are there for a reason and she is not going to get rid of them. The labels are what stop her being asked forty times a day, and her son’s memory is what makes the evening rush possible at all.

But she pays for them, and she pays every time something changes.

The oil went up eleven rupees in April. She changed the board that morning. The shelf label she changed at some point in the afternoon. Her son was at college, so for three days he sold it at the old price to anybody who did not look at the label, and she found out from the account book at the end of the week.

The worst one was a soap that went up and then came back down. The board was right, the label was right, and the boy was still charging the middle price a fortnight later, until a woman who had been coming for eleven years told her, quite sharply, that she had been overcharged.

So now there is a Sunday morning ritual, and it takes forty minutes. She walks the whole shop with the board in her hand and reads every label against it, and anything that disagrees gets fixed, and then she reads out to her son the six or seven things that have moved that week and makes him say them back.

She is very clear that the copies are worth having. She is equally clear that if you keep copies and do not have a Sunday morning, you do not have a shop — you have three shops that disagree.

3 The idea in plain English

The board is the **normalised** truth. The shelf labels and her son's memory are **denormalised copies**, kept for speed. The oil incident is **drift**. And Sunday morning is the **reconciliation job**, which is the part everybody forgets.

The rule underneath all of it

Every fact is stored in exactly one place. Everything else refers to it by key.

That is normalisation. The formal normal forms are three specific ways of failing it.

First normal form: one value per cell

1NF: each column holds a single value, and there are no repeating groups.

Broken:

orders

+-----+-----+-----+-----+			
id	cust	products	
+-----+-----+-----+-----+			
1	Asha	"soap, oil, rice"	← three values in one cell
2	Ravi	"sugar"	
+-----+-----+-----+-----+			

You cannot index it, cannot join on it, cannot ask “how many orders included oil?” without string matching, and cannot enforce that the products exist. The fix is a row per item:

order_items

+-----+-----+-----+-----+		
order_id	product_id	
+-----+-----+-----+-----+		
1	12	
1	7	
1	3	
2	19	
+-----+-----+-----+-----+		

Also broken: columns named `product_1`, `product_2`, `product_3`. Same problem wearing a different shape, and the tell is that adding a fourth product means altering the table.

Second normal form: no partial dependency on a composite key

2NF: 1NF, and every non-key column depends on the *whole* primary key, not part of it.

This only ever applies when the primary key is **composite** — two or more columns together.

Broken:

order_items (primary key: order_id + product_id)

order_id	product_id	quantity	product_name
1	12	2	Coconut oil
2	12	1	Coconut oil

← copied

`quantity` genuinely depends on both — how many of *this product* in *this order*. But `product_name` depends only on `product_id`. So it is repeated on every order line that mentions that product, and renaming the product means updating every one of them.

The fix: `product_name` lives in `products`, and `order_items` keeps only `product_id`.

Third normal form: no dependency on a non-key column

3NF: 2NF, and no non-key column depends on another non-key column.

Broken:

employees

+-----+-----+-----+-----+			
id	name	department_id	department_name
+-----+-----+-----+-----+			
1	Asha	4	Engineering
2	Ravi	4	Engineering
+-----+-----+-----+-----+			

← copied

`department_name` does not depend on the employee at all; it depends on `department_id`, which is itself a non-key column. This is a **transitive dependency**: `id` → `department_id` → `department_name`.

The fix: a `departments` table, and `employees` keeps only `department_id`.

The one-line version people actually use, and it is worth saying because it is memorable:

Every non-key column depends on the key, the whole key, and nothing but the key.

3NF is where practice stops. There are higher forms — BCNF, 4NF, 5NF — and they address genuinely rare situations. **3NF is the working standard**, and saying “I’d normalise to 3NF and then denormalise deliberately where the numbers justify it” is the complete answer to the theory half.

The three anomalies normalisation prevents

Worth naming, because they are what the whole exercise is for:

- **Update anomaly** — the department is renamed and you update 400 rows; you miss 3, and now two answers exist. Vasanthi’s oil.
- **Insert anomaly** — you cannot record a new department until somebody works in it, because the department’s name only exists on employee rows.
- **Delete anomaly** — the last employee in a department leaves, and the department’s existence disappears with them.

Say all three by name. They are the reason, and most candidates can only produce the first.

Denormalisation, deliberately

Copies are legitimate. Four common ones:

COPY	INSTEAD OF	WHY
<code>posts.comment_count</code>	<code>COUNT(*)</code> on comments	counting on every read is unaffordable at a high read ratio
<code>orders.customer_name</code>	joining to customers	a historical snapshot : the name at the time of the order
<code>order_items.unit_price</code>	joining to products	the price <i>then</i> , which must not change when the catalogue does
a materialised view	recomputing a report	a dashboard that would otherwise scan millions of rows

Two of those four are not really denormalisation at all, and this is worth understanding because it gets asked. `orders.customer_name` and `order_items.unit_price` are storing a *different fact* — what the name and price **were at that moment** — not a copy of a current one. An invoice must not change because a product’s price changed last week. **That is correct design, not a shortcut**, and it never drifts because it is never supposed to match.

The genuine denormalisation is the first and fourth: a stored count and a materialised view, both of which *are* copies of something derivable and both of which *can* drift.

The decision procedure

Denormalise when you can answer all four:

- 1. **What is the read-to-write ratio?** From [day 017](#), a comment feed is around 265 reads per write. Below about 10:1, the copy is rarely worth it.
- 2. **What does the read cost without it?** Measure. If `COUNT(*)` on an indexed column takes 1 ms and you do 3,700 reads a second, that is nearly 4 cores of database time.
- 3. **How stale may it be?** Seconds? Minutes? “Never” means it must be updated in the same transaction, which slows the write and couples the two.
- 4. **What keeps it honest?** A reconciliation job, and a way to detect drift. **This is Sunday morning, and a design without it is not finished.**

If you cannot answer 4, do not denormalise.

4 The picture

The three normal forms, as three specific failures:

NOT 1NF many values in a cell	NOT 2NF depends on PART of the key	NOT 3NF depends on a NON-key column
<pre> +-----+ id products +-----+ 1 soap, oil, rice +-----+ </pre> ^ can't index, can't join, can't constrain	<pre> PK = (order_id, product_id) +-----+ ord prod qty prod_name +-----+ 1 1 4 soap +-----+ 2 1 4 oil +-----+ 3 1 4 rice +-----+ </pre> whole key whole only half fix: prod_name → products	<pre> +-----+ id name dept dept_name +-----+ 1 Asha 4 Engineering +-----+ 2 Ravi 4 Engineering +-----+ </pre> ^ depends on dept, not on id fix: dept_name → departments

What to notice: all three failures produce the same symptom — a value written down more than once — and therefore the same risk. The forms are just three different routes to it.

The trade, drawn with the numbers that decide it:

NORMALISED one fact, one place	DENORMALISED a copy, kept for speed
<pre> posts comments +-----+ +-----+ id id post_id +-----+ +-----+ </pre>	<pre> posts +-----+ id comment_count +-----+ </pre>
<pre> read : COUNT(*) WHERE post_id = 91 ~1 ms × 3,700 reads/s = 3.7 s of DB time per second = ~4 cores </pre>	<pre> already in the row you fetched 0 ms </pre>
<pre> write: nothing extra </pre>	<pre> write: one extra UPDATE 3.5 writes/s = negligible </pre>
<pre> never wrong </pre>	<pre> can drift → needs a reconciliation job </pre>

What to notice: the decision is made by the ratio, not by taste. At 265 reads per write, paying a tiny write cost to remove a large read cost is obviously right. At 1:1 it is obviously wrong. **Compute the ratio first and the answer falls out.**

The copy that is not a copy:

a snapshot — SHOULD differ, never drifts

order_items.unit_price
₹90 on 12 March

deliberately unrelated to

products.price
₹110 today

a real copy — can drift, needs a job

posts.comment_count

should equal

COUNT(*) of comments

FIGURE 29.1

What to notice: the dotted arrow is an obligation you must maintain. The solid arrow is not an obligation at all — the invoice is *supposed* to disagree with today's catalogue. Confusing the two is how people either build reconciliation jobs they do not need or skip ones they do.

5 How it actually works

Normalising a schema, worked

Start with the shape people actually produce first — one wide table:

```
CREATE TABLE orders_flat (
  order_id      BIGINT,
  order_date    DATE,
  customer_name TEXT,
  customer_email TEXT,
  customer_city TEXT,
  products      TEXT,      -- "soap, oil, rice"
  quantities    TEXT,      -- "1, 2, 5"
  total         NUMERIC
);
```

1NF: `products` and `quantities` hold lists. Split into `order_items`, one row per line.

2NF: with a composite key of `(order_id, product_id)`, anything depending only on `product_id` moves to `products`.

3NF: `customer_email` and `customer_city` depend on the customer, not on the order. Move them to `customers` and keep `customer_id`.

The result:

```
CREATE TABLE customers (
  id      BIGSERIAL PRIMARY KEY,
  name    TEXT NOT NULL,
  email   TEXT NOT NULL UNIQUE,
  city    TEXT NOT NULL
);

CREATE TABLE products (
  id      BIGSERIAL PRIMARY KEY,
  name    TEXT NOT NULL,
  price   NUMERIC(12,2) NOT NULL      -- the CURRENT price
);

CREATE TABLE orders (
  id          BIGSERIAL PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES customers(id),
  order_date  DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE order_items (
  order_id    BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id  BIGINT NOT NULL REFERENCES products(id),
  quantity    INT      NOT NULL CHECK (quantity > 0),
  unit_price  NUMERIC(12,2) NOT NULL,      -- the price AT THE TIME. Not a copy – a snapshot.
  PRIMARY KEY (order_id, product_id)
);
```

`unit_price` is the line worth defending. It looks like a violation and is not. `products.price` is *today's* price; `order_items.unit_price` is *what this customer was charged*. If you removed it and joined to `products` instead, every historical invoice would silently change whenever you updated the catalogue. Being able to say that sentence is worth more than reciting the normal forms.

Notice too that `total` has disappeared: it is derivable from the items, so storing it is a genuine denormalisation — one you might well want, but a decision rather than an accident.

Keeping a denormalised count honest

Three mechanisms, in increasing order of robustness.

In the same transaction as the write. Exact, and it couples the two operations: the comment insert now also takes a lock on the post row, which becomes a contention point on a popular post.

```
BEGIN;  
  INSERT INTO comments (post_id, author_id, body) VALUES (91, 42, '...');  
  UPDATE posts SET comment_count = comment_count + 1 WHERE id = 91;  
COMMIT;
```

A database trigger. The database maintains it, so no application path can forget — including the script somebody wrote in a hurry. The cost is logic living somewhere nobody looks, which is a real maintenance complaint.

Asynchronously, via an event or a queue. The write is fast and the count is eventually correct. Now it *will* be wrong sometimes, so you need the count to be recoverable — which is the job below.

And, whichever you pick, the reconciliation job:

```
UPDATE posts p  
SET    comment_count = c.n  
FROM   (SELECT post_id, COUNT(*) AS n FROM comments GROUP BY post_id) c  
WHERE  c.post_id = p.id AND p.comment_count <> c.n;
```

Run it nightly, and — this is the part people leave out — **log how many rows it corrected**. A count that is always zero means the maintenance is working. A count that grows means you have a bug, and you find out from a graph instead of from a customer.

Materialised views

Postgres can store the result of a query as a table and refresh it on demand:

```
CREATE MATERIALIZED VIEW daily_revenue AS  
SELECT order_date, SUM(quantity * unit_price) AS revenue  
FROM   orders o JOIN order_items i ON i.order_id = o.id  
GROUP BY order_date;  
  
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue;
```

This is denormalisation with the bookkeeping built in: the definition of correctness lives in the view, so it cannot drift silently — it can only be stale, and you control how stale by choosing the refresh schedule. **When the denormalised thing is a whole aggregate rather than one column, a materialised view is usually the right shape**, and naming it is a good signal.

The document-store escape hatch

MongoDB and DynamoDB take denormalisation as the default: you embed the comments inside the post document, so one read gets everything. That removes joins entirely and makes the “one fact, one place” rule impossible to keep — a user’s display name embedded in ten thousand comments is ten thousand copies.

Their answer is that you design around the access pattern and accept the duplication, updating copies in the background. It works, and it moves the entire cost from read time to write time and to your own discipline. Documents are [day 038](#); the point today is that **“denormalise everything” is a coherent position with a known price**, not a mistake.

6 The numbers

What the join actually costs

Rendering a post with its comment count, 3,700 reads per second:

```
COUNT(*) on an indexed post_id, ~200 comments : ≈ 1 ms  
3,700 × 1 ms = 3.7 s of database time per second → ~4 cores doing nothing else
```

With `comment_count` stored on the post:

```
read cost : 0 — it is a column of the row you already fetched  
write cost : one extra UPDATE per comment  
300,000 comments/day ÷ 86,400 = 3.5 writes/s → negligible
```

Four cores against three and a half writes a second. That is the trade, and it is not close.

The ratio that decides it

```
reads : 80,000,000/day → 926/s average  
writes : 300,000/day → 3.5/s average  
ratio : 265 : 1
```

Rule of thumb: below roughly 10:1, do not denormalise — the write cost starts to matter and the read saving does not. Above 100:1 it is nearly always right. Between the two, measure.

The cost of drift

If the count is maintained asynchronously and 0.1% of updates are lost:

```
300,000 comments/day × 0.001 = 300 wrong counts/day
                             = 109,500 wrong counts/year
```

Which is why the nightly reconciliation exists — and why it should report the number it fixed rather than silently correcting.

Storage of the copies

```
comment_count : 4 bytes per post × 1,000,000 posts      = 4 MB
customer_name copied onto every order:
  30 bytes × 10,000,000 orders                          = 300 MB
versus a foreign key: 8 bytes × 10,000,000              = 80 MB
```

Storage is almost never the argument. **The argument is always drift on one side and read cost on the other**, and saying that plainly is better than reaching for disk figures.

Joins avoided

Rendering a feed of 20 posts with author name, comment count and reaction count:

```
fully normalised : 1 query for posts + 3 joins, or 61 queries if an ORM does it lazily
denormalised     : 1 query, no joins

fully normalised, joined properly : ~5 ms
the ORM's N+1 version             : 61 × 0.5 ms = ~30 ms and 61 round trips
denormalised                     : ~1 ms
```

Note the middle row. **Often the real problem is not the normalisation but an ORM issuing 61 queries**, and the fix is to write the join rather than to duplicate the data. Check that before denormalising anything.

Update amplification

Renaming a department in a normalised schema versus a denormalised one:

```
normalised : UPDATE departments SET name = 'X' WHERE id = 4;          1 row
denormalised : UPDATE employees SET dept_name = 'X' WHERE dept_id = 4; 400 rows
               - and any row missed is now permanently inconsistent
```

7 The trade-offs

Normalised

You get one source of truth, so nothing can disagree with itself; smaller storage; and updates that touch one row. You also get a schema that can answer questions you have not thought of yet, because nothing has been shaped around one particular query.

You pay joins on every read, more complex queries, and — at high read volume — real database CPU.

Denormalised

You get fast reads, simple queries, and the ability to serve a screen from one row.

You pay drift risk, larger storage, more expensive writes, and a maintenance obligation that never goes away. And you pay it *later*, which is what makes it dangerous: the schema that was fast in year one is the one with three counts that disagree in year three.

Where to draw the line

Normalise by default. Denormalise specific fields, with numbers, and with a job. Not whole tables, not on instinct, and never without the reconciliation.

The specific things almost always worth denormalising:

- **Counts and aggregates** read far more often than written.
- **Historical snapshots** — price at time of order, name at time of invoice. Not really denormalisation, and mandatory.
- **Materialised views** for reports that would otherwise scan millions of rows.

The things almost never worth it:

- **A name or address you copy to avoid one join.** The join is cheap; the drift is not.
- **Anything with a read-to-write ratio under 10:1.**
- **Anything you cannot reconcile**, because you will never know whether it is right.

Do it in the transaction, or asynchronously?

Synchronous is exact and couples the write to the counter, which becomes a contention point — every comment on a viral post now contends for the same row. Asynchronous keeps writes fast and guarantees the count will sometimes be wrong. **For a count nobody makes decisions on, asynchronous plus a nightly job is right. For anything a person or a payment depends on, do it in the transaction or do not denormalise it at all.**

I would not denormalise without a reconciliation job. A stored count with nothing checking it is not an optimisation, it is a number that is right on the day you ship and slowly stops being right after that — and because nothing errors, you find out from a customer. So the design has three parts, not one: the copy, the thing that maintains it, and the thing that detects drift and reports how much it found. If I cannot describe all three, I would keep the join and fix the query instead — which, more often than people expect, turns out to be an ORM issuing sixty-one queries where one would do.

8 In the interview

How it gets asked

- *“When would you deliberately duplicate data across tables?”* — the direct version. They want a condition with numbers, not “for performance”.
- *“What is third normal form?”* — the theory check. Answer in one sentence, then move to the trade.
- *“Normalise this table.”* — shown a wide flat table with lists in cells.
- *“This page is slow because of joins. What do you do?”* — where the right first answer is *measure*, and the right second answer is often “fix the query, not the schema”.
- *“How do you keep a denormalised count correct?”* — the follow-up that separates people who have done it from people who have read about it.

What to say out loud, in the first ninety seconds

1. **Give the rule in one sentence.** *“Normalisation means each fact is stored in exactly one place, so nothing can disagree with itself.”*
2. **Give 3NF in the memorable form.** *“Every non-key column depends on the key, the whole key, and nothing but the key. 3NF is where practice stops.”*
3. **Name the anomalies.** *“It exists to prevent update, insert and delete anomalies — renaming a department and missing three rows, being unable to record a department with no employees, and losing the department when the last employee leaves.”*
4. **State your default.** *“I’d normalise to 3NF first, always.”*
5. **Then give the exception with a number.** *“And denormalise specific fields where the read-to-write ratio justifies it. On a comment feed that ratio is around 265 to 1 — counting comments on every read is four cores of database time, and maintaining a stored count is three and a half writes a second.”*

6. Distinguish the snapshot. *“Some things that look like duplication aren’t — storing the unit price on an order line is a historical fact, not a copy, and it must not change when the catalogue does.”*

7. Name the obligation, unprompted. *“And any real copy needs a reconciliation job that reports how many rows it corrected. Without that I’d keep the join.”*

The follow-ups

“What is third normal form, in one sentence?” Every non-key column depends on the key, the whole key, and nothing but the key. Unpacking that: 1NF means each cell holds one value, so no comma-separated lists and no `product_1`, `product_2` columns. 2NF adds that when the primary key is composite, no column may depend on only part of it — a product’s name on an order line depends on the product, not on the order-and-product pair, so it belongs in the products table. 3NF adds that no non-key column may depend on another non-key column — a department name depends on the department id, which is itself not the key, so it belongs in a departments table. There are higher forms, BCNF and above, and they address genuinely rare situations; 3NF is the working standard, and what I would actually do is normalise to 3NF and then denormalise deliberately where the numbers justify it.

“How do you keep a denormalised count correct?” Three options and I would choose by how much correctness matters. Updating it in the same transaction as the insert is exact, and the cost is that every comment now takes a lock on the post row, so a viral post becomes a contention point. A database trigger is the same thing enforced by the database, so no code path can forget it — including a one-off script — at the price of logic living somewhere nobody looks. Doing it asynchronously through an event keeps writes fast and guarantees the count will occasionally be wrong. Whichever I pick, I would also run a nightly reconciliation that recomputes the true counts and corrects any that disagree — and, importantly, logs how many it corrected, so a non-zero number becomes an alert rather than a silent repair. If I could not commit to that job, I would not store the count.

“Is storing the price on an order line a violation of normalisation?” No, and this is the case worth getting right. `products.price` is today’s price. `order_items.unit_price` is what this customer was actually charged on that date. Those are two different facts that happen to be equal at one moment, so storing both is not duplication. The proof is what happens if you remove it: join to `products` instead, and every historical invoice silently changes the next time you update the catalogue, which is wrong and in most places illegal. The same argument applies to a customer’s name and address on an invoice. The way I distinguish it in general is to ask whether the two values are *supposed* to stay equal. If yes, it is a copy and needs a reconciliation job. If they are supposed to diverge over time, it is a snapshot and needs nothing.

“This page does eight joins and it’s slow. Do you denormalise?” Not first. First I would look at the actual query plan, because “slow because of joins” is a diagnosis people reach for and it is often wrong. The three things I would check: is it really one query with eight joins, or is an ORM issuing sixty-one queries lazily — which is the $N + 1$ problem and is fixed by eager loading, not by changing the schema. Are the join columns indexed, since foreign keys are not indexed automatically and an unindexed one turns a join into a scan per row. And is the query selecting columns nobody uses, which prevents a covering index. Those three fix a large majority of “slow because of joins” complaints without touching the data model. If after that it is genuinely the joins, then I would denormalise the specific fields the page needs, compute the read-to-write ratio to justify each one, and build the reconciliation job — or use a materialised view, which gives me the same speed with the correctness definition written down in one place.

A model answer

“Normalisation means each fact is stored in exactly one place, and everything else refers to it by key. The formal levels are 1NF, no lists in a cell; 2NF, no column depending on only part of a composite key; 3NF, no column depending on another non-key column. The version worth memorising is that every non-key column depends on the key, the whole key, and nothing but the key.

It exists to prevent three specific things. An update anomaly — you rename a department, update four hundred rows, miss three, and now two answers exist. An insert anomaly — you cannot record a department until somebody works in it. And a delete anomaly — the last employee leaves and the department disappears with them.

So my default is to normalise to 3NF. But I'd denormalise specific fields deliberately, and I'd want a number for each one.

The clearest example is a comment count on a post. Normalised, rendering a post means `COUNT(*)` on the comments table — about a millisecond with an index. At 3,700 reads a second that's 3.7 seconds of database time per second, so roughly four cores doing nothing but counting. Storing the count on the post makes the read free and costs one extra UPDATE per comment, which at 300,000 comments a day is three and a half writes a second. The read-to-write ratio there is about 265 to 1, and that ratio is what makes the decision — below roughly 10 to 1 I wouldn't bother.

There's a case that looks like denormalisation and isn't, which I'd want to distinguish. Storing the unit price on an order line isn't a copy of the product's price — it's what the customer was actually charged that day. Two different facts. If I removed it and joined to the products table, every historical invoice would change the next time someone updated the catalogue. Same with a name on an invoice. Those never drift, because they're never meant to match.

The part I'd insist on for a real copy is the third piece. A denormalised count has three components, not one: the copy, something that maintains it — in the transaction, a trigger, or an async event — and a reconciliation job that recomputes the truth periodically and reports how many rows it had to correct. That last number is the whole point: zero means it's working, and a rising number is a bug I find from a graph rather than from a customer. If I couldn't commit to that job, I'd keep the join.

And before denormalising anything I'd check that the joins are actually the problem. Very often 'slow because of joins' turns out to be an ORM issuing sixty-one queries where one would do, or an unindexed foreign key — and both of those are cheaper

to fix than a data model.”

9 Recall card

- **Normalisation = each fact in exactly one place.** 3NF: every non-key column depends on the key, the whole key, and nothing but the key.
- **It prevents three anomalies:** update, insert, delete. Name all three.
- **Denormalise by the read-to-write ratio.** Below ~10:1, don't. At 265:1, a stored count saves four cores and costs 3.5 writes a second.
- **A snapshot is not a copy.** `order_items.unit_price` is what was charged — it *should* differ from today's price, and it never drifts.
- **Every real copy needs a reconciliation job that reports what it fixed.** No job, no copy — keep the join instead.

PRACTICE

Practice — Same direction: the read pointer and the write pointer

DSA topic: Same direction: the read pointer and the write pointer **System design topic:** Normalisation and when to break it

Code these, in this order

Four problems on the same-direction skeleton, chosen so that each one uses a different sub-shape. **Before writing any of them, say which sub-shape it is:** filter, compare-against-what-you-kept, partition by swapping, or walk from the back.

Before each one, ask:

1. In place? Do I return a count, or is the array itself checked?
2. Does the relative order of the survivors matter? Of the discarded group?
3. Am I comparing against the current element, or against something I already kept?
4. What happens on the empty input, and on one element?

#	PROBLEM	SOURCE	SUB-SHAPE, AND WHAT IT IS REALLY TESTING
1	Remove Duplicates from Sorted Array	LeetCode 26 (Easy)	Compare-against-what-you-kept. <code>nums[write - 1]</code> , and the empty guard that silently returns 1 without it.
2	Remove Duplicates from Sorted Array II	LeetCode 80 (Medium)	The same, at <code>write - 2</code> . Should be a one-character change. If it is a rewrite, problem 1 compared against the wrong thing.
3	Sort Colors	LeetCode 75 (Medium)	Partition, three pointers. The whole question is why <code>mid</code> advances on a 0 and not on a 2.
4	Backspace String Compare	LeetCode 844 (Easy)	Walk from the back. Easy with <code>O(n)</code> space; the question is the <code>O(1)</code> version.

On problem 1, find the silent bug

Write it, then delete the empty guard and run it on `[]`.

```
def remove_duplicates(nums):
    write = 1 # no empty check
    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1
    return write

print(remove_duplicates([1]))
```

It returns `1`, with no exception. Say why the loop never runs, why `nums[write - 1]` is therefore never evaluated, and why that makes this worse than a crash.

On problems 1 and 2, prove the habit

Solve problem 1 comparing against `nums[write - 1]`. Then solve problem 2 by changing **one character**.

Then write the deliberately wrong version and find the input that catches it:

```
def keep_at_most_two(nums):
    write = 0
    for read in range(len(nums)):
        if write < 2 or nums[read] != nums[read - 2]: # read, not write
            nums[write] = nums[read]
            write += 1
    return write

a = [1, 1, 1, 2, 2, 3]
k = keep_at_most_two(a)
print(k, a[:k]) # 4 [1, 1, 2, 3] - a 2 has been LOST
```

Then check that it gives the *right* answer on `[1, 1, 1, 1, 2, 3]`. That is the point: the wrong version passes the input you would naturally try.

Then generalise to `keep_at_most_k` and confirm `k = 1` reproduces problem 1.

On problem 3, break it and find the input yourself

Write the version that advances `mid` after swapping in a 2. Then, without looking anything up, find an input of length 3 that it gets wrong.

```
elif nums[mid] == 2:
    nums[mid], nums[high] = nums[high], nums[mid]
    high -= 1
    mid += 1 # the bug
```

Try `[2, 0, 1]` first. It works — which is why this bug survives. Now try `[1, 2, 0]` and `[2, 1, 2]`.

Then answer:

1. Why is advancing `mid` **correct** in the `0` branch and **wrong** in the `2` branch?
2. Why is the loop `mid ≤ high` and not `mid < high`?
3. Why does the loop terminate, given that `mid` sometimes stands still?
4. What are the four regions of the array, and what is true of each?

On problem 4, do it twice

Write the easy version — build both strings with a list and `join`, then compare. State its cost.

Then write the `O(1)`-space version walking backwards with a skip count, and answer:

1. Why can you not decide, going forwards, whether a character survives?
2. Why can you, going backwards?
3. What goes wrong if you use `zip` instead of `zip_longest`? Give an input.
4. Where else have you seen “go in the direction where the answer is already determined”?

The invariant drill

For each, state the invariant in one sentence and say what it makes safe:

1. The filter skeleton — `write` and `read`.
2. Dedupe — what does `items[0:write]` hold at any moment?
3. Sort Colors — name the four regions and what is true of each.
4. Merge sorted arrays from the back (day 018) — why can the write index never collide?

The stability drill

Run this and predict the output first:

```
def partition_even_odd(nums):
    write = 0
    for read in range(len(nums)):
        if nums[read] % 2 == 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1
    return write

a = [3, 1, 2, 4]
print(partition_even_odd(a), a)
```

Then answer:

1. Which group kept its order, and which did not? Why?
2. When would that matter, and when would it not?
3. If both groups must keep their order, what does that cost you?
4. Which of the copy version and the swap version does more writes, and on what kind of input?

The sub-shape drill

Classify each in under five seconds, and name the comparison:

1. Move all zeros to the end.
2. Remove every occurrence of a value.
3. Remove duplicates from a sorted array.
4. Keep each value at most three times.
5. Sort an array of 0s, 1s and 2s.
6. Partition an array around a pivot.
7. Compare two strings with backspaces.
8. Merge a sorted array into the tail of another.
9. String compression, `aabcccccaaa` to `a2b1c5a3`.

Numbers 6 and 5 are the same shape at different widths. Numbers 7 and 8 both run backwards, for the same reason.

The cost drill

State time and space, counting out loud, for each:

1. The filter skeleton.
2. `keep_at_most_k`.
3. Sort Colors.
4. Backspace compare, both versions.
5. Removing duplicates with `del nums[i]` inside a loop.

For number 5, give the number of element moves at `n = 100,000` and compare it with number 1.

The normalisation drill

Here is a table somebody actually wrote. Normalise it to 3NF, saying which form each fix satisfies:

bookings						
id	guest_name	guest_email	guest_city	room_nums	nights	room_type
1	Asha	asha@x.com	Coimbatore	"12, 14"	3	"AC, AC"
2	Ravi	ravi@x.com	Erode	"7"	1	"Non-AC"

Then answer:

1. Which column breaks 1NF, and what does that make impossible?
2. Once you have a `booking_rooms` table keyed on `(booking_id, room_num)`, which column breaks 2NF?
3. Which column depends on another non-key column, breaking 3NF?
4. Which column should stay duplicated on purpose, and why is it not a violation?

Number 4 is the rate charged per night. Say why it must be stored on the booking.

The denormalisation drill

For each, say **normalise** or **denormalise**, and give the deciding number or reason:

1. A post's comment count, read 3,700 times a second and written 3.5 times a second.
2. A customer's current address, needed on every order display.
3. The price a customer paid, shown on an old invoice.
4. A user's display name, shown on every one of their 10,000 comments.
5. A daily revenue total for a dashboard that would otherwise scan 50 million rows.
6. An order's total, derivable by summing its items.

Numbers 3 and 6 are interesting for opposite reasons. Say why.

The three-parts drill

Any denormalised copy has three parts. For a post's `comment_count`, write out all three:

1. The copy itself — what column, what type?
2. What maintains it — name three options and pick one, with a reason.
3. What detects drift — write the reconciliation query, and say what it should log.

If you cannot write part 3, say out loud what you would do instead.

The slow-page drill

"This page does eight joins and it's slow. Do you denormalise?"

Say the three things you would check **before** touching the schema, and for each one say what the fix would be. Then say when you would actually reach for denormalisation, and what you would build alongside it.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Remove duplicates in place and return the new length.* Contract questions. Name the shape. State the invariant — `write` never overtakes `read` — and say what it makes safe. Say you compare against `nums[write - 1]` and **why that specific choice**. Finish with $O(n) / O(1)$, and what `write` means at the end.
2. *When would you deliberately duplicate data across tables?* 3NF in one sentence, the three anomalies by name, then your default. Then the exception with the ratio: 265 to 1, four cores against 3.5 writes a second. Distinguish the snapshot from the copy. Finish with the reconciliation job, unprompted.
3. *Sort an array of 0s, 1s and 2s in one pass.* Four regions, three pointers. The whole answer is why `mid` advances on a 0 and not on a 2 — what is known about the value coming back from each end. Then termination, then $O(n) / O(1)$, then mention counting sort and why the problem forbids it.

Before you move on

- [] I can state the `write ≤ read` invariant and say exactly what it makes safe.
- [] I compare against `items[write - 1]`, not `items[read - 1]`, without thinking about it.
- [] I can turn dedupe into keep-at-most-`k` by changing one character.
- [] I write the empty guard on anything that pre-keeps element 0.
- [] I know the copy version is stable and the swap version is not, and can show it.
- [] I can say why `mid` does not advance on a 2, and name an input that proves it.
- [] I can define 3NF in one sentence and name all three anomalies.
- [] I decide denormalisation with a read-to-write ratio, not with taste.
- [] I can tell a snapshot from a copy, and I never denormalise without a reconciliation job.
- [] I can redraw the read/write gap diagram and the four-regions diagram from memory, in whatever tool I like.

Fast and slow pointers

DATA STRUCTURES AND ALGORITHMS

Fast and slow pointers

You can find the middle of a sequence in one pass and detect a loop in it.

SYSTEM DESIGN

Indexes: how a database finds a row fast

You can explain why an index makes reads fast and writes slower.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the middle element in a single pass.*
- *This query is slow. How do you find out why, and what do you do about it?*

1 What this is, and why they ask it

The third of the three two-pointer shapes. Both indices start at the same place and move in the same direction, but **at different speeds** — one step at a time for the slow one, two for the fast one.

That single arrangement answers two apparently unrelated questions.

Where is the middle? When the fast index reaches the end, it has taken twice as many steps, so the slow index is exactly halfway. One pass, no length needed in advance — which matters enormously for anything you can only walk forwards through.

Is there a loop? On a straight path the fast one pulls away and you never see it again. If the path bends back on itself, the fast one comes round and catches the slow one from behind. **That catching is the proof**, and it needs no memory of where you have been.

The technique is **Floyd's cycle detection**, universally called *tortoise and hare*. It is asked constantly, in three costumes: on linked lists from [day 083](#), on arrays that encode a function — *Find the Duplicate Number*, LeetCode 287 — and on number sequences — *Happy Number*, LeetCode 202. The costume changes and the six lines do not.

The reason interviewers like it is that the obvious solution to cycle detection is a hash set of everything seen, which is $O(n)$ space, and Floyd's is $O(1)$. **A candidate who produces the set version and then says “but I can do it in constant space” has answered the question**; one who only knows the set has not.

2 The story

There is a walking path all the way round the lake at the back of Girish's colony, and he does it every morning at ten past six, four rounds, about an hour.

His neighbour Mani runs it. Not fast, but properly, and roughly twice Girish's speed.

They start together at the gate most mornings and they talk for the first thirty seconds and then Mani goes on ahead. What Girish noticed in his first week — and it delighted him, in the way small regularities do — is that Mani comes past him again. From behind. Every eleven minutes or so, without fail, there is the sound of Mani's shoes coming up behind him and a hand on his shoulder as he goes by.

The path is a loop, so that is what happens. Mani gets further and further ahead, and then the ahead becomes behind, and he arrives from the other side.

Girish said something to him about it once and Mani, who thinks about these things, said the useful part back: on a loop, if one man goes twice as fast as another, he will always catch him. He cannot not. He gains steadily on him, and there is nowhere for the slow man to escape to.

The other thing happened in October, when the council dug up the far side for a drain and made the path an out-and-back — down to the far corner and turn round and come back. That morning Mani never came up behind him at all. Girish saw him coming the other way, waved, and that was that. A straight path out and a straight path back, and the faster man simply gets to the end and returns; he never laps anybody, because there is no lap.

There is one more thing they do, and it is the reason they still start together. Girish's wife drops them at the gate and comes back for them. So when Mani finishes his round and reaches the gate again, he tells her where Girish is — and he is always at the same place, right across the water, exactly halfway. Twice the speed, twice the distance, so when one of them is at the end the other is at the middle. She has stopped asking.

3 The idea in plain English

Girish is `slow`, moving one step at a time. Mani is `fast`, moving two. Everything today comes from that and from one fact: **if there is a loop, the fast one must catch the slow one.**

Finding the middle

```
slow = fast = 0
while fast + 1 < len(items):
    slow += 1
    fast += 2
return slow
```

`fast` covers two positions for every one of `slow`'s. When `fast` can go no further, it has travelled about `n` and `slow` has travelled about `n/2` — so `slow` is at the middle.

Why this is worth doing on an array at all, where you could just use `len(items) // 2`: because the same six lines work on a structure you can only walk forwards through and whose length you do not know. That is a linked list, and it is why this is the standard answer to “*find the middle of a linked list in one pass*”. On an array it is a demonstration; on a list it is the only sensible method.

The contract question: on an even-length sequence there are two middles. `[1,2,3,4]` — is it `2` or `3`? The loop above gives the **second** one, index 2, which is the LeetCode 876 convention. Ask. Changing the loop condition to `while fast + 2 < len(items)` gives the first. **Say which one you are returning and why.**

Detecting a loop

Now the same two speeds, on something that might bend back:

```
slow = fast = start
while True:
    slow = step(slow)
    fast = step(step(fast))
    if slow == fast:
        return True          # they met: there is a cycle
```

On a path with an end you never get here — the fast one runs off the end and you stop. On a loop, they meet.

Why the meeting must happen — this is the question

Interviewers ask “*why are you sure they meet?*” and the answer is short and worth having exactly.

Once **both** are inside the loop, look at the gap between them, measured the way round the loop from `fast` to `slow`. Every step, `slow` advances 1 and `fast` advances 2, so **the gap shrinks by exactly 1 per step.**

A quantity that starts at some value less than the loop length and decreases by exactly one each step must reach zero. And because it decreases by exactly one, **it cannot jump over zero** — there is no way for the fast one to step past the slow one without landing on it.

So they meet, in at most one loop-length of steps after both are inside. That is the whole proof, and “the gap shrinks by exactly one, so it cannot skip zero” is the sentence to say.

The reason the fast one moves at 2 and not 3 is precisely this. At speed 3 the gap shrinks by 2 per step, so it can go from 1 to -1 and step straight past. It still works out on a single loop for other reasons, but the argument stops being one line — and the second phase below breaks. Use 2.

Where the loop starts — Floyd’s second phase

Detecting a loop is often not enough; you want the node where it begins. There is a small piece of arithmetic that gives it, and it looks like magic until you see it once.

Let μ be the distance from the start to where the loop begins, and λ the loop's length.

When the two meet, `slow` has walked `d` steps and `fast` has walked `2d`. Both are on the loop, and `fast` has gone round some whole number of extra loops, so:

$$2d - d = k \cdot \lambda \quad \rightarrow \quad d = k \cdot \lambda$$

The slow one has walked a whole number of loop-lengths. Now, `slow` is `d - μ` steps into the loop. If you start a fresh index at the beginning and move both one step at a time, then after `μ` steps the fresh one is at the loop entry, and `slow` has moved `μ` further — reaching `d - μ + μ = d = k · λ` steps into the loop, which is the loop entry again.

So:

```
slow = start                # reset one index to the beginning
while slow  $\neq$  fast:
    slow = step(slow)
    fast = step(fast)
return slow                 # BOTH at speed 1 now
                             # the loop entry
```

Both move one step at a time in the second phase. Leaving `fast` on two steps is the standard bug and it gives a wrong answer with no error.

The loop's length

Once they have met, stand still and walk one of them round until it comes back:

```
length = 1
cur = step(meeting_point)
while cur  $\neq$  meeting_point:
    cur = step(cur)
    length += 1
```

Arrays as paths: the trick that makes this apply everywhere

An array of numbers where every value is a valid index is a **path**. From position `i`, the next position is `nums[i]`. Follow it and you are walking.

```
nums = [1, 3, 4, 2, 2]
```

0 → nums[0]=1 → nums[1]=3 → nums[3]=2 → nums[2]=4 → nums[4]=2 → nums[2]=4 → ...

^

the loop: 2 → 4 → 2 → 4 ...


```

index    0    1    2    3    4
+-----+-----+-----+-----+
|       |       |       |       |
+-----+-----+-----+-----+

start    s,f
step 1    s           f
step 2           s           f      fast + 1 = 5, not < 5, so stop
              ^
            slow = 2, the middle of 5

even length, n = 4:
start    s,f
step 1    s           f
step 2           s           (fast would be 4) → stop, slow = 2
              ^
            the SECOND of the two middles

```

The array-as-a-path view, for `[1, 3, 4, 2, 2]`:

```

index:  0    1    2    3    4
value:  1    3    4    2    2

walk:   0 → 1 → 3 → 2 → 4 → 2 → 4 → 2 → ...
        |---- tail ----|   |--- loop ---|
                mu = 3           lambda = 2

```

Two positions (2 and 4) both hold the value 2, so two arrows point at index 2 – and that is exactly what makes a loop. The loop entry is 2, which is the duplicated value.

What to notice: the duplicate is not found by comparing values at all. It is found by noticing that the path joins, and the join point is the duplicate. That reframing is the whole insight of LeetCode 287.

5 The code, built step by step

The middle

```

def middle_index(items: list[int]) → int:
    if not items:
        return -1
    slow = fast = 0
    while fast + 1 < len(items):
        slow += 1
        fast += 2
    return slow

```

`fast + 1 < len(items)` asks *can fast take two more steps?* If it can, take them; otherwise stop.

For a linked list, where you cannot ask for a length, the same shape reads:

```
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
return slow
```

`fast and fast.next`, in that order. Python's `and` short-circuits, so if `fast` is `None` the second test is never evaluated. Reverse them and you get `AttributeError: 'NoneType' object has no attribute 'next'` on any even-length list. Linked lists arrive properly on [day 078](#); the ordering rule is worth carrying now.

Cycle detection

```
def has_cycle(step, start):
    slow = fast = start
    while True:
        slow = step(slow)
        fast = step(step(fast))
        if slow == fast:
            return True
```

For a structure that can end, you need the escape:

```
while fast and fast.next:
    ...
return False          # fast ran off the end: no cycle
```

For a **function** that never ends — happy numbers, `nums[i]` where every value is a valid index — there is nothing to run off, so `while True` is correct and the loop is guaranteed to terminate by the gap argument.

Where the loop begins

```
# phase 1: find a meeting point
slow = fast = start
while True:
    slow = step(slow)
    fast = step(step(fast))
    if slow == fast:
        break

# phase 2: reset one to the start, then move BOTH at speed 1
slow = start
while slow != fast:
    slow = step(slow)
    fast = step(fast)
return slow
```

Two things that are easy to get wrong and are the whole of this half:

Reset `slow`, not `fast`. It does not matter which name you reset as long as one goes back to the start and one stays at the meeting point — but be deliberate, because swapping them by accident gives a wrong answer with no error.

Both move one step in phase two. The arithmetic in §3 depends on it.

Find the Duplicate Number

```
def find_duplicate(nums: list[int]) → int:
    slow = fast = nums[0]
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break
    slow = nums[0]
    while slow != fast:
        slow = nums[slow]
        fast = nums[fast]
    return slow
```

Note the starting point: `nums[0]`, which is one step in from index 0. Starting both at `0` works too as long as you are consistent about taking the first step before the first comparison — otherwise `slow == fast` is true immediately and you exit at once. **This off-by-one is the commonest way to get this problem wrong**, and the tell is that it returns the value at index 0.

Happy Number

```
def is_happy(n: int) → bool:
    def next_value(x: int) → int:
        total = 0
        while x:
            x, digit = divmod(x, 10)
            total += digit * digit
        return total

    slow = fast = n
    while True:
        slow = next_value(slow)
        fast = next_value(next_value(fast))
        if slow == fast:
            return slow == 1
```

The sequence either reaches 1 — which is itself a loop of length one, since $1 \rightarrow 1$ — or enters a different loop. So the two always meet, and **what they meet on is the answer**: 1 means happy, anything else means not.

The complete solutions

```
def middle_index(items: list[int]) → int:
    """Index of the middle. On even lengths, the SECOND middle (LeetCode 876 convention)."""
    if not items:
        return -1
    slow = fast = 0
    while fast + 1 < len(items):          # can fast take two more steps?
        slow += 1
        fast += 2
    return slow

def has_cycle(step, start):
    """Floyd's cycle detection on any repeated function. O(1) space."""
    slow = fast = start
    while True:
        slow = step(slow)
        fast = step(step(fast))
        if slow == fast:
            return True

def cycle_start_and_length(step, start) → tuple[object, int]:
    """The entry point of the loop, and its length."""
    slow = fast = start
    while True:                          # phase 1: meet somewhere inside the loop
        slow = step(slow)
        fast = step(step(fast))
        if slow == fast:
            break

    length = 1
    cur = step(slow)                     # phase 2a: walk one all the way round
    while cur != slow:
        cur = step(cur)
        length += 1

    slow = start                         # phase 2b: reset one, BOTH at speed 1
    while slow != fast:
        slow = step(slow)
        fast = step(fast)
    return slow, length

def find_duplicate(nums: list[int]) → int:
    """LeetCode 287. n+1 values in 1..n, one repeated. O(n) time, O(1) space,
    without sorting and without modifying the array.

    nums is a path: from i you go to nums[i]. A repeated value means two
    positions point at the same place, which makes a loop – and the loop
    entry IS the duplicate.
    """
    slow = fast = nums[0]                # one step in, so the first comparison is meaningful
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break
    slow = nums[0]
    while slow != fast:                  # both at speed 1
        slow = nums[slow]
        fast = nums[fast]
    return slow

def is_happy(n: int) → bool:
    """LeetCode 202. Repeatedly sum the squares of the digits. Loops or reaches 1."""
    def next_value(x: int) → int:
        total = 0
```

```

while x:
    x, digit = divmod(x, 10)
    total += digit * digit
return total

slow = fast = n
while True:
    slow = next_value(slow)
    fast = next_value(next_value(fast))
    if slow == fast:
        return slow == 1          # 1 is itself a loop of length 1

if __name__ == "__main__":
    for case in ([1], [1, 2], [1, 2, 3], [1, 2, 3, 4], [1, 2, 3, 4, 5], [1, 2, 3, 4, 5, 6]):
        i = middle_index(case)
        print(case, "→ index", i, "value", case[i])
    # [1,2,3,4] → index 2 value 3      ← the SECOND middle

    for case in ([1, 3, 4, 2, 2], [3, 1, 3, 4, 2], [1, 1], [1, 1, 2],
                 [2, 2, 2, 2, 2], [1, 4, 6, 6, 6, 6, 3, 2, 5]):
        print(case, "→ duplicate", find_duplicate(list(case)))
    # [1,3,4,2,2] → 2   [3,1,3,4,2] → 3   [1,4,6,6,6,6,3,2,5] → 6

    print([(n, is_happy(n)) for n in (19, 2, 1, 7, 4, 100)])
    # [(19, True), (2, False), (1, True), (7, True), (4, False), (100, True)]

```

6 What it costs

Finding the middle

`fast` advances by 2 each turn and stops near n , so the loop runs about $n/2$ times, each doing constant work. **$O(n)$ time, $O(1)$ space** — two integers.

Against the obvious alternative on a linked list — walk it once to count, then walk half of it again — that is $2n$ steps against $1.5n$. Same complexity; the reason to prefer one pass is that **on a stream you cannot walk it twice**, which is the honest answer when asked why it matters.

Cycle detection

Let μ be the tail length and λ the loop length.

Phase 1. `slow` reaches the loop after μ steps. From there the gap closes by one per step, and the gap is less than λ , so they meet within λ more. Total at most $\mu + \lambda$ iterations, and $\mu + \lambda \leq n$. **$O(n)$ time.**

Phase 2a, the length: one walk round the loop, λ steps. **Phase 2b**, the entry: μ steps.

Total $O(n)$ time, $O(1)$ space — a fixed number of variables regardless of the input size.

Against the hash-set version

```
seen = set()
while node:
    if node in seen: return True
    seen.add(node)
    node = node.next
```

Also $O(n)$ time, and $O(n)$ space. On ten million nodes:

```
hash set : 10,000,000 entries × ~50 bytes ≈ 500 MB
Floyd    : two variables                ≈ 16 bytes
```

That is the entire reason Floyd's algorithm is worth knowing. Write the set version first, state its cost, then offer the constant-space one — that sequence is the complete answer.

`find_duplicate` , and why the constraints matter

The problem forbids modifying the array and asks for constant space, which rules out:

```
sort then scan      :  $O(n \log n)$ , and it MODIFIES the array
a hash set          :  $O(n)$  time,  $O(n)$  space
counting array       :  $O(n)$  time,  $O(n)$  space
marking values negative :  $O(n)$  time,  $O(1)$  space — but MODIFIES the array
binary search on value :  $O(n \log n)$  time,  $O(1)$  space, no modification
Floyd                :  $O(n)$  time,  $O(1)$  space, no modification ← the intended answer
```

That table is worth having, because a good answer walks down it. The binary-search-on-value approach is a genuine alternative and worth naming: count how many values are $\leq \text{mid}$; if that count exceeds mid , the duplicate is in the lower half.

`is_happy`

The sequence of digit-square-sums drops quickly and then stays small: for any number below 1,000, the next value is at most $9^2 \times 3 = 243$. So the reachable set is tiny and both μ and λ are small constants — in practice under a dozen steps. **Effectively $O(\log n)$ time** to reduce the original number, then constant, and **$O(1)$ space**.

The number to have ready

Floyd's is $O(n)$ time and $O(1)$ space, against $O(n)$ space for the hash-set version — 16 bytes against 500 MB on ten million nodes. The meeting is guaranteed because the gap shrinks by exactly one per step and therefore cannot skip zero.

7 The traps

The real error: testing `fast.next` before `fast`

```
while fast.next and fast:
    slow = slow.next
    fast = fast.next.next
```

```
AttributeError: 'NoneType' object has no attribute 'next'
```

On any even-length list, `fast` becomes `None` and the first half of the condition dereferences it. `and` evaluates left to right and short-circuits, so **`fast` must be tested first**: `while fast and fast.next`. The ordering is doing real work, not stylistic.

The near-miss: exiting immediately

```
def find_duplicate(nums):
    slow = fast = 0                # both at index 0, no first step
    while slow != fast:           # false immediately!
        ...
```

`slow == fast` is true before anything moves, so the loop never runs. The `while True / break` structure exists precisely to force **one step before the first comparison**. If you prefer a condition-first loop, you must offset the starting positions:

```
slow, fast = nums[0], nums[nums[0]]
while slow != fast:
    slow = nums[slow]
    fast = nums[nums[fast]]
```

Both forms are correct; mixing them is not.

The near-miss: leaving `fast` at double speed in phase two

```
slow = start
while slow != fast:
    slow = step(slow)
    fast = step(step(fast))      # still doubling – WRONG
return slow
```

Returns a point inside the loop that is not the entry, with no error. The derivation in §3 requires both to move one step at a time — that is what makes the fresh index and `slow` meet exactly at the entry.

The near-miss: the even-length middle

```
print(middle_index([1, 2, 3, 4]))    # 2, the value 3
```

Is that right? It depends on the problem. LeetCode 876 wants the second middle; a “split the list in half” problem usually wants the **first**, so that the halves are equal. Changing `fast + 1 <` to `fast + 2 <` gives the first. **This is a contract question, and getting it wrong costs a submission for otherwise perfect code.**

The near-miss: assuming a cycle exists

```
def has_cycle(step, start):
    slow = fast = start
    while True:
        slow = step(slow)
        fast = step(step(fast))
        if slow == fast:
            return True
```

On a structure with an end, this never returns `False` — it runs off the end, or loops forever. For anything finite you need the escape condition. `while True` is only correct for a **total function** where every value maps to another valid value, which is true of happy numbers and of an array where every entry is a valid index, and false of a linked list.

The near-miss: speed 3

```
fast = step(step(step(fast)))
```

The gap now shrinks by 2 per step, so it can go from 1 to -1 and jump over zero. It happens to still work in many cases for other reasons, but the one-line proof is gone and the phase-two arithmetic is wrong. **There is no benefit and a real cost. Use 2.**

The contract corner: what “middle” means for length 1 and 2

`[1]` — the middle is index 0. `[1, 2]` — index 1 under this convention, index 0 under the other. Both are single-line answers and both are easy to get wrong by not thinking about them. Test them.

8 In the interview

How it gets asked

- “Find the middle element in a single pass.” — usually about a linked list, where the single-pass constraint is the whole point.
- “Does this linked list have a cycle?” — LeetCode 141, then “find where it starts”, which is LeetCode 142 and the interesting half.
- “Find the duplicate number without modifying the array and in constant space.” — LeetCode 287. The constraints are the question.
- “Is this a happy number?” — LeetCode 202, where “does it loop?” is the whole problem.

What to say out loud, in the first ninety seconds

1. **Name the technique.** “Two indices at different speeds — Floyd’s, tortoise and hare. Slow moves one step, fast moves two.”
2. **Say what the speeds buy.** “For the middle: when fast reaches the end it has gone twice as far, so slow is halfway. For cycles: on a straight path fast pulls away, but on a loop it comes round and catches slow from behind.”
3. **Give the obvious solution and its cost first.** “The straightforward way to detect a cycle is a hash set of everything visited — $O(n)$ time and $O(n)$ space.”
4. **Then offer the improvement.** “Floyd’s does it in $O(1)$ space, which on ten million nodes is two variables instead of half a gigabyte.”
5. **Give the meeting proof, unprompted.** “They must meet, because once both are in the loop the gap closes by exactly one per step — and since it changes by exactly one it can’t skip over zero.”
6. **Ask the contract question for the middle.** “On an even-length input there are two middles — do you want the first or the second?”
7. **Flag the short-circuit if it is a linked list.** “The condition is `fast and fast.next`, in that order, or it dereferences None on an even-length list.”

The follow-ups

“Why are you sure the fast pointer catches the slow one?” Once both are inside the loop, consider the gap from fast to slow measured the way round the loop. Each step, slow advances one and fast advances two, so the gap decreases by exactly one. It starts at some value smaller than the loop length and decreases by one every step, so it must reach zero — and because it changes by exactly one, it cannot step over zero and miss. So they land on the same point. The bound falls out too: slow enters the loop after μ steps,

and they meet within λ more, so it is at most $\mu + \lambda$ iterations, which is at most n . That is also why the fast pointer moves at two and not three: at three the gap shrinks by two per step, so it can go from 1 to -1 and jump the meeting, and the one-line argument is gone.

“Now find where the cycle starts.” There is a short derivation. Let μ be the distance from the start to the loop entry and λ the loop length. When they meet, slow has taken d steps and fast has taken $2d$, and since both are on the loop, fast has covered a whole number of extra loops — so $2d - d = k\lambda$, meaning d is a multiple of λ . Now reset one index to the start and move **both** one step at a time. After μ steps the reset one is at the loop entry, and the other has advanced μ from a position that was $d - \mu$ into the loop, putting it at d steps in — which is a multiple of λ , so it is at the entry too. They meet exactly at the loop start. The two things people get wrong are resetting the wrong one and leaving the fast pointer at double speed in the second phase; both give a wrong answer with no error.

“Find the duplicate number in an array of $n+1$ values from 1 to n , without modifying it and in $O(1)$ space.” The reframing is the whole answer: treat the array as a path where from index `i` you move to index `nums[i]`. Since every value is between 1 and n , every step lands on a valid index, so the walk never ends — which means it must eventually repeat, so there is a cycle. And because one value appears twice, two different positions point at the same index, and that convergence is exactly where the cycle begins. So the entry point of the cycle is the duplicated value, and Floyd’s finds it in $O(n)$ time and $O(1)$ space. It is worth saying why the alternatives are ruled out: sorting modifies the array, a hash set or a counting array is $O(n)$ space, and marking values negative modifies it. The one other legitimate answer is binary search on the value range — count how many entries are at most mid , and if that exceeds mid the duplicate is below — which is $O(n \log n)$ time and $O(1)$ space.

“How does this apply to a happy number?” Repeatedly replacing a number by the sum of the squares of its digits is just a function applied over and over, so the sequence is a path exactly like the array case. Either it reaches 1 — and 1 maps to itself, so that is a loop of length one — or it enters some other loop. Since the values are bounded — anything under a thousand maps to at most 243 — the sequence cannot escape to infinity, so a loop is guaranteed. Run tortoise and hare, and when they meet, look at what they met on: 1 means happy, anything else means not. That is $O(1)$ space, against the obvious solution of keeping a set of every value seen. The set version is perfectly acceptable here because the reachable set is tiny, so I would mention that the space saving is a demonstration rather than a necessity on this particular problem.

A model answer

“First a contract question, since this is a linked list: on an even-length list there are two middles — do you want the first or the second?

...The second. Fine.

The naive approach is two passes: walk the list to count the nodes, then walk half of it again. That is $O(n)$ and correct, and it needs the list twice.

One pass uses two pointers at different speeds. Slow advances one node, fast advances two. When fast reaches the end it has travelled twice as far, so slow is exactly at the middle.

```
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
return slow
```

The condition has to be `fast and fast.next` in that order. Python's `and` short-circuits, so if `fast` is `None` the second test is never evaluated — reverse them and you get an `AttributeError` on every even-length list. And this particular loop condition is what gives the second middle; stopping one step earlier gives the first.

$O(n)$ time and $O(1)$ space. The reason one pass matters rather than two isn't the constant factor — it's that on a stream, or anything you can only traverse once, there is no second pass available.

The same two speeds answer a different question, which is whether the list has a cycle. On a straight list, fast reaches the end and you stop. If there's a cycle, fast comes round and catches slow from behind, and that meeting is the proof.

The reason it must meet is worth stating: once both pointers are inside the cycle, the gap between them closes by exactly one node per step, because slow moves one and fast moves two. A quantity that decreases by exactly one each step must hit zero, and it can't jump over zero. So they meet within one cycle-length.

The obvious alternative is a hash set of visited nodes — same $O(n)$ time, but $O(n)$ space. On ten million nodes that's roughly half a gigabyte against two variables, and that gap is the entire reason the technique is worth knowing.

If you also want where the cycle starts, there's a second phase: reset one pointer to the head and move both one step at a time. They meet at the cycle entry, and the arithmetic is that when they first met, slow had walked a whole number of cycle-lengths.”

9 Recall card

- **Slow moves 1, fast moves 2.** When fast ends, slow is at the **middle**. If they ever meet, there is a **cycle**.
- **Why they must meet:** once both are in the loop the gap shrinks by **exactly one** per step, so it cannot skip zero. That is why fast moves 2, not 3.
- **Cycle start:** reset one to the beginning, then move **both at speed 1**. They meet at the entry.
- `while fast and fast.next` — that order, or you dereference `None` on an even-length list.
- **An array where every value is a valid index is a path.** A repeated value makes a loop, and the loop entry is the duplicate — `O(n)` time, `O(1)` space, no modification.

1 What this is, and why they ask it

An **index** is a second structure, stored alongside a table, that maps values of one or more columns to the rows holding them — kept in sorted order so it can be searched without reading everything.

Without one, finding the rows where `customer_id = 4217` means reading every row and discarding the ones that do not match. That is a **sequential scan**, and it is $O(n)$. With a B-tree index it is three or four page reads regardless of table size — the difference between 112 milliseconds and 0.02 on a million rows.

The reason this gets a whole day is the second half, which people miss: **an index is a copy, and every write has to update it**. A table with eight indexes does nine writes per insert. So indexing is not free and “index everything” is a real mistake, with a real cost you can quantify.

Interviewers ask “*this query is slow, what do you do?*” in almost every backend interview, and it has a right shape of answer: **measure first with EXPLAIN, then look at the specific thing it tells you**. Candidates who immediately say “add an index” without asking what the query is get marked down, because about half the time the problem is something else — an unindexed foreign key, an ORM issuing sixty-one queries, a function wrapped round the indexed column, or deep `OFFSET` pagination.

2 The story

The spare parts shop where Nadeem works is one room, and along three walls there are wooden cabinets of small drawers — six hundred and something drawers, four inches square, holding everything a two-wheeler mechanic asks for at eight in the morning.

Every drawer has a number painted on the front, and nothing else. Not what is in it. Just a number.

The list is what makes the shop work. It is a long list, ordered by part number, and against each part number it says a drawer. A man comes in with a part number off the old part, Nadeem runs his eye down the list — it is in order, so he lands on the right place in about four seconds — and goes straight to drawer 218.

The alternative is what happened on the day the list went missing, before Nadeem’s time, and the owner still tells the story. They opened drawers. All morning.

There is a second list, and it took the owner two years to agree to it. That one is ordered by the name of the part — clutch cable, brake shoe, chain sprocket — because half the customers do not have a part number, they have a broken thing in their hand and a word for it. Without that list, a man who says “clutch cable” is back to opening drawers, because the first list is in part-number order and a name tells you nothing about where you are in it.

And here is the thing the owner was right about, which is why he resisted.

Every time a new kind of part comes in, Nadeem has to write it into **both** lists, in the right place in each, and it has to be the right place or the list is worse than useless. A delivery of forty new part numbers on a Tuesday is forty entries in one list and forty in the other, and it takes him most of the afternoon. Two lists means twice the afternoon.

There was talk of a third list, by manufacturer. Nadeem was against it. He says he already spends more time keeping the lists right than he does finding parts, and finding parts is the job.

3 The idea in plain English

Nadeem’s part-number list is an **index on the part number column**. The name list is a **second index on a different column**. Opening every drawer is a **sequential scan**. And the Tuesday afternoon is the **write cost**, which is the half people forget.

What an index actually is

A **B-tree** — a shallow, wide, always-balanced tree stored on disk, where each node is one disk page. Each entry holds a value and a pointer to the row that has it.

The critical property is the **fanout**: because a page is 8 KB and an entry is small, one node holds hundreds of entries. Postgres manages around 500 per page for a typical integer key. So:

```
depth 1 :      500 entries
depth 2 :    250,000
depth 3 : 125,000,000
depth 4 : 62,500,000,000
```

Three levels covers ten million rows. Four covers sixty billion. That is why an index lookup is “three or four page reads” rather than a number that grows with your data — the tree gets *wider* before it gets deeper, and depth grows like $\log_{500}(n)$.

Trees properly arrive on [day 098](#). What you need today is: **it is sorted, it is shallow, and a lookup is a handful of reads.**

What an index gives you, beyond equality

Because it is **sorted**, one index serves five different query shapes:

QUERY	USES THE INDEX?
<code>WHERE id = 42</code>	yes — walk down to the value
<code>WHERE age > 30</code>	yes — find 30, then walk right along the leaves
<code>WHERE age BETWEEN 20 AND 30</code>	yes — a range scan
<code>ORDER BY created_at</code>	yes — the index is already in that order, so no sort needed
<code>WHERE name LIKE 'Ram%'</code>	yes — a prefix is a range
<code>WHERE name LIKE '%kumar'</code>	no — a suffix is not a range

That fourth row is worth pausing on. `ORDER BY` on an indexed column costs nothing, because the index is the order. That is often a bigger win than the lookup, since sorting a million rows means either $O(n \log n)$ in memory or a spill to disk.

The rule that catches everyone: don't wrap the column

```
WHERE YEAR(created_at) = 2026      -- index NOT used: sequential scan
WHERE created_at ≥ '2026-01-01'
  AND created_at < '2027-01-01'    -- index used
```

The index stores `created_at`, not `YEAR(created_at)`. Once you apply a function, the database has to compute it for every row to find out which match — so it reads every row. Same for `WHERE LOWER(email) = 'x'` and `WHERE price * 1.18 > 100`.

The fix is always the same: rearrange so the bare column is on one side. Or, when you genuinely need the function, build an index *on the expression*:

```
CREATE INDEX ON users (LOWER(email));
```

This one line accounts for a large share of real-world slow queries, and spotting it in a query someone shows you is a strong interview signal.

Composite indexes and the left-prefix rule

An index can cover several columns, and then **order matters enormously**:

```
CREATE INDEX ON orders (customer_id, created_at);
```

That index is sorted by `customer_id` first, and by `created_at` within each customer. So it serves:

WHERE customer_id = 5	yes
WHERE customer_id = 5 AND created_at > '2026-01-01'	yes — this is what it is FOR
WHERE customer_id = 5 ORDER BY created_at	yes — and no sort step
WHERE created_at > '2026-01-01'	NO — created_at is not the first column

An index on (A, B) can be used for A, and for A with B, but not for B alone. That is the **left-prefix rule**, and it is the most-asked detail in this topic.

It is Nadeem's list exactly. A list ordered by part number, then by name within each number, lets you find a part number instantly and a name within it — and is no help at all if all you have is a name.

An index on (A, B) also makes a separate index on A redundant, which is a useful thing to notice when someone shows you six indexes on one table.

The covering index

If the index contains **every column the query needs**, the database can answer from the index alone and never read the table row at all. Postgres calls this an *index-only scan*.

```
CREATE INDEX ON orders (customer_id, status, amount);

SELECT status, amount FROM orders WHERE customer_id = 5; -- served entirely from the index
```

On a wide table this is often a 5–10× win, because the index is far smaller than the rows. And it is the real argument against `SELECT *` — from [day 027](#), asking for columns you do not need makes a covering index impossible.

Selectivity: when an index is worse than useless

An index on a column with two distinct values — `is_active`, where 95% of rows are `true` — will not be used for `WHERE is_active = true`, and the planner is right to refuse.

Reading 950,000 rows through an index means 950,000 **random** reads scattered across the table. Reading the whole table sequentially is 200 MB in one continuous sweep. From [day 010](#), sequential reads are far faster per byte than random ones, so beyond roughly 5–10% of the table, **the scan wins**.

High selectivity — many distinct values — is what makes an index worth having. Email addresses and user ids: excellent. Booleans and status columns with three values: usually not, on their own.

The exception is a **partial index**, which is often the right answer for exactly that case:

```
CREATE INDEX ON orders (created_at) WHERE status = 'pending';
```

If 1% of orders are pending, this index is 1% of the size, and it is highly selective for the query that matters.

The cost: every write updates every index

This is Nadeem's Tuesday.

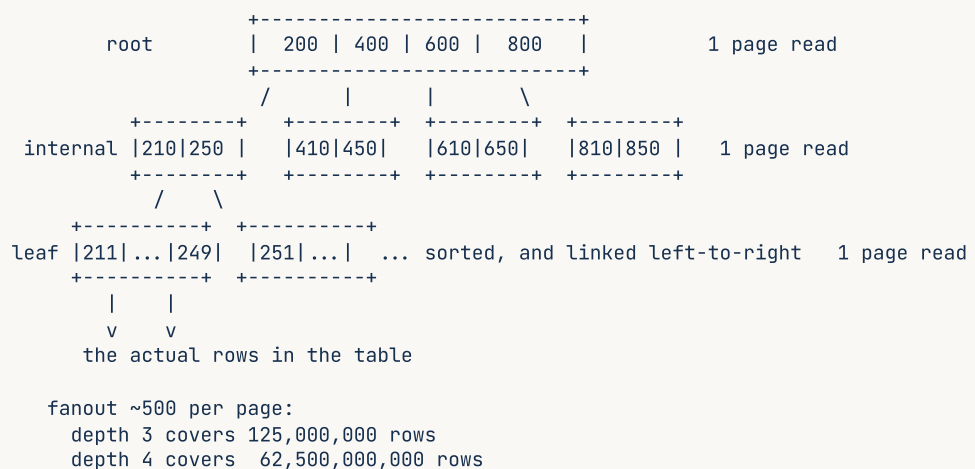
- **INSERT** — one row written, plus one entry inserted into every index on the table.
- **UPDATE** — the row, plus every index on a column you changed. In Postgres, an update writes a new row version, so it often touches **all** the indexes.
- **DELETE** — the row, plus an entry removed from every index.

So a table with eight indexes does roughly nine writes per insert. In practice that is three to five times slower than an unindexed table, plus the storage.

“Index everything” is the mistake this fact prevents.

4 The picture

A B-tree, and why the depth barely grows:



What to notice: the leaves are linked to each other in order. That is what makes **>**, **BETWEEN** and **ORDER BY** work — find the start, then walk sideways. An index is not just a lookup table; it is a **sorted** lookup table, and half its value is the sortedness.

Scan versus index, on a million rows:

SEQUENTIAL SCAN	INDEX SCAN
read every page, discard non-matches	walk down, then read the rows
<pre> [][][][][][][][][][][][][] ... [][][][][][][][][][][][][] ... [][][][][][][][][][][][][] 25,000 pages, 200 MB 112 ms 999,997 rows read and thrown away </pre>	<pre> root internal leaf → 3 matching row pointers 0.02 ms 3 rows read </pre>

The left-prefix rule, drawn as the sort order:

```
index on (customer_id, created_at) - sorted by the first, then the second
```

customer_id	created_at
5	2026-01-04
5	2026-02-11
5	2026-03-02
7	2026-01-09
7	2026-05-20
9	2026-01-01
9	2026-04-15

← all of customer 5 is together,
and in date order within that

WHERE customer_id = 5 → jump straight to the block. YES
WHERE customer_id = 5 AND created_at > X → jump, then walk. YES
WHERE customer_id = 5 ORDER BY created_at → already ordered. YES, no sort
WHERE created_at > '2026-04-01' → the dates are scattered
through the whole index. NO

What to notice: the dates are only sorted *within* a customer. Globally they are all over the place, which is why the index cannot help a query that filters on the date alone.

5 How it actually works

Finding out why a query is slow

Always start here. `EXPLAIN` shows the plan; `EXPLAIN ANALYZE` runs it and shows what actually happened.

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 4217;
```

```
Seq Scan on orders (cost=0.00..18334.00 rows=1 width=64)
    (actual time=0.021..112.4 rows=3 loops=1)
  Filter: (customer_id = 4217)
  Rows Removed by Filter: 999997
```

Four things to read, in order:

- 1. **Seq Scan on a large table with a filter** — the headline problem. It read a million rows to return three.
- 2. **Rows Removed by Filter: 999997** — confirms it, and quantifies how much work was wasted.
- 3. **Estimated rows=1 versus actual rows=3** — close enough. A large gap here means stale statistics, and `ANALYZE orders;` may fix the plan without any schema change.
- 4. **actual time** — where the time really went, as opposed to where the cost model guessed.

Add the index:

```
CREATE INDEX CONCURRENTLY ON orders (customer_id);
```

```
Index Scan using orders_customer_id_idx on orders
(cost=0.42..8.45 rows=3 width=64) (actual time=0.018..0.024 rows=3 loops=1)
Index Cond: (customer_id = 4217)
```

`CONCURRENTLY` matters in production: a plain `CREATE INDEX` takes a lock that blocks writes to the table for the whole build, which on a large table is minutes of downtime. `CONCURRENTLY` is slower and does not block. **Mentioning it unprompted is a strong operational signal.**

The scan types you will see

NODE	MEANS
Seq Scan	read the whole table. Fine on small tables, the thing to fix on large ones.
Index Scan	walk the index, then fetch each matching row.
Index Only Scan	answered entirely from the index — a covering index. The best case.
Bitmap Heap Scan	many matches: collect their locations first, sort them, then read the table in page order. Postgres chooses this between “a few rows” and “most rows”.
Nested Loop / Hash Join / Merge Join	the join algorithms from day 028 .

Bitmap Heap Scan is worth understanding, because people see it and assume something is wrong. It is the planner being clever: rather than making 50,000 random row reads in index order, it gathers the page numbers, sorts them, and reads the table in physical order — turning random I/O into something closer to sequential.

The six causes of a slow query

Have this list. It is the answer to the interview question.

1. **A missing index** — `Seq Scan` with a high “Rows Removed”. Add one.
2. **A function wrapped round the column** — `WHERE YEAR(x) = 2026`. Rewrite as a range, or index the expression.
3. **An unindexed foreign key** — from [day 026](#), primary keys are indexed automatically and foreign keys are not. This shows up as a slow join or a slow delete.
4. **N + 1 queries from an ORM** — the plan looks fine because each individual query is fine; there are just sixty-one of them. You find this in the application log, not in `EXPLAIN`.
5. **Deep OFFSET pagination** — `OFFSET 1000000` generates and discards a million rows. Use a keyset cursor.
6. **A sort or hash aggregate spilling to disk** — `Sort Method: external merge Disk: 52000kB` in the plan. Either raise `work_mem` or add an index that provides the order.

Index types beyond B-tree

Know that they exist and roughly what each is for:

- **B-tree** — the default. Equality, ranges, ordering, prefixes. Covers 95% of needs.
- **Hash** — equality only, no ranges. Rarely worth it in Postgres, since B-tree is nearly as fast for equality and does far more.
- **GIN** — for columns containing many values: full-text search, `JSONB`, arrays. `WHERE tags @> '{sql}'`.
- **GiST** — geometric and range types. “Points within 5 km of here.”
- **BRIN** — tiny indexes for huge tables where the data is physically ordered, such as append-only time-series. Stores a summary per block rather than an entry per row.

Writing indexes for a real workload

The practical procedure:

1. **Find the slow queries.** `pg_stat_statements` ranks them by total time — which surfaces the query that takes 20 ms and runs a million times a day, and that is usually the real problem rather than the 2-second report that runs nightly.
 2. `EXPLAIN ANALYZE` each one.
 3. **Index the columns in** `WHERE`, `JOIN` and `ORDER BY`, most selective first in a composite.
 4. **Check for redundancy.** An index on `(A, B)` makes one on `(A)` unnecessary.
 5. **Find unused indexes** — `pg_stat_user_indexes` shows scan counts; an index with zero scans is pure write cost, and dropping it is free speed.
-

6 The numbers

Scan against index lookup

1,000,000 orders at about 200 bytes:

```
table size                = 200 MB = ~25,000 pages of 8 KB
sequential scan @ 500 MB/s  ≈ 400 ms of I/O, ~112 ms when cached
index scan: 3 B-tree pages + 3 row pages = 6 page reads ≈ 0.02 ms
```

About 5,000×. And it gets better with size: at 10 million rows the scan is ten times slower and the index scan is unchanged, because the tree gained no depth.

Depth against table size

fanout ~500 per 8 KB page:

```
1,000 rows → 2 levels
250,000 rows → 2 levels
10,000,000 rows → 3 levels
100,000,000 rows → 4 levels
```

A thousand-fold increase in data costs you one extra page read. That is the single most persuasive fact about B-trees.

The write cost

```
table with 0 indexes : 1 write per insert
table with 1 index   : 2 writes
table with 8 indexes : 9 writes
```

measured effect: roughly 3-5x slower inserts with 8 indexes than with none

At 1,000 inserts a second:

```
0 indexes : 1,000 writes/s
8 indexes : 9,000 writes/s of index maintenance
```

Storage

```
1,000,000 rows, index on a bigint column:
~16 bytes per entry (key + pointer + overhead)
1,000,000 × 16 B ≈ 16 MB, plus tree overhead ≈ 25 MB
```

```
table itself : 200 MB
one index    : 25 MB   (12%)
eight indexes: 200 MB  (100% – the indexes now cost as much as the data)
```

Selectivity: where the scan wins

```
1,000,000 rows, matching:
    10 rows (0.001%) : index scan, 10 random reads      - obviously index
    10,000 rows  (1%)  : index or bitmap scan           - index
    100,000 rows (10%) : bitmap heap scan               - borderline
    500,000 rows (50%) : sequential scan                 - obviously scan
```

The crossover is roughly **5–10%** of the table, and the reason is that index scans produce random reads while a sequential scan is one continuous sweep. From [day 010](#), the ratio between random and sequential access is what drives the whole decision.

The covering index

```
orders row : 200 bytes
index on (customer_id, status, amount) : ~32 bytes per entry

query needing only those three columns, 10,000 matching rows:
    index scan + row fetch : 10,000 random row reads ≈ 2,000,000 bytes touched
    index only scan       : ~320,000 bytes, sequential within the index
                           about 6x less I/O, and no random access
```

OFFSET pagination, again

```
LIMIT 20 OFFSET 0          ≈ 1 ms
LIMIT 20 OFFSET 1,000,000 ≈ 2,000 ms - a million rows generated and discarded
keyset: WHERE id > :last ORDER BY id LIMIT 20 ≈ 1 ms at any depth
```

7 The trade-offs

Read speed against write speed and storage

The whole subject in one line. Each index makes some reads dramatically faster and every write on that table slower, and costs 10–25% of the table's size.

Index the columns your real queries filter, join and sort on. Then measure, and drop what is not used. `pg_stat_user_indexes` tells you which have never been scanned, and dropping those is free speed with no downside.

Composite or several single-column indexes

A composite `(A, B)` is much better than separate indexes on `A` and `B` for a query filtering on both, because it goes straight to the matching block instead of combining two result sets. It is also useless for a query on `B` alone.

Rule: build composites for the query shapes you actually run, most selective column first, and remember that `(A, B)` already covers `A`.

Selective or not

A boolean or a three-value status column is a poor index on its own — the planner will ignore it and be right. **A partial index is usually the better tool:** index only the rows matching a condition, which is both small and highly selective for the query that matters.

When to add the index

Adding an index to a live table takes a lock unless you use `CONCURRENTLY`, and on a large table a plain `CREATE INDEX` can block writes for minutes. `CONCURRENTLY` is slower and safe. **Say this — it is the difference between someone who has added an index in production and someone who has read about it.**

Index, or fix the query

Often the real answer is not an index at all: a function wrapped round the column, an ORM issuing sixty-one queries, `SELECT *` defeating a covering index, or `OFFSET 1000000`. **Check those before adding anything**, because an index is permanent write cost and a query fix is free.

The sentence that separates candidates

I would not add an index before running `EXPLAIN ANALYZE`. Roughly half the slow queries I have seen were not missing an index — they had a function wrapped round the indexed column, or an ORM issuing sixty-one queries where one would do, or a deep `OFFSET` discarding a million rows. Those are free to fix and an index is not: it is 10–25% of the table in storage and a permanent tax on every insert, update and delete. And when I do add one on a live table, it is `CREATE INDEX CONCURRENTLY`, because the plain form holds a lock that blocks writes for the length of the build.

8 In the interview

How it gets asked

- “*This query is slow. What do you do?*” — the main event. The first word of the answer should be “measure”.
- “*How does an index actually work?*” — B-tree, fanout, three levels for ten million rows.
- “*What’s the downside of adding an index?*” — write cost and storage, quantified.
- “*You have an index on `(a, b)`. Does it help `WHERE b = 5`?*” — the left-prefix rule, and the answer is no.

- “Why isn’t my index being used?” — a function on the column, low selectivity, or stale statistics.

What to say out loud, in the first ninety seconds

1. **Measure before guessing.** “First I’d run `EXPLAIN ANALYZE` on it, because ‘slow’ has about six different causes and they need different fixes.”
2. **Say what you are looking for.** “A `Seq Scan` on a big table with a high ‘Rows Removed by Filter’ means a missing index. A big gap between estimated and actual rows means stale statistics. A sort spilling to disk means the grouping exceeded `work_mem`.”
3. **Name the causes that are not missing indexes.** “I’d also check for a function wrapped around the column, which stops any index being used; an unindexed foreign key; an ORM doing $N + 1$; and deep `OFFSET` pagination.”
4. **Explain the index if asked.** “It’s a B-tree — sorted, and shallow because the fanout is about 500 entries per page, so three levels covers ten million rows. A lookup is three or four page reads instead of scanning 200 MB.”
5. **Say what else sortedness buys.** “It also serves ranges, `BETWEEN`, prefix `LIKE`, and `ORDER BY` with no sort step — which is often the bigger win.”
6. **Give the cost, unprompted.** “The price is that every insert, update and delete has to maintain every index — eight indexes means nine writes per insert — plus 10 to 25% of the table in storage.”
7. **Mention `CONCURRENTLY`.** “And on a live table I’d use `CREATE INDEX CONCURRENTLY`, because the plain form locks out writes for the whole build.”

The follow-ups

“You have an index on `(customer_id, created_at)`. Does it help `WHERE created_at > '2026-01-01'`?” No, and the reason is the left-prefix rule. The index is sorted by `customer_id` first and by `created_at` only within each customer, so globally the dates are scattered all through it — there is no contiguous block of “everything after January”. The index can serve `customer_id` alone, or `customer_id` together with `created_at`, or `customer_id` with an `ORDER BY created_at` and no sort step, but never `created_at` on its own. For that query I would need a separate index with `created_at` first. Worth adding: the composite already covers any query on `customer_id` alone, so a separate index on just `customer_id` would be redundant, and I would drop it if it existed.

“Why isn’t my index being used?” Four common reasons and I would check them in this order. First, a function or expression wrapped around the column — `WHERE YEAR(created_at) = 2026` or `WHERE LOWER(email) = ...` — because the index stores the raw column, not the computed value, so the database must compute it per row. The fix is to rewrite as a range, or create an index on the expression itself. Second, low selectivity: if the condition matches more than roughly 5 to 10 per cent of the table, the

planner deliberately prefers a sequential scan, because an index scan there means hundreds of thousands of random reads while a scan is one continuous sweep — and it is right to. Third, stale statistics: if the planner thinks a condition matches one row and it actually matches half the table, it will choose badly, and `ANALYZE` fixes it. Fourth, a type mismatch — comparing a `bigint` column to a string literal can prevent the index being used in some databases.

“What’s the actual cost of adding an index?” Three costs. Storage: roughly 10 to 25 per cent of the table per index, so eight indexes on a 200 MB table is another 200 MB. Write throughput: every insert adds an entry to every index, so eight indexes means nine writes per insert, and in practice inserts run three to five times slower than on an unindexed table. Updates are worse in Postgres, because an update writes a new row version and often has to touch every index rather than only the changed columns. And there is an operational cost: building it on a live table takes a lock that blocks writes unless you use `CONCURRENTLY`. Against all that, the read win can be several thousand-fold, which is why the answer is to index the columns your real queries actually use and then check `pg_stat_user_indexes` for ones that have never been scanned — those are pure cost.

“The query does `SELECT * FROM orders WHERE status = 'pending'` and status has three values.” A plain index on `status` will probably be ignored, and correctly so if pending is a large share of the table. But if pending is a small share — which it usually is, since orders move out of that state — a **partial index** is the right tool: `CREATE INDEX ON orders (created_at) WHERE status = 'pending'`. It only contains the pending rows, so it is perhaps one per cent of the size, it is cheap to maintain because most inserts do not qualify, and it is highly selective for exactly the query that matters. I would also change the `SELECT *`: naming the columns lets the index cover the query, so it can be answered without touching the table rows at all, which on a wide table is another five to ten times less I/O.

A model answer

“Before changing anything I’d run `EXPLAIN ANALYZE` on the query, because ‘slow’ has several different causes and they want different fixes.

What I’m reading in the plan: is there a `Seq Scan` on a large table, and what does ‘Rows Removed by Filter’ say? If it read a million rows to return three, that’s a missing index and it’s the headline problem. I’d also compare the estimated row count with the actual — a big gap means the statistics are stale and `ANALYZE` might fix the plan by itself. And I’d look for a sort or hash aggregate spilling to disk, which means the operation exceeded `work_mem`.

But I’d check three things that aren’t missing indexes before adding one, because they’re free to fix. Is there a function wrapped round the column — `WHERE YEAR(created_at) = 2026` — which stops any index being used, since the index stores the raw value? Is this actually one slow query, or is an ORM issuing sixty-one of them, which `EXPLAIN` won’t show me? And is it deep `OFFSET` pagination, where `OFFSET 1000000` generates and throws away a million rows?

If it really is a missing index: an index is a B-tree, so it’s sorted and it’s shallow. The fanout is around 500 entries per 8 KB page, which means three levels covers ten million rows and four covers sixty billion — so a lookup is three or four page reads whatever the table size, against reading 200 MB. That’s roughly five thousand times on a million rows.

Because it’s sorted, the same index also serves ranges, `BETWEEN`, prefix `LIKE`, and `ORDER BY` with no sort step at all — and that last one is often the bigger win, since sorting a million rows otherwise means a spill to disk.

The cost is real and I’d state it: every insert, update and delete has to maintain every index, so eight indexes means nine writes per insert and inserts three to five times slower, plus 10 to 25 per cent of the table in storage each. So I’d index what the real queries filter, join and sort on, and then check `pg_stat_user_indexes` for indexes that have never been scanned — those are pure write cost and dropping them is free speed.

And on a live table I’d use `CREATE INDEX CONCURRENTLY`. The plain form takes a lock that blocks writes for the entire build, which on a large table is minutes of downtime.”

9 Recall card

- **An index is a sorted B-tree kept alongside the table.** Fanout ~500, so **three levels covers ten million rows** — a lookup is 3–4 page reads, not a 200 MB scan.
- **Sortedness buys ranges**, `BETWEEN`, prefix `LIKE` and free `ORDER BY` — often the bigger win.
- **Left-prefix rule:** an index on `(A, B)` serves `A` and `A+B`, never `B` alone.
- **Every write updates every index.** Eight indexes = nine writes per insert, and 10–25% storage each.
- **Measure first.** `EXPLAIN ANALYZE`, and check for a function on the column, an `N+1`, or deep `OFFSET` before adding anything. On a live table, `CREATE INDEX CONCURRENTLY`.

DSA topic: Fast and slow pointers **System design topic:** Indexes: how a database finds a row fast

Code these, in this order

Four problems that are the same six lines in four costumes. **Before each one, say which costume it is and what the “path” is** — what plays the role of “the next node”.

Before each one, ask:

1. What is the step function? From here, where do I go next?
2. Can the path end, or does it go forever? That decides the loop condition.
3. Do I need only *whether* there is a loop, or *where* it starts?
4. What is the naive solution and what does it cost in space?

#	PROBLEM	SOURCE	THE COSTUME, AND WHAT IT IS REALLY TESTING
1	Middle of the Linked List	LeetCode 876 (Easy)	Two speeds, one pass. The condition order <code>fast and fast.next</code> , and which of the two middles.
2	Linked List Cycle	LeetCode 141 (Easy)	Detection. Give the hash-set answer with its space cost, then Floyd's.
3	Happy Number	LeetCode 202 (Easy)	The path is a function on integers. Note what the meeting value tells you.
4	Find the Duplicate Number	LeetCode 287 (Medium)	The path is <code>i → nums[i]</code> . The three constraints rule out every obvious approach, and the loop entry <i>is</i> the answer.

On problem 1, decide the convention first

Run your solution on `[1]`, `[1,2]`, `[1,2,3]`, `[1,2,3,4]`. Then answer:

1. Which of the two middles does your loop return on even lengths?
2. What single change gives the other one?
3. Which does LeetCode 876 want?
4. If you were splitting a list into two halves for a merge sort, which would you want and why?

Then deliberately reverse the condition to `while fast.next and fast` and run it on `[1,2]`. Read the exact error and say why the order matters.

On problem 2, write both and compare

Write the hash-set version first. State its cost out loud: $O(n)$ time, $O(n)$ space. Then write Floyd's and state its cost.

Then produce the number that justifies it: at 10 million nodes, how much memory does each use? That comparison is the reason the technique exists, and it is what you say in the interview.

On problem 4, work out why the obvious answers are banned

The problem says: $n+1$ values in the range $1..n$, one repeated, **do not modify the array**, use **constant space**. For each of these, say which constraint rules it out:

1. Sort, then scan for adjacent equals.
2. A hash set of values seen.
3. A counting array of size $n+1$.
4. Negate `nums[abs(x)]` as a marker.
5. Binary search on the value range, counting how many are $\leq \text{mid}$.
6. Floyd's.

Number 5 is not ruled out — say its cost and why it is a legitimate second answer.

Then explain, in two sentences, why a repeated value creates a loop and why the loop's entry point is the duplicate.

The proof drill

Say each of these out loud, without looking:

1. Why must the fast pointer catch the slow one if there is a loop? (The answer is about a quantity that changes by exactly one.)
2. Why does the fast pointer move at 2 and not 3?
3. In the second phase, why do **both** pointers move at speed 1?
4. Why, when they first meet, has the slow pointer walked a whole number of loop-lengths?
5. What is the total number of steps, in terms of μ and λ ?

Number 1 is the one you will actually be asked. Number 4 is the one that makes number 3 make sense.

The break-it drill

Run each and say what goes wrong:

```
# A
def find_duplicate(nums):
    slow = fast = 0
    while slow != fast:           # condition checked before any step
        slow = nums[slow]
        fast = nums[nums[fast]]
    return slow
```

```
# B (phase two)
slow = start
while slow != fast:
    slow = step(slow)
    fast = step(step(fast))      # still doubling
return slow
```

```
# C
while fast.next and fast:
    ...
```

A exits immediately. B returns a point inside the loop that is not the entry, with no error. C raises. Say which of the three is most dangerous and why.

The costume drill

For each, say what the “path” is — what the step function does — and whether the path can end:

1. A linked list with a cycle.
2. `nums[i]` where every value is a valid index.
3. Sum of the squares of the digits.
4. A function that maps each user to the person who referred them.
5. Repeatedly halving a number and rounding down.

Number 5 is the one where there is always a loop, and a boring one. Say what it is.

The middle-versus-length drill

Answer without running anything:

1. Two passes — count, then walk half — versus one pass with two speeds. Same complexity. Give the real reason to prefer one pass.
2. On an array, why would you ever use this instead of `len(items) // 2`?
3. What structure makes this technique not merely nicer but necessary?

The index drill

Answer each in one or two sentences, out loud:

1. What is an index, physically, and why is it shallow?
2. Fanout of 500 — how many levels for ten million rows? For sixty billion?
3. Name five query shapes an index on one column can serve. Name one it cannot.
4. Why does `WHERE YEAR(created_at) = 2026` not use an index on `created_at`? Rewrite it.
5. An index on `(a, b)` — which of `a`, `b`, and `a AND b` does it serve?
6. What is a covering index, and what does `SELECT *` do to it?
7. At roughly what fraction of the table does a sequential scan beat an index scan, and why?
8. What does every extra index cost, in writes and in storage?
9. What does `CREATE INDEX CONCURRENTLY` buy you, and what does it cost?
10. Name three reasons a query is slow that are **not** a missing index.

Number 10 is the one that makes the difference in an interview.

The plan-reading drill

For each plan fragment, say the diagnosis and the fix:

```
Seq Scan on orders (actual time=0.021..112.4 rows=3 loops=1)
  Filter: (customer_id = 4217)
  Rows Removed by Filter: 999997
```

```
Seq Scan on users (cost=... rows=1 width=...) (actual rows=480000 loops=1)
```

```
Sort (actual time=...) Sort Method: external merge Disk: 52000kB
```

```
Index Scan using orders_pkey on orders (actual rows=1 loops=61)
```

The last one has `loops=61`. Say what that almost certainly means and where you would look for it.

The arithmetic drill

From memory, in under two minutes:

- 1,000,000 rows at 200 bytes — table size, pages, and time for a sequential scan.
- The same lookup through a three-level B-tree — page reads and time. The ratio.
- Fanout 500 — rows covered at depths 2, 3 and 4.
- A table with eight indexes — writes per insert, and the rough slowdown.

- One index on a bigint column over a million rows — its size, and as a percentage of the table.
- The selectivity crossover — what fraction, and why random beats sequential below it.

The design drill

You are given this table and these three queries. Decide the minimum set of indexes, and justify each.

```
CREATE TABLE orders (
  id          BIGSERIAL PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES customers(id),
  status      TEXT  NOT NULL,          -- 'pending', 'shipped', 'delivered', 'cancelled'
  total       NUMERIC(12,2) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL
);
```

1. `SELECT * FROM orders WHERE customer_id = ? ORDER BY created_at DESC LIMIT 20;`
2. `SELECT id, total FROM orders WHERE status = 'pending' ORDER BY created_at;`
3. `SELECT customer_id, SUM(total) FROM orders WHERE created_at ≥ ? GROUP BY customer_id;`

Then answer: which of your indexes is redundant with another? Which query would benefit from a **partial** index, and why? And which one could be made an index-only scan, and what would you have to change?

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the middle element in a single pass.* Ask which middle. Give the two-pass version and its cost. Then two speeds, and the reason one pass matters — not the constant factor, but that a stream has no second pass. Name the `fast` and `fast.next` ordering.
2. *This query is slow. How do you find out why, and what do you do about it?* Measure first. Say what you read in the plan and in what order. Then the six causes, at least three of which are not missing indexes. Then the B-tree explanation with the fanout number, then the cost of the index, then `CONCURRENTLY`.
3. *Why are you sure the fast pointer catches the slow one?* The gap shrinks by exactly one per step, so it cannot skip zero. Then the bound, $\mu + \lambda$. Then why speed 3 breaks the argument. Ninety seconds.

Before you move on

- [] I can state, in one sentence, why the two pointers must meet.
- [] I write `while fast and fast.next` in that order without thinking.
- [] I ask which of the two middles before writing the loop.
- [] I give the hash-set solution and its space cost before offering Floyd's.
- [] I can explain why a repeated value in an index-valued array creates a loop.
- [] In phase two I reset one pointer and move **both** at speed 1.
- [] I say “`EXPLAIN ANALYZE` first” before “add an index”.
- [] I can name three causes of a slow query that are not a missing index.
- [] I know the left-prefix rule and can say why `(a, b)` does not serve `b`.
- [] I can quantify the write cost of an index, and I mention `CONCURRENTLY` unprompted.
- [] I can redraw the rho diagram and the B-tree fanout diagram from memory, in whatever tool I like.

Fixed-size sliding window

DATA STRUCTURES AND ALGORITHMS

Fixed-size sliding window

You can answer every window-of-size- k question in $O(n)$ instead of $O(n*k)$.

SYSTEM DESIGN

B-trees and why indexes are shaped that way

You can draw a B-tree and explain why the database does not use a binary tree on disk.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- Find the maximum sum of any subarray of size k .
- Why is a database index a B-tree and not a binary search tree?

1 What this is, and why they ask it

A **sliding window** is a range of fixed width k that moves along the array one position at a time. The whole technique is one observation:

Consecutive windows overlap in $k - 1$ elements. So do not recompute — adjust.

When the window moves one step right, exactly one element enters on the right and exactly one leaves on the left. Everything else is unchanged. So instead of adding up k numbers again, you add the entering one and subtract the leaving one: two operations instead of k .

That turns $O(n \times k)$ into $O(n)$, and the saving grows with k . On 100,000 elements with $k = 1,000$, the brute force does about 100 million additions and the window does 200,000.

This is the same family as the two-pointer work of the last four days — the window is two indices moving in the same direction — but the mental model is different enough to be worth its own day: you think about *the range between them* rather than about the two indices. That reframing is what makes tomorrow's variable-size windows possible.

Interviewers ask it because the brute force is obvious and the improvement is a genuine insight, and because the follow-ups are excellent. “Now find the maximum in each window” is not a small change — a sum can be adjusted by arithmetic and a maximum cannot, and the answer needs a data structure. That follow-up, LeetCode 239, is a hard problem sitting one sentence away from an easy one.

2 The story

The Sunday match on the ground behind the water tank is a serious business, and Prasad has kept the score for it for six years. He sits on a plastic chair under the one tree with the book on his knee and he does not miss a ball.

The thing people ask him, every single week, is what the run rate has been over the last five overs. It matters — it is how the captain decides whether to push or hold — and somebody shouts it up to him about twice an hour.

For the first season he did it the obvious way. Somebody asked, and he added the last five overs and divided. Eight, four, eleven, six, nine. Thirty-eight. Seven point six. It took maybe twenty seconds, and he was doing it from scratch every single time, so by the fortieth over he had added up the same numbers over and over.

What he does now takes him about two seconds and he does it without stopping watching the game.

He carries one number in his head: the total for the last five overs. When an over finishes, he adds that over's runs to the number, and then he takes away the runs from the over that has just dropped out of the five — the one that is now six back. Add one, take away one, and the number in his head is correct again.

Somebody asks, he divides by five, done.

The part he had to get right, and got wrong twice in his first month, is *which* over drops out. After the twelfth over finishes, the last five are eight, nine, ten, eleven and twelve — so the one leaving is the seventh, not the eighth. He counts back five from the one that just went in. He says once you have made that mistake in front of forty people you do not make it again.

There is one question he still cannot do this way, and it annoys him. Sometimes they ask for the **biggest** over in the last five, not the total. Adding and subtracting does not work for that — if the over that drops out happened to be the biggest one, the new biggest could be any of the other four, and he has to look at them. So for that one he still looks.

3 The idea in plain English

Prasad's number is the window sum. Adding the new over and subtracting the one that dropped out is the slide. And the question he cannot answer that way is §5's hard follow-up, which needs a structure rather than arithmetic.

The core move

```
total = sum(nums[:k])           # build the first window
best = total
for i in range(k, len(nums)):
    total += nums[i] - nums[i - k]    # add entering, remove leaving
    best = max(best, total)
return best
```

Six lines. The whole technique is the one arithmetic line in the middle.

The index that leaves — the part people get wrong

When `nums[i]` enters, `nums[i - k]` leaves.

Work it out on a small case rather than remembering it. With `k = 3` and the window currently covering indices 0, 1, 2: bringing in index 3 must push out index 0, and `3 - 3 = 0`. Correct. Now index 4 enters and index 1 leaves: `4 - 3 = 1`. Correct.

That is Prasad counting back five from the over that just went in. `i - k`, not `i - k + 1` and not `i - k - 1`.

After the line runs, the window covers `nums[i - k + 1 .. i]`, which is `k` elements. **State that range out loud when you write the loop** — it is the fastest way to check yourself.

Why the loop starts at `k`

The first `k` elements form the first window, and it is built before the loop by `sum(nums[:k])`. The loop then produces windows 2, 3, 4 and so on, so it starts at index `k`. Starting at 0 or 1 either double-counts or produces a window of the wrong size, and both are silent.

Why it is `O(n)`

Each element enters the window exactly once and leaves exactly once. So across the whole run there are `n` additions and `n` subtractions — `2n` operations regardless of `k`.

The brute force is `O(n * k)` because it recomputes all `k` elements at each of the roughly `n` positions, and it re-adds each overlapping element `k` times.

Say it as: *“the windows overlap in `k - 1` elements, so recomputing throws away almost all the work you just did.”*

What can be maintained incrementally, and what cannot

This is the real content of the day, and it decides which follow-up you can answer.

QUANTITY	MAINTAINABLE IN <code>O(1)</code> PER SLIDE?	WHY
sum	yes	add and subtract are exact inverses
average	yes	it is the sum $\div$ <code>k</code>
count of a value	yes	increment and decrement a counter
set of distinct values	yes, with a count map	decrement, and delete the key at zero
whether a duplicate exists	yes, with a set	add entering, discard leaving
maximum / minimum	no	removing the maximum tells you nothing about the next one

Subtraction only works when the operation has an inverse. Addition does; `max` does not. If the element leaving was the maximum, the new maximum could be any of the remaining `k - 1`, and you have thrown away the information needed to know which. That is exactly Prasad's complaint.

The maximum, and the monotonic deque

The fix for `max` is to keep a **deque** — a double-ended queue, from [day 074](#) — holding **indices**, arranged so that their values are decreasing from front to back.

```
while dq and nums[dq[-1]] ≤ x:
    dq.pop()          # anything smaller than x can never be the max again
dq.append(i)
if dq[0] ≤ i - k:
    dq.popleft()      # the front has fallen out of the window
```

Two rules, and both have a one-line reason.

Pop from the back while the back is smaller than the entering value. Those elements are both older *and* smaller than `x`, so `x` outlives them and beats them — they can never be the maximum of any future window. Discard them for good.

Pop from the front when it has fallen out of the window. The front holds the largest value; if its index is `≤ i - k`, it is no longer inside.

Then `nums[dq[0]]` is the maximum of the current window, always, because the deque holds exactly the elements that could still be a maximum, in decreasing order.

Each index is pushed once and popped at most once, so the total work is $O(n)$ even though there is a `while` inside a `for`. **That is the argument to give**, and it is the same “count the travel, not the nesting” reasoning as [day 023](#).

The counting variants

When the window's content matters rather than its sum, carry a `Counter` and update two entries:

```
window[s[i]] += 1          # entering
left = s[i - k]
window[left] -= 1          # leaving
if window[left] == 0:
    del window[left]        # delete at zero, or the comparison fails
```

The `del` is not optional. `Counter({'a': 1, 'b': 0})` does not equal `Counter({'a': 1})`, so leaving zero-count keys behind makes every comparison fail. This is the single most common bug in window-plus-map problems, and it is why *Find All Anagrams* — LeetCode 438 — catches people.

4 The picture

The window sliding, and what changes:

```
nums    2    1    5    1    3    2    k = 3
+-----+-----+-----+-----+
|  2  |  1  |  5  |  1  |  3  |  2  |
+-----+-----+-----+-----+
|<----- 8 ----->|          first window: 2+1+5 = 8

          |<----- 7 ----->|          +1 (enters), -2 (leaves) → 8 + 1 - 2 = 7
                        ^           ^
                    leaves    enters

          |<----- 9 ----->|          +3, -1 → 7 + 3 - 1 = 9

          |<----- 6 ----->|          +2, -5 → 9 + 2 - 5 = 6

best = 9
Each step: TWO operations, not three. With k = 1000: two, not a thousand.
```

What to notice: the two shaded elements at each step are all that changed. Everything in the overlap was already counted and is left alone — that is the entire idea.

The index bookkeeping, which is where the bugs are:

```
i enters    →    i - k leaves    window afterwards is [i-k+1 .. i]

k = 3:
i = 3 enters,  0 leaves    window = [1, 2, 3]
i = 4 enters,  1 leaves    window = [2, 3, 4]
i = 5 enters,  2 leaves    window = [3, 4, 5]
                        ^^^^^^^^^ always exactly k = 3 wide
```

The monotonic deque, on `[1, 3, -1, -3, 5, 3, 6, 7]` with `k = 3`:

```
i=0 x=1 deque empty      push 0      dq: [0]      values (1)
i=1 x=3 back is 1 ≤ 3    pop 0, push 1 dq: [1]      values (3)
i=2 x=-1 back is 3 > -1  push 2      dq: [1,2]    values (3,-1) → max 3
i=3 x=-3 back is -1 > -3 push 3      dq: [1,2,3]  values (3,-1,-3)→ max 3
i=4 x=5  pop 3, pop 2, pop 1      dq: [4]      values (5) → max 5
i=5 x=3  back is 5 > 3    push 5      dq: [4,5]    values (5,3) → max 5
i=6 x=6  pop 5, pop 4      push 6      dq: [6]      values (6) → max 6
i=7 x=7  pop 6, push 7     dq: [7]      values (6,7) → max 7

answers: [3, 3, 5, 5, 6, 7]
```

What to notice at `i = 4`: three indices are popped at once, and that looks expensive. It is not, because each of those was pushed exactly once and can only be popped once — across the whole run there are `n` pushes and at most `n` pops. **Count the pushes, not the loop nesting.**

And notice what the deque holds: at any moment, exactly the elements that are still *candidates* to be a future maximum. Anything both older and smaller than something else is gone for good.

5 The code, built step by step

Building the first window

```
if k ≤ 0 or len(nums) < k:
    return None
total = sum(nums[:k])
best = total
```

The guard first — a window bigger than the array has no answer, and asking for one is a real test case. `sum(nums[:k])` builds window one; `best` starts as that rather than as `0`, for the same reason trackers never start at zero on [day 014](#): the array may be all negative.

The slide

```
for i in range(k, len(nums)):
    total += nums[i] - nums[i - k]
    best = max(best, total)
return best
```

`range(k, ...)` because windows 2 onwards start when index `k` enters. The single arithmetic line is the technique. Writing it as `total += nums[i] - nums[i - k]` rather than two statements makes the “one in, one out” symmetry visible, which helps you check it.

Maximum average — the same thing with a division at the end

```
return best / k
```

Divide **once, at the end**, not inside the loop. Dividing each window introduces floating-point error `n` times and buys nothing, because dividing every candidate by the same positive constant does not change which is largest.

Maximum of each window: the deque version

```
from collections import deque

dq: deque[int] = deque()      # holds INDICES, values decreasing front to back
out: list[int] = []
```

Indices and not values, because you need to know when something falls out of the window, and only the index tells you that.

```
for i, x in enumerate(nums):
    while dq and nums[dq[-1]] ≤ x:
        dq.pop()
    dq.append(i)
```

Anything at the back that is smaller than or equal to the entering value is finished — it is older and smaller, so `x` beats it in every window they could both be in. `≤` rather than `<` also discards equal values, which is fine and keeps the deque smaller.

```
if dq[0] ≤ i - k:
    dq.popleft()
if i ≥ k - 1:
    out.append(nums[dq[0]])
```

The front is the maximum. Remove it if it has aged out; record an answer once the first full window exists, which is from index `k - 1`.

Duplicate within distance k

```
seen: set[int] = set()
for i, x in enumerate(nums):
    if i > k:
        seen.discard(nums[i - k - 1])
    if x in seen:
        return True
    seen.add(x)
return False
```

Here the window is `k + 1` wide, because “within distance `k`” means indices `i - k` to `i` inclusive — so the element leaving is at `i - k - 1`. **Read the problem’s definition of the window before choosing the index**; LeetCode 219 is off by one from the sum problems for exactly this reason.

`discard` rather than `remove`, because `remove` raises when the element is absent and duplicates in the array make that possible.

Anagram windows: window plus a Counter

```
need = Counter(p)
window = Counter(s[:len(p)])
out = [0] if window == need else []

for i in range(len(p), len(s)):
    window[s[i]] += 1
    left = s[i - len(p)]
    window[left] -= 1
    if window[left] == 0:
        del window[left]
    if window == need:
        out.append(i - len(p) + 1)
```

The `del` at zero is mandatory, as above. And the recorded index is `i - len(p) + 1`, the **start** of the window, not `i`.

Comparing two `Counter` s of at most 26 keys is constant work, so this stays `O(n)`.

The complete solutions

```
from collections import Counter, deque

def max_sum_of_size_k(nums: list[int], k: int) → int | None:
    """Maximum sum of any window of exactly k elements. O(n) time, O(1) space."""
    if k ≤ 0 or len(nums) < k:
        return None
    total = sum(nums[:k])          # window 1
    best = total                  # never start at 0 - values may be negative
    for i in range(k, len(nums)):
        total += nums[i] - nums[i - k] # i enters, i-k leaves
        best = max(best, total)
    return best

def max_average_of_size_k(nums: list[int], k: int) → float:
    """LeetCode 643. Divide once, at the end."""
    total = sum(nums[:k])
    best = total
    for i in range(k, len(nums)):
        total += nums[i] - nums[i - k]
        best = max(best, total)
    return best / k

def max_sliding_window(nums: list[int], k: int) → list[int]:
    """LeetCode 239. Maximum of every window. O(n) time, O(k) space.

    The deque holds INDICES whose values decrease from front to back -
    exactly the elements that could still be a future maximum.
    """
    dq: deque[int] = deque()
    out: list[int] = []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] ≤ x:
            dq.pop()          # older AND smaller: can never win again
        dq.append(i)
        if dq[0] ≤ i - k:
            dq.popleft()      # the front has aged out of the window
        if i ≥ k - 1:
            out.append(nums[dq[0]]) # the front is the window maximum
    return out

def contains_nearby_duplicate(nums: list[int], k: int) → bool:
    """LeetCode 219. A duplicate within index distance k. Window is k+1 wide."""
    seen: set[int] = set()
    for i, x in enumerate(nums):
        if i > k:
            seen.discard(nums[i - k - 1]) # note: k+1 wide, so i-k-1 leaves
        if x in seen:
            return True
        seen.add(x)
    return False

def find_anagrams(s: str, p: str) → list[int]:
    """LeetCode 438. Start indices of every anagram of p inside s."""
    if len(p) > len(s):
        return []
    need = Counter(p)
```

```

window = Counter(s[:len(p)])
out: list[int] = [0] if window == need else []

for i in range(len(p), len(s)):
    window[s[i]] += 1
    left = s[i - len(p)]
    window[left] -= 1
    if window[left] == 0:
        del window[left]          # MANDATORY: a zero entry breaks equality
    if window == need:
        out.append(i - len(p) + 1) # the START of the window
return out

if __name__ == "__main__":
    print([max_sum_of_size_k(a, k) for a, k in
          (([2, 1, 5, 1, 3, 2], 3), ([2, 3, 4, 1, 5], 2), ([1], 1),
           ([1, 2], 3), ([-1, -2, -3], 2))])
    # [9, 7, 1, None, -3]

    print(max_average_of_size_k([1, 12, -5, -6, 50, 3], 4))      # 12.75

    print(max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3))     # [3, 3, 5, 5, 6, 7]
    print(max_sliding_window([1], 1), max_sliding_window([9, 11], 2)) # [1] [11]

    print([contains_nearby_duplicate(a, k) for a, k in
          (([1, 2, 3, 1], 3), ([1, 0, 1, 1], 1), ([1, 2, 3, 1, 2, 3], 2))])
    # [True, True, False]

    print(find_anagrams("cbaebabacd", "abc"))                  # [0, 6]
    print(find_anagrams("abab", "ab"))                          # [0, 1, 2]
    print(find_anagrams("a", "ab"))                             # []

```

6 What it costs

The sum version

Building the first window is k additions. The loop then runs $n - k$ times, each doing one addition, one subtraction and one comparison — constant work.

$$k + 3(n - k) \text{ operations} \rightarrow O(n) \text{ time}$$

Space: `total` and `best`. **$O(1)$ extra space.**

Against the brute force

Recomputing each window is k additions at each of $n - k + 1$ positions:

$$(n - k + 1) \times k \rightarrow O(n \times k)$$

Measured, on random arrays:

n = 10,000	k = 100	window 0.00170 s	brute 0.0108 s	6x
n = 50,000	k = 500	window 0.00911 s	brute 0.1901 s	21x
n = 100,000	k = 1,000	window 0.01722 s	brute 0.7111 s	41x

The ratio grows with k , exactly as $O(n \cdot k)$ versus $O(n)$ predicts — roughly 6, 21, 41 as k goes 100, 500, 1,000.

An honest note worth making in an interview: the brute-force numbers here use Python's `sum()` on a slice, which runs as compiled C. A hand-written inner loop would be several times slower again, so the *true* gap between the two ideas is larger than the measured one. The operation counts are the honest comparison: at $n = 100,000$ and $k = 1,000$, that is about **100 million** element additions against **200,000**.

`max_sliding_window`

The `while` inside the `for` looks like it could be quadratic and is not. **Each index is appended exactly once and popped at most once**, so across the entire run there are at most n pushes and n pops. Total $O(n)$ time.

Space: the deque holds at most k indices, so $O(k)$.

Against the naive version — `max(nums[i:i+k])` at every position — which is $O(n \times k)$:

```
n = 100,000, k = 1,000:
naive : ~100,000,000 comparisons
deque : ~200,000 deque operations
```

There is a middle answer worth naming: a **max-heap** of size k gives $O(n \log k)$, which is better than $O(n \cdot k)$ and worse than $O(n)$, and it needs extra work to remove elements that have aged out. Heaps arrive on [day 113](#). **Mention it, then give the deque.**

The counting variants

`find_anagrams` does two `Counter` updates per step and one comparison of two dictionaries with at most 26 keys — all constant. $O(n)$ time, $O(1)$ space for a bounded alphabet, $O(k)$ in general.

`contains_nearby_duplicate` is one set add, one discard and one membership test per step: $O(n)$ time, $O(k)$ space for the window's worth of values.

The number to have ready

Windows of size `k` overlap in `k - 1` elements, so recomputing throws away almost all the work. One in and one out makes it $O(n)$ instead of $O(n \cdot k)$ — at `n = 100,000` and `k = 1,000`, 200,000 operations against 100 million.

7 The traps

The near-miss: the wrong element leaving

```
total += nums[i] - nums[i - k + 1]      # off by one
```

The window is now `k - 1` or `k + 1` wide depending on which way you got it wrong, and — this is the problem — **the answer still looks plausible**. There is no error, no crash, and the sums are in the right ballpark.

The check that catches it in five seconds: after the line runs, the window should be `nums[i - k + 1 .. i]`. Count the elements in that range: `i - (i - k + 1) + 1 = k`. **State the range out loud as you write the line.**

The near-miss: `best = 0`

```
best = 0                                # instead of the first window's total
```

On an all-negative array — `[-1, -2, -3]` with `k = 2` — this returns `0`, which is not the sum of any window. The same bug as [day 014](#): a tracker must start as a real value, not a convenient constant. `best = total` after building the first window.

The near-miss: starting the loop in the wrong place

```
for i in range(1, len(nums)):           # should be range(k, ...)
```

The first few iterations use a negative `i - k`, which in Python indexes from the **end** of the array rather than raising. So it silently subtracts elements from the far end of the input. No error, a completely wrong answer, and it is the negative-index trap from [day 016](#) again.

The near-miss: forgetting to delete zero counts

```
window[left] -= 1
# no deletion at zero
if window == need:                # never true again
```

`Counter({'a': 2, 'b': 0})` is not equal to `Counter({'a': 2})`, so once any character's count reaches zero the comparison fails forever. On `"cbaebabacd"` with `"abc"` you get `[0]` instead of `[0, 6]` — the first match is found before any count hits zero, and nothing is found afterwards. **A partly-right answer, which is the hardest kind to notice.**

The near-miss: `remove` instead of `discard`

```
seen.remove(nums[i - k - 1])
```

```
KeyError: 1
```

On `[1, 1, 1]`, the value leaving the window may already have been removed, or may equal a value still present. `discard` does nothing when the element is absent; `remove` raises.

The near-miss: the deque holding values instead of indices

```
while dq and dq[-1] ≤ x:
    dq.pop()
dq.append(x)                # values, not indices
```

Everything works until you need to know whether the front has aged out — and with only values you cannot tell, because a value does not say where it came from. **Store indices, read values through them.**

The contract corner: `k` larger than the array

```
max_sum_of_size_k([1, 2], 3)
```

There is no window of size 3, so the answer is undefined. Return `None`, raise, or return 0 — but **decide, and say which**. Without a guard, `sum(nums[:3])` quietly returns `3` (the sum of the whole array) and the loop never runs, so you get a confident answer to a question with no answer.

8 In the interview

How it gets asked

- “Find the maximum sum of any subarray of size k .” — the base version, and the one where you say the overlap insight.
- “Find the maximum average of any k consecutive elements.” — LeetCode 643, the same thing with one division.
- “Find the maximum in every window of size k .” — LeetCode 239, and a genuinely harder problem hiding one sentence away.
- “Find all anagrams of p in s .” — LeetCode 438, window plus a counter.
- “Does the array contain a duplicate within k positions?” — LeetCode 219, window plus a set, and off by one from the rest.

What to say out loud, in the first ninety seconds

1. **State the brute force and its cost.** “The obvious approach recomputes the sum of each window, which is $O(nk)$.”
2. **Say the insight in one sentence.** “But consecutive windows overlap in $k - 1$ elements, so recomputing throws away almost all the work I just did.”
3. **Say the move.** “Instead I keep a running total: when the window slides, one element enters on the right and one leaves on the left, so I add one and subtract one. Two operations instead of k .”
4. **Say the index rule, and check it aloud.** “When index i enters, index $i - k$ leaves — so afterwards the window covers $i - k + 1$ through i , which is exactly k elements.”
5. **Give the cost with the counting.** “Each element enters once and leaves once, so $2n$ operations regardless of k . $O(n)$ time, $O(1)$ space.”
6. **Flag the boundary cases.** “I’ll guard k larger than the array, and start `best` from the first window rather than zero, in case the values are negative.”
7. **Anticipate the follow-up.** “If you ask for the maximum of each window rather than the sum, this arithmetic won’t work — subtraction has an inverse and max doesn’t — and I’d need a monotonic deque.”

The follow-ups

“Now find the maximum in each window instead of the sum.” That is a genuinely different problem, and the reason is worth stating: a sum can be maintained incrementally because addition has an inverse — subtracting the leaving element exactly undoes adding it. Maximum has no inverse. If the element leaving happens to be the maximum, the new maximum could be any of the remaining $k - 1$ and I have kept no information about them. So I need a structure: a deque of **indices** whose values decrease from front to back. When a new value arrives I pop from the back everything smaller than it,

because those elements are both older and smaller, so they can never be the maximum of any window that also contains the new one. Then I pop the front if it has aged out of the window, and the front is the answer. Each index is pushed once and popped at most once, so it is $O(n)$ overall despite the inner `while` — and $O(k)$ space. A max-heap of size k is a legitimate middle answer at $O(n \log k)$, but it needs extra bookkeeping to evict aged-out elements lazily.

“How do you know the deque version is $O(n)$ and not $O(n \cdot k)$?” Because I count the pushes rather than the loop nesting. Every index is appended to the deque exactly once, and once popped it never returns. So across the entire run there are at most n pushes and at most n pops, which bounds all the work in the inner `while` loops added together — even though a single iteration can pop many elements at once. That is the same argument as any amortised bound: the expensive step is paid for by the cheap steps that preceded it. If I look at just one iteration it seems it could be $O(k)$; looking at the whole run, the total is linear.

“What if the window size changes, or the condition is ‘the longest window with at most k distinct characters’?” Then it is a variable-size window rather than a fixed one, and the structure changes: instead of sliding both edges together in lockstep, I extend the right edge to grow the window and advance the left edge only while some condition is violated. The cost argument is the same — each index is visited at most twice, once by each edge — so it stays $O(n)$. The fixed version is the easy case because the left edge’s position is completely determined by the right edge’s; in the variable version it is decided by the condition, which is what makes those problems harder.

“The array is a stream — you can’t index backwards. What changes?” The sum version needs the leaving element, so I must remember the last k values, which means a deque or a circular buffer of size k . That takes the space from $O(1)$ to $O(k)$, which is unavoidable — you cannot subtract something you have not kept. The maximum version already keeps a deque of up to k indices, so it works on a stream with no change at all beyond storing the values alongside. And a useful thing to say: this is exactly what a moving average in a monitoring system is, and why those systems have a configured window size — the window size is the memory cost.

A model answer

“The brute force is to compute the sum of each window separately: for each of the $n - k + 1$ starting positions, add up k elements. That’s $O(n \cdot k)$, and at a hundred thousand elements with k a thousand it’s about a hundred million additions.

But consecutive windows overlap in $k - 1$ elements. So recomputing throws away almost everything I just calculated. Instead I keep a running total and adjust it: when the window slides one step, exactly one element enters on the right and exactly one leaves on the left.

```
def max_sum_of_size_k(nums: list[int], k: int) → int | None:
    if k ≤ 0 or len(nums) < k:
        return None
    total = sum(nums[:k])
    best = total
    for i in range(k, len(nums)):
        total += nums[i] - nums[i - k]
        best = max(best, total)
    return best
```

Three details worth calling out. When index i enters, index $i - k$ leaves — so after that line the window covers $i - k + 1$ through i , which is exactly k elements. I check that by counting the range out loud, because getting it wrong by one gives a plausible-looking answer with no error.

The loop starts at k , because the first window was built before the loop. Starting earlier makes $i - k$ negative, and in Python a negative index reads from the *end* of the array rather than raising, so that’s another silent wrong answer.

And `best` starts as the first window’s total, not zero — otherwise an all-negative array returns zero, which isn’t the sum of any window.

Cost: each element enters once and leaves once, so $2n$ operations regardless of k . $O(n)$ time and $O(1)$ extra space, against $O(n \cdot k)$ for the brute force. I measured that: at a hundred thousand elements with k of a thousand it was about forty times faster, and the ratio grows with k exactly as you’d expect.

If you changed the question to the maximum of each window rather than the sum, this wouldn’t work. Subtraction undoes addition exactly, but there’s no way to ‘un-maximum’ — if the element leaving was the largest, the next largest could be any of the others and I’ve kept nothing about them. That one needs a deque of indices with decreasing values, and it’s still $O(n)$ because each index is pushed and popped at most once.”

9 Recall card

- **Consecutive windows overlap in $k - 1$ elements.** Do not recompute — **add the entering, subtract the leaving.** $O(n \cdot k)$ becomes $O(n)$.
- **When i enters, $i - k$ leaves,** and the window is then `nums[i-k+1 .. i]`. Say that range out loud to check yourself.
- **Build the first window before the loop; start the loop at k ; start `best` from that window,** not from 0.
- **Sums, counts and sets slide; `max` does not** — subtraction needs an inverse. `max` needs a monotonic deque of indices, still $O(n)$.
- **With a `Counter`, delete keys at zero** — a zero entry makes every equality comparison fail.

1 What this is, and why they ask it

Yesterday you learned what an index buys you. Today is *why it has the shape it has*, and the answer is one fact about hardware:

Reading from disk costs the same whether you read 1 byte or 8,000 bytes.

A disk read has an enormous fixed cost — the operating system fetches a whole page, typically 8 KB, whatever you asked for. So the number that matters is not “how many comparisons” but **how many page reads**. Everything about a B-tree’s shape follows from minimising that one number.

A binary search tree makes one decision per node, so it needs $\log_2(n)$ nodes — 24 levels for ten million rows, and if each node is on a different page, 24 disk reads. A **B-tree** packs hundreds of keys into each node, so one page read makes a several-hundred-way decision. Three or four page reads cover ten million rows. **Same number of comparisons, eight times fewer trips to the disk**, and the trips are what cost.

Interviewers ask this because it is the clearest example in the whole subject of a data structure shaped by hardware rather than by theory. A candidate who says “B-trees are balanced” has half the answer; one who says “because a disk read is a fixed cost, so you want the fattest node that fits in one page” has all of it. It also opens the door to the LSM-tree comparison, which is the natural follow-up and the reason Cassandra and RocksDB behave differently from Postgres.

2 The story

Habib’s godown is one big room with a loft above it, reached by a wooden ladder in the corner, and the loft is where everything that is not moving this week goes.

The ladder is the whole problem. It takes him about two minutes to go up it properly — he is sixty-one, and the third rung has been going for a while — and about two minutes to come down. Once he is up there, walking around and looking at things costs him nothing. He can read the labels on forty boxes in twenty seconds.

So the arithmetic of his day is entirely about trips, not about looking.

When he started, things went up in the order they arrived, and when a customer wanted something he went up, looked, came down, went up again. Six trips was a normal morning — twenty-four minutes on a ladder to find one box of clutch plates.

What he does now is that there is a board nailed to the wall at the top of the ladder, and on it is written what is in each of the eight bays — the ranges, roughly. Bay one, part numbers up to 4000. Bay two, 4000 to 9000. And so on. He climbs once, stands at the top, reads the board, and walks to the right bay.

At the front of each bay there is a smaller board doing the same thing for the twelve shelves in it.

So it is: one climb, read the big board, walk to the bay, read the small board, take the box. **One climb.** Not six, not twenty-four.

His nephew suggested something during the renovation that Habib turned down flat. The nephew wanted a neat system where at the top of the ladder there was a sign saying “under 5000, go left; over 5000, go right”, and then another sign at each place saying the same thing again, halving each time. It is tidier, and it needs fewer words on each board.

Habib’s objection was simple and correct. Every one of those signs is a place you have to *walk to*, and walking to a place is what costs. Reading eight ranges off one board costs the same as reading two. So put as much on each board as you can possibly fit, and make the number of places you have to walk to as small as possible.

3 The idea in plain English

The ladder climb is a **disk page read**. Reading the board once you are up there is **comparisons in memory**, which are effectively free. And Habib’s objection to his nephew is the entire argument for B-trees over binary trees.

The hardware fact everything rests on

From [day 009](#) and [day 010](#):

one CPU comparison	≈ 1 nanosecond
one main-memory access	≈ 100 nanoseconds
one SSD page read	≈ 100,000 nanoseconds (0.1 ms)
one spinning-disk seek	≈ 10,000,000 nanoseconds (10 ms)

An SSD page read is about **100,000 times** the cost of a comparison. And crucially, the disk delivers a whole **page** — 8 KB in Postgres, 16 KB in MySQL’s InnoDB — whether you wanted one byte or all of it.

So a structure stored on disk should be designed to answer the question “**how few pages must I touch?**” and should treat everything inside a page as free.

Why a binary tree is the wrong shape here

A binary search tree stores one key per node and makes one decision at each. For n items, the depth is $\log_2(n)$:

$$n = 10,000,000 \rightarrow \log_2(10,000,000) \approx 24 \text{ levels}$$

Each node is small — a key and two pointers, maybe 24 bytes — so it sits somewhere unpredictable in memory or on disk. Walking down the tree touches 24 different nodes, and in the worst case that is **24 separate page reads**:

$$24 \text{ page reads} \times 0.1 \text{ ms} = 2.4 \text{ ms}$$

And 8,000 bytes arrive with each read of which you use 24. You are paying for a full page and using 0.3% of it.

That is the nephew’s tidy scheme: many signs, each carrying one bit of information, each requiring a walk.

What a B-tree does instead

A **B-tree** node is exactly one page, and it is filled with as many keys as will fit. With an 8 KB page and roughly 16 bytes per entry:

$$8,192 \text{ bytes} / 16 \text{ bytes} \approx 500 \text{ entries per node}$$

That is the **fanout** — the number of children a node can have. One page read now makes a 500-way decision instead of a 2-way one, so the depth is $\log_{500}(n)$:

$$\log_{500}(10,000,000) \approx 2.7 \rightarrow 3 \text{ levels}$$

ROWS	BINARY TREE DEPTH	B-TREE DEPTH (FANOUT 500)
1,000	10	2
250,000	18	2
10,000,000	24	3
62,500,000,000	36	4

Three page reads instead of twenty-four. Same total number of comparisons — about 24 either way, since $\log_2(500) \approx 9$ and $3 \times 9 = 27$ — but 24 of them happen *inside pages you have already paid for*.

That is the whole answer: **the comparisons are free and the page reads are not, so you want the fattest node that fits in one page.**

The three properties of a B-tree

1. **Every leaf is at the same depth.** The tree is perfectly balanced by construction, so every lookup costs the same. There is no bad case.
2. **Nodes are kept at least half full.** A node that drops below half merges with a sibling. This is what stops the tree degenerating into something tall and sparse after many deletions.
3. **Keys within a node are sorted, and so are the children.** That is what makes range queries work.

B+ trees, which is what databases actually use

Almost every real database uses a **B+ tree**, a variant with two differences that both matter:

All the actual data pointers live in the leaves. Internal nodes hold only keys, used for navigation. That makes internal nodes smaller, so the fanout is higher, so the tree is shallower.

The leaves are linked to each other in order. Once you have found the start of a range, you walk sideways along the leaves without going back up the tree.

That second property is why range queries are cheap:

```
SELECT * FROM orders WHERE created_at BETWEEN '2026-01-01' AND '2026-03-31';
```

Three page reads down to the first matching leaf, then a **sequential** walk along linked leaves. From **day 010**, sequential reads are far cheaper than random ones, so a range scan over a million rows is genuinely fast — and it is the reason **ORDER BY** on an indexed column needs no sort step, which is often a bigger win than the lookup itself.

Everyone says “B-tree” and means B+ tree. **Saying “it’s actually a B+ tree, with the data in the leaves and the leaves linked” is a cheap, strong signal.**

What it costs to write

Inserting a key means finding the right leaf — three page reads — and writing into it. If the leaf is full, it **splits**: half the keys move to a new page, and a key is pushed up to the parent. If the parent is full too, that splits as well, and in the rare worst case the split propagates to the root and the tree gains a level.

So a write is a handful of page reads plus one or a few page writes, and occasionally more. That is the concrete cost behind yesterday’s “every index makes writes slower”.

The other cost is **fragmentation**. Splits leave pages half full, and random insertion order — random UUID primary keys, from [day 026](#) — spreads writes across the whole tree, so pages fill unevenly and the index grows larger than the data it indexes. Sequential keys append to the rightmost leaf, which stays in cache and fills completely. **That is the real reason BIGSERIAL beats random UUIDs**, and it is a much better answer than “UUIDs are bigger”.

The alternative: LSM trees

The natural follow-up is “*is a B-tree always right?*”, and the answer is no.

A **log-structured merge tree** — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade. Writes go into an in-memory sorted structure and are appended to a log; when it fills, it is flushed to disk as one sorted immutable file. Background compaction merges those files.

	B-tree	LSM tree
writes	random page updates, read-modify-write	sequential appends only (much faster writes)
reads	3-4 page reads, predictable	may check several files (bloom filters help; still slower)
space	fragmentation from splits	write amplification from compaction
best for	read-heavy, ranges, OLTP	write-heavy, time series, logs

B-trees favour reads; LSM trees favour writes. Postgres, MySQL, Oracle and SQLite use B-trees. Cassandra, RocksDB and ScyllaDB use LSM trees. MongoDB’s WiredTiger can do either. Naming that contrast is what turns a textbook answer into a design answer.

4 The picture

The two shapes, side by side, for the same ten million rows:

BINARY SEARCH TREE
one key per node, 24 levels

```
(o)
 / \
(o) (o)
 / \ / \
(o) (o)(o) (o)
...
... 24 levels ...
```

24 page reads × 0.1 ms = 2.4 ms
uses 24 bytes of each 8 KB page

B+ TREE (fanout ~500)
~500 keys per node, 3 levels

```
+-----+
| 200 | 400 | ... | 99800 |
+-----+
 /      |      \
+-----+ +-----+ +-----+
|210|...| |410|...| |610|...|
+-----+ +-----+ +-----+
|
leaf |211|212|...|249| → |251|...|
+-----+
leaves LINKED in order
```

3 page reads × 0.1 ms = 0.3 ms
uses all 8 KB of each page

What to notice: the comparison count is nearly the same — about 24 either way — but the binary tree pays a disk read for each one, and the B-tree pays for three. **The unit of cost is the page, not the comparison.**

Why the fanout is what it is:

```
one page = 8,192 bytes
one entry = key (8 bytes) + child pointer (~6) + overhead (~2) ≈ 16 bytes
```

```
8,192 / 16 ≈ 500 entries per node
```

```
depth 1:          500
depth 2:          250,000      (500 x 500)
depth 3:        125,000,000    (500 x 500 x 500)
depth 4: 62,500,000,000
```

A 1,000-fold increase in rows costs ONE extra page read.

The range query, which is what the linked leaves are for:

```
WHERE created_at BETWEEN 'A' AND 'B'
```

```
    root ----- 1 page read, navigate down
    |
  internal ----- 1 page read
    |
  leaf → leaf → leaf → leaf ...
    ^               ^
  find A          stop at B
```

3 random reads to find the start, then SEQUENTIAL reads along the leaves. No going back up the tree. This is why ORDER BY on an indexed column needs no sort step at all.

5 How it actually works

A lookup, step by step

```
SELECT * FROM orders WHERE id = 4217
```

 on a ten-million-row table:

1. **Read the root page.** It holds ~500 separated key ranges. Binary-search *within the page*, in memory, to find which child covers 4217. One page read, about 9 comparisons.
2. **Read that internal page.** Same again. One page read.
3. **Read the leaf page.** It holds ~500 keys with pointers to rows. Binary-search it, find 4217, get the row location. One page read.
4. **Read the table page** holding the row itself. One page read.

Four page reads total. And in practice the root and often the whole first internal level are already in the buffer pool, because they are read constantly — so the real cost is frequently one or two physical reads.

That caching point is worth making: **the top of the tree is hot by definition**, because every single lookup goes through it.

An insert, and the split

1. Navigate to the correct leaf — three page reads.
2. If the leaf has room, insert the key in sorted position and write the page. Done.
3. If it is full, **split**: create a new page, move half the keys into it, and insert a separator key into the parent.
4. If the parent is now full, split it too. If that reaches the root, the root splits and **the tree grows by one level** — the only way a B-tree gets taller, and it happens from the top, which is what keeps every leaf at the same depth.

Splits are why the “at least half full” rule exists: after a split both halves are half full, so there is room for future inserts without immediately splitting again.

Why insertion order matters so much

```
sequential keys (BIGSERIAL):
    every insert goes to the RIGHTMOST leaf
    that page stays in cache and fills completely
    → few splits, dense pages, small index

random keys (UUIDv4):
    every insert goes to a DIFFERENT leaf
    each one must be read from disk, modified, written back
    → the working set is the whole index, pages half full, index much larger
```

This is the concrete mechanism behind yesterday’s advice, and behind [day 026](#)’s preference for `BIGSERIAL` or `UUIDv7` over random `UUIDv4`. **UUIDv7 and ULID put a timestamp in the high bits precisely to restore this locality.**

Bloating, and what to do about it

Deletions leave pages under-full, and Postgres’s MVCC — from [day 025](#) — leaves dead row versions behind. Over time an index can become several times larger than it needs to be, which means more pages to read, which means slower.

The fixes: `VACUUM` reclaims space for reuse; `REINDEX CONCURRENTLY` rebuilds an index compactly without blocking writes. **Index bloat is one of the classic causes of “it got slower and nothing changed.”**

Page size, and why 8 KB

Not arbitrary. It balances three things: the operating system reads in 4 KB pages, so 8 KB is two of them; larger pages give higher fanout and shallower trees but waste more when you only want one row; and the whole page must be written back for any change, so bigger pages mean more write amplification. Postgres uses 8 KB, InnoDB 16 KB, and both are configurable and almost never changed.

What else is a B-tree

Worth knowing, because it shows the idea is general: **filesystems** use them — NTFS, ext4's HTree, XFS, APFS — for exactly the same reason, since a directory lookup is also a disk-bound search. The constraint produces the same shape wherever it appears.

6 The numbers

Depth, and the read cost

```
fanout 500:
  depth 2 :      250,000 rows
  depth 3 :   125,000,000 rows
  depth 4 : 62,500,000,000 rows

binary tree, same 10,000,000 rows:  $\log_2(10,000,000) \approx 24$  levels

page reads : 3 vs 24
at 0.1 ms per SSD read : 0.3 ms vs 2.4 ms      → 8x
at 10 ms per HDD seek  : 30 ms vs 240 ms       → 8x, and both are terrible
```

The ratio is the same on either device; the absolute numbers are what made this design mandatory in the disk era and still worthwhile on SSDs.

Comparisons versus page reads

```
B-tree   : 3 pages ×  $\log_2(500) \approx 3 \times 9 = 27$  comparisons, 3 page reads
binary   : 24 nodes                      = 24 comparisons, up to 24 page reads

comparisons: 27 vs 24 — the B-tree does slightly MORE
page reads : 3 vs 24 — the B-tree does 8x FEWER
```

That table is the answer to the interview question. The B-tree is not saving comparisons. It is moving them inside pages you have already paid to fetch.

Page utilisation

```
binary tree node : 24 bytes used of an 8,192-byte page = 0.3%
B-tree node      : ~8,000 bytes used of 8,192         = 98%
```

Index size

```
10,000,000 rows, bigint key:
  entry ≈ 16 bytes
  leaves : 10,000,000 × 16      = 160 MB
  internal levels add ~1/500th ≈ 0.3 MB
  with ~70% average fill      ≈ 230 MB

table itself, 200 bytes/row      = 2 GB
so the index is ~11% of the table
```

Sequential versus random insertion

```
1,000,000 inserts:

sequential keys : nearly all hit the rightmost leaf, which is cached
                  ≈ 1,000,000 / 500 = 2,000 page writes
random keys     : each hits a different leaf; the working set is the whole index
                  ≈ up to 1,000,000 page read-modify-writes

difference: up to 500x in page I/O, and the index ends up ~30-40% larger
            from half-full pages
```

That is why sequential ids matter, stated in a way that survives a follow-up.

The range scan

```
WHERE created_at BETWEEN A AND B, matching 100,000 rows:

  navigate to the first leaf : 3 random page reads
  walk the linked leaves    : 100,000 × 16 B ≈ 1.6 MB, sequential
  at 500 MB/s                ≈ 3 ms

versus 100,000 independent lookups : 100,000 × 3 page reads = 300,000 random reads ≈ 30 s
```

Four orders of magnitude, and it comes entirely from the leaves being linked.

7 The trade-offs

B-tree against a hash index

A hash index is $O(1)$ for equality and cannot do anything else — no ranges, no `BETWEEN`, no prefix `LIKE`, no `ORDER BY`. A B-tree is $O(\log n)$, but $\log_{500}$ of anything realistic is 3 or 4, so the “slower” one is slower by nothing measurable and does five extra things.

That is why Postgres’s default is a B-tree and hash indexes are rarely used. The constant-factor win does not exist in practice and the lost functionality is real.

B-tree against an LSM tree

The real trade, and the one worth arguing:

- **B-tree:** predictable reads at 3–4 page reads; in-place updates, so writes are random I/O; space lost to fragmentation. Right for read-heavy transactional workloads with range queries.
- **LSM tree:** writes are pure sequential appends, which is dramatically faster; reads may have to check several sorted files, mitigated by bloom filters but still slower and less predictable; background compaction consumes I/O and causes latency spikes.

Choose an LSM tree when writes dominate — time-series, event logs, metrics, sensor data. **Choose a B-tree when reads and ranges dominate**, which is most transactional applications.

Higher fanout, or smaller pages

A larger page means higher fanout and a shallower tree — but every modification rewrites the whole page, so write amplification goes up, and a lookup that wants one 200-byte row fetches 16 KB. 8 KB is the settled compromise and almost nobody changes it.

Sequential or random keys

Sequential keys give dense pages, few splits and a cached hot spot. They also create a **write hot spot**: in a distributed system every insert goes to the same shard, which is exactly what you do not want. **That is why Cassandra and DynamoDB push you towards randomly-distributed partition keys**, and why the answer differs between a single Postgres instance and a distributed store. UUIDv7 is the compromise: roughly ordered for locality, random enough in the low bits to avoid perfect contention.

The sentence that separates candidates

I would not use a binary search tree for anything stored on disk. The comparison count is essentially the same either way — about 24 for ten million rows — but a binary tree pays a page read for every one of those comparisons, while a B-tree pays for three and does the other 24 inside pages it has already fetched. The unit of cost is the page, not the comparison. In memory, where there is no page to fetch, a balanced binary tree is perfectly reasonable and is what an in-memory sorted map usually is — the shape follows from where the data lives.

8 In the interview

How it gets asked

- “Why is a database index a B-tree and not a binary search tree?” — the direct version. The answer is about page reads, not about balance.
- “How many disk reads to find a row in a ten-million-row table?” — three or four, and you should be able to derive it from the fanout.
- “Why do range queries work on an index but hash lookups don’t?” — the linked leaves.
- “Why are sequential ids better than random UUIDs for a primary key?” — page locality and splits.
- “When would you not use a B-tree?” — LSM trees, and write-heavy workloads.

What to say out loud, in the first ninety seconds

1. **Lead with the hardware fact.** “Because the unit of cost on disk is the page, not the comparison. A read fetches 8 KB whether you wanted one byte or all of it, and it costs about 0.1 ms — roughly a hundred thousand times a comparison.”
2. **Say what follows from it.** “So you want to make each page read decide as much as possible. A binary node uses 24 bytes of an 8 KB page and makes a two-way decision. A B-tree node fills the page with about 500 keys and makes a 500-way decision.”
3. **Give the depths.** “For ten million rows that’s log base 500, so three levels, against log base 2, which is 24. Three page reads instead of twenty-four.”
4. **Make the comparison point explicitly.** “The comparison count is about the same — 27 against 24 — so this isn’t saving comparisons. It’s moving them inside pages I’ve already paid to fetch.”
5. **Add the B+ detail.** “And it’s really a B+ tree: all the row pointers are in the leaves, so internal nodes are smaller and the fanout is higher, and the leaves are linked in order.”
6. **Say what the linking buys.** “That’s why ranges and `ORDER BY` are cheap — find the start, then walk sideways sequentially instead of doing independent lookups.”
7. **Name the alternative.** “If the workload were write-heavy I’d consider an LSM tree instead — Cassandra and RocksDB — which turns random writes into sequential appends at the cost of slower, less predictable reads.”

The follow-ups

“How many disk reads to find one row in a ten-million-row table?” Three or four, and I can derive rather than recall it. A page is 8 KB and an index entry is roughly 16 bytes — a key plus a pointer plus overhead — so about 500 entries fit in a page, which is the fanout. That gives 500 at depth one, 250,000 at depth two, and 125 million at depth three, so ten million rows needs three levels. Three page reads walks me down to the leaf, and one more fetches the actual table row, so four. In practice fewer are physical

reads, because the root and usually the whole second level are permanently in the buffer pool — every lookup goes through them, so they are hot by definition. And if the index covers every column the query needs, the fourth read disappears entirely, because it can be answered from the leaf without touching the table.

“Why not just use a balanced binary tree?” Because the cost model is wrong for disk. A red-black or AVL tree does $\log_2(n)$ comparisons, which for ten million rows is 24 — and that is fine in memory. On disk, each of those 24 nodes is likely on a different page, so it is 24 page reads at 0.1 ms each, about 2.4 milliseconds, and each read delivers 8 KB of which the node uses about 24 bytes. A B-tree does roughly the same number of comparisons — three levels times nine comparisons within each page is 27 — but only three page reads, because 24 of those comparisons happen inside pages already in memory. So the B-tree is not a cleverer search; it is the same search reorganised so that the expensive operation happens as rarely as possible. In memory, where there is no page to fetch, a balanced binary tree is entirely reasonable, and that is what an in-memory sorted map usually is.

“Why can an index do range queries when a hash index can’t?” Because a B-tree is sorted and a hash is not. A hash function deliberately scatters similar keys to unrelated buckets — that is what makes it uniform — so there is no sense in which the entries “next to” a key are the next values. A B-tree keeps keys in order within each node and orders the children the same way, and in a B+ tree the leaves are additionally linked left to right. So a range query navigates down to the first matching leaf in three reads and then walks sideways along linked leaves, reading sequentially. That is also why `ORDER BY` on an indexed column costs nothing extra — the index already is that order, so the sort step disappears, which for a large result is often a bigger win than the lookup. It is also why prefix `LIKE 'Ram%'` works and suffix `LIKE '%kumar'` does not: a prefix is a contiguous range in sorted order and a suffix is not.

“When would you not use a B-tree?” When writes dominate. A B-tree updates pages in place, so a write is a random read-modify-write, and at very high insert rates that becomes the bottleneck — plus every split fragments the tree. An LSM tree inverts it: writes go to an in-memory sorted structure and are appended sequentially to a log, then flushed as immutable sorted files that background compaction merges. Sequential appends are far faster than random page updates, so write throughput is much higher. The price is on reads — a lookup may have to check several files, which bloom filters mitigate but do not remove — and compaction consumes I/O in bursts, so latency is less predictable. So: time-series, event logs and metrics go to Cassandra or RocksDB; transactional workloads with ranges and predictable read latency stay on a B-tree, which is Postgres, MySQL, Oracle and SQLite.

A model answer

“Because the unit of cost on disk is the page, not the comparison.

A disk read fetches a whole page — 8 KB in Postgres — whether you wanted one byte or all of it, and it costs around 0.1 milliseconds on an SSD. A CPU comparison is about a nanosecond. So a page read is roughly a hundred thousand times more expensive than a comparison, and any structure that lives on disk should be designed to minimise page reads and treat everything inside a page as free.

A binary search tree gets that exactly backwards. Each node holds one key and makes a two-way decision, so for ten million rows you need log base 2, which is about 24 levels. If each node is on a different page, that’s 24 page reads — about 2.4 milliseconds — and each read delivered 8 KB of which you used 24 bytes.

A B-tree makes each node exactly one page and fills it. With 8 KB pages and about 16 bytes per entry, that’s roughly 500 keys per node, so one page read makes a 500-way decision. The depth becomes log base 500, which for ten million rows is three levels. Three page reads instead of 24.

The point I’d want to make explicitly is that this isn’t saving comparisons. Three levels times about nine comparisons inside each page is 27, against 24 for the binary tree — slightly more. What changed is that 24 of those comparisons now happen inside pages I’ve already paid to fetch. The structure was reshaped around which operation is expensive.

And the fanout is why the depth barely grows: 500 at one level, 250,000 at two, 125 million at three, 62 billion at four. A thousand-fold increase in data costs one extra page read.

In practice it’s a B+ tree, with two refinements. All the row pointers live in the leaves, so internal nodes carry only keys and the fanout is even higher. And the leaves are linked in order, which is what makes ranges cheap — you navigate down once and then walk sideways sequentially, instead of doing an independent lookup per row. That’s also why `ORDER BY` on an indexed column needs no sort step.

Where I’d reconsider is a write-heavy workload. A B-tree updates pages in place, so writes are random I/O and splits fragment it. An LSM tree — Cassandra, RocksDB — turns writes into sequential appends and pays for it on reads, which may have to consult several files. So time-series and event logs go that way; transactional workloads with ranges stay on a B-tree.”

9 Recall card

- **The unit of cost on disk is the page, not the comparison.** A page read is ~ 0.1 ms and fetches 8 KB whatever you asked for.
- **So make each node one page and fill it.** Fanout ~ 500 , so $\log_{500}(10M) = 3$ levels against $\log_2(10M) = 24$.
- **It does not save comparisons** — 27 against 24 — **it saves page reads**, 3 against 24.
- **B+ tree: pointers in the leaves, leaves linked in order.** That is what makes ranges and `ORDER BY` cheap.
- **Sequential keys fill the rightmost leaf and stay cached; random keys scatter and split.** B-trees favour reads; **LSM trees** favour writes.

DSA topic: Fixed-size sliding window **System design topic:** B-trees and why indexes are shaped that way

Code these, in this order

Four problems with the same window and four different things carried inside it. **For each one, say what you are maintaining and whether it can be updated in $O(1)$ per slide** — that single question decides whether the problem is easy or hard.

Before each one, ask:

1. What quantity does the window carry — a sum, a count, a set, a maximum?
2. Does that quantity have an inverse, so it can be adjusted rather than recomputed?
3. Which index enters and which leaves? State the resulting range out loud.
4. What is the answer when k is larger than the input?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Maximum Average Subarray I	LeetCode 643 (Easy)	The core slide, and dividing once at the end rather than inside the loop.
2	Contains Duplicate II	LeetCode 219 (Easy)	The window is $k + 1$ wide, so the element leaving is at $i - k - 1$. Off by one from every other problem here, on purpose.
3	Find All Anagrams in a String	LeetCode 438 (Medium)	Window plus a <code>Counter</code> , and the <code>del</code> at zero without which every comparison fails after the first zero.
4	Sliding Window Maximum	LeetCode 239 (Hard)	The monotonic deque. Easy problem, one sentence away, and a completely different technique — because <code>max</code> has no inverse.

On problem 1, check the index rule out loud

Before writing the loop, complete these three lines:

```
k = 3, window currently covers indices 0,1,2
index 3 enters → index ___ leaves → window is now [___ .. ___]
index 4 enters → index ___ leaves → window is now [___ .. ___]
```

Then write the loop and confirm that after `total += nums[i] - nums[i - k]` the window is `nums[i-k+1 .. i]`, which has exactly k elements.

On problem 1, break it three ways

Run each and say what happens:

```
# A
total += nums[i] - nums[i - k + 1]
```

```
# B
best = 0
print(max_sum([-1, -2, -3], 2))
```

instead of the first window's sum

```
# C
for i in range(1, len(nums)):
    # instead of range(k, ...)
```

A gives a plausible wrong answer. B returns 0, which is not the sum of any window. C uses a negative index, which in Python reads from the **end** of the array rather than raising. Say which of the three you would be least likely to notice, and why.

On problem 2, work out the width yourself

The problem says “there exist two distinct indices `i` and `j` such that `nums[i] == nums[j]` and `abs(i - j) ≤ k`”. Before coding, answer:

1. How many elements does the window need to hold?
2. Therefore, which index leaves when `i` enters?
3. Why is it different from problems 1 and 3?

Then run it on `[1,0,1,1]` with `k = 1` — the answer is `True` — and on `[1,2,3,1,2,3]` with `k = 2`, where it is `False`.

Then swap `discard` for `remove` and run it on `[1,1,1]`. Read the error and say why `discard` is the right choice here.

On problem 3, find the silent failure

Write it, then delete the two lines that remove a zero count:

```
window[left] -= 1
# if window[left] == 0:
#     del window[left]
```

Run it on `find_anagrams("cbaebabacd", "abc")`. You should get `[0]` instead of `[0, 6]`.

Then explain, in one sentence, why `Counter({'a': 1, 'b': 0})` is not equal to `Counter({'a': 1})`, and why that makes every comparison after the first zero fail.

On problem 4, do the two easy things first

1. Write the naive version: `max(nums[i:i+k])` at each position. State its cost.
2. Write the heap version and state its cost. Say what extra bookkeeping it needs.
3. Only then write the deque version.

Then answer, out loud:

1. Why can a sum be maintained incrementally and a maximum cannot?
2. In the deque, why do you store **indices** rather than values?
3. Why do you pop from the back everything smaller than the entering value?
4. Why is it `O(n)` and not `O(n·k)`, given there is a `while` inside a `for`?

Number 4 is the argument you will be asked for. It is about counting pushes.

The maintainability drill

For each quantity, say whether a fixed window can maintain it in `O(1)` per slide, and why:

1. Sum.
2. Average.
3. Product. (Careful — there is a subtlety.)
4. Maximum.
5. Number of distinct values.
6. Whether all values are distinct.
7. Median.
8. The count of a particular character.

Number 3 has an inverse in principle — division — and one input breaks it. Say which. Number 7 needs a structure you have not met yet; name what it would have to support.

The measurement drill

Run this and read the ratio column.

```

import time, random
random.seed(3)

def window(nums, k):
    total = sum(nums[:k]); best = total
    for i in range(k, len(nums)):
        total += nums[i] - nums[i - k]
        best = max(best, total)
    return best

def brute(nums, k):
    best = None
    for i in range(len(nums) - k + 1):
        s = sum(nums[i:i + k])
        if best is None or s > best: best = s
    return best

for n, k in ((10_000, 100), (50_000, 500), (100_000, 1000)):
    arr = [random.randint(-100, 100) for _ in range(n)]
    s = time.perf_counter(); window(arr, k); t1 = time.perf_counter() - s
    s = time.perf_counter(); brute(arr, k); t2 = time.perf_counter() - s
    print(f"n={n:>7} k={k:>5} {t1:.5f}s {t2:.4f}s ratio {t2/t1:>6.0f}x")

```

Then answer:

1. Why does the ratio grow as `k` grows, rather than staying constant?
2. The brute version uses `sum()` on a slice, which runs as compiled C. Does that make the measured ratio bigger or smaller than the true gap between the two ideas?
3. Give the operation counts at `n = 100,000`, `k = 1,000` for both.

The B-tree drill

Answer each in one or two sentences, out loud:

1. Why is the unit of cost on disk a page rather than a comparison? Give both timings.
2. Fanout 500 — rows covered at depth 2, 3 and 4.
3. Compare page reads **and** comparison counts for a B-tree against a binary tree on ten million rows. Which one is the point?
4. What are the two differences in a B+ tree, and what does each buy?
5. Why do range queries and `ORDER BY` cost almost nothing on an indexed column?
6. What happens on an insert when the leaf is full? How does the tree ever get taller?
7. Why do sequential primary keys produce a smaller, faster index than random UUIDs?
8. When would you choose an LSM tree instead, and what do you give up?

Number 3 is the one that separates a memorised answer from an understood one.

The derivation drill

Do not recall these — derive them, out loud, in under a minute:

- 1. Page 8 KB, entry 16 bytes → fanout.
- 2. Fanout → depth for 10 million rows.
- 3. Depth × 0.1 ms → lookup time.
- 4. Binary tree depth for the same rows → its lookup time.
- 5. The ratio.

Then do the same for a 16 KB page and say what changes and what does not.

The comparison drill

Fill in this table from memory:

	B-TREE	HASH INDEX	LSM TREE
equality lookup			
range query			
ORDER BY for free			
write cost			
used by			

Then say, in one sentence each, why Postgres defaults to a B-tree rather than a hash index, and why Cassandra does not use a B-tree at all.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

- 1. Find the maximum sum of any subarray of size k. Brute force and its cost. Then the overlap insight in one sentence. Then the slide, saying the index rule and checking the resulting range aloud. Then O(n) with the counting. Then flag the two boundary cases — k too large, and best starting from the first window.
- 2. Why is a database index a B-tree and not a binary search tree? Lead with the hardware fact and both timings. Derive the fanout and the depth. Then the point that makes it: the comparison counts are the same and the page reads differ by eight times. Then B+ refinements, then LSM as the alternative.

3. Now find the maximum in each window instead of the sum. Say why the arithmetic stops working — addition has an inverse and `max` does not. Then the deque of indices, both pop rules with their reasons, and the `O(n)` argument by counting pushes.

Before you move on

- [] I say “consecutive windows overlap in `k-1` elements” before writing any window code.
- [] I state the resulting range `[i-k+1 .. i]` out loud to check the index rule.
- [] I start `best` from the first window, never from 0.
- [] I guard `k` larger than the array and say what I return.
- [] I know which quantities have an inverse and which do not, and why that decides the technique.
- [] I delete zero counts from a `Counter`, and use `discard` rather than `remove` on a set.
- [] I can give the deque’s two pop rules with a one-line reason for each.
- [] I can argue the deque is `O(n)` by counting pushes rather than loop nesting.
- [] I can derive fanout $\rightarrow$ depth $\rightarrow$ page reads without recalling any of the numbers.
- [] I can say why the B-tree does *more* comparisons than a binary tree and is still eight times faster.
- [] I can redraw the sliding-window diagram and the B-tree fanout diagram from memory, in whatever tool I like.

Variable-size sliding window

DATA STRUCTURES AND ALGORITHMS

Variable-size sliding window

You can grow and shrink a window against a condition and never lose track of the invariant.

SYSTEM DESIGN

Query plans and the slow query

You can read an EXPLAIN output and say which part of the query is the problem.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the smallest subarray with a sum at least k .*
- *Here is an EXPLAIN plan. What is wrong with this query?*

1 What this is, and why they ask it

Yesterday the window was `k` wide and both edges moved together. Today **the width is not given** — it is whatever the condition says it should be. The right edge always advances by one; the left edge advances only when a rule tells it to.

That one change turns a small technique into a large family. *The shortest subarray summing to at least k . The longest substring with no repeats. The longest substring with at most k distinct characters. The smallest window containing all the letters of another string.* All of them are eight lines with a different condition, and all of them are `O(n)`.

The cost argument is the same as it has been all week and worth saying precisely: **each index is visited at most twice** — once when the right edge passes it and once when the left edge does. Neither edge ever moves backwards. So a `while` loop nested inside a `for` loop is still linear, and being able to explain that is half of what is being tested.

The other half is knowing **when the technique does not apply**. A sliding window needs the condition to behave predictably as the window grows — and the moment an array contains negative numbers, “the sum only increases as I add elements” stops being true and the whole method silently breaks. Candidates who apply a window to “*shortest subarray with sum at least k , values may be negative*” get a wrong answer with no error. Knowing that boundary is the difference between using a technique and reciting one.

2 The story

Devappa farms about four acres outside Belur, and the thing that decides his year is when the rain comes and how fast.

He has the daily figures on his phone — the taluk office publishes them, millimetre by millimetre, going back years. And the number he actually cares about is a strange one: **what is the shortest run of consecutive days that gave him fifty millimetres?** Because fifty millimetres is roughly what soaks the field properly, and if it takes four days he can sow on the fifth, and if it takes nineteen days it has been useless — it ran off or evaporated between falls.

The first year he looked it up he did it the long way. Start at the first of June and add days until he got to fifty. Then start at the second of June and add days until fifty. Then the third. It took him an evening and he was fairly sure he had made an arithmetic mis-

take somewhere.

His son showed him the quick way and it takes about a minute.

You keep a running total and you keep two dates — where the run starts and where it ends. You move the end date forward one day at a time, adding that day's rain to the total.

The moment the total reaches fifty, you stop adding and do the other thing: **you push the start date forward**, taking days off the front, for as long as the total is still fifty or more. Each time you push it, you have a shorter run that still works, so you write down the length. When taking one more day off would drop you below fifty, you stop, and go back to moving the end date forward.

The part that surprised Devappa is that you never put a day back at the front. Once a day has been dropped off the start, it is gone for the rest of the calculation. His son explained it and Devappa repeated it back in his own words, which were better: any run that still includes that day is at least as long as one that starts later, and I have already looked at the ones that start later. So there is nothing back there worth going back for.

The whole thing goes forward, once, and never turns round.

3 The idea in plain English

Devappa's end date is `right`, his start date is `left`, and his running total is the window's sum. Everything today is the two rules for moving them.

The two shapes

There are exactly two, and picking the wrong one is the commonest error in this family.

Shape A — shrink while valid. Use it to MINIMISE.

Grow the window until the condition is satisfied, then shrink from the left **while it is still satisfied**, recording the length at each step.

```

left = 0
total = 0
best = float("inf")
for right, x in enumerate(nums):
    total += x
    while total ≥ target:          # while still VALID
        best = min(best, right - left + 1)
        total -= nums[left]
        left += 1
return 0 if best == float("inf") else best

```

Note where the recording happens: **inside the shrink loop, before shrinking**. Every time the window is valid you have a candidate, and you want the smallest of them.

Shape B — shrink while invalid. Use it to MAXIMISE.

Grow the window; whenever it becomes invalid, shrink from the left **until it is valid again**; then record.

```

left = 0
best = 0
for right, ch in enumerate(s):
    add(ch)
    while invalid():          # while still BROKEN
        remove(s[left])
        left += 1
    best = max(best, right - left + 1)  # record AFTER restoring validity
return best

```

Here the recording is **after** the shrink loop, outside it, because only then is the window guaranteed valid.

The rule to remember: minimising means shrink-while-valid and record inside; maximising means shrink-while-invalid and record after. Get those two the wrong way round and the code runs and gives plausible nonsense.

Why it is $O(n)$

`right` advances exactly `n` times, once per turn of the `for` loop. `left` only ever advances, and it can never pass `right`, so across the whole run it advances at most `n` times too.

Total: at most $2n$ moves. Each element is added once and removed at most once. That is $O(n)$, despite the `while` inside the `for` — and the argument is *count how far the pointers travel*, not *count the nested loops*. Same reasoning as [day 023](#) and [day 031](#).

The condition that makes this legal

A sliding window is not always allowed, and this is the part most people never learn.

The technique needs the condition to be **monotonic in the window**: extending the window on the right must move you in one direction only, and shrinking on the left must move you back.

- *Sum of positive numbers $\geq$ target* — adding elements only increases the sum. **Monotonic. Window works.**
- *At most k distinct characters* — adding can only increase the distinct count, removing can only decrease it. **Monotonic. Window works.**
- *Sum $\geq$ target, values may be negative* — adding an element might **decrease** the sum, so a window that is invalid now might become valid later, and shrinking might make it valid rather than invalid. **Not monotonic. The window is wrong**, and the correct technique is prefix sums plus a monotonic deque, which is LeetCode 862 and considerably harder.

Say this out loud when the array might contain negatives. “A sliding window needs the sum to grow monotonically as I extend, which negatives break — so I’d use prefix sums with a deque instead.” That sentence is worth more than any of the code on this page.

The four variants you will actually be asked

PROBLEM	SHAPE	THE CONDITION	WHAT THE WINDOW CARRIES
Minimum size subarray sum (209)	A, minimise	<code>total $\geq$ target</code>	a running sum
Longest substring without repeating characters (3)	B, maximise	a character repeats	a set, or a last-seen map
Longest substring with at most k distinct (340)	B, maximise	<code>len(counts) > k</code>	a count map
Minimum window substring (76)	A, minimise	all of <code>t</code> is covered	a count map plus a “missing” counter

Two ways to handle repeats, and why one is nicer

For *longest substring without repeating characters* there are two correct approaches.

Shrink one at a time, which is the pure shape-B template:

```
while ch in seen:
    seen.discard(s[left])
    left += 1
seen.add(ch)
```

Jump straight there, using a map of last-seen positions:

```
if ch in last and last[ch] ≥ left:
    left = last[ch] + 1
last[ch] = right
```

The jump version is what you met on [day 024](#), and `last[ch] ≥ left` is the guard without which `left` moves **backwards** on input like `"abba"` — which breaks the whole `O(n)` argument as well as the answer. **The while version cannot have that bug**, because it only ever increments, which is a reason to prefer it under pressure.

The subtle one: longest repeating character replacement

LeetCode 424: *you may change at most `k` characters; what is the longest run of one repeated character you can make?*

The window is valid when `window_length - count_of_the_most_frequent_character ≤ k`, because those are the characters you would have to change.

The famous subtlety is that the standard solution **never decreases the maximum-frequency tracker** when the window shrinks — and it is still correct. The reason: the answer only ever grows, so a window carried forward with a stale (too high) maximum frequency can only fail to *shrink*, never produce a wrong larger answer. The window size never decreases, which is exactly what you want when maximising.

You do not have to use that trick. Recomputing `max(count.values())` inside the loop is correct and `O(26)` per step, which is constant. Say the simple version, and mention the optimisation.

4 The picture

Shape A, minimising. `nums = [2,3,1,2,4,3]`, `target = 7`:

right=0	[2]	total=2	< 7, grow
right=1	[2,3]	total=5	< 7, grow
right=2	[2,3,1]	total=6	< 7, grow
right=3	[2,3,1,2]	total=8	≥ 7 → record 4, drop 2, left=1
	[3,1,2]	total=6	< 7, stop shrinking, grow
right=4	[3,1,2,4]	total=10	≥ 7 → record 4, drop 3, left=2
	[1,2,4]	total=7	≥ 7 → record 3, drop 1, left=3
	[2,4]	total=6	< 7, stop shrinking, grow
right=5	[2,4,3]	total=9	≥ 7 → record 3, drop 2, left=4
	[4,3]	total=7	≥ 7 → record 2, drop 4, left=5
	[3]	total=3	< 7, stop

answer 2

What to notice: `left` moves 5 times in total across the whole run, and `right` moves 6. Eleven moves for six elements — that is the $2n$ bound made concrete, and it is why the nested `while` does not make it quadratic.

Shape B, maximising. `s = "abcabcbb"`, no repeats allowed:

```

right=0 [a]      ok      best=1
right=1 [ab]     ok      best=2
right=2 [abc]    ok      best=3
right=3 [abca]   'a' repeats → shrink: drop 'a', left=1
          [bca]    ok      best=3
right=4 [bcab]   'b' repeats → shrink: drop 'b', left=2
          [cab]    ok      best=3
right=5 [cabcb]  'c' repeats → shrink: drop 'c', left=3
          [abc]    ok      best=3
right=6 [abcb]   'b' repeats → shrink: drop 'a', left=4
          [bcb]    'b' repeats → shrink: drop 'b', left=5
          [cb]     ok      best=3
right=7 [cbb]    'b' repeats → shrink: drop 'c', left=6
          [bb]     'b' repeats → shrink: drop 'b', left=7
          [b]      ok      best=3

```

What to notice at `right = 6`: the shrink loop runs twice in one turn. That looks expensive and is not, because `left` has moved two positions it will never revisit.

The two shapes side by side — this is the picture to memorise:

MINIMISE (shape A)		MAXIMISE (shape B)
-----		-----
for right:		for right:
add(right)		add(right)
while VALID:		while INVALID:
record(right - left + 1)	←	remove(left); left++
remove(left); left++		record(right - left + 1) ←
 record INSIDE the shrink loop		record AFTER the shrink loop
shrink while it still works		shrink until it works again

Why `left` never goes back — Devappa's own argument:

window `[left .. right]` is the smallest valid one ending at `right`.

Now `right` moves to `right+1`. Could the answer start EARLIER than `left`?
 No: any window starting before `left` and ending at `right+1` CONTAINS `[left .. right+1]`, so it is longer. For a minimisation problem, longer is worse. So there is nothing behind `left` worth revisiting.

5 The code, built step by step

Shape A: minimum size subarray sum

```
left = 0
total = 0
best = float("inf")
```

`best` starts at infinity, not at 0 or at `len(nums)`, because you need a value that any real answer beats and a way to detect “no answer at all”.

```
for right, x in enumerate(nums):
    total += x
```

The right edge always advances, unconditionally. That is the outer loop and it never has a condition of its own.

```
while total >= target:
    best = min(best, right - left + 1)
    total -= nums[left]
    left += 1
```

Record first, then shrink. The window is valid right now, so its length is a candidate; then remove the leftmost element and check again. `while` and not `if`, because several elements may need to come off — look at `right = 4` in §4, where two come off in one turn.

```
return 0 if best == float("inf") else best
```

The problem’s convention for “no such subarray” is 0. **Ask** — some versions want `-1`.

Shape B: longest substring without repeats

```
seen: set[str] = set()
left = 0
best = 0
for right, ch in enumerate(s):
    while ch in seen:
        seen.discard(s[left])
        left += 1
    seen.add(ch)
    best = max(best, right - left + 1)
```

Note the order inside the loop: **shrink first, then add**. If you added `ch` before the `while`, the condition `ch in seen` would be true because of the character you just added, and `left` would run all the way to `right`.

And the recording is **after** the shrink loop, because only there is the window guaranteed valid.

Shape B with a count map: at most k distinct

```
count: defaultdict[str, int] = defaultdict(int)
left = 0
best = 0
for right, ch in enumerate(s):
    count[ch] += 1
    while len(count) > k:
        count[s[left]] -= 1
        if count[s[left]] == 0:
            del count[s[left]]          # or len(count) never goes down
        left += 1
    best = max(best, right - left + 1)
```

The `del` at zero is what makes `len(count)` mean “distinct characters in the window”. Leave the zero entries in and the count never decreases, the shrink loop never terminates properly, and the answer is wrong. This is the same rule as [day 031](#), and here it is not merely a comparison failure — it breaks the loop condition itself.

Shape A with a count map: minimum window substring

The hardest of the four, and the trick is the `missing` counter that makes the validity check `0(1)` rather than a dictionary comparison.

```
need = Counter(t)
missing = len(t)          # how many characters of t are still unmatched
left = 0
best = (float("inf"), 0, 0)
```

```
for right, ch in enumerate(s):
    if need[ch] > 0:
        missing -= 1      # only counts if we still NEEDED this character
    need[ch] -= 1         # goes negative for surplus characters
```

`need[ch]` going negative is deliberate: it records how many spare copies of `ch` the window holds, which is exactly what the shrink step needs to know.

```
if missing == 0:
    while need[s[left]] < 0:      # trim surplus characters off the front
        need[s[left]] += 1
        left += 1
    if right - left + 1 < best[0]:
        best = (right - left + 1, left, right)
    need[s[left]] += 1          # give up one needed character
    missing += 1                # and stop being valid
    left += 1
```

The last three lines are the shape-A shrink: having recorded the best window, deliberately break validity by removing one *needed* character, so the outer loop goes back to growing.

The complete solutions

```
from collections import Counter, defaultdict

def min_subarray_len(target: int, nums: list[int]) → int:
    """LeetCode 209. Shortest subarray with sum ≥ target. POSITIVE values only.

    Shape A: grow, then shrink while still valid, recording inside the shrink.
    """
    left = 0
    total = 0
    best = float("inf")
    for right, x in enumerate(nums):
        total += x
        while total ≥ target:
            # while STILL valid
            best = min(best, right - left + 1) # record BEFORE shrinking
            total -= nums[left]
            left += 1
    return 0 if best == float("inf") else best

def longest_unique(s: str) → int:
    """LeetCode 3. Shape B: shrink while invalid, record after."""
    seen: set[str] = set()
    left = 0
    best = 0
    for right, ch in enumerate(s):
        while ch in seen:
            # shrink BEFORE adding
            seen.discard(s[left])
            left += 1
        seen.add(ch)
        best = max(best, right - left + 1) # record AFTER restoring validity
    return best

def longest_at_most_k_distinct(s: str, k: int) → int:
    """LeetCode 340. Shape B with a count map."""
    count: defaultdict[str, int] = defaultdict(int)
    left = 0
    best = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best

def character_replacement(s: str, k: int) → int:
    """LeetCode 424. Valid when (window length - most frequent count) ≤ k.

    max_freq is never decreased on shrink, and that is still correct: a stale
    high value can only stop the window growing, never produce a larger answer.
    """
    count: defaultdict[str, int] = defaultdict(int)
    left = 0
    best = 0
    max_freq = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        max_freq = max(max_freq, count[ch])
        while (right - left + 1) - max_freq > k:
```

```

        count[s[left]] -= 1
        left += 1
        best = max(best, right - left + 1)
    return best

def min_window(s: str, t: str) → str:
    """LeetCode 76. Smallest window of s containing every character of t.

    `missing` makes the validity check O(1) instead of comparing two maps.
    Negative values in `need` record surplus characters.
    """
    if not t or not s:
        return ""
    need = Counter(t)
    missing = len(t)
    left = 0
    best = (float("inf"), 0, 0)

    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
            need[ch] -= 1

        if missing == 0:
            while need[s[left]] < 0:          # trim surplus off the front
                need[s[left]] += 1
                left += 1
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            need[s[left]] += 1                # break validity deliberately
            missing += 1
            left += 1

    return "" if best[0] == float("inf") else s[best[1]:best[2] + 1]

if __name__ == "__main__":
    print([min_subarray_len(t, a) for t, a in
           ((7, [2, 3, 1, 2, 4, 3]), (4, [1, 4, 4])),
           (11, [1, 1, 1, 1, 1, 1, 1, 1]), (1, [1]), (5, [1]))])
    # [2, 1, 0, 1, 0]

    print([longest_unique(x) for x in
           ("abcbcb", "bbbb", "pwwkew", "", "au", "dvdf", "abba")])
    # [3, 1, 3, 0, 2, 3, 2]

    print([longest_at_most_k_distinct(s, k) for s, k in
           (("eceba", 2), ("aa", 1), ("abaccc", 2), ("", 2), ("abc", 0))])
    # [3, 2, 4, 0, 0]

    print([character_replacement(s, k) for s, k in
           (("ABAB", 2), ("AABABBA", 1), ("AAAA", 0), ("", 1))])
    # [4, 4, 4, 0]

    print([min_window(s, t) for s, t in
           (("ADOBECODEBANC", "ABC"), ("a", "a"), ("a", "aa"), ("ab", "b"))])
    # ['BANC', 'a', '', 'b']

```

6 What it costs

Every problem on this page

`right` advances exactly `n` times. `left` only advances and never passes `right`, so it advances at most `n` times across the entire run.

at most $2n$ pointer moves, each doing constant work $\rightarrow O(n)$ time

Space depends on what the window carries: $O(1)$ for a running sum, $O(k)$ for a set or map bounded by the alphabet or by `k`.

The nested `while` does not make it quadratic, and the argument is that a single turn of the `for` loop may run the `while` many times, but those moves are permanently spent — `left` never returns. Total work across all turns is bounded by how far `left` can travel, which is `n`.

Against the brute force

The brute force tries every start and extends until the condition is met: $O(n^2)$.

Measured, on inputs where no window qualifies so both do their full work:

<code>n = 2,000</code>	<code>window 0.00009 s</code>	<code>brute 0.0800 s</code>	854x
<code>n = 5,000</code>	<code>window 0.00025 s</code>	<code>brute 0.5268 s</code>	2,100x
<code>n = 10,000</code>	<code>window 0.00048 s</code>	<code>brute 2.0750 s</code>	4,350x

The ratio roughly doubles when `n` doubles — 854, 2,100, 4,350 — which is the $O(n)$ versus $O(n^2)$ signature. At `n = 10,000` that is about 10,000 operations against 50 million.

`min_window`, and why the `missing` counter matters

Without it, checking validity means comparing two dictionaries on every step. That is $O(k)$ where `k` is the alphabet size — constant, but with a real constant factor, and it is done `n` times.

With `missing`, validity is `missing == 0`: a single integer comparison. Same complexity class, several times faster in practice, and it is the difference between a solution that reads well and one that looks laboured.

$O(|s| + |t|)$ time, $O(|t|)$ space.

`character_replacement`, both ways

Recomputing `max(count.values())` inside the shrink loop is $O(26)$ per step — constant for a fixed alphabet, so the whole thing stays $O(n)$. Carrying `max_freq` and never decreasing it removes that constant entirely.

Both are $O(n)$. The trick is a constant-factor improvement with a subtle correctness argument, so in an interview: write the simple version, get it right, then offer the optimisation with its justification.

The number to have ready

Both edges only ever move forward, so each index is visited at most twice — $2n$ moves, $O(n)$ time. The brute force is $O(n^2)$: measured at about 4,350 times slower at ten thousand elements, with the ratio doubling every time n doubles.

7 The traps

The big one: negative numbers break the whole technique

```
print(min_subarray_len(3, [-3, 5]))
```

```
0
```

0 means “no such subarray exists”. But [5] sums to 5, which is at least 3, so the answer is 1. The window silently reported that nothing works.

Here is what happened. After `right = 0` the total is -3, below target, so nothing shrinks. At `right = 1` the total is 2 — still below target — so the loop ends having never been valid. The 5 was never considered on its own, because the window never dropped the -3 off the front.

That is the general failure. Shape A’s shrink loop assumes **removing an element from the left can only decrease the sum**, so “not valid yet” means “keep growing”. With negatives, removing a negative element *increases* the sum, so a window that looks hopeless can be fixed by shrinking — which the algorithm never tries. Testing over twenty thousand random small arrays containing negatives, this version disagrees with the brute force about **12% of the time**, sometimes reporting no answer at all and sometimes reporting one that is too long — [-1,-3,2,4,1,4] with target 6 gives 3 where the answer is 2.

Sliding windows require the condition to be monotonic as the window grows. For sums, that means non-negative values. The correct technique for negatives is prefix sums plus a monotonic deque — LeetCode 862 — and it is a genuinely harder problem. Say “does the array contain negative numbers?” before writing a window over sums.

The near-miss: recording in the wrong place

```
for right, x in enumerate(nums):
    total += x
    while total ≥ target:
        total -= nums[left]
        left += 1
    best = min(best, right - left + 1)    # recorded AFTER the shrink
```

By the time this records, the shrink loop has already broken validity — it exits when the sum has dropped *below* target — so it measures an invalid window. On `[2,3,1,2,4,3]` with target 7 it returns 1, which is wrong.

Minimise: record inside the shrink loop, before shrinking. Maximise: record after. Getting these backwards is the single most common structural error in this family, and the code runs perfectly.

The near-miss: `if` instead of `while`

```
if total ≥ target:                # should be while
```

On `[1, 1, 1, 10]` with target 4, the window needs to shrink three positions in one turn and this shrinks one. The answer comes out too large, with no error. **Look at `right = 4` in §4** — two removals in one turn is normal, not exceptional.

The near-miss: adding before shrinking in shape B

```
seen.add(ch)
while ch in seen:                # always true – we just added it
    seen.discard(s[left])
    left += 1
```

`left` runs all the way up to `right` every time, and the answer is always 1. **Shrink first, then add.**

The near-miss: `left` moving backwards

```
if ch in last:                    # missing: and last[ch] ≥ left
    left = last[ch] + 1
```

On `"abba"`: at the final `a`, `last["a"]` is 0, which is *before* the current `left` of 2 — so `left` jumps backwards to 1, the window contains a repeated `b`, and the answer is 3 instead of 2.

It also destroys the `O(n)` argument, since `left` no longer travels monotonically. Guard with `last[ch] ≥ left`, or write `left = max(left, last[ch] + 1)`, which says the intent plainly. **The while-based version cannot have this bug at all**, because it only increments.

The near-miss: forgetting the `del` at zero

```
count[s[left]] -= 1
# no deletion at zero
left += 1
```

`len(count)` now counts characters that are no longer in the window, so it never drops back to `k`, the shrink loop keeps running, and `left` marches to the end. The answer becomes 1 or 0. **Here the missing `del` breaks the loop condition itself**, which is worse than the equality-comparison failure it caused yesterday.

The contract corner: what “no answer” returns

`min_subarray_len(11, [1,1,1,1,1,1,1,1])` — the whole array sums to 8, so no window qualifies. LeetCode 209 wants `0`. Other phrasings want `-1`. Returning `float("inf")` because you forgot to translate it is a real and easy mistake. **Decide and say it.**

8 In the interview

How it gets asked

- “Find the smallest subarray with a sum at least k .” — LeetCode 209, shape A.
- “Longest substring without repeating characters.” — LeetCode 3, shape B, and the most-asked window problem there is.
- “Longest substring with at most k distinct characters.” — LeetCode 340, shape B with a map.
- “Minimum window substring.” — LeetCode 76, shape A with a map, and genuinely hard.
- “What if the array has negative numbers?” — the question that checks whether you know the boundary.

What to say out loud, in the first ninety seconds

1. **Ask the question that decides legality.** “Are all the values positive? A sliding window needs the sum to only increase as I extend the window, and negatives break that.”

2. **Name the shape and say which of the two it is.** *“This is a minimisation, so I’ll grow the window until it’s valid and then shrink from the left while it’s still valid, recording the length each time.”*
3. **Say the invariant.** *“The window from left to right is always the shortest valid one ending at right.”*
4. **Say why `left` never goes back.** *“Any window starting earlier and ending here contains this one, so it’s longer — and for a minimisation, longer is worse. There’s nothing behind left worth revisiting.”*
5. **Say where the recording goes and why.** *“The record goes inside the shrink loop, before I remove an element, because that’s the moment the window is valid.”*
6. **Give the cost with the argument.** *“Both pointers only move forward, so each index is visited at most twice — $2n$ moves, $O(n)$ — even though there’s a `while` inside a `for`.”*
7. **Name the boundary cases.** *“Empty array, and no window ever reaching the target — I’ll return 0 for that, though I’d check the convention.”*

The follow-ups

“What if the array can contain negative numbers?” Then a sliding window is the wrong technique, and I would say so rather than adapt it. The window depends on monotonicity: extending it can only increase the sum, and shrinking can only decrease it, which is what makes “shrink while still valid” safe. With negatives, adding an element can decrease the sum and removing one can increase it — so a window I rejected might have become valid later, and a window I shrank might have had a shorter valid version I never saw. The result is a wrong answer with no error, which is the dangerous kind. The correct approach is prefix sums plus a monotonic deque: build the prefix sums, and for each right endpoint find the largest earlier prefix that is at least `target` smaller, maintaining a deque of increasing prefix values. That is LeetCode 862 and it is a hard problem rather than a variation.

“Why is this $O(n)$ when there’s a while loop inside a for loop?” Because the right pointer advances exactly n times and the left pointer also advances at most n times across the entire run — it only ever increases and it can never pass the right pointer. So the total number of pointer movements is bounded by $2n$, and every unit of work is attached to a movement. A single turn of the outer loop might run the inner `while` many times, but each of those iterations spends a move that will never be repeated, so it is paid for. The way I check this in general is to ask whether either pointer can move backwards — if neither can, it is linear; if either can, as in the last-seen-map version without a guard, the argument collapses along with the correctness.

“Now find the minimum window in `s` containing all characters of `t`, including duplicates.” Same shape A, with two refinements. The window carries a count map initialised to what `t` needs, and I decrement it as characters arrive — letting it go negative, which records surplus copies. And rather than comparing two maps to check validity, I keep a single `missing` counter of how many required characters are still unmatched, decremented only when I consume a character that was still needed. So validity is `missing == 0`, an integer comparison rather than a dictionary comparison. When valid, I first trim surplus characters off the front — those with negative counts — then record, then deliberately remove one *needed* character to break validity so the window grows again. $O(|s| + |t|)$ time, $O(|t|)$ space.

“How do you decide whether it’s shape A or shape B?” By what the question is optimising. If it asks for the **smallest** window satisfying something, I grow until it is satisfied and then shrink while it is *still* satisfied, recording inside the shrink loop — because each shrink gives a shorter valid candidate. If it asks for the **largest** window satisfying something, I grow and shrink only while the window is *broken*, recording after the shrink loop, because that is the first moment the window is valid again. Stated as a rule: minimise means shrink-while-valid and record inside; maximise means shrink-while-invalid and record after. Swapping them produces code that runs and returns plausible wrong answers, which is why I say which one I am doing before writing it.

A model answer

“First, are all the values positive? That decides whether a sliding window is even legal here — the technique relies on the sum only increasing as I extend the window, and negative numbers break that.

...All positive. Good.

The brute force is to try every starting position and extend until the sum reaches the target, which is $O(n^2)$.

Instead I'll use a window with two edges that both only move forward. The right edge advances unconditionally, adding to a running total. The moment the total reaches the target, the window is valid, so I record its length — and then I shrink from the left while it is *still* valid, recording each shorter version, until removing another element would drop below the target.

```
def min_subarray_len(target: int, nums: list[int]) → int:
    left = 0
    total = 0
    best = float("inf")
    for right, x in enumerate(nums):
        total += x
        while total ≥ target:
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best == float("inf") else best
```

Two structural points. The record goes *inside* the shrink loop and *before* the removal, because that is the moment the window is valid — putting it after the loop measures a window that has just been broken, and returns an answer that is too small. And it is a `while`, not an `if`, because several elements may need to come off in one turn: on `[1,1,1,10]` with target 4 the left edge moves three places when the 10 arrives.

The left edge never goes backwards, and the reason is worth stating: any window starting before the current left and ending at the current right *contains* the current window, so it is longer — and for a minimisation, longer is worse. There is nothing back there to reconsider.

That also gives the cost. The right pointer advances n times and the left pointer at most n times, so at most $2n$ moves total — $O(n)$ time, $O(1)$ space, even though there is a `while` inside a `for`. I measured it against the brute force: about four thousand times faster at ten thousand elements, with the ratio doubling each time n doubles.

Edge cases: an empty array, and the case where no window ever reaches the target — the convention there is to return 0, and I'd confirm that."

9 Recall card

- **Right edge always advances. Left edge advances only when the rule says so.** Neither ever goes back — so $2n$ moves, $O(n)$.
- **Minimise → shrink while VALID, record INSIDE the loop. Maximise → shrink while INVALID, record AFTER.** Swapping them gives plausible nonsense.
- **The condition must be monotonic.** Negative numbers break sum-windows — that needs prefix sums plus a deque instead. Ask first.
- **Shape B: shrink before adding**, or the character you just added triggers the shrink.
- **With a count map, del at zero** — here it breaks the loop condition, not just a comparison.

1 What this is, and why they ask it

SQL is declarative — from [day 027](#), you say *what* you want and the database decides *how*. The thing that decides is the **query planner**, and the decision it produces is the **query plan**: a tree of physical operations, chosen from many possible ones, that together answer your query.

The planner chooses by **estimating**. It keeps statistics about each table — how many rows, how many distinct values in each column, the most common values — and uses them to guess how many rows each step will produce. Then it costs the alternatives and picks the cheapest.

Which means the plan can be wrong in exactly one way: **the estimates can be wrong**. A planner that believes a filter matches one row will happily choose a strategy that is catastrophic when it actually matches half a million. And the way you find out is `EXPLAIN ANALYZE`, which shows the estimate and the reality side by side.

Interviewers ask this because it is the most practical database skill there is. Every backend engineer meets a slow query in their first month. The bad answer is “add an index”, offered before looking. The good answer is a **procedure**: run `EXPLAIN ANALYZE`, read the plan from the inside out, find where the row counts explode or the estimate diverges from reality, and fix *that*. Roughly half the time it is not a missing index at all.

2 The story

Ramanathan does the errands on Saturday morning because his wife works, and he has got it down to a system.

He works out the order before he leaves. Bank first, because he thinks that takes ten minutes; then the electricity office, twenty; then the vegetable market on the way back, fifteen; then the tailor, five. Forty-five minutes plus the riding, back by ten, and he plans the route around that order so he is never doubling back.

The plan is a good plan. It is the right order given what he believes.

The Saturday it went wrong, he got to the bank at half past eight and there were about seventy people outside it, because it was the first Saturday of the month and everybody's pension had come in. He was in that queue for an hour and forty minutes. By the time he

got out, the electricity office had shut for lunch, so he had to go back in the afternoon, and by then the tailor had closed as well.

The thing he says about it, and he is right, is that the plan was not the mistake. Ten minutes for the bank was the mistake. Everything downstream — the order, the route, the timing — followed correctly from a number that happened to be wrong by a factor of ten.

His son asked him afterwards how he could have known, and Ramanathan said the honest thing: he couldn't have, not from where he was standing. But he could have found out. The bank has a board outside with the pension dates on it and he has walked past it four hundred times.

Now he does two things. Before he leaves, he checks the two or three things that change — is it pension week, is there a strike, is it the end of the quarter at the electricity office. And afterwards, if it took longer than he thought, he makes himself say where the time actually went, out loud, rather than just saying it was a bad morning. He says the second one has taught him more, because his guesses about the tailor were wrong for six years and he never once noticed.

3 The idea in plain English

Ramanathan's order of errands is the **query plan**. His ten-minutes-for-the-bank is a **row estimate**. Checking the pension board before leaving is **ANALYZE**, which refreshes the statistics. And making himself say where the time actually went is **EXPLAIN ANALYZE**.

What the planner is actually doing

For a query like `SELECT ... FROM a JOIN b ... WHERE ...`, the planner must choose:

- **How to read each table** — sequential scan, index scan, index-only scan, bitmap scan.
- **Which join algorithm** — nested loop, hash join, merge join, from [day 028](#).
- **In what order to join** — with three tables there are twelve possible orderings, and the choice can change the cost by orders of magnitude.
- **Where to filter and where to sort.**

It enumerates the plausible combinations, assigns each a **cost** — an abstract number, not milliseconds — and picks the smallest. The cost model weighs sequential page reads, random page reads, CPU per row and CPU per operator.

Everything depends on knowing how many rows each step produces. That comes from the statistics.

The statistics

Postgres stores, per column: the number of distinct values, the fraction that are null, a list of the most common values with their frequencies, and a histogram of the rest. They are refreshed by `ANALYZE`, which autovacuum runs automatically — but not instantly, and not after a bulk load unless you ask.

From those numbers the planner estimates **selectivity**: what fraction of rows a condition keeps.

```
WHERE status = 'pending'
  status has 4 distinct values, 'pending' is 3% by the most-common-values list
  → estimate 3% of 1,000,000 = 30,000 rows
```

Stale statistics are the single most common cause of a bad plan, and the fix is one command.

Reading a plan

Two rules make plans readable, and without them they are impenetrable.

Read from the inside out — the most indented lines run first. The output is a tree printed with the root at the top, so execution goes bottom-up.

Every line has the same shape:

```
Node Type (cost=START..TOTAL rows=EST width=BYTES)
        (actual time=START..TOTAL rows=ACTUAL loops=N)
```

- `cost=0.42..8.45` — the planner's abstract estimate. The first number is the cost to produce the *first* row, the second to produce *all* of them. The first number matters for `LIMIT` queries.
- `rows=` in the cost line is the **estimate**; `rows=` in the actual line is the **truth**.
- `loops=N` — this node ran N times. **The actual time is per loop, so multiply.** This is the single most misread field in the whole output.

`EXPLAIN` shows only the estimates. `EXPLAIN ANALYZE` **actually runs the query** and adds the actual line — which means it also executes any `UPDATE` or `DELETE`, so wrap those in a transaction you roll back.

The four things to look for, in order

1. Estimate versus actual. If `rows=1` estimated and `rows=480000` actual, everything downstream was chosen for the wrong problem. This is the first thing to check and the most valuable. Fix: `ANALYZE`, or increase the statistics target on that column.

2. A `Seq Scan` on a large table with a filter. Look for `Rows Removed by Filter`. Reading a million rows to return three is a missing index.

3. `loops=` greater than one on a large node. A nested loop running its inner side sixty-one times is often an ORM N + 1 problem or a bad join order.

4. Anything that says `Disk`. `Sort Method: external merge Disk: 52000kB` means the operation exceeded `work_mem` and spilled. Either raise `work_mem` for that session or give it an index that provides the order.

The scan nodes, and what each means

NODE	WHAT IT IS DOING	WHEN IT IS RIGHT
<code>Seq Scan</code>	read every page of the table	small tables, or when returning most rows
<code>Index Scan</code>	walk the index, then fetch each row	few matching rows
<code>Index Only Scan</code>	answered from the index alone	the index covers every column needed — the best case
<code>Bitmap Heap Scan</code>	collect row locations, sort them, read pages in order	many matches — turns random reads into sequential ones

`Bitmap Heap Scan` is not a problem. People see it and assume something is wrong; it is the planner being clever in the middle ground where an index scan would do too many random reads and a sequential scan would read too much.

The six causes of a slow query

The list from [day 030](#), now with what each looks like in a plan:

CAUSE	IN THE PLAN	FIX
missing index	<code>Seq Scan</code> + high <code>Rows Removed by Filter</code>	add the index
function on the column	<code>Seq Scan</code> despite an index existing	rewrite as a range, or index the expression
stale statistics	estimated rows wildly different from actual	<code>ANALYZE</code>
N + 1 from an ORM	each plan looks fine; there are just 61 of them	eager loading — not visible in <code>EXPLAIN</code>
deep <code>OFFSET</code>	<code>Limit</code> with a huge <code>rows</code> beneath it	keyset pagination
spilling to disk	<code>Sort Method: external merge Disk:</code>	raise <code>work_mem</code> , or index the sort column

Two of those six do not appear in the plan at all. The N + 1 is invisible because each individual query is fine, and you find it in the application log or in `pg_stat_statements`. That is worth saying in an interview, because it is the one people never mention.

4 The picture

A plan, annotated:

```
Sort (cost=1250.30..1252.80 rows=1000 width=68)
  (actual time=45.2..45.6 rows=987 loops=1)                                ← runs LAST
  Sort Key: o.created_at DESC
  Sort Method: quicksort Memory: 128kB                                    ← in memory: good
  → Hash Join (cost=25.10..1200.45 rows=1000 width=68)
    (actual time=0.8..42.1 rows=987 loops=1)
    Hash Cond: (o.customer_id = c.id)
    → Seq Scan on orders o (cost=0.00..1100.00 rows=50000 width=40)
      (actual time=0.01..20.4 rows=50000 loops=1)
      Filter: (status = 'completed')
      Rows Removed by Filter: 950000                                     ← READ THIS
    → Hash (cost=20.00..20.00 rows=400 width=32)
      (actual time=0.7..0.7 rows=400 loops=1)
      → Seq Scan on customers c (rows=400)                               ← runs FIRST

read bottom-up: scan customers → build hash → scan orders → join → sort
```

What to notice: `Rows Removed by Filter: 950000`. The orders scan read a million rows and threw away 95% of them. That is the line that tells you where the 20 milli-seconds went, and it is the argument for an index on `status` — or better, a partial index on the pending rows only.

Estimate against actual, and why a bad estimate poisons everything downstream:

BAD ESTIMATE	WHAT ACTUALLY HAPPENED
Nested Loop (rows=1)	Nested Loop (actual rows=480000)
→ Index Scan on a (rows=1)	→ Index Scan on a (actual rows=480000)
→ Index Scan on b (rows=1 loops=1)	→ Index Scan on b (loops=480000) ← !!
"one row from a, so one lookup in b – a nested loop is obviously cheapest"	480,000 index lookups instead of one. A hash join would have been right.
The plan was correct given the estimate. The estimate was wrong.	

What to notice: this is Ramanathan’s bank queue exactly. The plan followed correctly from a number that was wrong by a factor of half a million, and the fix is not to change the plan — it is to fix the number, with `ANALYZE`.

The `loops` trap:

→ Index Scan on orders (actual time=0.02..0.05 rows=3 loops=61)

"0.05 ms – that's fine" WRONG

actual time is PER LOOP.

real total = 0.05 x 61 = 3.05 ms, and 61 round trips.

loops=61 on a table you expected to touch once

almost always means an N+1 or a nested loop over too many rows.

5 How it actually works

The procedure, in order

1. Find the right query first. Do not optimise the one somebody complained about; optimise the one consuming the most time.

```
SELECT query, calls, total_exec_time, mean_exec_time
FROM   pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

`total_exec_time`, not `mean_exec_time`. A 20 ms query called a million times a day costs far more than a 2-second report run nightly, and the 20 ms one is usually the real problem. Sorting by the mean is how people spend a week optimising something that runs once.

2. EXPLAIN ANALYZE it, with `BUFFERS` for the I/O picture:

```
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
```

Buffers: shared hit=12 read=8340 — “hit” came from the cache, “read” came from disk. A high `read` count is where the time is.

3. Read bottom-up and find the biggest `actual rows` and the biggest `actual time`.

4. Check estimate against actual at every level. The first place they diverge badly is usually the root cause; everything above it inherited the mistake.

5. Apply the specific fix, then re-run and compare.

Fixing each cause

Stale statistics.

```
ANALYZE orders;
ALTER TABLE orders ALTER COLUMN status SET STATISTICS 1000; -- default is 100
```

Raising the statistics target makes the histogram finer, which helps on columns with skewed distributions. Postgres also supports **extended statistics** for correlated columns — if `city` and `pincode` are related, the planner otherwise multiplies their selectivities as if independent and underestimates badly:

```
CREATE STATISTICS orders_city_pin (dependencies) ON city, pincode FROM orders;
```

A function on the column.

```
-- before: Seq Scan, even though created_at is indexed
WHERE DATE(created_at) = '2026-03-01'
-- after: Index Scan
WHERE created_at ≥ '2026-03-01' AND created_at < '2026-03-02'
```

Or index the expression: `CREATE INDEX ON orders (DATE(created_at));`

Deep pagination.

```
-- before: Limit over a node with rows=1000020
LIMIT 20 OFFSET 1000000
-- after
WHERE (created_at, id) < (:last_created_at, :last_id)
ORDER BY created_at DESC, id DESC
LIMIT 20
```

The row-comparison syntax `(a, b) < (x, y)` handles the tie-break correctly in one condition, which is neater than the `OR` version people usually write.

Spilling to disk.

```
SET work_mem = '64MB'; -- for this session only
```

Or add an index that provides the order, so the sort disappears entirely rather than being made faster.

The N + 1. Not visible in any plan. You find it by counting queries per request in the application log, and you fix it in the ORM with eager loading — `select_related` / `prefetch_related` in Django, `includes` in Rails, `joinedload` in SQLAlchemy.

Making the planner do something else

Postgres deliberately has no query hints. What you can do:

- `ANALYZE` — fix the estimates, which fixes the plan properly.
- **Rewrite the query** — `EXISTS` instead of `IN`, a lateral join, a CTE.
- `SET enable_seqscan = off` — a diagnostic sledgehammer, useful to ask “what would you do otherwise, and what did you think it would cost?” Never leave it on in production.
- **Materialise a CTE deliberately** — in Postgres 12+, `WITH x AS MATERIALIZED (...)` forces the old behaviour of computing it once.

The absence of hints is a design position: the Postgres view is that a hint freezes a decision that should adapt as the data changes. Oracle and SQL Server disagree and provide them. Knowing that this is a deliberate disagreement rather than a missing feature is a good signal.

`EXPLAIN` versus `EXPLAIN ANALYZE`

`EXPLAIN` plans without running: instant, safe, estimates only. `EXPLAIN ANALYZE` **executes the query** — so it takes as long as the query does, and on an `UPDATE` or `DELETE` it really updates or deletes. Wrap it:

```
BEGIN;
EXPLAIN ANALYZE DELETE FROM orders WHERE id = 5;
ROLLBACK;
```

6 The numbers

What a bad estimate costs

10,000 customers, 1,000,000 orders. The planner estimates a filter yields 1 row; it actually yields 480,000.

```
plan chosen: nested loop
  estimated : 1 outer row x 1 index lookup = 1 lookup      ≈ 0.02 ms
  actual    : 480,000 outer rows x 1 lookup = 480,000 lookups
              480,000 × 0.02 ms                      ≈ 9.6 seconds

plan it would have chosen with correct estimates: hash join
  build hash on 10,000 rows + scan 1,000,000              ≈ 0.3 seconds
```

Thirty times slower, from one wrong number. `ANALYZE` takes a few seconds and fixes it. This is the most important arithmetic in the lesson.

Missing index

1,000,000 rows × 200 bytes = 200 MB = ~25,000 pages

Seq Scan	: read 25,000 pages, discard 999,997 rows	≈ 112 ms
Index Scan	: 3 index pages + 3 row pages = 6 reads	≈ 0.02 ms
		~5,000x

The `loops` multiplication

Index Scan on orders (actual time=0.02..0.05 rows=3 loops=61)

misread : 0.05 ms

actual : 0.05 × 61 = 3.05 ms of database time

plus : 61 network round trips at ~0.5 ms = 30 ms of latency

one query with a join instead: ~2 ms and one round trip
→ about 15x on wall-clock

Spilling to disk

sort 1,000,000 rows × 100 bytes = 100 MB

work_mem = 4 MB (default)

in memory : quicksort, ~20,000,000 comparisons ≈ 1.2 s

external : 25 passes writing and reading 100 MB ≈ 8-15 s
~10x

raise work_mem to 128 MB : back to ~1.2 s

add an index on the sort column : the sort disappears entirely ≈ 0 s

Choosing which query to fix

Query A : 2,000 ms × 1 call/day = 2 seconds/day

Query B : 20 ms × 1,000,000/day = 20,000 seconds/day = 5.6 hours

Query B costs ten thousand times more, and it is the one nobody complains about because each individual call feels fast. Sorting `pg_stat_statements` by `total_exec_time` is what surfaces it.

OFFSET

LIMIT 20 OFFSET 0 : produce 20 rows ≈ 1 ms

LIMIT 20 OFFSET 1,000,000: produce 1,000,020, keep 20 ≈ 2,000 ms

keyset : index seek + 20 rows ≈ 1 ms at any depth

7 The trade-offs

Fix the query, the index, or the schema

In increasing order of cost and permanence:

- **Rewrite the query** — free, reversible, and it is the right answer for a function on a column, an `OFFSET`, or a `SELECT *` defeating a covering index.
- **Add an index** — permanent write cost and 10–25% storage, from [day 030](#). Right when the query shape is stable and common.
- **Change the schema** — denormalise, add a materialised view, partition. Expensive and hard to undo. Only when the first two are not enough, and with the reconciliation job from [day 029](#).

Always try them in that order. Reaching for the schema first is the classic mistake.

Raise `work_mem`, or add an index

Raising it fixes the sort spill immediately, and it is **per operation, per connection** — set it to 256 MB globally with 200 connections each running two sorts and you have promised 100 GB you do not have. Set it per session or per role for the queries that need it. An index that provides the order removes the sort entirely and costs write throughput instead.

`EXPLAIN` or `EXPLAIN ANALYZE`

`EXPLAIN` is instant and safe and only shows guesses — which is exactly the thing you are trying to verify. `EXPLAIN ANALYZE` shows the truth and runs the query, so on a query that takes four minutes it takes four minutes, and on a write it really writes. Use `ANALYZE` **inside a rolled-back transaction**, and use plain `EXPLAIN` first when you suspect the query may be catastrophic.

Optimise the mean or the total

Mean time finds the query that is individually painful. Total time finds the query that is actually costing you the machine. **Sort by total** — but keep an eye on the mean too, because a query with a two-second mean is a user-visible problem even if it runs rarely.

I would not add an index before running `EXPLAIN ANALYZE`, and I would look at the estimated row counts before I look at the scan types. A plan is only ever as good as the numbers it was built from, and the most expensive failure in this area is not a missing index — it is a planner that believed a step would return one row when it returned half a million, chose a nested loop on that basis, and did half a million index lookups. The plan was correct for the problem it thought it had. The fix there is `ANALYZE`, which costs seconds, and adding an index would not have helped at all.

8 In the interview

How it gets asked

- “Here’s an `EXPLAIN` plan. What’s wrong with this query?” — the direct version, with a plan on the board.
- “This endpoint got slow last week and nothing was deployed. What happened?” — usually data growth crossing a threshold, or stale statistics after a bulk load.
- “How do you find the slow queries in the first place?” — `pg_stat_statements`, sorted by total time.
- “What does `loops=61` mean?” — the field everybody misreads.
- “Why is the planner choosing a sequential scan when there’s an index?” — selectivity, a function on the column, or stale statistics.

What to say out loud, in the first ninety seconds

1. **Say the procedure before any diagnosis.** “I’d run `EXPLAIN ANALYZE` on it — with `BUFFERS` — and read the plan from the inside out, since the most indented nodes execute first.”
2. **Say what you check first, and it is not the scan type.** “The first thing I compare is estimated rows against actual rows at each level. If they diverge badly, everything above that point was chosen for the wrong problem, and no amount of indexing fixes it — that’s an `ANALYZE`.”
3. **Then the scan.** “Then I look for a `Seq Scan` on a large table with a high `Rows Removed by Filter` — reading a million rows to return three is a missing index.”
4. **Then loops.** “Then `loops` greater than one on a big node, remembering that `actual time` is per loop, so it multiplies. `loops=61` usually means an $N+1$ or a nested loop over too many rows.”

5. **Then disk.** “And anything mentioning `Disk` — a sort spilling means the operation exceeded `work_mem`.”
6. **Name the causes that are invisible in the plan.** “Two things won’t show up at all: an ORM issuing sixty-one separate queries, where each plan looks perfectly healthy, and deep `OFFSET` pagination. I’d check the application log and `pg_stat_statements` for those.”
7. **Order the fixes.** “Then I’d fix in order of cost: rewrite the query first, add an index second, change the schema last.”

The follow-ups

“**The planner is choosing a sequential scan even though there’s an index on that column. Why?**” Three likely reasons and I would check them in order. First, selectivity: if the condition matches more than roughly five to ten per cent of the table, a sequential scan genuinely is cheaper, because an index scan there means hundreds of thousands of random page reads while a scan is one continuous sweep. The planner is right and there is nothing to fix. Second, the statistics are stale — it thinks the condition matches half the table when it matches a hundred rows — and `ANALYZE` fixes it. Third, something makes the index unusable: a function wrapped around the column, so it must compute the value per row, or a type mismatch in the comparison. To distinguish them I would look at the estimated row count in the plan: if the estimate is wildly wrong it is statistics, and if the estimate is right and it still scans, the planner has correctly decided the index is not worth it.

“**What does `loops=61` tell you?**” That this node executed sixty-one times, and — crucially — that the `actual time` shown is **per loop**, so the real total is sixty-one times that number. It is the most misread field in the output: a node showing `actual time=0.02..0.05` with `loops=61` looks trivially fast and actually consumed about three milliseconds of database time plus, if these are separate round trips, thirty milliseconds of network latency. It normally means one of two things. Inside a nested loop join, it means the outer side produced sixty-one rows and the inner side was probed once for each — which is fine if sixty-one is small and a disaster if the estimate was wrong and it is really half a million. Or, if you are looking at sixty-one *separate* plans rather than one node, it is an ORM $N + 1$.

“**An endpoint got slow last week and nothing was deployed. What happened?**” Most likely one of three things, and none of them is a code change. The data crossed a threshold: a table that was small enough for a sequential scan to be fine is no longer, or a sort that fitted in `work_mem` now spills to disk — and those changes are sudden rather than gradual, because the planner flips strategy. Or the statistics went stale, typically after a bulk load or a large migration, so the planner is now choosing on the basis of a distribution that no longer exists. Or index bloat: after a lot of updates and deletes, an

index can grow several times larger than it needs to be, so more pages must be read. I would compare `EXPLAIN ANALYZE` now against what the plan used to be, run `ANALYZE` first because it is free, and check `pg_stat_user_indexes` and table sizes for bloat.

“How do you find which query to optimise?” `pg_stat_statements`, sorted by `total_exec_time` rather than by mean. The reason is that the expensive query is usually not the one people complain about: a report taking two seconds once a night costs two seconds a day, while a twenty-millisecond query running a million times a day costs five and a half hours a day of database time. The second one is invisible to users and is the one actually consuming the machine. I would look at both — total to find what is costing capacity, mean to find what is costing user experience — and I would also check `calls`, because a query whose call count jumped is often an N + 1 that appeared with a new feature rather than a query that got slower.

A model answer

“I’d start by running `EXPLAIN ANALYZE`, with `BUFFERS`, and I’d read it from the inside out — the most indented nodes execute first, and the top line runs last.

The first thing I look at is not the scan types. It’s the estimated row count against the actual row count at each level. The planner chooses everything — join algorithm, join order, whether to use an index — from those estimates, so if it thought a step would return one row and it returned four hundred thousand, every decision above that point was made for a different problem. That’s the failure that costs the most, and no index fixes it: it’s stale statistics, and `ANALYZE` takes seconds.

Concretely, that failure looks like a nested loop with `rows=1` estimated and `rows=480000` actual on the outer side, and then `loops=480000` on the inner index scan. The planner reasoned ‘one outer row, so one lookup — a nested loop is obviously cheapest’, and it ended up doing half a million lookups where a hash join would have taken about a third of a second.

Then I look for a `Seq Scan` on a large table with a high `Rows Removed by Filter`. Reading a million rows to return three is a missing index — about 112 milliseconds against 0.02.

Then `loops` greater than one anywhere significant, remembering that `actual time` is per loop and multiplies. And anything mentioning `Disk` — `Sort Method: external merge` means the sort exceeded `work_mem`, which is often ten times slower than sorting in memory.

Two causes won’t appear in the plan at all, and I’d check for them separately. An ORM issuing sixty-one queries instead of one — each plan looks perfectly healthy, and you only see it in the application log or in the call counts in `pg_stat_statements`. And deep `OFFSET` pagination, where `OFFSET 1000000` produces and discards a million rows.

When I fix it, I’d go in order of cost: rewrite the query first, since that’s free and reversible — unwrapping a function from an indexed column, replacing `OFFSET` with a keyset cursor, naming columns so a covering index can be used. Then add an index, which is a permanent tax on every write and 10 to 25 per cent of the table in storage. Then change the schema, which is the expensive and hard-to-undo option.

And to decide *which* query to work on at all, I’d sort `pg_stat_statements` by total execution time rather than by mean — the twenty-millisecond query running a million times a day costs far more than the two-second report nobody runs.”

9 Recall card

- **The plan is only as good as the estimates.** Compare **estimated rows against actual rows** first — a bad estimate poisons every decision above it, and the fix is `ANALYZE`.
- **Read plans inside out.** `cost` is an estimate; the `actual` line is truth; **actual time is per loop**, so `loops=61` multiplies.
- **Look for:** `Seq Scan` with high `Rows Removed by Filter`, `loops > 1` on a big node, and anything saying `Disk`.
- **Two causes never appear in the plan:** an ORM $N+1$, and deep `OFFSET`. Check the app log and `pg_stat_statements`.
- **Fix in order of cost:** rewrite the query, then add an index, then change the schema. And pick the query by **total** time, not mean.

DSA topic: Variable-size sliding window **System design topic:** Query plans and the slow query

Code these, in this order

Four problems, two of each shape. **Before writing a line of any of them, say out loud whether it is shape A or shape B**, and therefore where the recording goes. Getting that wrong is the failure mode of this whole family, and it produces code that runs.

Before each one, ask:

1. Minimise or maximise? Therefore shrink-while-valid or shrink-while-invalid?
2. Is the condition monotonic as the window grows? (Any negative numbers?)
3. What does the window carry, and can it be updated in $O(1)$ per move?
4. What is returned when no valid window exists?

#	PROBLEM	SOURCE	SHAPE, AND WHAT IT IS REALLY TESTING
1	Minimum Size Subarray Sum	LeetCode 209 (Medium)	Shape A. Record inside the shrink loop. And whether you ask about negative values.
2	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	Shape B. Shrink before adding, record after .
3	Longest Repeating Character Replacement	LeetCode 424 (Medium)	Shape B with a subtle validity test, and an optimisation whose correctness argument is the real question.
4	Minimum Window Substring	LeetCode 76 (Hard)	Shape A with a count map and a missing counter. The hardest window problem asked routinely.

On problem 1, get the recording place wrong on purpose

Write it correctly, then move the recording line to after the `while` loop:

```
for right, x in enumerate(nums):
    total += x
    while total >= target:
        total -= nums[left]
        left += 1
    best = min(best, right - left + 1)    # moved OUT of the loop
```

Run it on `[2,3,1,2,4,3]` with target 7. It returns 1 instead of 2. Say exactly why — what is true about the window at the moment the `while` loop exits?

Then say the rule in one sentence: minimise records where, maximise records where?

On problem 1, break it with negatives

```
print(min_subarray_len(3, [-3, 5]))
```

It returns `0`, meaning “no such subarray”, and the answer is `1` — the single element `5`.

Trace it: what is the total after `right = 0`? After `right = 1`? Why does the shrink loop never run, and why is that the *correct* behaviour for the code and the wrong answer for the problem?

Then say what makes a sliding window legal, in one sentence, and name the technique you would use instead.

On problem 2, do it both ways

Write the `while`-shrink version and the jump-with-a-last-seen-map version. Confirm both give `[3, 1, 3, 0, 2, 3, 2]` for `["abcabcbb", "bbbbbb", "pwwkew", "", "au", "dvdvf", "abba"]`.

Then delete the `last[ch] ≥ left` guard from the jump version and run it on `"abba"`. You should get 3 instead of 2.

Then answer:

1. Where exactly did `left` move backwards, and why?
2. Why can the `while` version never have this bug?
3. Why does `left` moving backwards break the `O(n)` argument as well as the answer?

On problem 3, write the honest version first

Write it recomputing `max(count.values())` inside the shrink condition. Confirm it gives `[4, 4, 4, 0]` for `[("ABAB", 2), ("AABABBA", 1), ("AAAA", 0), ("", 1)]`. State its cost — and say why `O(26)` per step is still `O(n)` overall.

Only then write the version that carries `max_freq` and never decreases it. Then answer the question that matters:

1. Why is it correct never to decrease `max_freq` when the window shrinks?
2. What could go wrong if the problem asked for the *smallest* window instead?

On problem 4, build it in three stages

1. First, a version that checks validity by comparing two `Counter` s. Get it correct.

2. Then replace the comparison with the `missing` counter and confirm the answers are unchanged.
3. Then explain what a **negative** value in `need` represents, and why the trim loop uses `< 0`.

Test on `[("ADOBECODEBANC", "ABC"), ("a", "a"), ("a", "aa"), ("ab", "b")]` — expect `['BANC', 'a', '', 'b']`.

The shape drill

For each problem, say **shape A** or **shape B**, and where the recording goes, in under five seconds:

1. Shortest subarray with sum at least k.
2. Longest subarray with sum at most k.
3. Longest substring with at most 2 distinct characters.
4. Smallest window containing all letters of t.
5. Longest subarray where max minus min is at most k.
6. Number of subarrays with at most k distinct values.
7. Shortest substring containing at least k of some character.

Number 6 is the one that becomes tomorrow's material — say why counting is different from finding a best.

The monotonicity drill

For each, say whether a sliding window is legal, and why:

1. $\text{Sum} \geq \text{target}$, all values positive.
2. $\text{Sum} \geq \text{target}$, values may be negative.
3. $\text{Product} \leq \text{target}$, all values ≥ 1 .
4. $\text{Product} \leq \text{target}$, values may be 0 or between 0 and 1.
5. At most k distinct characters.
6. The window contains all letters of `t`.
7. Maximum minus minimum $\leq k$.

Numbers 2 and 4 fail. For number 7, the window is legal but needs two extra structures — name them.

The cost-argument drill

Say each of these out loud, without notes:

1. Why is a `while` inside a `for` still `O(n)` here?

2. How many times can `left` move in total, and why?
3. What single property of the pointers is the whole argument resting on?
4. What breaks the argument in the buggy `"abba"` version?

The measurement drill

```
import time, random
random.seed(5)

def window(target, nums):
    left = 0; total = 0; best = float('inf')
    for right, x in enumerate(nums):
        total += x
        while total ≥ target:
            best = min(best, right - left + 1)
            total -= nums[left]; left += 1
    return 0 if best == float('inf') else best

def brute(target, nums):
    best = float('inf')
    for i in range(len(nums)):
        s = 0
        for j in range(i, len(nums)):
            s += nums[j]
            if s ≥ target:
                best = min(best, j - i + 1); break
    return 0 if best == float('inf') else best

for n in (2000, 5000, 10000):
    arr = [random.randint(1, 10) for _ in range(n)]
    tgt = 10 ** 6 # unreachable, so both do full work
    s = time.perf_counter(); window(tgt, arr); t1 = time.perf_counter() - s
    s = time.perf_counter(); brute(tgt, arr); t2 = time.perf_counter() - s
    print(f"n={n:>6} {t1:.5f}s {t2:.4f}s ratio {t2/t1:>6.0f}x")
```

Then answer: why is the target chosen to be unreachable, and what does the ratio doubling tell you?

The plan-reading drill

For each fragment, give the diagnosis and the fix in one sentence:

```
Seq Scan on orders (cost=0.00..18334.00 rows=1 width=64)
      (actual time=0.021..112.4 rows=3 loops=1)
  Filter: (customer_id = 4217)
 Rows Removed by Filter: 999997
```

```
Nested Loop (cost=... rows=1) (actual rows=480000 loops=1)
  → Index Scan on a (rows=1) (actual rows=480000 loops=1)
  → Index Scan on b (rows=1) (actual rows=1 loops=480000)
```



```
Sort (actual time=8200..8400 rows=1000000 loops=1)
  Sort Method: external merge  Disk: 102400kB
```

```
Index Scan using orders_pkey (actual time=0.02..0.05 rows=1 loops=61)
```

```
Seq Scan on users (actual rows=480000 loops=1)
  Filter: (lower(email) = 'a@b.com'::text)
```

The second one is the most important. Say why adding an index would not help it.

The EXPLAIN drill

Answer each in one or two sentences, out loud:

1. In `(cost=0.42..8.45 rows=3 width=64)`, what is each number?
2. What is the difference between `EXPLAIN` and `EXPLAIN ANALYZE`, and what precaution does the second need?
3. Which do you read first — the top line or the most indented line?
4. What does `Rows Removed by Filter` tell you?
5. What does `loops=N` do to the reported `actual time`?
6. Estimated rows 1, actual rows 480,000 — what is the cause and what is the fix?
7. Name two causes of a slow query that never appear in a plan.
8. Why sort `pg_stat_statements` by total time rather than mean time?
9. Why does Postgres have no query hints?

The arithmetic drill

From memory, in under two minutes:

- A nested loop chosen on an estimate of 1 row where the reality is 480,000 — lookups and time, against the hash join it should have chosen.
 - 1,000,000 rows at 200 bytes — sequential scan against index scan.
 - `actual time=0.05` with `loops=61` — real database time, and real wall-clock with round trips.
 - Sorting 100 MB with `work_mem` at 4 MB against 128 MB.
 - A 2,000 ms query run once a day against a 20 ms query run a million times — daily cost of each.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the smallest subarray with a sum at least k .* Ask about negatives first, and say why. Name the shape. State the invariant and why `left` never goes back. Say where the recording goes and why. Finish with the `2n` argument and the no-answer convention.
 2. *Here is an EXPLAIN plan. What is wrong with this query?* The procedure, in order: read inside out, compare estimated with actual **first**, then scan types, then `loops`, then `Disk`. Name the two causes that never appear in a plan. Then fix in order of cost.
 3. *Why does a sliding window stop working when the array has negative numbers?* Monotonicity. Give the concrete failure — `[-3, 5]` with target 3 returns “no answer” when the answer is 1 — and name the correct technique instead.
-

Before you move on

- [] I say “minimise or maximise?” before writing any variable window, and therefore where to record.
- [] I ask about negative values before applying a window to a sum.
- [] I can state the monotonicity requirement in one sentence.
- [] In shape B I shrink **before** adding, and record **after**.
- [] I can argue `O(n)` from “neither pointer ever moves backwards”.
- [] I can name the input where a missing `last[ch] ≥ left` guard makes `left` go backwards.
- [] I say `EXPLAIN ANALYZE` before “add an index”.
- [] I compare estimated rows with actual rows **before** looking at scan types.
- [] I know `actual time` is per loop and multiplies by `loops`.
- [] I can name two causes of slowness that never appear in a plan.
- [] I can redraw the two-shapes template and the bad-estimate diagram from memory, in whatever tool I like.

Window plus hash map: the longest-substring family

DATA STRUCTURES AND ALGORITHMS

Window plus hash map: the longest-substring family

You can solve longest substring without repeating characters and its five cousins.

SYSTEM DESIGN

Transactions and ACID

You can explain each of the four ACID letters with a bank-transfer example.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the length of the longest substring without repeating characters.*
- *What does ACID stand for, and why does it matter?*

Window plus hash map: the longest-substring family

1 What this is, and why they ask it

Yesterday's window carried a running **sum**. Today it carries a **map** — a count of what is inside it — and that single change unlocks the largest family of window problems there is.

The map answers questions a sum cannot:

- **How many distinct things are in the window?** `len(count)`.
- **How many of this particular thing?** `count[x]`.
- **What is the most frequent thing?** the largest value.
- **Does the window contain everything I need?** compare against a required map.

Every problem below is the same eight-line skeleton with one of those four questions as its condition. *Longest substring without repeats*. *Longest with at most k distinct*. *Longest after replacing k characters*. *Longest run of 1s after flipping k zeros*. *Fruit into baskets*. *Permutation in a string*. Six recognisably different questions, one shape.

Interviewers ask the first of them — LeetCode 3 — more than almost any other problem in existence. It is the standard medium warm-up at every product company. The reason it works as a filter is that the brute force is obviously $O(n^2)$ or worse, the window solution is obviously better once seen, and there is one specific bug — `left` moving backwards — that separates people who understand the invariant from people who have memorised a shape.

2 The story

Kamesh drives a share-auto on the road that runs from the bypass junction down to the hospital, and the way it works is that people stand along it and he picks them up as he goes.

The thing that decides whether a trip is worth doing is destinations. He will run a trip with **two** destinations. Not three. Three means he is going down two side lanes and coming back out, and the fifteen minutes that costs him is fifteen minutes not on the main road picking people up, and by the time he is back the whole road has been taken by somebody else.

So on a Saturday morning, standing at the junction and looking down the road at the people waiting, the question in his head is: **what is the longest stretch of this road I can serve in one trip?**

He works it forwards. He starts at the top of the road and walks down it in his head, taking people as he goes and keeping a little tally — not just *which* places they are going to, but *how many* people for each place, because that turns out to matter.

Hospital, hospital, market. Two places, fine, keep going. Hospital again — three for hospital, one for market, still two places. Then somebody for the college, and that is a third place, so this stretch has gone too far.

Now he does the other thing: he starts dropping people from the **front** of the stretch — the ones nearest the junction — until he is back to two destinations.

And here is where the tally earns its place. He drops the first person, who was going to the hospital, and that does **not** get him back to two destinations, because there are still two more hospital people further down. He has to keep dropping until the *last* hospital person is gone. Only when the count for a place reaches zero does that place actually leave his list.

He said once that he used to keep only the names of the places and not the numbers, and he kept getting it wrong — crossing a place off his list while there was still somebody in the auto going there.

By the time he has walked the whole road in his head, he knows the longest stretch he can take, and he starts the engine.

3 The idea in plain English

Kamesh's stretch of road is the window. His tally is the **map**. And his rule that a place only leaves the list when its count reaches zero is the single most important detail in this entire family.

The skeleton

```
count = defaultdict(int)
left = 0
best = 0
for right, ch in enumerate(s):
    count[ch] += 1
    while invalid(count):
        count[s[left]] -= 1
        if count[s[left]] == 0:
            del count[s[left]]
        left += 1
    best = max(best, right - left + 1)
return best
```

Eight lines. Everything today is a different `invalid()`.

The `del` at zero, and why it is not optional

`len(count)` is supposed to mean *how many distinct characters are in the window right now*. If you decrement a count to zero and leave the key in the dictionary, `len(count)` still counts it — so it never goes back down, the shrink loop never terminates properly, and `left` marches all the way to `right`.

That is Kamesh crossing a place off his list while somebody in the auto is still going there — except the code makes the opposite mistake, keeping a place on the list after everybody going there has got out.

Decrement, then delete at zero. Say it as one action.

There is a Python subtlety worth knowing: with `defaultdict(int)`, merely *reading* `count[x]` for a missing key **creates it** with value 0 — from [day 021](#). So `if count[s[left]] == 0` after the decrement is safe, because the key definitely exists, but a stray `if count[some_other_char] == 0` elsewhere silently inserts a key and corrupts `len(count)`. Use `x in count` to test membership.

The six problems, and the one line that differs

PROBLEM	THE CONDITION FOR INVALID	WHAT THE MAP HOLDS
Longest without repeating (3)	<code>count[ch] > 1</code>	character → count
Longest with at most k distinct (340)	<code>len(count) > k</code>	character → count
Fruit into baskets (904)	<code>len(count) > 2</code>	fruit type → count
Longest repeating replacement (424)	<code>window_len - max_freq > k</code>	character → count
Max consecutive ones III (1004)	<code>zeros > k</code>	just a counter, not a map
Permutation in string (567)	fixed window; compare maps	character → count

That table is the day. Learn the skeleton once and the six problems are one line each.

Notice the fifth row. When there are only two kinds of thing — 0 and 1 — you do not need a map at all, just an integer. **Recognising when the map collapses to a counter is worth saying**, because it is simpler and faster and shows you understand what the map was for.

Two ways to shrink, and which to write

One at a time — the pure template, and it cannot go wrong:

```
while count[ch] > 1:
    count[s[left]] -= 1
    left += 1
```

Jump straight there — using a last-seen map, from [day 024](#):

```
if ch in last and last[ch] ≥ left:
    left = last[ch] + 1
last[ch] = right
```

The jump version is fewer operations, and it has a bug waiting in it. `last[ch] ≥ left` **is mandatory**: without it, a character last seen *before* the window began drags `left` backwards.

On "abba": at the final `a`, `last["a"]` is 0, which is behind the current `left` of 2, so `left` jumps back to 1 — the window now contains two `b`s and the answer comes out 3 instead of 2. It also destroys the `O(n)` argument, because `left` is no longer monotonic.

Write the `while` version under pressure. It is one line longer and it cannot have this bug, because it only ever increments. Offer the jump version afterwards as an optimisation, with the guard.

The subtle one: longest repeating character replacement

LeetCode 424: *you may change at most `k` characters; what is the longest run of a single repeated character you can produce?*

Inside any window, the characters you would have to change are all of them except the most frequent one. So:

```
invalid when (right - left + 1) - max_freq > k
```

Now the famous part. The standard solution **never decreases** `max_freq` when the window shrinks, and it is still correct. The argument:

- `best` only ever increases, and the window length never decreases — when the window becomes invalid, `left` advances by exactly as much as `right` did, so the window slides rather than shrinking.
- A stale, too-high `max_freq` makes the condition *easier* to satisfy, so the window can only fail to shrink when it should have.
- But a window that is too long is never *recorded* as an improvement, because the recorded value can only grow when a genuinely valid longer window appears.

You do not need this trick. Recomputing `max(count.values())` is $O(26)$ per step — constant for a bounded alphabet, so the whole thing is still $O(n)$. Write the honest version, get it right, and offer the optimisation with its justification. That reads far better than a memorised trick you cannot defend.

The fixed-size cousin

Permutation in String — LeetCode 567 — asks whether **any** window of length `len(s1)` is an anagram of `s1`. The window size is fixed, so this is [day 031](#)'s shape with a map inside: build the first window, then slide, adding one and removing one, comparing maps each step.

Comparing two `Counter`s of at most 26 keys is constant work, so it stays $O(n)$. And the `del` at zero is mandatory again, for a different reason: `Counter({'a': 1, 'b': 0})` does not equal `Counter({'a': 1})`, so a leftover zero makes every comparison fail forever.

4 The picture

`"abcabcbb"`, no repeats allowed, with the map shown:

right	ch	count after adding	action	left	window	best
0	a	{a:1}	ok	0	"a"	1
1	b	{a:1,b:1}	ok	0	"ab"	2
2	c	{a:1,b:1,c:1}	ok	0	"abc"	3
3	a	{a:2,b:1,c:1}	a>1 → drop 'a'	1	"bca"	3
		{a:1,b:1,c:1}	ok			
4	b	{a:1,b:2,c:1}	b>1 → drop 'b'	2	"cab"	3
		{a:1,b:1,c:1}	ok			
5	c	{a:1,b:1,c:2}	c>1 → drop 'c'	3	"abc"	3
		{a:1,b:1,c:1}	ok			
6	b	{a:1,b:2,c:1}	b>1 → drop 'a'	4		
		{b:2,c:1}	b>1 → drop 'b'	5	"cb"	3
		{b:1,c:1}	ok			
7	b	{b:2,c:1}	b>1 → drop 'c'	6		
		{b:2}	b>1 → drop 'b'	7	"b"	3
		{b:1}	ok			

answer 3

What to notice at right = 6: the shrink loop runs twice and the map loses the key `a` entirely, because its count hit zero. If the `del` were missing, `{a:0, b:1, c:1}` would still have `len == 3`, which matters enormously for the at-most-k-distinct variant.

The `del`-at-zero rule, made visible:

WITH the `del`
 window "bca", drop 'b'
`{a:1,b:1,c:1} → {a:1,c:1}`
`len = 2` correct

WITHOUT the `del`
 window "bca", drop 'b'
`{a:1,b:1,c:1} → {a:1,b:0,c:1}`
`len = 3` WRONG

For "at most 2 distinct", the left version stops shrinking here and the right version keeps going until left reaches right.

Why `left` must never move backwards, on "abba":

```
index    0    1    2    3
char     a    b    b    a
```

right=3, ch='a', last['a'] = 0

WITH the guard (`last[ch] ≥ left`):
 left is currently 2
 last['a'] = 0, and `0 ≥ 2` is FALSE
 so left stays at 2
 window = `s[2..3]` = "ba" correct

WITHOUT it:
 left = last['a'] + 1 = 1
 window = `s[1..3]` = "bba"
 → contains two b's!
 → answer 3, should be 2

5 The code, built step by step

The base case, the safe way

```
count: defaultdict[str, int] = defaultdict(int)
left = 0
best = 0
for right, ch in enumerate(s):
    count[ch] += 1
    while count[ch] > 1:
        count[s[left]] -= 1
        if count[s[left]] == 0:
            del count[s[left]]
        left += 1
    best = max(best, right - left + 1)
```

Add the entering character. While it now appears twice, drop characters off the front. Then record.

The condition is `count[ch] > 1` and not `len(count) < right - left + 1`, because the first says directly what is wrong — **this specific character repeats** — and shrinking is guaranteed to fix it, since the only way to remove the duplicate is to pass its earlier occurrence.

The base case, the fast way

```
last: dict[str, int] = {}
left = 0
best = 0
for right, ch in enumerate(s):
    if ch in last and last[ch] ≥ left:
        left = last[ch] + 1
    last[ch] = right
    best = max(best, right - left + 1)
```

Fewer operations, and `last[ch] ≥ left` is load-bearing. An equivalent and clearer way to write the same guard:

```
left = max(left, last.get(ch, -1) + 1)
```

That says *left never goes backwards* in the code itself, which is better than relying on the reader to notice the comparison.

At most k distinct

Only the condition changes:

```

while len(count) > k:
    count[s[left]] -= 1
    if count[s[left]] == 0:
        del count[s[left]]
    left += 1

```

`len(count) > k` is where the `del` earns its keep: without it `len(count)` counts characters that have left, so it never falls back to `k` and the loop runs until `left == right`.

Guard `k = 0` — the answer is 0, and the loop handles it, but say it aloud.

Max consecutive ones, where the map collapses

```

zeros = 0
left = 0
best = 0
for right, x in enumerate(nums):
    if x == 0:
        zeros += 1
    while zeros > k:
        if nums[left] == 0:
            zeros -= 1
        left += 1
    best = max(best, right - left + 1)

```

Two kinds of value, so one integer replaces the whole map. **Noticing that is worth mentioning** — it is the same algorithm with the data structure simplified because the alphabet is size two.

Longest repeating character replacement

```

count: defaultdict[str, int] = defaultdict(int)
left = 0
best = 0
max_freq = 0
for right, ch in enumerate(s):
    count[ch] += 1
    max_freq = max(max_freq, count[ch])
    while (right - left + 1) - max_freq > k:
        count[s[left]] -= 1
        left += 1
    best = max(best, right - left + 1)

```

Note there is no `del` here, because nothing depends on `len(count)` — the condition uses `max_freq` and the window length. **The `del` is needed exactly when `len(count)` is part of the condition**, and saying that shows you know why the rule exists rather than applying it superstitiously.

Permutation in string: fixed window plus map

```
if len(s1) > len(s2):
    return False
need = Counter(s1)
window = Counter(s2[:len(s1)])
if window == need:
    return True

for i in range(len(s1), len(s2)):
    window[s2[i]] += 1
    left_char = s2[i - len(s1)]
    window[left_char] -= 1
    if window[left_char] == 0:
        del window[left_char]
    if window == need:
        return True
return False
```

Fixed size, so both edges move together — `i` enters, `i - len(s1)` leaves, exactly [day 031](#). And the `del` is mandatory again, because `Counter` equality is affected by zero entries.

The complete solutions

```
from collections import Counter, defaultdict

def longest_unique(s: str) → int:
    """LeetCode 3. The safe version: shrink one at a time, so left cannot go back."""
    count: defaultdict[str, int] = defaultdict(int)
    left = 0
    best = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while count[ch] > 1:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best

def longest_unique_jump(s: str) → int:
    """The same, jumping straight to the new left. max() makes the guard explicit."""
    last: dict[str, int] = {}
    left = 0
    best = 0
    for right, ch in enumerate(s):
        left = max(left, last.get(ch, -1) + 1)  # left NEVER goes backwards
        last[ch] = right
        best = max(best, right - left + 1)
    return best

def longest_k_distinct(s: str, k: int) → int:
    """LeetCode 340. len(count) must mean 'distinct in the window' – hence the del."""
    count: defaultdict[str, int] = defaultdict(int)
    left = 0
    best = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best

def longest_ones(nums: list[int], k: int) → int:
    """LeetCode 1004. Two kinds of value, so the map collapses to one integer."""
    zeros = 0
    left = 0
    best = 0
    for right, x in enumerate(nums):
        if x == 0:
            zeros += 1
        while zeros > k:
            if nums[left] == 0:
                zeros -= 1
            left += 1
        best = max(best, right - left + 1)
    return best
```

```

def character_replacement(s: str, k: int) → int:
    """LeetCode 424. No del needed: the condition uses max_freq, not len(count)."""
    count: defaultdict[str, int] = defaultdict(int)
    left = 0
    best = 0
    max_freq = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        max_freq = max(max_freq, count[ch])
        while (right - left + 1) - max_freq > k:
            count[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best

def check_inclusion(s1: str, s2: str) → bool:
    """LeetCode 567. FIXED window plus a map – day 031's shape, not day 032's."""
    if len(s1) > len(s2):
        return False
    need = Counter(s1)
    window = Counter(s2[:len(s1)])
    if window == need:
        return True

    for i in range(len(s1), len(s2)):
        window[s2[i]] += 1
        left_char = s2[i - len(s1)]
        window[left_char] -= 1
        if window[left_char] == 0:
            del window[left_char]          # zero entries break Counter equality
        if window == need:
            return True
    return False

if __name__ == "__main__":
    cases = ("abcabcbb", "bbbbbb", "pwwkew", "", "au", "dvdf", "abba", " ")
    print([longest_unique(x) for x in cases])      # [3, 1, 3, 0, 2, 3, 2, 1]
    print([longest_unique_jump(x) for x in cases]) # identical

    print([longest_k_distinct(s, k) for s, k in
           (("eceba", 2), ("aa", 1), ("abaccc", 2), ("", 2), ("abc", 0),
            ("abcadccacacaca", 3))])
    # [3, 2, 4, 0, 0, 11]

    print([longest_ones(a, k) for a, k in
           (([1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0], 2), ([0, 0, 1, 1, 1, 0, 0], 0),
            ([0], 1), ([1], 0))])
    # [6, 3, 1, 1]

    print([character_replacement(s, k) for s, k in
           (("ABAB", 2), ("AABABBA", 1), ("AAAA", 0), ("", 1), ("ABBB", 2))])
    # [4, 4, 4, 0, 4]

    print([check_inclusion(a, b) for a, b in
           (("ab", "eidbaooo"), ("ab", "eidbaooo"), ("a", "a"), ("adc", "dcda"))])
    # [True, False, True, True]

```

6 What it costs

Every problem on this page

`right` advances exactly n times. `left` only advances and never passes `right`, so at most n times across the whole run. Each move does a constant number of dictionary operations, which are $O(1)$ on average.

$O(n)$ time, and the argument is the same as [day 032](#): count how far the pointers travel, not how the loops are nested.

Space: $O(\min(n, \text{alphabet}))$ — one entry per distinct character currently in the window. For lowercase English that is at most 26, so $O(1)$; for arbitrary Unicode it is bounded by n .

Say the space bound carefully. “ $O(k)$ where k is the alphabet size” is the precise answer, and “ $O(1)$ because the alphabet is bounded at 26” is the right thing to say when the problem promises lowercase letters.

Against the brute force

Checking every substring for repeats: $n^2/2$ substrings, each checked in $O(k)$:

```
n = 1,000    brute ≈ 500,000 substring checks × up to 1,000 chars = 500,000,000 ops
            window ≈ 2,000 pointer moves
```

At $n = 100,000$ the brute force is roughly 10^{15} operations and the window is 200,000.

`character_replacement`, both versions

```
recompute max(count.values()) :  $O(26)$  per step →  $26n$  operations
carry max_freq                :  $O(1)$  per step →  $n$  operations
```

Both are $O(n)$. The trick is a constant-factor improvement of about $26\times$, which is real but not a complexity change. **Say that when you offer it**, so it is clear you are not confusing an optimisation with an asymptotic improvement.

`check_inclusion`

One `Counter` comparison per position. Two `Counter`s of at most 26 keys compare in constant time, so:

```
n comparisons ×  $O(26)$  =  $O(26n)$  =  $O(n)$ 
```

There is a further optimisation — carry a `matches` counter of how many characters have exactly the right count, updated as the two edges move — which makes each step genuinely $O(1)$. Worth naming; not worth writing unless asked.

The number to have ready

One pass, both pointers forward only, so $O(n)$ time and $O(k)$ space where k is the alphabet size — $O(1)$ for lowercase English. The brute force checks every substring, which at a thousand characters is 500 million operations against 2,000.

7 The traps

The near-miss: forgetting the `del` at zero

```
count[s[left]] -= 1
# no deletion
left += 1

print(longest_k_distinct("eceba", 2))
```

1

The answer is 3 ("ece"). Once any character's count hits zero, `len(count)` still includes it, so the shrink loop's condition `len(count) > k` never becomes false again and `left` marches up to `right`. The result collapses to 1.

The `del` is needed exactly when `len(count)` is part of the condition. In `character_replacement` it is not, and there is no `del` there — knowing *why* rather than doing it everywhere is the mark of understanding it.

The near-miss: `left` moving backwards

```
if ch in last:
    left = last[ch] + 1
# missing: and last[ch] ≥ left
```

On "abba" this returns 3 instead of 2, and the mechanism is in §4's diagram: at the final `a`, the last-seen position 0 is behind the current window, so `left` jumps backwards and the window swallows a repeated `b`.

Two fixes, and the second is better because it states the intent:

```
if ch in last and last[ch] ≥ left:
    left = last[ch] + 1
# or
left = max(left, last.get(ch, -1) + 1)
```


And the `while -shrink` version cannot have this bug at all, which is a reason to prefer it when you are being watched.

The near-miss: adding before shrinking

```
count[ch] += 1
seen.add(ch)
while ch in seen:                                # always true – you just added it
```

If you use a set and add before checking, the condition is trivially true and `left` runs to `right`, so every answer is 1. With a counter and `count[ch] > 1` this does not arise, which is another small reason to prefer the counter.

The real error: `defaultdict` inserting on read

```
count = defaultdict(int)
if count[ch] == 0:                                # this INSERTS ch with value 0
    ...
print(len(count))                                # now includes characters never seen
```

Reading a missing key from a `defaultdict` creates it — from [day 021](#). That silently corrupts `len(count)`, which is exactly the quantity the shrink loop depends on. **Test membership with `ch in count`, never by reading the value.**

The near-miss: recording inside the shrink loop

```
while len(count) > k:
    best = max(best, right - left + 1)    # WRONG PLACE
    count[s[left]] -= 1
    left += 1
```

This is [day 032](#)'s shape-A placement applied to a shape-B problem. It records the window while it is still **invalid** — too many distinct characters — so the answer comes out too large. **Maximise: record after the shrink loop. Minimise: record inside it.**

The contract corner: what counts as a character

`" "` is a valid input to LeetCode 3, and the answer is 1. So are digits, symbols and Unicode. If you reached for a 26-element array instead of a map, `ord(' ') - ord('a')` is negative and — from [day 021](#) — a negative index does not raise, it reads from the end. **Ask about the alphabet before choosing the structure.**

The subtle one: defending `max_freq`

If an interviewer asks “why don’t you decrease `max_freq` when the window shrinks?” and you cannot answer, the honest move is to say so and switch to recomputing it. A wrong justification is worse than the extra 26 operations. The correct argument is in §3, and it turns on the window never actually shrinking — only sliding.

8 In the interview

How it gets asked

- “Longest substring without repeating characters.” — LeetCode 3, the most-asked medium there is.
- “Longest substring with at most k distinct characters.” — LeetCode 340, and *fruit into baskets* (904) is the same problem with `k = 2`.
- “Longest run of 1s if you can flip at most k zeros.” — LeetCode 1004, where the map collapses to a counter.
- “Longest substring you can make uniform by changing k characters.” — LeetCode 424, the subtle one.
- “Does s_2 contain a permutation of s_1 ?” — LeetCode 567, a **fixed** window with a map.

What to say out loud, in the first ninety seconds

1. **Ask about the alphabet.** “Lowercase English only, or arbitrary characters? That decides whether I use a 26-element array or a map, and it changes the space bound I quote.”
2. **State the brute force and its cost.** “Checking every substring for repeats is $O(n^2)$ at best.”
3. **Name the shape and which one it is.** “This is a maximisation, so: grow the window, and shrink from the left only while it’s invalid, recording after the shrink.”
4. **Say what the window carries and what the condition is.** “The window carries a count map, and it’s invalid when the character I just added appears twice.”
5. **Flag the `del` and say why.** “When I decrement a count to zero I delete the key, because `len(count)` has to mean ‘distinct characters currently in the window’.”
6. **Say why `left` never moves backwards.** “Both pointers only advance, so each index is visited at most twice — that’s the $O(n)$ argument, and it’s also why I’d rather shrink one step at a time than jump, since jumping can move `left` backwards on input like `abba` if the guard is missing.”
7. **Give the costs.** “ $O(n)$ time; $O(k)$ space for the map, which is $O(1)$ for a bounded alphabet.”

The follow-ups

“Can you do it without shrinking one character at a time?” Yes — keep a map of the last position each character was seen, and when a repeat arrives, jump `left` straight past that earlier occurrence instead of walking there. It is fewer operations. The thing to be careful about is that the jump must never move `left` backwards: if the character was last seen *before* the current window began, its recorded position is behind `left`, and assigning `left = last[ch] + 1` unconditionally moves the window’s start backwards. On `"abba"` that gives 3 instead of 2, because the window ends up containing two `b`s. The fix is either a guard, `last[ch] ≥ left`, or writing it as `left = max(left, last[ch] + 1)`, which I prefer because it states the invariant in the code. Under time pressure I would write the one-step-at-a-time version, since it only ever increments and cannot have this bug at all.

“Now: at most k distinct characters.” Only the condition changes — invalid becomes `len(count) > k`. But that change makes one line load-bearing that was optional before: when I decrement a character’s count to zero I must delete the key, because `len(count)` is now the thing the loop tests. Leave a zero entry behind and `len(count)` counts characters that have already left the window, so it never drops back to `k`, the shrink loop keeps running, and `left` walks all the way to `right` — the answer collapses to 1 with no error. On `"eceba"` with k of 2 you get 1 instead of 3. It is worth noticing that *fruit into baskets* is exactly this problem with $k = 2$, and that if there were only two possible values I would drop the map entirely and keep a single counter.

“Why doesn’t `max_freq` need to decrease in the character-replacement problem?” Because the window never actually shrinks — it slides. When the window becomes invalid, `left` advances by one for the one step `right` advanced, so the length stays the same rather than dropping. A stale, too-high `max_freq` makes the validity test easier to pass, so the only thing it can do is fail to shrink a window that should have shrunk — and since the recorded answer only improves when a genuinely longer window appears, that never produces a larger wrong answer. I would add that I do not need the trick: recomputing `max(count.values())` is $O(26)$ per step, so constant for a bounded alphabet and still $O(n)$ overall. I would write that version first and offer this one as a constant-factor optimisation with the argument attached, because a trick I cannot defend is worse than 26 extra operations.

“What if the string is Unicode, or extremely long?” The map handles Unicode with no change, which is the main reason to prefer it over a 26-element array — that array silently misbehaves on anything outside `a–z`, because `ord(ch) - ord('a')` goes negative and Python indexes from the end rather than raising. The space bound becomes `O(min(n, alphabet))` rather than `O(1)`. For an extremely long input the algorithm is already one pass and the map only ever holds what is currently inside the window, so memory is bounded by the window’s distinct-character count rather than by the input

length — which means it works on a stream, provided you can buffer the window itself. What I would watch is slicing: never take `s[left:right+1]` inside the loop to inspect the window, because that copies and turns a linear algorithm into a quadratic one silently.

A model answer

“First, what’s the alphabet? Lowercase English, or could it be arbitrary characters including spaces and Unicode? That decides whether I use a fixed 26-element array or a dictionary, and it changes the space bound I’d quote.

...Arbitrary. Then a map.

The brute force checks every substring for repeats, which is $O(n^2)$ at best and worse if you rebuild a set each time. Instead I’ll use a sliding window with a count of what’s inside it.

This is a maximisation, so the shape is: grow the window on the right unconditionally, and shrink from the left only while the window is invalid — then record after the shrink, because that’s the first moment it’s valid again.

```
def longest_unique(s: str) → int:
    count = defaultdict(int)
    left = 0
    best = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while count[ch] > 1:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best
```

The condition is `count[ch] > 1` — the character I just added now appears twice — and shrinking is guaranteed to fix it, because the only way to remove the duplicate is to pass its earlier occurrence.

The `del` when a count reaches zero matters. Here it’s tidiness, but the moment the condition involves `len(count)` — as it does for the at-most-k-distinct version — it becomes essential, because `len(count)` has to mean ‘distinct characters currently in the window’. Leave zero entries behind and the shrink loop never terminates properly.

Cost: the right pointer advances n times, the left pointer at most n times, and neither ever moves backwards — so at most $2n$ moves, each doing constant dictionary work. $O(n)$ time, and $O(k)$ space where k is the alphabet size, bounded by n .

There’s a faster variant that jumps `left` straight past the previous occurrence using a last-seen map, and it has a specific bug worth mentioning: if the character was last seen before the window started, jumping unconditionally moves `left` backwards.

On `abba` that returns 3 instead of 2. The fix is `left = max(left, last[ch] + 1)`. I'd write the one-step version under time pressure, because it only ever increments and can't have that bug."

9 Recall card

- **The window carries a map**, and the condition is a property of the map: `len(count)`, `count[x]`, the maximum value, or a comparison against a needed map.
- **del at zero — mandatory whenever `len(count)` is in the condition**, or the shrink loop never ends.
- **Maximise: shrink while invalid, record after**. Six problems, one skeleton, one line different.
- **left must never move backwards**. Guard the jump with `max(left, last[ch] + 1)`, or shrink one step at a time and the bug cannot exist.
- **$O(n)$ time, $O(k)$ space** where `k` is the alphabet. When there are only two kinds of value, the map collapses to one integer.

1 What this is, and why they ask it

A **transaction** is a group of database operations treated as one indivisible unit. Either all of them take effect, or none of them do. You mark the boundaries yourself:

```
BEGIN;  
  UPDATE accounts SET balance = balance - 5000 WHERE id = 1;  
  UPDATE accounts SET balance = balance + 5000 WHERE id = 2;  
COMMIT;
```

Two statements, one transaction. If the process dies between them, neither happens. If another connection looks while it is in progress, it sees neither.

ACID names the four guarantees a transaction gives you:

- **Atomicity** — all of it, or none of it.
- **Consistency** — the database's own rules are never violated.
- **Isolation** — concurrent transactions do not see each other's unfinished work.
- **Durability** — once it says committed, it survives a crash.

Interviewers ask this constantly, and it is one of the questions most often answered badly — not because people cannot recite the four words, but because reciting is all they do. The answer that lands names each letter, gives the **specific failure it prevents**, and says **what mechanism delivers it**. And the most revealing follow-up is “*which of the four is the interesting one?*”, where the answer is isolation, because it is the only one that is not simply on or off — it comes in levels, which is [day 034](#).

You have already met three of the four mechanisms. The write-ahead log from [day 025](#) delivers atomicity and durability. Constraints from [day 026](#) deliver consistency. Today ties them together and adds isolation.

2 The story

Sudhir decided to sell his scooter in March, and the man who wanted it was a stranger who had answered the advertisement — perfectly pleasant, from two towns away, and completely unknown.

The problem they had is old and obvious. Sudhir was not going to hand over the keys and the papers and then wait to see whether the money arrived. And the buyer was not going to hand over sixty-two thousand rupees and then wait to see whether the papers turned up. Neither of them was being difficult. Each of them was simply not prepared to be the one who went first.

What they did was go to a man who does vehicle transfers near the RTO, on a Tuesday morning, both of them.

That man's whole job is that nothing happens until everything can happen. He took the papers from Sudhir and put them on his side of the desk. He watched the buyer make the transfer on his phone and waited for Sudhir's message to come. He checked the insurance was current and that there was no loan outstanding on the vehicle, because a transfer with a loan on it is not a thing he is allowed to do. And only when all of that was in front of him did he do the actual transfer — and then it was done, all of it, in about four minutes.

Sudhir asked him what happens if something goes wrong in the middle. He said: then nothing has happened. If the money does not come, he gives the papers back and everyone goes home and the scooter is still Sudhir's. There is no state of affairs where the buyer owns half a scooter, or where the money has moved and the papers have not.

And the third thing, which Sudhir only appreciated later. While it was going on, if somebody else had rung him about the scooter, the honest answer was still that it was his. It did not become half-sold. Right up to the moment it was done, it was one thing; after that moment, it was the other.

The papers came ten days later and the entry is in the government's records, and Sudhir says that is the part he actually paid for. Not the four minutes. The fact that it is written down somewhere that does not depend on anybody remembering.

3 The idea in plain English

The man at the RTO is the database. Everything he insisted on is one of the four letters.

A — Atomicity: all of it, or none of it

“There is no state of affairs where the money has moved and the papers have not.”

A transaction is indivisible. If any part fails — a constraint violation, a deadlock, the process being killed — everything done so far is **rolled back**, and it is as though the transaction never ran.


```
BEGIN;  
  UPDATE accounts SET balance = balance - 5000 WHERE id = 1;  
  -- process dies here  
  UPDATE accounts SET balance = balance + 5000 WHERE id = 2;  
COMMIT;
```

Without atomicity, ₹5,000 has vanished. With it, nothing happened at all.

The mechanism is the write-ahead log. Before changing anything, the database appends a record of what it intends to do. On restart it reads the log: transactions that reached `COMMIT` are replayed, and those that did not are undone. The word to use is **rollback**, and it is not a special error path — it is the normal machinery.

C — Consistency: the rules are never broken

“He checked there was no loan outstanding, because a transfer with a loan on it is not a thing he is allowed to do.”

A transaction moves the database from one valid state to another valid state. Every constraint you declared — `NOT NULL`, `UNIQUE`, `CHECK`, foreign keys, from [day 026](#) — holds before and after. If a statement would break one, the transaction fails and rolls back.

```
balance NUMERIC NOT NULL CHECK (balance ≥ 0)
```

Now a transfer that would overdraw an account cannot commit, whatever the application code believes.

C is the odd letter out, and saying so is a genuinely good signal. Atomicity, isolation and durability are things the database *does*. Consistency is largely something **you** define — the database enforces the rules you declared, and it has no opinion about rules you did not declare. Many people argue the C is in the acronym mostly because ACID is a better word than AID.

I — Isolation: nobody sees your half-finished work

“If somebody else had rung him about the scooter, the honest answer was still that it was his.”

Concurrent transactions must not observe each other’s intermediate states. Between the two `UPDATE`s above, ₹5,000 genuinely does not exist in either account — and **no other transaction may ever see that**.

The strongest form is **serialisable**: the result is as if the transactions had run one after another, in some order. That is expensive, so real databases offer weaker levels that permit specific anomalies in exchange for speed. Postgres defaults to **read committed**, which prevents dirty reads and permits several other things.

Isolation is the interesting letter, because it is the only one with a dial. That dial — and the four anomalies it trades against — is tomorrow’s lesson.

D — Durability: once committed, it stays committed

“It is written down somewhere that does not depend on anybody remembering.”

When `COMMIT` returns successfully, the data survives a power cut, a kernel panic or a process kill. It is not “probably written soon”; it is written.

The mechanism is `fsync` on the write-ahead log. The change is appended to the log and the operating system is told to force it to physical storage *before* the commit is acknowledged. The actual data pages are written later, in the background — because the log is written sequentially, which from [day 025](#) is far faster than the random writes the data pages would need.

That single design makes commits both **safe and fast**, which is a rare combination and the reason write-ahead logging is in every serious database.

The four, as one table

LETTER	PREVENTS	MECHANISM	FAILURE WITHOUT IT
Atomicity	half-done work	write-ahead log, rollback	₹5,000 vanishes mid-transfer
Consist- ency	invalid data	constraints, checked at commit	a negative balance is stored
Isolation	seeing others’ unfinished work	locking, MVCC	a report totals money that is mid-flight
Durability	losing committed work	<code>fsync</code> on the log	the payment is confirmed and then lost

What a transaction is *not*

Two clarifications that come up:

A transaction is not a lock on the whole database. Two transactions touching different rows run in parallel. Only conflicting access is serialised.

A transaction is not free. Holding one open takes resources, holds locks, and — under Postgres’s MVCC — prevents old row versions from being cleaned up. **A long-running transaction is one of the classic production problems:** an idle-in-transaction connection can block `VACUUM` across the whole database and cause table bloat that has nothing to do with the query it ran.

4 The picture

The transfer, with and without atomicity:

WITHOUT a transaction	WITH a transaction
-----	-----
UPDATE a: 10000 → 5000 [written]	BEGIN
*** process killed ***	UPDATE a: 10000 → 5000 [in the log]
	*** process killed ***
UPDATE b: never runs	(no COMMIT was reached)
	on restart: replay the log,
state: a = 5000, b = 3000	find no COMMIT for this
total 8000	transaction, UNDO it
₹5,000 has vanished	
	state: a = 10000, b = 3000
	total 13000 - unchanged

What to notice: the left column has no error message anywhere. The money is simply gone, and the only way anyone finds out is a reconciliation weeks later.

Where each letter acts, on one timeline:

```
BEGIN
|
|  UPDATE accounts SET balance = balance - 5000 WHERE id = 1;
|      → appended to the write-ahead log           [ A, D ]
|      → row locked / new version created           [ I ]
|      → CHECK (balance ≥ 0) evaluated               [ C ]
|
|  UPDATE accounts SET balance = balance + 5000 WHERE id = 2;
|      ... same four things ...
|
|  ← other transactions see NEITHER update          [ I ]
|
COMMIT
|  → fsync the log to physical storage               [ D ]
|  → only THEN report success                        [ D ]
|  → now other transactions see BOTH                 [ I ]
v
data pages written to disk later, in the background
```

What to notice: the `fsync` happens *before* `COMMIT` returns, and the data pages are written after. That ordering is the whole of durability, and reversing it — reporting success first — is what “fast but loses data on power failure” means.

The write-ahead log, and why it is fast as well as safe:

the data pages that need changing:

page	page	page	page
1092	47	8801	312

scattered: 4 RANDOM writes

the log:

txn 88: a -= 5000
txn 88: b += 5000
txn 88: COMMIT

one SEQUENTIAL append

random 8 KB writes to SSD : ~15,000/second

sequential writes to SSD : ~60,000 8 KB writes/second

So: append to the log, fsync THAT, report success.

Write the scattered pages later, lazily, batched.

5 How it actually works

Writing one

```
BEGIN;  
  UPDATE accounts SET balance = balance - 5000 WHERE id = 1;  
  UPDATE accounts SET balance = balance + 5000 WHERE id = 2;  
  INSERT INTO transfers (from_id, to_id, amount) VALUES (1, 2, 5000);  
COMMIT;
```

Three statements, one unit. Note the third: the audit record is inside the transaction, so there can never be a transfer row without the balance changes or the reverse.

Two rules worth stating:

Do the arithmetic in the database, not in your application. `SET balance = balance - 5000` is one atomic operation. Reading the balance, subtracting in Python and writing it back is a **lost update** — two concurrent transfers each read 10,000, each write 5,000, and one transfer vanishes. That is [day 025's](#) lost update, and a transaction alone does not prevent it at Postgres's default isolation level.

Keep transactions short. No network calls to third parties inside one, no waiting for user input, no sleeping. Every held lock is a queue behind you.

What actually happens on `COMMIT`

1. All the changes are already in the write-ahead log buffer in memory.
2. `fsync` — the operating system is told to force the log to physical storage and to wait until it has happened.
3. Only now is `COMMIT` reported as successful.
4. The modified data pages are written from the buffer pool to their real locations later, at a checkpoint.

Step 2 is the expensive one — it is a physical write barrier, and it is why commit latency is roughly one disk sync. **Group commit** amortises it: many transactions committing at nearly the same moment share one `fsync`, so throughput can be far higher than one commit per sync.

`synchronous_commit = off` in Postgres skips the wait — commits return before the log is durable. That makes writes dramatically faster and means a power failure can lose the last fraction of a second of committed transactions. **It is a real, deliberate setting for data where that is acceptable**, and knowing it exists, and what exactly it trades, is a good signal.

ROLLBACK, and how it happens without you

```
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE id = 1;
UPDATE accounts SET balance = balance - 999999 WHERE id = 2; -- violates CHECK
COMMIT;
```

The second statement raises, the transaction is aborted, and **the first update is undone too**. In Postgres, once a statement errors inside a transaction, every subsequent statement fails with *current transaction is aborted* until you `ROLLBACK` — which surprises people the first time and is exactly the guarantee working.

Savepoints allow partial rollback when you genuinely want it:

```
BEGIN;
INSERT INTO orders ...;
SAVEPOINT after_order;
INSERT INTO notifications ...;      -- if this fails, keep the order
ROLLBACK TO after_order;
COMMIT;
```

Use them sparingly; they are usually a sign the transaction is doing two jobs.

MVCC, and why readers do not block

Postgres, Oracle and MySQL's InnoDB use **multi-version concurrency control**. An `UPDATE` does not overwrite the row — it writes a **new version** and marks the old one as valid up to this transaction's id. Each transaction sees the versions that were committed when it started.

The consequences are worth knowing:

- **Readers never block writers, and writers never block readers.** That is the single biggest reason these databases perform well under mixed load.
- **Two writers to the same row still conflict** — the second waits for the first to commit or roll back.

- **Old versions accumulate** and must be cleaned up, which is `VACUUM`. A long-running transaction holds back the cleanup horizon for the *whole database*, which is why an idle-in-transaction connection can bloat tables it never touched.

Distributed transactions, and why they are avoided

A transaction across two databases needs **two-phase commit**: a coordinator asks everyone to prepare, and if all say yes, tells everyone to commit.

It works and it is avoided, because if the coordinator dies between the two phases, every participant is left holding locks and unable to decide. In a microservice architecture the usual answer is the **saga** pattern — a sequence of local transactions, each with a compensating action to undo it — which gives you eventual consistency and explicit undo instead of atomicity. Sagas arrive later in the course; naming them here is enough.

Which stores give you this

Full ACID: PostgreSQL, MySQL with InnoDB, Oracle, SQL Server, SQLite.

ACID within a limit: MongoDB has multi-document transactions since 4.0 but they are discouraged at scale. DynamoDB has transactions across up to 100 items. Cassandra has none across partitions — lightweight transactions are a single-partition compare-and-set.

Redis has `MULTI / EXEC`, which is atomic in the sense that commands are not interleaved — but there is **no rollback**: if one command in the block fails, the others still apply. It is worth being precise about that, because people call it a transaction and it is not an ACID one.

6 The numbers

What a commit costs

```
fsync to SSD           ≈ 0.5 - 1 ms
fsync to spinning disk ≈ 5 - 10 ms

so, naively, one commit per fsync:
  SSD : ~1,000 - 2,000 commits/second
  HDD : ~100 - 200 commits/second
```

With **group commit**, transactions arriving within the same window share one `fsync` :

```
100 transactions sharing one 1 ms fsync → effective 100,000 commits/second
```

Which is why real throughput on a single Postgres instance is tens of thousands of small transactions a second and not one thousand. **That gap between the naive figure and the real one is the thing to be able to explain.**

```
synchronous_commit = off
```

```
on  : commit waits for fsync    → ~1 ms latency, zero data loss
off : commit returns first      → ~0.05 ms latency, up to ~600 ms of committed
                                   transactions lost on a power failure
```

About $20\times$ faster writes, in exchange for a bounded window of loss. Right for analytics ingestion or click tracking; wrong for payments.

Transaction size

```
1,000 rows, one transaction each : 1,000 commits × 1 ms fsync = 1,000 ms
1,000 rows, one transaction total : 1 commit                    ≈      15 ms
                                   about 60x
```

Batching writes into one transaction is one of the largest easy wins there is, and it is the answer to “the import is slow”.

But there is an opposite limit:

```
1,000,000 rows in one transaction:
- holds locks for the whole duration
- the entire change must be undoable, so undo information grows
- one failure at row 999,999 discards everything
- blocks VACUUM for the whole database while it runs
```

Batch in chunks of a few thousand, not one row and not a million.

The cost of a long transaction

```
a transaction left open for 4 hours:
- VACUUM cannot remove any row version created in those 4 hours,
  ANYWHERE in the database
- a table taking 10,000 updates/second for 4 hours accumulates
  144,000,000 dead row versions
- at ~100 bytes each: ~14 GB of bloat from ONE idle connection
```

That number is worth carrying, because “an idle-in-transaction connection bloated the database” sounds implausible until you multiply it out.

Lock waiting

```
transaction A holds a row lock for 100 ms
99 other transactions want the same row

serialised: 99 × 100 ms = 9.9 seconds for the last one
```

Which is why “keep transactions short” is a throughput statement, not tidiness. **Never put an HTTP call to a payment provider inside a transaction** — you have just made every other writer wait for a third party’s latency.

7 The trade-offs

Durability against write latency

`synchronous_commit = on` is roughly $20\times$ slower and loses nothing. `off` is fast and can lose a fraction of a second of acknowledged commits on power loss. There is a middle setting, `remote_write`, which waits for a replica to have received the log but not necessarily flushed it.

Choose by what the data is. Payments, orders and anything a person will complain about: `on`. Metrics, clickstream, cache warming: `off` is defensible and the speed is real.

Transaction size

Too small and you pay an `fsync` per row — $60\times$ slower on a bulk import. Too large and you hold locks for minutes, generate huge undo, block `VACUUM` and lose everything if the last row fails. **A few thousand rows per transaction is the usual sweet spot**, and the shape of that curve is worth knowing.

Isolation strength against concurrency

Stronger isolation means more locking or more aborted transactions, and therefore less throughput. Serialisable is correct and slow; read committed is fast and permits anomalies you must think about. **That is tomorrow’s entire lesson**, and the honest position is that most applications run at read committed and handle the specific anomalies that matter to them explicitly.

ACID against distributed scale

Two-phase commit across services is technically possible and operationally miserable — a coordinator failure leaves participants stuck holding locks. Sagas replace atomicity with compensating actions: each step commits locally and has a defined undo. **You give up “all or nothing” and get “eventually all, or eventually undone”, with the undo written by you.**

The sentence that separates candidates

I would not put a call to an external service inside a transaction. The transaction holds locks for its entire lifetime, so a payment provider having a slow afternoon becomes every other writer in my system waiting on their latency — and a transaction open for hours stops `VACUUM` reclaiming dead rows across the whole database, which is how one idle connection bloats tables it never touched. The pattern I would use instead is to commit the local state first, then make the external call outside the transaction, and reconcile with an idempotency key — which is [day 018's](#) answer arriving here for a different reason.

8 In the interview

How it gets asked

- “What does ACID stand for?” — the definition, and the answer that lands attaches a failure and a mechanism to each letter.
- “Walk me through a bank transfer.” — the canonical example. Two updates, one transaction.
- “Which of the four is the interesting one?” — isolation, because it has levels.
- “How does the database actually guarantee durability?” — the write-ahead log and `fsync`.
- “Two users transfer money at the same time. What goes wrong?” — the lost update, which a transaction alone does not prevent at the default isolation level.

What to say out loud, in the first ninety seconds

1. **Define a transaction before the acronym.** “A transaction is a group of operations treated as one indivisible unit — all of them take effect or none do.”
2. **Give each letter with a failure attached.** “Atomicity: the transfer can’t leave money debited and not credited. Consistency: it can’t leave a negative balance if I declared that constraint. Isolation: nobody else sees the moment when the money is in neither account. Durability: once commit returns, a power cut can’t undo it.”
3. **Name the mechanism for each.** “Atomicity and durability both come from the write-ahead log — append the intent, `fsync` it, and only then report success. Consistency is the constraints I declared. Isolation is locking, or MVCC.”
4. **Say which letter is odd.** “Consistency is arguably the odd one out — the other three are things the database does, and consistency is mostly rules I define and it enforces.”

5. **Say which is interesting.** *“Isolation is the one worth talking about, because it’s the only one with a dial. The others are on or off; isolation has levels, and each level permits specific anomalies.”*
6. **Give the practical rule.** *“And in practice the thing that matters most is keeping transactions short — no external calls inside one, because you’re holding locks the whole time.”*

The follow-ups

“How does the database actually guarantee durability?” With a write-ahead log. Before changing any data page, it appends a description of the change to a log file, and on `COMMIT` it calls `fsync` to force that log to physical storage and waits for the operating system to confirm — only then does the commit return successfully. The actual data pages are written later, in the background at a checkpoint. That ordering is the whole guarantee: if the machine loses power, the log is replayed on restart, transactions that reached commit are reapplied and those that did not are undone. The design is also *faster*, not just safer, because the log is written sequentially while the data pages are scattered — roughly four times the throughput on an SSD — and because many transactions committing at once can share a single `fsync`, which is called group commit and is why a single instance does tens of thousands of commits a second rather than the thousand you would get from one `fsync` each.

“Which of the four letters is the most interesting, and why?” Isolation, because it is the only one that comes in degrees. Atomicity, durability and consistency are essentially on or off — a database either rolls back partial work or it does not. Isolation has levels, because full serialisability is expensive, so databases offer weaker ones that permit specific, named anomalies in exchange for concurrency: dirty reads, non-repeatable reads, phantom reads, and lost updates. Postgres defaults to read committed, which prevents dirty reads and permits the rest. So choosing an isolation level is a real design decision with real consequences, and the anomalies it allows are things you then have to handle in the application — typically with explicit locking, or by doing the arithmetic in the database rather than in application code.

“Two users transfer money from the same account at the same time. What happens?” It depends entirely on how the update is written, and this is where a transaction alone is not enough. If I write `UPDATE accounts SET balance = balance - 5000 WHERE id = 1`, the database performs the arithmetic itself under a row lock, so the two transfers serialise and both are applied correctly. If instead the application reads the balance, subtracts in code, and writes the result back, then at Postgres’s default read-committed level both transactions can read 10,000, both compute 5,000, and both write it — one transfer disappears with no error. That is a lost update, and being inside a transaction does not

prevent it. The fixes are to do the arithmetic in the SQL, to take an explicit lock with `SELECT ... FOR UPDATE`, to use optimistic locking with a version column, or to raise the isolation level to repeatable read so the second transaction is aborted and can retry.

“Why not just use transactions everywhere, across services too?” Because across a network the guarantee gets very expensive. Within one database it is nearly free — locks are local and the coordinator is the database itself. Across two databases you need two-phase commit: a coordinator asks every participant to prepare, and if all agree, tells them to commit. The problem is the window between the two phases — if the coordinator dies there, every participant is holding locks and cannot decide on its own whether to commit or abort, so it blocks, potentially indefinitely. In practice most systems avoid it and use sagas instead: a sequence of local transactions, each with a compensating action that undoes it. You trade atomicity for eventual consistency plus explicit undo logic that you have to write and test — and you accept that there are moments when the system is visibly half-done, which the business has to be able to tolerate.

A model answer

“A transaction is a group of database operations treated as one indivisible unit — all of them happen or none do — and ACID names the four guarantees it gives you. The clearest example is a bank transfer, which is two updates: subtract from one account, add to the other.

Atomicity is that there is no state where the money has left one account and not arrived in the other. If the process dies between the two updates, the whole thing is rolled back and it is as if nothing ran. The mechanism is the write-ahead log: on restart the database replays the log, reapplies anything that reached commit, and undoes anything that did not.

Consistency is that the database’s own rules hold before and after. If I declared ``CHECK (balance`

`= 0)``, a transfer that would overdraw the account cannot commit, regardless of what the application code thinks. I’d add that C is arguably the odd letter out — the other three are things the database does for you, and consistency is mostly rules you define and it enforces.

Isolation is that no other transaction sees the moment in the middle when the ₹5,000 is in neither account. A report running concurrently either sees both updates or neither, never one.

Durability is that once `COMMIT` returns, a power cut cannot undo it. The mechanism is that the log is `fsync`ed to physical storage *before* the commit is acknowledged — the data pages themselves are written later in the background, which is both safer and faster, because the log is a sequential append while the data pages are scattered.

If you asked which of the four is worth talking about, I’d say isolation, because it’s the only one with a dial. The others are on or off. Isolation has levels — read committed, repeatable read, serialisable — and each weaker level permits specific anomalies in exchange for concurrency, which becomes something you have to handle deliberately.

And the practical thing I’d add: keep transactions short. They hold locks for their whole lifetime, so an external HTTP call inside one makes every other writer wait on a third party’s latency — and a transaction left open for hours prevents `VACUUM` from cleaning up dead row versions across the entire database, which is how a single idle connection ends up bloating tables it never touched.”

9 Recall card

- **A transaction is one indivisible unit.** `BEGIN` ... `COMMIT`, and all of it or none of it.
- Atomicity and Durability both come from the **write-ahead log**: append the intent, `fsync`, *then* report success.
- Consistency is the constraints you declared — the odd letter out, since you define it and the database enforces it.
- Isolation is the interesting one, because it is the only one with **levels** — that is [day 034](#).
- **Keep transactions short.** No external calls inside one; batch bulk writes in thousands, not one at a time and not a million.

PRACTICE

Practice — Window plus hash map: the longest-substring family

DSA topic: Window plus hash map: the longest-substring family **System design topic:** Transactions and ACID

Code these, in this order

Four problems, all the same eight-line skeleton with a different condition. **Before writing a line of any of them, say the condition out loud** — what the map holds, and what makes the window invalid. That sentence is the problem; the rest is typing.

Before each one, ask:

1. What does the map hold, and is it a map at all — or does it collapse to one integer?
2. What is the invalid condition, in one sentence?
3. Does the condition use `len(count)` ? If yes, the `del` at zero is mandatory — say why.
4. Maximise, so: shrink while invalid, record after. Say it before you type it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	The skeleton itself, and whether <code>left</code> can ever move backwards.
2	Fruit Into Baskets	LeetCode 904 (Medium)	Recognising “at most 2 distinct” through a story about fruit.
3	Max Consecutive Ones III	LeetCode 1004 (Medium)	Noticing the map collapses to a single counter when the alphabet is size two.
4	Longest Repeating Character Replacement	LeetCode 424 (Medium)	The <code>window_len - max_freq > k</code> condition, and defending the <code>max_freq</code> optimisation.

On problem 1, break it both ways

Write the `while`-shrink version and get it passing. Then write the jump version with a last-seen map, and delete the `last[ch] ≥ left` guard. Run both on `"abba"`. The broken one returns 3 instead of 2.

Then answer, out loud:

1. At which index did `left` move backwards, and what did the window contain afterwards?
2. Why can the `while` version never have this bug?

3. Why does `left` moving backwards also destroy the `O(n)` argument?

On problem 2, forget the `del` on purpose

Write it correctly first — it is “longest with at most 2 distinct” wearing a costume. Confirm `[1,2,1]` gives 3 and `[1,2,3,2,2]` gives 4.

Then remove the `del` at zero and run `[1,2,3,2,2]` again. Watch the answer collapse. Say exactly which quantity `len(count)` stopped meaning, and finish the sentence: “the `del` is mandatory exactly when...”

On problem 3, say the simplification before you use it

Before coding, say: “two kinds of value, so the map collapses to one integer counting zeros.” Then write it with just `zeros`, no dictionary. Confirm `([1,1,1,0,0,0,1,1,1,1,0], 2)` gives 6.

Then say what the invalid condition would have been with a full map, and why the integer version is the same algorithm.

On problem 4, write the honest version first

Write it recomputing `max(count.values())` in the condition. Confirm `("AABABBA", 1)` gives 4. State the cost, and why `O(26)` per step is still `O(n)`.

Only then write the version that carries `max_freq` and never decreases it, and answer the question that actually gets asked: why is a stale, too-high `max_freq` safe? The argument is in §3 of the lesson — say it without looking, in three sentences.

The condition drill

For each, give the invalid condition and what the window carries, in under five seconds:

1. Longest substring without repeating characters.
2. Longest substring with at most k distinct characters.
3. Fruit into baskets.
4. Longest run of 1s after flipping at most k zeros.
5. Longest substring you can make uniform by changing k characters.
6. Does s2 contain a permutation of s1?

Number 6 is the odd one out — say why it is day 031’s shape, not day 032’s.

The ACID drill

Say the table from memory: for each letter — the failure it prevents, and the mechanism that delivers it. Four letters, eight facts, under a minute.

Then the two one-liners an interviewer listens for:

1. Which letter is the odd one out, and why?
2. Which letter is the interesting one, and why?

The commit-timeline drill

Say what happens on `COMMIT`, in order: where the changes already are, what gets `fsync` ed, when success is reported, and when the data pages are actually written. Then say why that ordering is both the safety and the speed.

The arithmetic drill

From memory, in under two minutes:

- One `fsync` at 1 ms — naive commits per second, and what group commit does to that number.
 - 1,000 rows inserted one transaction each, against one transaction total. Roughly how long each?
 - A transaction left open 4 hours on a table taking 10,000 updates a second — how many dead row versions pile up, and what does that do to the database?
 - `synchronous_commit = off` — how much faster, and what exactly can be lost?
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the length of the longest substring without repeating characters.* Ask about the alphabet first. Name the shape and where the recording goes. Say why the `del` at zero matters, why `left` never moves backwards, and finish with the `O(n)` argument.
 2. *What does ACID stand for, and why does it matter?* Each letter with a failure attached and a mechanism named. Then the two follow-up one-liners: the odd letter, and the interesting letter.
 3. *Two users transfer money from the same account at the same time. What can go wrong, and how do you stop it?* The lost update, why being inside a transaction does not prevent it at read committed, and two fixes — arithmetic in the SQL, or `SELECT ... FOR UPDATE`.
-

Before you move on

- [] I say the invalid condition in one sentence before writing any window-plus-map problem.

- [] I `del` at zero whenever `len(count)` is in the condition, and I can say why.
- [] I can name the input where a missing `last[ch] ≥ left` guard makes `left` go backwards.
- [] I notice when the map collapses to one integer, and I say so.
- [] I can give each ACID letter with a failure and a mechanism, from memory.
- [] I can say what happens on `COMMIT`, in order, including where the `fsync` sits.
- [] I answered all three questions above out loud.

At-most-K, and the exactly-K trick

DATA STRUCTURES AND ALGORITHMS

At-most-K, and the exactly-K trick

You can turn an exactly-K problem into two at-most-K problems and explain the subtraction.

SYSTEM DESIGN

Isolation levels and the anomalies they allow

You can name dirty read, non-repeatable read and phantom read, and which level stops each.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Count the subarrays with exactly k distinct integers.*
- *What isolation level would you use, and what anomaly are you accepting?*

1 What this is, and why they ask it

Every window so far has answered “what is the **longest**?” or “what is the **shortest**?”. Today’s windows answer “**how many**?” — count the subarrays that satisfy a condition, not find the best one. The counting version has its own one-line trick: **the number of valid subarrays ending at `right` is the window length**, so you add `right - left + 1` instead of taking a `max`. And when the question says **exactly k** instead of **at most k** , you do not fight the window — you count *at most k* , count *at most $k - 1$* , and subtract.

Interviewers love this family because it looks like yesterday’s material and is not. A candidate who has only memorised the longest-substring skeleton walks straight into it, writes a `max` where a sum belongs, and produces code that runs cleanly and answers a different question. *Subarrays with K Different Integers* — LeetCode 992 — is the standard hard version, and *Count Number of Nice Subarrays* and *Binary Subarrays With Sum* are the same trick asked at medium. The subtraction is the part they actually want to hear you explain.

2 The story

Sports day at the school where Shanti has taught for eleven years, and forty children are standing in one long line on the field, each wearing a house colour — red, blue, yellow or green.

The first game needs a team, and a team is any run of children standing next to each other. The rule this year is that a team must have children from **exactly two** houses in it. Not one, not three. The younger teacher, Priya, has been given the job of counting how many different teams are possible, and she has started badly. She walks the line trying to spot runs with exactly two colours, and she keeps losing her place, because inside every run that works there are shorter runs that have only one colour, and runs that start one child earlier suddenly have three.

Shanti watches for a minute and then tells her the trick she always uses. Stop counting the thing you want. Count something easier, twice.

First, count the runs with **at most two** colours. That one is friendly. Walk the line child by child, and for each child, work out how far back you can go before a third colour sneaks in. If you can go back five children, then five teams end at this child — the one of

length one, the one of length two, up to the one of length five. Add five and move on. The nice part, she says, is that the starting place never moves backwards as you walk forward, so you are never redoing work.

Then count the runs with **at most one** colour, the same way. Even friendlier.

Now subtract. Every run you counted the first time has one colour or two. Every run you counted the second time has one. What survives the subtraction is the runs with exactly two — which is the number the game needed, and neither count ever made you think about “exactly” at all.

Priya asks why she cannot just count exactly-two directly. Shanti says: because “too many colours” is a clean thing to fix — you shrink from the back until it is fixed, and it stays fixed. “Not enough colours” and “too many colours” at the same time is two jobs, and a walk in one direction does one job well.

3 The idea in plain English

Shanti’s line of children is the array. A run of children standing together is a **subarray** — from [day 024](#), a contiguous slice. Houses are values, and “how many houses in this run” is how many **distinct** values the window holds — `len(count)`, from [day 033](#). Today changes only the question: not *how long is the best window* but *how many windows qualify*.

Counting at-most-k: the window-length insight

Keep exactly yesterday’s window: grow `right`, and shrink from `left` while the window holds more than `k` distinct values. After the shrink, the window `nums[left..right]` is the **longest** valid window ending at `right`.

Here is the insight that turns a longest-finder into a counter. Every subarray that **ends at** `right` and starts at `left` or later is also valid — because dropping elements off the front can only remove values, never add new distinct ones. And every subarray ending at `right` that starts *before* `left` is invalid — that is exactly why `left` moved. So the valid subarrays ending at `right` are:

```
nums[left..right], nums[left+1..right], ..., nums[right..right]
```

and there are `right - left + 1` of them — **the window length**. Add that to a running total at every step, and the total is the number of valid subarrays anywhere, because every subarray ends somewhere, and each is counted at the one step where `right` sits on its last element.

One line changes from yesterday:

```
best = max(best, right - left + 1)    # yesterday: the longest
total += right - left + 1            # today: the count
```

That is Shanti’s “if you can go back five children, five teams end here”.

Why exactly-k resists a direct window

Yesterday’s shrink loop repairs one kind of wrongness: **too many**. The condition is monotonic — shrinking always moves you toward valid, and once valid you stop. “Exactly k” is wrong in two directions at once. A window can have too many distinct values *or too few*, and shrinking a too-many window can overshoot straight past “exactly k” into “too few”. The single `left` boundary cannot serve both jobs. That is Shanti’s “two jobs, one walk”.

The subtraction

So do not count exactly. Count at-most, twice:

```
exactly(k) = at_most(k) - at_most(k - 1)
```

The argument is one sentence of set thinking. Every subarray with at most `k` distinct values has either at most `k - 1` of them, or exactly `k` — those two groups do not overlap, and together they are everything `at_most(k)` counted. Subtract the first group and the second remains.

For `[1, 2, 1, 2, 3]` and `k = 2`: `at_most(2)` is 12, `at_most(1)` is 5, so `exactly(2)` is `12 - 5 = 7`. §4 counts all twelve and all five in front of you.

The same trick beyond distinct counts

“At most” is not about distinct values — it is about any condition where shrinking helps, the **monotonic** conditions from [day 032](#). Each of these is the identical skeleton with a different thing counted:

PROBLEM	“EXACTLY K” OF WHAT	WHAT THE WINDOW CARRIES
Subarrays with K Different Integers (992)	distinct values	a count map
Count Number of Nice Subarrays (1248)	odd numbers	one integer: odds in the window
Binary Subarrays With Sum (930)	ones (the sum)	one integer: the window sum
Subarray Product Less Than K (713)	— already an at-most question	one integer: the product

The last row is the gift version: the question already says “less than k ”, so there is no subtraction at all — just the counting window. And rows two and three show [day 033](#)’s observation again: when the property tracks one kind of thing, the map collapses to a single integer.

The one guard the subtraction needs

`exactly(k)` calls `at_most(k - 1)`. When `k` is 0 — or the target sum is 0 in problem 930 — that inner call receives `-1`, and no window has at most `-1` of anything. The function must return 0 for a negative `k`, and §7 shows what happens when it does not. It is one `if` at the top, and forgetting it is the most common way this family goes wrong.

4 The picture

Counting `at_most(2)` on `[1, 2, 1, 2, 3]` — watch the running total, not a best:

index	0	1	2	3	4
value	1	2	1	2	3
right	window after shrink				left
0	[1]				0
1	[1,2]				0
2	[1,2,1]				0
3	[1,2,1,2]				0
4	[2,3]				3
				adds	subarrays ending here
				1	[1]
				2	[2] [1,2]
				3	[1] [2,1] [1,2,1]
				4	[2] [1,2] [2,1,2] [1,2,1,2]
				2	[3] [2,3]
				12	= at_most(2)

What to notice: at `right = 4`, the value 3 arrives, the window briefly holds three distinct values, and `left` walks from 0 to 3 to repair it. The two subarrays counted there start at or after `left` — everything starting earlier would still contain all three values, which is exactly why `left` moved past it.

The same array, `at_most(1)` :

right	window	left	adds	(only single-value runs survive)
0	[1]	0	1	
1	[2]	1	1	
2	[1]	2	1	
3	[2]	3	1	
4	[3]	4	1	
			5	= at_most(1)

And the subtraction, as two nested boxes:

```
+-----+
| at_most(2) = 12 subarrays |
|                               |
| +-----+                   |
| | at_most(1) = 5           | the ring between |
| | (one distinct value)    | the boxes is     |
| +-----+                   | exactly(2) = 7  |
+-----+

12 - 5 = 7:  [1,2] [2,1] [1,2,1] [2,1,2] [1,2,1,2] [1,2] [2,3]
```

What to notice: the inner box sits entirely inside the outer one — every one-colour run is also an at-most-two-colours run. That containment is what makes the subtraction legal. Subtracting two counts that overlap only partially would be meaningless.

5 The code, built step by step

The counting window

```
def at_most_k_distinct(nums: list[int], k: int) → int:
    if k < 0:
        return 0
    count: defaultdict[int, int] = defaultdict(int)
    left = 0
    total = 0
```

The guard comes first, because `exactly` is about to call this with `k - 1`. An at-most-minus-one window can never exist, so the count is 0.

```
    for right, x in enumerate(nums):
        count[x] += 1
        while len(count) > k:
            count[nums[left]] -= 1
            if count[nums[left]] == 0:
                del count[nums[left]]
            left += 1
        total += right - left + 1
    return total
```

Yesterday's skeleton, line for line — including the `del` at zero, which is mandatory here for yesterday's reason: `len(count)` is the condition. The only new line is the last one. After the shrink, `nums[left..right]` is the longest valid window ending at `right`, and every later start also works, so `right - left + 1` subarrays end here. No `max`, no `best`.

The subtraction

```
def exactly_k_distinct(nums: list[int], k: int) → int:
    return at_most_k_distinct(nums, k) - at_most_k_distinct(nums, k - 1)
```

Two passes over the array, one subtraction. This two-line function is the answer to a problem LeetCode marks **hard**.

Nice subarrays: the map collapses

Count Number of Nice Subarrays — LeetCode 1248 — asks for subarrays with exactly `k` **odd** numbers. Odd or even is two kinds of thing, so the map collapses to one integer, as in [day 033](#):

```
def at_most(nums: list[int], k: int) → int:
    if k < 0:
        return 0
    left = odds = total = 0
    for right, x in enumerate(nums):
        odds += x % 2
        while odds > k:
            odds -= nums[left] % 2
            left += 1
        total += right - left + 1
    return total
```

`x % 2` is 1 for odd and 0 for even, so `odds` is a count of odd values in the window. Everything else is unchanged.

Binary subarrays with sum: the same collapse

Binary Subarrays With Sum — LeetCode 930 — counts subarrays of 0s and 1s summing to exactly `goal`. The sum of 0s and 1s is a count of 1s, so this is the previous problem wearing different clothes — and here the `goal = 0` case makes the negative guard load-bearing, because `exactly(0)` calls `at_most(-1)`.

Product less than k: no subtraction needed

Subarray Product Less Than K — LeetCode 713 — is already an at-most question, with one edge of its own:


```
def num_subarray_product_less_than_k(nums: list[int], k: int) → int:
    if k ≤ 1:
        return 0
    left = total = 0
    product = 1
    for right, x in enumerate(nums):
        product *= x
        while product ≥ k:
            product /= nums[left]
            left += 1
        total += right - left + 1
    return total
```

The guard is `k ≤ 1` this time. The values are positive integers, so every window's product is at least 1, and “product strictly less than 1” can never be satisfied — without the guard the shrink loop runs off the end of the array. §7 has the crash.

The complete solutions

```
from collections import defaultdict

def at_most_k_distinct(nums: list[int], k: int) → int:
    """Count subarrays holding at most k distinct values. The guard feeds exactly()."""
    if k < 0:
        return 0
    count: defaultdict[int, int] = defaultdict(int)
    left = 0
    total = 0
    for right, x in enumerate(nums):
        count[x] += 1
        while len(count) > k:
            count[nums[left]] -= 1
            if count[nums[left]] == 0:
                del count[nums[left]]          # len(count) is the condition: del is mandatory
            left += 1
        total += right - left + 1             # every start from left to right works
    return total

def exactly_k_distinct(nums: list[int], k: int) → int:
    """LeetCode 992. The whole trick."""
    return at_most_k_distinct(nums, k) - at_most_k_distinct(nums, k - 1)

def number_of_nice_subarrays(nums: list[int], k: int) → int:
    """LeetCode 1248. Exactly k odd numbers: the map collapses to one integer."""
    def at_most(k: int) → int:
        if k < 0:
            return 0
        left = odds = total = 0
        for right, x in enumerate(nums):
            odds += x % 2
            while odds > k:
                odds -= nums[left] % 2
                left += 1
            total += right - left + 1
        return total
    return at_most(k) - at_most(k - 1)

def num_subarrays_with_sum(nums: list[int], goal: int) → int:
    """LeetCode 930. Sum of 0s and 1s is a count of 1s; goal = 0 needs the guard."""
    def at_most(goal: int) → int:
        if goal < 0:
            return 0
        left = window_sum = total = 0
        for right, x in enumerate(nums):
            window_sum += x
            while window_sum > goal:
                window_sum -= nums[left]
                left += 1
            total += right - left + 1
        return total
    return at_most(goal) - at_most(goal - 1)

def num_subarray_product_less_than_k(nums: list[int], k: int) → int:
    """LeetCode 713. Already an at-most question;  $k \leq 1$  can never be satisfied."""
    if k ≤ 1:
        return 0
    left = total = 0
    product = 1
    for right, x in enumerate(nums):
        product *= x
        while product ≥ k:
            product /= nums[left]
```

```

        left += 1
        total += right - left + 1
    return total

if __name__ == "__main__":
    print(exactly_k_distinct([1, 2, 1, 2, 3], 2))          # 7
    print(exactly_k_distinct([1, 2, 1, 3, 4], 3))          # 3

    print(number_of_nice_subarrays([1, 1, 2, 1, 1], 3))    # 2
    print(number_of_nice_subarrays([2, 4, 6], 1))          # 0
    print(number_of_nice_subarrays([2, 2, 2, 1, 2, 2, 1, 2, 2, 2], 2)) # 16

    print(num_subarrays_with_sum([1, 0, 1, 0, 1], 2))      # 4
    print(num_subarrays_with_sum([0, 0, 0, 0, 0], 0))      # 15 ← the guard at work

    print(num_subarray_product_less_than_k([10, 5, 2, 6], 100)) # 8
    print(num_subarray_product_less_than_k([1, 2, 3], 0))  # 0

```

Run it. Eight answers, all correct, including the two edge rows the guards exist for.

6 What it costs

One at-most pass

The counting window is [day 032](#)'s argument unchanged: `right` advances exactly `n` times, `left` only ever advances and never passes `right`, so at most `n` moves. At most `2n` pointer moves, each with constant map work. **$O(n)$ time**. Space is the map: **$O(k)$** entries at most — and $O(1)$ when the map has collapsed to an integer.

The subtraction doubles it, and no more

`exactly(k)` runs the pass twice:

```

at_most(k)      : up to 2n moves
at_most(k - 1) : up to 2n moves
total           : 4n moves          → still  $O(n)$ 

```

Two passes is a constant factor, not a complexity change — the same distinction [day 033](#) drew for `max_freq`. Say “two linear passes” and nobody will blink.

Against the brute force

Counting by checking every subarray: $n(n+1)/2$ subarrays, and checking the distinct count of each from scratch costs up to $O(n)$ more:

```

n = 10,000:  brute ≈ 50,000,000 subarrays × up to 10,000 each ≈  $5 \times 10^{11}$  ops
              window ≈ 40,000 pointer moves

```

Even the smarter brute force — fix the start, extend the end, grow a set as you go — is $n^2/2 = 50,000,000$ steps against 40,000. That is the gap the interviewer wants named.

The number to have ready

Two at-most passes, each $O(n)$ because both pointers only move forward, then one subtraction — $O(n)$ time, $O(k)$ space for the map. The brute force enumerates $n^2/2$ subarrays, which at $n = 10,000$ is fifty million against forty thousand pointer moves.

7 The traps

The near-miss: `max` where the sum belongs

Yesterday's reflex, applied to today's question:

```
while len(count) > k:
    ...
    best = max(best, right - left + 1)    # counting problem, longest-window line
```

On `[1, 2, 1, 2, 3]` with `k = 2` this returns `4` — the length of `[1,2,1,2]` — when the question asked for the count 7. It runs, it looks like every window you have ever written, and it answers the wrong question. **Read the ask: “how many” means `total +=`, “longest” means `max`**. Say which one you are doing before you type it.

The near-miss: at-most where exactly was asked

The counting window alone, with `len(count) > k` as the condition, happily returns 12 for `[1, 2, 1, 2, 3]`, `k = 2` — the at-most count, not the exactly count of 7. Nothing crashes. The tell is in your own test: build one tiny example by hand, count the qualifying subarrays with your eyes, and 12 against 7 exposes it immediately. This is why the two-minute hand-count is not optional.

The real error: the missing negative guard

`exactly(0)` — or `goal = 0` in Binary Subarrays With Sum — calls `at_most(-1)`. Without the guard, the shrink loop can never be satisfied: the window sum is always at least 0, which is greater than `-1`, so `left` marches past `right`, keeps subtracting elements it never should, and runs off the end:

```

Traceback (most recent call last):
  File "day34.py", line 20, in <module>
    print(at_most([0, 0, 1, 0], 0) - at_most([0, 0, 1, 0], -1))
                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "day34.py", line 14, in at_most
    window_sum -= nums[left]
                  ~~~~~^~~~~~
IndexError: list index out of range

```

One line prevents it: `if goal < 0: return 0`. And note the sneakier variant: add a `left ≤ right` guard to the `while` instead, and the crash disappears but the counting silently continues with an empty window — plausible numbers, wrong answer, no traceback.

The real error: product less than 1

The same failure wears a second costume in LeetCode 713. Positive integers mean every product is at least 1, so `k = 1` can never be satisfied and the shrink loop eats the whole array:

```
print(count([1, 1, 1], 1))      # no k ≤ 1 guard
```

```

Traceback (most recent call last):
  File "day34.py", line 14, in <module>
    print(count([1, 1, 1], 1))
  File "day34.py", line 9, in count
    product /= nums[left]
IndexError: list index out of range

```

And on `[2, 3]` with `k = 1` the same code does not crash — it returns `-1`. A count of minus one subarrays, delivered with a straight face. `right - left + 1` goes negative once `left` passes `right`, and the total absorbs it silently. **Guard `k ≤ 1` with `return 0`.**

The near-miss: forgetting the `del`, again

The condition is `len(count) > k`, so [day 033](#)'s rule applies with full force: decrement, then delete at zero. Leave zero entries behind and `len(count)` counts ghosts, the shrink loop overshrinks, and every window's contribution is too small. The count comes out wrong with no error — and because it is a count, not a length, you cannot eyeball it from the output the way you could spot a too-short “longest”.

The contract corner: what exactly counts

Ask what `k = 0` should return — for distinct values it is 0 by convention, for a sum it is a real question with real subarrays answering it, as `[0,0,0,0,0]` with `goal = 0` returning 15 shows. And confirm whether “integers” can be negative: for *distinct counts* negatives change nothing, but the moment the condition is a **sum** of arbitrary integers, monotonicity dies — [day 032](#)’s rule — and this whole family hands over to prefix sums, which arrive on [day 038](#).

8 In the interview

How it gets asked

- “Count the subarrays with exactly k distinct integers.” — LeetCode 992, marked hard, and two lines once you own the trick.
- “Count the subarrays containing exactly k odd numbers.” — LeetCode 1248, the same problem after one observation.
- “How many subarrays of this 0/1 array sum to the target?” — LeetCode 930, where the target 0 edge hides.
- “Count the subarrays whose product is under 100.” — LeetCode 713, the version with no subtraction needed.
- And the follow-up form: you solve “longest with at most k distinct”, and the interviewer leans in — “now count the subarrays with exactly k .”

What to say out loud, in the first ninety seconds

1. **Name the shift.** “This is a counting question, not a longest question — so I’ll accumulate, not take a max.”
2. **State the counting insight.** “With an at-most window, after the shrink, every subarray ending at `right` and starting at or after `left` is valid — that’s `right - left + 1` new subarrays per step, because removing elements from the front can never add distinct values.”
3. **Explain why exactly resists.** “Exactly- k is wrong in two directions — too many and too few — and one left boundary can only repair one of them.”
4. **Give the subtraction.** “So: subarrays with at most k , minus subarrays with at most $k - 1$. Those with at most $k - 1$ sit entirely inside those with at most k , so the difference is exactly k .”
5. **Flag the guard.** “`at_most(k - 1)` gets -1 when k is 0, and my helper returns 0 for negative k — otherwise the shrink loop runs off the array.”
6. **Give the costs.** “Two linear passes and a subtraction — $O(n)$ time, $O(k)$ space for the map.”

The follow-ups

“Why is the number of valid subarrays ending at `right` equal to the window length?” Because validity here survives shortening. After the shrink, `nums[left..right]` has at most `k` distinct values. Any subarray ending at `right` that starts later than `left` is that window with elements removed from the front — and removing elements can only lose values, never introduce new ones, so its distinct count is at most the window’s. That gives one valid subarray per start position from `left` to `right: right - left + 1` of them. And nothing starting before `left` qualifies, because `left` only moved when the window holding those elements exceeded `k`. Both halves matter — the count is not just an upper bound, it is exact — and the property doing the work is monotonicity: the condition can only improve as the window shortens. A condition without that property, say “the window’s maximum minus minimum is exactly 5”, would not let me count this way.

“Can you do exactly-`k` in one pass instead of two?” Yes — keep two left boundaries over the same walk. `left1` is the tightest start with at most `k` distinct values, `left2` the tightest with at most `k - 1`; every start in between gives exactly `k`, so each step adds `left2 - left1`. It is one pass with two maps, against two passes each with one — the same total work, $O(n)$ either way, so it is a constant-factor rearrangement rather than an improvement. I would write the subtraction version in an interview: two calls to a helper I have already tested, one line of arithmetic, and far less state to get wrong under pressure. I would name the three-pointer version only to show I know the trade I am making.

“What if the array contains negative numbers?” For distinct-count conditions, nothing changes — distinctness does not care about sign, and the monotonicity that justifies the counting still holds. But if the condition is a *sum* — count the subarrays summing to exactly `k` over arbitrary integers — the trick dies, because with negatives a window’s sum does not shrink as the window shrinks, so “at most” is no longer a monotonic condition and the two-boundary walk is unjustified. That is day 032’s rule surfacing again. The right tool there is a prefix-sum map — count, for each position, how many earlier prefix totals differ from the current one by exactly `k` — which is $O(n)$ with a hash map and handles negatives natively. That technique is a few days ahead of me in this course, but I would name it as the correct escape hatch.

A model answer

“Count the subarrays with exactly k distinct integers — this is a counting problem, so I will not be taking a max of window lengths; I will accumulate a total.

The direct version is awkward, because ‘exactly k ’ can be violated in two directions — too many distinct values or too few — and a sliding window’s left edge only knows how to repair ‘too many’. So I’ll use the standard decomposition: the answer is the number of subarrays with at most k distinct values, minus the number with at most $k - 1$. Every at-most- $(k-1)$ subarray is also an at-most- k subarray, so the subtraction leaves exactly the exactly- k ones.

Counting at-most is where the window earns its keep. I grow `right`, shrink while the map holds more than k distinct values, and then every subarray ending at `right` starting anywhere from `left` onward is valid — shortening from the front can only remove values. So each step contributes the window length, `right - left + 1`.

```
def at_most(nums, k):
    if k < 0:
        return 0
    count, left, total = defaultdict(int), 0, 0
    for right, x in enumerate(nums):
        count[x] += 1
        while len(count) > k:
            count[nums[left]] -= 1
            if count[nums[left]] == 0:
                del count[nums[left]]
            left += 1
        total += right - left + 1
    return total

def exactly_k(nums, k):
    return at_most(nums, k) - at_most(nums, k - 1)
```

Two details are load-bearing. The `k < 0` guard: `exactly(0)` calls `at_most(-1)`, and without the guard the shrink loop can never be satisfied and walks off the array. And the `del` when a count hits zero, because `len(count)` is the loop condition — leave ghosts behind and it overshrinks.

Cost: each pass is $O(n)$ — both pointers only move forward, at most $2n$ moves — so two passes and a subtraction is $O(n)$ time, $O(k)$ space. On `[1,2,1,2,3]` with $k = 2$: at-most-2 is 12, at-most-1 is 5, answer 7 — and I’d check that little example by hand before trusting the code.”

9 Recall card

- **Counting window:** `total += right - left + 1` — after the shrink, every start from `left` to `right` gives a valid subarray ending here.
- **Why it works: shortening never hurts** — removing front elements cannot add distinct values. Monotonicity is the licence to count this way.
- `exactly(k) = at_most(k) - at_most(k - 1)` — `at-most-(k-1)` nests inside `at-most-k`, so the difference is exactly-`k`. Two linear passes, still `O(n)`.
- **Guard the inner call: negative k returns 0** — else `at_most(-1)` walks off the array (`IndexError`), and product-less-than-k needs `k ≤ 1 → 0` for the same reason.
- “How many” → **accumulate**; “longest” → **max**. Say which before typing. And sums with negatives break the trick — that is prefix sums, [day 038](#).

1 What this is, and why they ask it

Yesterday established that isolation is the only ACID letter with a dial. Today is the dial. There are four standard **isolation levels** — read uncommitted, read committed, repeatable read, serialisable — and each weaker level permits specific, named **anomalies**: ways concurrent transactions can see things that a one-at-a-time world could never show them. Choosing a level is choosing which anomalies you are prepared to handle yourself.

This is a favourite interview question because it has a memorisable table on the surface and real understanding underneath, and the two are easy to tell apart. Anyone can recite “repeatable read stops non-repeatable reads”. The candidate who gets hired can describe each anomaly as a story with two transactions in it, say what their database’s default actually permits, and answer the hub question — *which level, and what are you accepting?* — with a decision rather than a definition. It appears in backend interviews at every product company, usually right after ACID, and the follow-up about **write skew** is where senior candidates are separated from the table-reciters.

2 The story

Kavita caters weddings, and the part of the job that has nothing to do with cooking is the guest count. The count lives in the family’s phone chat, and the family will not stop touching it.

Three things have gone wrong often enough that she has names for them.

The first she calls the taken-back number. Two years ago, a groom’s uncle typed that ninety extra people were coming from Nagpur, and Kavita saw it and ordered two more vats of oil. An hour later he deleted the message — it was the wrong chat, the ninety people were for a different function. The order was already placed. She had acted on something that was never actually true.

The second is the number that will not sit still. On a Thursday she checked the chat at ten in the morning — 240 guests — and planned the plates. At noon she checked again to plan the sweets, and it said 265. Same job, same day, two answers. Nothing was wrong, exactly. The family had every right to change it. But her plates and her sweets were now planned for two different weddings.

The third is the count that grows while you are counting. She once went through the chat adding up the out-of-town guests, got 38, and went through again to be sure — 41. Three new names had arrived between her first pass and her second, typed in by an aunt while Kavita was still counting.

Her fix, these days, is a rule. At noon on the day she plans, she takes a screenshot of the list and works only from the screenshot. The chat can boil all it likes; her plates, sweets and seating all come from one frozen moment, so they at least agree with each other. Anything typed after noon goes into the next day's screenshot.

And for the last hour before the wedding, she asks the family to stop editing altogether. They grumble. It is the only way the seating chart and the reality in the hall come out the same.

3 The idea in plain English

Kavita's three disasters are the three read anomalies, in the official order. Her screenshot and her stop-editing hour are two isolation levels. All that changes in the technical version is that the family chat is a table, and Kavita and the family are **concurrent transactions**.

The three read anomalies

Dirty read — the taken-back number. A transaction reads data that another transaction has written but **not yet committed**. If the writer rolls back, the reader acted on a value that officially never existed. Kavita bought oil for ninety guests who were never coming.

Non-repeatable read — the number that will not sit still. A transaction reads the same row twice and gets different values, because another transaction **committed a change** in between. Both reads saw committed truth; they just saw different days of it. Plates for 240, sweets for 265.

Phantom read — the count that grows while you are counting. A transaction runs the same *search* twice — “all out-of-town guests” — and the second run returns **rows that did not exist the first time**. The difference from a non-repeatable read is worth being able to say: non-repeatable is an existing row *changing*; a phantom is a *new row appearing* that matches your condition.

And from yesterday, the fourth horseman, which is a write anomaly rather than a read one: the **lost update** — two transactions read the same value, both compute, both write, one overwrites the other. Keep it in the set, because interviewers do.

The four levels, and what each one buys

Each level is defined by which anomalies it forbids:

LEVEL	DIRTY READ	NON-REPEATABLE READ	PHANTOM READ
Read uncommitted	allowed	allowed	allowed
Read committed	stopped	allowed	allowed
Repeatable read	stopped	stopped	allowed by the standard
Serialisable	stopped	stopped	stopped

Read uncommitted is the no-rules chat: you can see half-typed messages. Almost nothing runs here.

Read committed — Postgres’s default — promises only that you never see uncommitted work. Every statement sees the freshest committed truth, which is precisely why two statements in one transaction can disagree: truth moved between them. This is Kavita checking the live chat twice.

Repeatable read is the screenshot. The transaction takes a **snapshot** at its first read and every statement in it sees that snapshot, however long it runs. Internally consistent, possibly slightly stale.

Serialisable is the stop-editing hour. The database guarantees the outcome is the same as if the transactions had run one at a time, in some order — and anything that cannot be made to look serial is **aborted** and must retry.

The screenshot has a hole in it

Here is the subtlety the follow-up questions live in. Repeatable read gives every transaction a consistent *picture*, but two transactions can each act on their own picture and produce a combined result no serial order could ever create. The classic is **write skew**: two rows, an invariant across both, and two transactions that each check the invariant on their snapshot, then write to *different* rows. Each write is fine alone; together they break the rule. Snapshots cannot catch it, because neither transaction touched the row the other wrote. Only serialisable — which watches what you *read*, not just what you wrote — stops it. §5 has the worked example.

4 The picture

The three anomalies, as timelines. Time runs left to right; each line is one transaction.

DIRTY READ (stopped by: read committed and above)

```
T1: BEGIN -- UPDATE stock = 0 ----- ROLLBACK
T2:      SELECT stock → 0
      ^ read a value that was never committed:
      T2 refuses an order it could have taken
```

NON-REPEATABLE READ (stopped by: repeatable read and above)

```
T1: ----- UPDATE price = 120; COMMIT
T2: BEGIN -- SELECT price → 100 ----- SELECT price → 120 -- COMMIT
      ^ both reads saw committed data,      ^ same row, same
      just from different moments            transaction, new answer
```

PHANTOM READ (stopped by: serialisable; in Postgres, by repeatable read too)

```
T1: ----- INSERT guest ('Pune'); COMMIT
T2: BEGIN -- COUNT guests in Pune → 8 ----- COUNT guests in Pune → 9
      ^ no existing row changed:                ^ a NEW row appeared
      the SET of matching rows grew              inside one transaction
```

What to notice: in all three, every individual read is perfectly reasonable. The anomaly is only visible when you compare two moments — which is exactly why code that “worked in testing”, where nothing ran concurrently, produces these in production.

The levels, as how much of the chat Kavita is looking at:

read uncommitted	the live chat, half-typed messages included
read committed	the live chat, sent messages only ← Postgres default
repeatable read	the noon screenshot
serialisable	the screenshot, plus nobody may edit anything you LOOKED AT until you finish

5 How it actually works

Snapshots, not locks: MVCC does the reading

Postgres, and InnoDB under MySQL, deliver read isolation with **multi-version concurrency control** from [day 033](#): writers create new row versions rather than overwriting, so readers never block writers. The difference between the two everyday levels is only **when the snapshot is taken**:

- **Read committed:** a fresh snapshot per *statement*. Each statement sees everything committed before it started. Hence non-repeatable reads between statements.
- **Repeatable read:** one snapshot per *transaction*, taken at the first read. Every statement sees it. In Postgres this snapshot also makes phantoms impossible — the second `COUNT` cannot see rows committed after the snapshot — so **Postgres repeatable read**

is stronger than the standard requires. The standard's table says "phantoms allowed"; Postgres's implementation says no.

Two Postgres quirks worth stating in an interview. First, `READ UNCOMMITTED` exists as a keyword but **behaves as read committed** — Postgres never shows uncommitted data to anyone. Second, MySQL's InnoDB defaults to repeatable read, not read committed, so "what is the default?" has a per-database answer.

What repeatable read does on a write conflict

Snapshots handle reads. When two repeatable-read transactions **update the same row**, the second waits for the first to commit, and then Postgres refuses to apply a write on top of data that changed under the snapshot:

```
ERROR:  could not serialize access due to concurrent update
```

The application must catch this and **retry the whole transaction**. That error is not a bug — it is repeatable read doing its job, and it is how repeatable read stops lost updates that read committed would allow through silently.

Write skew: the anomaly the snapshot cannot see

The canonical example: a hospital roster requires **at least one doctor on call**. Alice and Bob are both on call. Both try to go off duty at the same moment, at repeatable read:

```
invariant: at least one row must have on_call = true

T1 (Alice): count on-call doctors → 2    [her snapshot]
T2 (Bob):   count on-call doctors → 2    [his snapshot]
T1: UPDATE alice SET on_call = false; COMMIT -- fine: 2 → 1
T2: UPDATE bob   SET on_call = false; COMMIT -- fine on HIS snapshot: 2 → 1

result: zero doctors on call. No serial order could produce this.
```

No error is raised, because the transactions wrote to **different rows** — the concurrent-update check never fires. Each decision was valid on its own snapshot; the combination is invalid. The same shape hides in "book the seat if it is free", "spend if the balance across accounts stays positive", "claim the username if unused" — any **check-then-act across rows**.

Serialisable: how the database catches it

Postgres's serialisable level is **serialisable snapshot isolation (SSI)**: repeatable read plus tracking of read/write **dependencies** between concurrent transactions. When it detects a pattern that could not occur in any serial order — Alice read what Bob wrote around, and Bob read what Alice wrote around — it aborts one of them:

```
ERROR: could not serialize access due to read/write dependencies among transactions
HINT: The transaction might succeed if retried.
```

The hint is literal: **serialisable code must run inside a retry loop**. That is the real cost of the level — not slow reads, but a background abort rate that your application has to absorb. SQL Server and older systems instead take **range locks** at serialisable, blocking writers that would create phantoms; same guarantee, different price — waiting instead of retrying.

The targeted alternative: lock exactly what you check

You rarely raise the level globally. At read committed, you can buy just the strength you need:

```
SELECT * FROM doctors WHERE on_call = true FOR UPDATE;
```

`FOR UPDATE` locks the rows you read, so Bob's identical `SELECT` waits until Alice commits — then sees one doctor and refuses. This converts the write skew into a wait. It works when you can name the rows to lock; it cannot protect a *count of rows that do not exist yet*, which is where serialisable or a separate constraint-carrying row comes in. Tomorrow — locking and deadlocks — is entirely about this tool and its failure mode.

6 The numbers

The price of each step up

Read committed to repeatable read is nearly free on reads — same MVCC machinery, one snapshot instead of many. The costs appear as aborts and as bloat:

```
repeatable read, two writers on the same row:
  loser gets "could not serialize access due to concurrent update"
  cost = one full transaction retry
```

```
serialisable, measured on TPC-C-like workloads (the Postgres SSI paper):
  low contention : ~5-10% throughput below repeatable read
  high contention : abort rates climb into double digits, and
                   every abort is a retry your app must perform
```

What a retry rate does to capacity

```
1,000 transactions/second at serialisable
3% abort rate          → 30 retries/second
each retry repeats ~5 ms of work → 150 ms of extra work per second (0.15 cores)

but at 20% aborts on a hot row:
200 retries/s, some aborting AGAIN on retry
→ effective throughput can fall by a third while CPU stays busy
```

The multiplication to say out loud: **abort rate** × **retry cost is a tax on every write**, and it grows with contention, not with data size. Serialisable on a low-contention workload is cheap. Serialisable on one hot counter row is a retry storm.

The cost of check-then-act done wrong, for scale

```
seat-booking site, 2,000 bookings/minute at peak, read committed, no locks:
check-seat-free → act window of ~10 ms per booking
two requests for the same seat inside 10 ms = a double booking

popular concert, 500 requests hit one seat in the first second:
window overlap is a certainty, not a risk -- every hot seat double-books
```

That is why “we will just check first” fails precisely when the product succeeds.

The snapshot you hold open

A repeatable-read transaction holds its snapshot for its whole life, so [day 033](#)’s bloat arithmetic applies with a sharper edge:

```
a 30-minute repeatable-read report on a database taking 5,000 updates/second:
5,000 × 1,800 s = 9,000,000 dead row versions VACUUM cannot reclaim
at ~100 bytes each ≈ 900 MB of bloat, from one report
```

Long reports at repeatable read are fine — that is what the level is for — but they are a scheduled cost, not a free lunch. Run them on a replica when they grow.

7 The trade-offs

Choose per transaction, not per database

The level is a property of each transaction (`BEGIN ISOLATION LEVEL SERIALIZABLE`), and the strong move in an interview is to refuse the global question. Payments and inventory adjustments: serialisable, or read committed with `FOR UPDATE`, inside a retry loop. Reports and exports: repeatable read, so every table in the report agrees with every other. Everything else: the default, with the arithmetic done in SQL — `SET balance = balance - 5000` — so lost updates cannot happen.

Read committed plus discipline, or serialisable plus retries

Most production systems run read committed and handle the anomalies that matter **explicitly**: database-side arithmetic, `FOR UPDATE` on check-then-act, unique constraints to kill duplicate inserts. That is more code and more thinking, and it is fast and predictable. Serialisable replaces all that thinking with one guarantee and one obligation — **every transaction must be retryable** — which means no side effects inside transactions (no emails, no HTTP calls: yesterday's rule with a second reason now).

I would not use it if...

I would not use serialisable if the workload concentrates writes on a few hot rows and I cannot afford retry storms — a like-counter, a stock ticker. I would use database-side arithmetic there, which needs no isolation at all. **I would not stay at read committed** if the money path contains a check-then-act across rows — that is write skew territory, and “it passed the tests” means only that the tests did not run concurrently. **I would not use repeatable read for a quick single-statement read** — a lone `SELECT` at read committed is already a consistent snapshot of one moment; the level buys nothing there.

The honest sentence

Most teams run Postgres at read committed and never change it, and the ones that get away with it are the ones that know *which* anomalies they are accepting and where. The level is not a safety setting you turn up; it is a contract about which failures are yours to prevent.

8 In the interview

How it gets asked

- “What isolation level would you use, and what anomaly are you accepting?” — the hub question, and it wants a decision with reasons, not the table.
- “What is the difference between a non-repeatable read and a phantom read?” — the classic discriminator; existing row changing versus new row appearing.
- “Your default is read committed. What can still go wrong?” — lost updates, non-repeatable reads, phantoms, write skew: everything except dirty reads.
- “Two transactions each check a condition and then write, and the invariant breaks anyway. What happened?” — write skew, being described to see if you can name it.

What to say out loud, in the first ninety seconds

1. **Frame the dial.** *“Isolation comes in four levels, and each weaker level admits specific named anomalies in exchange for concurrency — so the question is which anomalies this workload can tolerate.”*
2. **Name the anomalies as one-liners.** *“Dirty read: seeing uncommitted work. Non-repeatable read: the same row answering differently twice. Phantom: new rows appearing between two runs of the same search. And the write-side one: the lost update.”*
3. **State the defaults.** *“Postgres defaults to read committed — no dirty reads, everything else possible. MySQL’s InnoDB defaults to repeatable read.”*
4. **Give the decision rule.** *“I choose per transaction: read committed plus SQL-side arithmetic for ordinary writes, repeatable read for multi-statement reports, and serialisable — in a retry loop — for check-then-act invariants that span rows.”*
5. **Show the depth card early.** *“The subtle failure is write skew: two transactions check an invariant on their own snapshots and write different rows, so no conflict is detected — snapshot levels can’t see it; serialisable can.”*

The follow-ups

“Explain write skew, and why repeatable read does not stop it.” Write skew is two transactions reading an overlapping set of rows, each validating an invariant against its snapshot, then writing to *different* rows so that the combined result violates the invariant. The on-call roster is the standard example: two doctors, a rule that one must stay on call, and both go off duty simultaneously — each saw two on-call doctors in its snapshot, each reduced the count to one, and reality went to zero. Repeatable read cannot stop it because its machinery detects conflicting writes to the *same* row, and here there are none; the conflict is between what one transaction *read* and what the other *wrote*. Serialisable — SSI in Postgres — tracks exactly those read/write dependencies and aborts one transaction. The lightweight alternative at lower levels is to make the reads conflict deliberately: `SELECT ... FOR UPDATE` on the rows you checked, so the second transaction waits and then re-checks against the new truth.

“You are at read committed. How do you prevent a lost update?” Four tools, in the order I would reach for them. First, do the arithmetic in the database — `UPDATE accounts SET balance = balance - 5000` — because the read and the write become one atomic statement under a row lock, and the anomaly is impossible. Second, if the logic must live in the application, `SELECT ... FOR UPDATE` to lock the row across the read-think-write gap. Third, optimistic locking: carry a version column, write with `UPDATE ... SET version = version + 1 WHERE id = ? AND version = ?`, and treat zero affected rows as “someone got there first — reload and retry”; that is the right tool when contention is rare and you would rather not hold locks. Fourth, raise that transaction to

repeatable read and let `could not serialize access due to concurrent update` force the retry. What I would not do is read, compute in code, and write back with no guard — that is the lost update, and it fails silently.

“Does Postgres actually match the standard’s table?” No, in three ways worth knowing. `READ UNCOMMITTED` is accepted syntax but behaves as read committed — Postgres never exposes uncommitted rows, because MVCC always has a committed version to show instead. Repeatable read is stronger than the standard requires: the standard permits phantoms at that level, but a Postgres snapshot covers the whole transaction, so phantoms cannot occur either. And serialisable is implemented as serialisable snapshot isolation — optimistic detection and abort — rather than the range-locking the standard’s authors had in mind, so its cost appears as retries, not as blocking. The general lesson is that the standard defines levels by which anomalies they *forbid*, and implementations are free to forbid more — so the answer to “what does repeatable read allow?” is per-engine, and saying that is itself a good signal.

A model answer

“I’d pick the level per transaction, not for the database, and I’ll say what I’m accepting in each case.

For ordinary writes I’d stay at Postgres’s default, read committed, and keep the arithmetic in SQL — `balance = balance - 5000` — so lost updates can’t happen at any level. What I’m accepting is non-repeatable reads and phantoms inside multi-statement transactions, which for short writes touching one entity never bite.

For anything shaped like check-then-act — book the seat if free, go off call if someone else is still on — read committed is not enough even with careful code, because of write skew: two transactions validate an invariant on their own views and write different rows, so nothing conflicts and the invariant still breaks. There I either lock what I check with `SELECT ... FOR UPDATE`, or run serialisable with a retry loop around the transaction, since Postgres enforces it optimistically and aborts losers with a serialization error. I’m accepting a retry rate, so those transactions must be side-effect-free — no emails from inside a transaction.

For long reports I’d use repeatable read: one snapshot for the whole run, so every number in the report agrees with every other. I’m accepting staleness of a few minutes and the bloat cost of holding a snapshot open, which is why heavy reports belong on a replica.

And if you asked me the difference between the read anomalies: a non-repeatable read is an existing row changing between two reads; a phantom is a new row appearing in a repeated search; a dirty read is seeing data that was never committed at all — which Postgres never allows at any level.”

9 Recall card

- **Three read anomalies, in strength order:** dirty read (uncommitted data), non-repeatable read (same row, two answers), phantom (new rows appear in a repeated search). Plus the write one: the lost update.
- **The levels forbid them cumulatively:** read committed kills dirty; repeatable read kills non-repeatable; serialisable kills phantoms — and write skew.
- **Postgres: default read committed; repeatable read = one snapshot per transaction** and stops phantoms too; serialisable = SSI, aborts losers — **retry loop required**.
- **Write skew:** two snapshots, an invariant across rows, writes to *different* rows — no conflict fires. Fix with `FOR UPDATE` on what you checked, or serialisable.

- **Choose per transaction:** default + SQL arithmetic for writes, repeatable read for reports, serialisable (retried) for check-then-act. Say what you are accepting.

DSA topic: At-most-K, and the exactly-K trick **System design topic:** Isolation levels and the anomalies they allow

Code these, in this order

Four problems. **Before writing a line of each, say two sentences out loud:** is this a counting question or a longest question, and is it at-most as asked or exactly-needing-the-subtraction? Those two sentences are today's entire skill.

Before each one, ask:

1. “How many” or “longest”? Therefore `total +=` or `max` ?
2. At-most directly, or exactly via `at_most(k) - at_most(k - 1)` ?
3. What is the guard — negative `k`, `goal = 0`, `k ≤ 1` ?
4. Does the condition use `len(count)` ? Then the `del` at zero, as always.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Subarray Product Less Than K	LeetCode 713 (Medium)	The counting line <code>total += right - left + 1</code> , no subtraction needed — and the <code>k ≤ 1</code> guard.
2	Binary Subarrays With Sum	LeetCode 930 (Medium)	The subtraction, the collapsed map, and whether <code>goal = 0</code> crashes you.
3	Count Number of Nice Subarrays	LeetCode 1248 (Medium)	Seeing “exactly k odd numbers” as the same problem after one observation.
4	Subarrays with K Different Integers	LeetCode 992 (Hard)	The full trick with a real map — a hard problem that is two tested calls and a minus sign.

On problem 1, earn the counting line

Before coding, answer: after the shrink, why is every subarray ending at `right` and starting at or after `left` valid? Say the one-sentence reason — shortening from the front can only remove things.

Then write it, and test `[10, 5, 2, 6]` with `k = 100` — expect 8. List the eight subarrays out loud from the walk, not from the code.

On problem 1, remove the guard on purpose

Delete the `k ≤ 1` guard and run `[2, 3]` with `k = 1`, then `[1, 1, 1]` with `k = 1`. The first returns `-1` — a count of minus one subarrays, silently. The second raises `IndexError: list index out of range`. Say why the shrink loop cannot ever be satisfied, and why one input crashes while the other lies.

On problem 2, chase the zero

Write it with the subtraction and the `goal < 0` guard. Confirm `([1,0,1,0,1], 2)` gives 4 and `([0,0,0,0,0], 0)` gives 15.

Then explain: when `goal = 0`, what does `at_most(-1)` get asked, and what does your guard return? Now count the fifteen subarrays of `[0,0,0,0,0]` by hand — `5 + 4 + 3 + 2 + 1` — and say why that is the sum of window lengths.

On problem 3, find the collapse

Say before coding: what would the map hold, and why does it collapse to one integer? Then write it with just an `odds` counter. Confirm `([1,1,2,1,1], 3)` gives 2 and `([2,4,6], 1)` gives 0.

On problem 4, respect the hard label

Write `at_most` as its own function and test it alone first: `[1,2,1,2,3]` with `k = 2` must give 12, with `k = 1` must give 5. Only then write the two-line `exactly` and confirm 7.

Then answer the question that makes this a hard problem: why can a single shrink-while-invalid window not count exactly-k directly? Two directions of wrongness, one repair tool — say it in your own words.

The counting-versus-longest drill

For each phrasing, say `total +=` or `max`, in under five seconds:

1. Longest subarray with at most `k` distinct values.
2. Number of subarrays with at most `k` distinct values.
3. Count the subarrays whose product is under `k`.
4. Shortest subarray with sum at least `k`.
5. How many subarrays contain exactly `k` odd numbers?
6. Longest run of 1s after flipping at most `k` zeros.

The anomaly-matching drill

For each scene, name the anomaly and the weakest standard level that stops it:

1. A report reads a balance that a transaction later rolls back.

2. A transaction reads a price twice and gets 100, then 120.
3. A count of matching rows grows between two identical searches in one transaction.
4. Two processes read stock = 1, both decide to sell, both write stock = 0.
5. Two doctors both go off call after each checking that two were on call.

Numbers 4 and 5 are the write-side ones — say why no read level in the table fixes number 5, and which two tools do.

The level-choosing drill

For each transaction, pick a level (or a lock) and name the anomaly you are accepting:

1. Transfer money between two accounts, arithmetic in SQL.
2. A monthly report that reads forty tables over twenty minutes.
3. Book a seat if it is still free.
4. Increment a view counter ten thousand times a second.
5. Claim a username if nobody has it.

Number 5 has a better answer than any isolation level — name the constraint.

The error-text drill

Two real Postgres errors. For each: which level produces it, what happened, and what must the application do?

```
ERROR:  could not serialize access due to concurrent update
```

```
ERROR:  could not serialize access due to read/write dependencies among transactions
HINT:   The transaction might succeed if retried.
```

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Count the subarrays with exactly k distinct integers.* Counting, so the sum-of-window-lengths line, and why it is exact. Then the subtraction, why the nesting makes it legal, the negative- k guard, and the two-passes-still- $O(n)$ cost.
2. *What isolation level would you use, and what anomaly are you accepting?* Refuse the global question — choose per transaction. Default plus SQL arithmetic, repeatable read for reports, serialisable in a retry loop for check-then-act. Name what each choice accepts.

3. Two transactions each checked the rule and the rule still broke. What happened? Write skew, told as the on-call doctors. Why repeatable read misses it — different rows written, no conflict — and the two fixes: `FOR UPDATE` on what you checked, or serialisable with retries.
-

Before you move on

- [] I say “counting or longest” before every window problem, and place `total +=` or `max` accordingly.
- [] I can prove the counting line: after the shrink, `right - left + 1` subarrays end here, no more and no fewer.
- [] I guard the inner call — negative `k`, `goal - 1`, `k ≤ 1` — and I know the crash and the silent `-1` that appear without it.
- [] I can recite the three read anomalies with a two-transaction story each.
- [] I can say the levels table from memory, plus the two Postgres deviations from it.
- [] I can tell write skew as a story and name both fixes.
- [] I answered all three questions above out loud.

Choosing between two pointers and a window, under pressure

DATA STRUCTURES AND ALGORITHMS

Choosing between two pointers and a window, under pressure

You can look at a new problem and pick the right pattern in under a minute.

SYSTEM DESIGN

Locking and deadlocks

You can describe how two transactions deadlock and how the database resolves it.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *You have thirty seconds. What pattern does this problem want?*
- *What is a deadlock, and how would you prevent one?*

Choosing between two pointers and a window, under pressure

1 What this is, and why they ask it

Days 27 to 34 gave you six tools: opposite ends, read and write, fast and slow, the fixed window, the variable window, and the counting window. Today teaches the skill that sits above all six — **routing**. Given a problem you have never seen, you ask a fixed sequence of short questions, and the answers land you on one tool before you have written a line.

No interviewer asks “please choose a pattern” out loud, but every interviewer scores it. The first two minutes of a coding round are a routing test wearing a problem’s clothes: candidates who route correctly spend twenty-five minutes on code they already know; candidates who route wrongly discover it at minute twenty, with nothing left to salvage. The questions in this lesson are the same ones [day 026](#) built for strings, extended to cover the pointer and window families — and they are meant to be drilled until they run in under a minute, out loud.

2 The story

The out-patient queue at the district hospital starts forming before the gates open, and by eight o’clock there are sixty people on the benches. Sister Philomena sits at the front desk, and she has been sitting there for fourteen years.

She asks every person the same three questions, in the same order, and she needs about fifteen seconds per person. Where is the trouble? How long has it been there? Is there fever with it? Then she writes a room number on a slip of their token and calls the next person. Chest and breathing go left to Room 2. Anything with fever goes straight back to Room 5. Old aches, longer than a month, go upstairs. Cuts and swellings go to the dressing room. Anything that fits nowhere goes to Room 1, the general physician, who can handle whatever walks in.

A trainee sat beside her in June, and the queue nearly died of it. The trainee listened to each person’s whole account — the uncle who had the same thing, what the first medicine was, what the neighbour said — and tried to work out what was actually wrong with them. Twelve minutes for the second person. The benches filled and the shouting started.

Philomena told her the thing she tells every trainee. You are not here to find out what is wrong. The doctor finds out what is wrong, in the room, with time and instruments. You are here to find out **which room**. Those are different jobs, and the second one only needs three answers.

And the order of the questions matters, she said. Fever first would be a waste — most people say no, and you have learnt almost nothing. Where-is-the-trouble first cuts the whole building in half.

The trainee learnt it inside a week. The queue moves again. Sixty people, three questions each, and by half past eight every one of them is sitting outside the right door — none of them treated yet, every one of them in the right place to be.

3 The idea in plain English

Philomena's desk is the first two minutes of the interview. The rooms are the six patterns. Her three fixed questions are the routing checklist below — and her deepest rule carries over whole: **routing is a different job from solving**. You are not looking for the answer; you are looking for the door.

The routing checklist

Ask these in order. Each answer eliminates rooms.

Question 1 — is the answer about a contiguous run? Look for *subarray*, *substring*, *consecutive*, *run of*. Contiguous — from [day 024](#) — sends you to the **window side**. Pairs, rearrangement in place, or cycles send you to the **pointer side**. This is where-is-the-trouble: one question, half the building gone.

Question 2 (window side) — fixed size or best size? “Every window of size k ”, “any k consecutive” → the **fixed window**, [day 031](#). “Longest”, “shortest”, “at most”, “how many” → keep going.

Question 3 (window side) — best, or how many? “Longest” or “shortest” → the **variable window**, [day 032](#), and immediately say which shape: minimise shrinks while valid and records inside; maximise shrinks while invalid and records after. “How many” or “count” → the **counting window** and the at-most trick, [day 034](#), and the line is

```
total += right - left + 1.
```

Question 4 (window side) — is the condition monotonic? The window's licence, from [day 032](#): growing must only push the condition one way. A sum over values that can be **negative** is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [day 037](#). Asking “can the values be negative?” out loud is routing done where the interviewer can hear it.

Question 5 (pointer side) — which of the three shapes? From [day 027](#): sorted input and a pair to find → opposite ends, [day 028](#). Keep some elements, discard others, in place → read and write, [day 029](#). A cycle, a middle, or “no extra space” on something linked → fast and slow, [day 030](#). And the escape: **unsorted, and the answer needs the original indices** → not a pointer problem at all — a hash map, from [day 021](#), because sorting would destroy the very thing you must return.

The tells, on one table

THE PROBLEM SAYS	THE ROOM	DAY
“longest substring such that...”	variable window, shape B	032
“smallest subarray with...”	variable window, shape A	032
“count the subarrays with...”	counting window, at-most trick	034
“exactly k”	two at-most passes, subtract	034
“maximum sum of any k consecutive”	fixed window	031
“sorted array, find a pair”	opposite ends	028
“remove / move / compact in place”	read and write	029
“cycle”, “middle”, “linked, O(1) space”	fast and slow	030
“return the indices”, unsorted	hash map, not pointers	021
“subsequence”	not this family at all	024
sums that can go negative	prefix sums, not a window	037

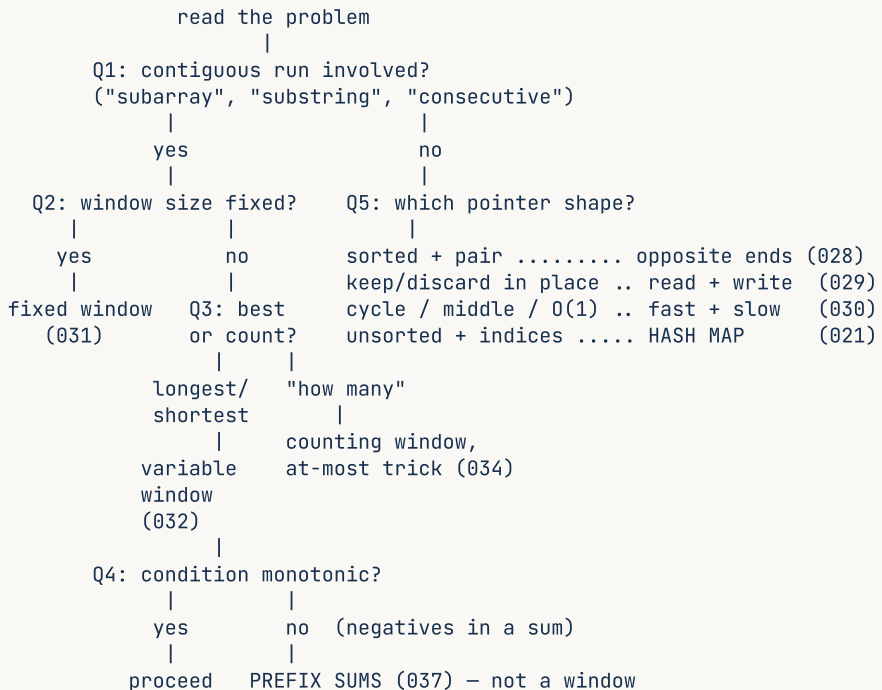
The last two rows are Philomena’s Room 1 — the cases that fit nowhere on this desk, and knowing they fit nowhere **is** the routing. Saying “this is a subsequence question, so none of my window tools apply” is a correct and impressive answer.

Routing is said out loud

The checklist is worthless run silently, because the interviewer cannot score what they cannot hear. The sentence to practise is one breath long: *“The word subarray means contiguous, it says longest, and the condition is a count of zeros which only grows as the window grows — so this is a variable window, maximise shape, shrink while invalid, record after.”* That sentence, before any code, is the first ninety seconds done properly.

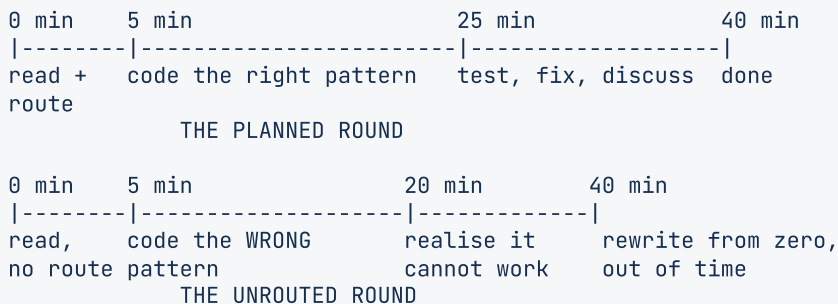
4 The picture

The desk, as a shape you can run in under a minute:



What to notice: two of the leaves are not rooms on this corridor at all. The checklist exists as much to route you *away* from these patterns as toward them — the hash-map leaf and the prefix-sums leaf are where wrong-pattern disasters are averted.

The cost of routing badly, drawn on the interview clock:



What to notice: the failure is not slower code — it is that the discovery arrives at minute twenty, when there is no time left to spend the lesson it teaches.

5 The code, built step by step

Three problems, worked as routing transcripts. Read them for the questions asked before the code, not for the code.

Problem one — Merge Sorted Array

“You are given two sorted arrays. The first has empty space at its end, exactly enough to hold the second. Merge the second into the first, in place, keeping sorted order.” — LeetCode 88.

Route it. No “subarray”, no condition on a run — Q1 says pointer side. Two sorted inputs walked position by position, and an in-place destination — this is the two-indices family, one per array, plus a write position, from [day 029](#).

Then the twist that makes it worth asking. Writing from the front overwrites values of `nums1` that have not been read yet. The free space is at the **back** — so walk everything backwards, largest first, and the write position can never catch up with the unread data.

```
write = m + n - 1
i = m - 1      # last real value in nums1
j = n - 1      # last value in nums2
```

Three positions, all starting at ends. `write` fills the array from the back; `i` and `j` point at the largest unread value of each source.

```
while j ≥ 0:
    if i ≥ 0 and nums1[i] > nums2[j]:
        nums1[write] = nums1[i]
        i -= 1
    else:
        nums1[write] = nums2[j]
        j -= 1
    write -= 1
```

The loop runs while `nums2` has anything left — if `nums2` is exhausted, the rest of `nums1` is already in place. The `i ≥ 0` guard matters: without it, `nums1[i]` with `i = -1` reads the *last* element — Python’s negative indexing, silently wrong, from [day 021](#). §7 shows the damage.

Problem two — Longest Subarray of 1's After Deleting One Element

“Delete exactly one element. Return the length of the longest run of 1s in what remains.”
— LeetCode 1493.

Route it. “Subarray” — Q1 says window. No fixed k — Q2 says variable. “Longest” — Q3 says shape B: shrink while invalid, record after. The condition: a window is workable if it contains **at most one zero** — deleting that zero leaves all 1s. Zeros only accumulate as the window grows, so Q4 passes. The window carries one integer, `zeros` — the collapsed map from [day 033](#).

```
left = zeros = best = 0
for right, x in enumerate(nums):
    if x == 0:
        zeros += 1
    while zeros > 1:
        if nums[left] == 0:
            zeros -= 1
        left += 1
    best = max(best, right - left)
```

One line differs from the standard shape: `right - left`, not `right - left + 1`, because exactly one element **must** be deleted — the answer is the window minus its sacrificial element. That handles the all-ones input for free: `[1, 1, 1]` gives 2, because you delete a 1 you would rather keep.

Problem three — Maximum Sum of Distinct Subarrays of Length k

“Return the maximum sum over all subarrays of length k whose elements are all distinct; 0 if none exists.” — LeetCode 2461.

Route it. “Subarrays of length k ” — Q1 window, Q2 **fixed**: this is [day 031](#)’s shape, not yesterday’s. The window carries two things: a running sum (slides by add-and-subtract) and a count map (to test distinctness). “All k distinct” is `len(count) == k`.


```

count: defaultdict[int, int] = defaultdict(int)
window_sum = 0
best = 0
for i, x in enumerate(nums):
    count[x] += 1
    window_sum += x
    if i ≥ k:
        out = nums[i - k]
        count[out] -= 1
        if count[out] == 0:
            del count[out]
        window_sum -= out
    if i ≥ k - 1 and len(count) == k:
        best = max(best, window_sum)

```

When `i` enters, `i - k` leaves — the fixed-window rhythm — and the `del` at zero is mandatory because `len(count)` is the test, [day 033](#)'s rule applying inside [day 031](#)'s shape. Two patterns composed, which is exactly why interviewers like this one.

The complete solutions

```
from collections import defaultdict

def merge(nums1: list[int], m: int, nums2: list[int], n: int) → None:
    """LeetCode 88. Overlapping destination: write from the free end, backwards."""
    write = m + n - 1
    i, j = m - 1, n - 1
    while j ≥ 0:
        if i ≥ 0 and nums1[i] > nums2[j]: # nums2 exhausted → rest already in place
            # i ≥ 0 guard: -1 would wrap silently
            nums1[write] = nums1[i]
            i -= 1
        else:
            nums1[write] = nums2[j]
            j -= 1
        write -= 1

def longest_subarray(nums: list[int]) → int:
    """LeetCode 1493. At most one zero in the window; one element must go."""
    left = zeros = best = 0
    for right, x in enumerate(nums):
        if x == 0:
            zeros += 1
        while zeros > 1:
            if nums[left] == 0:
                zeros -= 1
            left += 1
        best = max(best, right - left) # window minus the deleted element
    return best

def maximum_subarray_sum(nums: list[int], k: int) → int:
    """LeetCode 2461. Fixed window carrying a sum AND a map."""
    count: defaultdict[int, int] = defaultdict(int)
    window_sum = 0
    best = 0
    for i, x in enumerate(nums):
        count[x] += 1
        window_sum += x
        if i ≥ k:
            out = nums[i - k]
            count[out] -= 1
            if count[out] == 0:
                del count[out] # len(count) is the test: del is mandatory
            window_sum -= out
        if i ≥ k - 1 and len(count) == k:
            best = max(best, window_sum)
    return best

if __name__ == "__main__":
    a = [1, 2, 3, 0, 0, 0]
    merge(a, 3, [2, 5, 6], 3)
    print(a) # [1, 2, 2, 3, 5, 6]
    b = [0]
    merge(b, 0, [1], 1)
    print(b) # [1]

    print(longest_subarray([1, 1, 0, 1])) # 3
    print(longest_subarray([0, 1, 1, 1, 0, 1, 1, 0, 1])) # 5
    print(longest_subarray([1, 1, 1])) # 2 - must delete a 1
```

```
print(maximum_subarray_sum([1, 5, 4, 2, 9, 9, 9], 3)) # 15
print(maximum_subarray_sum([4, 4, 4], 3))           # 0 - no distinct window
```

6 What it costs

The three solutions, counted

Merge: every iteration writes one cell and retires one source element; there are $m + n$ cells. $O(m + n)$ time, $O(1)$ extra space — the point of merging backwards is precisely that no temporary array is needed.

Longest subarray of 1s: `right` advances n times, `left` at most n times, neither moves back — the $2n$ argument from [day 032](#). $O(n)$ time, $O(1)$ space.

Distinct k-sum: one pass, each step doing constant map work; the map holds at most k entries. $O(n)$ time, $O(k)$ space.

The cost that matters today: the wrong room

The routing checklist is five questions and costs under a minute. Compare what it buys:

```
45-minute round, well routed:
    ~2 min reading + routing, ~20 min code, ~15 min tests and follow-ups

45-minute round, routed wrong and discovered at minute 20:
    20 min sunk + ~5 min accepting it + 20 min to re-route, re-code, re-test
    = a rushed solution or none, in the same round, with the same knowledge
```

One minute of questions against twenty minutes of loss — a $20\times$ return, and it is the only complexity argument in this course measured in interview minutes rather than operations.

The number to have ready

Every tool in this family is $O(n)$ — the whole reason the family exists — with space $O(1)$ for pure pointers and $O(k)$ when the window carries a map. So the differentiator in the round is not asymptotics; it is choosing the right $O(n)$ tool in the first minute.

7 The traps

The near-miss: merging from the front

The instinct is to write the merged order starting at index 0:

```
c = [1, 2, 3, 0, 0, 0]
merge_front(c, 3, [2, 5, 6], 3)
print(c)
```

```
[1, 2, 2, 5, 6, 0]
```

The 3 is gone — overwritten by the incoming 2 before it was ever read, and every value after it is wrong. **When source and destination share memory, write into the free end.** Here the free space is at the back, so the merge runs backwards, largest first.

The near-miss: the missing `i >= 0` guard

Drop the guard from the backwards merge and run `nums1 = [2, 0]`, `m = 1`, `nums2 = [1]`:

```
if nums1[i] > nums2[j]:          # i is -1 here
```

```
[2, 2]
```

No crash. `nums1[-1]` reads the **last** element — Python’s negative indexing, the same silent wrap as [day 021’s](#) `ord` trap — and the 1 never gets placed. The expected answer is `[1, 2]`. An index that can go negative must be guarded *before* it is used, not after.

The real error: the unguarded pop

Route Backspace String Compare (LeetCode 844) to a stack — a fine first answer — and feed it a string that starts with deletes:

```
stack = []
for ch in "##a":
    if ch == "#":
        stack.pop()
    else:
        stack.append(ch)
```

```
Traceback (most recent call last):
  File "day35.py", line 5, in <module>
    stack.pop()
IndexError: pop from empty list
```

A backspace with nothing before it must do nothing: `if stack: stack.pop()`. And the follow-up — “*now do it in $O(1)$ space*” — is the routing question again: walk both strings **backwards** with a pointer each, counting skips, because a backspace’s effect runs right to left.

The misroute: “subsequence” through the window door

Longest increasing subsequence contains the word “longest”, and a window will happily produce an answer to it — the wrong answer. In `[5, 1, 2]` the longest increasing subsequence is `[1, 2]`, which is contiguous only by luck; in `[1, 9, 2, 3]` it is `[1, 2, 3]`, which no window can hold, because a window is a contiguous run and a subsequence may skip. **“Longest” routes you nowhere until you have answered Q1.** Subsequence questions belong to later tools ([day 024](#) drew the line; dynamic programming picks them up much later in the course).

The misroute: sorting away the answer

Two Sum returns **indices**, and the array is unsorted. Sort it to use opposite ends and the pointers find a correct *pair of values* — at positions in the sorted copy that mean nothing in the original:

```
nums = [3, 2, 4], target = 6
sorted → [2, 3, 4]; pointers find 2 + 4, positions 0 and 2
answer returned: (0, 2) → in the ORIGINAL array that is values 3 and 4 = 7
```

Correct answer: positions 1 and 2. If the answer is made of indices, sorting destroys it — the tell that routes to a hash map. (Sorting a *copy* and mapping back is possible, but it is $O(n \log n)$ and fiddly against the map’s clean $O(n)$ — [day 027](#)’s rule.)

The misroute: negatives through the window door

One line of yesterday repeated, because it is the most expensive misroute in the family: *shortest subarray with sum at least k* over values that may be negative is **not** a window problem — `[-3, 5]` with target 3 defeats the shrink logic, from [day 032](#). Ask “can values be negative?” before the window leaves your mouth. If yes: prefix sums, arriving [day 037](#).

8 In the interview

How it gets asked

- Never directly. The routing happens inside every problem from this family — the interviewer just says “Merge these two sorted arrays in place” or “Longest run of 1s if you may delete one element” and watches your first two minutes.
- The pressure version: you finish one problem and get “okay, quick one” — a second problem with fifteen minutes left, where routing speed is openly the thing being measured.

- The trap version: a problem whose surface words point to the wrong room — “longest” on a subsequence, “indices” on a sortable array — checking whether you route on words or on structure.

What to say out loud, in the first ninety seconds

This section is the script today. Run the checklist audibly:

1. **Q1, contiguity.** *“It says subarray, so contiguous — window side.”* Or: *“pairs in a sorted array, in place — pointer side.”*
2. **Q2/Q3, the shape.** *“No fixed k , and it says longest — variable window, maximise shape: shrink while invalid, record after.”* Or: *“count — so I accumulate $right - left + 1$, and ‘exactly’ means two at-most passes.”*
3. **Q4, the licence.** *“The condition is a count of zeros, which only grows as the window grows — monotonic, so the window is legal. If these could be negative-weighted I’d switch to prefix sums.”*
4. **The carry.** *“The window carries one integer — the zero count — so $O(1)$ space.”*
5. **Only now, code.** *“Brute force is every subarray at $O(n^2)$; the window gives $O(n)$. Writing it.”*

The follow-ups

“Why do you merge from the back?” Because the destination and one source are the same array, and the free space sits at the end. Merging forwards writes the smallest values into `nums1[0]`, `nums1[1]`, ... — cells that still hold unread values of `nums1`, so the write destroys data before the read reaches it; on `[1,2,3,0,0,0]` and `[2,5,6]` the 3 is overwritten and the result is wrong with no error. Walking backwards, largest first, writes only into cells that are either free space or already read — `write` starts at `m + n - 1` and `i` at `m - 1`, and `write` can never overtake `i`, because exactly `j + 1` of the cells between them are reserved for `nums2`’s remaining values. Same comparisons, same cost, and the overlap problem vanishes. The general rule: when source and destination overlap, write from the end where the free space is — it is the back here for the same reason the write pointer ran forwards in remove-duplicates, where the free space opens at the front.

“You picked a window in ten seconds. What would make you abandon it?” Three discoveries, and I try to force all three before coding. First, contiguity failing — if clarification reveals the answer may skip elements, it is a subsequence question and no window applies. Second, monotonicity failing — if the condition can improve *and* worsen as the window grows, the shrink logic has no licence; the concrete case is a sum over values that can be negative, where `[-3, 5]` style inputs break the invariant, and the correct move is prefix sums. Third, the answer’s shape failing — if the problem wants original

indices and my plan involved sorting, I have destroyed the answer to make the tool fit, and a hash map was the right room. All three are cheap questions before code and expensive discoveries after, which is exactly why the checklist runs first.

“Two problems, fifteen minutes. How do you spend them?” Ninety seconds routing both, out loud, before solving either — because routing is cheap and misrouting is the only real risk at this time scale. Then the one I am surest of first, coded straight from the pattern skeleton with the two or three standard tests — empty, single element, all-same — not a full test sweep. Then the second, and if I am short of time on it I state the route, the invariant and the cost, and write the skeleton honestly rather than rushing opaque code: “this is a maximise window over the zero count, record after the shrink, $O(n)$ ” earns most of the marks even half-typed. What I never do is start typing the first problem in the first thirty seconds — at this pace a wrong room is unrecoverable, so the routing minute matters more, not less.

A model answer

“Longest run of 1s if you may delete exactly one element” — the first ninety seconds, verbatim:

“Subarray of the original — so contiguous, a window question, not a subsequence one. It says longest, so a variable window, maximise shape: grow the right edge, shrink from the left only while the window is invalid, record after the shrink.

What makes a window workable? At most one zero inside it — I delete that zero and the rest are 1s. Zeros only accumulate as the window grows, so the condition is monotonic and the window is legal. The window carries a single integer, the zero count — no map needed, values are only 0 or 1.

One edge case before I write it: all ones. I must still delete something, so the answer is the window length minus one — which suggests recording `right - left` rather than the usual `right - left + 1`, and then the all-ones case needs no special branch at all.

Brute force is every subarray at $O(n^2)$; the window is $O(n)$, both pointers only move forward. Writing it now — eight lines.”

Then the eight lines from §5, and the tests `[1,1,0,1] → 3`, `[1,1,1] → 2`, `[0] → 0` — spoken before they are run.

9 Recall card

- **Route before solving, out loud.** Q1 contiguous? Q2 fixed k? Q3 best or count? Q4 monotonic? Q5 which pointer shape? Under a minute, every problem.
- **Words route wrong; structure routes right.** “Longest” on a subsequence is no window; “indices” plus sorting destroys the answer — hash map.
- **Two leaves lead out of the family:** negatives in a sum → prefix sums ([day 037](#)); “subsequence” → not this corridor at all.
- **Overlapping source and destination** → **write from the free end** — merge sorted arrays runs backwards, and `i ≥ 0` guards the wrap to `nums1[-1]`.
- **Everything here is $O(n)$.** The differentiator is the first minute, not the asymptotics — one minute of routing against twenty minutes of misroute.

1 What this is, and why they ask it

A **lock** is the database's claim on a piece of data: while a transaction holds it, conflicting access by others must wait. Locks are how yesterday's isolation promises are physically kept — and they create one famous failure of their own. A **deadlock** is two transactions each holding a lock the other needs, waiting on each other forever, until the database notices the circle and kills one of them.

This question is an interview staple because it is a story with a mechanism inside it, and both halves get tested. The story half — two transactions crossing — checks that you can narrate concurrency at all. The mechanism half — how the database detects the cycle, which victim it picks, what your application must do next — separates people who have read a definition from people who have seen `deadlock detected` in a production log. It also carries the single most useful prevention rule in all of backend engineering, and interviewers listen for it by name: **lock things in a consistent order**.

2 The story

Sunday morning is biryani morning in Lata's house, which means her son Arun is also in the kitchen, making the raita and the salad, and the kitchen is small.

There is one good knife and one big chopping board. Around eleven o'clock, Lata picks up the knife — she needs to cut the marinated chicken, and after that she will need the board to lay the pieces out. At the same moment, on the other side of the counter, Arun pulls the chopping board toward himself for the cucumbers, and reaches for the knife.

The knife is in Lata's hand. So Arun waits, board in front of him, hand out.

Lata finishes trimming, turns to lay the pieces out, and the board is gone. So she waits too, knife in hand, looking at her son.

Neither of them is being unreasonable. Each is holding one thing they genuinely need and waiting for one thing they genuinely need, and each is waiting for the other. And there they stand. The chicken is not moving, the cucumbers are not moving, and the pressure cooker is getting ahead of both of them. It is a small, silly, complete standstill — nobody can take a single step until the other moves first, and nobody can move first.

Lata's mother settled it from the doorway, the way she settles most things, with a sentence: somebody put something down. Arun, being the junior cook, slid the board across. He started his cucumbers again from the beginning two minutes later, when both things were free.

But it is the grandmother's second sentence that fixed the kitchen for good. In her kitchen, she said, the rule was that you take the board first, then the knife, always in that order, everyone. If both of them had followed that rule, one of them would have got the board and gone on to finish, and the other would have simply waited a minute at the start — a queue, not a standstill. The standstill only happens because the two of them picked the same two things up in opposite orders.

3 The idea in plain English

The knife and the board are two rows in a database. Lata and Arun are two **transactions**. Holding a thing while waiting for the other thing is the whole story of today.

What a lock is

When a transaction updates a row, the database gives it an **exclusive lock** on that row: nobody else may change the row until the transaction commits or rolls back. The lock is not politeness — it is what makes [day 033's](#) guarantees physical. Anyone else wanting the same row simply **waits** in a queue.

Waiting is normal and healthy. Arun waiting for the knife while Lata uses it is not a problem; it is a queue, and queues drain. Yesterday's `SELECT ... FOR UPDATE` was exactly this: deliberately taking a lock so that others wait.

Two flavours matter for interviews. A **shared lock** is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other. An **exclusive lock** is for writing: one holder, and it excludes readers-with-locks and writers both. (Plain readers in Postgres do not take row locks at all — MVCC gives them a version to read, which is why readers and writers do not block each other.)

What a deadlock is

A deadlock is waiting arranged in a **circle**. Lata holds the knife and waits for the board; Arun holds the board and waits for the knife. Transaction 1 holds row A's lock and wants row B's; transaction 2 holds row B's and wants row A's. Every member of the circle is waiting for another member, so no member can ever proceed. This is not slowness — no amount of patience fixes it. The circle is permanent until someone breaks it.

The distinction to say out loud: **blocking is a queue; deadlock is a cycle**. A queue drains on its own. A cycle never does.

How it gets resolved: the grandmother in the doorway

The database is the grandmother. It watches who waits for whom, and when it finds a circle, it picks a **victim** — one transaction in the cycle — and kills it with an error. The victim's work rolls back (atomicity doing its job), its locks are released, the circle is broken, and the survivor proceeds. Arun slid the board across and started his cucumbers again — the victim **retries from the beginning**.

The deadlock is therefore not a catastrophe. It is a resolved collision, costing one roll-back and one retry. A *high rate* of them is the real problem, and prevention is about the rate.

How it gets prevented: the second sentence

Lock objects in a consistent order, everywhere. Board first, then knife, for everyone in the kitchen. If every transaction that touches accounts 7 and 42 locks the lower id first, the circle cannot form: whoever gets account 7 proceeds; the other waits at the *start*, holding nothing. A wait instead of a standstill. This one rule, applied at code-review time, prevents the large majority of real deadlocks — and it costs nothing at runtime.

4 The picture

The deadlock, on two timelines:

T1 (transfer 7 → 42)	T2 (transfer 42 → 7)
BEGIN	BEGIN
UPDATE accounts SET ...	UPDATE accounts SET ...
WHERE id = 7;	WHERE id = 42;
→ holds lock on row 7	→ holds lock on row 42
UPDATE accounts SET ...	UPDATE accounts SET ...
WHERE id = 42;	WHERE id = 7;
→ WAITS for T2's lock	→ WAITS for T1's lock
..... both wait forever: the cycle is closed	
after deadlock_timeout (1 s), the detector runs:	
T1: proceeds	T2: ERROR: deadlock detected (rolled back, must retry)

What to notice: each transaction, read alone, is completely sensible. The deadlock exists only in the *pair*, and only because they locked the same two rows in opposite orders.

The waits-for graph the detector actually builds:

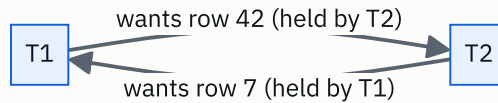


FIGURE 35.1

What to notice: deadlock detection is cycle detection in this graph — the same “does this loop?” question as [day 030](#), asked of transactions instead of list nodes. A cycle found means one node in it gets aborted.

The fix, as timelines — both transactions lock the **lower id first**:

T1 (7 then 42) BEGIN lock row 7 ← wins lock row 42 ... work, COMMIT locks released ----->	T2 (7 then 42 – same order) BEGIN lock row 7 → WAITS, holding nothing acquires row 7, then 42, proceeds
---	--

What to notice: T2 still waits — ordering does not remove contention — but it waits at the start, holding nothing, so no circle can close. A queue, not a standstill.

5 How it actually works

Who takes locks, and when

`UPDATE`, `DELETE` and `INSERT` take exclusive row locks as they touch rows. `SELECT ... FOR UPDATE` takes the same lock without changing anything — yesterday’s check-then-act tool. Plain `SELECT` in Postgres takes no row lock at all: MVCC hands readers a committed version, which is why “readers block writers” is false there. Locks live exactly as long as the transaction — released at `COMMIT` or `ROLLBACK`, never earlier, which is the deep reason [day 033](#) said to keep transactions short.

There are also table-level locks (schema changes take the big ones — an unguarded `ALTER TABLE` queues behind and ahead of everything), but row locks are where interview deadlocks live.

Detection in Postgres

Waiting for a lock is passive — the transaction just sleeps in the queue. When a wait exceeds `deadlock_timeout` (default one second), Postgres builds the waits-for graph and searches for a cycle. If it finds one, the transaction that *triggered the check* is usually the victim:

```
ERROR: deadlock detected
DETAIL: Process 9182 waits for ShareLock on transaction 5501; blocked by process 9075.
        Process 9075 waits for ShareLock on transaction 5502; blocked by process 9182.
HINT: See server log for query details.
```

The victim is rolled back entirely. The application must catch this and **retry the transaction from BEGIN** — the same obligation serialisable imposed yesterday, arriving one level down. MySQL’s InnoDB checks eagerly at each wait and typically chooses the transaction that has done less work as the victim; it also takes **gap locks** at its default repeatable read — locks on the spaces *between* index entries — which produce deadlocks that surprise people who learnt on Postgres.

The tools around waiting

Waiting has dials, and naming them is senior-signal:

```
SET lock_timeout = '2s';           -- give up waiting after 2s, with an error
SELECT ... FOR UPDATE NOWAIT;      -- error immediately if the row is locked
SELECT ... FOR UPDATE SKIP LOCKED; -- skip locked rows, take what is free
```

SKIP LOCKED deserves its own sentence, because it turns a table into a **job queue**. Ten workers each run:

```
SELECT * FROM jobs WHERE status = 'pending'
ORDER BY id LIMIT 1 FOR UPDATE SKIP LOCKED;
```

Each worker locks a *different* pending row — nobody waits, nobody collides, and a crashed worker’s lock vanishes with its transaction, so its job is automatically picked up again. This is the standard “background jobs on Postgres” pattern, used by Sidekiq-style systems and worth producing unprompted when queues come up.

Optimistic control: the no-lock alternative

Pessimistic locking assumes collisions are likely and pays for certainty with waiting.

Optimistic locking assumes they are rare: read the row with its **version** column, do the work unlocked, then

```
UPDATE items SET stock = 4, version = 8
WHERE id = 91 AND version = 7;
```

Zero rows affected means someone got there first — reload and retry. No locks held during the think time, so no deadlocks at all; the cost moves to retries, which is fine when contention is rare and miserable when it is hot. This is the same optimistic-versus-pessimistic trade yesterday’s serialisable-versus-**FOR UPDATE** choice made, one storey lower.

6 The numbers

The detector's price

```
deadlock_timeout = 1s (Postgres default)

a genuine deadlock therefore costs:
  ~1 s of two transactions frozen
+ one rollback and one full retry
≈ 2+ s of user-visible latency on the victim's request
```

One deadlock is an incident of two seconds. The arithmetic to fear is the rate: at 50 deadlocks a minute, you have a design problem, not bad luck.

How likely is the collision?

Two transfer types crossing between the same two accounts, no ordering rule:

```
each transfer holds its first lock for ~10 ms before wanting the second
transfers between this pair arrive at 20/second in each direction

probability a given transfer meets an opposite one mid-flight:
  ≈ arrival rate × window = 20/s × 0.010 s = 0.2 → 20% of transfers
at 20/s each way, that is ~4 deadlocks per second on ONE hot pair
```

Two lines of `ORDER BY id` — locking the lower id first — takes that to zero. That before/after is the most persuasive numbers story in today's lesson.

What one held lock does to a queue

The hot-row arithmetic, because interviewers reuse it constantly:

```
one row, exclusive lock held 100 ms per transaction
100 transactions want it in the same second

throughput through that row: 1 / 0.1 s = 10/s → 90 queue up, and
the last waiter sees ~10 s of latency (100 × 100 ms, serialised)
```

A single hot row is a serial bottleneck no index can fix. The escapes: make the hold shorter, make the operation database-side arithmetic (`SET count = count + 1`, held for microseconds), or split the row (e.g. 16 counter rows summed on read).

The queue pattern's capacity

```
jobs table with SKIP LOCKED, 10 workers, each job ~200 ms:
  10 workers × 5 jobs/s = 50 jobs/s, zero lock waits
without SKIP LOCKED (plain FOR UPDATE on the same top row):
  all 10 workers queue on the SAME first row → effectively 1 worker
```

Same table, same workers — a $10\times$ difference from three words of SQL.

7 The trade-offs

Pessimistic against optimistic

Locks buy certainty and pay in waiting and deadlock risk; version columns buy freedom from waiting and pay in retries and in every writer having to cooperate. Choose by contention: a seat at a concert on-sale is pessimistic territory (collisions are the normal case); a user editing their own profile is optimistic territory (collisions are freak events). **I would not use optimistic locking on a hot row** — the retry storm is worse than the queue.

Fail fast against wait

`NOWAIT` and short `lock_timeout`s turn contention into visible, immediate errors the caller can handle — right for interactive requests, where a user staring at a spinner for ten seconds is worse than “try again”. Plain waiting is right for background work, where throughput matters and nobody is watching. The anti-pattern is the default everywhere: infinite patience on an interactive path is how one held lock becomes a site-wide hang.

Ordering discipline against flexibility

The lock-in-order rule costs nothing at runtime and everything in vigilance — it lives in code review, not in the database, and one new code path that touches rows in reverse order reintroduces the standstill. Teams encode it structurally where they can: always `ORDER BY id` when locking multiple rows (`SELECT ... WHERE id IN (7, 42) ORDER BY id FOR UPDATE`), so the discipline is in the query, not the reviewer’s memory.

The sentence that separates candidates

I would treat a deadlock error as a retry, and a deadlock *rate* as a design smell. The database already resolves individual deadlocks — detection, victim, roll-back — so my application’s job is a retry loop with a couple of attempts and jitter. But if the rate is nonzero and growing, something structural is wrong: two code paths locking in opposite orders, or transactions holding locks across external calls — and the fix is ordering and shorter transactions, not a cleverer retry.

8 In the interview

How it gets asked

- “What is a deadlock, and how would you prevent one?” — the hub question: the story, the cycle, the ordering rule.
- “Two requests are stuck and one eventually fails with ‘deadlock detected’. Walk me through what happened.” — the production-flavoured version; they want the two timelines.
- “What’s the difference between blocking and a deadlock?” — queue against cycle, drains against never.
- “How would you build a job queue on Postgres?” — `FOR UPDATE SKIP LOCKED`, workers, crashed worker’s lock evaporating.
- “When would you use optimistic locking instead?” — contention rare, no held locks, version column, retry on zero rows.

What to say out loud, in the first ninety seconds

1. **Define with the circle.** “A deadlock is two transactions each holding a lock the other needs — T1 has row A and wants B, T2 has B and wants A. Neither can ever proceed; it’s a cycle, not a queue.”
2. **Distinguish it from blocking.** “Ordinary lock waiting is fine — a queue drains. A deadlock never drains without intervention.”
3. **Say how the database resolves it.** “Postgres waits a second, builds the waits-for graph, finds the cycle, and aborts one transaction with ‘deadlock detected’. The application retries the victim from BEGIN.”
4. **Give the prevention rule by name.** “Prevention is mostly one rule: lock rows in a consistent order — lowest id first, everywhere. Opposite orders are the only way the cycle closes.”
5. **Add the operational habits.** “Plus short transactions — no external calls while holding locks — and `lock_timeout` on interactive paths so contention fails fast instead of hanging.”

The follow-ups

“Why exactly does locking in a consistent order prevent deadlocks?” Because a deadlock needs a cycle in the waits-for graph, and a global order makes cycles impossible. Give every lockable row a rank — its primary key — and let every transaction acquire locks only in increasing rank. Now suppose a cycle existed: T1 waits for something T2 holds, T2 for something T1 holds. T1 holds a lower-ranked row than the one it wants; T2 likewise. Follow the circle and each step must point at a higher rank than something the waiter already holds — but the circle comes back to where it started, which would need a rank higher than itself. Contradiction, so no cycle. In practice it means one habit: when a transaction touches several known rows, sort before locking —

`WHERE id IN (7, 42) ORDER BY id FOR UPDATE` — and when two tables are involved, agree the table order too. Contention remains; the second transaction still waits — but it waits holding nothing, which is a queue, and queues drain.

“The deadlock error is rare but nonzero in your logs. What do you actually ship?” Two things, in this order. First, a retry wrapper on write transactions: catch the deadlock error (and its serialisation cousins from yesterday), retry the whole transaction up to two or three times with a small randomised backoff — which requires those transactions to be side-effect-free, no emails or HTTP calls inside, so the retry is safe. That makes the user never see the error. Second, an investigation, because the rate is a smell: pull the two queries from the server log — Postgres logs both parties — and almost always it is two code paths locking the same pair of rows in opposite orders, or one transaction holding locks across something slow. Fix the order or shorten the transaction. What I would not do is raise `deadlock_timeout` to make errors rarer — that just makes each real deadlock freeze longer before resolution.

“How do multiple workers take jobs from a table without stepping on each other?” `SELECT ... FROM jobs WHERE status = 'pending' ORDER BY id LIMIT 1 FOR UPDATE SKIP LOCKED`, inside the transaction that processes the job. `FOR UPDATE` claims the row; `SKIP LOCKED` is the crucial part — a worker that finds the first pending row already claimed does not queue behind it, it takes the next free one. Ten workers therefore hold ten different rows with zero waiting. Mark the job done and commit; if the worker crashes mid-job, its transaction dies, the lock evaporates, and the job is pending again for someone else — crash recovery with no extra machinery. Without `SKIP LOCKED` all ten workers queue on the same top row and you have one effective worker. It is the standard Postgres job-queue pattern, and it is why “do we need RabbitMQ?” is often answered “not yet”.

A model answer

“A deadlock is two transactions holding one lock each and waiting for each other’s. The concrete version: T1 transfers money from account 7 to 42, T2 from 42 to 7, at the same moment. T1 updates row 7 and holds its lock; T2 updates row 42 and holds its lock. Then T1 asks for row 42 and waits, and T2 asks for row 7 and waits. Each is waiting for the other — a cycle. That’s different from ordinary blocking: a queue drains on its own, a cycle never does.

The database resolves it, not the application. Postgres lets a lock wait run for `deadlock_timeout` — a second by default — then builds the graph of who waits for whom and looks for a cycle. It aborts one member as the victim: that transaction gets ‘ERROR: deadlock detected’, rolls back completely, and its locks release, so the survivor proceeds. My application’s duty is a retry loop — catch that error and rerun the transaction from BEGIN, which also means keeping side effects out of transactions so a rerun is safe.

Prevention is mostly one rule: acquire locks in a consistent global order. Both transfers should lock the lower account id first — then whichever wins proceeds, and the other waits at the start holding nothing. A wait, not a deadlock. When I lock several rows at once I put the order in the query itself: `WHERE id IN (...) ORDER BY id FOR UPDATE`.

And around that rule, the supporting habits: transactions short, no external calls while holding locks, `lock_timeout` on user-facing paths so contention fails fast, and — if the workload is a job queue — `FOR UPDATE SKIP LOCKED` so workers claim different rows instead of queueing on the same one.”

9 Recall card

- **Blocking is a queue; deadlock is a cycle.** T1 holds A wants B, T2 holds B wants A — nobody can ever move. Queues drain; cycles need a victim.
- **The database resolves it:** after `deadlock_timeout` (1 s), find the cycle in the waits-for graph, abort one transaction — `ERROR: deadlock detected` — and the app retries from BEGIN.
- **Prevention rule number one: lock in a consistent order** — lowest id first, everywhere; `WHERE id IN (...) ORDER BY id FOR UPDATE` puts the discipline in the query.
- **FOR UPDATE SKIP LOCKED turns a table into a job queue** — workers claim different rows, a crashed worker’s job frees itself.

- **Optimistic locking (version column) has no deadlocks** — right when contention is rare; a hot row wants pessimistic locks or database-side arithmetic instead.

PRACTICE

Practice — Choosing between two pointers and a window, under pressure

DSA topic: Choosing between two pointers and a window, under pressure **System design topic:** Locking and deadlocks

Code these, in this order

Four problems chosen so that the route is the work. **For each one, run the checklist out loud and name the room before writing anything** — Q1 contiguous? Q2 fixed k? Q3 best or count? Q4 monotonic? Q5 which pointer shape? Time the routing: under a minute each.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Merge Sorted Array	LeetCode 88 (Easy)	Routing to pointers, then the real lesson: overlapping destination → write from the back.
2	Backspace String Compare	LeetCode 844 (Easy)	First route (stack) versus the follow-up route (two pointers, backwards, O(1) space).
3	Longest Subarray of 1's After Deleting One Element	LeetCode 1493 (Medium)	Window, maximise shape, collapsed map — and the <code>right - left</code> twist for the forced deletion.
4	Maximum Sum of Distinct Subarrays of Length K	LeetCode 2461 (Medium)	Hearing “length k” over “distinct”: fixed window carrying a sum and a map together.

On problem 1, break it twice

Write it correctly, from the back. Then write the front-first version on `([1, 2, 3, 0, 0, 0], 3, [2, 5, 6], 3)` and watch the 3 disappear: `[1, 2, 2, 5, 6, 0]`. Say the rule — overlapping source and destination, write from the free end.

Then drop the `i ≥ 0` guard and run `([2, 0], 1, [1], 1)`. You get `[2, 2]`, no crash. Say why `nums1[-1]` did not raise, and which earlier day warned you about it.

On problem 2, do both routes

Stack version first — and feed it `"##a"` before you guard the pop, so you have seen `IndexError: pop from empty list` once on purpose. Then the O(1)-space route: why must the pointer version walk backwards? Say it in one sentence before writing it — a backspace's effect runs right to left, so certainty only exists from the back.

On problem 3, route it aloud first

The full routing sentence before code: contiguous, longest, condition “at most one zero” which is monotonic, carries one integer. Then the twist: why `right - left` and not `right - left + 1`, and which input that one character handles for free. Confirm `[1,1,1]` gives 2 and `[1,1,0,1]` gives 3.

On problem 4, name both parents

This is day 031’s shape carrying day 033’s map. Say which rule from each day applies — enter/leave at `i` and `i - k`; `del` at zero because `len(count)` is the test. Confirm `([1,5,4,2,9,9,9], 3)` gives 15 and `([4,4,4], 3)` gives 0.

The routing drill

Name the room for each, in under ten seconds, out loud — pattern and day:

1. Longest substring with at most two distinct characters.
2. Given a sorted array, count pairs summing below a target.
3. Remove every occurrence of a value, in place, return the new length.
4. Does this linked list contain a cycle?
5. Count subarrays with exactly three odd numbers.
6. Maximum average of any subarray of length k.
7. Longest increasing subsequence.
8. Two Sum, unsorted, return the indices.
9. Shortest subarray with sum at least k — values may be negative.
10. Reverse the vowels of a string in place.
11. How many subarrays have a product below 100?
12. Find the middle of a linked list in one pass.

Numbers 7, 8 and 9 are the exits — say where each one routes away to, and why.

The abandon drill

For each discovery, say what you do next:

1. Mid-problem, the interviewer confirms elements may be skipped.
2. Mid-problem, you learn the values can be negative and your condition is a sum.
3. Your plan was to sort, and the expected output is a pair of original indices.

The deadlock-telling drill

Tell the two-transaction transfer story from memory — T1 does 7→42, T2 does 42→7 — with the two lock acquisitions, the two waits, and the moment the cycle closes. Under a minute, no notes. Then the one-line distinction: blocking is a queue, deadlock is a cycle.

The prevention drill

Answer each in one or two sentences, out loud:

1. Why does locking in a consistent order make the cycle impossible?
2. What does the application do when it receives `deadlock detected`?
3. Why must retried transactions be free of side effects?
4. What does `deadlock_timeout` control, and why is raising it the wrong fix?
5. Ten workers, one jobs table — which three words of SQL stop them queueing on one row?
6. When is optimistic locking the better tool, and what column does it need?

The arithmetic drill

From memory, in under two minutes:

- One row, lock held 100 ms, 100 waiters in one second — throughput through that row, and the last waiter's latency.
 - Transfers between one account pair at 20/s each way, 10 ms first-lock hold — roughly what fraction collide, and what the ordering rule takes that to.
 - A genuine deadlock with `deadlock_timeout` at one second — the victim's user-visible cost, all in.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *You have thirty seconds. What pattern does this problem want: “longest run of 1s if you may flip at most two 0s”?* Run the checklist audibly — contiguous, variable, maximise, monotonic, carries one integer — and finish with the shape sentence: shrink while invalid, record after.
 2. *What is a deadlock, and how would you prevent one?* The two-transfer story, queue against cycle, detection and the victim, then the ordering rule by name — plus short transactions and `lock_timeout` as the supporting habits.
 3. *How would you build a job queue on Postgres?* `FOR UPDATE SKIP LOCKED`, why workers stop colliding, and what happens to a crashed worker's job — then the honest sentence about when this pattern runs out and a real broker earns its place.
-

Before you move on

- [] I run the five routing questions out loud before writing any code, in under a minute.
- [] I can name the three exits — subsequence, indices-plus-sorting, negatives-in-a-sum — and where each routes to.
- [] I write from the free end whenever source and destination overlap, and I guard indices that can go negative.
- [] I can tell the deadlock story with both timelines and say why it is a cycle, not a queue.
- [] I can state the ordering rule and put it inside a query with `ORDER BY id FOR UPDATE`.
- [] I know what the application does on `deadlock detected`, and why retries must be side-effect-free.
- [] I answered all three questions above out loud.

Two pointers revision and mock round

DATA STRUCTURES AND ALGORITHMS

Two pointers revision and mock round

You can solve two unseen pointer or window problems cold.

SYSTEM DESIGN

NoSQL: what it actually means

You can stop saying NoSQL is faster and start naming the real trade-off.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *What is the difference between SQL and NoSQL?*

1 What this is, and why they ask it

Days 27 to 35 built the pointer-and-window toolkit: the two-pointers idea, opposite ends, read and write, fast and slow, the fixed window, the variable window, the window with a map, the counting window, and yesterday's routing checklist that chooses among them. Today is the third mock round, and it has a different job from [day 018](#)'s process and [day 026](#)'s recognition.

Today is about **execution under match conditions**. You know the six beats. You know the routing questions. The remaining question is whether they still run when the problem is unseen, the clock is on, and someone is watching — because knowledge that evaporates under pressure scores the same as knowledge you never had. This phase is also the single most-asked family at product companies: a majority of screening-round mediums are pointer or window problems, so this is the mock most worth doing honestly.

The two problems in §5 are deliberately ones no lesson has solved. Route them yourself before reading the transcripts.

2 The story

Tuesday and Thursday evenings, the community hall becomes a badminton court, and coach Sebastian runs the junior group from six to eight.

For the first hour it is drills, and in drills his best student is unbeatable. Drop shots into a hoop, twenty out of twenty. Serves onto a coin. Footwork ladders faster than anyone. Her name is Divya, she is thirteen, and until October she had never won a match that mattered.

Sebastian watched her lose one in the district round and saw exactly what he expected to see. Every stroke she owned in the drills was still there — but between strokes, where a match actually happens, everything went. She chose the wrong shot when the rally got long. She rushed her serve after losing a point. At nine points down she started playing shots she had never practised at all, as if the drills belonged to a different girl.

So he changed Thursdays. The second hour is now nothing but practice matches, with scoring, with an umpire, with her parents allowed to sit and watch — because an audience changes the body, and she had never once practised being watched. And he gave her a rule for the gap between points, the only part of a match nobody drills. Before

every serve she must say her plan in her head in one short sentence — serve wide, then attack the backhand. Not a paragraph. One sentence, every point, no exceptions, even at match point, *especially* at match point.

The first three Thursdays were ugly. She lost to boys she beat in every drill, and cried once in the car. The fourth Thursday she stopped rushing. By December the sentence had become automatic — plan, serve, play — and in January she won the district under-four-
teens, beating the same girl from October in straight games.

The strokes did not improve between October and January. The strokes were never the problem.

3 The idea in plain English

Divya’s drills are days 27 to 35. Today is Thursday’s second hour — full matches, scored, watched. And her one-sentence rule between points is exactly yesterday’s routing sentence said before every problem: the piece of match play that must be drilled until pressure cannot remove it.

The phase, on one table

Nine days compressed. Each row is a tool, its invariant — the sentence that makes it correct — and its tell.

TOOL	THE INVARIANT TO SAY	THE TELL	DAY
Opposite ends	any answer lies between <code>left</code> and <code>right</code> ; each move discards only proven-impossible pairs	sorted + pairs	028
Read and write	<code>items[0:write]</code> is finished and correct; <code>write ≤ read</code> always	compact in place	029
Fast and slow	the gap shrinks by exactly one per step inside a loop	cycle, middle, O(1)	030
Fixed window	when <code>i</code> enters, <code>i - k</code> leaves; add and subtract, never recompute	“of length k”	031
Variable window	neither edge ever moves backwards; ~2n moves	“longest”, “shortest”	032
Window + map	the map means <i>what is in the window now</i> — hence <code>del</code> at zero	condition on counts	033
Counting window	after the shrink, <code>right - left + 1</code> valid subarrays end here	“how many”	034

Under pressure you will not re-derive these. You will either say the invariant from memory or write code you cannot defend. That is what today’s two matches test.

What pressure actually removes, and the counter-habit

Watching candidates fail mock rounds shows the same three losses, in order:

The routing minute goes first. The heart speeds up, the hands want to type, and the checklist gets skipped — the exact skill [day 035](#) built. The counter is Divya's rule: one routing sentence, out loud, before any code, no exceptions. It costs thirty seconds and it is the thing being scored.

Edge cases go second. The empty input, the single element, the all-same input — known cold on a calm day, forgotten at minute thirty. The counter is a fixed list run at a fixed time: after the code compiles in your head, before you declare done, the same five inputs from [day 026](#), every time, as a ritual rather than a decision.

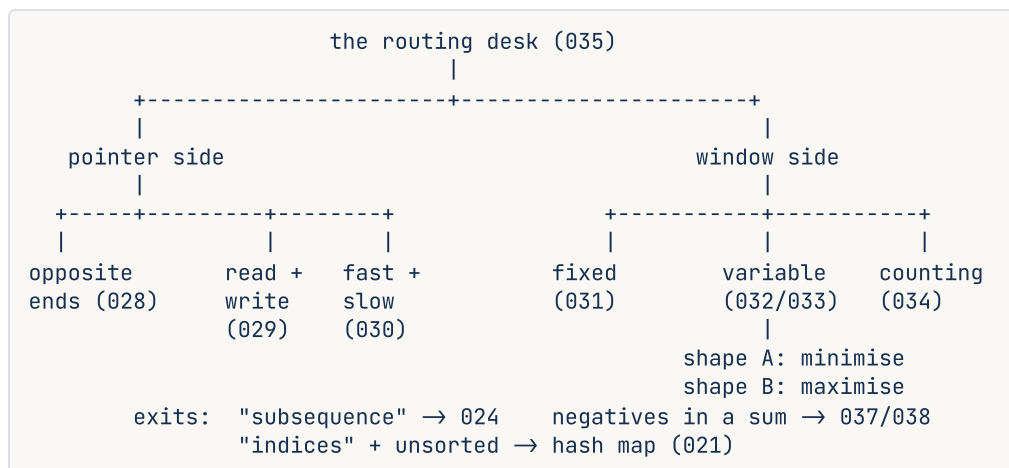
Honesty goes third. Stuck candidates go quiet, and quiet is the one unrecoverable state — the interviewer cannot help, cannot score the thinking, cannot even tell it is happening. The counter: narrate the stuck. *"I have the window but my condition isn't monotonic — let me check what breaks it."* Saying you are stuck, with specifics, reads as method; silence reads as absence.

How to run today's mock

Same protocol as [day 026](#). Set a timer for thirty-five minutes per problem. Stand up, talk continuously, no notes, no lesson files open. Route out loud, name the invariant, write the code in one pass, then run the five test inputs by voice. Only then read the transcript in §5 and score yourself against it — not on whether the code matches, but on whether the *sentences before the code match*.

4 The picture

The whole phase as one corridor, with yesterday's desk at the front:



What to notice: seven rooms, three exits. A mock is passed as much by walking *out* of the corridor at the right moment as by picking the right room in it.

The match ritual, as the clock you run today:

```

min 0-2    read twice; ask the contract questions (alphabet? negatives? empty?)
min 2-3    ROUTE OUT LOUD: one sentence, pattern + shape + invariant
min 3-5    example by hand; state brute force and its cost
min 5-20   code, narrating; invariant said before each loop
min 20-25  the five test inputs, by voice, including the empty one
min 25-35  follow-ups, cost argument, the honest "what I'd check with more time"
  
```

What to notice: the code gets fifteen of thirty-five minutes. Candidates who type at minute one spend those fifteen minutes twice.

5 The code, built step by step

Two problems, worked as transcripts. Route each yourself before reading on.

Match one — Container With Most Water

“You are given heights of vertical lines at positions 0 to $n - 1$. Pick two lines; together with the x -axis they hold water. Return the largest area you can trap. `[1, 8, 6, 2, 5, 4, 8, 3, 7]` → 49.” — LeetCode 11.

Beat 1, clarify. *“Area is distance between the lines times the shorter height? And the lines between don’t matter?”* Yes and yes. *“Can heights be zero?”* They can — area just comes out zero.

Beat 2, route out loud. “The answer is about a pair of positions, not a contiguous run — so pointer side, not a window. It isn’t sorted, but I don’t need sortedness for a pair chosen by position: I need an argument for discarding pairs. Widest pair first, then move inward — opposite ends, if I can find the discard argument.”

Beat 3, brute force. “Every pair: $n^2/2$ areas, $O(n^2)$. Fine to name, not to write.”

Beat 4, the invariant — the whole question. “Start at the outermost pair. The area is width times the shorter line. Now: moving the taller line inward can never help — the width shrinks, and the height is still capped by the same shorter line. So every pair using the current shorter line at this width or less is already beaten, and I can discard the shorter line entirely. That is the opposite-ends move: each step throws away a whole set of pairs that are proven no better.”

That paragraph is [day 028](#)’s row-and-column deletion argument with area instead of a target sum. Producing it — not the code — is what the problem is for.

Beat 5, code.

```
left, right = 0, len(height) - 1
best = 0
while left < right:
    width = right - left
    if height[left] < height[right]:
        best = max(best, width * height[left])
        left += 1
    else:
        best = max(best, width * height[right])
        right -= 1
```

“Record the area this pair makes, then discard the shorter line. Ties: moving either is safe — both are capped by the same height — so the `else` taking the right one is fine.”

Beat 6, test. `[1,8,6,2,5,4,8,3,7]` → 49 — the pair of 8s... no: width 7 × height 7, the 8 at index 1 and the 7 at the end. Say the winning pair out loud, that is the point of testing by hand. `[1,1]` → 1. `[4,3,2,1,4]` → 16, the two ends. `[1,2,1]` → 2. Single element → loop never runs, 0.

Match two — Number of Substrings Containing All Three Characters

“Given a string of only *a*, *b* and *c*, count the substrings that contain at least one of each. “`“abcabc”` → 10.” — LeetCode 1358.

Beat 1, clarify. “Exactly the letters *a*, *b*, *c*, nothing else? Count substrings, so positions matter — the same text at two places counts twice?” Yes.

Beat 2, route out loud. “Substrings — contiguous, window side. ‘Count’, not ‘longest’ — a counting window, day 034. But the condition is ‘contains all three’, an at least, where day 034 counted at most — so the counting line lands differently, and I want to re-derive it rather than paste it.”

Saying that a memorised line does not fit, before it bites, is the mark of the invariant being real.

Beat 3, the re-derivation. “For at-most conditions, validity survives shortening, so I count windows ending at `right` that start at or after `left`. For at-least conditions it is the mirror: validity survives lengthening — if `s[left..right]` contains all three, then any start earlier than `left` also works. So: shrink the window while it is still valid, walking `left` past every start that works, and then `left` itself is the count of valid starts — positions 0 to `left - 1` — for this `right`. Add `left`, not `right - left + 1`.”

Beat 5, code.

```
count: defaultdict[str, int] = defaultdict(int)
left = 0
total = 0
for ch in s:
    count[ch] += 1
    while len(count) == 3:          # still valid: keep shrinking
        count[s[left]] -= 1
        if count[s[left]] == 0:
            del count[s[left]]      # len(count) is the condition → del, as always
        left += 1
    total += left                  # starts 0 .. left-1 were all valid
```

“The shrink loop deliberately overshoots — it stops only when the window is invalid again — which is exactly what makes `left` equal the number of valid starts. And the `del` at zero is load-bearing for the same reason as ever: `len(count)` is the condition.”

Beat 6, test. “`“abcabc”` → 10, and check the first hit by hand: at the first `c`, `left` walks to 1, adding 1 — the substring `“abc”`. `“aaacb”` → 3. `“abc”` → 1. `“aab”` → 0 — no `c`, the while never fires, total stays 0. Empty string → 0.

The complete solutions

```
from collections import defaultdict

def max_area(height: list[int]) → int:
    """LeetCode 11. Opposite ends: discard the shorter line – moving the taller
    one can only shrink width under the same height cap."""
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        width = right - left
        if height[left] < height[right]:
            best = max(best, width * height[left])
            left += 1
        else:
            best = max(best, width * height[right])
            right -= 1
    return best

def number_of_substrings(s: str) → int:
    """LeetCode 1358. At-least counting: shrink while VALID, then add left –
    every start before left works for this right."""
    count: defaultdict[str, int] = defaultdict(int)
    left = 0
    total = 0
    for ch in s:
        count[ch] += 1
        while len(count) == 3:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        total += left
    return total

if __name__ == "__main__":
    print(max_area([1, 8, 6, 2, 5, 4, 8, 3, 7])) # 49
    print(max_area([1, 1])) # 1
    print(max_area([4, 3, 2, 1, 4])) # 16
    print(max_area([1, 2, 1])) # 2

    print(number_of_substrings("abcabc")) # 10
    print(number_of_substrings("aaacb")) # 3
    print(number_of_substrings("abc")) # 1
    print(number_of_substrings("aab")) # 0
```

6 What it costs

Match one

Each iteration retires one line — `left` up or `right` down — and they start `n - 1` apart, so exactly `n - 1` iterations of constant work. **$O(n)$ time, $O(1)$ space.** Against the brute force's $n^2/2$ pairs: at `n = 100,000`, five billion areas against a hundred thousand steps.

Match two

The standard two-edges count: the loop body runs `n` times, and `left` advances at most `n` times across the whole run — at most `2n` moves, constant map work each (the map holds at most three keys). **$O(n)$ time, $O(1)$ space** — the alphabet is fixed at three, so even the map is constant.

The cost that is actually being examined today

Neither problem's asymptotics are the test — everything in this corridor is $O(n)$, as [day 035](#) said. What the mock measures is the **argument per move**: why discarding the shorter line is safe, why `left` equals the count of valid starts. An $O(n)$ loop you cannot defend scores like a guess, because the follow-up — “*why is that correct?*” — arrives every single time.

7 The traps

The near-miss: moving the taller line

The plausible-but-backwards version of match one:

```
if height[left] > height[right]:
    left += 1          # moving the TALLER side
else:
    right -= 1
```

8

On `[1,8,6,2,5,4,8,3,7]` it returns 8 instead of 49. It walks straight past the winning pair, because it keeps the short line — the binding constraint — and discards the tall one that cost nothing. The fix is the argument, not the sign: the shorter line caps every pair it could ever form at this width or less, so the shorter line is the one with nothing left to offer.

The near-miss: yesterday's counting line in today's problem

Paste `day 034`'s line into match two:

```
while len(count) == 3:
    ...shrink...
    total += right - left + 1          # at-most line in an at-least problem
```

11

Eleven, not ten — and the wrongness is instructive: after shrinking *while valid*, the window is invalid, so `right - left + 1` counts the substrings ending here that do **not** contain all three. It computes the exact complement: 21 total substrings of `"abcabc"` minus the 10 good ones. The counting line is not a formula to paste; it is the answer to “*which starts are valid for this right?*” — asked fresh each time. At-most: starts at or after `left`. At-least: starts before `left`.

The real error: the empty-input reflex, again

Match one with a careless initial:

```
left, right = 0, len(height) - 1
print(height[left])
```

On `height = []`:

```
Traceback (most recent call last):
  File "day36.py", line 2, in <module>
    print(height[left])
IndexError: list index out of range
```

The loop version survives (`while 0 < -1` never runs) — but any code that touches `height[0]` before the loop dies on empty input. The five test inputs exist precisely because at minute thirty you will not *think* of the empty case; you will only *run the ritual* — which is why it is a ritual.

The mock-round trap: the silent stall

Not a code bug, and it fails more mocks than any code bug: sixty seconds of silence while stuck. There is always a legal sentence available — the invariant you are trying to prove, the input that is bothering you, the two candidate rooms you are torn between. “*I want opposite ends but I haven’t justified discarding a line yet — let me test the discard on a three-element example*” is progress the interviewer can score. Silence is not. If you catch yourself quiet during today’s timer, that — not the code — is the thing to repeat next Thursday.

8 In the interview

How it gets asked

- *“Let’s do a couple of problems.”* — the screening round itself; this phase supplies most of its mediums.
- *“Talk me through your approach before you code.”* — an explicit request for the routing sentence and the invariant; candidates who code first are stopped and asked again.
- *“Why is that correct?”* — the invariant question, arriving mid-code, on schedule.
- *“That works — can you do it without the extra memory?”* — the pointer-side upgrade question: hash map to opposite ends, stack to backwards pointers.

What to say out loud, in the first ninety seconds

The compressed match ritual — this is the script, for any problem in the family:

1. **The two contract questions this family always needs.** *“Can values be negative? Can the input be empty?”* — one kills windows-over-sums, the other kills careless initials.
2. **The routing sentence.** *“Contiguous and counting, so a counting window”* — or wherever the five questions land, said as one breath.
3. **The invariant, before any code.** *“Each move discards only pairs that are proven no better” / “every start before left is valid”* — the sentence you will be asked to defend later, volunteered now.
4. **Brute force and target.** *“Pairs at $O(n^2)$; the pointer argument gives $O(n)$.”*
5. **The test list, promised.** *“I’ll finish with empty, single, all-same, all-different, and the adversarial one.”* Saying it early means you cannot forget it late.

The follow-ups

“Why is discarding the shorter line safe? Convince me.” Fix the current pair: left line height 3, right line height 8, width 10 — area 30, capped by the 3. Every other pair that still uses the left line has width at most 9 and height still at most 3 — the shorter line caps the height whatever partner it gets — so all of those pairs are at most 27: strictly worse than what I already recorded. The left line’s future is exhausted, and I discard it. The taller line makes no such promise: a still-taller partner inside could pair with it for a bigger area, so it must stay. That is the same shape of argument as the pair-sum walk from day 028 — each move deletes a set of candidates that have been *proven* beaten, never merely guessed — and it is why meeting in the middle having recorded the best along the way is a proof, not a heuristic.

“Your counting answer added `left`, but yesterday you added `right - left + 1`. Which is right?” Both — for different conditions, and the difference is which direction validity survives. An at-most condition survives shortening: if the window is valid, every later start also gives a valid window, so after shrinking to the tightest valid window I count starts from `left` to `right` — `right - left + 1`. An at-least condition survives lengthening: if `s[left..right]` contains all three letters, every *earlier* start does too, so I shrink while still valid, overshooting to the first invalid start, and the valid starts are 0 to `left - 1` — exactly `left` of them. Pasting one formula into the other problem computes the complement — 11 instead of 10 on `"abcabc"`, which is 21 total substrings minus the 10 good ones. The habit that protects me is asking “which starts are valid for this right?” fresh, every time, rather than remembering a line.

“You’re three minutes in and stuck between two approaches. What do you do?”

Say so, with the specifics: name both rooms and the missing piece — “this is either opposite ends or a window; what decides it is whether the answer is about a pair or a run, and the phrase ‘substring’ settles it — window.” If it does not settle in one sentence, I take the smallest example that could distinguish them and run both intuitions on it by hand for sixty seconds; a three-element input usually kills one of the two. What I do not do is pick silently and start typing, because a wrong room discovered at minute twenty is unrecoverable, and because the interviewer scores the deciding process — which only exists for them if it happens out loud.

A model answer

The first two minutes of match two, as one continuous piece — compare your recording against it:

“Count the substrings containing at least one a, one b and one c. Substrings — contiguous, so window side. Count, not longest — so a counting window. But my counting line from at-most problems assumes validity survives shortening, and here it’s the opposite: containing all three survives *lengthening*. So I’ll re-derive: for each right edge, if the window from `left` still contains all three, every earlier start works too. I’ll shrink while the window is still valid, and then the number of valid starts for this right is exactly `left`. I add `left` each step.

The window carries a three-key count map, and since my loop condition is `len(count) == 3`, I must delete keys at zero or the length lies. Both edges only move forward — at most $2n$ moves — so $O(n)$ time, $O(1)$ space, the map never holds more than three keys.

Before I code: empty string gives 0, a string missing one letter gives 0 — the while never fires and `left` stays 0 — and `"abc"` should give exactly 1. Those are my first three tests. Writing it now.”

9 Recall card

- **The invariant is the answer.** Seven tools, seven sentences — say the sentence before the code, every time; a loop you cannot defend scores as a guess.
- **Container: discard the shorter line** — it caps every pair it could still form at smaller width. Moving the taller line returns 8 where 49 exists.
- **Counting has two directions.** At-most survives shortening → add `right - left + 1`. At-least survives lengthening → shrink while valid, add `left`. The wrong one computes the complement.
- **Pressure eats, in order: the routing minute, the edge cases, the narration.** Counter each with a ritual — one sentence before code, five inputs by voice, stuck-narration instead of silence.
- **Everything here is $O(n)$ and the interviewer knows it.** The score lives in the sentences between the lines of code.

1 What this is, and why they ask it

NoSQL is a family name for databases that do not organise data as normalised relational tables — key-value stores, document stores, wide-column stores, graph stores. The name says what they are *not*, which is why it confuses people: the four families have less in common with each other than some of them have with Postgres. What they share is one design decision: **store the data in the shape you will read it, instead of one fact in one place assembled by joins.**

Interviewers ask “SQL versus NoSQL?” at every level, and it is a trap question in a specific way: the popular answer — *“NoSQL is faster and scales better”* — is a failing answer, because it is a slogan where a trade-off should be. The answer that passes names what you give up (joins, ad-hoc questions, cross-entity transactions, single-copy facts) and what you buy (access-path speed, horizontal scaling, flexible records), and then refuses to pick a side without knowing the access patterns. The next four days walk the families one by one; today builds the frame they hang on.

2 The story

Two women in one family feed people for a living, and their kitchens disagree about everything.

Meena runs a tiffin service from her flat — forty steel boxes out of the door by nine, a different menu every day. Her kitchen is arranged like a shop. Every ingredient has one labelled jar, every jar has one place on the racks: dals on the second shelf, spices in the drawer in alphabetical order, rice and flours at the bottom. Ask her how much cumin is left and she answers in five seconds, from one jar. When the wholesaler changes her turmeric brand, she changes one jar. But cooking any single dish means walking the kitchen — eight jars from five shelves for a simple sambar — and on a morning when three dishes run in parallel, most of her time is collecting.

Her sister-in-law Farida runs a dosa cart near the college, and the cart is arranged like the serving window, not like a shop. One tub per item on the menu. The masala dosa tub holds everything masala dosas need — the batter, the potato filling, the chutney portions, the ladle. Order comes; she pulls one tub; everything is in her hand. No walking, no collecting. That is why her queue moves.

But ask Farida how much cumin she has and she laughs. There is cumin in the dosa tub, cumin in the upma tub, cumin in three chutney boxes — she would have to open every tub and add it up, and she has never once done it. When her salt supplier changed, she repacked nine tubs. And when her nephew suggested a new item for the menu, the hard part was not the recipe — it was that nothing on the cart was arranged for it.

Neither kitchen is wrong. Meena's is arranged for **questions** — any question, asked once, answered from one place. Farida's is arranged for **serving** — one known order, served fast, a thousand times a day. Each pays for its arrangement exactly where the other collects.

3 The idea in plain English

Meena's kitchen is a relational database. Farida's cart is NoSQL. The whole of today is in the last line of the story: **arranged for questions, or arranged for serving** — and each pays where the other collects.

The relational deal, restated

Everything since [day 026](#) has described Meena's kitchen. **Normalisation** — [day 029](#) — puts each fact in exactly one place: one jar per ingredient. **Joins** — [day 028](#) — assemble a dish by walking the shelves. The payoff is enormous and easy to forget: *any* question can be asked, including ones nobody predicted, and every answer is consistent because no fact has a second copy that could disagree. The price is the walking — every read that spans entities pays for assembly at read time.

The NoSQL deal

Turn it around. Decide **first** what the application will ask — “give me this user's cart”, “render this product page” — and store each answer pre-assembled, like Farida's tubs. A read becomes one lookup of one shaped record. The price arrives on the other side of the counter:

- **Unplanned questions are expensive or impossible.** “How much cumin?” means opening every tub — a full scan, or a separate analytics copy of the data.
- **Facts get copied**, and copies drift. The supplier change touched nine tubs. Every copy needs the reconciliation discipline [day 029](#) demanded.
- **Guarantees shrink to the tub.** Most NoSQL stores make their ACID promises about one record, not across records — [day 033](#)'s transaction, at its full strength, is a relational feature.

The four families, in one pass

“NoSQL” covers four different data models. One line each today; the next three days take them up properly:

FAMILY	THE RECORD IS	PRODUCTS	BUILT FOR
Key-value	an opaque value behind a key	Redis, DynamoDB	exact-key lookups, blindingly fast — day 037
Document	a nested JSON-like document	MongoDB	one entity’s whole world in one read — day 038
Wide-column	rows grouped and sorted within partitions	Cassandra	huge write volume, known queries — day 039
Graph	nodes and edges	Neo4j	relationship-walking questions

Why “NoSQL is faster” is not a sentence

Faster **at what**? Farida serves a dosa faster than Meena could — and answers the cumin question somewhere between slowly and never. A document store renders the page it was shaped for in one read; ask it for “all users who bought X and Y last month” and it is slower than the relational database would have been, or silent. Speed is a property of a *query against an arrangement*, not of a product. The honest comparison is: **NoSQL is faster at the access paths you designed in, by paying for them at write time and giving up the paths you did not**. Say it that way and the interview changes tone.

There is a second, real reason for the family’s reputation, and it deserves its own sentence: **horizontal scaling**. Records that never need joining can be spread across a hundred machines by key, and each request touches one machine. Relational joins want the data together; splitting a relational database across machines is the hard, later part of this course. When people say “NoSQL scales”, this — not raw speed — is the thing that is true.

4 The picture

The same blog post, served by both kitchens:

RELATIONAL – arranged for questions

users

+-----+-----+

| id | name |

+-----+-----+

posts

comments

tags

+-----+-----+

+-----+-----+

+-----+

| id | title |

| post | body |

| post |

+-----+-----+

+-----+-----+

+-----+

read = 1 query joining 4 tables

(or 4 round trips, done badly)

write "rename Meena" = 1 row

"all posts tagged X" = easy

"top commenters" = easy

DOCUMENT – arranged for serving

posts collection, one document:

```
{
  "title": "Why B-trees",
  "author": {"id": 7,
             "name": "Meena"},
  "comments": [ ... ],
  "tags": ["storage", "trees"]
}
```

read = 1 lookup, already shaped

write "rename Meena" = every
document she ever wrote

"all posts tagged X" = only if

you shaped an index for it;

"top commenters" = a scan

What to notice: neither column is winning. Read the two “rename” lines, then the two bottom-left questions — every advantage on one side has its bill on the other.

Why the split-across-machines argument favours the tubs:

by key, no joins:

machine A: users a-h

machine B: users i-q

machine C: users r-z

request for
user "meena"
→ ONE
machine

with joins:

posts on machine A,
comments on machine B,
users on machine C →
every join crosses the network

What to notice: the key-shaped arrangement gives each request a single home. The join-shaped arrangement makes the network part of every query — which is the hard problem the distributed phase of this course exists to study.

5 How it actually works

Key-value: Redis and DynamoDB

The store is a giant dictionary: `GET key`, `PUT key value`, the value opaque. Redis keeps it in memory — sub-millisecond reads, hundreds of thousands of operations a second on one node — and is the standard cache, session store and counter. DynamoDB puts the same model on disk across many machines with hash partitioning by key: single-digit-millisecond reads at any scale, priced per request. Neither can answer “which values contain X?” — there is no *inside* to the value, as far as the store is concerned. Tomorrow’s lesson.

Document: MongoDB

Records are JSON-like documents, nested, flexible per record, gathered into collections. The unit of atomicity is **one document** — updating one document is atomic; updating three is your problem unless you opt into its newer multi-document transactions, which cost. Secondary indexes exist, so it is far more queryable than a key-value store — but the design centre is still “one entity, one document, one read”. [Day 038](#).

Wide-column: Cassandra

Rows live inside **partitions** chosen by a partition key; within a partition, rows are sorted by clustering columns. Writes append to a log-structured merge tree — from [day 031](#), the write-favouring shape — which is why Cassandra swallows enormous write volumes across hundreds of nodes with no single leader. The price is that **you must know your queries when you design the table**: data is queryable by its partition key and sort order, and very little else. [Day 039](#).

Graph: Neo4j

Nodes and edges as first-class records, so “friends of friends who like X” walks pointers instead of joining tables repeatedly. Niche but unbeatable in its niche — recommendation, fraud rings, networks. It gets this one line today, and reappears in the graphs phase of the DSA track.

The blur, which interviewers respect you for naming

The border is not clean, and saying so signals currency. Postgres has `JSONB` — documents inside a relational database, indexed, queried, transactional — which quietly serves most “we might need Mongo” cases. MongoDB grew multi-document transactions. And **distributed SQL** — Google Spanner, CockroachDB — puts real SQL and real transactions on top of horizontal partitioning, at the cost of coordination machinery that belongs to the later phases of this course. The families are converging; the *trade-offs* are what stay fixed, which is why today taught the trade-offs.

6 The numbers

The read the document was shaped for

Rendering one product page:

```
relational, done badly (one query per table, sequentially):  
  4 round trips × (0.5 ms network + query time ~0.5 ms) ≈ 4 ms  
relational, done properly (one join): ≈ 1-1.5 ms  
document store, one shaped read:      ≈ 0.5-1 ms  
  
at 1,000 pages/second the gap between properly-joined SQL and a  
document read is real but small. The 4-round-trip version is the  
one that hurts – and that is an application bug, not a database law.
```

The honest number: most of the speed people attribute to NoSQL is the speed of **one read versus several round trips** — which a well-written join usually also achieves.

The write the document pays

The duplicated fact, priced:

```
author's name embedded in 40,000 of her documents, 20 bytes each:  
  storage cost: 800 KB → nothing. storage is not the issue.  
  rename cost: 40,000 document writes vs 1 row update  
  at 5 ms per write, done naively: 200 seconds of writing
```

Duplication is cheap to hold and expensive to change — which is why the modelling question tomorrow-but-one is *“how often does this fact change?”*, not *“how big is it?”*.

Where horizontal scaling actually bites

```
one well-tuned Postgres node comfortably serves:  
  ~10,000-50,000 simple transactions/second, terabytes of data  
  
Cassandra at Discord/Netflix scale:  
  millions of writes/second across hundreds of nodes  
  
the gap between those two lines is the honest domain of NoSQL-for-scale.  
Most products live their whole lives inside the first line.
```

The multiplication to carry: 50 million users × 20 events a day = 1 billion writes ≈ 11,600 a second average — **inside one Postgres node's range**. NoSQL-for-scale is a real need at a scale most systems never reach, and knowing which side of the line a design sits on is the entire game of [day 040](#).

7 The trade-offs

What you hand over at the door

Choosing a NoSQL store for a system's main record of truth gives up, in rough order of pain: **ad-hoc questions** (the analytics team now needs a copy of the data somewhere queryable), **cross-entity transactions** (the all-or-nothing of [day 033](#) shrinks to one record), **single-copy facts** (every embedded copy needs an owner and a reconciliation path), and **joins** (assembly becomes application code). What you get: reads shaped to the product's hot paths, writes that scale by adding machines, and records that vary without schema ceremony.

I would not use it if...

I would not choose NoSQL if the access patterns are still unknown — early products pivot, and a relational schema is the arrangement that survives a pivot, because it is arranged for questions you have not thought of yet. **I would not choose it if** the domain is transactional across entities — money moving between accounts is a two-row atomic write, and that is the relational deal. **And I would not choose relational alone if** one access path utterly dominates at a scale one node cannot hold — a feed, a session store, an event firehose — which is precisely where the next three days' tools earn their keep.

The honest default

Start relational — Postgres — and add specialised stores per access path as measurements demand: Redis in front for hot keys, a document or wide-column store when a specific read or write path outgrows the node. “SQL or NoSQL” is rarely the real decision in production; the real decision is **which access path gets its own arrangement**, and most systems end up polyglot for exactly Meena-and-Farida reasons.

8 In the interview

How it gets asked

- “*What is the difference between SQL and NoSQL?*” — the direct form; they are listening for a trade-off, not a feature list.
- “*Why is NoSQL faster?*” — the baited form; the right answer disputes the premise politely.
- “*Would you use MongoDB or Postgres for this?*” — the applied form, which is [day 040](#)'s whole lesson.

- “How can Cassandra handle writes Postgres can’t?” — the mechanism form: partitioning by key, no joins to keep together, log-structured writes.

What to say out loud, in the first ninety seconds

1. **Refuse the slogan in the first sentence.** *“NoSQL isn’t one thing and isn’t simply faster — it’s four data models sharing one decision: store data in the shape you’ll read it, instead of normalised tables assembled by joins.”*
2. **Name the trade in both directions.** *“You buy your designed access paths — one shaped read, partitionable by key across machines. You sell ad-hoc queries, cross-record transactions, and single-copy facts.”*
3. **Give the families ten seconds.** *“Key-value for exact lookups — Redis, DynamoDB. Document for one entity’s world in one read — MongoDB. Wide-column for huge write volume with known queries — Cassandra. Graph for relationship-walking — Neo4j.”*
4. **Say where the scaling claim is true.** *“The real scaling advantage is horizontal: no joins means data splits cleanly by key, one machine per request. That’s true — and it matters above a scale one good Postgres node can’t hold, which is further away than people think.”*
5. **Land on the default.** *“So my default is relational until a named access path and a real number push a piece of the data into a specialised store.”*

The follow-ups

“So is NoSQL ever actually faster?” Yes — at the access path it was shaped for, and the mechanism is worth naming precisely. A document read replaces a join: the assembly work moved from read time to write time, so the read is one lookup of pre-assembled data. A key-value read replaces even the parsing: Redis serves from memory in under a millisecond. A Cassandra write appends to a log-structured tree instead of updating a B-tree in place — day 031’s read-favouring versus write-favouring shapes — so sustained write throughput per node is genuinely higher. Every one of those is a real speedup, and every one is paid for: the document store answers unplanned questions badly, the key-value store not at all, and Cassandra requires the queries to be designed into the table. What is *not* true is the blanket claim — put a well-indexed Postgres and a document store side by side on an ad-hoc analytical question and the relational side wins, because that is the query it is arranged for.

“Why does NoSQL scale horizontally more easily than a relational database?” Because of what has to stay together. If records never join, you can split them across a hundred machines by hashing the key, and every request computes its machine and touches only it — no cross-machine coordination on the read path, so capacity grows nearly linearly with machines. A relational database’s value is precisely the cross-record operations — joins and multi-row transactions — and those want the participating rows on the same machine. Split the tables across machines and a join becomes network traffic

and a transaction becomes a distributed commit, which is the expensive machinery from day 033's two-phase-commit discussion. It is not that relational databases cannot be distributed — Spanner and CockroachDB do it — it is that they must solve the coordination problem NoSQL sidesteps by giving up the operations that need it. Same trade as everywhere today: the families that scale easiest are the ones that promised least across records.

“MongoDB has transactions now. Doesn't that end the debate?” It narrows it, and the direction of travel is real — Mongo grew multi-document ACID, Postgres grew JSONB documents, and distributed SQL grew both. But defaults and design centres still differ. In Postgres, a five-row transaction is the free, normal case; in MongoDB it is an opt-in with latency and abort costs, on top of a data model that encourages you not to need it — the idiomatic Mongo design embeds related data in one document so the single-document atomic update suffices. Choosing by feature checklist misses this: the question is not “does it have transactions?” but “is the workload's normal case the store's normal case?”. A domain that constantly needs cross-entity atomicity is fighting a document store's grain even when the feature exists — and a domain that reads one aggregate at a time is paying Postgres's assembly cost for guarantees it rarely uses. Grain over checklist is the durable answer.

A model answer

“NoSQL is an umbrella for four data models — key-value, document, wide-column, graph — that share one decision: store the data in the shape you’ll read it, rather than normalised tables assembled by joins at read time.

That decision trades in both directions. You buy the designed access paths: a product page is one shaped read instead of a four-table join, and because records don’t join, data splits cleanly by key across machines — that’s the honest version of ‘NoSQL scales’. You sell the things normalisation gave you: ad-hoc questions the design didn’t anticipate, transactions that span records, and facts that exist in one place — an embedded author name lives in forty thousand documents, and renaming her is forty thousand writes instead of one.

So ‘NoSQL is faster’ isn’t a sentence I’d say. Faster at what? A document store beats Postgres at the read it was shaped for and loses on the analytical question nobody planned. Speed belongs to a query against an arrangement, not to a product.

Concretely: Redis and DynamoDB for exact-key lookups, MongoDB when one entity’s whole world is read as a unit, Cassandra when write volume outgrows a node and the queries are known in advance, Neo4j for relationship-heavy questions. And one honest number for calibration: a billion events a day is about twelve thousand writes a second, which a single tuned Postgres node handles — so my default is relational until a named access path with a real number outgrows it, and then that path, not the whole system, moves to the store shaped for it.”

9 Recall card

- **NoSQL = four models, one decision:** store data in the shape you will read it. Arranged for serving; relational is arranged for questions.
- **The trade, both directions:** buy designed access paths + horizontal scaling by key; sell ad-hoc queries, cross-record transactions, single-copy facts.
- **“Faster” needs an object.** Faster at the shaped path (assembly moved to write time); slower or silent elsewhere. Speed belongs to a query against an arrangement.
- **The scaling claim, stated honestly:** no joins → split by key → one machine per request. It matters above what one node holds (~tens of thousands of TPS) — most systems never get there.
- **Families and homes:** Redis/DynamoDB key-value, MongoDB document, Cassandra wide-column, Neo4j graph — and Postgres JSONB quietly covers many “need Mongo” cases. Default relational; move access paths, not systems.

DSA topic: Two pointers revision and mock round **System design topic:** NoSQL: what it actually means

Code these, in this order

Today is a mock, so the protocol matters more than the list. **Thirty-five minutes per problem, a timer, standing, talking continuously, nothing open.** Route out loud, say the invariant, code in one pass, run the five test inputs by voice. Score yourself afterwards against §5 of the lesson — on the sentences before the code, not the code.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Container With Most Water	LeetCode 11 (Medium)	Whether you can <i>produce</i> the discard argument — moving the taller line can never help.
2	Number of Substrings Containing All Three Characters	LeetCode 1358 (Medium)	Re-deriving the counting line for an at-least condition instead of pasting yesterday's.
3	3Sum Closest	LeetCode 16 (Medium)	Opposite ends inside a fixed outer loop, unseen variant — tracking nearest instead of equal.
4	Minimum Window Substring	LeetCode 76 (Hard)	The phase's summit, revisited cold: shape A with a need map and a missing counter.

Problems 3 and 4 have no transcript in the lesson. That is deliberate — they are the second match of each pair, done truly unseen.

On problem 1, argue before you move

Before writing the loop, say the discard argument to the wall: the shorter line caps every pair it could still form at this width or less, so it is spent. Then break it on purpose — move the taller line instead — and run `[1, 8, 6, 2, 5, 4, 8, 3, 7]`. You get 8, not 49. Say which pair the broken version walked past.

On problem 2, derive the counting line fresh

Ask the question aloud before coding: *for this right edge, which starts are valid?* If the window still contains all three after shrinking, the valid starts are everything before `left` — so add `left`. Then paste yesterday's `right - left + 1` instead and run `"abcabc"`: you get 11, the exact complement (21 total substrings minus the 10 good ones). Explain why it counts the bad ones.

On problem 3, name the composition

This is [day 028](#)'s 3Sum skeleton with one change. Say what stays (sort, fix one, walk two) and what changes (record the nearest sum instead of hunting an exact one — no duplicate-skipping needed, and the update is an `abs` comparison). Test with `([-1, 2, 1, -4], 1) → 2`.

On problem 4, respect the summit

If it takes the full thirty-five minutes, that is the honest result. The pieces you own already: shape A (shrink while valid, record inside), a `need` map, and the `missing` counter that makes validity $O(1)$. Test on `("ADOBECODEBANC", "ABC") → "BANC"` and `("a", "aa") → ""`.

The invariant drill

One sentence each, from memory, under five seconds — the sentence that makes the tool correct:

1. Opposite ends.
2. Read and write.
3. Fast and slow.
4. Fixed window.
5. Variable window.
6. Window with a map.
7. Counting window, at-most.
8. Counting window, at-least.

The stall drill

Practise the recovery sentence. For each stuck-state, say the narration that keeps the round alive:

1. Torn between two rooms.
2. The invariant will not come.
3. A test input just failed and you do not know why.

The slogan-repair drill

Each sentence below is the failing answer. Say the passing version out loud:

1. "NoSQL is faster."
2. "NoSQL scales better."
3. "MongoDB is schemaless."
4. "We should use NoSQL because we have a lot of data."

The families drill

For each need, name the family and one product, in under five seconds:

1. Session tokens looked up by id, sub-millisecond.
2. A product page read as one unit, fields varying by category.
3. A million sensor readings a second, queried by device and time.
4. “Friends of friends who bought this.”
5. Money moving between two accounts, atomically.

The arithmetic drill

From memory, in under two minutes:

- A billion events a day — writes per second, and which side of one Postgres node that lands on.
- An author’s name embedded in 40,000 documents — the storage cost, the rename cost, and which one matters.
- Four sequential round trips at 1 ms against one joined query — the gap, and why it is an application bug rather than a database law.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why is it always safe to move the pointer at the shorter line?* The capping argument, a concrete pair with numbers, and the connection to day 028 — each move discards only pairs proven beaten, so the walk is a proof.
2. *What is the difference between SQL and NoSQL?* Arranged for questions against arranged for serving. The trade in both directions, the four families with a product each, where the scaling claim is honestly true, and the relational default.
3. *You counted subarrays two different ways this week. When is it `right - left + 1` and when is it `left`?* At-most survives shortening; at-least survives lengthening. Which starts are valid each time, and what the wrong choice computes on `"abcabc"`.

Before you move on

- [] I did both mock problems standing, timed, talking — and scored the sentences, not the code.
- [] I can say all eight invariants from the drill without looking.

- [] I narrated at least one stuck moment instead of going quiet.
- [] I can repair all four NoSQL slogans into trade-offs.
- [] I can name the family and a product for all five needs in the drill.
- [] I know which scale honestly needs NoSQL, with the events-per-day arithmetic.
- [] I answered all three questions above out loud.



PART V

Prefix sums

Days 37 to 41 · 5 chapters

Alongside, on the system design track:

Databases from zero

037 Prefix sums: answering range queries instantly

038 Subarray sum equals K: prefix plus hash map

039 Difference arrays: range updates, cheaply

040 2D prefix sums and inclusion-exclusion

041 Prefix sums revision and mock round

Prefix sums: answering range queries instantly

DATA STRUCTURES AND ALGORITHMS

Prefix sums: answering range queries instantly

You can precompute once and then answer any range-sum question in $O(1)$.

SYSTEM DESIGN

Key-value stores

You can say what Redis and DynamoDB are good at, and what they cannot do.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Answer many range-sum queries on a fixed array.*
- *When would you use a key-value store instead of a relational database?*

Prefix sums: answering range queries instantly

1 What this is, and why they ask it

A **prefix sum** is a running total: for each position, the sum of everything up to it. Build that once, in one pass, and the sum of *any* stretch of the array becomes a single subtraction — the total up to the stretch's end, minus the total up to just before its start. One $O(n)$ preparation buys unlimited $O(1)$ answers.

This opens a new phase because it is a new *kind* of idea. The window days made one pass clever; prefix sums **precompute** — spend memory and preparation time now so that questions cost nothing later. That trade is everywhere in computing, and interviewers use this family to see whether you recognise it: the tell is *many queries against data that does not change*. Range Sum Query — LeetCode 303 — is the direct form, easy to code and easy to get subtly wrong at index zero. And tomorrow this same running total plus [day 021](#)'s map solves the subarray-sum problems that yesterday's windows could not touch — the negatives case has been waiting since [day 032](#).

2 The story

Ramu drives a delivery van for a wholesale cloth merchant, the same route every day: the godown, then eleven shops strung along the highway, ending at the big showroom in the next town.

The owner is a man for questions. How far is it from the third shop to the seventh? From the godown to the fifth? If we drop the ninth shop and go straight from the eighth to the tenth, what does that save? He asks because diesel is money and the van is old.

For the first month, Ramu answered the slow way. He knew every leg of the route — godown to first shop four kilometres, first to second seven, second to third three — so for each question he would add the legs up in his head, one by one, from the start of the stretch to the end. Third shop to seventh meant adding four legs. The owner never asked just one question, either. He asked six or seven in a row, and Ramu stood there adding.

Then Ramu noticed the van had been keeping a better answer all along. The meter behind the steering wheel only ever counts up, and one morning, on a whim, he noted the reading into his phone at every stop. Leaving the godown: 40,180. First shop: 40,184. Second: 40,191. Third: 40,194. All the way to the showroom.

Now every question is one subtraction. Third shop to seventh? Reading at the seventh minus reading at the third. Done before the owner finishes the sentence. It does not matter whether the stretch covers two stops or all eleven — one subtraction, always the same speed.

The detail he learnt to be careful about: the godown reading, the 40,180 from **before the first shop**. The first time he forgot to note it, every question that started from the beginning of the route — godown to anywhere — had no answer, because there was nothing to subtract. The list in his phone starts with the reading at the start of the day, before anything has been driven at all.

3 The idea in plain English

Ramu's legs between shops are the array. His noted meter readings are the **prefix sums** — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

From legs to readings

Take a seven-element array of leg lengths:

```
nums = [4, 7, 3, 6, 2, 5, 1]
```

The prefix sums are the meter readings — what the counter shows *after* each leg, starting from the reading before any leg at all:

```
prefix = [0, 4, 11, 14, 20, 22, 27, 28]
```

`prefix[i]` is the sum of the **first i elements** — `prefix[3]` is `4 + 7 + 3 = 14`, the reading after three legs. The array has 7 elements; `prefix` has 8, because the reading before the first leg — the plain 0 — is a real entry. That extra entry is called a **sentinel**: a value placed at the boundary so that the boundary needs no special treatment.

The subtraction

The sum of any stretch is reading-at-the-end minus reading-before-the-start:

```
sum of nums[i..j] = prefix[j + 1] - prefix[i]
```

Sum of `nums[2..4]` — the stretch `3, 6, 2` — is `prefix[5] - prefix[2] = 22 - 11 = 11`. The `+ 1` is not decoration: `prefix[j + 1]` is the reading *after* element `j`, and `prefix[i]` is the reading *before* element `i`, so the difference is exactly the elements

from `i` to `j`, both ends included. Say the formula as a sentence — *after the end, minus before the start* — and the off-by-ones lose their grip.

And the sentinel earns its place at `i = 0`: sum of `nums[0..3]` is `prefix[4] - prefix[0] = 20 - 0`. Without the leading zero there is no “before the start” reading for stretches that begin at the beginning — Ramu’s forgotten godown reading — and §7 shows the two distinct ways that bug bites.

Why this is a new kind of tool

Every day since 27 has answered one question in one pass. Today’s shape is different:

```
prepare once:  O(n)      - build the readings
answer often:  O(1) each - one subtraction per question
```

The trade is worth it exactly when the data holds still and the questions keep coming — **many queries, fixed array** is the tell in the problem statement. One query only? The plain loop is already optimal; building the prefix first is ceremony. Data that changes between queries? Every change invalidates readings to its right, and rebuilding is `O(n)` per change — the honest limitation, and the interviewer’s favourite follow-up.

The running-total form

Half the problems in this family never store the prefix array at all — they carry the running total in one variable, updating it as they walk, and consult it on the spot. *Find the Pivot Index* — LeetCode 724 — asks for the position where the sum on the left equals the sum on the right. With `total` known, the right side is always `total - left_sum - nums[i]`, so one walk with one running number settles it. Same idea, `O(1)` extra space — the stored array is for when *later* questions need *arbitrary* stretches; the running variable is for when the walk itself is the only customer.

4 The picture

The array and its readings, boundaries marked:

index	0	1	2	3	4	5	6	
	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	
nums	4	7	3	6	2	5	1	
	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	
prefix	0	4	11	14	20	22	27	28
	^							^
	before							after
	everything							everything

prefix has one entry per BOUNDARY, not per element: 8 entries for 7 elements.

What to notice: the readings live on the boundaries between elements — that is why there is one more reading than there are elements, and why the leading 0 is not padding but the first boundary.

The subtraction, drawn:

```
sum of nums[2..4] = prefix[5] - prefix[2] = 22 - 11 = 11
```

```

nums      +-----+-----+-----+-----+-----+-----+-----+-----+
           | 4 | 7 | 3 | 6 | 2 | 5 | 1 |
           +-----+-----+-----+-----+-----+-----+-----+-----+
           |<--- wanted --->|
           0      4      11      14      20      22      27      28
                   ^              ^
           prefix[2]      prefix[5]
           before start      after end

```

everything up to the end of the stretch,
minus everything before it began – the wanted part is what remains.

What to notice: both readings already existed. No element of the stretch was visited to answer the question — that is where the $O(1)$ comes from.

5 The code, built step by step

Building the readings

```
prefix = [0] * (len(nums) + 1)
for i, x in enumerate(nums):
    prefix[i + 1] = prefix[i] + x
```

One pass. Each reading is the previous reading plus the leg just driven — `prefix[i + 1] = prefix[i] + nums[i]` — and the sentinel 0 is written before the loop by the initialisation itself.

The standard library builds it too, and in an interview you would say so:

```
from itertools import accumulate

prefix = list(accumulate(nums, initial=0))
```

`accumulate` yields running totals; `initial=0` supplies the sentinel. Write the loop today so the `+ 1` lives in your hands once; use `accumulate` from tomorrow.

Answering a query

```
def range_sum(i: int, j: int) → int:
    return prefix[j + 1] - prefix[i]    # after the end, minus before the start
```

That is the entire query path. No loop anywhere near it.

Range Sum Query as the interview asks it

LeetCode 303 wraps this in a class — build once in the constructor, subtract in the method:

```
class NumArray:
    def __init__(self, nums: list[int]) → None:
        self.prefix = list(accumulate(nums, initial=0))

    def sumRange(self, left: int, right: int) → int:
        return self.prefix[right + 1] - self.prefix[left]
```

The class shape is the lesson: the expensive work happens once, where it can be amortised; the method the caller hammers is a subtraction.

The pivot index, with a running total

```
total = sum(nums)
left_sum = 0
for i, x in enumerate(nums):
    if left_sum == total - left_sum - x:
        return i
    left_sum += x
```

`left_sum` is the reading just before `i`; the right side is everything minus the left minus the element itself. Check *before* adding — the element at `i` belongs to neither side. Note the edge this order gets right for free: on `[2, 1, -1]` the answer is index 0, whose left side is the empty sum 0.

The complete solutions

```
from itertools import accumulate

def build_prefix(nums: list[int]) → list[int]:
    """Readings at every boundary: prefix[i] = sum of the first i elements."""
    prefix = [0] * (len(nums) + 1)
    for i, x in enumerate(nums):
        prefix[i + 1] = prefix[i] + x
    return prefix

class NumArray:
    """LeetCode 303. Build once, subtract forever."""

    def __init__(self, nums: list[int]) → None:
        self.prefix = list(accumulate(nums, initial=0))

    def sumRange(self, left: int, right: int) → int:
        return self.prefix[right + 1] - self.prefix[left]

def pivot_index(nums: list[int]) → int:
    """LeetCode 724. The running-total form: no stored array needed."""
    total = sum(nums)
    left_sum = 0
    for i, x in enumerate(nums):
        if left_sum == total - left_sum - x:
            return i
        left_sum += x
    return -1

if __name__ == "__main__":
    nums = [4, 7, 3, 6, 2, 5, 1]
    prefix = build_prefix(nums)
    print(prefix) # [0, 4, 11, 14, 20, 22, 27, 28]
    print(prefix[5] - prefix[2], sum(nums[2:5])) # 11 11
    print(prefix[4] - prefix[0], sum(nums[0:4])) # 20 20 ← the sentinel at work

    arr = NumArray([-2, 0, 3, -5, 2, -1])
    print(arr.sumRange(0, 2), arr.sumRange(2, 5), arr.sumRange(0, 5)) # 1 -1 -3

    print(pivot_index([1, 7, 3, 6, 5, 6])) # 3
    print(pivot_index([2, 1, -1])) # 0 ← empty left side counts
    print(pivot_index([1, 2, 3])) # -1
```

Note the `NumArray` test: negatives everywhere, and nothing anywhere in today's code cares. The subtraction never needed the values to be positive — which is exactly what [day 032's](#) windows could not say, and why tomorrow exists.

6 What it costs

Build, then query

```
build:  one pass over n elements, one addition each      → O(n) time
store:  n + 1 readings                                    → O(n) extra space
query:  two lookups and one subtraction, regardless of span → O(1) each
```

The comparison that sells it

q queries on an array of length n, summing each stretch by loop against subtracting readings:

```
n = 100,000 and q = 100,000, stretches averaging n/2 long:

loop per query : 100,000 × 50,000 = 5,000,000,000 additions
prefix          : 100,000 build + 100,000 × 1 ≈ 200,000 operations

25,000 times less work – bought with one array of 100,001 numbers.
```

Say the trade explicitly: **O(n) memory purchased a 25,000× speedup**. Interviewers want the cost named on both sides, not just the win.

When the trade goes wrong

One query only: the loop costs n and the prefix costs n to build *plus* the query — no win, just ceremony. Updates between queries: one changed element invalidates every reading to its right, so each update costs O(n) rebuild; at that point the honest answers are “rebuild if updates are rare” or “a Fenwick or segment tree if they are not” — name them, both O(log n) per operation, and say they are beyond today’s scope. Recognising *when the tool loses* is worth as much as the tool.

The number to have ready

Build once in O(n), answer each range in O(1) — for a hundred thousand queries on a hundred thousand elements that is two hundred thousand operations against five billion. Cost: one extra array, and the array must hold still — an update invalidates everything to its right.

7 The traps

The near-miss: skipping the sentinel

Build the readings without the leading zero and use the tempting formula `prefix[j] - prefix[i - 1]`:

```

prefix = list(accumulate(nums))          # length n – no leading zero
def range_sum(i, j):
    return prefix[j] - prefix[i - 1]

print(range_sum(2, 5), sum(nums[2:6]))
print(range_sum(0, 3), sum(nums[0:4]))

```

```

19 19
-22 9

```

Interior stretches come out right, which is what makes this version survive a casual test. Any stretch starting at 0 reaches for `prefix[-1]` — the **last** reading, by Python’s negative indexing — and returns garbage with no error. Third appearance of this exact trap, after [day 021](#) and [day 035](#): an index that can reach `-1` is a bug that does not crash. The sentinel exists precisely so `i = 0` subtracts a real, honest 0.

The real error: the sentinel forgotten the other way

Keep the zero-less array but use today’s formula:

```

prefix = list(accumulate(nums))          # length 8, no leading zero
print(prefix[7 + 1] - prefix[2])         # sum of nums[2..7]

```

```

Traceback (most recent call last):
  File "day37.py", line 4, in <module>
    print(prefix[7 + 1] - prefix[2])      # sum of nums[2..7]
    ~~~~~^~~~~~
IndexError: list index out of range

```

The formula assumes `n + 1` readings; the array has `n`. Same root cause as the silent version — **the sentinel and the formula are a set**. Adopt them together: `initial=0`, size `n + 1`, and `prefix[j + 1] - prefix[i]`, always, and neither failure can exist.

The near-miss: answering queries after the array changed

```

arr = NumArray([4, 7, 3, 6])
nums_changes_somewhere_else()          # element 1 becomes 9
print(arr.sumRange(0, 2))               # still answers 14 – built from the old values

```

No error, ever — the readings are a snapshot, and they go stale the moment the array moves. LeetCode splits this into two problems deliberately: 303 is immutable, and 307 — *Range Sum Query, Mutable* — exists because the answer changes completely. If the interviewer says “now support updates”, the correct first sentence is that the prefix array’s deal is off, not a patch to it.

The contract corner: which ends are included

“Sum from i to j ” — inclusive of both? Half-open? The formula differs by one at each end, and both conventions are common. Ask, then anchor yourself with the sentence: *after the end, minus before the start*. For inclusive `[i, j]` that is `prefix[j + 1] - prefix[i]`; for half-open `[i, j)` it is `prefix[j] - prefix[i]`. One clarifying question, ten seconds, and the whole class of off-by-one bugs never happens.

The near-miss: overflow, elsewhere

In Python the running total grows without limit — `int` is arbitrary precision, from [day 005](#). In Java or C++ the prefix of a large array of large values overflows a 32-bit `int` silently. Worth one sentence in an interview if the interviewer’s language is not Python: “I’d hold the prefix in a 64-bit type.” It signals you know where the trap is even when your language hides it.

8 In the interview

How it gets asked

- “Given an array, answer q queries: the sum from index i to j .” — the direct form, LeetCode 303.
- “Design a class that returns range sums efficiently.” — the same thing wearing design clothes; the constructor/method split is the answer.
- “Find the index where the left sum equals the right sum.” — LeetCode 724, the running-total form.
- And as a **component**: “as a first step I’ll build a prefix array” inside harder problems — tomorrow’s subarray-sum family, [day 039](#)’s updates, [day 040](#)’s matrices. From today, prefix sums are vocabulary.

What to say out loud, in the first ninety seconds

1. **Spot the tell and say it.** “Many queries, and the array doesn’t change — that’s a precompute-once shape. I’ll trade $O(n)$ memory for $O(1)$ per query.”
2. **Name the object.** “I’ll build a prefix array: `prefix[i]` is the sum of the first i elements — $n + 1$ entries, with a 0 sentinel at the front so queries touching index 0 need no special case.”
3. **Give the formula as a sentence.** “A range sum is: after the end minus before the start — `prefix[j + 1]` minus `prefix[i]`, both ends inclusive.”
4. **Confirm the contract.** “Are both ends inclusive? And can queries repeat / can the array change between them?”

5. **State the costs, both sides.** “ $O(n)$ build and $O(n)$ extra space, then $O(1)$ per query — for a hundred thousand queries that’s two hundred thousand operations against five billion by looping.”

The follow-ups

“**What if the array can be updated between queries?**” Then the prefix array’s contract is broken — changing one element invalidates every reading to its right, so keeping it current costs $O(n)$ per update, and I would say so before proposing anything. The decision is a ratio: with very rare updates and floods of queries, rebuild on update and keep $O(1)$ reads. With mixed traffic, the standard tools are a Fenwick tree or a segment tree — both keep running totals in a tree shape so that update and query are each $O(\log n)$. I would name them, say the trade — $O(1)$ queries with unpayable updates, against $O(\log n)$ for both — and ask which the problem wants before coding either, since they are a step up in implementation weight and this course meets trees properly later.

“**Why the extra zero at the front? Can’t you handle $i = 0$ with an if?**” You can — `return prefix[j] if i == 0 else prefix[j] - prefix[i - 1]` — and it is strictly worse in three small ways. It doubles the surface for off-by-one mistakes, because now two formulas must each be right instead of one. It hides the meaning: with the sentinel, prefix has one entry per *boundary*, and a range sum is always after-the-end minus before-the-start, one rule with no exceptions. And the failure mode of getting it wrong is vicious: without the guard, `prefix[i - 1]` at $i = 0$ is `prefix[-1]` — Python reads the *last* element, silently — so the bug ships instead of crashing. A sentinel that removes a special case is nearly always worth one extra slot; this is the cleanest example of that principle I know, and it is why I write `initial=0` reflexively.

“**The queries arrive faster than you can precompute — say, the first query comes immediately.**” Build lazily or build anyway — and mostly build anyway, because the numbers say so: the build is one pass, and at a hundred thousand elements it is a fraction of a millisecond, cheaper than a single looped query over a long stretch. If the input is truly enormous and the first response matters, I can answer the first query with a direct loop while building, or build incrementally — extend the readings only as far as the furthest index any query has touched, since a query needs only prefix values up to $j + 1$. That keeps worst-case work the same and front-loads nothing. The deeper point I would make: this is an amortisation question — the build cost divided by the query count is the real per-query price, and it falls toward zero as queries arrive.

A model answer

“Many range-sum queries on an array that doesn’t change — so I’ll precompute. One $O(n)$ pass builds a prefix array where `prefix[i]` is the sum of the first i elements; it has $n + 1$ entries because I include a 0 at the front, the sum of nothing, so that ranges starting at index 0 fall out of the same formula as everything else.

Then any query is one subtraction: the sum from i to j inclusive is `prefix[j + 1]` minus `prefix[i]` — everything up to and including the end, minus everything before the start.

```
class NumArray:
    def __init__(self, nums):
        self.prefix = list(accumulate(nums, initial=0))
    def sumRange(self, left, right):
        return self.prefix[right + 1] - self.prefix[left]
```

Costs on both sides of the trade: $O(n)$ once to build, $O(n)$ extra space held for the object’s lifetime, and $O(1)$ per query after that. For a hundred thousand queries on a hundred thousand elements that’s about two hundred thousand operations, against five billion if each query loops.

Two things I’d flag. Negative values are completely fine — the subtraction never cared about sign, which matters because sliding windows can’t say that. And the deal requires the array to hold still: one update invalidates every reading to its right. If updates arrive, I’d say the options out loud — rebuild if they’re rare, or a Fenwick or segment tree for $O(\log n)$ both ways — rather than quietly patching a structure whose contract has changed.”

9 Recall card

- **prefix[i]** = **sum of the first i elements** — $n + 1$ entries, one per *boundary*, sentinel 0 in front. `accumulate(nums, initial=0)`.
- **Range sum** = **after the end minus before the start**: `prefix[j + 1] - prefix[i]` for inclusive `[i, j]`. Say the sentence, not the indices.
- **The trade**: **$O(n)$ build and space buys $O(1)$ per query** — worth it when queries are many and the array holds still; one update invalidates everything to its right.
- **Without the sentinel**: ranges from 0 read `prefix[-1]` — the last element, silently. The sentinel and the formula are a set; adopt both or neither.
- **Negatives are fine here** — which windows could not say. Tomorrow: prefix + hash map counts subarrays summing to k , negatives included.

1 What this is, and why they ask it

A **key-value store** is the simplest database there is: give it a key and a value, and later give it the key again to get the value back. `PUT`, `GET`, `DELETE` — that is close to the whole interface. The value is opaque to the store; it does not look inside, cannot search inside, and does not care what is inside. In exchange for promising so little, it is the fastest and most scalable member of yesterday's four families.

Interviewers reach for this constantly because the two flagship products — **Redis** and **DynamoDB** — appear in almost every real system design, and because the question has a sharp edge: candidates who can say what these stores *do* are common, and candidates who can say what they *cannot do* — and therefore where they sit in an architecture, always next to something else — are the ones who sound like they have operated one. Sessions, carts, counters and rate limiters are the standard interview uses, and every one of them will appear in the high-level design phase of this course.

2 The story

The cloakroom at the city railway station is a long room of steel racks behind one counter, and Bhaskar has run it for nine years. You hand your bag across; he puts it in the next empty slot, hands you a brass disc with a number stamped on it, and writes nothing at all.

Give him disc 217 this evening and your bag is in your hand in about ten seconds. It does not matter whether the room holds forty bags or four hundred — the number takes him straight to one slot on one rack. He never searches. That is the whole design of his room: the disc is everything, and the disc is enough.

The system shows its other face maybe once a week. A man arrives with no disc — lost it on the platform — and says: it is a black bag, with a red strap, and there is a tiffin box inside. And Bhaskar's ten seconds become forty minutes, because the racks know nothing about black or red or tiffin boxes. Slot by slot, rack by rack, opening nothing, comparing every bag against a description. The room was never arranged for questions. It was arranged for discs.

His nephew, who studies commerce and thinks about these things, once asked why he does not keep a register — every bag, its colour, its owner's name, the slot number. Bhaskar laughed. Four hundred bags a day come across that counter in the morning rush, he

said, sometimes three in a minute. The register would make every drop-off slower, all day, every day — to speed up one lost-disc search a week. The queue is the business. The lost man can wait.

There is one more rule, printed on the wall: bags left beyond seven days are cleared out. The racks are big, but they are not infinite, and a slot holding a forgotten bag is a slot that earns nothing. Old things make way, automatically, without anyone deciding bag by bag.

3 The idea in plain English

Bhaskar's room is a key-value store. The disc is the **key**. The bag is the **value**. And every strength and weakness of Redis and DynamoDB is already on his racks.

The deal: everything by key, nothing by value

The disc number takes Bhaskar to one slot with no searching — that is the **hash lookup** from [day 021](#), grown up: the store computes, from the key alone, exactly where the value lives. One step, whether it holds a thousand entries or a billion. That is the promise: **GET by exact key, in constant time, at any size**.

The lost-disc man is the other half. “The black bag with the red strap” is a query by *contents*, and the racks have no answer except a full scan. A key-value store is exactly this: no **WHERE** clause, no secondary questions, no “all sessions belonging to user 7” — unless you planned for that question and stored *another key* for it. The store never looks inside the bag.

And the nephew's register is the relational alternative, priced honestly: richer questions, paid for on every single write, all day — when the business is the queue, the register loses.

The seven-day rule: TTL

Bhaskar's clear-out is **time-to-live**: attach an expiry to a key and the store deletes it, automatically. This is why key-value stores own the *ephemeral* data of a system — sessions that should die after 30 days of silence, rate-limiter counters that reset every minute ([day 023](#)'s buckets live in exactly this kind of store), links that expire. In a relational database, expiry is a cleanup job you write and babysit; here it is a property of the data.

Where it sits in a system

Because it answers only one question, a key-value store is almost never the *only* database — it is the fast shelf **next to** the system of record. The pattern to internalise: Postgres holds the truth, and the key-value store holds the copies you need at speed — the session

blob checked on *every* request, the rendered fragment, the counter. The next lesson layers document stores on the same skeleton; the “which store is the truth?” discipline from [day 029](#) — every copy needs an owner — applies from today onward.

4 The picture

The two operations, and the one that does not exist:

PUT "sess:9f2a" → {user: 7, cart: [...]}
|
v

GET "sess:9f2a"
|
v

+-----+
| hash("sess:9f2a") → slot 8,113,402 |
| slot 8,113,402: {user: 7, cart: [...]} [ttl: 29d 23h] |
+-----+

one computed step, ~same cost at any size

"find every session belonging to user 7"
|
v
X no such operation – the store never looks inside values.
either scan everything, or you stored a second key for it:
"user:7:sessions" → ["sess:9f2a", "sess:c41b"]

What to notice: the second key at the bottom is the whole modelling discipline of key-value stores — every question you will ask must be *a key you wrote*. Questions are designed in advance, Farida’s cart from [day 036](#), taken to its extreme.

Where the store sits in a real request:

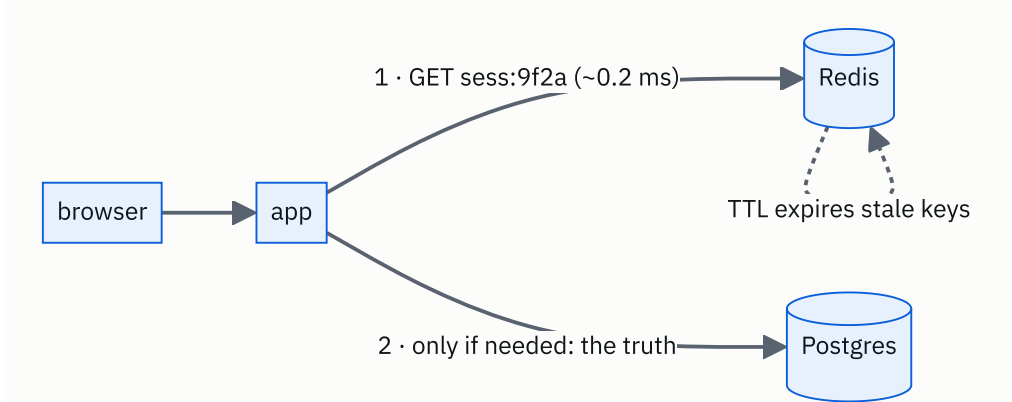


FIGURE 37.1

What to notice: the key-value store is in front of the relational database, not instead of it — it absorbs the read that happens on *every* request so that Postgres only sees the requests that need the truth.

5 How it actually works

Redis: the shelf in memory

Redis keeps everything in **RAM**, which from [day 009](#)'s speed hierarchy is the whole explanation of its numbers: reads and writes in well under a millisecond, over a hundred thousand operations a second on one node. One process, one event loop — commands execute one at a time, which is why `INCR` is atomic without locks: there is no concurrency inside to defend against. That single-threaded simplicity is a feature you can cite.

Its values are not quite opaque — Redis gives them *shapes*: strings, hashes (field → value maps), lists, sets, and sorted sets. Each shape has commands: `INCR` for counters, `LPUSH` / `RPOP` for simple queues, `ZADD` / `ZRANGE` for leaderboards. The discipline stands — you still fetch by key — but the value behind the key has structure the server can update in place.

Memory is the constraint that runs the design. You set `maxmemory` and an **eviction policy** — usually `allkeys-lru`: when full, quietly drop the least-recently-used keys. For a cache that is correct behaviour; for anything you cannot lose it is a catastrophe, which is why the next paragraph exists.

Durability is optional, and you must say so. Redis defaults to periodic snapshots (RDB): a crash loses everything since the last snapshot — minutes. Turn on the append-only file (AOF) with `everysec` and a crash loses at most about a second — [day 033](#)'s `synchronous_commit = off` trade, in different clothing. Full-safety `fsync-per-write` exists and costs Redis its speed. The interview sentence: **Redis is a fast shelf, not a ledger — nothing lives *only* in Redis unless losing it is acceptable.**

DynamoDB: the shelf across a thousand machines

DynamoDB is Amazon's managed key-value store, and its answer to “what if the data outgrows RAM, or one machine?” is [day 036](#)'s horizontal argument made real: the key is **hashed to choose a partition**, partitions spread across fleets of machines, and every request touches the one machine its key names. Single-digit-millisecond reads and writes at essentially any scale — the store behind Amazon's own cart.

Its model is one notch richer than pure key-value: a **partition key** chooses the machine, and an optional **sort key** orders items within the partition — so “all items for order 7, in order” is one cheap request. (That partition-plus-sort shape is exactly where

day 039's wide-column stores begin.) You pay per request — capacity units — and it is durable by default, replicated across data centres: unlike Redis, it is a ledger, priced like one.

What neither will ever do

No joins. No `WHERE colour = 'black'` (DynamoDB can bolt on secondary indexes — each one is literally a second copy of the table keyed differently, maintained on every write; Bhaskar's register, priced per write). No multi-key transactions worth leaning on: Redis's `MULTI` batches commands without rollback, DynamoDB's transactions cap at 100 items — both from day 033, both fine for small invariants, neither a home for the bank transfer. The design consequence is always the same: **the questions you will ask are keys you must write.**

6 The numbers

Latency: why the shelf goes in front

```
Postgres, warm, simple indexed read      : ~0.5 - 2 ms
Redis GET, same data centre               : ~0.1 - 0.3 ms
DynamoDB GetItem                         : ~1 - 5 ms

session check on EVERY request, 20,000 requests/s:
  on Postgres: 20,000 extra queries/s on the system of record
  on Redis:    absorbed by one node at ~15% of its capacity
```

The win is usually not the milliseconds — it is **taking the every-request read off the database that holds the truth**, so its capacity serves the requests that need it.

Memory: what fits on the shelf

```
50 million active sessions × ~1 KB each = 50 GB
  → one large Redis node, or a small cluster. Sessions fit.

product catalogue, 200 million items × 4 KB = 800 GB
  → not a RAM problem you want. That is DynamoDB or Postgres territory.
```

The multiplication to always run: **count** × **size**, against “RAM is hundreds of gigabytes, disk is effectively unlimited”. It decides Redis-versus-DynamoDB in one line.

Throughput: one node's honest ceiling

```
Redis, one node: ~100,000+ simple ops/s
a like-counter at 5,000 INCR/s: 5% of one node – trivial, and atomic for free
(the same counter as a Postgres row: 5,000 locked writes/s on one hot row
– day 035's queue arithmetic says that hurts)
```

Cost shape: DynamoDB charges per question

```
provisioned at 10,000 reads/s, item under 4 KB:
10,000 read units × ~$0.00013/unit-hour × 730 h ≈ $950/month
(order of magnitude – prices move; the SHAPE is the lesson:
you pay per request, so chatty access patterns cost linearly)
```

A relational node costs the same whether you ask it one question or a thousand; DynamoDB bills the thousand. Designs that read one item per request love it; designs that fan out to fifty keys per page view discover the bill.

7 The trade-offs

What you give up

Everything yesterday's frame predicted, at maximum strength: no ad-hoc questions (every future question must already be a key), no joins, transactions in miniature only, and — in Redis — a durability dial that defaults toward speed. Plus one new cost with no relational equivalent: **modelling by access path**. Renaming a concept, adding a question, changing a page's shape — each is a re-keying exercise, and the migration touches every entry.

When it wins

Data that is **fetches whole, by identity, constantly**: sessions, carts, profiles-by-id, feature flags, rendered fragments. Data that **expires**: anything with a natural lifetime, where TTL replaces a cleanup job. Data that **counts**: rate limiters, likes, view counters — atomic increments with no lock queue. And at DynamoDB's end: key-shaped data whose *scale* has outgrown a node but whose questions never outgrew the key.

I would not use it if...

I would not put the system of record in Redis — eviction and default persistence make it a shelf, not a ledger. **I would not choose a key-value store when the questions are still being discovered** — that is the relational arrangement's home ground, as [day 036](#) said. **I would not use DynamoDB for a fan-out-heavy read pattern** — fifty keys per page at per-request pricing is a bill, and a relational join or a document aggregate serves it better. **And I would not add a key-value layer at all** until a measured hot path exists — a shelf in front of an idle counter is pure moving parts.

The honest sentence

A key-value store is the fastest database because it answers the easiest question. The design work is making sure the questions your system asks really are that easy — and keeping an owner, somewhere slower and safer, for every fact the shelf holds.

8 In the interview

How it gets asked

- “When would you use a key-value store instead of a relational database?” — the hub question; they want uses *and* refusals.
- “Where do sessions live in your design?” — the applied version, in nearly every system design round.
- “Redis or DynamoDB?” — RAM shelf against partitioned ledger; the answer is the data’s size, lifetime, and durability needs.
- “How would you build a rate limiter / like counter?” — atomic `INCR` with a TTL, and the hot-row contrast with a relational counter.
- “What happens if Redis goes down?” — the durability dial, and whether your design treated it as a shelf or accidentally as the ledger.

What to say out loud, in the first ninety seconds

1. **Define by the promise.** “A key-value store answers one question — GET by exact key — in constant time at any size. It never looks inside the value.”
2. **Name the two flagships and their difference.** “Redis: in-memory, sub-millisecond, durability optional — a shelf. DynamoDB: partitioned by hashed key across machines, durable, per-request pricing — a key-shaped ledger at any scale.”
3. **Give the uses in one breath.** “Sessions, carts, counters, rate limiters, feature flags — fetched whole, by identity, on every request, often with a TTL.”
4. **Say what it cannot do, unprompted.** “No queries by value, no joins, no real cross-key transactions — every question must be a key I wrote in advance.”
5. **Place it in the architecture.** “So it sits next to the system of record, not instead of it: Postgres holds the truth, Redis absorbs the every-request reads.”

The follow-ups

“Sessions on every request — Redis or Postgres? Defend it.” Redis, and the defence is arithmetic plus a failure story. Arithmetic: the session read happens on every request — at 20,000 requests a second that is 20,000 reads that carry no business value individu-

ally, and putting them on Postgres spends the system of record's capacity on its least interesting question; one Redis node absorbs them at a fraction of capacity, at a fifth of the latency, with TTL giving me session expiry for free instead of a cleanup job. The failure story is what makes it safe: a session is *re-creatable* — if Redis dies, users log in again; annoying, not corrupting. That is precisely the class of data a shelf may own. What I would not do is quietly promote that Redis to holding anything non-re-creatable — a cart that is the only copy, money-ish state — because then eviction and snapshot-gap loss become data loss, and the right home was the ledger with a cache in front, not the cache alone.

“How do you count likes at five thousand a second?” `INCR likes:post:812` in Redis — one atomic command, no read-modify-write cycle, no lock, about five percent of one node's throughput. The relational contrast is worth saying: the same counter as a Postgres row is five thousand exclusive locks a second on one hot row — [day 035](#)'s queue arithmetic — which is a serial bottleneck. Then the durability caveat, unprompted: Redis's default persistence can lose the last seconds of increments in a crash, so if the count matters commercially I flush it to Postgres on an interval — Redis absorbs the write storm, Postgres owns the durable total, and the flush job is the reconciliation [day 029](#) demands of every copy. If the count is merely decorative, Redis alone with AOF-everysec is a fine and honest answer.

“You said no queries by value. So how does anyone find anything?” By writing the question down as a key at write time. If sessions must be findable by user, then alongside `sess:9f2a` I maintain `user:7:sessions` — a set of session ids — and update both together; the read is then two GETs, not a search. That is the general method: every access path gets its own key shape, chosen when the data is written, which is Farida's cart from [day 036](#) taken to the limit. DynamoDB industrialises exactly this as global secondary indexes — each one is a second copy of the table, keyed by a different attribute, maintained automatically on every write and billed accordingly, which tells you what it really is: Bhaskar's register, priced per drop-off. And when the questions are genuinely open-ended — “all black bags with red straps” — the honest answer is that this data's questions outgrew the key, and it belongs in the relational store or a search system, not behind a cleverer key scheme.

“A key-value store gives me one operation done perfectly: GET by exact key, constant time at any size, because the key alone determines where the value lives. It never looks inside the value — so no queries by contents, no joins, and only miniature transactions. Every question I’ll ever ask has to be a key I wrote in advance.

That makes it the right home for data that’s fetched whole, by identity, extremely often: sessions, carts, counters, rate-limit buckets, feature flags. Two products cover the space. Redis is the in-memory shelf — sub-millisecond, atomic increments, TTL expiry built in, and a durability dial that defaults toward speed, so nothing irreplaceable lives only there. DynamoDB is the same idea made durable and partitioned — the key is hashed to a partition across a fleet, so it’s single-digit milliseconds at any scale, priced per request.

Where it sits: next to the relational database, not instead of it. Postgres stays the system of record; Redis absorbs the read that happens on every request. The concrete win is capacity, not just latency — at twenty thousand requests a second, moving the session check to Redis takes twenty thousand queries a second off the store that holds the truth.

And the refusals: I wouldn’t put the only copy of anything important in Redis; I wouldn’t pick a key-value store while the product’s questions are still changing weekly; and I wouldn’t reach for one at all until a measured hot path justifies the extra moving part. The sizing check is one multiplication — count times size: fifty million sessions at a kilobyte is fifty gigabytes, which fits the shelf. Eight hundred gigabytes of catalogue does not, and that’s when I’m talking about DynamoDB or staying relational.”

9 Recall card

- **One promise: GET by exact key, $O(1)$, any size — and it never looks inside the value.** Every future question must be a key you wrote at write time.
- **Redis = RAM shelf:** sub-ms, atomic `INCR`, TTL, single-threaded; durability optional (RDB gap / AOF ≈ 1 s) — nothing irreplaceable lives only there.
- **DynamoDB = durable, partitioned by hashed key** (+ optional sort key), single-digit ms at any scale, **priced per request** — chatty designs pay linearly.
- **Homes: sessions, carts, counters, rate limiters, flags** — fetched whole, by identity, often expiring. Sizing: count $\times$ size against RAM.

- **It sits beside the system of record, not instead of it** — Postgres owns the truth, the shelf absorbs every-request reads, and every copy has an owner and a reconciliation path.

PRACTICE

Practice — Prefix sums: answering range queries instantly

DSA topic: Prefix sums: answering range queries instantly **System design topic:** Key-value stores

Code these, in this order

Four problems on one idea. **Before each, say the formula as a sentence** — *after the end, minus before the start* — and say where the sentinel is.

Before each one, ask:

1. One query or many? Is the precompute actually earning its keep here?
2. Do I need the stored array, or just a running total?
3. Both ends inclusive, or half-open?
4. Can the values be negative — and does anything in my plan care? (It should not.)

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Running Sum of 1d Array	LeetCode 1480 (Easy)	The build loop itself — <code>prefix[i + 1] = prefix[i] + nums[i]</code> with nothing else in the way.
2	Range Sum Query — Immutable	LeetCode 303 (Easy)	The sentinel and the formula as a set; the constructor/method split.
3	Find Pivot Index	LeetCode 724 (Easy)	The running-total form — no stored array, check before adding, empty left side at index 0.
4	Left and Right Sum Differences	LeetCode 2574 (Easy)	Two directions of the same idea; whether your boundaries stay honest when one runs backwards.

On problem 2, break it both ways

Build the prefix without `initial=0` and query `(0, 3)` using `prefix[j] - prefix[i - 1]`. You get a silently wrong number — say which entry `prefix[-1]` actually read. Then keep the zero-less array but use the lesson's formula on the last index, and collect the `IndexError: list index out of range`. Finish the sentence: *the sentinel and the formula are...*

On problem 3, honour the empty side

Run `[2, 1, -1]` and explain, out loud, why the answer is 0 — what is the sum of nothing, and why does checking before adding get it right with no special case?

On problem 2 again, the follow-up rehearsal

Say the two-minute answer to “*now the array can be updated between queries*”: the contract that broke, the rebuild-if-rare option, and the two tree names with their $O(\log n)$ — without pretending to implement them.

The formula drill

For `nums = [4, 7, 3, 6, 2, 5, 1]`, `prefix = [0, 4, 11, 14, 20, 22, 27, 28]`, answer from the readings alone, under five seconds each, out loud:

1. Sum of `nums[2..4]` .
2. Sum of `nums[0..6]` .
3. Sum of `nums[3..3]` .
4. Sum of `nums[0..2]` .
5. Which two readings answer “sum of the last three elements”?

The tell drill

For each, say whether prefix sums are the right tool, and why in one sentence:

1. One range-sum query on a ten-element array.
2. A hundred thousand range-sum queries on a fixed array.
3. Range sums where the array takes an update every few queries.
4. Longest subarray with sum at most k , all values positive.
5. Count subarrays summing to exactly k , negatives allowed.

Number 4 is yesterday’s tool; number 5 is tomorrow’s. Saying which is which is the drill.

The shelf-or-ledger drill

For each piece of data, say Redis, DynamoDB, or Postgres — and the one-line reason:

1. Session blobs, 30-day expiry, read on every request.
2. Order history, seven years, auditors visit.
3. A like counter at 5,000 increments a second.
4. Cart items at Amazon scale, fetched by cart id.
5. Product search by colour, size and price.

The refusal drill

Answer each in one or two sentences, out loud:

1. Why does “find every session for user 7” not exist in a key-value store — and what do you write at write time to make it exist?

2. What can Redis lose in a crash at its default settings, and which day's Postgres setting is the same trade?
3. Why is a Redis `INCR` counter free of the hot-row queue that the same counter has in Postgres?
4. What makes DynamoDB's pricing punish a fifty-key page view?

The arithmetic drill

From memory, in under two minutes:

- 100,000 queries $\times$ average stretch 50,000 by loop, against build-plus-subtract. The two totals and the ratio.
 - 50 million sessions $\times$ 1 KB — where does it fit? 200 million items $\times$ 4 KB — where does that fit?
 - A session check on every one of 20,000 requests/s — what does moving it to Redis take off Postgres?
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Answer many range-sum queries on a fixed array.* The precompute trade named on both sides, the sentinel and why, the formula as a sentence, and the follow-up about updates handled with the two tree names.
 2. *When would you use a key-value store instead of a relational database?* The one promise and the refusals, Redis against DynamoDB in one breath each, the four standard uses, and where it sits — beside the system of record, never instead of it.
 3. *Why does the prefix array have one more entry than the array?* Boundaries, not elements. What `prefix[0]` means, which queries need it, and what Python does — silently — when it is missing and `i` is 0.
-

Before you move on

- [] I can build the prefix array with the sentinel and say what each entry means.
- [] I say “after the end, minus before the start” instead of reciting indices.
- [] I know both failure modes of the missing sentinel — the silent one and the crash.
- [] I can name when the precompute loses: single queries, and updates between queries.

- [] I can say what Redis and DynamoDB each promise, and the questions neither can answer.
- [] I know which data may live only on the shelf, and which needs a ledger behind it.
- [] I answered all three questions above out loud.

Subarray sum equals K: prefix plus hash map

DATA STRUCTURES AND ALGORITHMS

Subarray sum equals K: prefix plus hash map

You can count subarrays with a given sum in one pass, including the negative-number case.

SYSTEM DESIGN

Document databases

You can model a domain as documents and say where the model breaks.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Count the subarrays whose sum equals k .*
- *Model a blog in MongoDB. What happens when a user changes their name?*

Subarray sum equals K: prefix plus hash map

1 What this is, and why they ask it

Count the subarrays that sum to exactly k , in one pass, on an array that may contain negative numbers. The tool is yesterday's running total plus [day 021](#)'s frequency map: walk the array carrying the prefix sum, and at each step ask the map *how many earlier prefixes equal this one minus k* — because each such earlier moment marks the start of a subarray ending here with sum exactly k .

This is LeetCode 560, one of the most-asked mediums at product companies, and it is asked as a **trap with a pedigree**. The problem looks exactly like the window problems of days 32 to 34 — and windows are illegal here, because negatives destroy monotonicity, the debt [day 032](#) recorded and [day 034](#) renewed. Interviewers watch for one specific moment: do you ask about negative numbers before choosing the tool? Today pays that debt, and the same three-line pattern then solves a whole family — equal 0s and 1s, sums divisible by k , and their cousins.

2 The story

Deepa's bank app shows two columns: each credit or debit, and next to it the balance after it. She scrolls it the way other people scroll photos — month-end discipline, she calls it.

Her brother started it, with an accusation. Somewhere in March, he said, that badminton kit plus the shoes plus whatever else — you dropped exactly five thousand in one stretch, I am sure of it. Find the stretch.

The obvious way is miserable. Pick a starting entry, add every amount from there forward, see if any stopping point gives five thousand, move the start one down, do it again. Forty entries in March. She tried it for two starts and gave up.

Then she stopped looking at the amounts column and looked only at the balances. If some stretch of entries cost exactly five thousand net, then the balance at the end of that stretch is exactly five thousand less than the balance just before it began. That is not clever, she thought — that is just what a balance is.

So the question flips. Standing at any entry, balance showing 41,200, she does not add anything. She asks: **was there an earlier moment when the balance showed 46,200?** If yes — there is her stretch, ending right here. If the balance showed 46,200 at *three* earlier moments, that is three different stretches, all ending here, all costing exactly five thousand.

And the three-moments case is real, not a technicality — her balance climbs on payday and falls all month, so the same figure comes around again and again. Money goes up and down; that is exactly why the same balance repeats, and exactly why she counts moments rather than remembering one.

One scroll, top to bottom. At each entry she asks her one question about the balances she has already passed, keeps a rough tally of how often each balance has appeared, and moves on. By the bottom of March she had found it — two stretches, actually, which cost her brother an apology and her a coffee.

The first moment she almost missed: a stretch that starts at the very top of the month. Its “balance just before” is the balance March *opened* with — a moment with no entry attached. She had to remember the opening balance counted too.

3 The idea in plain English

Deepa’s balance column is yesterday’s prefix sum. Her one question — *how often has balance-minus-5000 appeared before?* — is the whole algorithm. And her opening balance is the sentinel again, wearing new clothes.

The flip

Yesterday established: sum of `nums[i..j]` = `prefix[j + 1] - prefix[i]` — after the end minus before the start. So a subarray ending at position `j` sums to `k` exactly when

```
prefix_before_its_start = prefix_here - k
```

Walk left to right carrying `running`, the prefix so far. At each element, the subarrays that end here and sum to `k` correspond **one for one** with the earlier moments whose prefix equalled `running - k`. Do not search for them — that is Deepa adding from every start. **Count them as you go:** keep a map from prefix value to how many times it has occurred, ask the map, then record the current prefix.

```
count += seen[running - k]    # how many valid starts end a subarray here
seen[running] += 1           # this moment is a future "earlier moment"
```


The order matters and §7 has the input that punishes getting it wrong: ask first, record after, or a subarray of length zero sneaks in when `k = 0`.

The opening balance: `seen[0] = 1`

A subarray that starts at index 0 has “the prefix before its start” equal to 0 — the sum of nothing, yesterday’s sentinel. That moment happened — before the walk began — so the map starts as `{0: 1}`, not empty. Forget it and every subarray touching the left edge goes uncounted: on `[1, 2, 3]` with `k = 3` you get 1 instead of 2, silently missing `[1, 2]`. This is the single most common bug in this problem, and it is Deepa’s opening balance, exactly.

Why this beats the window — and when the window is still better

Negatives are no problem here, because nothing is assumed about direction: the map does not care whether prefixes climb or fall, it only counts what appeared, and Deepa’s balance falling and rising is why the same value repeats and why occurrences are *counted*, not just remembered.

But say the comparison properly, because it is a favourite follow-up. With all-positive values and a question like “longest at most k ”, the window is still the better tool — $O(1)$ space against the map’s $O(n)$. The map earns its space exactly when **negatives** are possible or the question is **exactly** — the two places monotonicity fails. This is [day 035’s](#) routing question 4 getting its full answer.

The same trick in costumes

Two transformations turn famous problems into this one:

Contiguous Array — LeetCode 525: longest subarray with equal 0s and 1s. Recast each 0 as -1 ; now “equal 0s and 1s” means **sum zero**. Because it asks *longest*, not *how many*, the map stores each prefix’s **first** index instead of a count — the earliest occurrence gives the longest stretch — and is never overwritten.

Subarrays Divisible by K — LeetCode 974: count subarrays whose sum is divisible by `k`. A stretch is divisible by `k` when the prefixes at its two ends leave the **same remainder** — so carry `running % k` and count matching remainders. Python’s `%` always returns a non-negative remainder for positive `k` (`-7 % 5 = 3`), so negatives cost nothing here; in Java or C++ the remainder keeps the sign and needs `(r % k + k) % k` — worth one sentence if the interview is not in Python.

One pattern, three problems: **carry a running quantity, and ask a map about the past**. What varies is the quantity (sum, recast sum, remainder) and what the map stores (count, or first index).

4 The picture

The one-pass count on `[3, 4, 7, 2, -3, 1, 4, 2]`, `k = 7` — answer 4:

index	0	1	2	3	4	5	6	7
nums	3	4	7	2	-3	1	4	2
step	running	ask	seen[running-7]	found		seen afterwards		
(start)								{0:1}
0	3	seen[-4] = 0						{0:1, 3:1}
1	7	seen[0] = 1	[3,4]	1st				{0:1, 3:1, 7:1}
2	14	seen[7] = 1	[7]	2nd				{..., 14:1}
3	16	seen[9] = 0						{..., 16:1}
4	13	seen[6] = 0						{..., 13:1}
5	14	seen[7] = 1	[2,-3,1,4]...	no: [7,2,-3,1]	3rd			{..., 14:2}
6	18	seen[11] = 0						{..., 18:1}
7	20	seen[13] = 1	[-3,1,4,2]...	no: [1,4,2]...	4th			{..., 20:1}

What to notice, three things. The very first hit uses the `{0: 1}` sentinel — subarray `[3, 4]` starts at the left edge. At step 5, `running` returns to 14 — the negative at index 4 pulled it back down — and the map now holds `14: 2`: revisited values are why occurrences are counted. And the negative number caused no special handling anywhere.

Same balance, two moments — the picture of why counting matters:

```
running: 3    7    14    16    13    14    18    20
           ^      ^
           +--- same value ---+
           the stretch between sums to 0 -
           and BOTH moments serve as valid starts
           for later subarrays summing to k
```

What to notice: a repeated prefix value means some stretch in between summed to zero — only possible with negatives, and exactly the case windows cannot navigate.

5 The code, built step by step

The core

```
seen: defaultdict[int, int] = defaultdict(int)
seen[0] = 1                      # the opening balance: prefix before index 0
running = 0
count = 0
```

Four names. `seen[0] = 1` is the line that separates this from a wrong answer on day one.

```

for x in nums:
    running += x
    count += seen[running - k]      # ask about the past first
    seen[running] += 1             # then file the present

```

Three lines of loop. `seen[running - k]` on a `defaultdict` returns 0 for never-seen values — harmless here, and the read-creates-key quirk from [day 021](#) does not bite because inserting `running - k` with count 0 never changes a future answer. (With a plain dict you would write `seen.get(running - k, 0)` — §7 shows the crash if you forget.)

Contiguous array: first index, not count

```

first: dict[int, int] = {0: -1}      # sum 0 exists before index 0
running = 0
best = 0
for i, x in enumerate(nums):
    running += 1 if x == 1 else -1
    if running in first:
        best = max(best, i - first[running])
    else:
        first[running] = i           # only the FIRST occurrence – never overwrite

```

Two changes, both forced by “longest”: the map stores the earliest index of each prefix value (`{0: -1}` is the sentinel in index form — the moment before index 0), and an existing entry is **never overwritten**, because a later duplicate could only shorten the stretch. §7 shows the overwriting version producing a confident wrong answer.

Divisible by k: remainders

```

seen: defaultdict[int, int] = defaultdict(int)
seen[0] = 1
running = 0
count = 0
for x in nums:
    running = (running + x) % k
    count += seen[running]          # same remainder = divisible stretch between
    seen[running] += 1

```

The ask changes from `seen[running - k]` to `seen[running]` — equal remainders, rather than a fixed difference. Everything else is the same machine.

The complete solutions

```
from collections import defaultdict

def subarray_sum(nums: list[int], k: int) → int:
    """LeetCode 560. Count subarrays summing to k; negatives welcome."""
    seen: defaultdict[int, int] = defaultdict(int)
    seen[0] = 1 # prefix 0 exists before the walk starts
    running = 0
    count = 0
    for x in nums:
        running += x
        count += seen[running - k] # ask first...
        seen[running] += 1 # ...record after (k = 0 depends on this)
    return count

def find_max_length(nums: list[int]) → int:
    """LeetCode 525. Longest subarray with equal 0s and 1s: recast 0 as -1,
    then longest stretch with sum 0 – map stores FIRST index of each prefix."""
    first: dict[int, int] = {0: -1}
    running = 0
    best = 0
    for i, x in enumerate(nums):
        running += 1 if x == 1 else -1
        if running in first:
            best = max(best, i - first[running])
        else:
            first[running] = i # never overwrite: earliest = longest
    return best

def subarrays_div_by_k(nums: list[int], k: int) → int:
    """LeetCode 974. Same remainder at both ends = divisible stretch between.
    Python's % is already non-negative for k > 0."""
    seen: defaultdict[int, int] = defaultdict(int)
    seen[0] = 1
    running = 0
    count = 0
    for x in nums:
        running = (running + x) % k
        count += seen[running]
        seen[running] += 1
    return count

if __name__ == "__main__":
    print(subarray_sum([1, 1, 1], 2)) # 2
    print(subarray_sum([1, 2, 3], 3)) # 2
    print(subarray_sum([1, -1, 1, 1], 2)) # 2 – needs the negatives
    print(subarray_sum([-1, -1, 1], 0)) # 1
    print(subarray_sum([3, 4, 7, 2, -3, 1, 4, 2], 7)) # 4 – the walk in §4

    print(find_max_length([0, 1])) # 2
    print(find_max_length([0, 1, 0])) # 2
    print(find_max_length([0, 0, 1, 0, 0, 1, 1])) # 6

    print(subarrays_div_by_k([4, 5, 0, -2, -3, 1], 5)) # 7
    print(subarrays_div_by_k([5], 9)) # 0
```

6 What it costs

The count, out loud

One loop over n elements. Each iteration: one addition, one map read, one map write — all $O(1)$ on average, [day 021](#)’s deal. **$O(n)$ time**. The map holds at most one entry per distinct prefix value — up to $n + 1$ of them. **$O(n)$ extra space**.

Against the alternatives

$n = 20,000$, $k = 0$, worst case:

all starts × all ends, summing each stretch afresh	: $\sim n^3/6$	$\approx 1.3 \times 10^{12}$	ops
all starts × all ends, running sum per start	: $\sim n^2/2$	$\approx 2 \times 10^8$	ops
prefix + map	: $\sim 3n$	$\approx 6 \times 10^4$	ops

The $O(n^2)$ version is the one interviewers actually see written — it passes small tests and dies on large ones. Name it as the honest brute force, then leave it.

The space is the price of the negatives

Yesterday’s window solved its problems in $O(1)$ space; today costs $O(n)$. That is not carelessness — it is the exact price of dropping monotonicity. When values can go both ways, any earlier moment might matter, so some record of *all* of them must be kept. **Say the trade in the interview:** windows when monotonic and $O(1)$ space matters; prefix-plus-map when negatives or “exactly” appear, at $O(n)$ space.

The number to have ready

One pass, constant map work per element: $O(n)$ time, $O(n)$ space for the prefix counts. The running-sum brute force is $O(n^2)$ — two hundred million operations at twenty thousand elements against sixty thousand.

7 The traps

The near-miss: the missing opening balance

Start the map empty instead of `{0: 1}`:

```
seen = defaultdict(int)          # no seen[0] = 1
...
print(subarray_sum_noinit([1, 2, 3], 3))
```

1

The answer is 2: `[1, 2]` and `[3]`. The version without the sentinel finds only `[3]` — every subarray starting at index 0 needs an “earlier moment with prefix 0”, and that moment was never filed. Wrong by a little, on most inputs, with no error: the worst kind. `seen[0] = 1` is the first line after the imports, every time.

The real error: `get` forgotten on a plain dict

Swap the `defaultdict` for a plain dict and keep the bracket read:

```
seen = {0: 1}
...
count += seen[running - k]
```

```
Traceback (most recent call last):
  File "day38.py", line 8, in <module>
    count += seen[running - k]
              ~~~~~^~~~~~
KeyError: -1
```

First element 1, `k = 2`, and `seen[-1]` has never been filed. With a plain dict the read must be `seen.get(running - k, 0)`. Either spelling is fine in an interview — mixing them is not.

The near-miss: recording before asking

Swap the two loop lines and run `k = 0`:

```
seen[running] += 1          # record first – wrong order
count += seen[running - k]  # k = 0: this now counts the CURRENT moment
```

With `k = 0`, `seen[running - 0]` includes the entry you just filed — the empty subarray from this moment to itself — and every position overcounts by one. On `[-1, -1, 1]` with `k = 0` it answers 4 instead of 1. **Ask about the past, then file the present.** The order encodes “earlier moment”, and `k = 0` is the input that checks you meant it.

The near-miss: overwriting the first index in Contiguous Array

```
if running in first:
    best = max(best, i - first[running])
first[running] = i          # overwrites every time
```

On `[0, 0, 1, 0, 0, 0, 1, 1]` the true answer is 6; the overwriting version says 4. Each overwrite drags the stored index rightward, shortening every stretch measured against it. For *counting* problems you add occurrences; for *longest* problems you keep the *first* index untouched. Which map you are keeping — count or first-index — is a sentence to say before coding, because the two bodies differ by one line and answer different questions.

The misroute, formally retired

The window version of this problem — grow right, shrink left while the sum exceeds `k` — returns plausible answers on all-positive tests and wrong ones the moment a negative appears: shrinking can *raise* the sum, so the shrink loop discards subarrays it never examined. Day 032 showed the failure; today names the resolution. **“Can values be negative?” is the first question this problem statement should trigger** — asked aloud, before any tool is chosen. If no: windows remain legal and cheaper for at-most/longest forms. If yes, or if the word is “exactly”: today’s map.

The contract corner: `k = 0` and empty subarrays

Two clarifications worth fifteen seconds: do empty subarrays count (universally no, and the ask-then-record order enforces it), and can `k` be negative (yes — nothing in the machine cares, `seen[running - k]` is just a different lookup). Saying the second unprompted signals the machine is understood, not memorised.

8 In the interview

How it gets asked

- “Count the subarrays whose sum equals k .” — LeetCode 560, verbatim.
- “Longest subarray with equal numbers of 0s and 1s.” — LeetCode 525; the recast-and-sum-zero costume.
- “Count the subarrays whose sum is divisible by k .” — LeetCode 974; the remainder costume.
- “Is there a subarray summing to k ?” — the existence version: same machine, return on first hit.
- And the escalation path: you solve a window problem, and the interviewer adds “*now the values can be negative*” — the sentence that hands over from days 32–34 to today.

What to say out loud, in the first ninety seconds

1. **Ask the routing question.** “Can the values be negative?” — and say why: “negatives break the sliding window’s monotonicity, so they decide my tool.”

2. **Name the brute force honestly.** “Every start with a running sum is $O(n^2)$ — correct, and too slow at scale.”
3. **State the flip.** “A subarray ending here sums to k exactly when some earlier prefix equals the current prefix minus k — so I count earlier prefixes with a frequency map instead of re-summing.”
4. **Flag the two load-bearing details.** “The map starts as $\{0: 1\}$ — the prefix before the array — or I miss every subarray touching the left edge. And I ask the map before recording the current prefix, or $k = 0$ counts empty subarrays.”
5. **Give the costs, both sides.** “ $O(n)$ time, $O(n)$ space — the space is the price of negatives; with all-positive values and an at-most question I’d be back to an $O(1)$ -space window.”

The follow-ups

“**Why exactly does the sliding window fail here?**” The window’s shrink logic is licensed by monotonicity: growing the window must push the sum one way and shrinking must push it the other, so that “too big” is a state shrinking always repairs and never overshoots reversibly. With negatives, both directions are broken — extending the window can shrink the sum, and shrinking it can grow the sum — so when the window discards a start position, it has not *proven* anything about that start: a later extension might have brought the sum back to k . Concretely, on `[1, -1, 1, 1]` with $k = 2$, a window that shrinks past index 0 on seeing the sum reach 2 throws away the start of `[1, -1, 1, 1]`, which also sums to 2. The map version never discards: every prefix stays counted forever, which is exactly why it costs $O(n)$ space — the fair price of making no directional assumption at all.

“**Now make it the longest subarray with sum k , not the count.**” Same machine, different map contract. I still walk with the running prefix and still ask about `running - k`, but the map now stores the **first index** at which each prefix value occurred, because for length I want the match that is furthest left: `best = max(best, i - first[running - k])` when present. Two details change with it. The sentinel becomes `{0: -1}` — index form, the moment before the array — so a stretch from the very start measures correctly as `i - (-1) = i + 1`. And I never overwrite an existing entry: a later occurrence of the same prefix could only produce shorter stretches, so the earliest one is the only one worth keeping. That is exactly Contiguous Array — LeetCode 525 — after recasting 0s as -1 s; count-maps for “how many”, first-index-maps for “longest”, and saying which contract the map carries is the difference between the two problems.

“**What if the array is enormous — streaming, can’t hold it in memory?**” The algorithm is already one forward pass that never revisits elements, so it runs on a stream as-is: I keep the running sum and the map, and each arriving value costs $O(1)$. What grows is the map — one entry per distinct prefix value, unbounded in general. Whether that is a problem depends on structure I would ask about: if values are integers in a mod-

est range, prefix values are bounded and so is the map; for the divisible-by-k variant the map is at most k entries — $O(k)$, fully streaming-safe; for arbitrary values, an adversarial stream makes every prefix distinct and the map grows linearly, and no exact algorithm can avoid that, because any forgotten prefix could be the partner of a future element. So: time is stream-friendly, space is the honest constraint, and the divisible variant is the one that streams perfectly.

A model answer

“First question: can the values be negative? ...Yes — then a sliding window is off the table, because shrinking no longer reliably reduces the sum, and I’d be discarding starts I’ve proven nothing about. I’ll use prefix sums with a frequency map instead.

The idea: the sum of a subarray is the prefix at its end minus the prefix before its start. So a subarray ending at the current position sums to k exactly when some earlier prefix equals my current prefix minus k. Instead of searching for those, I count them as I walk — a map from prefix value to how many times it has occurred.

```
def subarray_sum(nums, k):
    seen = defaultdict(int)
    seen[0] = 1
    running = count = 0
    for x in nums:
        running += x
        count += seen[running - k]
        seen[running] += 1
    return count
```

Two lines carry the correctness. `seen[0] = 1` — the prefix before the array exists, or every subarray starting at index 0 goes missing; on `[1,2,3]` with `k = 3` the answer drops from 2 to 1. And I ask the map *before* recording the current prefix — otherwise `k = 0` counts each position’s empty subarray.

Cost: one pass, $O(1)$ map work per element — $O(n)$ time, $O(n)$ space. The space is the honest price of negatives: since sums can come back down, any earlier moment may matter, so all of them stay counted. With all-positive values and an at-most question I’d use the $O(1)$ -space window instead.

And the pattern generalises: recast 0s as `-1`s and ‘equal 0s and 1s’ becomes sum zero; carry the remainder instead of the sum and ‘divisible by k’ becomes matching remainders. Same machine, different quantity.”

9 Recall card

- **A subarray ending here sums to $k \Leftrightarrow$ some earlier prefix equals `running - k`.** Count earlier prefixes with a map; never re-sum.
- **`seen[0] = 1` first, always** — the prefix before the array. Without it, every left-edge subarray is silently missed.
- **Ask, then record.** `count += seen[running - k]` before `seen[running] += 1` — `k = 0` is the input that checks the order.
- **Count-map for “how many”; first-index map (never overwritten, sentinel `{0: -1}`) for “longest”.** Recast $0 \rightarrow -1$ gives equal-0s-and-1s; remainders give divisible-by- k .
- **$O(n)$ time, $O(n)$ space — the space is the price of negatives.** All positive + at-most $\rightarrow$ yesterday’s $O(1)$ window instead. Ask “can values be negative?” before choosing.

1 What this is, and why they ask it

A **document database** stores each record as a self-contained document — a nested, JSON-shaped value holding an entity and the things that belong to it, read and written as one unit. Where yesterday's key-value store treats the value as an opaque bag, a document store can see inside: index a field, query by it, update one nested piece in place. **MongoDB** is the product that defines the category.

Interviewers use document modelling as a design test with teeth, and the hub question is the classic form: model a blog, then — *what happens when a user changes their name?* The question is a trap for people who embedded everything and a trap for people who referenced everything; the passing answer knows the two tools, chooses per relationship using how the data is read and how often it changes, and can name the failure modes — the unbounded array, the 16-megabyte ceiling, the fact that atomicity stops at the document's edge. Everything needed to answer well is a database costume over ideas already met: [day 029](#)'s copies that need owners, and [day 036](#)'s arranged-for-serving.

2 The story

Salim shoots weddings, and his phone gallery is organised the way his work arrives: one album per wedding. Everything about the Fernandes wedding lives in the Fernandes album — the photos, the short clips, a screenshot of the booking message, the venue pin, the shot list the bride sent. When the family calls a year later, he opens one album and their whole world is in his hand. Nothing to gather. That is the design, and on delivery day it is unbeatable.

The design has a seam, and he knows exactly where it runs.

His cousin Rafiq assists at most shoots, and Salim used to note Rafiq's number inside each album's notes, so everything stayed in one place. Then Rafiq changed his number. Salim fixed it in the Colaco album, missed it in nine others, and spent a furious Sunday months later, opening every album one by one, because a caterer had reached a dead number found in an old shared album. These days Rafiq's number lives in exactly one place — the phone's contacts — and albums say only *Rafiq*. One fact, one home. Looking it up is one extra tap, and he pays that tap gladly.

Some things he still copies into the album deliberately. The venue's address goes in even though it is also in his contacts — because the address *as it was on that day* is part of that wedding's record. If the hall renames itself next year, the Fernandes album should still say what the invitation said. A copy that is *meant* to stay old, he has learnt, is not a mistake.

And one seam he only found at scale: the D'Souza wedding, three days, four thousand photos, and the album so fat his phone stutters opening it. The gallery grumbles above a certain size. He split it — main album, plus one per day — and made a note for the future: an album is for what belongs together *and stays a sensible size*. Things that grow forever get their own album.

3 The idea in plain English

Salim's albums are documents. His three lessons — the number that changed, the address that should not change, the album that grew too fat — are the entire discipline of document modelling.

What a document is

One entity and its belongings, nested, stored and fetched as a unit:

```
{
  "_id": "post_812",
  "title": "Why B-trees",
  "author": { "id": "u7", "name": "Meena" },
  "tags": ["storage", "trees"],
  "comments": [
    { "user": "u41", "text": "Cleared it up for me", "at": "2026-03-02" }
  ]
}
```

Documents live in **collections** (MongoDB's tables), and unlike rows, two documents in one collection may have different fields — there is no declared schema. The blog post above is read by one lookup: the page renders from one document, no joins, [day 036's](#) arranged-for-serving made concrete.

The one real decision: embed or reference

Every relationship in the model gets one of two treatments, and the whole craft is choosing:

Embed — put the data inside the document, like photos in the album. One read serves the page; the data travels with its owner.

Reference — store only an id, like *Rafiq* pointing at the contacts entry. One fact, one home; readers pay an extra lookup.

The choice is made by three questions, and they are Salim's three lessons in order:

1. **Is it read together with the parent, nearly always?** Read-together data wants embedding — that is what the one-read benefit is *for*.
2. **How often does it change, and how many copies would a change touch?** Rafiq's number: changes rarely but catastrophically when copied — reference it. The venue address on the day: *should* never change — embed it, deliberately, as a snapshot. This is [day 029](#)'s snapshot-versus-copy distinction, verbatim: a copy that is meant to stay old is a snapshot, not a bug.
3. **Does it grow without limit?** Comments on a viral post, events on a device — unbounded arrays fatten the document until reads slow and the 16 MB ceiling nears. Unbounded children get their own collection, referenced back — the D'Souza split.

The hub question, answered in the frame

A user changes their name. If the name is embedded in every post and comment for display — the usual choice, because pages read it constantly — then a rename must **fan out** to every copy: one update on the `users` document, plus a background pass rewriting the embedded `author.name` everywhere it appears. The alternatives are honest but different trades: reference-only (every page pays a lookup to render a byline) or accept staleness until the fan-out completes (what most real systems do — the rename propagates over minutes, and nobody minds). What fails the interview is not noticing the copies exist. [Day 029](#)'s law stands: **every copy needs an owner and a reconciliation path** — here, the `users` document owns the truth, and the fan-out job is the reconciliation.

4 The picture

The same blog, both treatments, seams marked:

EMBED (read-optimised)	REFERENCE (change-optimised)
<pre>posts: { title: "Why B-trees", author: {id:"u7", name:"Meena"}, ← copy comments: [...500...], ← grows } tags: [...]} </pre>	<pre>posts: users: { { title: "...", _id: "u7", author_id: "u7", ---> name: "Meena" comments: [...]} </pre>
<pre>page render: 1 read rename Meena: 1 + fan-out to every post/comment viral post: document fattens toward 16 MB</pre>	<pre>page render: 2 reads (post + author) rename Meena: 1 write, done viral post: comments in their own collection, unbounded</pre>

What to notice: neither column is “correct” — the left is Salim’s album, the right is his contacts app, and a real model uses both at once: embed the read-together and the snapshot-worthy, reference the shared and the unbounded.

The growth seam, drawn as a timeline:

```
comments embedded in the post document:

day 1      [c1]                4 KB   fine
month 1    [c1..c120]          250 KB  fine, reads slowing
viral      [c1..c48,000]        15 MB   every read hauls 15 MB;
                                         next comment nears the
                                         16 MB hard ceiling → writes fail

the split: posts hold a count + the newest 10;
           comments live in their own collection, keyed by post_id
```

What to notice: the failure is gradual and then sudden — reads degrade quietly for months, and the ceiling arrives as a hard write error on your most popular content, which is the worst possible place to discover a modelling decision.

5 How it actually works

MongoDB’s machinery, briefly

Documents are stored as **BSON** (binary JSON, with real types — dates, numbers, binary), each with a unique `_id` serving as its primary key. **Secondary indexes** work on any field, nested fields and array elements included — B-trees, exactly [day 030’s](#) and [day 031’s](#) machinery, so `db.posts.find({"tags": "storage"})` is an index lookup, not a scan. This is the real distance from yesterday’s key-value store: the store can see inside the value, and you can ask questions you did not pre-key — at [day 030’s](#) standard price of one more index maintained per write.

The atomicity boundary

A write to **one document is atomic**, however deep and complex the update — set a field, push to an array, increment a counter, all in one operation, all-or-nothing. That single-document guarantee is the design centre: model so that things which must change together *live* together, and the everyday writes never need more. Multi-document transactions exist since MongoDB 4.0 — [day 033](#)’s machinery, opt-in — and they work, at a real cost in latency and coordination; the grain of the tool, as [day 036](#) put it, is to rarely need them. If every workflow in the domain updates three documents atomically, the domain is telling you it wanted [day 026](#)’s tables.

The limits that shape models

The **16 MB document cap** is a hard error, not a warning. The practical limit arrives earlier: big documents make every read haul the whole album, and growth forces the storage engine to relocate and rewrite. Hence the standing rules: **embed the bounded, reference the unbounded**, and for the middle ground, hybrid patterns — keep the newest ten comments embedded for the first paint, the full history in its own collection.

Where the truth lives, and the Postgres question

Two closing placements, both interview-ready. MongoDB replicates (replica sets) and partitions (sharding by a key) — the distributed machinery arrives properly in a later phase; today’s point is only that the *modelling* discipline is identical at any scale. And the honest rival: **Postgres JSONB** puts indexed, queryable documents inside the relational store — so “we need flexible nested records” alone does not justify a second database. The document store earns its place when the *whole domain* is document-shaped and the one-read-per-page access pattern dominates; a document-shaped corner of a relational domain is usually a JSONB column.

6 The numbers

The read the model buys

```
blog page, relational: post + author + comments + tags
    4 index lookups (1 query joined, ~1-2 ms; 4 round trips done badly, ~4-8 ms)
blog page, document:    1 lookup, ~0.5-1 ms, one disk page if small

at 2,000 page views/s: the document model does 2,000 reads/s
                      where the naive relational app does 8,000
```

The rename the model pays

```
Meena's name embedded in 12,000 posts and 90,000 comments:

storage cost of the copies: ~102,000 × 20 bytes ≈ 2 MB      – irrelevant
rename fan-out:           102,000 document writes
  batched at 5,000 writes/s ≈ 20 seconds of background work – fine, IF built
  never built              = day 029's Sunday: stale names forever
```

The multiplication to say out loud: **copies** × **change-rate**, not copies × bytes. Two megabytes is nothing; 102,000 writes nobody wrote a job for is a data-quality incident.

The growth arithmetic

```
comments at 2 KB each, embedded:

    100 comments = 200 KB    fine
   1,000 comments =  2 MB    every page view hauls 2 MB – reads hurt
   8,000 comments = 16 MB    writes FAIL with a hard error

an "events" array at 1 KB × 100/day = 36 MB/year: breaches in ~5 months
```

Run this arithmetic at design time for every array in the model: **size** × **growth rate** **against 16 MB**, and against “every read carries the whole document” long before that.

Sizing the store

```
1 million posts × ~20 KB average (text + embedded recent comments) = 20 GB
→ one node, comfortably; indexes add 10-25% each (day 030's rule holds here)
```

Document stores are rarely chosen for raw size — [day 037](#)'s count-times-size still decides where things fit. They are chosen for the shape of the read.

7 The trade-offs

What the model gives up

The mirror of the one-read page, all inherited from [day 036](#) and now concrete: **cross-entity queries** (“top commenters this month” walks every post or needs a separate arrangement), **cross-document invariants** (nothing like a foreign key stops a comment pointing at a deleted post — integrity is application code now), and **fan-out on shared facts** (every embedded copy is a write you owe on change). Plus the quiet one: **schema flexibility means schema in the code** — the database will happily store five generations of document shapes, and every reader carries the if-else archaeology to handle them. Real teams version their documents (`"schema": 3`) and run migrations anyway.

When it breaks

The model breaks where relationships stop being tree-shaped. A blog is a tree: post owns comments, owns tags — documents fit. A marketplace is a web: orders touch products, buyers, sellers, inventory, payments, and every entity is shared by every other — model that as documents and you either embed copies of everything (fan-out hell) or reference everything (joins in application code, without the database's help). **Webs want tables; trees tolerate documents.** That sentence routes most modelling questions.

I would not use it if...

I would not use a document store where money or inventory moves between records — cross-record atomicity is the relational deal, and needing the opt-in transactions constantly means the grain is wrong. **I would not use one for an analytics-heavy domain** — arranged-for-serving is arranged against ad-hoc questions. **And I would not add MongoDB beside Postgres just for nested data** — JSONB covers that inside the store that already holds the truth. The document store earns its place when the domain is genuinely tree-shaped, page reads dominate, and the one-read pattern is worth owning the fan-out discipline.

The honest sentence

Embedding is not a performance trick; it is a *promise* — that this data is read with its parent, bounded in growth, and either never changes or has a job that changes every copy. Model documents by which promises you can keep.

8 In the interview

How it gets asked

- “*Model a blog / e-commerce catalogue / chat app in MongoDB.*” — the design form; they are watching the embed-versus-reference reasoning, not the field names.
- “*What happens when a user changes their name?*” — the fan-out probe, and the reason this hub question exists.
- “*When would you embed and when would you reference?*” — the direct form; answer with the three questions.
- “*What are the limits of a document?*” — 16 MB, unbounded arrays, atomicity at the document edge.
- “*MongoDB or Postgres for this domain?*” — tree or web, plus the JSONB escape hatch; [day 040](#) makes this a full lesson.

What to say out loud, in the first ninety seconds

1. **Name the unit and its guarantee.** *“A document is one entity and its belongings, read and written as a unit — and atomicity is per document, which drives the whole model.”*
2. **State the one decision.** *“Every relationship gets embedded or referenced, and I choose with three questions: read together? how often does it change, times how many copies? does it grow without bound?”*
3. **Model the blog in one breath.** *“Posts embed tags and the author’s name for display; comments get their own collection — unbounded; the author’s truth lives in users — shared.”*
4. **Answer the rename before it is asked.** *“A rename updates users, then fans out to the embedded display names as a background job — the copy has an owner and a reconciliation path, or I don’t make the copy.”*
5. **Name the ceilings.** *“16 MB hard cap, and reads haul the whole document long before that — so bounded things embed, growing things reference.”*

The follow-ups

“Walk me through the rename fan-out. What breaks, and how do you make it safe?” The truth changes in one place — the `users` document — and then 102,000 embedded display names are stale until rewritten. I would run the fan-out as an idempotent background job: query the indexed `author.id` fields, rewrite in batches with retries, and record progress so a crash resumes rather than restarts — day 018’s idempotency arriving in a data job. Two failure modes to name. Mid-fan-out reads see mixed names — old on some posts, new on others — which is an eventual-consistency window, acceptable for display names, and I would say so explicitly rather than let it be discovered. And writes racing the job: a comment created with the old cached name after the job passed its range — prevented by having creation always read the name from `users` at write time, never from another embedded copy. If the domain cannot tolerate the staleness window at all — legal names on invoices — then the name was never embeddable, and that field becomes a reference or, for the invoice case, a deliberate snapshot that *should not* change: day 029’s distinction deciding the model.

“How do you handle something needing atomic updates across two documents?”

First I check whether the model is telling me something: the standard move is to *re-draw the boundary* so the invariant lives inside one document — if an order and its line items must change together, the line items belong embedded in the order, and the single-document atomic write covers it. That redesign answers most cases and is the idiomatic one. Where the invariant genuinely spans entities — decrement stock and record the order — MongoDB’s multi-document transactions exist and work, with real costs: coordination latency, aborts under contention, and pressure to keep them short — day 033’s rules apply unchanged. If I find them everywhere in the design, the honest conclusion is that

the domain is web-shaped and wants a relational store. And there is the pattern in between: make one document the record of intent (an order document with a status field) and drive the side effects from it asynchronously with retries — which is eventual consistency with an audit trail, the same shape sagas took in day 033.

“Why not just always reference, like a relational database?” Because then the document store is a worse relational database — every page render becomes application-side joins, without the query planner, join algorithms, or foreign keys that make joins safe and fast in Postgres; `$lookup` exists but is not the tool’s grain. The point of the document model is the locality: the page’s read is one document because the things read together live together. Reference-everything spends that advantage and keeps all the costs — no cross- document integrity, weaker ad-hoc queries. So all-reference is a smell in both directions: if everything is referenced, the data is web-shaped and belongs relational; if the model is genuinely tree-shaped, embedding the read-together, bounded, own-able parts is what buys the one-read page. The craft is per-relationship, not a global policy — embed the album’s photos, reference Rafiq.

A model answer

“I’ll model the blog as three collections, choosing embed or reference per relationship.

Posts embed what the page reads together and what is bounded: the title, body, tags — a small fixed array — and the author’s id *plus display name*, a deliberate copy so rendering a page is one read. Comments do not embed: they are unbounded — a viral post’s comments would fatten the document toward the 16 MB cap and make every read haul megabytes — so they get their own collection, keyed and indexed by post_id, with maybe the newest ten duplicated into the post for first paint. Users own the truth about people.

Now the rename. The user document updates atomically — that is the easy part. The embedded display names on every post and comment are now stale: those are copies, and every copy needs an owner and a reconciliation path. The owner is the users collection; the reconciliation is an idempotent background job that fans the new name out — a hundred thousand embedded copies is about twenty seconds of batched writes. Until it finishes, pages show mixed names; for display names that eventual consistency is acceptable, and I am saying so out loud rather than letting it be discovered. If it weren’t acceptable, the name was never embeddable — reference it and pay a lookup per render.

Where this model breaks: cross-entity questions — ‘top commenters this month’ — want a different arrangement; nothing enforces that a comment’s post exists, so integrity is my application’s job; and if the domain grew into a marketplace — orders, inventory, payments, everything shared by everything — the relationships stop being a tree, and webs want tables. I’d also flag that if this blog lived inside an existing Postgres system, JSONB would carry the flexible parts without a second database.”

9 Recall card

- **A document = one entity + its belongings, read and written as one unit; atomicity ends at the document’s edge.** Model so everyday writes never cross it.
- **Embed or reference, by three questions:** read together? change-rate $\times$ copies? unbounded growth? Bounded + read-together + own-able $\rightarrow$ embed; shared or growing $\rightarrow$ reference.

- **The rename answer:** truth in `users`, embedded display names fanned out by an idempotent background job — every copy has an owner and a reconciliation path, or the copy is not made.
- **Ceilings:** 16 MB hard cap; reads haul the whole document far sooner. Run size $\times$ growth on every array at design time.
- **Trees tolerate documents; webs want tables.** And nested data alone justifies JSONB, not a second database.

Practice — Subarray sum equals K: prefix plus hash map

DSA topic: Subarray sum equals K: prefix plus hash map **System design topic:** Document databases

Code these, in this order

Four problems, one machine: carry a running quantity, ask a map about the past. **Before each, say two things out loud:** what the running quantity is (sum, recast sum, remainder), and what the map stores (count, or first index).

Before each one, ask:

1. Can values be negative — and did the problem just retire the window for me?
2. “How many” (count map) or “longest” (first-index map, never overwritten)?
3. What is the sentinel — `{0: 1}` or `{0: -1}` — and what does it mean?
4. Ask first, record after — which input checks that order?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Subarray Sum Equals K	LeetCode 560 (Medium)	The machine itself: <code>seen[0] = 1</code> , ask-then-record, and why the window is illegal.
2	Subarray Sums Divisible by K	LeetCode 974 (Medium)	The remainder costume — same remainder at both ends — and Python’s friendly <code>%</code> .
3	Contiguous Array	LeetCode 525 (Medium)	The longest variant: recast $0 \rightarrow -1$, first-index map, sentinel <code>{0: -1}</code> , never overwrite.
4	Maximum Size Subarray Sum Equals k	LeetCode 325 (Medium)	Problem 1’s quantity with problem 3’s map — proof you know which contract is which.

On problem 1, remove the sentinel on purpose

Delete `seen[0] = 1` and run `[1, 2, 3]` with `k = 3`. You get 1 instead of 2. Name the subarray that went missing and say why every left-edge subarray needs the opening balance.

Then swap the two loop lines and run `[-1, -1, 1]` with `k = 0` — you get 4 instead of 1. Say what got counted.

On problem 2, check the language edge

Print `(-7) % 5` and say why Python needs no fix-up. Then say the one sentence you would add in a Java or C++ interview about `(r % k + k) % k`.

On problem 3, overwrite on purpose

Store `first[running] = i` unconditionally and run `[0, 0, 1, 0, 0, 0, 1, 1]`. You get 4 instead of 6. Say which direction the stored index drifted and why “longest” wants the earliest occurrence frozen.

On problem 4, say the hybrid before coding

One sentence: “running sum like 560, first-index map like 525, sentinel `{0: -1}`.” If you can say it, the code is four edits from problem 3.

The contract drill

For each phrasing, say the quantity, the map contract, and the sentinel — five seconds each:

1. Count subarrays summing to k .
2. Longest subarray summing to k .
3. Count subarrays with sum divisible by k .
4. Longest subarray with equal 0s and 1s.
5. Does any subarray sum to k ?

The routing echo

Say which day’s tool serves each, and why, in one sentence:

1. Longest subarray with $\text{sum} \leq k$, all positive.
2. Longest subarray with $\text{sum} = k$, negatives allowed.
3. Count subarrays with at most k odd numbers.
4. Count subarrays with $\text{sum} = k$, negatives allowed.

The embed-or-reference drill

For each relationship in a marketplace, say embed or reference, and the deciding question:

1. An order’s line items.
2. The seller’s display name shown on a product card.
3. A product’s reviews.
4. The buyer’s default address, as it was when the order shipped.
5. A device’s event log, 100 events a day.

Number 4 is the snapshot — say why this copy is *meant* to stay old.

The ceiling drill

From memory, in under two minutes:

- Comments at 2 KB embedded: the size at 100, at 1,000, and where the hard error lands.
- An events array at 1 KB $\times$ 100/day — months until 16 MB.
- A rename fanning out to 102,000 embedded copies at 5,000 writes/s — the background job's duration, and the two failure modes to name (mixed names mid-job; a write racing the job).

The breakage drill

Answer each in one or two sentences, out loud:

1. Why does “top commenters this month” hurt in a document model?
 2. What stops a comment pointing at a deleted post — and in Postgres, what would have?
 3. When do MongoDB's multi-document transactions signal the domain wanted tables?
 4. Why does “just use JSONB” answer many MongoDB proposals?
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Count the subarrays whose sum equals k.* Ask about negatives first. The flip — earlier prefix equals running minus k — then the sentinel, the ask-then-record order with the $k = 0$ input, and $O(n)/O(n)$ with the space named as the price of negatives.
 2. *Model a blog in MongoDB. What happens when a user changes their name?* Three collections, embed-versus-reference with reasons, then the rename: truth in users, fan-out job for the display-name copies, the staleness window said out loud, and when the name was never embeddable at all.
 3. *Why does the sliding window fail on this problem, when it worked all last week?* Monotonicity, both directions broken by negatives, the discarded start it cannot justify — and the honest counterpart: when the window is still the better tool, and what today's $O(n)$ space buys instead.
-

Before you move on

- [] I can write the 560 machine from memory: sentinel, ask, record — in that order.
- [] I know which map contract serves “how many” and which serves “longest”, and the two sentinels.
- [] I can name the input that punishes each of the three classic bugs — missing sentinel, swapped order, overwritten first index.
- [] I ask “can values be negative?” before choosing between window and map.
- [] I can model the blog with reasons per relationship, and answer the rename without pausing.
- [] I can run the 16 MB and fan-out arithmetic for any array or copy in a model.
- [] I answered all three questions above out loud.

Difference arrays: range updates, cheaply

DATA STRUCTURES AND ALGORITHMS

Difference arrays: range updates, cheaply

You can apply a thousand range updates in $O(1)$ each and rebuild the array at the end.

SYSTEM DESIGN

Wide-column and time-series stores

You can say what Cassandra's data model really is and when a time-series store is right.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Apply many range increments and then report the final array.*
- *Which database would you pick for storing metrics, and why?*

Difference arrays: range updates, cheaply

1 What this is, and why they ask it

A **difference array** is the prefix sum run backwards. Instead of storing values and pre-computing totals, you store only the **changes at boundaries**: to add 5 to every element from index `l` to `r`, write `+5` at `l` and `-5` just past `r` — two writes, whatever the range's length. When all updates are in, one prefix-sum pass turns the recorded changes back into the final array.

It is [day 037](#)'s trade mirrored: prefix sums made *reads* of ranges $O(1)$ by preparing once; difference arrays make *writes* to ranges $O(1)$ by settling once at the end. Interviewers like the family — *Corporate Flight Bookings*, *Car Pooling*, *Range Addition* — because the trick is three lines, and everything they want to probe lives around the lines: whether you see why two boundary marks equal a whole range of additions, whether you handle the inclusive-end boundary without an off-by-one, and whether you know when the deal is off — the moment someone needs to *read* between the updates.

2 The story

The wedding was a ten-day affair, and Sarasu, who runs the kitchen for that side of the family, had one question that decided everything else: how many people eat here, each day?

Relatives do not arrive together. The Chennai cousins were coming Sunday and leaving Thursday. The grand-aunt, Saturday to the last day. The Bombay lot, just Tuesday-Wednesday-Thursday. Fourteen families, each with their own dates, each a message in the family chat.

Her nephew offered to work it out and did it the honest, slow way: a list of ten days on his phone, and for each family, he went down every day of their stay adding their headcount. The Chennai cousins alone meant touching five days. Fourteen families, every day of every stay — his thumb was sore, and then a message arrived changing one family's dates and he groaned out loud.

Sarasu does not do it that way. For each family she records exactly two things: the day they arrive, plus so-many mouths; the day they leave, minus so-many. The Chennai cousins are two entries — Sunday `+6`, Thursday `-6` — not five. A family changing plans is

two entries corrected, not a stay's worth of days redone.

Of course, her page of pluses and minuses does not answer the cook's question directly. Ask her on Tuesday what Wednesday's count is and she cannot say — not yet. When the entries are all in, she walks the ten days once, keeping a running count: start at zero, Saturday plus four, Sunday plus six, and so on down the page. Each day's figure is just the running count as she passes it. One walk, and the whole calendar is settled.

The one thing she is careful about — because it went wrong at her own daughter's wedding — is what “leaving Thursday” means. Leaving Thursday *morning* means no Thursday lunch: the minus goes on Thursday. Leaving Thursday *night* means the minus belongs to Friday. That year, one aunt was subtracted a day early, the count came up short, and Sarasu has asked every family the same question since: last meal, or last night?

3 The idea in plain English

Sarasu's arrivals-and-departures page is the difference array. Her nephew's sore thumb is the naive version. Her one settling walk is a prefix sum. And her “last meal or last night?” is the boundary question that decides every off-by-one in this family.

Recording changes instead of values

To add `amount` to every element of `nums[l..r]`, the naive way touches `r - l + 1` elements. The difference array touches two:

```
diff[l] += amount      # from here on, everything is `amount` higher
diff[r + 1] -= amount  # from here on, that stops
```

`diff` records, at each index, **how much the running level changes there** — Sunday +6, Thursday −6. A range update is one step up at its start and one step down just past its end, and the thousand elements in between are never touched, because *staying raised* needs no marks.

Settling: the prefix sum comes home

When all updates are recorded, rebuild with a running total:

```
result = list(accumulate(diff))[:-1]
```

The running count walking Sarasu's page is [day 037](#)'s prefix sum — the two tools are one idea facing opposite directions. Prefix sums: store values, subtract readings to get ranges. Difference arrays: store changes, accumulate to get values. Formally, `diff` is built so that its prefix sums are the final array — differencing and prefix-summing undo each other, the way subtraction undoes addition.

The `r + 1`, and the extra slot

The `-amount` lands **just past** the range's end — the raise stops *after* `r`, because `r` itself is still inside. Two consequences, both bug factories:

- `diff` needs `length + 1` slots, so that a range ending at the last element can write its step-down at `diff[length]` without crashing. The settling pass drops that final slot.
- When a problem's "end" is exclusive — passengers who get off *at* stop 5 stopped occupying the seat *at* stop 5 — the minus goes at `end`, not `end + 1`. Sarasu's question, exactly: last meal, or last night? §7 shows both versions of getting it wrong.

When the deal is off

The difference array answers questions **only after settling**. Ask for `nums[3]` halfway through the updates and there is no answer without a walk — the marks are promises, not values. So the tool's contract is: **all updates first, then one settle, then reads**. Interleaved reads and range updates are the same escalation as [day 037's](#) mutable follow-up — name the Fenwick and segment trees, $O(\log n)$ both ways, and say they are heavier machinery for a genuinely mixed workload.

4 The picture

Three updates on a five-element array, recorded and settled:

updates, each becoming exactly two marks:

```
add 2 to [1..3]: diff[1] += 2, diff[4] -= 2
add 3 to [2..4]: diff[2] += 3, diff[5] -= 3
add -2 to [0..2]: diff[0] -= 2, diff[3] += 2
```

index	0	1	2	3	4	5	← the extra slot
	+-----+-----+-----+						
diff	-2	+2	+3	+2	-2	-3	
	+-----+-----+-----+						

settle (running total): -2 0 3 5 3 (drop slot 5)

What to notice: the six marks from three different updates all live in one array, in no particular order of arrival, and the settle adds them up in passing — had two marks landed on the same slot they would simply have summed. Boundary marks compose by plain addition; nothing ever needs to know which update wrote what.

The step shape, drawn as Sarasu's running count:

```

level
5           +-----+
4           |         |
3       +-----+     +-----+
2       |         .     .     |
1       |         .     .     |
0  +-----+     .     .     +
-1  |         .     .     .
-2  +         .     .     .
    day0  day1  day2  day3  day4

```

each +N in diff is a step UP at that index;
each -N is a step DOWN. The array is the skyline the steps trace.

What to notice: a range update is one step up and one step down — the flat top in between is free. That is the entire $O(1)$ -per-update argument, drawn.

5 The code, built step by step

Range Addition — the pattern, pure

LeetCode 370: length `n`, a list of `[l, r, amount]` updates (inclusive ends), report the final array.

```

diff = [0] * (length + 1)
for left, right, amount in updates:
    diff[left] += amount
    diff[right + 1] -= amount

```

The extra slot makes `right + 1` always legal — a range ending at the last element steps down in the slot the settle will discard.

```

return list(accumulate(diff))[:-1]

```

Settle and drop the scratch slot. `accumulate` is [day 037](#)'s build, reused verbatim — no `initial=0` this time, because `diff[0]` already is the change from zero.

Corporate Flight Bookings — the same, 1-indexed

LeetCode 1109: bookings `[first, last, seats]` with flights numbered from 1, ends inclusive.

```

diff = [0] * (n + 1)
for first, last, seats in bookings:
    diff[first - 1] += seats      # shift to 0-based
    diff[last] -= seats          # (last - 1) + 1 = last: two shifts cancel
return list(accumulate(diff))[:-1]

```

The 1-based shift and the exclusive step-down cancel into plain `diff[last]` — worth deriving out loud rather than pattern-matching, because deriving it is the protection against the off-by-one.

Car Pooling — Sarasu's question decides it

LeetCode 1094: trips `[people, start, end]`, passengers ride from `start` to `end`, capacity check. The crux: passengers get off **at** `end` — the seat is free *at* `end`, so the step-down goes at `end`, not `end + 1`. Last meal, or last night — this problem says last night was `end - 1`.

```

diff = [0] * 1001                # stops are bounded: 0..1000
for people, start, end in trips:
    diff[start] += people
    diff[end] -= people           # off AT end - exclusive, no +1

```

Then the settle doubles as the check — no final array needed, just the running level against capacity:

```

riding = 0
for change in diff:
    riding += change
    if riding > capacity:
        return False
return True

```

The complete solutions

```
from itertools import accumulate

def get_modified_array(length: int, updates: list[list[int]]) → list[int]:
    """LeetCode 370. Inclusive ends: step down at right + 1; settle once."""
    diff = [0] * (length + 1)
    for left, right, amount in updates:
        diff[left] += amount
        diff[right + 1] -= amount    # extra slot makes this always legal
    return list(accumulate(diff))[:-1]    # drop the scratch slot

def corp_flight_bookings(bookings: list[list[int]], n: int) → list[int]:
    """LeetCode 1109. 1-indexed inclusive ends: the two shifts cancel."""
    diff = [0] * (n + 1)
    for first, last, seats in bookings:
        diff[first - 1] += seats
        diff[last] -= seats
    return list(accumulate(diff))[:-1]

def car_pooling(trips: list[list[int]], capacity: int) → bool:
    """LeetCode 1094. Passengers leave AT end: exclusive end, no +1.
    The settle walk doubles as the capacity check."""
    diff = [0] * 1001
    for people, start, end in trips:
        diff[start] += people
        diff[end] -= people
    riding = 0
    for change in diff:
        riding += change
        if riding > capacity:
            return False
    return True

if __name__ == "__main__":
    print(get_modified_array(5, [[1, 3, 2], [2, 4, 3], [0, 2, -2]]))
    # [-2, 0, 3, 5, 3] – the walk from §4

    print(corp_flight_bookings([[1, 2, 10], [2, 3, 20], [2, 5, 25]], 5))
    # [10, 55, 45, 25, 25]

    print(car_pooling([[2, 1, 5], [3, 3, 7]], 4))    # False – 5 aboard on [3, 5)
    print(car_pooling([[2, 1, 5], [3, 3, 7]], 5))    # True
    print(car_pooling([[2, 1, 5], [3, 5, 7]], 3))    # True – clean handover AT 5
```

The last test is the boundary made executable: two passengers off at stop 5, three on at stop 5, capacity 3 — legal only because “off at 5” frees the seats *at* 5. Change the `diff[end]` to `diff[end + 1]` and this test flips to `False`.

6 What it costs

Counted from the loops

```
recording:  u updates × 2 writes each           → O(u)
settling:   one pass over length + 1 slots       → O(n)
total:      O(n + u)  – and O(1) per update, which is the headline
space:      the diff array itself, O(n); no other structure
```

The naive version costs the sum of the range lengths — $O(n)$ per update in the worst case, $O(n \times u)$ overall.

Measured, not just argued

A thousand updates averaging 50,000 elements wide, on a 100,000-element array — both versions run for real:

```
naive (touch every element per update):  2.225 s
difference array (record + one settle):  0.003 s    ~780× faster

the arithmetic behind it:
  naive : 1,000 × ~50,000 = 50,000,000 element writes
  diff  : 1,000 × 2 + 100,001 ≈ 102,000 operations – ~500× fewer, as observed
```

The trade said in one line

Prefix sums and difference arrays are the same purchase in opposite shops: **prefix** pays $O(n)$ once to make range *reads* $O(1)$; **difference** pays $O(n)$ once — at the end — to make range *writes* $O(1)$. Each is helpless at the other's job until it settles, and the mixed workload belongs to the log- n trees.

The number to have ready

Two writes per update regardless of range width, one $O(n)$ settle at the end — $O(n + u)$ total. A thousand wide updates on a hundred-thousand-element array: about a hundred thousand operations against fifty million naive — measured at roughly $780\times$ faster.

7 The traps

The real error: the missing extra slot

Size the array at `length` and update a range that touches the last element:

```
diff = [0] * 5                                # length 5, no extra slot
diff[1] += 2
diff[4 + 1] -= 2                               # range [1..4]: steps down at index 5
```

```
Traceback (most recent call last):
  File "day39.py", line 3, in <module>
    diff[4 + 1] -= 2
IndexError: list index out of range
```

The step-down for a range ending at the last element has nowhere to land. `length + 1 slots, always`, and the settle drops the last one. (The tempting alternative — `if right + 1 < length:` guards around every write — works and doubles the code's surface for the same reason [day 037](#)'s sentinel beat the special-case `if`.)

The near-miss: the inclusive minus at the inclusive end

Put the step-down at `r` instead of past it, in Range Addition:

```
diff[left] += amount
diff[right] -= amount                        # wrong: right is still INSIDE the range
```

On `get_modified_array(5, [[1, 3, 2]])` this yields `[0, 2, 2, 0, 0]` — the range's own last element never got its raise. Every range comes out one short at the tail, silently. The sentence that prevents it: **the raise stops *after* the last raised element.**

The near-miss: `+ 1` where the end was already exclusive

The same instinct, inverted, in Car Pooling:

```
diff[start] += people
diff[end + 1] -= people                     # wrong: passengers got off AT end
```

```
bad handover: False
```

On `[[2, 1, 5], [3, 5, 7]]` with capacity 3 — two off at stop 5, three on at stop 5 — the correct answer is `True`; this version keeps the leavers aboard one stop too long, sees five riders, and refuses a legal trip. Two problems, two conventions: Range Addition's ends are inclusive (`r + 1`), Car Pooling's are exclusive (`end`). **The convention is in the problem statement, not in the pattern** — Sarasu's question, asked of every problem: last meal, or last night?

The near-miss: reading before settling

```
diff[2] += 5          # update: add 5 to [2..6]
diff[7] -= 5
print(diff[4])        # "what is nums[4] now?"
```

```
0
```

Not 5 — nothing. The marks are promises; only the settle walk redeems them. If the problem interleaves reads with range updates, the difference array is the wrong tool and saying so is the answer: rebuild-per-read is $O(n)$ a time, and the honest upgrade is a Fenwick or segment tree at $O(\log n)$ for both operations.

The contract corner: bounds and non-integer positions

Car Pooling's `diff = [0] * 1001` leans on the promise that stops live in `0..1000` — ask for that bound before writing it. When positions are huge or unbounded (timestamps, coordinates), a dense array is off the table; the same boundary marks go into a **map** — `{position: change}` — settled by walking the sorted keys: $O(u \log u)$ for the sort, memory $O(u)$, and no dependence on the coordinate range at all. Same idea, sparse clothing — worth naming whenever the interviewer says “positions can be up to a billion”.

8 In the interview

How it gets asked

- “Apply these range increments and return the final array.” — Range Addition, LeetCode 370; the pattern verbatim.
- “Seat bookings across flights 1 to n — how many seats per flight?” — Corporate Flight Bookings, LeetCode 1109; 1-indexed, inclusive.
- “Can this van complete all trips without exceeding capacity?” — Car Pooling, LeetCode 1094; exclusive ends, settle-as-check.
- “A calendar of guest arrivals and departures — peak occupancy?” — the same tool wearing interval clothes; it reappears when the sorting phase treats interval problems.
- And the escalation: “now queries arrive between the updates” — the handover question to log- n trees, mirroring [day 037](#)’s.

What to say out loud, in the first ninety seconds

1. **Name the shape.** “Many range updates, one final report — that’s a difference array: record changes at boundaries, settle once with a prefix sum.”

2. **Give the two-line mechanic.** “Add at the start, subtract just past the end — two writes per update, any width. The elements between never get touched because staying raised needs no marks.”
3. **Ask the boundary question.** “Is the end inclusive — do they fly the last flight, eat the last meal? That decides whether the minus goes at end or end + 1.”
4. **State the contract.** “All updates, then one settle, then reads — if reads interleave, this is the wrong tool and I’d name a Fenwick tree instead.”
5. **Give the cost.** “ $O(1)$ per update, $O(n)$ settle — against $O(\text{range width})$ per update naively; at a thousand wide updates that’s measured hundreds of times faster.”

The follow-ups

“Why does writing two numbers update a thousand elements? Convince me.”

Because the settle walk turns marks into levels, and between marks the level cannot move. The running total only changes where `diff` is non-zero — so `+5` at index `l` raises the level as the walk passes `l`, and that raise persists through every subsequent index for free, because the walk carries its total forward; the `-5` just past `r` is the one and only thing that ends it. Formally: the settled value at index `i` is the sum of `diff[0..i]`, so adding 5 to `diff[l]` adds 5 to every settled value from `l` onward — a suffix — and subtracting at `r + 1` cancels it from `r + 1` onward; a range is the difference of two suffixes. That is also why overlapping updates compose without any book-keeping: each is a pair of boundary marks, addition is commutative, and the walk adds them all in passing — §4’s index 3, where two updates’ marks share a slot and simply sum.

“Now the updates are multiplicative — double everything in the range. Still works?” Not with sums, and the honest first answer is why: the whole machine rests on the settle being a running *sum* and range effects being the difference of two suffix-sums — addition’s structure. Doubling is multiplication; the equivalent trick needs a running *product*: record $\times 2$ at `l` and $\times \frac{1}{2}$ past `r`, settle by multiplying. That works exactly when every factor is invertible — no zeros, and floating-point or modular arithmetic to hold the halves — so for “double from `l` to `r`” over integers it survives only in log-space or mod- p form, and I would say that rather than force it. The general statement, if the interviewer wants it: the trick works over any operation with an inverse — sums with subtraction, products with division, XOR with itself — because “start the effect, cancel the effect” is the whole design. For non-invertible updates like “set the range to 5” or “max with 5”, boundary marks cannot cancel, and the tool is a segment tree with lazy propagation — machinery for a later phase, named and left.

“Peak riders rather than a yes/no capacity check — and what if stop numbers go to a billion?” Peak is the settle walk keeping a maximum instead of a threshold test — `peak = max(peak, riding)` — same cost, and it falls out of the same running level.

Unbounded coordinates kill the dense array, not the idea: put the marks in a dictionary keyed by position — `changes[start] += people`, `changes[end] -= people` — then settle by walking the keys in sorted order. Cost becomes $O(u \log u)$ for the sort with memory $O(u)$, independent of the coordinate range; the level between marked positions is constant, so nothing between them ever needed a slot. That sparse form is worth knowing by name because it is the standard answer for time-stamped intervals — bookings, meetings, server load — where the axis is huge and the events are few, and it reappears when the sorting phase does interval problems properly.

A model answer

“Bookings across flights 1 to n , report seats per flight — many range updates, one final report. I’ll use a difference array: record each booking as two boundary marks, then settle once.

The mechanic: to add `seats` to flights `first` through `last`, I add at the start and subtract just past the end. The elements between are never touched — when I rebuild with a running total, a mark raises the level and the level stays raised until the closing mark cancels it. Two writes per booking, any width.

One boundary question before I code: ends are inclusive here — they do fly the last flight — so the subtract goes past it. The array is 1-indexed, so shifting to 0-based, the marks are `diff[first - 1] += seats` and `diff[last] -= seats` — the two off-by-ones cancel, and I’d rather derive that than remember it. I size the array $n + 1$ so a booking ending at flight n has a slot to step down in, and the settle drops that slot.

```
def corp_flight_bookings(bookings, n):
    diff = [0] * (n + 1)
    for first, last, seats in bookings:
        diff[first - 1] += seats
        diff[last] -= seats
    return list(accumulate(diff))[:-1]
```

Cost: $O(1)$ per booking and one $O(n)$ settle — $O(n + u)$ total, against $O(n \times u)$ touching every flight of every booking. And the contract worth stating: this answers nothing until it settles — if queries arrived between bookings I’d move to a Fenwick tree at $O(\log n)$ both ways, and if flight numbers were unbounded I’d keep the same marks in a sorted map instead of a dense array.”

9 Recall card

- **Range update = two boundary marks:** `diff[l] += amount`, `diff[r + 1] -= amount` — $O(1)$ each, any width. Settle once with a running total; the marks compose by plain addition.
- **It is day 037 mirrored:** prefix sums buy $O(1)$ range *reads*; difference arrays buy $O(1)$ range *writes*. Each settles once; mixed read/write workloads go to Fenwick/segment trees, $O(\log n)$.
- **Size** `length + 1` — the step-down for a range ending at the last element needs its slot, and the settle drops it. No guards.
- **Ask the boundary question every time:** inclusive end $\rightarrow$ minus at `r + 1` (flights, bookings); exclusive end $\rightarrow$ minus at `r` (car pooling — off *at* the stop). Last meal, or last night?
- **Marks are promises, not values** — no reads before the settle. Huge coordinates $\rightarrow$ same marks in a sorted map, $O(u \log u)$, range-independent.

1 What this is, and why they ask it

A **wide-column store** — Cassandra is the name to know — organises data by two keys: a **partition key** that decides which machine (and which bucket on it) a row belongs to, and **clustering columns** that keep rows *sorted inside* that bucket. One request fetches a slice of one partition, in order — and almost nothing else is cheap. A **time-series store** — InfluxDB, TimescaleDB, Prometheus — is the same instinct specialised for measurements over time: enormous append-only write volume, queries that are always “this thing, this time range”, and data that ages out on schedule.

Interviewers reach for these when the design has a firehose in it — metrics, chat messages, sensor readings, activity feeds — because the stores that survive firehoses all made the same trade, and they want to hear you say it: **you must know your queries when you design the table**. Cassandra powers exactly these workloads at Discord, Netflix and Apple scale, and “why not just Postgres?” is the follow-up that sorts slogan-repeaters from engineers.

2 The story

The mailroom in the old head office is a wall of pigeonholes, one per department, and Xavier has sorted the post there for eleven years.

His rule is the wall’s whole design: everything for a department goes in that department’s hole, and inside the hole, newest on top. A courier drops forty envelopes at ten o’clock; Xavier reads only the department name, walks the wall once, and drops each envelope on top of its stack. He never rearranges anything, never reads past the first line of an address, never goes back. Sorting happens by *where things land*, not by work done later. That is why the morning flood, three hundred envelopes some days, takes him under twenty minutes.

Collection is the same story in reverse. The accounts clerk comes at four and says: whatever came for us since Tuesday. One hole, top of one stack, stop when the dates go past Tuesday — ten seconds, and the envelopes are already in order. Nobody ever asks him for anything else, because the wall has trained the whole building in what it is good at.

Once, a new auditor asked for every envelope from a particular insurance company, across all departments, for the whole year. Xavier did it — every hole, every stack, top to bottom, an afternoon on a stool — and then told her, kindly, that next time she should ask the sender for their list instead. The wall answers by department and date. It does not answer by sender. No amount of effort at collection time fixes what the landing rule never recorded.

And every January, the bottom of every stack goes into boxes for the basement, and after three years the boxes are pulped. Nobody decides envelope by envelope. Old post ages out on a schedule, because a pigeonhole that keeps everything forever eventually holds everything and finds nothing.

The wall, Xavier says, was designed by his predecessor around one question asked a hundred times a day. Design it around the question, and the question costs nothing.

3 The idea in plain English

Xavier's wall is a Cassandra table. The department name is the **partition key**. Newest-on-top is the **clustering order**. The auditor is the query you did not design for. And January is **retention**.

The two-part key, and what each part buys

A Cassandra table's primary key has two jobs, split explicitly:

```
CREATE TABLE messages (  
    channel_id    bigint,           -- partition key: WHICH pigeonhole  
    sent_at       timeuuid,        -- clustering column: ORDER inside it  
    author_id     bigint,  
    body          text,  
    PRIMARY KEY ((channel_id), sent_at)  
    ) WITH CLUSTERING ORDER BY (sent_at DESC);
```

The **partition key** is hashed to place the row — [day 037](#)'s key-value placement, deciding both the machine in the cluster and the bucket on it. The **clustering columns** sort rows *within* the partition, physically. So the query the table was built for — *the latest 50 messages in this channel* — is: hash to one machine, find one partition, read the top of one sorted run. One seek, sequential read, already in order — Xavier's ten seconds.

This is [day 026](#)'s primary key split into its two real jobs, and it is the whole data model. There are no joins, no foreign keys, and — effectively — no queries that do not start with the partition key. The auditor's question ("by sender, across all partitions") is a

full cluster scan, and the honest Cassandra answer is Xavier's: that question needs **its own table**, written at the same time — one table per query shape, copies maintained at write time, [day 036](#)'s arranged-for-serving taken to its disciplined extreme.

Why writes are the superpower

Cassandra's storage engine is the **log-structured merge tree** from [day 031](#) — writes append; sorting happens by where things land, then by background merging; nothing is updated in place. Add that every node accepts writes (no single leader — the distributed machinery is a later phase; today, just the consequence), and you get the headline: write throughput that scales nearly linearly with machines. Discord's message history, Netflix's viewing events, Apple's iCloud metadata — this store, this reason.

Time series: the same instinct, specialised

Metrics are the extreme case: writes are *only* appends of `(thing, timestamp, value)`; reads are *only* “this thing, this window, aggregated”; and old data loses value on a schedule. Time-series stores hard-code all three:

- **Partition by thing and time bucket**, so every query is one pigeonhole slice — the Cassandra idea, made automatic (TimescaleDB calls the buckets chunks; Influx, shards).
- **Compress brutally** — timestamps arrive at fixed intervals and values drift slowly, so delta-encoding gets $10\text{--}20 \times$ ([day 037](#)'s running differences, used as a compression trick: store the small deltas, not the big values).
- **Age data out by policy** — keep raw data 15 days, keep 5-minute averages a year, drop the rest: **retention and downsampling**, Xavier's January, built into the store.

Prometheus owns the pull-based metrics niche; InfluxDB the push-based one; TimescaleDB is the “Postgres with time-series superpowers” answer — a real Postgres extension, so SQL, joins and one operational stack survive.

4 The picture

The wall, as a cluster:

hash(channel_id) picks the node

node A	node B	node C
+-----+	+-----+	+-----+
channel 17	channel 4	channel 92
msg 09:41 ←	msg 09:40	msg 09:44
msg 09:38	msg 09:22	msg 09:41
msg 09:31	...	...
...sorted...	+-----+	+-----+
+-----+	channel 51	
channel 33	...	
...	+-----+	
+-----+		

"latest 50 in channel 17" → one node, one partition, top of one sorted run
"all messages by user 7" → EVERY node, EVERY partition (the auditor)

What to notice: the fast query never fans out; the undesigned query always does. The difference is not indexes or tuning — it is where the landing rule put things.

The time-series lifecycle, on one axis:

now	15 days	1 year	
----- -----	----- -----	----- -----	> age
raw points	downsampled to	dropped	
1/sec, compressed	5-min averages		
~1.5 bytes/point	288 points/day/series		

writes only ever land at "now" → the newest chunk stays hot in memory, old chunks are sealed, compressed, and eventually deleted whole.

What to notice: deletion happens by dropping sealed chunks whole — cheap, like pulp- ing a box — never by hunting individual rows, which is exactly what a B-tree store would have to do.

5 How it actually works

Writing and reading a partition

A write hashes the partition key, lands on the owning nodes, appends to the commit log (the write-ahead log from [day 025](#), same job) and into an in-memory table; sealed memtables flush to sorted files that background **compaction** merges — the LSM lifecycle from [day 031](#), verbatim. A read finds the partition, merges the memtable and the few sorted files that could hold the slice, and streams rows in clustering order. Point lookups and slices: fast. Anything without the partition key: a cluster-wide scatter, which the query language makes you say out loud (`ALLOW FILTERING` — a phrase that should read as a warning label).

Modelling: one table per query

The discipline that follows from the model, and the thing interviewers probe: you write the queries first, then a table *per query shape*, and the application writes each event to all of them. Messages by channel *and* messages by author? Two tables, two writes per message — the application's own fan-out, [day 029](#)'s copies-with-owners again. What Cassandra will not give you: joins, ad-hoc filters, cross-partition transactions ([day 033](#)'s lightweight transactions are a single-partition compare-and-set, and expensive). The store is a set of pigeonhole walls, each built for its one question.

Two Cassandra failure modes worth naming

Hot partitions: partition by `celebrity_user_id` and one pigeonhole takes the whole flood — one node melts while the cluster idles. The fix is in the key: add a bucket to spread it (`(user_id, day)` or `(user_id, hash % 16)`), at the cost of collecting from several partitions on read — [day 035](#)'s hot-row arithmetic, at partition scale. **Unbounded partitions:** a partition that only ever grows (one channel, forever) slows reads and compaction as it fattens — same disease as [day 038](#)'s unbounded embedded array, same cure: bucket by time, `(channel_id, month)` . Both are landing-rule decisions; neither is fixable later by tuning.

The time-series engines, one line deeper

TimescaleDB: a Postgres extension that auto-partitions a normal table into time chunks — you keep SQL, joins against business tables, and one backup story; right when metrics live *next to* an application. **InfluxDB:** purpose-built, its own query language, tags-and-fields model; right when metrics *are* the application. **Prometheus:** pull-model scraping for infrastructure monitoring, local storage, PromQL — the operational default for watching services, less a general database than the monitoring stack's heart. All three share the machinery in §4: time-bucketed chunks, delta compression, retention by policy.

6 The numbers

The firehose, sized

```
10,000 servers × 100 metrics × every 10 s = 100,000 points/second
    × 86,400 s ≈ 8.6 billion points/day

as naive Postgres rows (~100 bytes each incl. index): ~860 GB/day
time-series compressed (~1.5-2 bytes/point): ~15-17 GB/day (~50×)
```

That 50× is delta encoding earning its keep: timestamps at fixed intervals compress to almost nothing, and slowly-drifting values to a few bits.

Retention arithmetic

```
raw forever:          15 GB/day × 365          ≈ 5.5 TB/year, growing
15 days raw + 5-min averages for a year:
  raw window:         15 × 15 GB                ≈ 225 GB
  downsampled: 10,000 × 100 series × 288/day × 365 × ~2 B ≈ 210 GB
  total                ≈ half a TB, FLAT
```

Retention is not housekeeping; it is the difference between a system that grows forever and one that plateaus. Say the two-tier policy in interviews — raw briefly, aggregates long.

Why per-node write throughput differs

```
B-tree write (Postgres): find the page, read-modify-write in place,
                        plus every index          → thousands-tens of
                                                thousands of rows/s/node
LSM write (Cassandra):  append to log + memtable → ~10,000-50,000+
                                                writes/s/node, and
                        nearly linear with nodes → millions/s per cluster
```

Day 031's read-favouring versus write-favouring shapes, now with product names attached. The honest counterweight: at 100,000 points a second, a *single* TimescaleDB node with batched inserts also copes — the cluster is for when one node's ceiling, or one node's failure, is the problem.

The query the wall answers

```
"CPU for server-42, last 6 hours, 1-min averages":
  one series, one or two chunks, ~2,160 points → a few ms

"which of 10,000 servers averaged > 90% yesterday":
  10,000 series × 8,640 points = 86M points scanned → seconds,
  UNLESS yesterday's 5-min rollups exist: 10,000 × 288 = 2.9M → sub-second
```

Downsampling is not only retention — it is precomputation, [day 037](#)'s prepare-once trade applied to aggregates.

7 The trade-offs

What the wall costs

Everything is decided at landing time: **queries must be known up front** (a new question means a new table and a backfill, not a new index), **copies multiply** (one table per query shape, all written by the application, all needing owners), and **the relational comforts**

are gone — no joins, no ad-hoc filters, transactions only within a partition. Against that: writes that scale with machines, reads that never fan out, and data that ages out for free.

Choosing within the family

Metrics beside an existing Postgres application, at thousands of points a second: **TimescaleDB** — one stack, SQL, joins to business data. Infrastructure monitoring: **Prometheus**, because the ecosystem (scraping, alerting, dashboards) is the product. Metrics as the product, or write volume beyond one node: **InfluxDB or Cassandra**, and the operational weight that comes with a cluster. The deciding questions are always the same three: points per second, queries known or ad-hoc, and who operates it.

I would not use it if...

I would not use Cassandra when the queries are still being discovered — every new question is a new table, and a young product asks new questions weekly; that is [day 036](#)'s relational home ground. **I would not use it below real scale** — a cluster's operational cost buys nothing that one Postgres node was not already doing at 10,000 writes a second. **And I would not put metrics in plain Postgres rows** — uncompressed points and per-row deletion lose $50\times$ storage and make retention a nightly `DELETE` that fights `VACUUM` ([day 033](#)'s bloat, self-inflicted); TimescaleDB exists precisely to keep Postgres and fix those two things.

The honest sentence

A wide-column store is a promise to the database: *I already know my questions*. Keep the promise and it serves them at any scale; break it and every new question is a schema project. The interview answer is knowing which promise the product can actually make.

8 In the interview

How it gets asked

- “Which database for metrics, and why?” — the hub question; they want the write rate, the query shape, and retention named before a product is.
- “What is Cassandra’s data model, actually?” — partition key places, clustering columns sort; one table per query.
- “Design Discord’s message storage.” — the canonical wide-column case study: partition by channel (bucketed), cluster by time.

- “Why not just Postgres?” — the counter-question; answer with numbers, and concede it below scale.
- “How do you handle a celebrity / a channel that never stops growing?” — hot and unbounded partitions; the fix is in the key.

What to say out loud, in the first ninety seconds

1. **Size the firehose first.** *“Ten thousand servers, a hundred metrics, every ten seconds — a hundred thousand points a second, 8.6 billion a day. That number picks the family.”*
2. **Name the two keys and their jobs.** *“Wide-column means: partition key decides where — machine and bucket; clustering columns keep rows sorted inside. The designed query reads one sorted slice of one partition.”*
3. **Say the promise.** *“The model requires knowing the queries up front — one table per query shape, no joins, no ad-hoc filters. Metrics keep that promise naturally: always ‘this series, this window’.”*
4. **Add the time-series specials.** *“A time-series store adds delta compression — 50× on metrics — and retention with downsampling: raw for 15 days, 5-minute averages for a year, so storage plateaus instead of growing.”*
5. **Give the decision.** *“Beside an existing app at modest volume: TimescaleDB. Infra monitoring: Prometheus. Metrics as the product at cluster scale: InfluxDB or Cassandra.”*

The follow-ups

“Design the message table for Discord. Walk me through the key.” The query is fixed and known: the latest N messages in a channel, older pages on scroll. So: partition by channel, cluster by a time-ordered id descending — the read is one partition’s top slice, in order, no sort at read time. Then the two failure modes force one refinement each. A channel lives for years, so the partition is unbounded — it fattens forever and compaction suffers — so the partition key becomes `(channel_id, time_bucket)`, say a month: reads usually touch one bucket and occasionally step to the previous one at a page boundary. And a huge public channel is a hot partition — one bucket taking a whole flood — which the time bucket already softens and a further split `(hash % k)` added to the key) can spread at the cost of merging k slices on read. Any second question — messages by author, search — is its own table or its own system, written at the same time; I would say that at design time, not discover it later. This is, concretely, how Discord stores messages, and the reasoning is the reusable part: query first, key second, then stress the key with “does it grow forever?” and “can one value take the flood?”.

“Why is Cassandra so fast at writes when Postgres isn’t? Same hardware.” Different promises, different physics. A Postgres write finds the row’s page in a B-tree, modifies it in place, and updates every index — random reads and writes, multiplied by day 030’s every-index-is-a-write rule, all to keep the data readable in any order at any moment. A

Cassandra write appends to a commit log and an in-memory table, and sorting into files happens later, in the background, by sequential merges — day 031's LSM shape: it defers and batches the expensive part. Add no-single-leader, so every node takes writes, and throughput scales with the cluster. The price is symmetrical and worth saying: reads must merge several sorted files, ad-hoc reads are poor, and compaction consumes background I/O forever. Postgres spent the write to make every future read cheap; Cassandra spends the read to make every write cheap — neither is free, and the workload picks the winner. Metrics and messages are write-dominated with narrow reads, so they pick the LSM.

“You said don’t use plain Postgres for metrics — but the team knows only Postgres. Now what?” TimescaleDB, and it is a genuinely good answer rather than a compromise. It is a Postgres extension: the metrics table *is* a Postgres table, so the team keeps SQL, joins to business data, backups and the operational knowledge they have — one stack. Underneath, it fixes exactly the two things plain rows get wrong: it auto-partitions by time into chunks, so retention is `drop_chunks` — deleting sealed chunks whole, no million-row `DELETE` fighting `VACUUM` — and its columnar compression gets metrics within sight of the purpose-built stores' 10–20×. Continuous aggregates give the down-sampling tier. Honest limits, also worth saying: one node's write ceiling (batched, hundreds of thousands of points a second — usually plenty), and clustering beyond that is where the purpose-built systems pull ahead. So the decision sentence is: below cluster scale, the team's stack wins; at cluster scale, the workload's stack wins.

A model answer

“Metrics first means sizing the firehose: say ten thousand servers, a hundred metrics each, every ten seconds — a hundred thousand points a second, 8.6 billion a day. Three properties follow: writes are append-only and enormous; reads are always ‘this series, this time range, aggregated’; and old data loses value on a schedule. The right store hard-codes all three.

That is a time-series database. It partitions by series and time bucket, so a query reads one sorted slice — Cassandra’s partition-and-clustering model, specialised. It delta-compresses — fixed-interval timestamps and drifting values go from a hundred bytes a point as database rows to under two, a $50\times$ difference that decides whether a year of data is terabytes or hundreds of gigabytes. And it retains by policy: raw points for fifteen days, five-minute averages for a year, dropped chunks deleted whole — storage plateaus instead of growing.

Which product depends on context. Metrics beside an existing Postgres application: TimescaleDB — it’s a Postgres extension, so the team keeps SQL and one operational stack, and it handles this write rate on a node. Infrastructure monitoring: Prometheus, because scraping, alerting and dashboards come with it. Metrics as the product, or volume past one node: InfluxDB, or Cassandra with a hand-built schema.

What I’d not do is plain Postgres rows — 860 gigabytes a day uncompressed, and retention as a nightly DELETE that fights VACUUM — and I’d not reach for a Cassandra cluster below the scale that needs one. The model’s price is the same everywhere: these stores require the queries to be known up front. Metrics keep that promise; a product still discovering its questions does not, and belongs relational.”

9 Recall card

- **Wide-column = two-part key:** partition key → which machine and bucket (hashed); clustering columns → sort order inside. The designed query reads one sorted slice; everything else fans out to the whole cluster.
- **One table per query, written at write time** — no joins, no ad-hoc filters; a new question is a new table, not a new index. The promise: *I already know my queries.*
- **Writes scale because of the LSM** — append, flush, background-merge (day 031), every node accepts writes: millions/s per cluster. Price: read merges and compaction forever.
- **Key stress-test: unbounded partitions (bucket by time) and hot partitions (spread the key)** — both are landing-rule decisions, unfixable by tuning later.

- **Time series = the model + compression + retention:** delta encoding $\sim 50\times$; raw briefly, downsampled aggregates long, chunks dropped whole. TimescaleDB beside an app; Prometheus for infra; InfluxDB/Cassandra at cluster scale.

DSA topic: Difference arrays: range updates, cheaply **System design topic:** Wide-column and time-series stores

Code these, in this order

Four problems, one mechanic: two marks per update, one settle. **Before each, ask Sarasu's question out loud** — is the end inclusive or exclusive, and therefore where does the minus go?

Before each one, ask:

1. Many range updates, one final report? (If reads interleave — wrong tool, say so.)
2. Inclusive end ($r + 1$) or exclusive end (r)?
3. How big is the coordinate space — dense array, or sorted map of marks?
4. Does the settle need the whole array, or just a running maximum / threshold check?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Range Addition	LeetCode 370 (Medium, premium — or code it from §5)	The pure pattern: two marks, extra slot, one settle.
2	Corporate Flight Bookings	LeetCode 1109 (Medium)	1-indexed inclusive ends — deriving <code>diff[first-1]</code> and <code>diff[last]</code> instead of memorising.
3	Car Pooling	LeetCode 1094 (Medium)	The exclusive end — off <i>at</i> the stop — and the settle doubling as a capacity check.
4	Number of Zero-Filled Subarrays	LeetCode 2348 (Medium)	A cooldown problem: no difference array at all — proof you route by structure, not by phase.

Problem 4 is a deliberate ringer: it says “subarrays” and “count”, and it wants a run-length walk, not today’s tool. Route it honestly.

On problem 1, size it wrong on purpose

Make `diff` exactly `length` long and apply an update ending at the last element. Collect the `IndexError: list index out of range`, then say why `length + 1` slots removes the crash and the guard at once.

On problem 2, derive the indices aloud

Flights are 1-indexed, ends inclusive. Start from the 0-based rule — plus at start, minus past the end — apply the shift, and watch the two off-by-ones cancel into `diff[last]`. Confirm `[[[1,2,10],[2,3,20],[2,5,25]], 5)` gives `[10, 55, 45, 25, 25]`.

On problem 3, run the handover test

`[[2,1,5],[3,5,7]]` with capacity 3 — two off at stop 5, three on at stop 5. Correct code says True. Now write the minus at `end + 1` and watch it say False. Say which convention this problem uses and which line of the statement told you.

On problem 3 again, the peak variant

Change the settle to track `peak = max(peak, riding)` and report the maximum riders instead of a yes/no. One line — say why the running level already had the answer.

The boundary drill

For each, say where the minus goes — `r` or `r + 1` — in under five seconds:

1. Add bonus marks to questions 3 through 7, inclusive.
2. Guests staying from day 2, leaving the morning of day 6.
3. Seats booked on flights 4 to 9, all flown.
4. A machine occupied from minute 10 until minute 25, freed at 25.
5. Discount applies from item index 0 to the last item.

The routing drill

Which tool — difference array, prefix + map (day 038), window (day 032/034), or plain prefix (day 037)? One sentence each:

1. Apply 10,000 range increments, then print the array.
2. Count subarrays summing to k, negatives allowed.
3. Answer 10,000 range-sum queries, array never changes.
4. Longest subarray with at most two distinct values.
5. Peak simultaneous meetings, given start and end times up to 10^9 .

Number 5 is today's tool in sparse clothing — say what replaces the dense array.

The two-keys drill

For each query need, write the Cassandra-style key — `(partition key)` then clustering columns — out loud:

1. Latest 50 messages in a channel.
2. A device's sensor readings for the last hour.

3. A user's orders, newest first — and the channel that never stops growing: what gets added to the partition key, and why?
4. All messages by one author across channels — and what does this question actually require?

The metrics drill

Answer each in one or two sentences, out loud:

1. Why do fixed-interval timestamps compress $\sim 50\times$, and which running trick from this phase is delta encoding?
2. Raw for 15 days, 5-minute averages for a year — what does this do to storage growth?
3. Why is retention-by-dropping-chunks cheap where a nightly `DELETE` is not?
4. When does TimescaleDB beat InfluxDB, and on what grounds?
5. What is a hot partition, and where does the fix live?

The arithmetic drill

From memory, in under two minutes:

- 10,000 servers $\times$ 100 metrics $\times$ every 10 s — points/second and points/day.
 - That day of points at ~ 100 bytes a row against ~ 2 bytes compressed.
 - 1,000 updates averaging 50,000 wide on a 100,000-element array — naive writes against marks-plus-settle, and the measured ratio from the lesson.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Apply many range increments and then report the final array.* The two marks and why the untouched middle is free, the extra slot, the settle as day 037's prefix sum, the boundary question, and $O(n + u)$ against $O(n \times u)$ with the measured ratio.
 2. *Which database would you pick for storing metrics, and why?* Size the firehose first, then the three properties it forces, the two-part key, compression and retention with numbers, and the TimescaleDB / Prometheus / InfluxDB decision by context.
 3. *What is Cassandra's data model, actually?* Partition key places — machine and bucket; clustering columns sort within; one table per query written at write time; the two key stress-tests — unbounded and hot partitions — and the promise the model demands.
-

Before you move on

- [] I can write the two marks and the settle from memory, with the `length + 1` slot.
- [] I ask the inclusive-or-exclusive question on every range problem, and I can name the two problems that answer it differently.
- [] I know the contract — all updates, one settle, then reads — and the log-n trees to name when it breaks.
- [] I can switch the dense array for a sorted map when coordinates are huge.
- [] I can say Cassandra's two keys and their jobs, and design a message table key under stress.
- [] I can size a metrics firehose and defend a retention policy with arithmetic.
- [] I answered all three questions above out loud.

2D prefix sums and inclusion-exclusion

DATA STRUCTURES AND ALGORITHMS

2D prefix sums and inclusion-exclusion

You can answer any rectangle-sum query in $O(1)$ and derive the four-term formula yourself.

SYSTEM DESIGN

Choosing SQL or NoSQL in an interview

You can make the choice out loud, with the access patterns as your evidence.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- Answer many rectangle-sum queries on a fixed matrix.
- SQL or NoSQL for this system, and what made you decide?

2D prefix sums and inclusion-exclusion

1 What this is, and why they ask it

A **2D prefix sum** extends [day 037](#) to matrices: precompute, for every position, the sum of the whole rectangle from the top-left corner down to it. Any rectangle's sum then comes from **four** stored values instead of two — big rectangle, minus the strip above, minus the strip to the left, **plus the corner that got subtracted twice**. That add-back is **inclusion-exclusion**, a counting principle that outlives this problem by decades of interviews.

Interviewers use Range Sum Query 2D — LeetCode 304 — as a derivation test. The 1D version can be memorised; the 2D formula, with its four terms and two signs, punishes memorisation the moment a nervous mind drops a term. What they want to see is the formula *rebuilt from a picture*: overlap counted twice, added back once. Show that and the code is eight lines; miss it and the wrong answer arrives with confident arithmetic. The same four-corner move powers image processing (summed-area tables), block-sum problems, and the 2D dynamic programming that arrives later in the course.

2 The story

The sports-day forms came back on Friday, and Nirmala, who has taught Class 8 for nine years, sat down after lunch to get the one number the sports teacher wanted: how many of her forty-two students are playing **neither** cricket nor football, because those are the ones he needs for the ground events.

She had two stacks of forms on the desk. Eighteen had ticked cricket. Fifteen had ticked football. So: forty-two, minus eighteen, minus fifteen — nine students free for ground events. She wrote 9 on the slip, and had actually stood up to send it before something itched.

Arjun. She had seen his form in the cricket stack — and she was almost sure she had seen it in her mind's eye in the football pile too, because the boy plays everything. She sat back down and went through properly. Seven forms had ticked **both** games.

And there was the itch, caught. When she subtracted the eighteen cricket players, Arjun went out once. When she subtracted the fifteen football players, Arjun went out *again*. Every both-games child had been thrown out twice — but each of them is only one

child, and only needs removing once. Her count of nine was seven short of the truth. The both-games seven had to come back in.

Forty-two, minus eighteen, minus fifteen, **plus seven**: sixteen students for the ground events. She checked it the slow way — went through all forty-two forms, counting the ones with no ticks — and got sixteen, and felt the particular satisfaction of a shortcut that survives an audit.

She explained it to the sports teacher in one line when he raised an eyebrow at the crossed-out 9: when you remove two overlapping groups, the overlap leaves twice, so you must invite it back once. He nodded slowly, the way people do when a small trick turns out to be a law.

3 The idea in plain English

Nirmala's crossed-out 9 is the bug every candidate writes. Her rule — *the overlap leaves twice, so invite it back once* — is inclusion-exclusion, and it is the entire content of today's formula.

The stored object

`prefix[r][c]` holds the sum of the rectangle from the matrix's top-left corner `(0, 0)` down to — but not including — row `r` and column `c`: the first `r` rows' first `c` columns. As in [day 037](#), the table is one bigger in each direction — a **sentinel row and column of zeros** along the top and left — so `prefix` for a 5×5 matrix is 6×6 , and rectangles touching the edges need no special case. Everything from yesterday-but-three carries over: entries live on *boundaries*, and the extra zeros are the reading before anything was driven.

Deriving the query, corner by corner

Wanted: the sum of the rectangle from `(r1, c1)` to `(r2, c2)`, both ends inclusive. Four stored rectangles, all anchored at the top-left corner, do it:

1. Start with everything down to the target's bottom-right: `prefix[r2+1][c2+1]`.
2. Remove the strip **above** the target: `prefix[r1][c2+1]`.
3. Remove the strip to the **left**: `prefix[r2+1][c1]`.
4. The top-left corner block — above *and* left — was inside both strips. It left twice; it is Arjun. Invite it back once: `+ prefix[r1][c1]`.

```
sum = prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1]
```


Never memorise the four terms; rebuild them. The picture in §4 takes ten seconds to draw, and the signs fall out of it: one big, two strips minus, one corner plus.

Building the table is the same formula, smaller

Each build step also uses inclusion-exclusion, on the three neighbours already computed:

```
prefix[r+1][c+1] = matrix[r][c] + prefix[r][c+1] + prefix[r+1][c] - prefix[r][c]
```

The rectangle ending at `(r, c)` is: this cell, plus the rectangle above, plus the rectangle to the left — and those two rectangles share their top-left overlap, counted twice, so it comes off once. Same law, pointing the other way: overlap *entered* twice, so it leaves once. If you can say both sentences — added twice therefore subtract, subtracted twice therefore add — the whole day is yours.

Inclusion-exclusion beyond rectangles

Nirmala’s form of the law is worth keeping in its own right, because interviewers ask it bare: *how many numbers below 100 are divisible by 3 or 5?* — $33 + 20 - 6$, the shared multiples of 15 counted twice. *How many strings avoid both patterns?* — total, minus each, plus both. Two overlapping groups: $|A \text{ or } B| = |A| + |B| - |A \text{ and } B|$. The rectangle formula is this law applied to strips, and the law generalises to three and more sets with alternating signs. One principle, many costumes — today it wears a matrix.

4 The picture

The four terms, drawn — this is the picture to rebuild in interviews:

target: rows r1..r2, cols c1..c2 (X = wanted cells)

	c1	c2
	-----+	
	A B	
r1	+-----+	
	C X X X X	
	X X X X	
r2	+-----+	

prefix[r2+1][c2+1] the big rectangle
= A + B + C + X

minus prefix[r1][c2+1] = A + B (strip above)
minus prefix[r2+1][c1] = A + C (strip left)

A left TWICE → plus prefix[r1][c1] = A

X = (A+B+C+X) - (A+B) - (A+C) + A

What to notice: every term is a rectangle anchored at the top-left corner — the only kind the table stores — and the algebra on the last line is Nirmala’s forms: A is Arjun, subtracted with the cricket stack and again with the football stack, restored once.

The build step, same law at cell scale:

prefix[r+1][c+1]:	+-----+---+	
this cell m	P Q	
+ above Q (includes P)	+-----+---+	Q = prefix[r][c+1] (above)
+ left R (includes P)	R m	R = prefix[r+1][c] (left)
- overlap P	+-----+---+	P = prefix[r][c] (their overlap, entered twice)

What to notice: the overlap `P` arrives inside both `Q` and `R` — *added* twice during the build, so subtracted once; in the query it is *subtracted* twice, so added once. The sign flips with the direction; the law does not.

The sentinel frame, sized:

```
matrix 5x5 → prefix 6x6
```

0	0	0	0	0	0	← sentinel row: sum of zero rows
0	.	.	.	.	.	
0	.	.	.	.	.	queries touching row 0 or col 0
0	.	.	.	.	.	subtract a stored, honest 0 –
0	.	.	.	.	.	no ifs, day 037's argument squared
0	.	.	.	.	.	

5 The code, built step by step

The build

```
rows, cols = len(matrix), len(matrix[0])
prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
```

The frame of zeros, both directions. Note the list-comprehension build — `[[0] * (cols + 1)] * (rows + 1)` would alias one row object `rows + 1` times, [day 016's](#) trap, still live.

```
for r in range(rows):
    for c in range(cols):
        prefix[r + 1][c + 1] = (matrix[r][c]
                                + prefix[r][c + 1]      # rectangle above
                                + prefix[r + 1][c]      # rectangle left
                                - prefix[r][c])         # their overlap, once
```

Row by row, each entry from three already-finished neighbours — the build order is why iterating top-left to bottom-right matters.

The query

```
def sum_region(r1: int, c1: int, r2: int, c2: int) → int:
    return (prefix[r2 + 1][c2 + 1]
            - prefix[r1][c2 + 1]          # strip above
            - prefix[r2 + 1][c1]         # strip left
            + prefix[r1][c1])           # corner back in
```

Four lookups, three additions. No loop, no matrix access, any rectangle size.

The complete solutions

```
class NumMatrix:
    """LeetCode 304. Build once in O(rows × cols); every rectangle in O(1)."""

    def __init__(self, matrix: list[list[int]]) → None:
        rows, cols = len(matrix), len(matrix[0])
        self.prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
        for r in range(rows):
            for c in range(cols):
                self.prefix[r + 1][c + 1] = (matrix[r][c]
                                             + self.prefix[r][c + 1]
                                             + self.prefix[r + 1][c]
                                             - self.prefix[r][c])

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) → int:
        p = self.prefix
        return (p[row2 + 1][col2 + 1]
                - p[row1][col2 + 1]
                - p[row2 + 1][col1]
                + p[row1][col1])

def matrix_block_sum(mat: list[list[int]], k: int) → list[list[int]]:
    """LeetCode 1314. For every cell: the sum of the (2k+1)-square around it,
    clamped at the edges – one 2D prefix table, then one O(1) query per cell."""
    rows, cols = len(mat), len(mat[0])
    prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            prefix[r + 1][c + 1] = (mat[r][c] + prefix[r][c + 1]
                                    + prefix[r + 1][c] - prefix[r][c])

    answer = [[0] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            r1, c1 = max(0, r - k), max(0, c - k)
            r2, c2 = min(rows - 1, r + k), min(cols - 1, c + k)
            answer[r][c] = (prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1]
                            - prefix[r2 + 1][c1] + prefix[r1][c1])

    return answer

if __name__ == "__main__":
    m = [[3, 0, 1, 4, 2],
          [5, 6, 3, 2, 1],
          [1, 2, 0, 1, 5],
          [4, 1, 0, 1, 7],
          [1, 0, 3, 0, 5]]
    nm = NumMatrix(m)
    print(nm.sumRegion(2, 1, 4, 3))    # 8
    print(nm.sumRegion(1, 1, 2, 2))    # 11
    print(nm.sumRegion(1, 2, 2, 4))    # 12
    print(nm.sumRegion(0, 0, 4, 4))    # 58 – the whole matrix, edges included

    print(matrix_block_sum([[1, 2, 3], [4, 5, 6], [7, 8, 9]], 1))
    # [[12, 21, 16], [27, 45, 33], [24, 39, 28]]
```

`matrix_block_sum` is the pattern's second gear: the prefix table built once, then a *loop of $O(1)$ queries* — 9 cells, 9 rectangle sums, one table. The clamping (`max / min`) is where the sentinel frame pays again: a block hanging off the edge clamps to a smaller rectangle and the formula just works.

6 What it costs

Build and query, counted

build: $\text{rows} \times \text{cols}$ cells, constant work each	→ $O(\text{rows} \times \text{cols})$ time
store: $(\text{rows} + 1) \times (\text{cols} + 1)$ entries	→ $O(\text{rows} \times \text{cols})$ space
query: 4 lookups, 3 additions, any rectangle	→ $O(1)$

Against the alternatives, at $1,000 \times 1,000$ with 100,000 queries

sum each rectangle by loops	: $100,000 \times \text{up to } 10^6 \text{ cells}$	$\approx 10^{11} \text{ ops}$
row-prefixes only (day 037)		
per row, sum $r2-r1+1$ rows)	: $100,000 \times \text{up to } 1,000 \text{ rows}$	$\approx 10^8 \text{ ops}$
2D prefix	: $10^6 \text{ build} + 100,000 \times 4$	$\approx 1.4 \times 10^6 \text{ ops}$

The middle row is worth knowing: per-row 1D prefixes are the halfway house a candidate invents under pressure — a real improvement, `0(rows)` per query, and still a thousand times short of the four-corner answer. Name it as the stepping stone, then step past it.

The number to have ready

One $O(\text{rows} \times \text{cols})$ build — a million cells, a million operations — then every rectangle costs four lookups. A hundred thousand queries: about 1.4 million operations total, against a hundred billion by re-summing. Same deal as 1D: memory and one pass, traded for $O(1)$ answers on a matrix that holds still.

7 The traps

The near-miss: forgetting the add-back

Drop the fourth term — the most common 2D error in existence:

```
return p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1]    # no + p[r1][c1]
```

```
0
```

On the §5 matrix, region `(2,1)-(4,3)` should be 8; this returns 0, because the corner block — worth exactly 8 here — left twice and was never invited back. The wrongness is data-dependent: regions touching the top or left edges have an empty corner and come out *right*, so casual tests pass and interior queries lie. **Rebuild the picture, read off the signs** — Nirmala’s nine stood up from the desk before the itch caught it.

The real error: no sentinel frame

Store the prefix table the same size as the matrix and query a region starting at row 0:

```
prefix = [...] # rows × cols, no zero frame
return prefix[r2][c2] - prefix[r1 - 1][c2] - prefix[r2][c1 - 1] + prefix[r1 - 1][c1 - 1]
```

With `r1 = 0`, `prefix[-1][...]` reads the **bottom row** — the same silent negative-index wrap as [day 037](#), now with two chances per query to trigger it. And the build without a frame needs three `if` guards per cell for its missing neighbours. The frame of zeros deletes all of it: build with no guards, query with no guards, wrong answers with no route in. Day 037’s sentinel argument, squared — literally.

The real error: the aliased rows

```
prefix = [[0] * (cols + 1)] * (rows + 1)    # one row object, many names
prefix[1][2] = 5
print(prefix[3][2])
```

```
5
```

Every “row” is the same list, so the build smears each row’s values over all of them and the table is garbage — [day 016](#)’s trap, and 2D prefix sums are where it classically resurfaces. Always the comprehension: `[[0] * (cols + 1) for _ in range(rows + 1)]`.

The near-miss: mixing the two index worlds

The formula lives in *prefix* coordinates `(r2 + 1)`, the clamping in *matrix* coordinates `(min(rows - 1, r + k))`. Blur them — clamp with `min(rows, r + k)` — and the bottom edge reads one row too many; blur the other way and every block loses its last row. The discipline that prevents it: convert **once**, at the formula’s brackets — every `prefix[...]` index is a matrix index plus one on the bottom-right terms, verbatim — and never let a `+ 1` drift anywhere else.

The contract corner: inclusive ends, again

LeetCode 304’s `(row2, col2)` is inclusive. [Day 039](#)’s question — last meal or last night — applies in two dimensions now, and an exclusive-end variant shifts two of the four `+ 1`s. Ask once, then anchor on the sentence from day 037, upgraded: *through the bottom-*

right, minus the strips before the top and left, plus the corner before both.

8 In the interview

How it gets asked

- “Answer many rectangle-sum queries on a fixed matrix.” — LeetCode 304, the direct form.
- “For every cell, the sum of the k -neighbourhood around it.” — LeetCode 1314; the table plus a loop of $O(1)$ queries, with edge clamping.
- “How many integers under N are divisible by a or b ?” — inclusion-exclusion bare, no matrix anywhere.
- “Count the submatrices whose sum equals k .” — the escalation: fix a pair of rows, collapse columns to a 1D array, and run [day 038](#)’s map — 2D prefix meeting prefix-plus-map, a genuinely hard follow-up worth recognising by name.

What to say out loud, in the first ninety seconds

1. **Name the shape and the trade.** “Many queries, matrix fixed — 2D precompute: $O(\text{rows} \times \text{cols})$ once, $O(1)$ per rectangle.”
2. **Define the stored object.** “ $\text{prefix}[r][c]$ is the sum of the first r rows’ first c columns — one size bigger each way, with a zero frame on top and left so edge queries need no cases.”
3. **Derive, don’t recite.** “Any rectangle: everything through its bottom-right, minus the strip above, minus the strip left — and the top-left corner came off twice, so it comes back once. That’s inclusion-exclusion; I’d rather rebuild it from the picture than trust memory on the signs.”
4. **Say the build uses the same law.** “Each build cell: value plus above plus left minus their overlap — added twice, subtracted once.”
5. **Give the numbers.** “Million-cell build, then four lookups per query — a hundred thousand queries land around 1.4 million operations against a hundred billion by re-summing.”

The follow-ups

“Prove the four-term formula — why exactly those signs?” Name the four disjoint blocks: A is the corner above-and-left of the target, B the strip directly above, C the strip directly left, X the target. The stored table only knows rectangles anchored at the origin, and four of them cover the situation: the big one is $A + B + C + X$, the above-strip is $A + B$, the left-strip is $A + C$, the corner is A. Then it is bookkeeping: start with $A + B + C + X$, subtract $A + B$, subtract $A + C$ — B and C are gone once each, correct, but A has been removed twice — add A back once and exactly X remains. The general law underneath is inclusion-exclusion — subtracting two overlapping sets over-subtracts their inter-

section once — and it is the same law the build step uses in reverse, where the above-rectangle and left-rectangle *include* their overlap twice, so it is subtracted. If I ever doubt the signs under pressure, I redraw the four-block picture and read them off; it is ten seconds and it cannot be misremembered.

“Now count the submatrices summing to exactly k.” Recognise it as two known tools composed. Fix a top row and bottom row — $O(\text{rows}^2)$ pairs. For a fixed pair, collapse the matrix between them into a 1D array of column sums, where entry c is the sum of column c from the top row to the bottom row — maintainable incrementally as the bottom row extends, or read from the 2D prefix table. Now “submatrices spanning these rows summing to k ” is exactly “subarrays of this 1D array summing to k ” — day 038’s prefix-plus-hash-map, $O(\text{cols})$ per row pair with the seen-map and its $\{0: 1\}$ sentinel, negatives welcome. Total: $O(\text{rows}^2 \times \text{cols})$, against $O(\text{rows}^2 \times \text{cols}^2)$ enumerating column pairs too — on a 200×200 matrix, eight million steps against 1.6 billion. The composition is the answer they want to hear: 2D prefix collapses a dimension, then the 1D machine finishes — the course’s tools stacking rather than a new trick.

“The matrix gets one cell update between queries. How bad is it?” One changed cell dirties every prefix entry below and to its right — from a corner cell, that is the entire table, so the honest cost is an $O(\text{rows} \times \text{cols})$ rebuild per update, and patching only the dirty region is the same order in the worst case. Day 037’s escalation, squared. The upgrade path mirrors 1D: a 2D Fenwick tree gives $O(\log \text{rows} \times \log \text{cols})$ for both point-update and rectangle-query — the four-corner query survives, each corner becoming a tree query — and heavier machinery (2D segment trees) exists beyond it. I would state the ratio rule before choosing: rare updates and floods of queries, rebuild and keep $O(1)$ reads; genuinely mixed traffic, 2D Fenwick; and say the implementation cost out loud, since a 2D Fenwick under interview pressure is a decision, not a reflex.

A model answer

“Many rectangle sums on a fixed matrix — a precompute shape. I’ll build a 2D prefix table: $\text{prefix}[r][c]$ is the sum of the first r rows’ first c columns, one size bigger than the matrix in each direction with a frame of zeros on the top and left, so rectangles touching the edges subtract an honest zero instead of needing special cases — the 1D sentinel argument, squared.

The build is one pass: each entry is its cell, plus the rectangle above, plus the rectangle to the left, minus their overlap — the overlap entered twice, so it leaves once.

For a query from $(r1, c1)$ to $(r2, c2)$ inclusive, I’d rather derive than recite: take everything through the bottom-right corner; remove the strip above the target; remove the strip to its left; and the block that is both above and left has now been removed twice, so it comes back once —

$$p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1]$$

That plus-term is inclusion-exclusion, and dropping it is the classic bug — edge-touching queries still pass, because their corner block is empty, and interior queries quietly lie.

Costs: $O(\text{rows} \times \text{cols})$ build and space — a million operations on a thousand-square matrix — then four lookups per query. A hundred thousand queries: about 1.4 million operations, against a hundred billion re-summing, and about a hundred million even with clever per-row 1D prefixes.

Two cautions from the build: construct the table with a comprehension, because multiplying a row list aliases one object across all rows and smears the build; and if updates arrive between queries, the deal changes — one cell dirties everything below-right, so it’s rebuild-if-rare or a 2D Fenwick tree at log-times-log, and I’d say which before coding either.”

9 Recall card

- **prefix[r][c] = sum of first r rows × first c columns** — table one bigger each way, zero frame top and left; build with `cell + above + left - overlap`.
- **Query = big – strip above – strip left + corner:** $p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1]$. Derive from the picture; never trust memorised signs.
- **The law: removed twice → add back once; added twice → subtract once.** Inclusion-exclusion — the same answer for “divisible by 3 or 5” with no matrix in sight.

- **Costs: $O(\text{rows} \times \text{cols})$ build, $O(1)$ per rectangle** — 1.4M ops for 100k queries where re-summing needs 10^{11} . Updates break the deal → rebuild-if-rare or 2D Fenwick.
- **Two build traps:** `[[0]*(c+1)]*(r+1)` aliases rows (day 016); no zero frame → `prefix[-1]` wraps silently (day 037, squared). Comprehension + frame, always.

1 What this is, and why they ask it

Fifteen days of database lessons converge on one question that every system design interview contains, stated or not: **which store, and why?** Today is not a new technology — it is the decision procedure that turns days 25 to 39 into an answer you can deliver in two minutes: list the access patterns, check the invariants, run the scale arithmetic, and only then say a product's name.

Interviewers ask it because it is the cheapest possible probe of engineering judgement. The failing answers are both fashionable: “*NoSQL, because it scales*” (a slogan where arithmetic should be) and “*Postgres, always*” (a reflex where analysis should be). The passing answer shows its working — and, crucially, shows *what evidence would change it*. This lesson gives you the four questions, a worked example, and the script. It is also the close of the phase's argument: [day 036](#) built the frame, [days 037–039](#) walked the families, and today you make the call.

2 The story

Ibrahim sells furniture from a shop near the flyover, and deliveries are half his reputation. He owns a scooter with a carrier, keeps an auto-driver on call, and knows a lorry man for the big jobs.

Customers, he says, always ask the wrong question first: *which is your fastest?* The scooter is the fastest. It is also the wrong answer for a dining table, and the lorry is the wrong answer for a table lamp — it would arrive tomorrow, cost ten times the scooter, and spend the trip mostly empty.

So Ibrahim never answers that question. Instead he asks three of his own, always the same three, in the same order. **What is going?** A lamp, a mattress, a full bedroom set — the size and shape of the load rules out most of the fleet before anything else is said. **Where is it going?** The lanes behind the mosque are too narrow for the lorry, full stop; the highway township is too far for the scooter. **And how is it packed?** Glass tops ride differently from steel frames.

Only after the three answers does he name a vehicle, and by then the choice mostly makes itself. His nephew, who helps on Sundays, once watched him send a single mattress on the auto and asked why not the scooter, which was parked right there. Because

the customer had also bought two chairs, Ibrahim said. You do not choose for the mattress. You choose for the whole order.

The part his nephew remembers now, years later, is what happened when a rival shop bought a second lorry and painted *FASTEST DELIVERY IN THE DISTRICT* on it. For a month, every big purchase went out on it, lamps included. Customers in the narrow lanes waited while the lorry found parking three streets away and a boy walked their parcel over. The rival had bought an answer, Ibrahim said, and was now going around fitting it onto questions.

Ask about the load. Ask about the route. Ask about the packing. The vehicle names itself.

3 The idea in plain English

Ibrahim's three questions are today's decision procedure. The rival's lorry is every team that chose a database from a slogan. And "you choose for the whole order" is the sentence that saves you from choosing a store for one table.

The four questions, in order

Question 1 — what are the access patterns? Before any product name: list what the system actually reads and writes. *Users log in; a feed is rendered; an order is placed; support searches by email; finance runs monthly reports.* Every question after this one is answered by this list — which is why the interviewer's real test is whether you *start* here. This is the load: what is going?

Question 2 — what must be true at all times? The invariants. Money that must not vanish between accounts, stock that must not go negative, a booking that must not double-sell — these are [day 033](#)'s cross-entity transactions and [day 026](#)'s constraints, and they are the relational deal's home ground. A domain woven from such rules — a **web**, in [day 038](#)'s language — wants tables. A domain of independent records — a **tree** — tolerates anything.

Question 3 — what do the numbers say? The arithmetic, done aloud, because this is where "NoSQL because scale" dies or survives:

```
writes/second, reads/second, storage over three years, working set
against day 036's line: one tuned Postgres node ≈ 10-50k simple TPS,
terabytes of data — and day 037's: 50M × 1 KB sessions = 50 GB of RAM
```

Most systems live *inside* the node. A path that measurably crosses the line — a firehose from [day 039](#), a session check from [day 037](#) — is a named, sized exception.

Question 4 — how settled are the questions? Day 039's promise test: Cassandra and friends require the queries known up front; a young product asking new questions weekly cannot keep that promise. Ad-hoc questions and analysts belong to the relational arrangement — Meena's kitchen, from day 036.

The shape of the answer: default plus exceptions

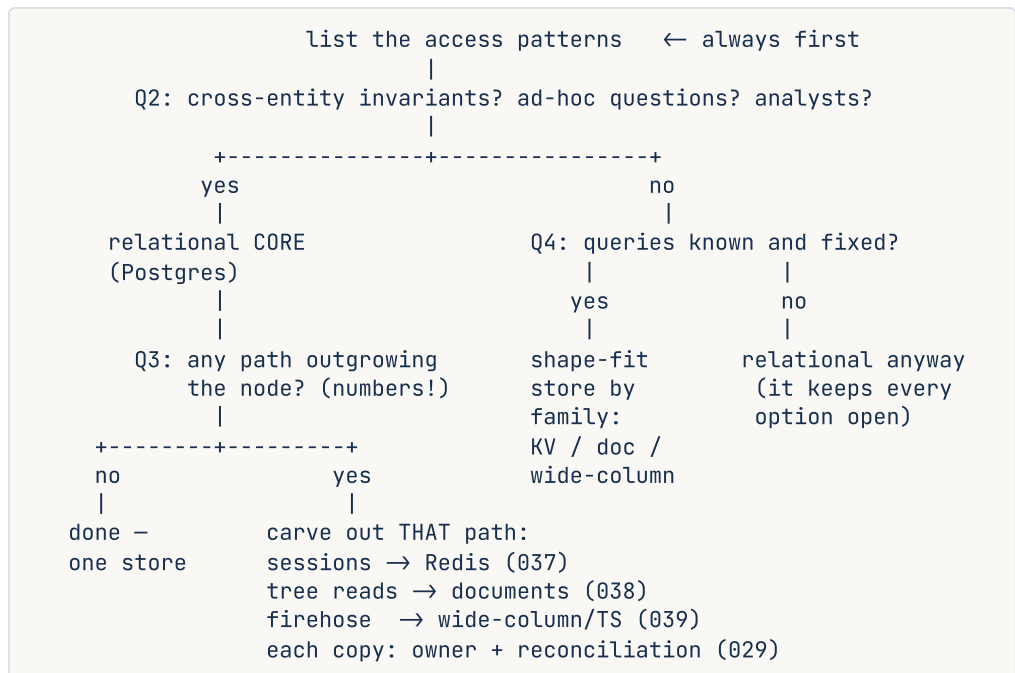
The procedure almost always lands in the same place, and saying it confidently is the win:

Postgres as the system of record, plus a specialised store per access path that measurably outgrows it — each with a named owner and reconciliation path.

Not “SQL versus NoSQL” — that framing is the rival's lorry. Real systems are polyglot by *path*: truth in the relational store, sessions on the day 037 shelf, the firehose in a day 039 store — each exception justified by a number, not a fashion. And the counterweight that marks senior judgement: **every additional store is an operational bill** — backups, monitoring, failure modes, a copy discipline — so the default for any path that has *not* proven itself is the store you already run.

4 The picture

The decision, as a shape you can run out loud in two minutes:



What to notice: a product name appears only at the leaves. Every failing answer names the product at the root.

The worked example from §5, as the final architecture:

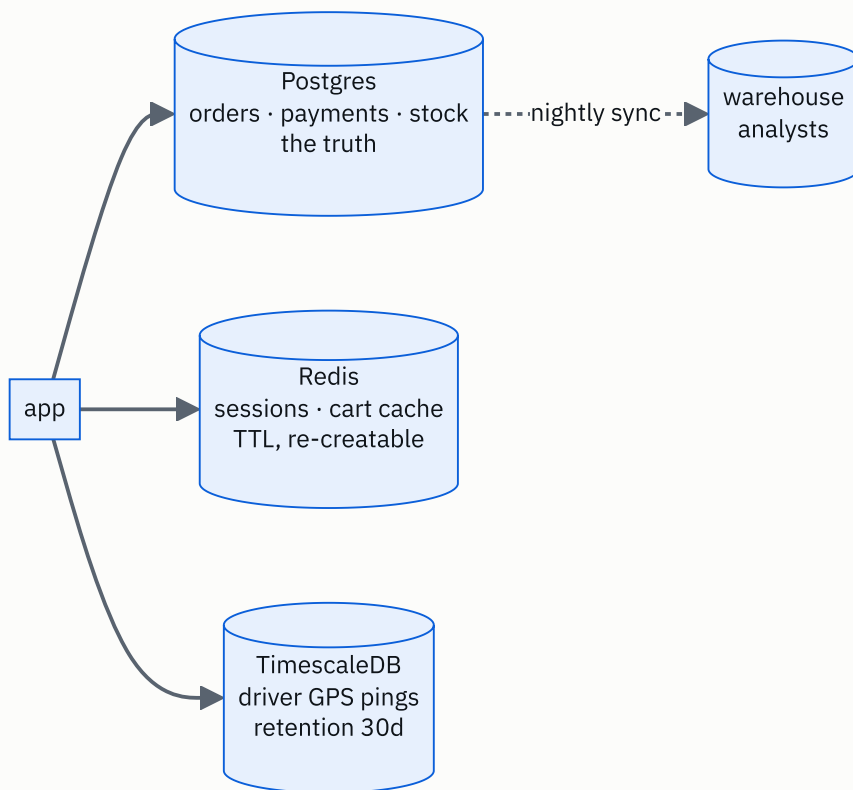


FIGURE 40.1

What to notice: three stores, and each edge has a *reason with a number* from §5 — none of them is there by fashion. The analysts get a copy, not access to the core.

5 How it actually works

The procedure, run honestly on one system: **a food-delivery app** — restaurants, menus, orders, riders, customers.

Step 1: the access patterns, listed

- | | |
|---|----------------------------|
| - customer browses restaurants and menus | (read-heavy, tree-shaped) |
| - customer places an order; pays | (multi-entity write) |
| - restaurant accepts; rider assigned | (state machine, contended) |
| - rider GPS pings every 3 s while delivering | (firehose, append-only) |
| - customer session checked on every request | (hot key-value read) |
| - support: "find this customer's orders" | (ad-hoc-ish) |
| - finance: monthly settlements per restaurant | (reports, joins) |

Step 2: the invariants

An order spans customer, restaurant, rider, payment and stock — pay only if placed, assign only if accepted, settle exactly once. That is a **web**: cross-entity transactions and constraints on every side. The core is relational, and no further debate is needed — this single step decides the centre of the design.

Step 3: the numbers, path by path

orders: 500,000/day $\approx$ 6/s average, $\sim$ 60/s peak	→ deep inside one node
menus: 50,000 restaurants $\times$ $\sim$ 50 KB = 2.5 GB	→ trivially relational (JSONB for the nested menu shape – day 038)
sessions: 2M active $\times$ 1 KB = 2 GB, checked at $\sim$ 20,000 reads/s	→ day 037 exactly: Redis, TTL, re-creatable
GPS: 50,000 active riders $\times$ 1 ping/3 s $\approx$ 17,000 writes/s, $\sim$ 1.5 B/day	→ the firehose: day 039 – time-series store, retention built in
queries: "this rider, this window"	
retention: 30 days	

Two paths cross the line; everything else stays home. Note the shape of the reasoning: the *number* carves the exception, not the product's reputation.

Step 4: the settledness

Order questions are still evolving (new offers, new report cuts) — relational keeps them open. GPS questions are fixed forever ("this rider, this window") — the wide-column promise is easy to keep there. The analysts get a nightly copy into a warehouse rather than queries against the core — [day 029](#)'s copy-with-an-owner, at system scale.

The delivery

Postgres core (orders, payments, restaurants, menus-as-JSONB); Redis for sessions and cart cache; TimescaleDB for pings with 30-day retention. Three stores, each with a number attached, each copy owned. That answer — with its working shown — is the whole lesson.

6 The numbers

The line, restated as a checklist

The calibration numbers this phase has built, in one place — these are the ones to say:

one tuned Postgres node	~10-50k simple TPS, terabytes on disk
one Redis node	~100k+ ops/s, RAM-bound (count × size!)
Cassandra/TS cluster	millions of writes/s, queries fixed up front
1 billion events/day	≈ 11,600/s average — INSIDE one node's range
50M × 1 KB	= 50 GB — RAM-shaped
10k servers × 100 metrics/10s	= 100k points/s — firehose-shaped

The cost of the exception

The arithmetic slogan-answers never do — what one extra store costs:

```

one more store = backups + monitoring + upgrades + a failure mode
                  + every copy needing an owner and reconciliation (day 029)
                  ≈ real engineer-days per month, forever

carve-out is justified when: (load it removes from the core) is measurable
                           AND the core would need scaling without it

```

The premature-scale check

```

startup, 10,000 users, "but we're planning for 100×":
today:      10k × 20 requests/day ≈ 2.3 req/s      — a laptop's load
at 100×:    230 req/s, orders ~6/s                — still one node
the honest trigger points, written down in advance:
  sessions to Redis    when session reads > a few thousand/s
  firehose to TS store  when appends > ~50k/s or retention hurts
  core partitioning    when TPS or working set outgrows the node

```

“Plan for scale” means writing the trigger numbers down — not paying the operational bill years early.

7 The trade-offs

The meta-trade: optionality against shape-fit

Relational keeps every question open and pays at read time; the specialised stores serve one shape brilliantly and slam the others shut. Early product: optionality wins, because the questions change weekly. Fixed, proven path: shape-fit wins, because the question will never change. The decision is really about **how much you know**, which is why access patterns come first — they are the measure of what you know.

The failure modes of each reflex

Choosing NoSQL by fashion: the rival’s lorry — ad-hoc questions impossible, invariants pushed into application code, and the analysts building a shadow relational copy within a year (they always do). Choosing Postgres by reflex and never carving out: the session

check eats the core's capacity, the firehose bloats it, and the eventual migration happens under fire instead of by plan. The procedure exists because *both* reflexes fail — just on different days.

I would not decide it if...

I would not answer “SQL or NoSQL” as asked — I would answer per access path, and say so. **I would not name a product before listing the patterns** — the order of the answer is the evidence of judgement. **And I would not carve out a path without a number and an owner** — a carve-out is a copy, and [day 029's](#) law has no exceptions: no reconciliation job, no copy.

The honest sentence

The strong answer is rarely a store; it is a *procedure* — patterns, invariants, numbers, settledness — landing on a relational core with named, numbered exceptions. Interviewers remember the candidate who said what evidence would change their mind.

8 In the interview

How it gets asked

- “SQL or NoSQL for this system?” — the direct form; answer with the procedure, per path.
- “What database would you use?” — inside every design round, usually early; buy time by listing access patterns first.
- “Why not MongoDB / why not just Postgres?” — the challenge form; concede the honest part, produce the number that decides.
- “Your traffic grows 100×. What breaks first?” — the trigger-points question; answer with the carve-out order and the numbers that fire each.

What to say out loud, in the first ninety seconds

1. **Refuse the binary, politely.** “I’d choose per access path rather than for the system — so let me list the paths first.”
2. **List the patterns.** “Reads: browse, feed, session-check. Writes: orders, payments, the GPS firehose. Plus support’s ad-hoc queries and finance’s reports.”
3. **Anchor the core on invariants.** “Orders span customer, payment, stock — cross-entity transactions and constraints — so the system of record is relational.”

4. **Carve exceptions with arithmetic.** *“Two paths outgrow the node: sessions at twenty thousand reads a second — Redis, TTL, re-creatable; GPS at seventeen thousand appends a second with fixed queries — a time-series store with thirty-day retention.”*
5. **Close with the discipline.** *“Everything else stays in Postgres until a number moves it — each carve-out is a copy with an owner, and each extra store is an operational bill.”*

The follow-ups

“The interviewer pushes: ‘if you had to pick exactly one store, which?’” Then Postgres, and the reasoning matters more than the name. It is the only member of the fleet that does a tolerable job of *every* path on the list: transactions and constraints for the order web, JSONB for the document-shaped menus, an unlogged or short-TTL table for sessions at this scale, even the ping firehose for a while with a Timescale extension or plain partitioned tables — day 025’s point that the relational store is the generalist. The specialised stores do not return the favour: Redis cannot hold the order invariants, Cassandra cannot serve the analysts, Mongo makes the settlement joins my application’s problem. Picking the generalist keeps every option open while the product discovers its questions — optionality is the asset a one-store constraint should maximise. And I would add the honest boundary: the single-store answer has trigger points written on it — the session read rate and the ping volume — and when they fire, the first carve-outs are exactly the ones I named before.

“Support says: ‘we chose MongoDB because we might scale.’ Interview-grade critique?” Two substitutions happened in that sentence, and naming them is the critique. “Might scale” substituted a hope for a number — the check is arithmetic: expected writes per second and working set against one node’s range, and a billion events a day is still inside it; “we might” almost never survives that division. And “MongoDB” substituted a product for an access pattern — Mongo’s actual strength is tree-shaped, read-as-a-unit domains, day 038’s lesson, which has nothing to do with scale; if the domain is order-shaped webs, Mongo at scale delivers the *worst* of both. The generous version of the critique, worth saying too: the instinct to think about scale is right, and the fix is cheap — keep the relational core, write the trigger numbers down, and pre-agree which paths carve out first when they fire. That converts a fashion decision into an engineering plan without buying the lorry today.

“How does this answer change at a big company versus a startup?” The procedure is identical; two inputs change value. Settledness: a startup’s questions churn weekly, so optionality dominates and the relational default hardens — almost nothing should carve out early, because every specialised store is a bet that a question stops changing. At an established company the paths are measured and settled — the feed’s read shape, the event firehose — so shape-fit stores are justified by data that actually exists, and the org can pay their operational bills with dedicated teams. Second, the cost of the exception: a

startup's three engineers cannot own four stores' failure modes — day 029's reconciliation discipline alone would eat them; a platform team amortises that across dozens of services. So the same four questions land differently: startups end with Postgres-plus-maybe-Redis and a list of trigger numbers; big companies end with the polyglot diagram — and the interview answer is knowing that *the procedure, not the diagram*, is the transferable part.

A model answer

“Rather than pick SQL or NoSQL for the whole system, I'll choose per access path — so first, the paths. Customers browse menus and place orders; payments settle; riders stream GPS pings every three seconds; every request checks a session; support asks ad-hoc questions; finance runs monthly settlement reports.

The core is decided by invariants, not load: an order weaves customer, restaurant, rider, payment and stock together — pay only if placed, settle exactly once. Cross-entity transactions and constraints are the relational deal, so the system of record is Postgres, with the nested menu shapes in JSONB rather than a second database.

Then the arithmetic, path by path. Orders: half a million a day is about six a second — deep inside one node; stays. Sessions: two million at a kilobyte is two gigabytes, read twenty thousand times a second — that's key-value shaped and re-creatable, so Redis with a TTL, taking that load off the core. GPS: fifty thousand riders pinging every three seconds is seventeen thousand appends a second, queries always 'this rider, this window', value fading in weeks — a time-series store with thirty-day retention. Finance and support get a nightly copy in a warehouse, not queries against the core.

So: a relational core and two carve-outs, each justified by a number, each copy with an owner and a reconciliation path — and everything else stays put until a measurement moves it, because every extra store is an operational bill. If our scale assumptions are wrong, the trigger points are written down: session reads in the thousands per second fire the Redis move, and append volume fires the time-series one. That's also what would change my mind.”

9 Recall card

- **Never answer the binary — answer per access path.** Patterns → invariants → numbers → settledness, and a product name only at the end.
- **Invariants pick the core:** cross-entity transactions and constraints = relational. Webs want tables; trees tolerate anything (day 038).

- **Numbers carve the exceptions:** one node $\approx$ 10–50k TPS / terabytes; 1B events/day $\approx$ 11.6k/s — inside it. Sessions $\rightarrow$ Redis (037); firehose with fixed queries $\rightarrow$ wide-column/TS (039).
- **Every carve-out is a copy with an owner (029) and an operational bill** — default to the store you already run; write trigger numbers instead of paying for scale early.
- **The winning close:** say what evidence would change your mind. The procedure is the answer; the diagram is just today's output.

DSA topic: 2D prefix sums and inclusion-exclusion **System design topic:** Choosing SQL or NoSQL in an interview

Code these, in this order

Four problems around one formula. **Before each, redraw the four-block picture** — in any tool you like, or trace it in the air — and read the signs off it. The picture is the answer; the formula is its transcript.

Before each one, ask:

1. Can I say the derivation? Big, minus strip above, minus strip left, plus the corner — why plus?
2. Where is the zero frame, and what size is the prefix table?
3. Which indices are matrix coordinates and which are prefix coordinates?
4. Is this a build question, a query question, or a loop-of-queries question?

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Range Sum Query 2D — Immutable	LeetCode 304 (Medium)	The build and the four-term query, with the zero frame doing the edge work.
2	Matrix Block Sum	LeetCode 1314 (Medium)	A loop of $O(1)$ queries with edge clamping — and keeping the two index worlds separate.
3	Count Square Submatrices with All Ones	LeetCode 1277 (Medium)	A ringer: it looks 2D-prefix-shaped and wants a different recurrence — route by structure.
4	Number of Submatrices That Sum to Target	LeetCode 1074 (Hard)	The composition: fix row pairs, collapse columns, run day 038's map on the 1D remainder.

Problem 3 is the deliberate misfit again: prefix sums *can* verify squares, but the clean solution is a different idea. Spend five minutes routing before committing.

On problem 1, drop the plus-term on purpose

Query an interior region without `+ p[r1][c1]` and watch a wrong answer arrive with confident arithmetic — on the lesson's matrix, region (2,1)-(4,3) returns 0 instead of 8. Then query a region touching row 0 and see it come out *right* anyway. Say why edge regions hide this bug and interior ones expose it.

On problem 1 again, alias the rows

Build the table with `[[0] * (cols + 1)] * (rows + 1)` and print two entries from different rows after one write. Same value. Name the day this trap first appeared, then rebuild with the comprehension.

On problem 2, police the index worlds

Say before coding: clamping happens in matrix coordinates `(max(0, r - k), min(rows - 1, r + k))`, the formula's `+ 1`s happen only inside the `prefix[...]` brackets. Confirm `([[1,2,3],[4,5,6],[7,8,9]], 1)` gives `[[12,21,16],[27,45,33],[24,39,28]]`.

On problem 4, name the composition first

One sentence before coding: “O(rows²) row pairs; for each, collapse to 1D column sums; then day 038’s seen-map with its `{0: 1}` sentinel counts subarrays summing to target.” If the sentence is right, the code is two tested pieces glued.

The derivation drill

Out loud, from the picture, under thirty seconds each:

1. The query formula, with why each sign.
2. The build formula, with why its sign points the other way.
3. Nirmala’s count: 42 students, 18 cricket, 15 football, 7 both — neither?
4. Integers under 100 divisible by 3 or 5.
5. Why does a query touching row 0 need no special case?

The decision-procedure drill

Run the four questions — patterns, invariants, numbers, settledness — out loud for each system, landing on a core plus carve-outs:

1. A hospital appointment system.
2. A multiplayer game’s match telemetry (every player action, replay and analytics).
3. A note-taking app with offline sync.
4. An ad-click counting pipeline (billions of clicks, revenue reports).

The challenge drill

One or two sentences each, out loud — the pushback answers:

1. “Just pick one store.” — which, and why that one?
2. “We chose MongoDB because we might scale.” — the two substitutions in that sentence.
3. “Your traffic grows $100\times$. What breaks first?” — the trigger numbers, in order.

4. “Why not put the analysts on the production database?” — whose copy, and what law applies?

The arithmetic drill

From memory, in under two minutes:

- $1,000 \times 1,000$ matrix, 100,000 rectangle queries: re-summing vs per-row 1D prefixes vs 2D prefix — the three totals.
 - A food-delivery app: 500k orders/day and 50k riders pinging every 3 s — which is inside one node and which is a firehose, with the per-second numbers.
 - 2M sessions $\times$ 1 KB at 20k reads/s — the store, the size, and what it takes off the core.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Answer many rectangle-sum queries on a fixed matrix.* The stored object with its zero frame, the derivation from the picture with both signs explained, the build as the same law reversed, and the 1.4M-versus- 10^{11} arithmetic.
 2. *SQL or NoSQL for this system, and what made you decide?* Refuse the binary; run the four questions on a system of your choosing; land on the core plus numbered carve-outs; close with what evidence would change your mind.
 3. *Why do you add the corner back?* Inclusion-exclusion in one breath: removed twice, restored once — then the same law in two other costumes: the build step, and the divisible-by-3-or-5 count.
-

Before you move on

- [] I can derive the four-term formula from the picture, signs included, without reciting.
- [] I can say why edge queries hide the missing-plus-term bug and interior queries expose it.
- [] I build 2D tables with a comprehension and a zero frame, reflexively.
- [] I can name the composition that solves submatrix-sum-to-target from two known tools.
- [] I can run patterns $\rightarrow$ invariants $\rightarrow$ numbers $\rightarrow$ settledness on an unseen system in two minutes.

- [] I have trigger numbers, not slogans, for when a path leaves Postgres.
- [] I answered all three questions above out loud.

Prefix sums revision and mock round

DATA STRUCTURES AND ALGORITHMS

Prefix sums revision and mock round

You can recognise a prefix-sum problem from the question alone.

SYSTEM DESIGN

Connection pools, ORMs, and the N + 1 query

You can spot an N + 1 query in code and fix it.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *Your page makes 200 database calls per request. What happened?*

1 What this is, and why they ask it

Days 37 to 40 built the precompute family: the prefix array and its sentinel, prefix-plus-map for exact sums with negatives, the difference array for cheap range updates, and the 2D table with inclusion-exclusion. Four tools, one instinct — **spend a pass now so questions cost nothing later** — and one shared border with the window family, patrolled by a single question about negative numbers.

Today is the phase's mock, and its skill is the one the hub names: **recognition from the question alone**. The window phase's tells were loud — “longest”, “substring”. This family's tells are quieter: *many queries, fixed data, exactly, negatives allowed, range updates, rectangle*. Interviewers rarely say “use prefix sums”; they describe a situation whose shape you must hear. The two problems in §5 are unseen, chosen because their surfaces point away from the tools that solve them — route them yourself, on the clock, before reading the transcripts.

2 The story

The mock board exam was in November, and Rekha, who has taught Class 10 mathematics for seventeen years, watched her best student walk out of it with a face like a burnt chapati.

Pranav. Ninety-eight in every class test. He had answered the paper in order, from question one, like a man eating a thali left to right without looking at the plate. Question four was a monster — he gave it twenty-five minutes and got it, too — and then did the last three questions, worth eighteen easy marks, in a panicked scrawl in nine minutes. Sixty-one.

The Monday after, she taught the lesson she teaches every batch, and it has nothing to do with mathematics. Take the first five minutes, she said, and do not solve anything. Read the whole paper with a pencil in your hand, and against every question, write one word. Not the answer — the **chapter**. This one is mensuration. This one is quadratic. This one *looks* like trigonometry but the triangle is a red herring — it is similar-triangles dressed up.

The class groaned. Five minutes, madam, gone, and nothing written.

Then she did her demonstration, the one that convinces every batch. She put a question on the board and gave them ten seconds — not to solve it, only to name the chapter. Most got it. Then she asked: now that you know the chapter, how much of the method do you already know? Nearly all of it, someone said. The chapter *is* the method, she said. What costs you twenty-five minutes is not solving — it is solving inside the wrong chapter, or discovering the right one at minute twenty.

Pranav sat the December mock with his pencil and his five minutes. Named every question’s chapter, did the eighteen easy marks first, saw through the dressed-up question because naming it was now a separate act from answering it. Ninety-one.

She tells them one more thing, every year. The examiners do not write the chapter’s name on the question. That is not cruelty. Naming the chapter **is** the examination.

3 The idea in plain English

Rekha’s chapter-naming is today’s whole skill: recognition is a separate act from solving, done first, with its own five minutes. Her dressed-up trigonometry question is §5’s second problem. And “the chapter is the method” is exactly true here — name the tool and the code is lines you already own.

The phase, on one table

TOOL	THE TELL IN THE QUESTION	THE INVARIANT TO SAY	DAY
Prefix array	many range-sum queries, data fixed	sentinel 0; <i>after the end minus before the start</i>	037
Prefix + map	“exactly k”, negatives possible, counting or longest	<code>seen[0] = 1</code> ; ask the map, then record	038
Difference array	many range <i>updates</i> , report at the end	two marks per update; settle once; inclusive → <code>r + 1</code>	039
2D prefix	rectangle sums, block sums	zero frame; big – two strips + corner	040

And the border post, staffed by one question: “**can the values be negative?**” All positive and at-most/longest → the window family keeps the case at `0(1)` space. Negatives, or “exactly” → this family takes it, at `0(n)` space, and saying the handover out loud is worth marks by itself.

The five bugs of the phase, as a pre-flight list

Every lesson this week ended in the same handful of failure shapes. Before declaring any prefix-family solution done, run the list:

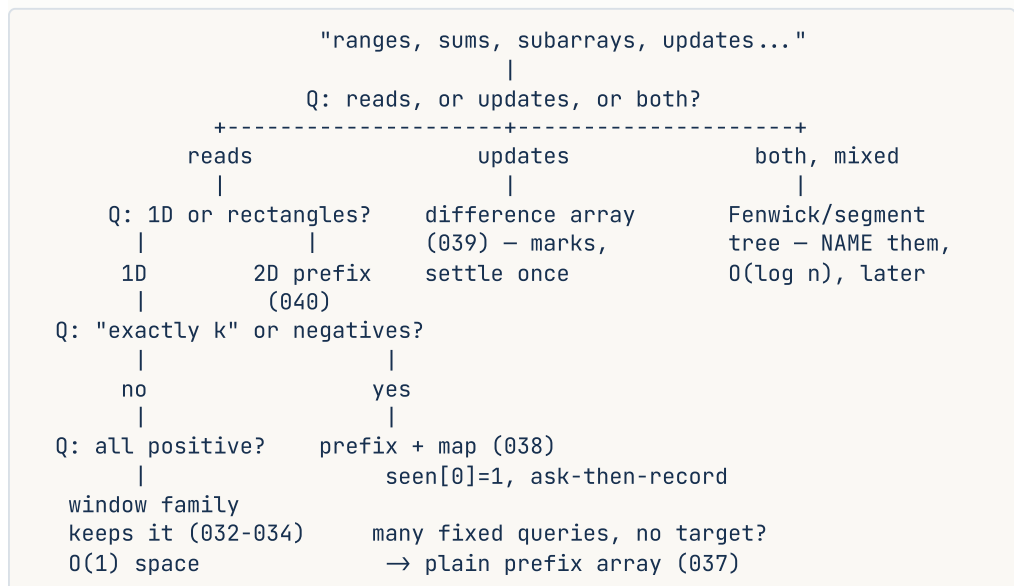
1. **The sentinel** — `initial=0`, `seen[0] = 1`, `{0: -1}`, the zero frame. Is the before-anything moment represented?
2. **The order** — ask the map, *then* record. (`k = 0` is the checking input.)
3. **The boundary** — inclusive or exclusive end? Minus at `r + 1` or `r`? Last meal, or last night?
4. **The map's contract** — count-map for “how many”, first-index map (never overwritten) for “longest”. Which am I keeping, and did I say so?
5. **The negative index** — can any subscript reach `-1`? Python will not crash; it will lie.

Running today's mock

The protocol is [day 036's](#), with one addition: thirty-five minutes a problem, standing, talking, nothing open — and **Rekha's pencil minute comes first**: before any solving, say the chapter. Which of the four tools, or the window family, or neither — and what tell decided it. Then the invariant, then code, then the five test inputs by voice, then the pre-flight list. Score yourself on the transcripts afterwards — on the naming, before the code.

4 The picture

The phase as one desk, with its border crossing:



What to notice: two exits again — the window border and the log-n trees — and both are *named*, not solved. Knowing where the family ends is part of knowing the family.

The mock clock, with Rekha's minute marked:

```

min 0-2    read twice; contract questions (negatives? ends inclusive? updates?)
min 2-3    NAME THE CHAPTER, out loud: tool + the tell that chose it
min 3-5    example by hand; brute force named with its cost
min 5-20   code, narrating; invariant before each loop
min 20-25  five test inputs by voice; then the five-bug pre-flight list
min 25-35  follow-ups; the escalation names (Fenwick, segment tree)

```

5 The code, built step by step

Two problems, worked as transcripts. Neither says “prefix” anywhere on its surface.

Match one — Product of Array Except Self

“Return an array where `answer[i]` is the product of every element except `nums[i]`. No division. $O(n)$.” — LeetCode 238.

Beat 1, clarify. *“Zeros allowed? Negatives? And division is banned even for the no-zero case?”* Yes, yes, yes — and the division ban is the routing hint in disguise.

Beat 2, name the chapter. *“Products, not sums — but the shape is day 037’s: `answer[i]` is everything before `i` times everything after `i`. Those are a prefix and a suffix. The chapter is prefix precompute, with multiplication as the running operation.”*

Saying *why division is banned* seals the naming: division is the subtraction of products — `total / nums[i]` is the 1D prefix trick run through an inverse — and zero has no inverse. §7 has the crash. The ban forces the two-directions form instead.

Beat 5, code — two sweeps, one output array.

```

answer = [1] * n
before = 1
for i in range(n):
    answer[i] = before      # product of everything left of i
    before *= nums[i]

```

“First sweep: `answer[i]` gets the running product of everything before `i` — the prefix, written directly into the output. `before` starts at 1, the empty product — the sentinel again, in multiplicative clothing.”

```

after = 1
for i in range(n - 1, -1, -1):
    answer[i] *= after      # times the product of everything right of i
    after *= nums[i]

```

“Second sweep, from the right: multiply in the running product of everything after i . Same idea, mirrored — and because `after` rides in one variable, the suffix array never exists: $O(1)$ extra space beyond the output.”

Beat 6, test. `[1,2,3,4]` → `[24,12,8,6]`. One zero: `[-1,1,0,-3,3]` → `[0,0,9,0,0]` — say why index 2 alone is non-zero. Two zeros: `[0,0]` → `[0,0]` — every position sees a zero on some side. Single element → `[1]`, the empty product, and worth saying before running.

Match two — Continuous Subarray Sum

“Does the array contain a subarray of length at least two whose sum is a multiple of k ?”
— LeetCode 523.

Beat 1, clarify. “Is zero a multiple of k ? So `[0, 0]` is True for any k ?” Yes — the contract corner that decides two test cases. “Values non-negative? k positive?” Say the questions even when the constraints answer them.

Beat 2, name the chapter — through the disguise. “Sum is a multiple of k is day 038’s remainder costume: two prefixes with the same remainder mod k bracket a stretch summing to a multiple of k . And it asks does one exist of length ≥ 2 — an existence-with-length question, so the map stores first index per remainder, day 038’s longest-contract, not the count.”

Beat 5, code.

```
first = {0: -1}          # remainder 0 before the array - index sentinel
running = 0
for i, x in enumerate(nums):
    running = (running + x) % k
    if running in first:
        if i - first[running] ≥ 2:
            return True    # stretch from first[running]+1 .. i, length ≥ 2
    else:
        first[running] = i
return False
```

“Three familiar pieces: the sentinel `{0: -1}` so a qualifying stretch from index 0 measures correctly; first-index-never-overwritten, because an earlier match can only make the stretch longer; and the length check `i - first[running] ≥ 2`. Note what the `else` protects: a repeat remainder that fails the length check must NOT update the map — §7 shows the input that punishes that.”

Beat 6, test. `[23,2,4,6,7]`, 6 → True (`[2,4]`). `[23,2,6,4,7]`, 13 → False. `[0,0]`, 1 → True — the contract corner. `[5,0,0,0]`, 3 → True. `[6]`, 6 → False — length one is not enough.

The complete solutions

```
def product_except_self(nums: list[int]) → list[int]:
    """LeetCode 238. Prefix and suffix products, two sweeps, no division."""
    n = len(nums)
    answer = [1] * n
    before = 1
    for i in range(n):
        answer[i] = before          # product of nums[0..i-1]
        before *= nums[i]
    after = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= after         # times product of nums[i+1..]
        after *= nums[i]
    return answer

def check_subarray_sum(nums: list[int], k: int) → bool:
    """LeetCode 523. Same remainder twice = multiple-of-k stretch between;
    first-index map + the length-2 rule."""
    first: dict[int, int] = {0: -1}
    running = 0
    for i, x in enumerate(nums):
        running = (running + x) % k
        if running in first:
            if i - first[running] ≥ 2:
                return True
        else:
            first[running] = i      # first occurrence only – never overwrite
    return False

if __name__ == "__main__":
    print(product_except_self([1, 2, 3, 4]))      # [24, 12, 8, 6]
    print(product_except_self([-1, 1, 0, -3, 3])) # [0, 0, 9, 0, 0]
    print(product_except_self([0, 0]))           # [0, 0]

    print(check_subarray_sum([23, 2, 4, 6, 7], 6)) # True
    print(check_subarray_sum([23, 2, 6, 4, 7], 13)) # False
    print(check_subarray_sum([0, 0], 1))           # True – 0 is a multiple
    print(check_subarray_sum([5, 0, 0, 0], 3))     # True
    print(check_subarray_sum([6], 6))             # False – length 1
```

6 What it costs

Match one

Two sweeps of n iterations, one multiplication and one write each — $2n$ operations. **$O(n)$ time.** Extra space beyond the output: two running variables — **$O(1)$** , and saying “beyond the output” is the precise phrasing the follow-up fishes for.

Match two

One sweep, one modulo and one map operation per element. $O(n)$ time, $O(\min(n, k))$ space — at most `k` distinct remainders can exist, so the map is bounded by `k` regardless of `n`. That sharper bound is [day 038](#)'s streaming observation made concrete, and producing it unprompted is a small flag of ownership.

The phase's costs, one line each

```
prefix array (037):  O(n) build, O(1)/query, O(n) space — data must hold still
prefix + map (038):  O(n) time, O(n) space — the space buys the negatives
difference (039):    O(1)/update + O(n) settle — no reads before settling
2D prefix (040):     O(rows*cols) build, O(1)/rectangle — four lookups
```

Four tools, one sentence of cost each — recitable in twenty seconds, and worth having at that speed.

7 The traps

The real error: division as the shortcut

The banned version of match one, on the input with a zero:

```
total = 1
for x in nums:
    total *= x
print([total // x for x in nums])
```

```
Traceback (most recent call last):
  File "day41.py", line 4, in <module>
    print([total // x for x in nums])
ZeroDivisionError: integer division or modulo by zero
```

And patching the zero cases breeds a three-branch swamp (no zeros, one zero, two-plus zeros) that is longer than the honest two sweeps. The ban is not arbitrary: division is the inverse that products do not always have — the same reason [day 039](#)'s multiplicative difference array needed invertibility. One principle, surfacing twice.

The near-miss: the overwritten first index

Move the map write out of the `else` in match two:

```
if running in first and i - first[running] ≥ 2:
    return True
first[running] = i          # overwrites on every step
```

```
bad: False      # on [5, 0, 0, 0], k = 3 - answer is True
bad2: False     # on [0, 0],          k = 1 - answer is True
```

Each repeat remainder that fails the length check now *drags the stored index forward*, so the gap can never reach 2. `[5, 0, 0, 0]`: remainder 2 repeats at every index, the stored index shadows `i` one step behind, and a True instance is walked straight past. Day 038's rule, now with an existence check attached: **first index means first, forever**.

The near-miss: the forgotten length rule

Return True on any repeated remainder and `[6], 6` comes back True — the single element 6, sum 6, “length at least two” violated. The subarray between two equal prefixes at indices `first[r]` and `i` has length `i - first[r]`; the rule is ≥ 2 , and deriving that length — rather than guessing > 1 versus ≥ 1 — is ten seconds of boundary arithmetic that prevents the bug class entirely. (And the index sentinel `{0: -1}` makes the derivation uniform for stretches from the start: $i - (-1) = i + 1$ elements.)

The contract corner: zero is a multiple

`[0, 0]` with any `k` is True — sum $0 = 0 \times k$. Candidates who never asked “is zero a multiple?” lose this case, and it is on every judge's test list precisely because the maths and the instinct disagree. Contract questions are free; this one is two test cases.

The mock trap, repeated on purpose

The silent stall from day 036 has a phase-specific variant: the candidate who recognises *nothing* and starts writing nested loops to stay busy. The recovery sentence exists here too: “the surface says products, but the shape is everything-before times everything-after — let me check whether the prefix idea transfers.” Naming a candidate chapter, even tentatively, restarts the round; busy hands do not.

8 In the interview

How it gets asked

- “Let's do two problems.” — the screening round; this phase supplies the quieter mediums.
- “Product of the array except self, no division.” — the transfer test: does your prefix instinct survive a change of operation?
- “Does a subarray of length ≥ 2 sum to a multiple of k ?” — the disguise test: remainder thinking under a divisibility costume.
- “Now the updates arrive between the queries.” — the escalation, every time this family appears: name Fenwick and segment trees, $O(\log n)$, and stop.

What to say out loud, in the first ninety seconds

Rekha's pencil minute, as a script:

1. **The contract questions.** *“Negatives possible? Ends inclusive? Is zero a multiple? Can the data change between queries?”* — four questions, ten seconds each, and two of them are routing questions in disguise.
2. **The chapter, named with its tell.** *“Everything-before times everything-after — prefix shape with multiplication” / “multiple of k means equal remainders — prefix-plus-map in the remainder costume.”*
3. **The map's contract, if there is a map.** *“Existence with a length rule, so first-index, never overwritten, sentinel $\{0: -1\}$.”*
4. **The brute force, priced.** *“Every pair of ends is $O(n^2)$; the prefix idea makes it one or two passes.”*
5. **The pre-flight promise.** *“Before I call it done I'll check the sentinel, the ask-record order, the boundary, the map contract, and any index that can reach -1 .”*

The follow-ups

“Why exactly is division banned in Product Except Self?” Because division is the trick's inverse operation, and the inverse does not always exist. The one-pass fantasy is: total product, then `total / nums[i]` per position — which is the prefix family's subtraction, transplanted to products. Subtraction always works, which is why range sums never faced this; division fails at zero, and floats add precision rot besides. One zero in the array makes the total 0 and every quotient meaningless — the crash is immediate — and handling it by cases (no zeros, one zero, more) yields three code paths, each wrong somewhere under pressure. The two-sweep form sidesteps inverses entirely: build the left products forward, ride the right products backward, multiply — every position pays two multiplications and no position ever divides. It is the same lesson day 039 taught about multiplicative range updates: the boundary-mark and running-total tricks are theorems about invertible operations, and when the inverse goes missing, you restructure the computation instead of forcing it.

“Your remainder solution — why store the first index and not the count, like in 560?” Because the two problems ask different questions of the same walk, and the map's contract follows the question. LeetCode 560 asks *how many* subarrays — every earlier matching prefix is one more answer, so the map must remember multiplicities: a count. This problem asks *does one exist with length at least two* — a single sufficiently-old match settles it, and the oldest match is the most likely to satisfy the length rule, so the map keeps each remainder's first index and never updates it: any later occurrence is strictly worse. The same fork appeared on day 038 between 560 and longest-subarray-with-sum-k. And the overwrite bug is the fork ignored: refresh the index on every repeat and the

stored position shadows the current one, the gap never reaches two, and `[5,0,0,0]` with $k = 3$ returns False against a plain True. Saying “count-map for how-many, frozen first-index for longest-or-exists” before coding is the one-sentence vaccine.

“Updates now arrive between the range-sum queries. Sketch your options.”

Three, in cost order, and the ratio between updates and queries picks one. Rare updates: keep the prefix array and rebuild on write — $O(n)$ per update, $O(1)$ per query, unbeatable when writes are once-a-day. Mixed traffic: a Fenwick tree — the prefix idea stored as a tree of partial sums, giving $O(\log n)$ point-update and $O(\log n)$ prefix-query; range sums stay two-prefix-subtraction; it is ~ 20 lines and the standard answer. Range *updates* with range queries, or operations beyond sums: a segment tree with lazy propagation — heavier, more general, $O(\log n)$ both ways. I would name all three with those costs, say the workload decides, and be honest that under interview time I can produce a Fenwick tree while a lazy segment tree is a design conversation — both structures get their full treatment later in this course, and at this point in a round the *naming with costs* is usually what was being tested.

A model answer

The pencil minute of match two, continuous and verbatim — compare your recording against it:

“Subarray of length at least two whose sum is a multiple of k . Two contract questions first: is zero a multiple of k — I’ll assume yes, so `[0,0]` is True — and are values non-negative? Either way, ‘multiple of k ’ is the tell: a stretch sums to a multiple of k exactly when the prefixes at its two ends leave the same remainder mod k . So this is the prefix-plus-map family in its remainder costume, from day 038.

It asks for existence with a length rule, not a count — so the map stores each remainder’s first index, never overwritten, because an earlier match only makes the stretch longer. The sentinel is remainder 0 at index -1 , so a qualifying stretch from the very start measures correctly. The stretch between indices `first[r]` and `i` has length $i - \text{first}[r]$, so the test is $i - \text{first}[r] \geq 2$ — I derived that rather than guessing the sign.

One subtlety I’ll flag before coding: when a repeated remainder fails the length check, I must NOT update the map — updating drags the first index forward and can walk straight past a true answer; `[5,0,0,0]` with $k = 3$ is the input that catches it.

Brute force is every pair of ends at $O(n^2)$. This is one pass, $O(n)$ time, and the map holds at most k remainders, so $O(\min(n, k))$ space. Tests before I run anything: the two examples, `[0,0]` True, `[6]` with $k = 6$ False for the length rule, `[5,0,0,0]` True for the no-overwrite rule.”

9 Recall card

- **Name the chapter before solving — it is a separate act.** The tells: many queries / exactly / negatives / range updates / rectangle — and the tool names itself.
- **The border question is “can values be negative?”** All-positive at-most stays with windows at $O(1)$ space; negatives or “exactly” cross to prefix-plus-map at $O(n)$.
- **Pre-flight, every time:** sentinel · ask-then-record · boundary (r vs $r + 1$) · map contract (count vs frozen first-index) · any index that can reach -1 .
- **Two transfer lessons:** products = prefix idea minus the inverse (no division — restructure as two sweeps); multiple-of- k = equal remainders, map bounded by k .
- **The escalation names, with costs:** rebuild-if-rare; Fenwick tree $O(\log n)$ point-update; segment tree + lazy for range-update-range-query. Name, price, stop.

1 What this is, and why they ask it

Today closes the database phase where most real slowness actually lives: not in the database, but in **how applications talk to it**. Three ideas, one lesson. A **connection pool** keeps a set of opened database connections and lends them out, because opening one is expensive and every request needs one. An **ORM** (object-relational mapper — Django's models, SQLAlchemy, Hibernate) translates between objects in code and rows in tables, writing the SQL for you. And the **N + 1 query** is the classic disease at their intersection: one query for a list, then one *more* query per item — 1 + N round trips where 2 queries would do — usually emitted invisibly by the ORM.

Interviewers love the hub question because it is a real incident, not a puzzle: every working engineer has met the page that makes two hundred calls. The passing answer diagnoses from the symptom (identical queries differing only in an id), names the cause (lazy loading in a loop), fixes it with the ORM's own tools (eager loading), and knows the pool arithmetic underneath. It is the most directly job-like question in the whole phase.

2 The story

The first time Suresh cooked for his in-laws, the meal was fine and the afternoon was a farce, and his wife still tells the story.

He had decided on five dishes and started confidently at noon. Ten minutes in, no mustard seeds — chappals on, down three floors, to the corner shop, back up. Started the dal. No tamarind. Chappals on, down again. The third trip was coriander, the fourth was curd, and when he went down the fifth time — matchsticks — the shopkeeper's boy laughed openly and asked whether he was buying the shop item by item.

Each trip was short. Seven or eight minutes, door to door. That was precisely the poison: no single trip felt bad enough to stop and think, so the afternoon leaked away four hundred rupees of time in eight-minute instalments. Five dishes, eleven trips. The cooking itself took less time than the stairs.

His mother-in-law, who has cooked for forty years and has opinions, delivered the verdict at dinner, kindly. The problem was not his cooking, she said. The problem was that he decided what he needed one ingredient at a time, at the moment he needed it. She plans differently: read all five recipes *first*, write one list, make one trip. Everything on the counter before the first flame.

And she added the detail Suresh thinks about most. In her building, the shop sends a delivery boy who knows her flat — but even so, she said, you do not send him eleven times. The boy is quick, but every trip is still the stairs, the road, the queue at the counter. The stairs do not get shorter because the boy is fast.

The next family lunch, Suresh read the recipes the night before, made the list on his phone, and went down once. His father-in-law, who notices things, remarked that the food arrived hot and the cook arrived calm — and that both, in his experience, tend to happen together.

3 The idea in plain English

Suresh's eleven trips are the $N+1$ query. The stairs are the network round trip. His mother-in-law's read-all-recipes-first is eager loading. And the delivery boy who is quick but still climbs the stairs is the milliseconds that fooled everyone: **short trips, many times, is still slow.**

The $N+1$ query, mechanically

The page lists 50 blog posts with their authors. The code reads naturally:

```
posts = Post.objects.all()[:50]           # query 1: the list
for post in posts:                         # queries 2..51: one per post
    print(post.author.name)
```

The ORM fetched the posts. Each `post.author` is a *different table*, not yet loaded — so the ORM, helpfully and silently, fetches it **on first touch**. That is **lazy loading**: the ingredient bought at the moment it is needed. In a loop, lazy loading becomes $1 + N$ queries — the list, plus one per item. Nothing errors. Every query is fast. The page is slow, and [day 032](#) already warned that this cause **never appears in a query plan** — every individual query planned perfectly.

Why 200 fast queries make a slow page

Each query pays the stairs before it pays the query:

```
per call: network round trip ~0.5 ms + parse/plan ~0.2 ms + the query ~0.3 ms
200 calls ≈ 200 ms — sequentially, because each result decides nothing;
the loop just hasn't asked for the next one yet
2 calls  ≈ 2 ms for the same data
```

The database is innocent — it answered 200 easy questions quickly. The *shape of the conversation* is the disease, and that is why the fix lives in the application.

The fix: read the recipes first

Every ORM has eager loading — the one-trip list, in two flavours:

- **Join it in** — one query bringing posts and authors together: Django's `select_related("author")`, SQLAlchemy's `joinedload` — right for to-one relationships (day 028's joins doing what joins are for).
- **Batch it in** — two queries: the posts, then *all* needed authors in one `WHERE id IN (...)`: Django's `prefetch_related`, SQLAlchemy's `selectinload` — right for to-many relationships, where a join would duplicate rows (day 028's row-multiplication warning).

Two queries instead of fifty-one. The loop code does not change — only the first line declares what the loop will need.

The connection pool: the delivery boy on retainer

Under all of this, every query needs a **connection** — and opening one is genuinely expensive: TCP, then TLS (day 006's handshake), authentication, and in Postgres a **whole operating-system process** per connection (day 008's processes, ~5–10 MB each). Five to ten milliseconds and real memory, per opening. No sane application opens one per query, or even per request: a **pool** opens, say, 20 connections at startup, lends one per request, takes it back at the end. Borrowing from the pool costs microseconds. When the fleet of application servers grows past what the database can host — `max_connections` defaults to 100 — a shared pooler in front (**PgBouncer**) multiplexes thousands of application connections onto tens of real ones.

4 The picture

The two conversations, drawn to scale:

N+1 (lazy loading in a loop)

```
app                                db
|---- posts? ----->|
|<--- 50 posts -----|
|---- author 7? ----->|
|<--- row -----|
|---- author 12? ----->|
|<--- row -----|
|      ... 48 more ... |
|                      |
51 round trips ≈ 50+ ms – each fast,
and the total is the stairs × 51
```

EAGER (batched)

```
app                                db
|---- posts? ----->|
|<--- 50 posts -----|
|---- authors IN (7,   |
|      12, 41, ...)?) ----->|
|<--- 34 rows -----|
|
2 round trips ≈ 2 ms
```


What to notice: the right side sends *the same information* in two envelopes. Nothing about the data changed — only the shape of the conversation.

The pool, and why it exists:

open a fresh connection:	borrow from the pool:
TCP handshake ~0.2 ms	take a lease ~0.01 ms
TLS handshake ~1-2 ms	run queries
auth + fork a ~2-5 ms	return the lease
process (5-10 MB)	
= 5-10 ms + memory, PER OPEN	the 20 connections were opened once, at application startup

500 app instances × 20 pooled each = 10,000 wanted connections
Postgres max_connections = 100 → PgBouncer in front:
10,000 app-side ----> [PgBouncer] ----> 50 real connections

What to notice: the pool solves the *per-request* cost; PgBouncer solves the *fleet* cost. They are the same idea at two levels — never pay the stairs when a lease will do.

5 How it actually works

What an ORM actually does

It maps classes to tables and objects to rows, tracks what you touch, and emits SQL. The translation is honest work — parameterised queries, escaping, migrations — and the danger is exactly its convenience: **the code stops looking like round trips.** `post.author.name` reads like a field access and costs a query. The engineer's job is not to abandon the ORM but to know what it emits — every serious ORM will show you (`django-debug-toolbar` , SQLAlchemy's `echo` , Rails' log), and the habit of glancing at the emitted SQL for any new page is the cheapest performance practice in the industry.

The N+1 signature, in the wild

In the query log or APM trace, the fingerprint is unmistakable:

```
SELECT ... FROM posts ORDER BY created_at LIMIT 50;
SELECT ... FROM users WHERE id = 7;
SELECT ... FROM users WHERE id = 12;
SELECT ... FROM users WHERE id = 41;
... 47 more, identical but for the id ...
```

Many identical statements differing only in a bound value, inside one request. `pg_stat_statements` — [day 032's](#) tool — shows the same truth aggregated: a trivial query with an enormous *call count*, ranked high by total time. Diagnosis is reading, not cleverness.

Eager loading, precisely

Django: `Post.objects.select_related("author")` adds a SQL `JOIN` — one query, right for foreign keys pointing to *one*. `Post.objects.prefetch_related("comments")` runs the second query with `WHERE post_id IN (...)` and stitches in memory — right for *to many*, avoiding join row-multiplication. SQLAlchemy mirrors them as `joinedload` and `selectinload`. Rails: `includes`. The deeper rule outlives every framework: **declare the shape of the read before the loop starts** — the recipes before the shopping.

Pool sizing, and the counterintuitive truth

Bigger pools are not faster. The database does useful work on roughly as many things as it has cores; beyond that, extra connections are queue positions with memory costs (day 008's context-switching, plus a process apiece in Postgres). The standard starting point (HikariCP's guidance) is around `cores × 2 + effective_spindle_count` — for an 8-core database, a pool near 20, not 200. When requests wait for a lease, the pool is not too small until the database is idle — check *its* CPU first; a bigger pool in front of a saturated database only lengthens the queue, day 035's arithmetic again.

PgBouncer's price

PgBouncer in **transaction mode** lends a real connection per transaction, which is how 10,000 app connections fit on 50 real ones — but between transactions you have no fixed connection, so **session state breaks**: session-level prepared statements, `SET` parameters, advisory locks held across transactions. Most ORMs have a compatible mode; the interview point is knowing the trade exists: the multiplexing is bought by giving up per-session state.

6 The numbers

The hub incident, priced

```
page renders 50 posts + author + comment count, lazily:
  1 + 50 + 50 = 101 queries × ~1 ms round trip ≈ 100+ ms in the database
  conversation alone – before rendering anything

eagerly: 3 queries (posts; authors IN; counts grouped) ≈ 3-5 ms
           ~25-30× on one code change
```

What N+1 does to the database fleet

```
that page at 200 req/s:
  lazy : 200 × 101 = 20,200 queries/s    → the database's capacity spent
  eager: 200 × 3   =   600 queries/s      on stairs, not on work
```

The same server that “needs sharding” at twenty thousand queries a second is idling at six hundred — **most “we outgrew Postgres” stories are conversation-shape stories**, which is why this lesson sits at the end of the phase, before the scaling chapters can be blamed.

Connection arithmetic

```
open-per-request at 200 req/s: 200 × ~5 ms opening = 1 full second of
  handshake work per second, plus 200 process forks — the database
  spends itself on greetings
pooled: 20 connections opened once; lease cost ~0.01 ms — noise

fleet: 500 instances × 20 = 10,000 wanted vs max_connections 100
  → PgBouncer: 10,000 app-side on ~50 real; memory saved ≈
    9,950 × ~7 MB ≈ 70 GB of process overhead that never exists
```

The pool-size sanity check

```
8-core database, queries averaging 5 ms:
  one connection ≈ 200 queries/s; ~16-20 busy connections saturate the cores
  a pool of 200 adds: 180 queue positions + 180 processes × 7 MB ≈ 1.3 GB
  and zero extra throughput
```

7 The trade-offs

The ORM bargain

Kept: productivity, safety (parameterised SQL by default), migrations, and code the whole team reads. Paid: the SQL becomes invisible, and invisible SQL breeds N+1s, accidental huge fetches (`SELECT *` of a row with a megabyte column), and queries no index can love. The senior position is neither worship nor abandonment: **use the ORM, read its output** — and drop to hand-written SQL for the five queries per system that are actually hot, which every ORM allows. I would not ban the ORM to prevent N+1s; I would turn on query logging in development and fail the test suite on query-count regressions — the guard rail that scales.

Eager loading is not “always”

Eager-load what the code path *will* use; eagerly fetching everything is its own disease — hauling comment bodies for a page that shows counts, joining five tables for two fields. The declaration-before-the-loop rule cuts both ways: declare what you need, and only that. **I would not `select_related` speculatively** — I would let the query log tell me what each page touches.

Pool and pooler

A per-instance pool is non-negotiable — open-per-request is never right. PgBouncer earns its place only when the fleet's connection count threatens `max_connections`: it is one more moving part (day 040's operational bill) and its transaction mode taxes session state. **I would not add PgBouncer at ten app instances** — arithmetic first: instances $\times$ pool size against the database's comfortable connection count.

The honest sentence

The database phase ends where it began on day 025: the database keeps its promises. When the page is slow with two hundred fast queries in it, the promise-breaker is the conversation — and the fix is one line that reads the recipes before the shopping.

8 In the interview

How it gets asked

- “Your page makes 200 database calls per request. What happened?” — the hub incident; diagnose, name, fix.
- “What is an $N+1$ query?” — the definition form; give the loop, the signature, both fixes.
- “Why do we need a connection pool?” — the opening-cost arithmetic, and Postgres's process-per-connection.
- “The app has 500 instances — what happens to the database's connections?” — the fleet form: the multiplication, `max_connections`, PgBouncer and its transaction-mode trade.
- “ORMs: good or bad?” — the maturity probe; the answer is a bargain, not a side.

What to say out loud, in the first ninety seconds

1. **Diagnose from the shape.** “Two hundred calls in one request is almost certainly $N+1$: one query for a list, then one per item — I'd confirm in the query log by looking for identical statements differing only in an id.”
2. **Name the mechanism.** “The ORM lazy-loads relations on first touch; touched in a loop, that's a round trip per element — each query fast, the sum slow, and none of it visible in a query plan.”

3. **Fix it in one line.** “Eager-load what the loop needs: a join for to-one (`select_related`), a batched `IN` query for to-many (`prefetch_related`) — two or three queries total, a $25\times$ on this page.”
4. **Add the guard rail.** “Then keep it fixed: query logging in development and a test that fails when a page’s query count jumps.”
5. **Check the floor underneath.** “And I’d confirm connections are pooled — opening is 5–10 ms and a whole Postgres process; at fleet scale that’s PgBouncer arithmetic.”

The follow-ups

“Each query takes one millisecond. Why is the page slow at all?” Because the two hundred milliseconds are sequential and none of them is the database working. Each call pays the stairs — network round trip, parse, plan — before the query proper, and the loop issues call 41 only after call 40 returns, since the code touches `post.author` one iteration at a time; nothing overlaps. So the request spends ~ 200 ms mostly *in transit*, while the database itself would happily have answered one `IN` query in three. That is also why this disease hides from every database-side tool: the plan for each statement is perfect, and `EXPLAIN` has nothing to confess — day 032 listed $N+1$ as one of the two slownesses that never appear in a plan. The general principle, worth saying because it transfers to every remote call in system design: **latency is paid per conversation turn, not per byte** — the fix is always to change the shape of the conversation, fewer and fatter turns, whether the far side is Postgres, Redis, or a microservice.

“How would you catch $N+1$ s before production, structurally?” Three layers, cheapest first. In development: query visibility on by default — `django-debug-toolbar` or `SQLAlchemy`’s `echo` — so the number sits in the developer’s face while the page is being built; most $N+1$ s die here, free. In CI: assert query counts on key endpoints — Django’s `assertNumQueries`, or a test middleware that fails a page whose count jumps from 4 to 104 — which converts the regression from a production incident into a red build. In production: APM traces (the waterfall of 51 identical spans is unmistakable) and `pg_stat_statements` sorted by call count — a trivial statement with millions of calls is an $N+1$ broadcasting its location. What I would not rely on is code review alone: the whole character of the bug is that the code reads innocently — `post.author.name` — and the reviewer sees a field access, not a query. The structural fixes work because they measure the emitted conversation, not the source’s appearance.

“When would you route around the ORM entirely?” For the handful of queries where the SQL is the design. Reports and analytics — multi-way joins, window functions, grouping — where the ORM’s rendition is either impossible or unreadable; hot-path statements worth hand-tuning against the plan, day 032 style; bulk operations, where a million ORM object instantiations lose to one `INSERT ... SELECT` or `COPY` by two orders of magnitude; and anything needing database-side arithmetic under concurrency — day

033's `SET balance = balance - 5000` — where object-think (read, mutate, save) is precisely the lost-update anti-pattern. Every mature ORM has the escape hatch (`raw()`, `text()`) and using it is not defeat; the codebase ends up ~95% ORM for shape and safety, 5% SQL where SQL is the better language. The signal I watch for in either direction: ORM contortions three method-calls deep trying to express a join — write the SQL; or hand-written SQL doing what `select_related` would have done — use the tool.

A model answer

“Two hundred calls per request is the N+1 signature: one query fetched a list, and then the loop touched a lazy relation per item — the ORM emits a fresh query on each first touch. I’d confirm in thirty seconds from the query log or an APM trace: it’ll show the same statement repeated with only the id changing.

Why it’s slow when every query is a millisecond: the cost is the round trip, not the query, and the loop pays it sequentially — two hundred turns of conversation for data that fits in two. The database side looks healthy throughout, which is why EXPLAIN finds nothing; this is one of the slownesses that never appears in a plan.

The fix is to declare the read’s shape before the loop: for a to-one relation like the author, a join — `select_related` in Django, `joinedload` in SQLAlchemy — and for to-many like comments, a batched IN query — `prefetch_related` or `selectinload`. That page goes from 101 queries to 2 or 3, and at 200 requests a second the database drops from twenty thousand queries a second to six hundred — which is often the difference between ‘we need to shard’ and ‘we’re fine’.

Then two guard rails so it stays fixed: query counts visible in development, and a CI assertion on key endpoints’ query counts so a regression fails the build instead of shipping.

And I’d check the layer underneath while I’m there: connections must come from a pool — opening one is five to ten milliseconds and a whole Postgres process, so per-request opening is a self-inflicted outage — sized around cores-times-two, not in the hundreds; and if we’re running hundreds of app instances, the multiplication against `max_connections` says whether PgBouncer belongs in front, with its transaction-mode trade on session state.”

9 Recall card

- **N+1 = one query for the list + one per item**, emitted by lazy loading in a loop. Signature: identical statements differing only in an id. Invisible to EXPLAIN — day 032’s warning.
- **Latency is paid per conversation turn:** 101×1 ms sequential beats the database at nothing; 2–3 eager queries carry the same data. Fix: declare the read before the loop — join for to-one (`select_related`), batched `IN` for to-many (`prefetch_related`).
- **Guard rails, not vigilance:** query counts visible in dev, `assertNumQueries` in CI, `pg_stat_statements` by call count in production.
- **Pool always:** opening = TCP + TLS + auth + a Postgres process (~5–10 ms, ~7 MB). Size near cores $\times 2$ — bigger pools are queue positions, not throughput.
- **Fleet arithmetic:** instances $\times$ pool vs `max_connections` → PgBouncer multiplexes (transaction mode trades away session state). Most “outgrew Postgres” stories are conversation-shape stories.

DSA topic: Prefix sums revision and mock round **System design topic:** Connection pools, ORMs, and the N + 1 query

Code these, in this order

A mock, so the protocol rules: **thirty-five minutes a problem, standing, talking, nothing open — and Rekha’s pencil minute first**. Name the chapter and the tell before solving anything. Score yourself against §5 of the lesson on the naming, before the code.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Find the Highest Altitude	LeetCode 1732 (Easy)	The warm-up: running prefix in one variable, sentinel 0 as the starting altitude.
2	Product of Array Except Self	LeetCode 238 (Medium)	Whether the prefix instinct survives a change of operation — and why division is banned.
3	Continuous Subarray Sum	LeetCode 523 (Medium)	The remainder disguise, the frozen first-index map, and the length-2 rule.
4	Range Sum Query 2D — Immutable	LeetCode 304 (Medium)	The phase’s four-corner summit, re-derived cold a day later.

On problem 1, respect the warm-up

One pass, one variable, track the maximum — and say where the sentinel is (the starting altitude 0 counts, and it can be the answer). If this takes more than ten minutes, the phase needs re-reading, not more problems.

On problem 2, crash the shortcut first

Write the division version and run it on `[-1, 1, 0, -3, 3]` — collect the `ZeroDivisionError: integer division or modulo by zero`. Then say the deeper reason the ban exists — subtraction always has an inverse, division does not — and write the two-sweep version. Say “O(1) extra space beyond the output” in exactly those words.

On problem 3, run the punishing inputs

After your solution passes the samples, run the four from the lesson: `[0, 0]` with $k = 1$ (True — zero is a multiple), `[6]` with $k = 6$ (False — length rule), `[5, 0, 0, 0]` with $k = 3$ (True — and the input that catches an overwritten first index), `[1, 0]` with $k = 2$ (False). If any surprises you, say which of the five pre-flight bugs it maps to.

On problem 4, derive from the picture again

No peeking at day 040: draw the four blocks in any tool, read off the signs, then code. The test of yesterday is whether the derivation — not the formula — survived the night.

The pre-flight drill

Say the five bugs of the phase from memory, each with its checking input or symptom:

1. The sentinel (which four costumes has it worn?).
2. Ask-then-record (which k checks it?).
3. The boundary — minus at r or $r + 1$ (which two problems answer differently?).
4. The map's contract — count or frozen first-index (which question picks which?).
5. The index that reaches -1 (what does Python do instead of crashing?).

The chapter-naming drill

Name the tool — or the exit — for each, in under ten seconds:

1. Answer 100,000 range-sum queries on a fixed array.
2. Count subarrays summing to k , negatives allowed.
3. Apply 10,000 range increments, then print the array.
4. Rectangle sums on a fixed matrix.
5. Longest subarray with sum at most k , all positive.
6. Range sums with updates arriving between queries.
7. Does a subarray of length ≥ 2 sum to a multiple of k ?
8. Product of the array except self, no division.

Numbers 5 and 6 are exits — name where each goes.

The $N+1$ drill

Answer each in one or two sentences, out loud:

1. What does the $N+1$ signature look like in a query log?
2. Why is 101×1 ms slow when the database is fast? (The sentence about conversation turns.)
3. Which eager-loading flavour for to-one, and which for to-many — and why the difference?
4. Name the three structural guard rails, cheapest first.
5. Why does $N+1$ never appear in a query plan, and which day taught that?

The pool drill

From memory, in under two minutes:

- What opening a Postgres connection costs — the three steps and the process — and what a pool lease costs instead.
- The pool-size starting rule for an 8-core database, and why a pool of 200 helps nothing.
- 500 instances $\times$ 20 connections against `max_connections = 100` — the tool, and the session state its transaction mode trades away.
- The hub page at 200 req/s: lazy versus eager queries per second, and the sentence about “we outgrew Postgres” stories.

The phase-closing drill

The database phase ends next lesson-block. Say the one-line cost of each tool from the phase’s recall cards — prefix array, prefix + map, difference array, 2D prefix — then the four-question store-choosing procedure from day 040, then the N+1 fix. Under three minutes, standing. That is the phase, portable.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Product of the array except self — no division allowed. Why not, and what instead?* The inverse argument, the zero crash, the two sweeps with the multiplicative sentinel, and the space claim said precisely.
 2. *Your page makes 200 database calls per request. What happened?* Diagnose from the shape, name lazy loading, fix with the right eager-loading flavour, add the guard rails, and check the pool underneath — the full model answer, compressed.
 3. *Does the array contain a subarray of length at least two summing to a multiple of k?* The remainder tell, the frozen first-index map with its `{0: -1}` sentinel, the derived length rule, and the input that punishes an overwritten index.
-

Before you move on

- [] I did all four problems with the pencil minute first, and scored the naming separately.
- [] I can recite the five pre-flight bugs with their checking inputs.
- [] I can name the chapter for all eight drill phrasings, including both exits.
- [] I can tell the N+1 story end to end: signature, mechanism, fix, guard rails.

- ☐ I can do the pool arithmetic — opening cost, sizing rule, fleet multiplication — from memory.
- ☐ I can deliver the phase-closing drill in under three minutes.
- ☐ I answered all three questions above out loud.

PART VI

Binary search

Days 42 to 50 · 9 chapters

Alongside, on the system design track:

Databases from zero, Object-oriented design

- 042** Binary search: the idea and the invariant
- 043** Writing binary search without off-by-one bugs
- 044** First and last occurrence
- 045** Search in a rotated sorted array
- 046** Binary search on the answer
- 047** Minimise the maximum: the capacity family
- 048** Binary search on floats, and the epsilon question
- 049** Peak finding, and searching data that is structured but not sorted
- 050** Binary search revision and mock round

Binary search: the idea and the invariant

DATA STRUCTURES AND ALGORITHMS

Binary search: the idea and the invariant

You can state, at every step, which half of the array still contains the answer.

SYSTEM DESIGN

Database revision and interview questions

You can answer schema, index, transaction and scaling questions about a database cold.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Search a sorted array. Why is it $O(\log n)$?*
- *Design and defend the database layer for an unseen product.*

Binary search: the idea and the invariant

1 What this is, and why they ask it

Binary search finds a value in a **sorted** list by looking at the middle element and throwing away the half that cannot contain the answer. Each look halves what is left, so a million elements are settled in twenty looks instead of a million. The whole method rests on one sentence you must be able to say at every step: *if the answer exists at all, it is still inside the stretch I am holding*.

They ask it because it is the shortest interview problem that most candidates get wrong. Everyone knows the idea; a large fraction cannot write it without an off-by-one bug or an infinite loop, and interviewers know that. It also opens the largest pattern family in this course — days 043 to 050 are all binary search wearing different clothes, and by [day 046](#) the thing being searched is not even an array. So today is not “learn a trick”. Today is where the idea and its guarantee get nailed down, because everything after leans on both.

2 The story

Shobha is going to a wedding at half past eleven, and the address on the invitation says house number 214, Second Cross Lane. She gets off the bus at the top of the lane at ten to eleven, in a silk sari and the wrong shoes, holding a gift box with both hands.

The lane is long. It runs straight for nearly a kilometre and there are gates down both sides, four hundred of them at least, and the numbers climb steadily as you walk. The first gate says 2. She cannot see the far end.

The obvious thing is to start walking and read every gate until 214 comes up. She has done that before, in another lane, in the rain, and it took twenty minutes.

So she does something else. She walks to roughly the middle of the lane — a stretch of shuttered shops, about five minutes — and reads the nearest gate. It says 260. That one number tells her something large: 214 is *behind* her. Everything in front of her, half the lane, is finished with. She does not have to look at any of it again.

She turns and walks back, but not all the way. She goes to the middle of what is left, between the top of the lane and where she is standing, and reads that gate. 130. Too small. So 214 is ahead of this gate and behind the one that said 260. She now knows two

things at once, and she says them to herself: after 130, before 260.

She keeps doing it. Halfway between: 196. Too small — after 196, before 260. Halfway: 228. Too big — after 196, before 228. Halfway: 212. Then the next gate but one is 214, with a shamiana out front and the sound of a nadaswaram.

Six stops. Not two hundred.

One stretch nearly ruined it. Near the school there is a block a builder renumbered, where 88 sits between 300 and 74. Where the numbers climb steadily, the trick works. Where they jump about, it is worth nothing, and Shobha has learnt to check that before she starts.

3 The idea in plain English

Shobha's lane is a sorted array. Her two remembered gates — *after 130, before 260* — are the two variables the code keeps. And the renumbered block near the school is the precondition: **the trick requires order, and gives nothing without it.**

The setup

Take a seven-element sorted array and look for the value 23:

```
nums = [3, 9, 14, 23, 31, 42, 55]
target = 23
```

Sorted means: reading left to right, the values never go down. That is the only thing binary search needs. It does not need them evenly spaced, it does not need them positive, and it does not need them to be numbers at all — only that “less than” means something and the list respects it.

You keep two variables:

- `low` — the leftmost index that could still hold the answer.
- `high` — the rightmost index that could still hold the answer.

They start at the two ends: `low = 0`, `high = 6`. The stretch `nums[low..high]` is called the **search space** — the part of the array you have not ruled out yet. At the start it is everything.

The invariant

An **invariant** — a word you met on [day 028](#) — is a statement that is true before the loop starts, stays true after every pass, and is therefore true when the loop ends. Binary search has exactly one, and it is the whole lesson:

If `target` is anywhere in the array, its index is between `low` and `high`, both included.

Say it out loud. Every line of the code exists to keep it true. When it stops being true, the search breaks; when it stays true and the space empties, the honest conclusion is that the target was never there.

One step

Look at the middle of the current stretch:

```
middle = (low + high) // 2
```

`//` is integer division, so $(0 + 6) // 2 = 3$. Now compare `nums[3]`, which is 23, with the target 23. Three things can happen, and only three:

- `nums[middle] == target`. Found it. Return `middle`.
- `nums[middle] < target`. The middle value is too small. Because the array is sorted, *everything at or to the left of* `middle` is also too small. None of it can be the answer, so the new search space starts one past the middle: `low = middle + 1`.
- `nums[middle] > target`. The middle is too big, so is everything to its right, and `high = middle - 1`.

The `+ 1` and the `- 1` are not decoration. The middle has been checked; leaving it in the space is how infinite loops happen, and §7 shows exactly that.

When to stop

The loop runs while `low ≤ high` — while the stretch still holds at least one index. If `low` ever passes `high`, the stretch is empty, the invariant says the target would have been inside it, and therefore the target is not in the array at all. Return `-1`.

That is the part people skate over. The failure case is not a guess; it is the invariant reporting a fact.

Why it is fast

Each pass throws away half. Starting from 7 elements: $7 \rightarrow 3 \rightarrow 1 \rightarrow 0$. Starting from a million: $1,000,000 \rightarrow 500,000 \rightarrow 250,000 \rightarrow \dots \rightarrow 1$, which takes twenty steps. The number of halvings needed to get from `n` down to 1 is the **base-2 logarithm** of `n`, written $\log_2 n$ — the growth curve you met on [day 004](#). That is where **$O(\log n)$** comes from, and in an interview you say it by counting halvings, not by naming the curve.

4 The picture

The array, and the three variables that matter:

index	0	1	2	3	4	5	6
value	3	9	14	23	31	42	55
	^			^			^
	low=0		middle=3		high=6		
search space = nums[0..6], everything. target = 42.							

What to notice: `middle` is not a third traveller. It is recomputed from `low` and `high` on every pass and forgotten immediately after.

Searching for 42, pass by pass:

```
pass 1  low=0 high=6 middle=3  nums[3]=23 < 42 → go right, low = 4
        [ 3   9   14  23 ]  31  42  55
        ^^^^^^^^^^^^^^^^^^
                        all thrown away, never read

pass 2  low=4 high=6 middle=5  nums[5]=42 = 42 → found, return 5
        31 [ 42 ]  55

two comparisons for seven elements.
```

What to notice: in pass 1 the four discarded cells were never visited. That is what “throwing away half” costs — nothing.

And the failure case, searching for 40:

```
pass 1  low=0 high=6 middle=3  nums[3]=23 < 40 → low = 4
pass 2  low=4 high=6 middle=5  nums[5]=42 > 40 → high = 4
pass 3  low=4 high=4 middle=4  nums[4]=31 < 40 → low = 5

now low=5, high=4. low > high, the stretch is empty. 40 is not here.
```

What to notice: `low` ends up sitting exactly where 40 *would* go if you inserted it. That is not a coincidence, and [day 043](#) turns it into a tool.

5 The code, built step by step

The two ends

```
low, high = 0, len(nums) - 1
```

`high` is the last **valid index**, not the length. `len(nums)` is 7; the last index is 6. Getting this wrong is the most common first-line mistake, and it costs you an `IndexError` on the very first pass when the target is large.

The loop condition

```
while low ≤ high:  
    ...
```

`≤`, not `<`. When `low = high` the stretch still holds one index, and that one index might be the answer. Stopping at `low < high` skips it — a bug that is invisible on most inputs and fires on single-element searches. §7 shows it failing.

The middle, and the three-way comparison

```
middle = (low + high) // 2  
if nums[middle] == target:  
    return middle  
if nums[middle] < target:  
    low = middle + 1  
else:  
    high = middle - 1
```

Read the two assignments as sentences. `low = middle + 1` says “everything up to and including the middle is too small, so the space now starts after it”. `high = middle - 1` says the mirror image. Both shrink the space by at least one element, which is what guarantees the loop ends.

The empty answer

```
return -1
```

Reached only when `low > high` — the space is empty. By the invariant, the target was never in the array.

The complete solution

```
def binary_search(nums: list[int], target: int) → int:
    """Index of target in the sorted list nums, or -1 if it is not there.

    Invariant: if target is in nums, its index is in nums[low..high].
    """
    low, high = 0, len(nums) - 1

    while low ≤ high:
        middle = (low + high) // 2      # stretch still holds at least one index
        if nums[middle] == target:      # integer division: floor of the midpoint
            return middle
        if nums[middle] < target:
            low = middle + 1            # middle and everything left of it is too small
        else:
            high = middle - 1           # middle and everything right of it is too big

    return -1                           # low passed high: the space is empty

if __name__ == "__main__":
    nums = [3, 9, 14, 23, 31, 42, 55]
    print(binary_search(nums, 42))      # 5
    print(binary_search(nums, 3))       # 0 ← first element
    print(binary_search(nums, 55))      # 6 ← last element
    print(binary_search(nums, 40))      # -1 ← absent, between two present values
    print(binary_search(nums, 1))       # -1 ← absent, below everything
    print(binary_search(nums, 99))      # -1 ← absent, above everything
    print(binary_search([], 5))         # -1 ← empty input, no crash
    print(binary_search([7], 7))        # 0 ← single element, found
    print(binary_search([7], 8))        # -1 ← single element, absent
```

Run it. The last four lines are the ones worth watching: an empty list must not crash, and a one-element list must be able to answer both ways. Those three inputs catch most beginner versions.

The recursive form, and why the loop is preferred

```
def binary_search_rec(nums: list[int], target: int, low: int, high: int) → int:
    if low > high:
        return -1
    middle = (low + high) // 2
    if nums[middle] == target:
        return middle
    if nums[middle] < target:
        return binary_search_rec(nums, target, middle + 1, high)
    return binary_search_rec(nums, target, low, middle - 1)
```

Identical logic, and some interviewers ask for it. It costs $O(\log n)$ **stack space** — each call waits for the one below it — where the loop costs $O(1)$. Say that trade if you write this version. Recursion gets its own phase from [day 087](#); today, prefer the loop.

In an interview you would use this

```
from bisect import bisect_left

def binary_search_lib(nums: list[int], target: int) → int:
    i = bisect_left(nums, target) # first index where target could go
    return i if i < len(nums) and nums[i] == target else -1
```

`bisect_left` returns the **insertion point**: the leftmost index where `target` could be placed while keeping the list sorted. It does not tell you whether the value is there, which is why the check after it exists. Write the loop when asked to write binary search; reach for `bisect` when binary search is a step inside a larger problem. [Day 043](#) is entirely about that insertion-point idea.

6 What it costs

Time, counted by halving

The search space starts at `n` and at least halves every pass:

pass:	0	1	2	3	4	...
size:	n	n/2	n/4	n/8	n/16	...

The loop ends when the size reaches 0, so the number of passes is the number of halvings that take `n` to 1, plus one. That count is $\log_2 n$. Each pass does a fixed amount of work — one addition, one division, one or two comparisons, one assignment — so the total is $O(\log n)$.

Put real numbers on it, because the interviewer wants the number, not the letter:

n = 1,000	10 comparisons	($2^{10} = 1,024$)
n = 1,000,000	20 comparisons	($2^{20} \sim 1.05$ million)
n = 1,000,000,000	30 comparisons	($2^{30} \sim 1.07$ billion)
n = 8,000,000,000	33 comparisons	(every person alive)

Thirty-three looks to find one person among eight billion, sorted. Against [day 012](#)'s linear search, which averages four billion. The sentence to have ready: **every doubling of the input adds one comparison.**

Space

low, high, middle: three integers, whatever n is → $O(1)$ extra space

The iterative version allocates nothing. The recursive version holds one stack frame per pass, so $O(\log n)$ — thirty frames at a billion elements, which is nothing, but it is not $O(1)$ and saying so is free marks.

The cost that is not in the loop

Binary search needs a sorted array. If the array arrives unsorted, sorting it costs $O(n \log n)$ — more than a single linear scan. So:

```
one search on unsorted data : scan it,  $O(n)$ . Sorting first is a loss.
many searches on fixed data : sort once,  $O(n \log n)$ , then  $O(\log n)$  each.
                             A win by the second or third search.
```

That is [day 037](#)'s trade again, in new clothes: **pay a preparation cost once, buy cheap answers forever**. Say it that way and you have connected two phases in one sentence.

7 The traps

The near-miss: `while low < high`

```
def broken(nums, target):
    low, high = 0, len(nums) - 1
    while low < high:                # ← strict, not ≤
        middle = (low + high) // 2
        if nums[middle] == target:
            return middle
        if nums[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return -1

print(broken([3, 9, 14, 23, 31, 42, 55], 42))    # 5    looks fine
print(broken([3, 9, 14, 23, 31, 42, 55], 55))    # -1   WRONG, 55 is at index 6
print(broken([7], 7))                            # -1   WRONG, 7 is at index 0
```

```
5
-1
-1
```

It passes the case you test first and fails on the last element and on any single-element list. The reason is exactly the invariant: when `low = high` the stretch still contains one index, and the strict `<` refuses to look at it. **If the space is non-empty, look at it.** That is what `≤` means.

The near-miss: forgetting the + 1

```
if nums[middle] < target:
    low = middle                # ← should be middle + 1
```

On `[1, 3]` searching for 3: `low=0, high=1, middle=0`, `nums[0]=1 < 3`, so `low = 0`. Nothing changed. The next pass computes the same middle, takes the same branch, and sets the same `low`. The program does not crash — it hangs, silently, forever, and in an interview you will sit there watching a blank terminal. The middle has already been compared; it must leave the space.

The real error: `high = len(nums)`

```
nums = [3, 9, 14, 23, 31, 42, 55]
low, high = 0, len(nums)          # ← length, not last index
while low ≤ high:
    middle = (low + high) // 2
    if nums[middle] == 99:
        break
    low = middle + 1
```

```
Traceback (most recent call last):
  File "day42.py", line 6, in <module>
    if nums[middle] == 99:
    ~~~~^~~~~~
IndexError: list index out of range
```

`high` starts at 7, the loop allows `middle` to reach 7, and index 7 does not exist. This one at least crashes. The `low < high` bug does not, which is why it is the more dangerous of the two.

The trap that is not in the code: unsorted input

```
print(binary_search([5, 2, 9, 1, 7], 9))    # -1
```

Nine is in the list. Binary search says it is not, returns quietly, and no exception is raised. The function's contract says "sorted"; the input broke the contract; the answer is garbage. Shobha's renumbered block. **Ask "is the input sorted?" before you write a line** — it is a ten-second question that saves the whole answer.

The overflow trap, which Python hides

`(low + high) // 2` can overflow a 32-bit integer in Java or C++ when both indices are near two billion — a famous bug that sat in the Java standard library for nine years. The safe form is:

```
middle = low + (high - low) // 2      # identical result, cannot overflow
```

Python's integers grow without limit, so it never bites here. Mention it anyway if the interviewer's language is not Python: it takes one sentence and shows you know where the edge is.

8 In the interview

How it gets asked

- “Given a sorted array of integers and a target, return the index of the target, or -1.” — LeetCode 704, the direct form, often as a warm-up before something harder.
- “How would you find a name in a sorted list of ten million? Why is that fast?” — the same question wearing an explanation's clothes. They want the halving count out loud.
- “Implement it, then tell me the invariant.” — the version that separates people. Many candidates can produce the code and cannot say what it maintains.
- And as a **component**: “first I'd sort, then binary search for each” appears inside dozens of harder problems from here on.

What to say out loud, in the first ninety seconds

1. **Confirm the precondition.** “Binary search needs the array sorted ascending — can I assume that? And are there duplicates, or do you want any matching index?”
2. **State the invariant before writing anything.** “I'll keep two indices, low and high. The invariant is: if the target is in the array at all, its index is between low and high inclusive. Every step preserves that.”
3. **Say what each step does.** “I look at the middle. If it's too small, everything up to and including it is too small, so low becomes middle plus one. If it's too big, high becomes middle minus one. If it's equal, I'm done.”
4. **Say the termination condition and what it means.** “The loop runs while low is less than or equal to high. If low passes high, the space is empty, and by the invariant the target isn't there — so I return minus one.”
5. **Give the cost with a number.** “ $O(\log n)$ time, $O(1)$ space. A million elements is twenty comparisons; a billion is thirty.”

Then write it. The code takes ninety seconds because you have already said what every line does.

The follow-ups

“Why is it $O(\log n)$? Prove it to me without hand-waving.” Because the search space at least halves every pass, and I can count the halvings. It starts at n . After one pass it is at most $n/2$, after two at most $n/4$, after k passes at most n divided by 2 to the k . The loop stops when that reaches 1, so 2 to the k equals n , so k equals log base 2 of n . Each pass does constant work — one midpoint calculation and at most two comparisons — so the total is $O(\log n)$. The sentence I’d actually use in a room is shorter: every time the input doubles, I pay exactly one more comparison. A thousand elements is ten comparisons, a million is twenty, a billion is thirty. That is why it stays usable at sizes where a linear scan has stopped being an option.

“What if the array has duplicates? Which index do you return?” As written, an arbitrary one — whichever the halving lands on first — and that is worth saying explicitly rather than pretending it is deterministic. If the question wants the *first* occurrence, the change is small but the shape is different: on a match I don’t return, I record the index and keep searching left by setting high to middle minus one. The mirror gives the last occurrence. Both still cost $O(\log n)$, because I still halve every pass, I just refuse to stop early. That pair is a standard follow-up, LeetCode 34, and Python’s `bisect_left` and `bisect_right` are exactly those two answers in the standard library. I’d also flag the naive alternative and why it is bad: find any match, then walk left. That is $O(\log n)$ plus $O(k)$ for k duplicates, and degrades to $O(n)$ when the array is all one value.

“The array is rotated — sorted, but starting from somewhere in the middle. Now what?” The invariant survives, but the test that decides which half to keep does not, because `nums[middle] < target` no longer implies the answer is to the right. What is still true is that at least one of the two halves is properly sorted — compare `nums[low]` with `nums[middle]` to find out which. Then check whether the target lies inside that sorted half’s range; if it does, search there, and if it doesn’t, search the other half. Still $O(\log n)$, still one loop, and the extra work is a range check rather than a scan. That is the whole of [day 045](#), and it is a top-five interview question at product companies precisely because it tests whether you understand the invariant or memorised the template.

A model answer

“Sorted array, so binary search. Before I write anything, let me state what the code maintains: two indices, low and high, and the invariant that if the target is in the array, its index is between them inclusive. That single sentence decides every line.

I start with low at 0 and high at the last index — the last index, not the length. Then I loop while low is less than or equal to high, because as long as low equals high there is still one cell I haven’t looked at.

Each pass I take the middle. If it equals the target I return it. If it’s smaller, then it and everything to its left is too small, so low becomes middle plus one. If it’s bigger, high becomes middle minus one. Both branches shrink the space by at least one, which is why the loop always terminates.

```
def binary_search(nums: list[int], target: int) → int:
    low, high = 0, len(nums) - 1
    while low ≤ high:
        middle = (low + high) // 2
        if nums[middle] == target:
            return middle
        if nums[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return -1
```

If low passes high the space is empty, and the invariant tells me the target was never there, so minus one is a conclusion rather than a guess.

Cost is $O(\log n)$ time — twenty comparisons at a million elements, thirty at a billion — and $O(1)$ extra space, since it’s three integers regardless of n . A recursive version would be $O(\log n)$ stack instead.

Two things I’d flag. This is silently wrong on unsorted input — it returns minus one for values that are present — so ‘is it sorted?’ is my first clarifying question. And in Java or C++ I’d write the midpoint as low plus high-minus-low over two, because low plus high can overflow a 32-bit int; Python doesn’t have that problem, but the habit is worth keeping.”

9 Recall card

- **The invariant is the lesson:** *if the target is anywhere, its index is in* `nums[low..high]`. Every line exists to keep it true; an empty space is a conclusion, not a guess.

- **Four details, all off-by-one:** `high = len(nums) - 1`, `while low ≤ high`, `low = middle + 1`, `high = middle - 1`. Drop the `+1` and it hangs; use `<` and it misses the last element.
- **$O(\log n)$ time, $O(1)$ space** — count halvings out loud: 1,000 → 10, a million → 20, a billion → 30. Every doubling costs one more comparison.
- **Sorted is a precondition, not a hint.** On unsorted input it returns `-1` for values that are there, with no error. Ask before you write.
- `bisect_left` is the library answer and returns an *insertion point*, not a found flag — check `nums[i] == target` after. Tomorrow: one template that removes the off-by-ones for good.

1 What this is, and why they ask it

This closes the database phase — eighteen days, from [day 025](#)’s “what a database gives you that a file does not” to [day 041](#)’s connection pools. Today is not new material. It is the drill that turns eighteen separate lessons into one procedure you can run on a product you have never seen, under time pressure, out loud.

They ask it because the database layer is where a system design interview goes when it stops being polite. Every high-level design round — design Twitter, design a ride-hailing app, design a ticket booking system — reaches a moment where the interviewer says “and what does that table look like?” or “what happens if two people book the last seat?” Candidates who have read blog posts about sharding fall over at exactly that point, because those two questions are about schema and isolation, not scale. The material for both has been in front of you since day 026. Today assembles it into an order you can walk.

2 The story

Nasreen has run a tailoring shop in a lane behind the market for nineteen years. Two machines, one girl who does the finishing, and a curtain across the back for measurements.

The shop is not the point. The point is what happens when a customer she has never met walks in at four in the afternoon with a bag of cloth and no idea what she wants.

Nasreen does the same six things, in the same order, every single time, and she has done them so often she does not know she is doing them. First: what is it for. A wedding, an office, a child growing fast — the answer changes everything after it. Second: how many, and by when. Third: she lifts the cloth, feels the weight of it between her fingers, and decides what it will take and what it will not tolerate. Fourth: the measurements, taken in a fixed order, shoulder first, always, so nothing is forgotten. Fifth: she says the price and the date out loud, before the customer has committed to anything, because that is the moment for a surprise to happen, not the moment of delivery. And sixth, the one her competitors skip: she says what would change her mind. “If you want it in three days instead of six, I cannot use this cloth, it will pucker. We would have to pick something else.”

Her nephew opened a shop two lanes over and lasted eight months. He was quicker than her with a machine. But when a customer walked in he would start guessing — “you want it fitted, no?” — and half the time he was guessing at the wrong question. He would agree to a date without having felt the cloth. He never said what would change his mind, so when something did change, it came out as an apology instead of a plan.

Nineteen years of every kind of customer, and Nasreen has never needed a new set of questions. Only the answers change.

3 The idea in plain English

Nasreen’s six questions are the procedure. The interviewer’s product is her unfamiliar customer. And her nephew is the candidate who starts naming technologies before knowing what the system does.

Here is the procedure, in her order, with the days that supply each answer.

One — what is it for: the access paths

Before any schema, list the questions the system will ask of its data. Not the entities — the **access paths**. “Show a user’s last twenty orders.” “Show one order with its items.” “Count today’s signups by city.”

This is [day 040](#)’s rule, and it is the whole reason the SQL-versus-NoSQL question is a trap: you never answer it for a system, you answer it per access path. A product with four read patterns can want a relational store for three of them and a key-value store for the fourth.

Two — how many, and by when: the numbers

Users, writes per second, bytes per record, growth per year. §6 does the arithmetic; the point here is that it comes *second*, before any technology choice, because the numbers decide whether the choice is even interesting. Most systems fit on one node, and saying so early is a strength.

Three — feel the cloth: the invariants

What must never be false? “An order’s total equals the sum of its items.” “A seat is sold once.” “An account balance never goes negative.”

Invariants are the sharpest question in the whole phase, because they pick the core store almost by themselves. Anything that must hold **across entities**, **at write time** wants a relational database and a transaction ([day 033](#)). Anything that holds inside one entity can live in a document ([day 038](#)).

Four — measurements in a fixed order: the schema

Tables, columns, keys, and then indexes — always in that order, because an index is a decision about a query and you cannot make it before the query exists.

The fixed order that keeps you from forgetting anything, from [day 026](#):

1. One table per kind of thing. One row per one of them.
2. A surrogate **primary key** on each — `BIGSERIAL` — never a natural key, because meaningful things change.
3. **Foreign keys on the many side**. Many-to-many needs a third table with a composite key.
4. `NOT NULL` unless absence genuinely means something. `TIMESTAMPZ` for time, `NUMERIC` for money.
5. Index every foreign key. Primary keys are indexed for you; foreign keys are not.
6. Normalise to 3NF first, then denormalise deliberately and only by the read-to-write ratio ([day 029](#)) — and every copy gets an owner and a reconciliation job.

Five — say the price out loud: the failure modes

Before the interviewer asks, name what goes wrong. Two writers on the same row ([day 034](#)'s lost update). Two transactions in a cycle ([day 035](#)'s deadlock). One page issuing two hundred queries ([day 041](#)'s $N+1$). A query that was fast at ten thousand rows and is not at ten million ([day 032](#)).

Six — what would change your mind: the trigger

“I’d start with one Postgres instance. I’d move sessions to Redis when the session check is more than about a fifth of database load, and I’d revisit the write path if sustained writes go past roughly ten thousand a second.” That sentence — a named trigger with a number — is [day 040](#)'s winning close, and it is what her nephew never learnt to say.

4 The picture

The eighteen days, arranged as the four questions they answer:

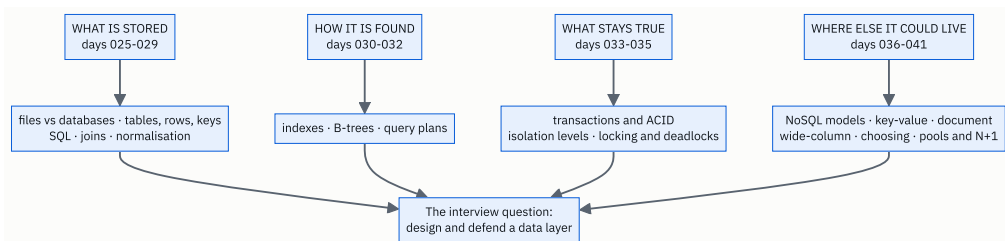
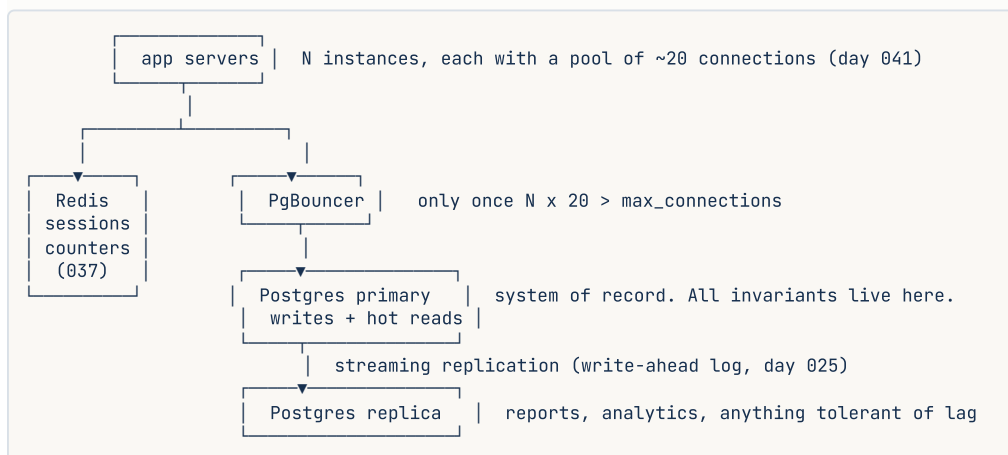


FIGURE 42.1

What to notice: the four groups are the order you should speak in. Schema, then access, then guarantees, then alternatives. Candidates who start at the fourth box — “I’d use Cassandra” — have skipped the three that justify it.

The one diagram to be able to draw from memory, because a design round asks for it constantly:



What to notice: the truth lives in exactly one box. Everything else is a copy with an owner and a reason. That single property is what makes a design defensible when the follow-ups start.

5 How it actually works

The phase in mechanism form — what each piece really does, condensed to the sentence you would say under pressure.

Durability: the write-ahead log

Postgres does not write your row to its table file and hope. It appends the *intent* to a **write-ahead log**, calls `fsync` so the operating system actually commits it to disk, and only then reports success (day 025, day 033). Crash halfway and recovery replays the

log. The same log, shipped to another machine, is streaming replication — which is why durability and replication are one mechanism, not two. MySQL's InnoDB does the same thing with its redo log.

Lookup: the B-tree

An index is a sorted **B + tree** kept beside the table ([day 030](#), [day 031](#)). Each node is one 8 KB page, so a node holds hundreds of keys — a **fanout** near 500 — and three levels cover ten million rows. A lookup is three or four page reads at about 0.1 ms each, against a 200 MB sequential scan. Leaves hold the row pointers and are linked in order, which is why the same index serves `BETWEEN`, prefix `LIKE` and `ORDER BY` for free. The **left-prefix rule**: an index on `(A, B)` serves `A` and `A + B`, never `B` alone.

Correctness under concurrency: MVCC and locks

Postgres gives each transaction a **snapshot** and keeps old row versions until nobody needs them — multi-version concurrency control, which is why readers never block writers. `READ COMMITTED` is the default and takes a fresh snapshot per statement. `REPEATABLE READ` takes one for the whole transaction. `SERIALIZABLE` uses SSI, detects conflicts and aborts the loser — which means **your application must have a retry loop** ([day 034](#)).

Writes take row locks. Blocking is a queue and drains; **deadlock is a cycle** and does not, so after `deadlock_timeout` (one second by default) Postgres finds the cycle in its waits-for graph, aborts one transaction with `ERROR: deadlock detected`, and the app retries from `BEGIN` ([day 035](#)).

The alternatives, and what each one really is

- **Redis** — a shelf in RAM. Sub-millisecond, single-threaded so `INCR` is atomic for free, TTL built in, durability optional. Sessions, carts, counters, rate limiters ([day 037](#)).
 - **DynamoDB** — a durable key-value store partitioned by hashed key, single-digit milliseconds at any size, priced per request.
 - **MongoDB** — documents. One entity plus its belongings, written and read as a unit; atomicity stops at the document's edge ([day 038](#)).
 - **Cassandra** — wide-column. A partition key picks the machine, clustering columns sort inside it, and writes are cheap because of the **LSM tree**. One table per query, decided at write time ([day 039](#)).
 - **Postgres JSONB** — quietly covers a large share of the cases people reach for MongoDB to solve, without a second system to operate.
-

6 The numbers

The arithmetic to have loaded before you walk in. Every one of these is a multiplication you can do out loud.

Sizing a table

```
an orders table, 2 million orders/month:
  2,000,000 x 12 = 24,000,000 rows/year
  ~300 bytes/row (ids, timestamps, status, amounts)
  24,000,000 x 300 B = 7.2 GB/year of table data
  + 4 indexes at ~15% each = 7.2 x 0.6 = 4.3 GB of index
  → ~11.5 GB/year. Five years is under 60 GB.
```

Say the conclusion, not just the number: **that fits in RAM on a mid-sized instance, so this is not a scale problem.** Most tables in most interviews land here.

Read throughput

```
5 million daily active users x 12 page views x 4 queries/page
  = 240,000,000 queries/day
  240,000,000 / 86,400 = ~2,800 queries/s average
  peak ~3x average = ~8,400 queries/s
```

Against one Postgres node's honest ceiling of roughly 10,000-50,000 simple indexed reads per second ([day 040](#)), that is inside one machine — with replicas for headroom, not because the primary cannot cope.

The index decision, priced both ways

```
index lookup : 3-4 page reads x 0.1 ms = ~0.4 ms
table scan   : 200 MB / 8 KB = 25,000 pages x 0.1 ms = ~2.5 s

but every write pays for every index:
  8 indexes = 9 writes per INSERT
  at 500 inserts/s that is 4,500 write operations/s instead of 500
```

That pair of numbers is the entire index trade-off, and it is why “add an index” is never a free answer.

The N+1, priced

```
one list page, 100 items, lazy-loaded author per item:
  1 + 100 = 101 queries x ~1 ms round trip = ~101 ms, all sequential
eager-loaded: 2 queries x ~1.5 ms = ~3 ms

at 200 requests/s: 20,200 queries/s (impossible) vs 400 queries/s (trivial)
```


The database was never slow. The **conversation** was.

Connection pool sizing

```
8-core database → pool of ~16-20 connections per app instance
opening a connection: TCP + TLS + auth + a new Postgres process
                      = ~5-10 ms and ~7 MB
40 app instances x 20 = 800 connections vs max_connections = 100
→ PgBouncer in transaction mode
```

7 The trade-offs

The four you will be asked to defend

Normalised against denormalised. Normalised means each fact in one place, which means no update anomaly and no drift, at the price of a join per read. Denormalised means a stored copy, which means a fast read at the price of a write path that must maintain it. The decision is the read-to-write ratio; below about 10:1, do not (day 029). *I would not denormalise if I could not name the job that reconciles the copy* — that job is the difference between a design and a future incident.

More indexes against faster writes. Each index is 10-25% more storage and one more write per insert. *I would not add an index before running* `EXPLAIN ANALYZE` — half the time the real cause is a function wrapped around the column, a deep `OFFSET`, or an $N+1$, and none of those are fixed by an index (day 032).

Stronger isolation against throughput. `SERIALIZABLE` removes write skew and phantoms and costs you aborts and a retry loop. `READ COMMITTED` is fast and lets a check-then-act pattern lose. *I would not run everything at serialisable* — I would pick per transaction: default plus SQL arithmetic for simple writes, repeatable read for reports that must be self-consistent, serialisable with retries for check-then-act (day 034).

Relational against NoSQL. Relational buys ad-hoc questions, cross-entity transactions and single-copy facts; it costs you joins at read time and a single node's write ceiling. NoSQL buys a designed access path and horizontal scaling by key; it costs you every question you did not think of at design time (day 036). *I would not move a system to Cassandra because it is growing* — I would move one access path, keep the system of record where it is, and write down the number at which that stops being enough.

The honest sentence

Almost every “we outgrew our database” story is a schema story, an index story, or a conversation-shape story. Reach for a bigger architecture only after you can point at the query plan, the pool, and the number of round trips per request, and say that all three are already right.

8 In the interview

How it gets asked

- “Design the data model for a food delivery app.” — the direct form. Schema, keys, indexes, and the one invariant that matters.
- “Two users tap ‘book’ on the last seat at the same moment. Walk me through what happens.” — the isolation question, always. It is testing whether you know the difference between a check and a lock.
- “This page takes 900 ms. Where do you look?” — diagnosis, in a fixed order: query log first, then the plan, then the schema.
- “Would you use SQL or NoSQL here?” — a trap in binary clothing. Never answer the binary.

What to say out loud, in the first ninety seconds

1. **Name the access paths, not the entities.** “Before the schema, let me list the reads: a customer sees their last twenty orders, a restaurant sees today’s open orders, and ops runs revenue by city per day. Three patterns, and the third is analytical.”
2. **Do one number.** “At two million orders a month, that’s twenty-four million rows and about ten gigabytes a year with indexes — comfortably one node, so I’ll design for correctness first, not scale.”
3. **State the invariant.** “The one thing that must never be false: a delivery slot is assigned to exactly one order. That’s a cross-row constraint, so it lives in a relational transaction.”
4. **Give the tables in one breath.** “customers, restaurants, orders, order_items. Surrogate BIGSERIAL keys, foreign key on the many side, order_items stores unit price as a snapshot because it must not follow today’s menu price.”
5. **Then the indexes, each with its query.** “Index on orders(customer_id, created_at DESC) for the customer’s recent list, and on orders(restaurant_id, status) for the open-orders board. Each index exists for a named query.”

6. Close with the trigger. *“I’d revisit this if writes go past roughly ten thousand a second, or if the analytics query starts affecting order latency — at which point it moves to a replica before it moves anywhere else.”*

The follow-ups

“Two customers book the last seat at the same instant. What happens, and what do you do about it?” Under the default isolation level, read committed, both transactions can read “one seat left”, both can decide it is fine, and both can write — that is the lost update, and it is the single most common correctness bug in booking systems. The reason it happens is that a plain `SELECT` takes no lock, so the check and the act are not one indivisible step. There are three fixes and I would say which I’d pick and why. First: do the arithmetic in the database — `UPDATE seats SET remaining = remaining - 1 WHERE id = ? AND remaining > 0`, then check the affected row count. That takes a row lock for the duration of the statement, it is one round trip, and it cannot lose. Second: `SELECT ... FOR UPDATE` on the row I am about to check, which locks it so the second transaction waits — correct, but it holds a lock for the length of my transaction, so it must be short and must contain no external calls. Third: `SERIALIZABLE`, which catches this and write skew as well, at the cost of aborts and a retry loop in the application. For a seat counter I’d take the first. For “the last seat, and also insert a booking row and also decrement a coupon”, where the invariant spans several rows, I’d take the third.

“Your orders table has two hundred million rows and the customer’s order history has got slow. What do you do, in order?” In order of cost, cheapest first, and I would not skip a step. First I look at the actual query and its plan with `EXPLAIN ANALYZE`, and the first thing I compare is estimated rows against actual rows, because a bad estimate poisons every decision above it and the fix might just be `ANALYZE`. Then I check the obvious structural causes that never show up in the plan: is this one query or is it an N+1 from the ORM, and is it using a deep `OFFSET` for pagination? Deep offset is the classic — `OFFSET 100000` makes the database produce and discard a hundred thousand rows, and the fix is keyset pagination: `WHERE (created_at, id) < (?, ?) ORDER BY created_at DESC, id DESC LIMIT 20`. Then the index: a composite on `(customer_id, created_at DESC)` serves both the filter and the sort, so the sort disappears from the plan. Only after all of that would I talk about partitioning by month, which helps because old partitions stop being touched, and only after that would I discuss moving history out. Most of the time the answer is at step three, and saying that is a strength.

“When would you actually add a second kind of database?” When a specific access path has an evidenced cost that the current store cannot serve, and I can name both the path and the number. Three real examples. A session check on every request at twenty thousand requests a second is twenty thousand queries a second against the system of

record for data that is disposable — that goes to Redis, and I'd say plainly that nothing irreplaceable lives only there, because eviction and default persistence make it a shelf, not a ledger. A firehose of telemetry, a billion events a day with a fixed set of queries, is 11,600 writes a second of data nobody updates — that goes to a wide-column or time-series store, because writes are appends and the questions are known. Full-text search across millions of documents goes to Elasticsearch, because that is a genuinely different data structure and Postgres full-text hits a ceiling. What all three share is that I am moving *one access path*, not the system, and every copy has an owner and a reconciliation path. The default answer is the store I already run, because a second database is a second thing to back up, monitor, patch and be paged for — and that bill is paid every week, not once.

A model answer

“Let me take the reads first, then the numbers, then the invariants, then the schema.

The reads are: a customer’s recent orders, a restaurant’s open orders, and daily revenue by city. The first two are per-key and latency-sensitive; the third is analytical and can tolerate lag, so it goes to a replica.

Numbers: two million orders a month is twenty-four million rows a year, about three hundred bytes each, so seven gigabytes of table plus four gigabytes of index. Five years is under sixty gigabytes. That fits on one node with room, so scale isn’t the interesting constraint here — correctness is.

The invariant that decides the store: an order’s items must be written with the order, and a delivery slot must be held by exactly one order. Both are cross-row, write-time constraints, so the core is relational — Postgres.

Schema: `customers`, `restaurants`, `orders`, `order_items`. `BIGSERIAL` surrogate keys, foreign keys on the many side, `NOT NULL` everywhere absence isn’t meaningful, `NUMERIC` for money and `TIMESTAMPTZ` for time. `order_items` stores `unit_price` as a snapshot of what was charged — it should differ from today’s menu price, and it never drifts.

Indexes, each tied to a query: `(customer_id, created_at DESC)` for the history page, which serves the filter and the sort together; `(restaurant_id, status)` for the open-orders board. I’d index every foreign key, since Postgres doesn’t do that for me, and I’d resist adding more until `EXPLAIN ANALYZE` asks for them — each index is another write per insert.

Concurrency: the seat-or-slot assignment does its arithmetic in the `UPDATE` with a `WHERE` guard, and I check the affected row count rather than reading first and writing after. Anything that has to check several rows and then write goes to serialisable with a retry loop, and I’d say out loud that the retry loop is not optional.

Operationally, connection pools of about twenty per app instance, and PgBouncer once instance count times pool size passes `max_connections`. And query counts visible in development, because the failure I’d actually expect first is an $N+1$ on the order-items page, not a shortage of hardware.

What would change my mind: sustained writes past roughly ten thousand a second, or a session or telemetry access path whose volume is dominated by data that isn’t the system of record. Those move one path to Redis or a time-series store — with an owner and a reconciliation path — not the whole system somewhere else.”

9 Recall card

- **Run the six questions in order, every time:** access paths → numbers → invariants → schema → failure modes → the trigger that would change your mind. Never name a product first.
- **Schema order is fixed:** table per thing, surrogate key, foreign key on the many side, `NOT NULL / NUMERIC / TIMESTAMPTZ`, index every foreign key, 3NF then denormalise only by read-to-write ratio with a reconciliation job.
- **The concurrency answer is one sentence:** do the arithmetic in the `UPDATE` with a `WHERE` guard; use `FOR UPDATE` for check-then-act on one row; use serialisable plus a retry loop when the invariant spans rows.
- **Diagnose in cost order:** `EXPLAIN ANALYZE` (estimated vs actual rows) → $N+1$ and deep `OFFSET`, which never appear in the plan → composite index → partition. Most answers are at step two.
- **Numbers to carry:** index lookup 3-4 pages ≈ 0.4 ms; one node $\approx 10\text{-}50\text{k}$ simple reads/s and tens of terabytes; $1\text{B events/day} = 11.6\text{k/s}$; pool $\approx \text{cores} \times 2$. A second database moves one access path, never the system.

DSA topic: Binary search: the idea and the invariant **System design topic:** Database revision and interview questions

Code these, in this order

Today is the first day of a new phase, so the rule is different from a mock day: **write the loop from memory before you look at §5, every single time**. Four problems, and the fourth is deliberately not an array search.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Binary Search	LeetCode 704 (Easy)	The template itself: <code>≤</code> , <code>middle + 1</code> , <code>middle - 1</code> , and <code>-1</code> as a conclusion.
2	Search Insert Position	LeetCode 35 (Easy)	Whether you noticed where <code>low</code> ends up when the target is absent.
3	Guess Number Higher or Lower	LeetCode 374 (Easy)	Binary search with no array at all — the space is a range of numbers.
4	Valid Perfect Square	LeetCode 367 (Easy)	The first sight of searching an answer range instead of data.

On problem 1, say the invariant before you type

Out loud, in one sentence: *if the target is anywhere, its index is between low and high inclusive*. Then write the five lines. Then run the three inputs that catch beginner versions — `[]` with any target, `[7]` with 7, `[7]` with 8. If any of them surprised you, rewrite the loop rather than patching it.

On problem 2, do not add a new loop

The whole answer is one word different from problem 1. When the loop ends without a match, `low` is already sitting at the insertion point — trace it by hand on `[1, 3, 5, 6]` with target 2 and watch where `low` finishes. Change `return -1` to `return low`, and say why that is correct rather than lucky: the invariant guarantees everything left of `low` is smaller and everything from `low` on is larger.

On problem 3, notice there is nothing to index

The search space is the numbers 1 to n, not cells in memory. Same two variables, same halving, same termination. This is the bridge to [day 046](#), and if it feels like the same problem, that is the point. Use `low + (high - low) // 2` here — n goes up to $2^{31} - 1$,

and the problem exists partly to make the overflow point.

On problem 4, name the monotone thing

Before coding, say what is sorted. It is not the input — the input is one number. It is the sequence of squares `1, 4, 9, 16, ...`, which never goes down, and that is the only property binary search needs. Search `low = 1` to `high = num` and compare `middle * middle` against `num`.

The off-by-one drill

Break your own working solution four ways, one at a time, and predict the symptom before you run it:

1. `while low < high` — which two inputs fail, and does it crash or lie?
2. `low = middle` instead of `low = middle + 1` — crash, wrong answer, or hang?
3. `high = len(nums)` instead of `len(nums) - 1` — paste the exact traceback.
4. Feed it `[5, 2, 9, 1, 7]` looking for 9 — what does it return, and why is that not a bug in the function?

Being able to predict the symptom is worth more than being able to fix it.

The counting drill

Answer in under ten seconds each, no calculator: comparisons needed for 1,000 elements; 1,000,000; 1,000,000,000. Then the sentence that generates all three without memorising any of them.

The database phase-closing drill

The database phase ends today. Run Nasreen's six questions on a product you have not seen written about here — pick a hostel room-booking system — and speak the answer in under four minutes, standing:

1. The access paths, three of them, as questions and not entities.
2. One number, multiplied out.
3. The one invariant that must never be false.
4. The tables, keys and two indexes, each index named with its query.
5. What happens when two people book the same room in the same second, and which of the three fixes you pick.
6. The trigger that would change your mind, with a number in it.

Then say the diagnosis order for a slow page — and remember which two causes never appear in a query plan.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Search a sorted array. Why is it $O(\log n)$?* The invariant first, then the code as sentences, then the halving count — n , $n/2$, $n/4$ — and the line about every doubling costing one more comparison.
 2. *Design and defend the database layer for an unseen product.* The six questions in order, ending on the trigger. If you reach for a product name before question five, start again.
 3. *Your binary search returns -1 for a value that is definitely in the list. What are the two possible causes?* The input was not sorted, or the loop used `<` and never looked at the last remaining cell. Say which one produces which symptom, and the clarifying question that prevents the first.
-

Before you move on

- ☐ I wrote binary search from memory, with no reference, and it handled `[]`, `[7]` found, and `[7]` absent.
- ☐ I can state the invariant in one sentence, and say which line of the code preserves it.
- ☐ I predicted the symptom of all four deliberate breakages before running them.
- ☐ I can give the comparison count for a thousand, a million and a billion without pausing.
- ☐ I ran the six database questions on the hostel booking system in under four minutes.
- ☐ I answered all three questions above out loud.

Writing binary search without off-by-one bugs

DATA STRUCTURES AND ALGORITHMS

Writing binary search without off-by-one bugs

You can write it correctly the first time, every time, using one template.

SYSTEM DESIGN

Why interviews ask object-oriented design at all

You can say what an OOP round is really testing, so you stop answering the wrong question.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Write binary search on the whiteboard. No compiler.*
- *Design a class structure for this feature.*

Writing binary search without off-by-one bugs

1 What this is, and why they ask it

Yesterday's binary search finds a matching element. Today you learn a second template that finds a **boundary** — the first position where something becomes true — and that template answers every binary search question you will ever be asked, including yesterday's. It is four lines, it has one invariant, and once it is in your hands the off-by-one bugs stop happening because there is nothing left to get wrong.

They ask you to write it without a compiler because that is where the bugs live. A candidate who knows only yesterday's three-way version has to re-derive the `+ 1` and `- 1` for every variant — first occurrence, last occurrence, insertion point, rotated array, search on the answer — and re-derives one of them wrong under pressure. A candidate with today's template writes the same four lines every time and changes only one line: the question being asked of the middle. That is the difference between remembering an algorithm and owning one.

2 The story

Vasu keeps a small boat tied at the ghat on the river at Ranganathittu, and he goes down to it most mornings at about six.

The ghat is a long flight of stone steps running down to the water, sixty-odd of them, wide and worn smooth. The river is not the same height two days running. After rain it climbs; in March it drops so far that the bottom twenty steps bake dry and the weeds on them go brown.

Every morning Vasu has the same small job before he does anything else. He carries a cloth bag with his flask and his slippers in it, and he wants to leave it on the lowest step that is still dry — as close to the water as he can get without soaking it, because he is coming back up wet and does not want to climb far.

He does not walk down testing every step. He learnt that when he was younger and it was cold.

What he does instead is this. He goes down to roughly the middle of the flight and puts his bare foot on that step. If it comes up dry, he knows two things at once: that step is dry, and so is every step above it, so there is no reason ever to look up there again. The

step he is standing on is not the one he wants either — he wants a lower one — so he moves on past it.

If it comes up wet, he knows the opposite: this step and everything below it is under water. But — and this is the part he is careful about — he does *not* move past a wet step. The step he just touched might be the very first wet one, the boundary itself, and the one he wants is directly above it. He keeps it in mind and searches upward from there, but he does not throw it away.

Dry step: step past it. Wet step: keep it. He does that five or six times, the range closing in each time, and then there is nothing left between his two marks, and the boundary is exactly there. Bag down, into the boat, gone by ten past six.

3 The idea in plain English

Vasu's steps are the array. Wet or dry is a **yes-or-no question** asked of one position. And the crucial asymmetry — *step past a dry one, keep a wet one* — is the whole template.

The one property the steps have

Reading from the top down, the steps go dry, dry, dry, ..., wet, wet, wet. They never go back. There is no dry step below a wet one, because water does not work like that.

A yes-or-no question with that shape is called **monotone**: once the answer flips from no to yes, it stays yes. Binary search does not actually need a sorted array. It needs a monotone question, and a sorted array is simply the most common way to get one.

Write the question as a function of the index:

```
def is_wet(i: int) → bool:  
    ...
```

and the array of answers looks like this:

index	0	1	2	3	4	5	6	7
	False	False	False	False	True	True	True	True
					^			
					the boundary:	the first True		

The boundary is the answer to every binary search problem. Yesterday's "find target" is one instance of it, as you will see in a moment.

The two marks, and what they mean

Two variables again, but they mean something different from yesterday:

- `low` — the first index that might still be True.
- `high` — one past the last index that might still be True.

So the live range is `[low, high)`: `low` included, `high` **not** included. That is called a **half-open range**, and it is the same convention Python slicing already uses — `nums[2:5]` gives you indices 2, 3 and 4, not 5. You have been using it since [day 005](#).

They start at `low = 0` and `high = len(nums)` — note, the *length*, not the last index. That is not a bug. `high` is deliberately one past the end, because the honest answer to “where is the first True?” when there is no True at all is “*one past the end*”.

The invariant

Everything at an index below `low` is False. Everything at an index at or above `high` is True. The boundary is somewhere in `[low, high)`.

That is the sentence to say in the interview. It is stronger than yesterday’s, and it is what makes the code write itself.

The four lines

```
while low < high:
    middle = (low + high) // 2
    if question(middle):
        high = middle          # middle might BE the boundary – keep it
    else:
        low = middle + 1       # middle is definitely not – step past it
```

Read the two branches as Vasu’s two rules.

- The question is **True** at `middle` — the step is wet. Then `middle` is a True, and the first True is at `middle` or earlier. So the range shrinks to `[low, middle)` ... except `middle` itself must stay in play, and it does, because the range is half-open: `high = middle` means “everything from `middle` on is known True”, and `middle` is still reachable through `low`.
- The question is **False** — the step is dry. Then `middle` is not the boundary and neither is anything before it, so `low = middle + 1`.

When the loop ends, `low == high`, the range is empty, and by the invariant everything below `low` is False and everything from `low` on is True. `low` **is the boundary**. No `-1`, no special case, no “did I find it” flag.

Why `while low < high` this time, and not `<=`

Yesterday's range was `[low, high]` with both ends included, so `low == high` still held one live index and the loop had to run. Today's range is `[low, high)`, so `low == high` holds *nothing* and the loop must stop. The comparison follows from the convention, which is why you should pick one convention and never mix them. Mixing is the actual source of nearly every off-by-one bug in this subject.

Everything else is this template with a different question

```
question(i) = nums[i] ≥ target    → low = first index whose value is ≥ target    (lower bound)
question(i) = nums[i] > target    → low = first index whose value is > target    (upper bound)
```

And from those two:

- **Does `target` exist?** Take the lower bound `i`, then check `i < len(nums)` and `nums[i] == target`.
- **Where would `target` be inserted?** The lower bound, unchanged. That is LeetCode 35.
- **First occurrence of `target` ?** The lower bound, when it matches.
- **Last occurrence?** The upper bound minus one.
- **How many equal `target` ?** Upper bound minus lower bound. One subtraction.

Five problems, one template, one changed line. That is why today exists.

4 The picture

The question's answers, and what the two marks mean:

index	0	1	2	3	4	5	6	7
	+	+	+	+	+	+	+	+
nums	2	4	4	4	7	9	11	15
	+	+	+	+	+	+	+	+
≥ 4 ?	no	YES	YES	YES	YES	YES	YES	YES
> 4 ?	no	no	no	no	YES	YES	YES	YES
		^			^			
	lower bound = 1			upper bound = 4				
count of 4s = upper - lower = 4 - 1 = 3.								

What to notice: both rows are monotone — once they turn to YES they stay. That, and nothing else, is what binary search requires.

The half-open range closing in, searching for the lower bound of 7:

```

start    low=0                                high=8    (one past the end)
         [ 2    4    4    4    7    9    11    15 )

pass 1   middle=4, nums[4]=7 ≥ 7 → YES → high = 4    (keep 4: it might be it)
         [ 2    4    4    4 ) 7    9    11    15

pass 2   middle=2, nums[2]=4 ≥ 7 → no  → low = 3      (step past it)
         [ 4 ) 7    9    11    15

pass 3   middle=3, nums[3]=4 ≥ 7 → no  → low = 4
         ) low = high = 4, loop ends

answer: low = 4.

```

What to notice: on pass 1 `high` became 4 and the answer is 4. If the code had written `high = middle - 1` — yesterday's reflex — it would have thrown away the answer on the first pass. That single line is the difference between the two templates.

And the case with no True at all:

```

nums = [2, 4, 4], question = "nums[i] ≥ 9"
every answer is no, low climbs to 3, high stays 3, loop ends, low = 3 = len(nums)

"one past the end" is the boundary's honest answer for "there is no True".

```

What to notice: no special case handled it. The convention handled it.

5 The code, built step by step

The template, alone

```

def first_true(lo: int, hi: int, question) → int:
    """Smallest i in [lo, hi) where question(i) is True, or hi if none is."""
    while lo < hi:
        mid = (lo + hi) // 2
        if question(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

```

Six lines, and they never change. Everything below is a different `question`.

Lower bound

```
def lower_bound(nums: list[int], target: int) → int:
    """First index whose value is  $\geq$  target; len(nums) if there is none."""
    lo, hi = 0, len(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid]  $\geq$  target:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

The only line carrying any thought is `nums[mid]  $\geq$  target`. Say it out loud as a question — *is this position at or past where the target belongs?* — and the branch that follows is obvious.

Upper bound: change one character

```
if nums[mid] > target:    # was  $\geq$ 
    hi = mid
```

> instead of $\geq$. Now positions holding the target itself answer “no”, so `low` walks past all of them and lands one past the last copy.

Search, insertion point, and the count

```
def search(nums: list[int], target: int) → int:
    i = lower_bound(nums, target)
    return i if i < len(nums) and nums[i] == target else -1

def insert_position(nums: list[int], target: int) → int:
    return lower_bound(nums, target)    # LeetCode 35, entire answer

def count_of(nums: list[int], target: int) → int:
    return upper_bound(nums, target) - lower_bound(nums, target)
```

Note the guard in `search: i < len(nums)` **before** `nums[i]`. If the target is bigger than everything, `i` is `len(nums)` and indexing it raises. The `and` short-circuits, so the order of the two conditions is doing real work.

The complete solution

```
from bisect import bisect_left, bisect_right

def first_true(lo: int, hi: int, question) → int:
    """The template. Smallest i in [lo, hi) with question(i) True, else hi.

    Invariant: everything below lo answers False, everything from hi up answers True.
    """
    while lo < hi:
        mid = (lo + hi) // 2          # mid < hi always, so hi = mid strictly shrinks
        if question(mid):
            hi = mid                  # mid might be the boundary: keep it
        else:
            lo = mid + 1              # mid is not the boundary: step past it
    return lo

def lower_bound(nums: list[int], target: int) → int:
    """First index with nums[i] ≥ target. len(nums) if every value is smaller."""
    return first_true(0, len(nums), lambda i: nums[i] ≥ target)

def upper_bound(nums: list[int], target: int) → int:
    """First index with nums[i] > target. len(nums) if no value is bigger."""
    return first_true(0, len(nums), lambda i: nums[i] > target)

def search(nums: list[int], target: int) → int:
    """LeetCode 704, rebuilt on the boundary template."""
    i = lower_bound(nums, target)
    return i if i < len(nums) and nums[i] == target else -1

def insert_position(nums: list[int], target: int) → int:
    """LeetCode 35. The lower bound, with no extra work at all."""
    return lower_bound(nums, target)

def count_of(nums: list[int], target: int) → int:
    """How many copies of target the sorted list holds."""
    return upper_bound(nums, target) - lower_bound(nums, target)

if __name__ == "__main__":
    nums = [2, 4, 4, 4, 7, 9, 11, 15]

    print(lower_bound(nums, 4), upper_bound(nums, 4))    # 1 4
    print(count_of(nums, 4))                             # 3
    print(count_of(nums, 5))                             # 0 ← absent, no special case
    print(lower_bound(nums, 1), lower_bound(nums, 99))  # 0 8 ← below all, above all
    print(search(nums, 9), search(nums, 10))            # 5 -1
    print(insert_position([1, 3, 5, 6], 2))              # 1
    print(insert_position([1, 3, 5, 6], 7))              # 4 ← one past the end
    print(lower_bound([], 5))                            # 0 ← empty input, no crash

    # The standard library is this template, already written.
    print(bisect_left(nums, 4), bisect_right(nums, 4))  # 1 4
```

`bisect_left` is `lower_bound` and `bisect_right` is `upper_bound`. They are written in C and they are correct. Write the template when the interviewer asks you to write binary search; call `bisect` when binary search is a step inside a bigger problem and nobody is testing your loop.

6 What it costs

Time

Identical to yesterday, because it is the same halving:

```
range size:  n  → n/2 → n/4 → ... → 1 → 0
passes:      log2(n) + 1
work per pass: one midpoint, one question, one assignment → O(1)
```

$O(\log n)$, and the count is the same twenty comparisons at a million, thirty at a billion. One caveat worth saying: the cost is **$O(\log n)$ questions**, and if the question itself is expensive — which it will be from [day 046](#), where answering it means scanning the array — the real cost is **$O(\log n \times \text{cost of one question})$** .

Space

```
lo, hi, mid: three integers    → O(1) extra space
```

Why it always terminates

This is the part interviewers probe, because “it hangs” is the most common failure and candidates rarely know why theirs does not.

The loop runs only while **$lo < hi$** , so **$lo + hi < 2 \times hi$** , so **$mid = (lo + hi) // 2 < hi$** . Therefore **$hi = mid$** strictly *decreases* **hi** . And **$mid \geq lo$** always, so **$lo = mid + 1$** strictly *increases* **lo** . Every pass moves one of the two marks toward the other by at least one, and they cannot cross without the loop stopping. **The range shrinks by at least one every pass, so it cannot run more than n times, and in fact runs $\log n$ times.**

Say that in one sentence: **mid is always strictly less than hi** , so **$hi = mid$** always *makes progress*. That sentence is the reason **$high = middle$** is safe here and would hang in yesterday’s **$[low, high]$** template — where **mid** can equal **hi** , and assigning **$hi = mid$** changes nothing.

The comparison worth naming

```
yesterday's template : 3-way compare, returns -1 when absent, needs re-derivation per variant
today's template      : 2-way question, returns a POSITION always, one line changes per variant

five problems solved by changing one character:
    exists / insert position / first occurrence / last occurrence / count
```

7 The traps

The near-miss: `high = middle - 1` inside the boundary template

The reflex from yesterday, dropped into today's code:

```
def broken(nums, target):
    lo, hi = 0, len(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] ≥ target:
            hi = mid - 1          # ← yesterday's habit
        else:
            lo = mid + 1
    return lo

print(broken([2, 4, 4, 4, 7], 7))    # 5    should be 4
print(broken([1, 3, 5, 6], 3))      # 2    should be 1
```

```
5
2
```

It throws away the answer. `mid` was the first True — the very thing being looked for — and `hi = mid - 1` puts it outside the range. The rule that prevents this: **- 1 belongs to closed ranges, never to half-open ones**. In `[lo, hi)`, `hi = mid` already excludes nothing you needed.

The near-miss: half-open range with `<=`

```
lo, hi = 0, len(nums)
while lo ≤ hi:                      # ← yesterday's condition, today's range
```

Now the loop keeps running when `lo = hi = len(nums)`, `mid` becomes `len(nums)`, and `nums[mid]` raises. Or, in the version where the question never touches the array, it simply hangs. **The range convention and the loop condition are a set**, exactly as the sentinel and the formula were a set on [day 037](#). Closed `[lo, hi]` goes with `≤` and `± 1`. Half-open `[lo, hi)` goes with `<` and `hi = mid`. Pick one. Never blend them.

The real error: forgetting the bounds check after the lower bound

```
nums = [2, 4, 4, 4, 7]
i = lower_bound(nums, 99)
print(i)                      # 5
print(nums[i] == 99)          # boom
```

```

5
Traceback (most recent call last):
  File "day43.py", line 4, in <module>
    print(nums[i] == 99)          # boom
    ~~~~^^^
IndexError: list index out of range

```

The lower bound is a *position*, and the position “one past the end” is a legitimate answer. Any code that turns a position into a value must check `i < len(nums)` first. This is the price of a template that never returns `-1`: it hands you an index you must validate.

The trap that returns a plausible lie: a question that is not monotone

```

nums = [1, 5, 2, 8, 3]
print(first_true(0, 5, lambda i: nums[i] ≥ 4))    # 3

```

Index 1 holds 5, which is `≥ 4`, so the true answer to “first index whose value is at least 4” is 1. The template says 3. No error, no warning. The template’s contract is *monotone question*; this question is not, so the answer is meaningless. Before you use it, say out loud what the question is and why it can never flip back — that ten-second check is the same one as yesterday’s “is the array sorted?”, and it catches strictly more bugs.

The subtle one: `lambda i: nums[i] >= target` captures `target`

```

questions = [lambda i: nums[i] ≥ t for t in (4, 7)]    # both close over the SAME t
print([lower_bound(nums, 4), lower_bound(nums, 7)])  # fine – passed as arguments

```

Building a list of questions in a loop and expecting each to remember its own `target` is Python’s late-binding closure trap, and it produces two identical questions. It is not a binary search bug, but it is how people break this template when they try to be clever with it. Pass the target as an argument, as the code in §5 does, and it cannot happen.

8 In the interview

How it gets asked

- “Write binary search. No IDE.” — and then, thirty seconds later, “now return the first occurrence instead.” The second half is the real question, and it is where the three-way template falls over.
- “Find the position where this value should be inserted to keep the list sorted.” — LeetCode 35, the lower bound with no extra code.

- “How many times does x appear in this sorted array, in better than linear time?” — upper bound minus lower bound, and they are checking whether you say “one subtraction” or start writing a scan.
- “Your loop hangs. Why?” — a live-debugging prompt. The answer is about `mid` and `hi`, and §6 has it.

What to say out loud, in the first ninety seconds

1. **Declare the convention before writing.** “I’ll use a half-open range, low inclusive and high exclusive, so high starts at `len(nums)`. That means the loop condition is strictly less-than, and I never write minus one.”
2. **Turn the problem into a yes-or-no question.** “I’m looking for the first index where `nums[i] ≥ target` is true. That question is monotone on a sorted array — once true, always true — which is all binary search needs.”
3. **State the invariant.** “Everything below low answers false, everything from high up answers true. When they meet, low is the boundary.”
4. **Say the asymmetry, because it is the bug everyone makes.** “If the middle answers true I set high to middle, not middle minus one — the middle might be the boundary itself. If it answers false I set low to middle plus one, because the middle definitely isn’t.”
5. **Say what the return value is.** “It returns a position, always — never minus one. If the target is bigger than everything, it returns `len(nums)`, which is the correct insertion point. So any caller that wants a value has to check the bound first.”

The follow-ups

“Now give me the first and the last occurrence, and tell me the cost.” Both are this template with one character changed. The first occurrence is the lower bound — the first index where `nums[i] ≥ target` — and it is a genuine match only if that index is in range and holds the target. The last occurrence is the upper bound minus one: the upper bound is the first index where `nums[i] > target`, so the position just before it is the last copy. Two independent binary searches, so $O(\log n)$ plus $O(\log n)$, which is still $O(\log n)$, and $O(1)$ space. The reason I’d do two searches rather than one search plus a walk is the all-duplicates case: on an array of a million identical values, find-then-walk is $O(n)$ and this stays at forty comparisons. And the count of copies is the two bounds subtracted, which is worth pointing out because interviewers often expect a third pass for it.

“Why does your loop terminate? Convince me.” Because `mid` is strictly less than `hi` on every pass, and that is not an accident of the input. The loop only runs while `lo < hi`, so `lo + hi` is strictly less than `2 × hi`, so the floor of `(lo + hi) / 2` is strictly less than `hi`. That means the assignment `hi = mid` always makes `hi` smaller. The other branch, `lo = mid + 1`, always makes `lo` bigger, because `mid` is at least `lo`. So every single pass narrows the range by at least one, and a range of size n cannot narrow

more than n times. It actually halves, so it is $\log n$. The place this argument breaks — and it is worth mentioning because it is the bug people hit — is if you write `hi = mid` inside a *closed-range* loop with `lo ≤ hi`. There `mid` can equal `hi`, the assignment changes nothing, and it hangs forever. The termination argument belongs to the convention, not to binary search in general.

“Can you use this when there is no array — say, the input is a function or a stream?” Yes, and that is the reason I write it this way. Nothing in the template touches an array; it takes a range of integers and a question, and the question happens to consult an array today. If the input is an API that answers “is version i broken?”, the same six lines find the first broken version — that is LeetCode 278, and it is exactly this. If the question is “can all packages ship within d days using capacity c ?”, the range becomes the possible capacities and the answer is the smallest c that works, which is [day 046](#). The two things I would check before using it anywhere are: is the question monotone over the range, and how expensive is one call — because the cost is $\log n$ *questions*, and if each question is an $O(n)$ scan then the total is $O(n \log n)$, not $O(\log n)$.

A model answer

“Let me set the convention first, because that’s where the off-by-one bugs come from. I’ll use a half-open range: low is inclusive, high is exclusive, so high starts at `len(nums)` rather than the last index. With that convention the loop condition is strictly less-than and I never write a minus one anywhere.

Then I turn the problem into a yes-or-no question about a position. For ‘find the target’ the question is: is `nums[i] ≥ target`? On a sorted array that’s monotone — once it’s true it stays true — and monotone is the only thing binary search actually needs. Sorted is just the usual way to get it.

The invariant is: everything below low answers false, everything from high upward answers true, so the boundary is inside `[low, high)`.

```
def lower_bound(nums: list[int], target: int) → int:
    lo, hi = 0, len(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] ≥ target:
            hi = mid          # mid might be the boundary – keep it
        else:
            lo = mid + 1      # mid is definitely not – step past it
    return lo
```

The asymmetry in those two branches is the whole thing. If the middle answers true, the middle itself might be the first true, so I keep it by setting high to middle — not middle minus one, which is the classic bug and throws away the answer. If it answers false, the middle is ruled out, so low goes one past it.

When the loop ends low equals high, and low is the boundary. It always returns a position, never minus one, so ‘does the target exist’ is a check afterwards: `i < len(nums) and nums[i] == target`, in that order, because a target above everything gives me `len(nums)` and indexing that would raise.

That’s $O(\log n)$ time and $O(1)$ space. And it’s worth saying what I get for free: change `≥` to `>` and I have the upper bound, so the last occurrence is upper bound minus one and the number of copies is upper minus lower. Insertion position is this function unchanged — that’s LeetCode 35. Five problems, one template, one character. In production I’d call `bisect_left` and `bisect_right`, which are exactly these two.”

9 Recall card

- **One template, forever:** half-open `[lo, hi)`, `hi = len(nums)`, `while lo < hi`, `mid` in the middle, `question(mid) → hi = mid` else `lo = mid + 1`. Return `lo`.
- **The asymmetry is the lesson:** True keeps the middle (`hi = mid`), False steps past it (`lo = mid + 1`). `- 1` belongs to closed ranges only; blending the two conventions is the bug.
- **Invariant:** below `lo` all False, from `hi` up all True — so `lo` is the first True, and `len(nums)` is the honest answer for “no True at all”.
- **Change one character, get five problems:** `≥` is lower bound (first occurrence, insert position, exists); `>` is upper bound (last occurrence = upper - 1, count = upper - lower). `bisect_left` / `bisect_right` are these two.
- **It needs a monotone question, not a sorted array** — and it returns a *position*, so check `i < len(nums)` before reading `nums[i]`. Termination: `mid < hi` always, so `hi = mid` shrinks.

1 What this is, and why they ask it

Object-oriented design is the practice of splitting a program into objects — bundles of data together with the behaviour that acts on that data — so that each one has a clear job and a clear edge. An **object-oriented design round**, sometimes called low-level design or LLD, gives you a paragraph of requirements and forty-five minutes, and asks for the classes: what they are, what each one is responsible for, how they connect, and enough working code that one flow through the system actually runs.

They ask it because it is the only round that tests the thing you will do every working day. The algorithms round tests whether you can find an answer; the high-level design round tests whether you can size a system. This round tests whether the code you write can be changed six months later by somebody who is not you. Amazon runs it in nearly every loop and weights it heavily. Uber, Flipkart, Swiggy, Zomato, Atlassian, PhonePe and most Indian product companies run a dedicated LLD round; Microsoft folds it into the coding round. The failure mode is specific and common: candidates answer the wrong question — they design a schema, or they recite design patterns, or they write one enormous class — and the interviewer has no way to score any of it.

2 The story

Antony has a furniture workshop in a lane in Ernakulam, six men, and he has been taking on one new carpenter a year for about twenty years.

He does not ask them to build a cupboard. He tried that once, early on. It takes four days, you learn almost nothing until the end, and by then you have paid for the wood.

What he does now is smaller. He puts a man at the bench with two planks and says: make me one drawer that slides.

Then he goes and does something else, and watches out of the corner of his eye.

The first thing he watches for happens before any cutting. A good man asks what the drawer is for. Kitchen or office? What is going in it — cutlery, or a sewing machine head that weighs nine kilos? How wide is the gap it has to sit in, and is the gap square? Four questions, one minute, and every cut after them is different because of the answers. The men who do not ask start cutting in the first thirty seconds, and Antony has learnt that the confidence is the problem, not the speed.

The second thing he watches for is the joints. Anybody can nail four sides together and it will slide today. What he wants to see is a man who makes the front separately, so that when the handle is wrenched off in a year — and it will be — the front can come away without the sides being destroyed. That is not extra work. It is the same work, arranged so that the thing most likely to fail is the thing easiest to replace.

The third is smaller and it tells him the most. When the drawer is finished, does the man slide it in and out a few times, with weight in it, before he says it is done? Or does he wipe his hands and announce it?

One drawer, ninety minutes. Antony can tell more from it than from four days of cupboard, because a cupboard hides habits and a drawer does not. And the men who ask, arrange for failure, and test are the men who are still there five years later.

3 The idea in plain English

Antony's drawer is the object-oriented design round. It is small on purpose, so that habits show. His three tests — *ask before cutting*, *arrange so the likely failure is cheap*, *prove it works* — are the three things the interviewer is scoring, and almost nobody is told that in advance.

What an object actually is

An **object** is data and the behaviour that belongs to that data, kept together in one place. A **class** is the description; an object is one thing made from that description. If you have a `ParkingSpot` class, one particular spot — number 34, on level 2, currently occupied — is an object.

That “kept together” is the entire point and it is easy to miss. A design where `ParkingSpot` holds `is_free` and some other class reaches in to decide whether the spot is free has *not* used objects. It has used a record with a name on it. Where behaviour sits is what this whole phase is about, and [day 044](#) starts on it properly.

The four questions the round is actually asking

One: can you turn vague words into a bounded model? The prompt is always under-specified — “design a parking lot” is nine words. A parking lot has gates, spots, vehicles, tickets, payment, pricing, floors, displays, reservations and season passes, and forty-five minutes contains about a third of that. The skill being tested is choosing the third, saying which third, and saying what you are leaving out. Antony's four questions before the first cut.

Two: do you know where behaviour belongs? Given a rule — “a bike may park in a bike spot or a large spot, never in a compact spot” — where does that rule live? In the `Vehicle`? In the `ParkingSpot`? In a `ParkingLot` that knows about both? There is a defensible answer and there are indefensible ones, and the interviewer is watching you reason about it rather than watching you get it right.

Three: is your design cheap to change? They will ask, at minute thirty-five, “now add electric vehicle charging spots” or “now the pricing is per-hour on weekdays and flat on weekends”. A good design absorbs that in one new class and one line. A bad one needs a change in five places, and the interviewer will make you name all five. Antony’s replaceable drawer front.

Four: does it run? Not all of it. One flow — a vehicle arrives, gets a spot, gets a ticket, leaves, pays — written as real code, with the method bodies that matter actually filled in. Candidates who produce only class names and arrows are producing a diagram, not a design.

What it is not

It is **not schema design**. The commonest wrong turn is to hear “design a parking lot” and start drawing tables with foreign keys — the whole of the phase you finished [yesterday](#). Tables are about what is stored; objects are about where behaviour lives. A `Ticket` table and a `Ticket` class look similar and are answers to different questions.

It is **not a design-patterns quiz**. Naming Factory, Singleton and Observer in the first two minutes reads as revision, not as design. Patterns arrive in this course from [day 063](#), and the right time to say a pattern’s name is when the problem has already forced its shape.

It is **not high-level design**. No load balancers, no replicas, no queues, no request-per-second arithmetic. One process, in memory, and the hardest question is where a rule lives.

The forty-five minutes, spent

minutes 0-5	clarify. Scope in, scope out, said out loud and agreed.
minutes 5-10	the nouns and verbs. Candidate classes, and what each is responsible for.
minutes 10-20	relationships, and the two or three interfaces the design turns on.
minutes 20-35	code one flow end to end. Real method bodies on the parts that matter.
minutes 35-42	the extension they throw at you. Change the design out loud.
minutes 42-45	what you left out, and what you would do next.

Say that plan at minute one — “I’ll spend five minutes on scope, then classes, then I’ll code the main flow” — and the interviewer relaxes, because now they know you will get to code.

4 The picture

The three rounds, side by side, so you stop answering the wrong one:

	DSA round	OOD / LLD round	HLD round
input	a precise problem	a vague paragraph	a product name
output	one function	6-10 classes + code	boxes, arrows, numbers
hard part	finding the trick	where behaviour lives	sizing and trade-offs
they ask	"is it correct and is it fast?"	"can someone else change this in a year?"	"does it survive ten million users?"
wrong turn	over-thinking the edge cases	drawing tables, or reciting patterns	going too deep on one component

What to notice: the middle column never mentions scale and never mentions storage. If your answer contains a replica count, you are in the wrong column.

And the shape of what you produce — a parking lot, sketched the way it should be at minute twenty:

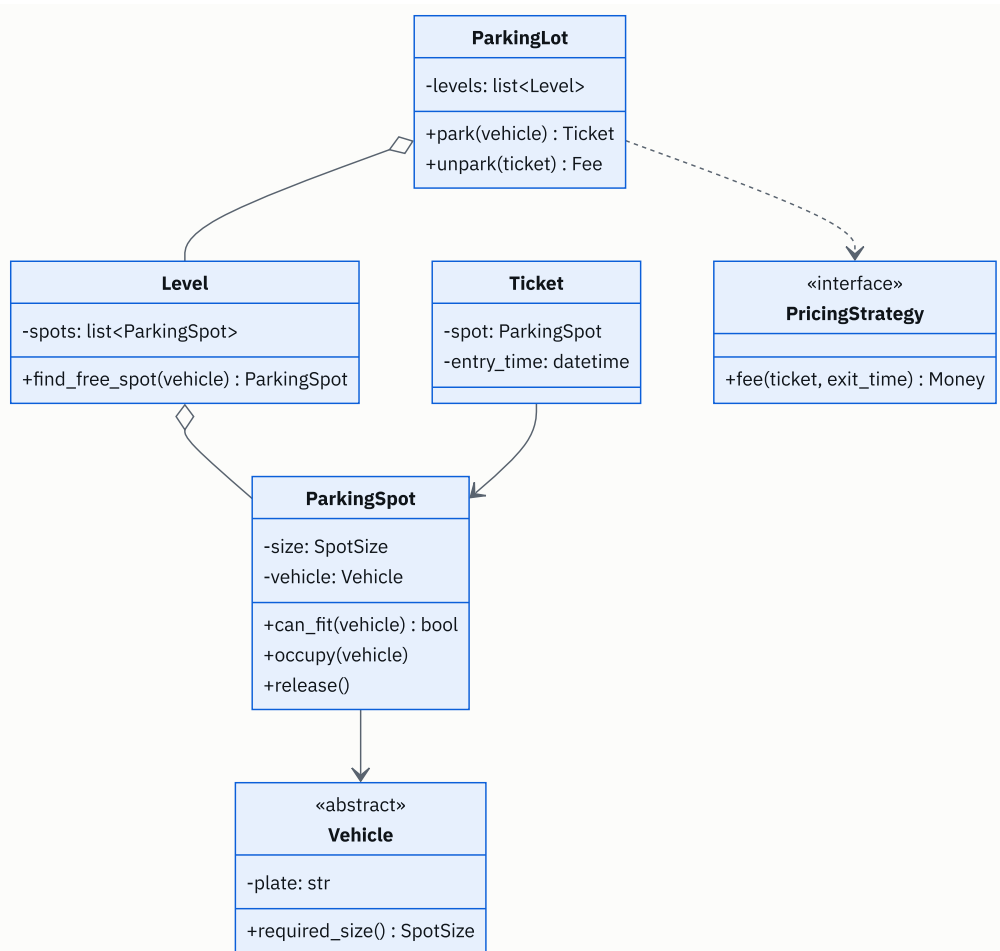


FIGURE 43.1

What to notice: `can_fit` lives on `ParkingSpot`, not in the lot. And `PricingStrategy` is an interface with nothing behind it yet — that empty box is what makes “now add weekend pricing” a one-class change instead of a five-file edit. Those two decisions are most of the score.

5 How it actually works

What the interviewer is filling in

Most product companies score this round against four or five named criteria, and they are close to identical across companies:

- **Requirement handling** — did the candidate scope, or did they start coding?

- **Modelling** — are the classes the right ones, with one clear responsibility each?
- **Extensibility** — did the design survive the change we threw at it?
- **Code quality** — does it compile in the reader's head? Are names honest? Is there a method longer than fifteen lines doing four things?
- **Communication** — did they say what they were doing and why, or did they go quiet and type?

You cannot see the form, but you can answer it. Saying “I’m putting this rule on the spot rather than the lot, because the spot owns whether it is free” is a sentence written *directly* into the modelling box.

The designs you already use every day

This is not theory. The libraries you have already touched in this course are object-oriented designs, and they are worth naming in an interview because they show the idea is load-bearing:

- **Python’s logging module** splits into `Logger`, `Handler`, `Formatter` and `Filter`. A logger does not know how to write to a file; a handler does. That is why you can add a handler that posts to Slack without touching a single line of logging code anywhere in your program. It is the textbook example of separating *what happened* from *what to do about it*.
- **Java’s List interface**, with `ArrayList` and `LinkedList` behind it. Code written against `List` does not change when you swap the implementation — the same idea you will meet as abstraction on [day 048](#).
- **Python’s collections.abc** — `Iterable`, `Sized`, `Mapping`. Implement `__len__` and your class is `Sized` and works everywhere `len()` is called. Behaviour defined by what an object can do, not by what it inherits from.
- **Django’s Storage** — `FileSystemStorage` and `S3Storage` behind one interface, which is why moving uploads to S3 is a settings change.
- **Stripe’s SDK** — `PaymentIntent`, `Customer`, `Refund`. Each object carries the operations that belong to it: `intent.confirm()`, not `stripe.confirm_intent(intent)`.

Every one of those is a design where behaviour was put next to the data it needs, and the payoff shows up years later as a change that touched one file.

The five failures, and what each looks like

- **The god class.** One `ParkingLot` with eleven methods that does spot allocation, pricing, ticket issue and payment. Symptom: every extension question modifies it.

Four edits to existing logic against one line. Each edit to existing logic is a chance to break something that worked, so the second design is not merely tidier — it is cheaper in the only unit that matters, which is risk.

How much this round is worth

```
a typical product-company loop: 4-6 rounds
  2 x DSA
  1 x OOD / LLD
  1 x HLD (often only for 3+ years of experience)
  1 x behavioural / hiring manager
```

so OOD is 1 in 4 to 1 in 6 of the decision -- roughly 20% -- and it is the round candidates prepare least, which makes it the cheapest place to gain ground.

7 The trade-offs

Modelling richly against finishing

Every extra class you introduce buys clarity and costs minutes. Ten well-chosen classes with two coded flows beats twenty classes with none. *I would not model the payment gateway, the receipt printer and the season-pass renewal in a parking-lot round* — I would name them as out of scope in minute three, and spend the time on spot allocation and pricing, because those are where the interviewer's extension questions live.

Interfaces against directness

An interface with one implementation is speculative; an interface with two is a design. *I would not introduce an interface for something I cannot name a second implementation of.* Pricing gets one because flat-rate and hourly are both real. `Ticket` does not, because there is only ever one kind. That single test — *can I name the second implementation?* — kills most over-engineering before it starts, and it is a good sentence to say out loud.

Patterns against plain code

A named pattern is a compression: it lets two engineers agree on a shape in one word. It is also the easiest thing in this subject to fake. *I would not reach for a pattern before the problem has forced its shape.* The honest order is to write the plain design, notice the pain, then say “this is essentially the strategy pattern” — which lands as recognition rather than recitation.

Depth against breadth, when the clock is against you

Given twelve minutes left and half a design, go deep on one flow rather than wide on all of them. A complete, working `park()` that handles a full lot and an oversized vehicle is worth more than five half-written methods. *I would not spread the last ten minutes evenly*

— I would say “I’ll make the parking flow complete and leave the exit flow as method signatures” and then do exactly that.

The honest sentence

This round is not asking whether you can build a parking lot. It is asking whether the person who inherits your code in a year will be able to add a requirement without being afraid. Everything you say should be aimed at that person.

8 In the interview

How it gets asked

- “Design a parking lot.” — the canonical one, at Amazon more than anywhere. Nine words, and the scoping is half the score.
- “Design an elevator system for a ten-floor building.” — the same round with state machines in it.
- “Design Splitwise / a vending machine / a chess game / a food-ordering system / a rate limiter.” Different nouns, identical scoring.
- “Here’s a feature request for our product. Sketch the classes.” — the honest version, and increasingly common at companies that dislike puzzle prompts.

What to say out loud, in the first ninety seconds

1. **Announce the plan.** “I’ll take about five minutes on requirements and scope, then identify the core classes and their responsibilities, then code the main flow properly. Stop me if you’d rather I go deeper somewhere.”
2. **Ask the four that change the model.** For a parking lot: “How many vehicle types and spot sizes? Is pricing flat or time-based? Is there one entrance or several — does that mean concurrent allocation? And do I need reservations, or is it first-come?” Four questions, one minute.
3. **Say the scope out loud and get agreement.** “I’ll model spots, vehicles, allocation, tickets and pricing. I’m leaving out payment processing, season passes and the display boards — tell me if any of those is actually the interesting part.”
4. **Name the classes with their responsibilities, not just their names.** “`ParkingSpot` owns whether it’s free and whether a given vehicle fits. `Level` owns finding a free spot. `ParkingLot` coordinates and owns nothing else. `PricingStrategy` is an interface because pricing is the thing most likely to change.”

5. Then start coding, and narrate the decisions. *“I’m putting `can_fit` on the spot rather than on the lot, because the spot is what knows its own size.”*

The follow-ups

“Now support electric vehicles that need charging, and charge them differently. What changes?” This is the extensibility probe and it is coming in every one of these rounds, so I would answer it by pointing at the seams I built rather than by improvising. Charging spots are a new kind of spot, so that is a new class implementing the same spot interface — it adds a charger and overrides `can_fit`, and nothing that allocates spots needs to know it exists, because allocation asks a spot whether it fits rather than checking its type. Charging fees are a new pricing rule, so that is a new `PricingStrategy` implementation, and the lot is handed one at construction rather than choosing one internally. So the change is two new classes and one line at the wiring point. What I want to draw attention to is the alternative: if I had written `if spot.type == "compact"` branches inside the lot, this same request would edit four existing methods, each of which currently works, and every one of those edits is a chance to break parking for ordinary cars. The point of the seams is not elegance, it is that new requirements land in new files.

“Two cars arrive at two entrances at the same instant and there’s one spot left. What happens?” As designed, both threads can read the spot as free and both can take it, which is the same check-then-act race as the lost update from [day 034](#) — the shape is identical even though there is no database here. The fix belongs on the object that owns the resource, which is the spot or the level, never on the caller. Concretely, I’d make the allocation an atomic claim rather than a read followed by a write: a single synchronised method on `Level` that finds and marks a spot while holding the level’s lock, or a compare-and-set on the spot’s occupancy so that the loser gets `None` back and retries. The wrong answer, which I’d say explicitly so the interviewer knows I considered it, is to lock the whole `ParkingLot` for the duration — that serialises every entrance in the building for the sake of a rare collision. The general rule I’m applying is that a lock should cover the smallest thing whose invariant it protects, and the invariant here is “one spot, one vehicle”.

“You’ve got five minutes left and the exit flow isn’t written. What do you do?” I say so and choose, out loud, rather than trying to finish everything badly. With five minutes I’d write `unpark` as a complete method — look up the ticket, compute the fee through the pricing strategy, release the spot, return the fee — because it exercises two collaborations and proves the design holds up in the direction I haven’t shown yet. I’d leave payment as a signature with a comment saying what it would do. Then I’d spend the last minute on what I deliberately left out and what I’d build next: reservations, multi-entrance concurrency beyond the single lock, and persistence. Naming your own gaps is worth real marks, because the alternative reading is that you did not see them.

A model answer

“Before I model anything, let me pin the scope, because ‘design a parking lot’ could be four classes or forty.

Questions: how many vehicle and spot types, is pricing flat or hourly, one entrance or several, and do I need reservations? Say three vehicle types, hourly pricing, multiple entrances, no reservations. Then I’ll model spot allocation, tickets and pricing, and I’ll leave payment processing and display boards out — tell me if either of those is where you wanted to go.

Now the classes, each with one responsibility. `Vehicle` is abstract with `Car`, `Bike` and `Truck` under it; the only thing it knows is what size of spot it needs. `ParkingSpot` owns its size and whether it’s occupied, and it answers `can_fit(vehicle)` — I’m putting that on the spot rather than on the lot, because the spot is the thing that knows its own dimensions, and if I put it in the lot I’ll be writing a type-switch that has to be edited every time a vehicle type is added. `Level` owns a collection of spots and finding a free one. `ParkingLot` owns the levels and coordinates the two flows, and deliberately owns no rules of its own. `Ticket` records the spot and the entry time. And `PricingStrategy` is an interface — flat and hourly implement it — because pricing is the requirement most likely to change, and I’d rather that change arrive as a new class than as an edit.

That’s six classes and one interface, which is about right for forty-five minutes.

Let me code the parking flow properly: `ParkingLot.park(vehicle)` asks each level for a free spot, the level asks each spot `can_fit`, the first match is occupied, a ticket is created with the entry time, and it comes back. Full-lot case returns `None` rather than raising, since it’s an expected outcome rather than an error.

Two things I’d flag as I go. Multiple entrances means two threads can claim the same last spot, so the claim has to be atomic at the level, not a read-then-write in the caller — same shape as a lost update in a database, and I’d fix it at the object that owns the resource. And I’d resist adding a factory or a builder here; there are six classes and the construction is trivial, so a pattern would be noise. If you add reservations later, that’s where a strategy for allocation starts to earn its place.”

9 Recall card

- **The round tests one thing: can somebody else change this code in a year.** Score comes from scoping, from where behaviour lives, from surviving the extension question, and from narrating all three.

- **It is not schema design, not a patterns quiz, not HLD.** No tables, no replica counts, no pattern names before the problem has forced the shape.
- **Spend the 45 minutes: 5 scope · 5 classes · 10 relationships · 15 code one flow · 7 extension · 3 gaps.** Say the plan at minute one. Aim for 6-10 classes and one flow that actually runs.
- **Behaviour goes next to the data it needs.** `spot.can_fit(vehicle)`, never a type-switch in the coordinator. The anaemic model — fields plus a service class — is objects in name only.
- **The extensibility test is “name the second implementation”.** If you can, make an interface; if you cannot, do not. Measure a design in edits to *existing* logic: 4 edits versus 1 line is the whole argument.

Practice — Writing binary search without off-by-one bugs

DSA topic: Writing binary search without off-by-one bugs **System design topic:** Why interviews ask object-oriented design at all

Code these, in this order

One rule today, and it is strict: **solve all four with the same six-line template**. If you find yourself writing `- 1` anywhere, you have slipped back to yesterday's convention — stop and restart the function.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Search Insert Position	LeetCode 35 (Easy)	The lower bound with nothing added — and that “one past the end” is a real answer.
2	First Bad Version	LeetCode 278 (Easy)	The template with no array at all; the question is an API call.
3	Find Smallest Letter Greater Than Target	LeetCode 744 (Easy)	The upper bound, plus a wrap-around that the template does not solve for you.
4	Find First and Last Position of Element in Sorted Array	LeetCode 34 (Medium)	Both bounds, and whether you resist the find-then-walk shortcut.

On problem 1, write the invariant as a comment first

Literally type the sentence — *below lo all False, from hi up all True* — as the first line of the function, then write the six lines under it. Delete the comment afterwards if you like. Doing this five times is what makes the template automatic.

On problem 2, notice what changed and what did not

Nothing about the loop changes. The question becomes `isBadVersion(mid)`, the range becomes `1` to `n + 1`, and the answer is still “the first True”. Use `lo + (hi - lo) // 2` here — `n` reaches $2^{31} - 1$ and this problem exists partly to make that point. Then say out loud what the cost is in *calls*, not comparisons: $\log_2(2^{31}) = 31$ calls to the API.

On problem 3, the template gives you most of it

The upper bound over the letters gives the first letter strictly greater than the target. It returns `len(letters)` when there is none — and this problem wants a wrap to `letters[0]` in that case. Write the wrap as a separate line after the search, not as a spe-

cial case inside the loop. Keeping the template pure and handling the problem's quirk outside it is the habit worth building.

On problem 4, do two searches, not one search and a walk

The tempting version finds any match and walks outward. Build the array `[8] * 200000` with target 8 and time both. The walk is $O(n)$ and the two bounds stay at eighteen comparisons each. Then say the answer for the empty case: both bounds are equal, and `[-1, -1]` falls out of `lo == hi` without a separate branch.

The convention drill

For each line below, say which convention it belongs to — closed `[lo, hi]` or half-open `[lo, hi)` — and what happens if you use it in the other one:

```
1. hi = len(nums)
2. while lo ≤ hi
3. hi = mid
4. hi = mid - 1
5. return lo
6. return -1
```

Then say the one-sentence rule that generates all six.

The termination drill

Answer out loud, in under a minute: why does `hi = mid` never hang in the half-open template, and why *does* it hang in the closed one? The answer is a single inequality about `mid` and `hi`. If you cannot produce it, re-read §6 — this is the question interviewers use to separate people who memorised the template from people who own it.

The one-character drill

Starting from your working `lower_bound`, produce each of these by changing one character or adding one line, and say the answer before you run it on `[2, 4, 4, 4, 7, 9]`:

1. First index with value `> 4`.
2. Last index holding 4.
3. How many 4s there are.
4. Whether 5 is present.
5. Where 5 would be inserted.

The OOD round drill

Set a timer for four minutes and speak, out loud, the opening of a *design a vending machine* round:

1. The plan sentence — how you will spend the forty-five minutes.
2. Four clarifying questions that would actually change the model.
3. The scope you are taking and the scope you are dropping, said explicitly.
4. Six to eight class names, each with the one thing it is responsible for.
5. The one interface you would introduce, and the second implementation that justifies it.

Then answer the extension you know is coming — “*now it accepts card payments as well as coins*” — in terms of files added versus existing methods edited. If your answer edits more than one existing method, the seams are in the wrong place; move them and say the new answer.

The wrong-round drill

For each sentence, say which round it belongs to — DSA, OOD, or HLD — and what the right sentence for an OOD round would be instead:

1. “I’d shard by user id.”
2. “That’s $O(n \log n)$ because of the sort.”
3. “`orders` would have a foreign key to `customers`.”
4. “I’d put a Redis cache in front of it.”
5. “I’ll use a Singleton for the inventory.”

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Write binary search on the whiteboard. No compiler.* Convention first, then the invariant, then the six lines, then what the return value means and what a caller must check before using it.
2. *Design a class structure for this feature.* The plan sentence, the four clarifying questions, the scope you keep and drop, and the rule for where behaviour lives. Do not name a pattern.
3. *Your binary search loop hangs. Talk me through why.* The `mid < hi` inequality, the two conventions, and the exact pairing that breaks — `hi = mid` inside a `while lo ≤ hi` loop.

Before you move on

- [] I solved all four problems with one template and wrote `- 1` nowhere.
- [] I can state the half-open invariant and say which line preserves it.
- [] I can explain termination with the `mid < hi` inequality, unprompted.
- [] I produced all five one-character variants without re-deriving the loop.
- [] I ran the four-minute vending machine opening, including the scope I dropped.
- [] I can sort the five wrong-round sentences and give the OOD replacement for each.
- [] I answered all three questions above out loud.

First and last occurrence

DATA STRUCTURES AND ALGORITHMS

First and last occurrence

You can find the boundaries of a repeated value in $O(\log n)$.

SYSTEM DESIGN

Classes and objects

You can turn a paragraph of requirements into classes with the right responsibilities.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the first and last position of a target in a sorted array.*
- *Model a library. What are your classes?*

1 What this is, and why they ask it

A sorted array can hold the same value many times, and those copies always sit together in one unbroken run. Today you find both ends of that run — the index of the first copy and the index of the last — in $O(\log n)$, using yesterday's template twice with one character different between the two calls. Everything else about the problem is a consequence of that.

They ask it because it is the shortest problem that punishes the obvious answer. Almost every candidate's instinct is to binary search for any copy and then walk outward to the edges, and that instinct is correct on the test cases and catastrophic on the input the interviewer is actually holding: an array of a million identical values, where the walk is $O(n)$ and undoes the whole point of the search. LeetCode 34 is a top-thirty interview question at product companies, and it is asked precisely to see whether you notice. It also unlocks a family — counting occurrences, finding the range of a value, and every “how many things satisfy this” question that reduces to two boundaries subtracted.

2 The story

Reshma needs a plain white shirt for her brother, and the shop on the second floor of the mall has one long rail of them against the back wall, maybe four hundred shirts, all the same design.

They are hung by size, smallest at the left, largest at the right. The tag is a small card sewn into the collar and you have to lift the shoulder to read it. Her brother takes a 38.

The first time she did this she started at the left end and flicked along, shirt by shirt, lifting collars. It took eleven minutes and her arms ached, and the 38s did not begin until nearly the middle of the rail.

Now she does something else, and she does it twice.

The first hunt is for the place the 38s begin. She pushes into the middle of the rail and lifts a collar: 42. Too big, so every shirt from here rightwards is 42 or more and none of them is what she wants. She moves left, into the middle of what is left: 34. Too small — so this one is not the start either, and neither is anything to its left. She keeps closing in, four or five lifts, until the two edges of her search meet, and that is where the 38s start.

The second hunt is for the place they stop, and it is the same hunt with the question changed. Not “where does 38 begin” but “where does anything bigger than 38 begin”. Middle, lift, too small, move right. Middle, lift, 40 — bigger than 38, so the 38s have ended at or before here. Four or five lifts again, and she has the far edge.

Two hunts, about ten collars lifted, and she knows exactly which stretch of the rail is hers. She takes the four shirts in it to the light by the window and picks one.

She learnt to do it as two hunts on the day the shop had a delivery and there were nearly two hundred 38s hanging in a row. She had found one 38 quickly enough, in the middle of the run, and then walked left to find where it started — and walking left through a hundred and something shirts took as long as flicking the whole rail had. Finding one of them is easy. Finding the *edge* is the job.

3 The idea in plain English

Reshma’s rail is the sorted array, the size on the tag is the value, and her two hunts are two calls to [yesterday’s template](#). The delivery day is the input that kills the shortcut.

The run is unbroken

In a sorted array, equal values are adjacent. There is no way to have a 4 at index 1 and another 4 at index 6 with a 7 in between, because that would break the ordering. So the copies of a value form one **run**: a contiguous stretch, described completely by its two ends.

```
nums = [2, 4, 4, 4, 7, 9, 11]
```

The run of 4s is indices 1 to 3. Two numbers describe it. That is the entire problem.

Two boundaries, one template

Yesterday’s template finds the **first index where a yes-or-no question becomes true**, and it needs the question to be monotone. Here are the two questions:

question A:	<code>nums[i] ≥ 4</code>	no, YES, YES, YES, YES, YES, YES	first True at 1
question B:	<code>nums[i] > 4</code>	no, no, no, no, YES, YES, YES	first True at 4

- The first `4` sits at the first index where `nums[i] ≥ 4`. That is the **lower bound**, and it is index 1.
- The **upper bound** is the first index where `nums[i] > 4`, which is index 4 — one past the run. So the last `4` is at `upper_bound - 1` = 3.

Both questions are monotone on a sorted array, so both are one call to the same six lines.

The check that must not be skipped

The lower bound always returns a position, even when the target is absent. Search for 5 in that array and you get 4 — the position where 5 *would* go. That index is in range and holds a 7, not a 5.

So the answer is only valid if:

```
first < len(nums) and nums[first] == target
```

Both halves matter. `first < len(nums)` guards the “target bigger than everything” case, where the lower bound is `len(nums)` and indexing it raises. `nums[first] == target` guards the “target absent but in range” case, which raises nothing at all and quietly returns a wrong answer. The `and` short-circuits, so the order is load-bearing.

If the check fails, return `[-1, -1]`. And note: you only need to run it once. If the lower bound is not a real match, there are no copies at all, so the upper bound cannot rescue it.

What falls out for free

```
count of target      = upper_bound - lower_bound
last occurrence     = upper_bound - 1
does target exist    = lower_bound is valid
how many are < x     = lower_bound(x)
how many are ≤ x     = upper_bound(x)
how many are in [a,b] = upper_bound(b) - lower_bound(a)
```

That last one is worth stopping on. “How many values lie between 10 and 20 inclusive” is two binary searches and a subtraction — `O(log n)` — where the obvious answer is a scan. This is the shape behind a large number of “count the things that satisfy a range condition” interview questions.

Why not find-then-walk

The tempting version: binary search for any copy, then walk left while the value is the same, then walk right. It is correct. It is also `O(log n + k)` where `k` is the number of copies, and `k` can be `n`. On `[8] * 1_000_000` searching for 8, the search takes twenty comparisons and the walk takes a million. **The cost of the walk is exactly the thing the problem is about**, which is why the interviewer chose an array with duplicates in it.

4 The picture

The two questions, drawn on the same array:

index	0	1	2	3	4	5	6
nums	2	4	4	4	7	9	11
≥ 4 ?	no	YES	YES	YES	YES	YES	YES
> 4 ?	no	no	no	no	YES	YES	YES

$\wedge$ $\wedge$
lower_bound = 1 upper_bound = 4

|<--- the run --->|
first = 1 last = upper - 1 = 3
count = 4 - 1 = 3

What to notice: the two rows are the same array asked two different questions, and both rows are monotone. Nothing else about the array matters.

The half-open range as a picture of ownership:

```
[ lower_bound , upper_bound )
      1           4
indices 1, 2, 3 -- exactly the copies of 4

the run is a half-open slice. nums[1:4] is [4, 4, 4].
That is not a coincidence: Python slicing uses the same convention.
```

What to notice: `nums[lower:upper]` is the run. If you can write the slice, you have the answer, and the `- 1` for “last index” is a display detail rather than part of the reasoning.

And the absent case, searching for 5:

```
lower_bound(5) = 4      nums[4] = 7, not 5      → no copies
upper_bound(5) = 4      count = 4 - 4 = 0      → the subtraction agrees, with no special case
```

What to notice: when the target is absent both bounds land on the same index, so the count is zero automatically. The `[-1, -1]` requirement is the problem’s output format, not a case the algorithm has to think about.

5 The code, built step by step

The template, once more

```
def _first_true(lo: int, hi: int, question) → int:
    while lo < hi:
        mid = (lo + hi) // 2
        if question(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Unchanged from [day 043](#). It will stay unchanged for the rest of the phase.

The two bounds

```
def lower_bound(nums: list[int], target: int) → int:
    return _first_true(0, len(nums), lambda i: nums[i] ≥ target)

def upper_bound(nums: list[int], target: int) → int:
    return _first_true(0, len(nums), lambda i: nums[i] > target)
```

One character apart. Say that out loud in the interview — it is the sentence that shows you understand the template rather than having memorised two functions.

The validity check

```
first = lower_bound(nums, target)
if first == len(nums) or nums[first] ≠ target:
    return [-1, -1]
```

Written as an early return, which is easier to read than nesting the happy path inside an `if`. Note `first == len(nums)` comes first, so `nums[first]` is never evaluated on an out-of-range index.

The last index

```
last = upper_bound(nums, target) - 1
```

Only reached when at least one copy exists, so the `- 1` cannot produce `-1` by accident. That ordering — check first, then compute last — is what makes the minus one safe.

The complete solution

```
from bisect import bisect_left, bisect_right

def _first_true(lo: int, hi: int, question) → int:
    """Smallest i in [lo, hi) where question(i) is True, or hi if none is."""
    while lo < hi:
        mid = (lo + hi) // 2
        if question(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

def lower_bound(nums: list[int], target: int) → int:
    """First index with nums[i] ≥ target; len(nums) if every value is smaller."""
    return _first_true(0, len(nums), lambda i: nums[i] ≥ target)

def upper_bound(nums: list[int], target: int) → int:
    """First index with nums[i] > target; len(nums) if no value is bigger."""
    return _first_true(0, len(nums), lambda i: nums[i] > target)

def search_range(nums: list[int], target: int) → list[int]:
    """LeetCode 34. First and last index of target, or [-1, -1].

    Two independent binary searches: O(log n) even when every value is the target.
    """
    first = lower_bound(nums, target)
    if first == len(nums) or nums[first] ≠ target:
        return [-1, -1] # no copies at all; upper cannot help
    last = upper_bound(nums, target) - 1 # safe: at least one copy exists
    return [first, last]

def count_of(nums: list[int], target: int) → int:
    """How many copies. Zero falls out with no special case."""
    return upper_bound(nums, target) - lower_bound(nums, target)

def count_in_range(nums: list[int], low: int, high: int) → int:
    """How many values lie in [low, high] inclusive. Two searches, one subtraction."""
    return upper_bound(nums, high) - lower_bound(nums, low)

if __name__ == "__main__":
    nums = [2, 4, 4, 4, 7, 9, 11]

    print(search_range(nums, 4)) # [1, 3]
    print(search_range(nums, 2)) # [0, 0] ← run of one, at the very start
    print(search_range(nums, 11)) # [6, 6] ← run of one, at the very end
    print(search_range(nums, 5)) # [-1, -1] ← absent, inside the range
    print(search_range(nums, 1)) # [-1, -1] ← below everything
    print(search_range(nums, 99)) # [-1, -1] ← above everything, lower = len(nums)
    print(search_range([], 4)) # [-1, -1] ← empty input
    print(search_range([4, 4, 4], 4)) # [0, 2] ← every element is the target

    print(count_of(nums, 4), count_of(nums, 5)) # 3 0
    print(count_in_range(nums, 4, 9)) # 5 → 4,4,4,7,9

    # The standard library is these two functions.
    print(bisect_left(nums, 4), bisect_right(nums, 4)) # 1 4
```

Run it. The three inputs that matter are the last three of the `search_range` block: an empty list, a target above everything, and an array that is nothing but the target. A version that passes those three is almost certainly right.

The other formulation, and why it is worth knowing

Some interviewers want to see the three-way comparison rather than the boundary template. The trick there is **record and keep going** instead of returning on a match:

```
def first_occurrence_three_way(nums: list[int], target: int) → int:
    low, high, answer = 0, len(nums) - 1, -1
    while low ≤ high:
        mid = (low + high) // 2
        if nums[mid] == target:
            answer = mid                # a match, but maybe not the leftmost
            high = mid - 1              # so keep looking left
        elif nums[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return answer
```

The one line that carries the idea is `answer = mid` followed by `high = mid - 1`: on a match you refuse to stop. Mirror it — `low = mid + 1` — for the last occurrence. It is the same $O(\log n)$. Know it, because it gets asked; prefer the boundary template, because it is one function instead of two and it does not need an `answer` variable that starts as a lie.

In an interview you would use this

```
def search_range_lib(nums: list[int], target: int) → list[int]:
    left = bisect_left(nums, target)
    if left == len(nums) or nums[left] ≠ target:
        return [-1, -1]
    return [left, bisect_right(nums, target) - 1]
```

Two library calls and the same validity check. Say “in production this is `bisect_left` and `bisect_right`” after you have written the loop by hand — it shows you know the loop is a teaching exercise, not a preference.

6 What it costs

Time

Two independent binary searches over the same array:


```

lower_bound:  log2(n) passes
upper_bound:  log2(n) passes
-----
total:        2 x log2(n) = O(log n)

```

Constants do not change the class, and it is worth saying the number rather than the letter:

```

n = 1,000,000    → 20 + 20 = 40 comparisons
n = 1,000,000,000 → 30 + 30 = 60 comparisons

```

Sixty comparisons to find both edges of a run inside a billion elements.

Against the shortcut

This is the comparison the whole problem exists for:

```

array = [8] * 1,000,000, target = 8

find-then-walk:  20 comparisons to land in the middle
                  + ~500,000 steps left + ~500,000 steps right
                  = ~1,000,020 operations           → O(log n + k), k = n here

two bounds:      20 + 20 = 40 operations             → O(log n)

25,000 times fewer operations, on the input the interviewer will hand you.

```

The general statement: **find-then-walk is $O(\log n + k)$ where k is the number of copies, and k is unbounded.** Say that sentence rather than “the walk is slow” — the variable is the point.

Space

```

lo, hi, mid, first, last: five integers, whatever n is    → O(1) extra space

```

No allocation, and the two searches do not interact, so nothing accumulates.

The build cost, when the array is not already sorted

Same warning as [day 042](#). If you have to sort first, that is $O(n \log n)$ and dominates everything here. Two binary searches on an unsorted array are two wrong answers, delivered quickly.

7 The traps

The near-miss: find-then-walk

```
def walker(nums, target):
    i = lower_bound(nums, target)
    if i == len(nums) or nums[i] != target:
        return [-1, -1]
    j = i
    while j + 1 < len(nums) and nums[j + 1] == target: # ← the walk
        j += 1
    return [i, j]

import time
big = [8] * 1_000_000
start = time.perf_counter(); walker(big, 8); print(f"{time.perf_counter() - start:.4f}s")
start = time.perf_counter(); search_range(big, 8); print(f"{time.perf_counter() - start:.6f}s")
```

```
0.0621s
0.000009s
```

Correct answers, both of them. One is roughly seven thousand times slower, and it is the one most candidates write. The interviewer does not have to say anything; they just ask “what’s the complexity of that?” and wait.

The real error: skipping the range guard

```
nums = [2, 4, 4, 4, 7]
first = lower_bound(nums, 99)
print(first)
print(nums[first] == 99)
```

```
5
Traceback (most recent call last):
  File "day44.py", line 4, in <module>
    print(nums[first] == 99)
    ~~~~^~~~~~
IndexError: list index out of range
```

The lower bound returned 5, which is `len(nums)` — the honest “it would go at the end” answer — and indexing it raises. Always `first < len(nums)` **before** `nums[first]`, in that order.

The near-miss that raises nothing: skipping the equality check

```
def broken(nums, target):
    first = lower_bound(nums, target)
    last = upper_bound(nums, target) - 1
    return [first, last]

print(broken([2, 4, 4, 4, 7], 5))    # [4, 3]
print(broken([2, 4, 4, 4, 7], 3))    # [1, 0]
```

```
[4, 3]
[1, 0]
```

No exception, and the numbers look like indices. They are nonsense: `last` is less than `first`, because the run is empty. That inversion is actually the tell — **when the target is absent, both bounds are equal, so `last` comes out one less than `first`**. You could test for it, but the equality check is clearer and says what you mean.

The trap: `- 1` computed before the check

```
first = lower_bound(nums, target)
last = upper_bound(nums, target) - 1
if first == len(nums) or nums[first] != target:
    return [-1, -1]
```

Correct output, wrong shape — and on a large input it does a second binary search that it then throws away. Worse, in a variant where `upper_bound` is expensive (which it will be from [day 046](#), where a “comparison” means scanning the whole array), that wasted call doubles the runtime. Check, then compute.

The trap: assuming a sorted-by-something-else array

```
people = [("Asha", 31), ("Bala", 24), ("Chetan", 45)]    # sorted by name
```

The array is sorted, so binary search is available — but only on the name. Searching it by age with `p[1] ≥ 30` gives a meaningless answer, because that question is not monotone here. **The monotone question must match the ordering the array actually has.** Ask which key the array is sorted on; it is the same ten-second question as “is it sorted?”, one level deeper.

8 In the interview

How it gets asked

- *“Find the first and last position of a target in a sorted array. Return [-1, -1] if it isn’t there.”* — LeetCode 34, word for word.
- *“How many times does x appear in this sorted array? Better than linear, please.”* — the same two searches, subtracted.
- *“Here’s a sorted list of timestamps. How many events happened between 9am and 5pm?”* — the range count, and the most realistic phrasing of the three.
- As a **follow-up to LeetCode 704**: they ask plain binary search first, then say “now the array has duplicates and I want the leftmost”. That is the whole of today, and it arrives ninety seconds after you thought you were finished.

What to say out loud, in the first ninety seconds

1. **Name the structure.** *“Sorted array, so equal values form one contiguous run. The answer is completely described by two indices, so I’ll find them independently.”*
2. **Say the two questions.** *“The first copy is the first index where `nums[i] ≥ target` — the lower bound. The last copy is one before the first index where `nums[i] > target` — the upper bound minus one. Same template, one character different.”*
3. **Pre-empt the shortcut, out loud.** *“I’m deliberately not finding any match and walking outward. That’s $O(\log n + k)$ and k can be the whole array — on a million identical values it’s a linear scan.”*
4. **State the guard.** *“The lower bound always returns a position, even when the target is absent, so I check `first < len(nums)` and `nums[first] == target` before trusting it. If that fails, no copies exist and I return [-1, -1] without the second search.”*
5. **Give the cost.** *“Two binary searches, so $O(\log n)$ — about forty comparisons at a million elements — and $O(1)$ space.”*

The follow-ups

“Can you do it in one pass instead of two searches?” Not meaningfully, and I’d explain why rather than trying. The two boundaries are answers to two different monotone questions, and a single binary search converges on one boundary — the moment it commits to going left it has stopped being able to find the right-hand edge. There is a version that shares the first few levels of the search: run one search until the range splits around the target, then run a lower bound on the left part and an upper bound on the right part. It saves a handful of comparisons in the average case, it is strictly more code, it has more edge cases, and it is still $O(\log n)$ — so it is a worse answer to the same question. What I would say is that two searches at $2 \log n$ is already optimal in the asymptotic sense, since you cannot identify a boundary in fewer than $\log n$ comparisons and there

are two boundaries. If they are asking because they want fewer *array accesses* specifically, I'd mention that `bisect_left` and `bisect_right` in C are so much faster per comparison than anything hand-written that the constant factor conversation stops being interesting.

“The array is huge and lives on disk, so every comparison is a page read. Does your answer change?” The shape does not, but the accounting does, and this is a good question because it moves the cost from comparisons to reads. Two searches at thirty reads each is sixty page reads on a billion elements, and a page read is about 0.1 ms from [day 031](#), so six milliseconds — against a sequential scan of a billion elements, which is hours. So binary search still wins by an enormous margin. But I would say two things. First, binary search on disk has terrible locality: each read lands in an unrelated part of the file, so none of the operating system's read-ahead helps, and that is exactly the reason real databases use B-trees rather than binary search over a sorted file — a B-tree node is one page holding hundreds of keys, so a fanout of 500 covers ten million rows in three reads instead of twenty-four. Second, if the two searches are close together in the file, the second one will hit pages the first one already warmed, so the real cost of the second search is often much less than thirty reads.

“Now the values aren't numbers — they're records, and 'equal' means equal on one field. Anything change?” The template does not care what is being compared; it needs one monotone question and nothing else. So if the records are sorted by the same field I'm searching on, the question becomes `records[i].field ≥ target` and everything works unchanged. The two things I'd check before writing it are: is the array sorted by *that* field, because an array sorted by name gives me nothing when I search by age, and is the field's ordering total — no `None`s mixed in, no comparison that raises. In Python I'd also mention that `bisect` has a `key` argument since 3.10, so `bisect_left(records, target, key=lambda r: r.field)` does this natively without building a projection list. The last thing worth flagging is that the *result* is a range of records with equal keys, and within that range the order is whatever the sort produced — so if the caller needs a specific one of them, that is a stability question about the sort, which is [day 057](#), not about the search.

A model answer

“Sorted array, so the copies of the target are contiguous — one unbroken run — and two indices describe it completely. I’ll find each one with its own binary search rather than finding a match and walking, and let me say why up front: walking outward from a match is $O(\log n + k)$ where k is the number of copies, and on an array of a million identical values that’s a linear scan. The duplicates are the whole point of the question.

Both searches are the same boundary template with one character different. The first copy is the first index where `nums[i] ≥ target`. The last copy is one before the first index where `nums[i] > target`.

```
def search_range(nums: list[int], target: int) → list[int]:
    first = lower_bound(nums, target)
    if first == len(nums) or nums[first] != target:
        return [-1, -1]
    return [first, upper_bound(nums, target) - 1]
```

The guard matters and it has two halves. The lower bound always returns a *position*, even when the target isn’t present — for a target bigger than everything it returns `len(nums)`, and indexing that raises an `IndexError`. And for a target that’s absent but in range it returns a perfectly valid index holding some other value, which raises nothing and quietly gives a wrong answer. So I check the bound first and the equality second, in that order, and I do it before the second search — if there’s no first copy there’s no last one either.

Cost is two binary searches, so $O(\log n)$ — about forty comparisons at a million elements — and $O(1)$ space.

Two things I get for free and would mention. The number of copies is upper bound minus lower bound, with no third pass and no special case for zero, because when the target is absent both bounds land on the same index. And ‘how many values are between a and b inclusive’ is upper bound of b minus lower bound of a — which is the shape behind a lot of range-counting questions that look harder than they are. In production these two are `bisect_left` and `bisect_right`.”

9 Recall card

- **Equal values in a sorted array form one contiguous run** — two indices describe it, so run [yesterday’s template](#) twice, one character apart.

- **first = lower_bound ($\geq$ target); last = upper_bound ($>$ target) - 1.** And `nums[lower:upper]` is the run — the slice, not the minus one, is the idea.
- **The guard is two halves, in order:** `first < len(nums)` (or it raises) **and** `nums[first] = target` (or it lies). Check before computing `last`.
- **Never find-then-walk.** That is $O(\log n + k)$ with k unbounded — a million identical values makes it a linear scan, and that is the input you will be given.
- **Free from the same two calls:** `count = upper - lower` (zero needs no special case); values in `[a, b]` = `upper(b) - lower(a)`. `bisect_left` / `bisect_right` in production.

1 What this is, and why they ask it

A **class** is a description of a kind of thing: what it knows, and what it can do. An **object** is one particular thing made from that description, with its own values for everything the class said it would know. `Book` is a class; the battered copy of *Malgudi Days* with the torn cover, currently lent to Priya, is an object.

They ask it because every low-level design round begins here and most candidates fumble the first five minutes. Given a paragraph — “we run a library, members borrow books, there are late fees” — the job is to name six to ten classes, say what each one is responsible for, and put each rule on the class that owns the data the rule needs. Candidates who have only used classes as containers for database rows produce a list of nouns with getters, which scores as nothing. The interviewer is watching for one thing above all: **does behaviour live next to the data it acts on?** That question is asked and answered today, and every remaining day of this phase is a refinement of it.

2 The story

Dhanraj makes idols in a lane behind the market in Pen, and from June onwards it is the only thing he does.

In the corner of his workshop there is a mould. It is heavy, it is made in two halves that clamp together, and it is the reason all his idols look the same: the same seated pose, the same height — a foot and a half — the same tilt of the head. He has had it eleven years. He does not sell it and he does not lend it. If it cracked in July his season would be finished.

From that one mould he makes something like three hundred idols between June and the end of August.

And here is the thing about the three hundred. They come out of the same shape, but from the moment each one leaves the mould it starts having a life of its own. Number forty-one dried in the sun and has a hairline mark down the back that nobody will see once it is painted. Number ninety is the one his nephew painted, so the eyes are slightly wrong, and it is going cheap. Number two hundred and twelve was paid for in advance on the fourteenth of July by a family from Alibag who are coming to collect it on the first day. Number two hundred and eighteen looks exactly like it — same pose, same paint, same everything — and it is still for sale.

That last pair is the one that matters, and Dhanraj learnt it the hard way, once, years ago. Two idols can be identical in every way you could measure and still not be the same idol. One has been paid for and one has not. When the Alibag family arrives, handing them 218 instead of 212 would be handing them a different thing that happens to look the same, and the books would be wrong for the rest of the season. So he ties a strip of cloth with a number inked on it round the base of every one, the day it comes out.

The mould tells you what an idol is. It cannot tell you which one is cracked, which one is sold, or which one is going to Alibag on Tuesday. Only the idol can tell you that.

3 The idea in plain English

The mould is the class. Each idol is an object. The numbered strip of cloth is the object's **identity**, and the hard-won lesson — two identical idols are still two idols — is the difference between being equal and being the same.

The class and the instance

A **class** names a kind of thing and says two things about it: what every one of them knows (**attributes**, sometimes called fields or state) and what every one of them can do (**methods**, the behaviour).

```
class Idol:
    def __init__(self, number: int, height_cm: int) → None:
        self.number = number          # this idol's own value
        self.height_cm = height_cm
        self.is_sold = False
```

`__init__` is the **constructor**: the code that runs when a new object is made, whose job is to leave the object in a valid state. `self` is the object being made — the particular idol, not the mould. Every attribute assigned through `self` belongs to that one object.

Making objects is called **instantiation**, and each object is an **instance** of the class:

```
a = Idol(212, 45)
b = Idol(218, 45)
a.is_sold = True
print(a.is_sold, b.is_sold)      # True False
```

One class, two objects, separate state. That separateness is the whole reason objects exist.

Where behaviour goes

Now the important part, and the part that decides your score in an interview. Consider the rule “*an idol can be sold only if it is not already sold.*” Two places it could live:

```
# Wrong: the rule is outside the object that owns the data.
if not idol.is_sold:
    idol.is_sold = True
    ledger.record(idol.number)
```

```
# Right: the object that owns is_sold owns the rule about is_sold.
class Idol:
    def sell(self) → None:
        if self.is_sold:
            raise ValueError(f"Idol {self.number} is already sold")
        self.is_sold = True
```

In the first version, the rule is written at one call site, and the second call site somewhere else in the program will forget it. In the second version, there is nowhere to sell an idol that skips the check. **Behaviour belongs to whoever owns the data it needs.** That is the sentence, and it is worth saying in exactly those words when the interviewer asks why you put a method where you put it.

A design where classes hold only data and all the rules live in some `IdolService` is called an **anaemic model** — objects in name only — and it is the most common way candidates lose this round.

One class, one responsibility

Ask of every class: *what is the one thing it is responsible for?* If the answer needs an “and”, you probably have two classes.

<code>Idol</code>	→ its own condition and whether it is sold
<code>Order</code>	→ which idols a customer has reserved, and the total
<code>Customer</code>	→ who they are and how to reach them
<code>Workshop</code>	→ the collection of idols, and finding an unsold one
<code>PriceList</code>	→ what an idol of a given finish costs

Five sentences, no “and” in any of them. Compare that with a single `Workshop` class holding three hundred idols, the orders, the prices and the customers — that is a **god class**, and every future requirement edits it.

Finding the classes: nouns and verbs

Given a paragraph of requirements, the mechanical starting point is to underline the nouns and the verbs.

“A member may borrow up to five books. Each book has copies. If a copy is returned late, a fee is charged at ₹2 a day.”

Nouns become candidate classes: `Member`, `Book`, `Copy`, `Loan`, `Fee`. Verbs become candidate methods, and — this is the step people skip — each verb tells you *which* class it belongs to. “Borrow” needs to know the member’s current loan count, so it starts on `Member`. “Charge a late fee” needs the due date and the return date, which live on `Loan`, so it belongs there.

Two warnings. Not every noun is a class — “five” is not, and neither is “day”. And the most valuable class in a design is often a noun that was *not* in the paragraph: `Loan` does not appear in that sentence at all, but without it there is nowhere to put the due date, and every design that omits it ends up with a mess of dates on `Copy`.

Being equal against being the same

Two objects can hold identical values and still be different objects.

```
a = Idol(212, 45)
b = Idol(212, 45)
print(a == b)      # False by default
print(a is b)      # False
```

`is` asks *are these the same object* — identity, Dhanraj’s thread. `==` asks *are these equal* — and by default Python answers by identity, which is why `a == b` is False even though every value matches. If your class has a natural notion of equality, you say so:

```
class Idol:
    def __eq__(self, other: object) → bool:
        return isinstance(other, Idol) and self.number == other.number

    def __hash__(self) → int:
        return hash(self.number)
```

Two rules go with this. Define `__hash__` whenever you define `__eq__`, or the object stops working in sets and as a dictionary key — Python sets `__hash__` to `None` for you as a warning. And **hash on something that never changes**; hashing on a mutable field means an object placed in a set can become unfindable in the set it is already in.

Class-level against instance-level

Some things belong to the mould, not to the idol:

```
class Idol:
    HEIGHT_CM = 45          # class attribute: shared by all instances
    made_this_season = 0    # a counter that belongs to the class

    def __init__(self, number: int) → None:
        self.number = number # instance attribute: this idol's own
        Idol.made_this_season += 1
```

The test is simple: *would every instance have the same value?* If yes, it is class-level.

4 The picture

The mould and the idols, as memory:

```
CLASS Idol (one, shared)
+-----+
| HEIGHT_CM = 45 | attributes every instance shares
| sell()         | ← one copy of the code
| repaint()      | ← one copy of the code
+-----+
  ^           ^           ^
  |           |           |
+-----+ +-----+ +-----+
| number | | number | | number |
| = 212  | | = 218  | | = 41   |
| is_sold | | is_sold | | is_sold |
| = True  | | = False | | = False |
+-----+ +-----+ +-----+
object a   object b   object c
```

every object holds a reference back to its class; the methods are NOT copied 300 times

What to notice: the code exists once and the state exists three hundred times. That is exactly why `Idol.HEIGHT_CM` is not stored three hundred times, and why `self` has to be passed to every method — the method needs telling which idol it is acting on.

Where behaviour goes, drawn as the decision it really is:

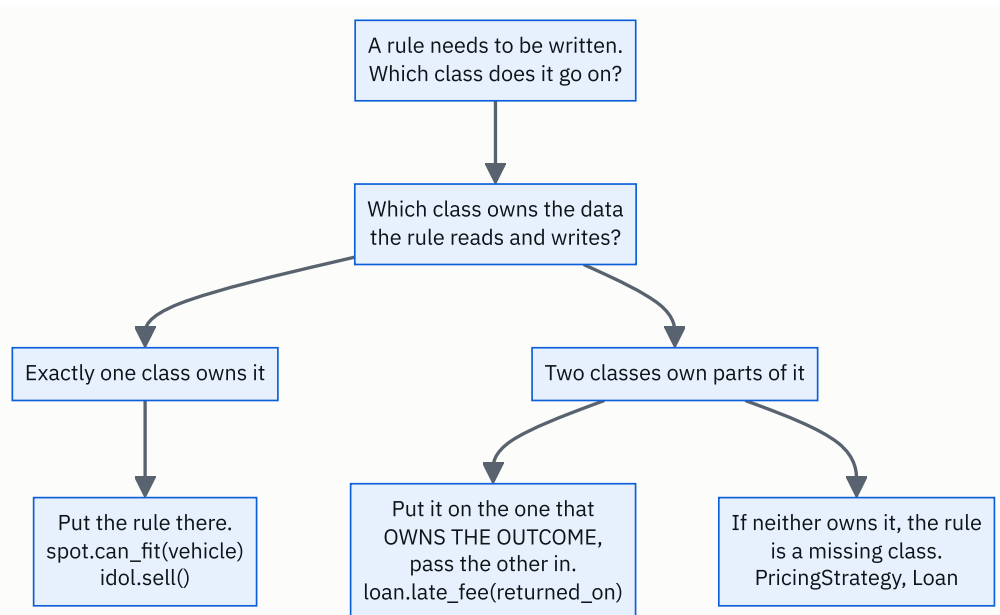


FIGURE 44.1

What to notice: the third branch is the one that produces good designs. When a rule fits nowhere, the answer is usually a class you have not thought of yet, not a service to dump it in.

5 How it actually works

What Python actually does with `self`

When you write `idol.sell()`, Python looks up `sell` on the object, does not find it, looks it up on the class, finds one function, and calls it with the object as the first argument. `idol.sell()` is literally `Idol.sell(idol)`. That is the whole mechanism, and knowing it kills two common confusions: why `self` is written out explicitly in Python and not in Java, and why the method code lives once rather than once per object.

Attributes work by dictionary lookup: each object carries a `__dict__` mapping names to values, and attribute access checks the object's dictionary first, then the class's. That is why an instance attribute silently shadows a class attribute of the same name — a real source of bugs when a counter is incremented as `self.count += 1` instead of `Idol.count += 1`.

`@dataclass` , and when to reach for it

Writing a constructor that just assigns five arguments to five attributes is noise. Python's `dataclass` writes it, plus a readable `__repr__` , plus `__eq__` if you want it:

```
from dataclasses import dataclass, field

@dataclass
class Idol:
    number: int
    height_cm: int = 45
    is_sold: bool = False

    def sell(self) → None:
        if self.is_sold:
            raise ValueError(f"Idol {self.number} is already sold")
        self.is_sold = True
```

You still add the methods yourself, which is the point — a `dataclass` removes the boilerplate, not the behaviour. `@dataclass(frozen=True)` makes instances immutable and hashable, which is the right choice for value-like things: a `Money` , a `Coordinate` , a `DateRange` .

The designs you use every day

- `datetime.date` is a frozen value object. `date(2026, 3, 14) == date(2026, 3, 14)` is True because the class defines equality by value, and it is hashable because it never changes. Compare that with a mutable `Order` , where equality by value would be wrong.
- **Django models** — `class Order(models.Model)` — are objects whose attributes map to table columns, and this is precisely where the anaemic-model habit comes from. The framework encourages fields; good Django code still puts `order.cancel()` on the model rather than in a view.
- **Stripe's SDK** gives you `PaymentIntent` objects with `intent.confirm()` and `intent.cancel()` . The alternative API — `stripe.confirm(intent_id)` — would be the same information arranged so that nothing owns the rules.
- `collections.namedtuple` and `NamedTuple` are classes for things that are only data, with no behaviour and no identity worth tracking. Reaching for one is a deliberate statement that this thing does not own any rules.

The order to build a model in, under time pressure

1. Underline the nouns; write them down as candidate classes.
2. Strike out the ones that are values rather than things — quantities, dates, names.

3. Add the missing nouns — the ones the requirements imply but never say. `Loan`, `Ticket`, `Reservation`, `Session`. These are usually the relationship between two other classes and they are where the interesting rules live.
4. For each class, write the one sentence saying what it is responsible for. If you need “and”, split it.
5. Take each verb from the requirements and place it on the class that owns the data it needs.
6. Only now, look for the rules that fit nowhere. Each one is either a missing class or an interface.

6 The numbers

What an object costs

```
a plain Python object with 3 attributes:
  object header           ~56 bytes
  instance __dict__       ~104 bytes (empty dict, grows in steps)
  3 references x 8 bytes   ~24 bytes
  -----
                        ~184 bytes, before the values themselves

the same class with __slots__ = ("number", "height_cm", "is_sold"):
  no per-instance dict
                        ~64 bytes -- roughly a third

300 idols:      184 x 300      = ~55 KB      -- irrelevant
10 million:    184 x 10^7     = ~1.8 GB     -- suddenly the design decision
                64 x 10^7     = ~640 MB      -- with __slots__
```

The rule that comes out of this: `__slots__` is a decision you make at ten million objects, not at three hundred. Say that number if an interviewer asks about object overhead; guessing “objects are expensive” without a threshold is worth nothing.

Method call cost

```
a Python attribute lookup + method call:  ~50-80 nanoseconds
a direct function call:                   ~40-60 nanoseconds
```

The difference is real and it is almost never the thing that matters. A page that is slow because of an $N+1$ query is paying 100 milliseconds — a million method calls is 60 milliseconds. Design for clarity; optimise the round trips.

How many classes a design should have

```
a 45-minute low-level design round:      6-10 classes
    fewer than 5    → under-modelled; the interviewer has nothing to probe
    more than 15    → you will not code any of them

a real service module:                    ~5-15 classes per bounded area
a class longer than ~200 lines              → almost always two classes
a method longer than ~20 lines              → almost always two methods
```

The cost of the anaemic model, counted

Take the library. The rule “a member may not borrow more than five books” written at each call site:

```
borrow from the counter UI      1 place
borrow from the self-service kiosk 1 place
bulk borrow for a school group   1 place
the import script for old records 1 place
-----
                                4 copies of one rule

a new requirement -- staff may borrow ten -- edits 4 places, and the one
somebody forgets is the one that ships.

the same rule on Member.borrow(): 1 place. Always 1 place.
```

7 The trade-offs

A class against a dictionary

A dictionary is faster to write and needs no definition. It also has no validation, no auto-completion, no place to put behaviour, and a typo in a key is a silent `None` rather than an error. *I would not use a class for a bag of values that crosses a boundary once and is never reasoned about* — a parsed JSON payload on its way to a queue is fine as a dict. *I would use a class the moment a rule attaches to the data*, because that is the moment there is somewhere for the rule to live.

A mutable object against a frozen value

Frozen objects are safe to share, safe to hash, and impossible to corrupt from a distance. They also mean every change allocates a new object. *I would freeze anything that represents a value* — money, a date range, a coordinate, a size — *and leave mutable anything that represents a thing with a life*: an order, a session, a booking. The test is whether the identity matters. Two 45-centimetre heights are interchangeable; two idols are not.

Defining `__eq__` against leaving it alone

Value equality makes tests readable and makes objects usable as dictionary keys. It also makes it easy to write a subtle bug: two objects that compare equal but represent different real things. *I would not define equality on an entity that has an identity* — two `Idol` objects with the same number are the same idol, so equality on the number is right; equality on “same pose and same paint” would be catastrophic. And if you define `__eq__`, define `__hash__`, and hash only on fields that never change.

Rich model against speed of delivery

Every class you extract costs time now and saves time later, and “later” is not guaranteed to arrive. *I would not extract a class for a rule that exists in exactly one place and has never changed.* The signal to extract is the second call site, or the first time you catch yourself writing the same `if` in two files.

The honest sentence

A class earns its existence by giving a rule somewhere to live. If you cannot name a rule it owns, you have written a dictionary with extra steps — and the interviewer will find it by asking where the validation happens.

8 In the interview

How it gets asked

- “*Model a library. What are your classes?*” — the canonical warm-up, and the first ten minutes of a full LLD round.
- “*What’s the difference between a class and an object?*” — sounds like a definition question, is actually checking whether you can talk about state versus code with an example.
- “*Where would you put this rule?*” — the real question, asked about a specific requirement halfway through a design. This is the one that is scored.
- “*Why is that a class and not a dictionary?*” — the pushback on over-modelling, and the answer is always about a rule that needs a home.

What to say out loud, in the first ninety seconds

1. **Do nouns and verbs visibly.** “*Let me pull the nouns out first: member, book, copy, loan, fee. Then the verbs: borrow, return, reserve, charge.*”

2. **Distinguish the thing from the kind of thing.** “`Book` is the title — ‘Malgudi Days’. `Copy` is the physical object on the shelf with its own barcode and its own condition. Members borrow copies, not books, and separating those two is what makes reservations possible later.”
3. **Add the noun that was not in the paragraph.** “I want a `Loan` class even though the requirements never say the word. It owns which copy, which member, borrowed on, due on — and without it the dates end up scattered across `Copy`, which is where late-fee bugs come from.”
4. **Give each class its one sentence.** “`Member` owns who they are and their borrowing limit. `Copy` owns its condition and whether it’s on loan. `Loan` owns the dates and computes the fee. `Catalogue` owns finding a copy of a title.”
5. **Place one rule out loud, with the reason.** “The five-book limit goes on `Member.borrow()`, because `Member` is the only thing that knows how many loans it already has. If I put it in a service, the kiosk code will forget it.”

The follow-ups

“Why is `Copy` a separate class from `Book`? Isn’t that over-modelling?” It would be over-modelling if a library owned one of each title, but it does not, and the distinction carries real rules. `Book` owns the things that are true of the title — the ISBN, the author, the publication year — and there is exactly one `Book` object for *Malgudi Days* however many copies sit on the shelf. `Copy` owns the things that are true of one physical object: the barcode, the shelf it lives on, the coffee stain, and whether it is currently out. Merging them forces you to duplicate the author’s name once per copy, which is the update anomaly from [day 029](#) arriving in object form — change the author’s spelling and you must change it in six places or the copies disagree. It also makes the two most common queries awkward: “how many copies of this title are available” becomes a scan for duplicated titles rather than a count on one object. So the test I’d apply is the one I’d apply to any proposed split: is there a rule or a fact that belongs to one and not the other? Here there are several in each direction, so they are two classes.

“Where does the late fee calculation go, and why not on `Member`?” On `Loan`, because `Loan` is the only object that holds both dates. The fee is a function of the due date and the return date, and both of those belong to one particular borrowing event — not to the member, who has had forty of them, and not to the copy, which has been borrowed by strangers for years. Putting it on `Member` would mean `Member` reaching into a loan’s dates to compute something about the loan, which is the shape people call feature envy: a method more interested in another object’s data than its own. The signature I’d write is `loan.late_fee(returned_on: date) → Money`. And then I’d flag the seam: the *rate* — ₹2 a day, free for staff, capped at the book’s price — is the part most likely to change, so I would not hard-code it inside that method. I’d pass a fee policy in, which is

the same interface argument as the pricing strategy from [day 043](#), and I'd only actually do it if I can name the second policy — which here I can, because staff and students already differ.

“Two `Copy` objects with the same barcode — are they equal?” They should be, and this is worth being precise about because it decides whether the model works in a set. A copy is an entity: it has an identity that persists even as everything else about it changes. The barcode is that identity, so `__eq__` compares barcodes and nothing else — not the condition, not the shelf, not whether it is on loan, because a copy that gets damaged is still the same copy. And because I define `__eq__`, I define `__hash__` on the same field, or the object silently becomes unusable as a dictionary key. The critical rule there is that the hash field must never change: if I hashed on the shelf location, moving a copy to another shelf would make it unfindable inside a set it was already in, which is a genuinely horrible bug to diagnose. That is also why value objects like `Money` are frozen — immutability makes the hash safe by construction. The contrast worth stating is that `Loan` probably should *not* define equality at all: two loans of the same copy by the same member in different months are different events, and default identity comparison is exactly right.

A model answer

“Let me take the nouns and verbs out of the requirements first, then say what each class is responsible for, then place the rules.

Nouns: member, book, copy, loan, fee. Verbs: borrow, return, reserve, charge.

The first decision is that `Book` and `Copy` are different classes. `Book` is the title — ISBN, author, year — and there’s one of them per title. `Copy` is a physical object with a barcode, a shelf and a condition, and there are six of them for a popular title. Keeping them separate means the author’s name is stored once, and ‘how many are available’ is a count rather than a scan.

The second decision is that I’m adding `Loan`, which isn’t in the requirements at all. It owns the member, the copy, the borrowed date and the due date. Without it those dates end up on `Copy`, and then a copy can only remember its current loan, so history is gone and late fees have nowhere to live.

One sentence each: `Member` owns identity and the borrowing limit. `Book` owns bibliographic facts. `Copy` owns its physical condition and availability. `Loan` owns the dates and the fee calculation. `Catalogue` owns finding an available copy of a title. `FeePolicy` is an interface, because student and staff rates already differ — that’s the second implementation that justifies it.

Now the rules, each placed with a reason. The five-book limit goes on `Member.borrow()`, because `Member` is the only object that knows its current loan count — put it in a service and the kiosk code will forget it. Marking a copy unavailable goes on `Copy.check_out()`, so there’s no way to lend a copy that’s already out. The late fee goes on `Loan.late_fee(returned_on)`, because `Loan` holds both dates; it takes a `FeePolicy` rather than hard-coding ₹2 a day.

On equality: `Copy` compares by barcode and hashes on it, because that’s an identity that never changes. `Money` I’d make a frozen dataclass so it’s safe to share and hash. `Loan` gets no `__eq__` at all — two borrowings of the same copy by the same member are different events, and identity comparison is right.

That’s six classes and one interface. If you want, the next thing I’d add is reservations, which is another missing noun and would sit alongside `Loan`.”

9 Recall card

- **Class = the mould, object = one thing from it.** One copy of the code, one set of state per object; `idol.sell()` is literally `Idol.sell(idol)`, which is why `self` exists.
- **Behaviour lives with the data it needs.** `member.borrow()`, `loan.late_fee(date)` — never a rule written at four call sites. Fields plus a `Service` class is the **anaemic model**, and it is the commonest way to lose this round.
- **Nouns → classes, verbs → methods, then add the noun that was not in the paragraph** — `Loan`, `Ticket`, `Reservation`. That missing class is where the interesting rules live.
- **One responsibility, said in one sentence with no “and”.** If the sentence needs an “and”, it is two classes; a 200-line class or a 20-line method almost always is.
- **is is identity, = is equality.** Define `__eq__` on an entity’s unchanging identity, always define `__hash__` with it, never hash a mutable field. Freeze value objects (`Money`, `date`); leave entities mutable.

DSA topic: First and last occurrence **System design topic:** Classes and objects

Code these, in this order

Same rule as yesterday: **one template, two calls, no `- 1` inside any loop**. If you catch yourself writing a `while` that walks one step at a time, stop — that is the trap this day exists for.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Find First and Last Position of Element in Sorted Array	LeetCode 34 (Medium)	The two bounds, and whether you resist find-then-walk.
2	Number of Occurrences / Count of an element	Standard (<code>bisect</code> warm-up)	That the count is one subtraction, and zero needs no special case.
3	Kth Missing Positive Number	LeetCode 1539 (Easy)	A boundary question where the array is not what you search on.
4	Find K Closest Elements	LeetCode 658 (Medium)	The bound as a <i>starting point</i> for a window, not as the answer itself.

On problem 1, run the three inputs that decide it

Before you look at anything else: `[]` with any target, `[1]` with target 1, and `[8] * 200000` with target 8. Time the last one against a find-then-walk version you write deliberately. Say the ratio out loud. That number — roughly seven thousand times — is the reason the question exists.

On problem 2, do not write a loop

`upper_bound(nums, x) - lower_bound(nums, x)`. One line. Then check it on a value that is absent and confirm you got 0 without an `if`. Then check it on an empty list. If either needed a special case, your bounds are wrong, not your subtraction.

On problem 3, name the monotone question first

The array holds present numbers; you are asked about missing ones. The bridge is that at index `i`, the count of missing numbers below `nums[i]` is `nums[i] - (i + 1)`, and that count never goes down. Say that sentence before writing anything. Then the question is `missing_count(i) ≥ k` and the template does the rest — this is the first problem where the monotone quantity is *computed* rather than read.

On problem 4, the bound is the start of the work

`lower_bound` tells you where `x` belongs; the answer is a window of size `k` around it. Two ways: expand outward with two pointers from that position ([day 028](#)), or binary search directly on the *window's left edge*. Do the first, then read about the second and say why it is $O(\log(n - k))$ instead of $O(\log n + k)$.

The walk-cost drill

Answer without running anything:

1. `[8] * 1_000_000`, target 8 — how many operations does find-then-walk do?
2. Same array, two bounds — how many?
3. What is `k` in $O(\log n + k)$, and what is the worst case for `k`?
4. On which single input do the two approaches cost the same?

The guard drill

For each, say what happens — a crash with a named exception, a silent wrong answer, or a correct answer:

1. `lower_bound(nums, 99)` on a five-element array, then `nums[first]`.
2. `lower_bound` then `upper_bound - 1` with no equality check, target absent but in range.
3. `upper_bound - 1` computed before the validity check, target absent.
4. Two bounds run on `[("Asha", 31), ("BaLa", 24)]` sorted by name, asking about age.

The free-answers drill

From the two bounds alone, in one line each, no extra loops:

1. How many values are strictly less than `x`.
2. How many are less than or equal to `x`.
3. How many lie in `[a, b]` inclusive.
4. Whether `x` is present.
5. The slice containing exactly the copies of `x`.

The modelling drill

Take this paragraph and produce a model out loud in four minutes:

A cinema has several screens. A show is a film on one screen at one time. Customers book seats for a show. A booking can be cancelled up to an hour before the show, and the seats go back on sale.

1. Nouns underlined, verbs underlined.
2. Which nouns are **not** classes, and why.
3. The class that is **not** in the paragraph but that your model needs.
4. One sentence of responsibility per class, no “and” allowed in any of them.
5. Where the cancellation rule goes, and the one-sentence reason.
6. Which classes define `__eq__`, on which field, and which deliberately do not.

The anaemic-model drill

For each, say whether it is a real class or a dictionary wearing a class’s clothes, and if it is the latter, name the rule that would fix it:

1. `class Seat: def __init__(self, row, number): ...` with only getters.
 2. `class Booking: with cancel()` that checks the hour rule.
 3. `class BookingService:` holding every rule about bookings, with `Booking` holding only fields.
 4. `class Money:` frozen, with `add()` that refuses to add different currencies.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the first and last position of a target in a sorted array.* The contiguous-run observation, the two questions one character apart, the guard in the right order, and the pre-empted shortcut with its cost in terms of `k`.
 2. *Model a library. What are your classes?* Nouns and verbs, the `Book` / `Copy` split with its reason, the `Loan` class that was not in the paragraph, and one rule placed with a justification.
 3. *Where would you put this rule, and why?* Pick any rule from the cinema drill. The answer is always “on the class that owns the data the rule reads” — say it in those words, then name what breaks if it goes elsewhere.
-

Before you move on

- [] I solved LeetCode 34 with two template calls and no walking loop anywhere.
- [] I timed find-then-walk against two bounds on a million identical values and can quote the ratio.
- [] I can state the guard as two halves, in order, and say what each half prevents.
- [] I gave all five free answers from the two bounds without adding a loop.
- [] I ran the cinema modelling drill in four minutes, including the missing class.
- [] I can tell an anaemic model from a real one and name the rule that fixes it.
- [] I answered all three questions above out loud.

Search in a rotated sorted array

DATA STRUCTURES AND ALGORITHMS

Search in a rotated sorted array

You can binary search data that is sorted, but not from the start.

SYSTEM DESIGN

Encapsulation

You can say why public fields are a design bug, with a concrete failure.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Search a rotated sorted array in $O(\log n)$.*
- *Why make a field private if you are going to add a getter anyway?*

1 What this is, and why they ask it

A **rotated sorted array** is a sorted array that has been cut at one point and had its front piece moved to the back. `[1, 3, 5, 7, 9, 11]` rotated at index 4 becomes `[9, 11, 1, 3, 5, 7]`. It is not sorted, so [day 042](#)'s test — *is the middle smaller than the target?* — no longer tells you which way to go. But one fact survives the rotation and is enough: **at least one of the two halves is still properly sorted**, and you can find out which in one comparison.

They ask it constantly. It is a top-five interview question at product companies, and it exists to separate two kinds of candidate: the one who memorised binary search, and the one who understands what binary search actually needs. The template does not need a sorted array; it needs a way to discard half. That is a different requirement, and this problem is the first place the difference bites. Expect it as the medium after a warm-up, expect the follow-up about duplicates, and expect to be asked why your answer is still `O(log n)` when the array is not sorted.

2 The story

Prithvi is a volunteer at his school's annual day, and his job for the evening is showing parents to their chairs.

The parents' block is four hundred plastic chairs in one long row along the side of the ground, brought out that afternoon from the store room behind the stage. Every chair has a number stuck on the back in white tape, 1 to 400.

The boys who laid them out worked from a stack, and the stack had been put away the wrong way round the year before. So the row does not start at 1. It starts at 240. From the left end it goes 240, 241, 242, and climbs steadily all the way to 400 — and then, with no warning at all, the next chair says 1, and from there it climbs again, 2, 3, 4, up to 239 at the far right end.

Nobody is going to move four hundred chairs at six in the evening.

A man arrives with a ticket for chair 88 and Prithvi walks him down the row.

He does not read every number. He walks to roughly the middle and reads the chair there: 330. Then he does the thing that makes this work. He already knows what the chair at the left end says, because he checked it when he started — 240. So he asks him-

self one question: *from the left end to where I am standing, do the numbers only climb?* The left end says 240, here it says 330, 240 is smaller than 330, and there is no way for the numbers to have jumped down and come back up in between. So yes, that whole left stretch runs 240 up to 330, in order, with nothing missing.

Now the easy part. Is 88 between 240 and 330? It is not. So 88 cannot be anywhere in that left stretch, and he can forget all of it. He walks right.

He does it again. Middle of what is left, chair 20. Left mark now says 331, and 331 is bigger than 20, so *that* stretch is the one with the jump in it — no use to him. But the other side, from here to the right end, must then be in order: 20 climbing up to 239. Is 88 between 20 and 239? Yes. Go right.

Four or five walks, and he is standing at chair 88. The man sits down and the show starts.

3 The idea in plain English

Prithvi's row is the rotated array. The one place where 400 is followed by 1 is the **pivot**. And his question — *does this stretch only climb?* — is the single comparison that replaces the sorted test.

What rotation does, exactly

Take a sorted array and move the front `k` elements to the back:

```
sorted_arr = [1, 3, 5, 7, 9, 11]
rotated     = [9, 11, 1, 3, 5, 7]      # rotated by 4, or equivalently cut before 9
```

The result has one **break point** — one place where a value is followed by a smaller value. `11` followed by `1`. Everywhere else the values climb. That is the definition, and everything today follows from it.

Two edge cases worth naming now, because they are what interviewers test:

- Rotated by 0 (or by `n`) gives back the original sorted array. There is no break point. Your code must handle that without a special case.
- Rotated by 1 gives `[3, 5, 7, 9, 11, 1]`, where the break is at the very end.

The fact that survives

Pick any middle index. The array splits into `left = nums[low..mid]` and `right = nums[mid..high]`. The break point sits in exactly one of them — it cannot be in both, because there is only one break. Therefore **the other half has no break in it, which means the other half is properly sorted.**

At least one half is always sorted. That is the whole idea, and it is worth saying in exactly those words.

Finding out which half

One comparison:

```
if nums[low] ≤ nums[mid]:
    # the left half climbs steadily → left is sorted
else:
    # the left half contains the break → right must be sorted
```

Prithvi's question. If the value at the left mark is smaller than the value at the middle, the stretch between them only climbs, so it holds no break. If it is bigger, the break is in there somewhere, so it is in the left and therefore not in the right.

The `≤` rather than `<` matters. When the range has shrunk to one element, `low = mid`, and `nums[low] ≤ nums[mid]` is true — a single element is trivially sorted, which is the correct answer. Using `<` there makes a one-element range take the wrong branch.

Deciding where to go

Once you know which half is sorted, you can use ordinary range logic on it, because inside a sorted half the plain comparison works again:

```
if left half is sorted:
    if nums[low] ≤ target < nums[mid]:
        # target lies inside the sorted left
        high = mid - 1
        # search left
    else:
        low = mid + 1
        # it cannot be there, so search right
else:
    # right half is sorted
    if nums[mid] < target ≤ nums[high]:
        # target lies inside the sorted right
        low = mid + 1
    else:
        high = mid - 1
```

Read it as Prithvi does. *Which stretch is in order? Is the target inside that stretch's range? If yes, go there. If no, go to the other one.* The half you discard is discarded on a proof, not a guess: a sorted stretch running 240 to 330 cannot contain 88, full stop.

Note the asymmetry in the two range checks — `nums[low] ≤ target < nums[mid]` on the left, `nums[mid] < target ≤ nums[high]` on the right. `mid` is excluded from both because it has already been compared to the target and did not match. The endpoints `low` and `high` are included because they have not.

Finding the smallest element instead

A close cousin, LeetCode 153, and it is cleaner than the search:

The minimum is the element just after the break — the only element smaller than the one before it.

That is a monotone question, so [day 043](#)'s template applies directly. The question is *is `nums[i]` less than or equal to the last element?*

```
nums      = [ 9, 11,  1,  3,  5,  7]      last = 7
≤ 7 ?      | no| no |YES|YES|YES|YES|
              ^
            first True = index 2 = the minimum
```

One template call and no branching at all. Compare `nums[mid]` with `nums[high]`, never with `nums[low]` — comparing against `low` fails on a non-rotated array, and §7 shows it failing.

4 The picture

The rotation, drawn:

```
sorted      [  1    3    5    7    9    11 ]
              ^ cut here, move the front to the back

rotated      [  9   11    1    3    5    7 ]
index        0    1    2    3    4    5
              ^----^
            the break: 11 followed by 1. Exactly one, ever.
```

What to notice: everywhere except that one join, the values climb. One break is the entire structure of the problem.

Choosing a half, searching for 3:

index	0	1	2	3	4	5
nums	9	11	1	3	5	7
	low=0		mid=2		high=5	

nums[low]=9, nums[mid]=1. $9 \leq 1$? NO
 → the break is in the LEFT half, so the RIGHT half is sorted.

right half runs nums[2]=1 up to nums[5]=7, in order.
 Is 3 in (1, 7] ? yes. → low = mid + 1 = 3.

next pass: low=3, high=5, mid=4, nums[4]=5
 nums[3]=3 ≤ nums[4]=5 ? YES → left half sorted, runs 3..5
 Is 3 in [3, 5) ? yes → high = mid - 1 = 3

next pass: low=3, high=3, mid=3, nums[3]=3 = target. Return 3.

What to notice: every discard is justified by a range, never by a single comparison. That is the difference from plain binary search, and it is the sentence to say out loud.

Where the break can sit, and why it does not matter:

break in the LEFT half
 [9 11 | 1 3 5 7]
 ^^^^^ ^^^^^^^^^
 broken SORTED

break in the RIGHT half
 [5 7 9 | 1 3 4]
 ^^^^^ ^^^^^
 SORTED broken

not rotated at all: no break anywhere, so BOTH halves are sorted
 [1 3 5 | 7 9 11] -- nums[low] ≤ nums[mid] is true, left branch taken, correct

What to notice: the un-rotated case takes the “left is sorted” branch and behaves like ordinary binary search. No special case is needed, and adding one is how people break it.

5 The code, built step by step

The frame is unchanged

```
low, high = 0, len(nums) - 1
while low ≤ high:
    mid = (low + high) // 2
    if nums[mid] == target:
        return mid
```

Closed range, ≤, exactly as [day 042](#). Only the decision about which half to keep changes.

Which half is sorted

```
if nums[low] ≤ nums[mid]:
    ...    # left half is sorted
else:
    ...    # right half is sorted
```

One comparison, and it is total: one of the two branches is always correct. `≤` handles the one-element range, where `low == mid`.

Inside the sorted left half

```
if nums[low] ≤ target < nums[mid]:
    high = mid - 1
else:
    low = mid + 1
```

If the target's value falls inside the sorted stretch's range, it is in there if it is anywhere. If it does not, it cannot be in there, so the other half is the only remaining possibility.

Inside the sorted right half

```
if nums[mid] < target ≤ nums[high]:
    low = mid + 1
else:
    high = mid - 1
```

The mirror image. Note `nums[mid] <` on this side and `< nums[mid]` on the other — `mid` is excluded from both because it was already checked and did not match.

The complete solution

```
def search_rotated(nums: list[int], target: int) → int:
    """LeetCode 33. Index of target in a rotated sorted array of DISTINCT values, else -1.

    Invariant: if target is in nums, its index is in nums[low..high].
    At every pass at least one half is properly sorted; the discard is proved by a range.
    """
    low, high = 0, len(nums) - 1

    while low ≤ high:
        mid = (low + high) // 2
        if nums[mid] == target:
            return mid

        if nums[low] ≤ nums[mid]:
            # left half is sorted
            if nums[low] ≤ target < nums[mid]:
                # target lies inside it
                high = mid - 1
            else:
                low = mid + 1
        else:
            # right half is sorted
            if nums[mid] < target ≤ nums[high]:
                # target lies inside it
                low = mid + 1
            else:
                high = mid - 1

    return -1

def find_min_rotated(nums: list[int]) → int:
    """LeetCode 153. The smallest value in a rotated sorted array of distinct values.

    The minimum is the first index whose value is ≤ the last value: a monotone question,
    so the boundary template applies with no branching at all.
    """
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[high]:
            # the break is to the right of mid
            low = mid + 1
        else:
            # mid could be the minimum: keep it
            high = mid
    return nums[low]

if __name__ == "__main__":
    print(search_rotated([9, 11, 1, 3, 5, 7], 3))    # 3
    print(search_rotated([9, 11, 1, 3, 5, 7], 9))    # 0 ← first element
    print(search_rotated([9, 11, 1, 3, 5, 7], 7))    # 5 ← last element
    print(search_rotated([9, 11, 1, 3, 5, 7], 4))    # -1 ← absent
    print(search_rotated([1, 3, 5, 7, 9, 11], 7))    # 3 ← NOT rotated at all
    print(search_rotated([3, 5, 7, 9, 11, 1], 1))    # 5 ← break at the very end
    print(search_rotated([5], 5), search_rotated([5], 6)) # 0 -1
    print(search_rotated([], 1))                    # -1

    print(find_min_rotated([9, 11, 1, 3, 5, 7]))    # 1
    print(find_min_rotated([1, 3, 5, 7, 9, 11]))    # 1 ← not rotated
    print(find_min_rotated([2, 1]))                # 1 ← two elements
```

The three inputs that matter are the not-rotated array, the break-at-the-end array, and the single-element array. A version that passes those three is almost certainly correct; a version that special-cases any of them is almost certainly wrong.

With duplicates, the guarantee breaks

LeetCode 81 allows repeats, and it changes the answer in a way worth understanding rather than patching:

```
nums = [3, 1, 3, 3, 3]
# low=0, mid=2, high=4:  nums[0]=3, nums[2]=3, nums[4]=3 -- all equal.
# Which half is sorted? There is no way to tell.
```

When `nums[low] == nums[mid]`, the comparison carries no information. The only safe move is to shrink the range by one and try again:

```
def search_rotated_dupes(nums: list[int], target: int) → bool:
    """LeetCode 81. Worst case O(n) -- and that is unavoidable, not a flaw in this code."""
    low, high = 0, len(nums) - 1
    while low ≤ high:
        mid = (low + high) // 2
        if nums[mid] == target:
            return True
        if nums[low] == nums[mid] == nums[high]:      # no information: shrink both ends
            low += 1
            high -= 1
        elif nums[low] ≤ nums[mid]:
            if nums[low] ≤ target < nums[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:
            if nums[mid] < target ≤ nums[high]:
                low = mid + 1
            else:
                high = mid - 1
    return False
```

On `[1] * 100000 + [2]` searching for 2, that degrades to a linear scan. Say why in the interview: **with duplicates, no comparison can distinguish `[1,1,1,1,2]` from `[1,1,2,1,1]` without looking at more elements**, so `O(n)` worst case is a property of the problem, not of the solution.

6 What it costs

Time

```
each pass: 1 equality check, 1 sortedness check, 1 range check → O(1)
each pass discards at least half the remaining range           → log2(n) passes
                                                             → O(log n)
```

The comparisons per pass went from one to about three. The class did not move, and the number is still small:

```
n = 1,000,000      20 passes x ~3 comparisons = ~60 comparisons
n = 1,000,000,000 30 passes x ~3 comparisons = ~90 comparisons
```

Say the constant honestly — “three comparisons a pass instead of one, so about three times the work of plain binary search, and still logarithmic” — rather than pretending it is free.

Space

```
low, high, mid: three integers    → 0(1) extra space
```

With duplicates

```
best/average: 0(log n)
worst:        0(n)    -- [1] * 100000 + [2], searching for 2

why: when nums[low] = nums[mid] = nums[high], no half can be ruled out,
     so the only safe step removes one element from each end.
```

The alternative that is worth naming and rejecting

Find the pivot first with one binary search, then binary search the correct piece:

```
find_min:      log2(n) passes
then search:   log2(n) passes
               -----
               2 log2(n)  -- same class, twice the passes, two functions to get right
```

It is easier to reason about and slower by a constant, and some interviewers prefer it because the two halves are each plainly correct. Know both. Offer the one-pass version, and mention the two-pass version as the clearer alternative if they want to see the pivot found explicitly.

7 The traps

The near-miss: comparing `nums[mid]` against the target instead of a range

```
def broken(nums, target):
    low, high = 0, len(nums) - 1
    while low ≤ high:
        mid = (low + high) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:      # ← plain binary search logic on rotated data
            low = mid + 1
        else:
            high = mid - 1
    return -1

print(broken([9, 11, 1, 3, 5, 7], 9))    # -1    should be 0
```

-1

The array is not sorted, so “the middle is smaller than the target” says nothing about where the target is. `nums[mid] = 1`, which is less than 9, so it goes right — into the half that does not contain 9. No error, wrong answer. **A discard must be justified by a sorted range, never by a single comparison.**

The near-miss: `<` instead of `≤` in the sortedness test

```
if nums[low] < nums[mid]:      # ← should be ≤
```

On a two-element range where `low = mid`, `nums[low] < nums[mid]` is false, so the code decides the right half is sorted when in fact `mid = low` and the “right half” claim is meaningless:

```
print(broken_lt([3, 1], 1))    # -1    should be 1
print(broken_lt([1], 1))      # 0      this one happens to work
```

A single element is sorted. The `≤` says so. This is the single most common bug in this problem, and it only shows up on ranges of size one or two — which is exactly what the last two passes always are.

The real error: comparing against `nums[low]` in find-minimum

```
def broken_min(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[low]:           # ← against low, not high
            low = mid + 1
        else:
            high = mid
    return nums[low]

print(broken_min([1, 3, 5, 7, 9, 11]))    # 11    should be 1
```

11

On an array that is not rotated, every middle is bigger than `nums[low]`, so `low` marches all the way to the end and returns the maximum. Comparing against `nums[high]` is correct because the question “is `nums[mid]` at or below the last element?” is monotone in both the rotated and the un-rotated case. **Compare against `high` for the minimum. Always.**

The real error: the empty array

```
nums = []
low, high = 0, len(nums) - 1      # high = -1
mid = (low + high) // 2           # mid = -1
print(nums[mid])
```

```
Traceback (most recent call last):
  File "day45.py", line 4, in <module>
    print(nums[mid])
    ~~~~^~~~~~
IndexError: list index out of range
```

The `while low ≤ high` in the real solution never runs on an empty array, because `0 ≤ -1` is false — so the correct code handles it for free. The crash above is what happens if you hoist the midpoint out of the loop, or write a `do-while`-shaped version. Fourth appearance of an index that reaches `-1`; in Python it wraps rather than crashing, and here it happens to crash only because the list is empty.

The trap: assuming a rotation exists

Half the failing submissions on this problem contain a special case for “if the array is not rotated”. It is unnecessary — the un-rotated array takes the left-is-sorted branch on every pass and behaves exactly like plain binary search — and the special case is another place

to be wrong. Test the un-rotated input; do not branch on it.

8 In the interview

How it gets asked

- “You have a sorted array that’s been rotated at an unknown pivot. Find the target in $O(\log n)$.” — LeetCode 33, word for word, and one of the most-asked medium questions anywhere.
- “Find the minimum in a rotated sorted array.” — LeetCode 153, usually first, as the easier warm-up that establishes the break-point idea.
- “Now the array can contain duplicates.” — LeetCode 81, the follow-up that comes ninety seconds after you finish, and the correct answer includes the words “ $O(n)$ worst case”.
- “How many times was this array rotated?” — the same find-minimum search; the answer is the index of the minimum.

What to say out loud, in the first ninety seconds

1. **Say the structure before the code.** “A rotated sorted array has exactly one break point — one place where a value is followed by a smaller one. Everywhere else it climbs.”
2. **Say the fact that makes it work.** “The break sits in one half or the other, never both. So at any midpoint, at least one half is properly sorted, and I can find out which in one comparison: is `nums[low] ≤ nums[mid]`?”
3. **Say how you discard.** “Once I know which half is sorted, I check whether the target’s value falls inside that half’s range. If it does, I search there. If it doesn’t, it can’t be there, so I search the other one. Every discard is justified by a range, not by one comparison — that’s the part plain binary search gets to skip.”
4. **Ask the question that changes the answer.** “Can the array contain duplicates? If it can, the worst case becomes $O(n)$ and I’d write it differently.”
5. **State the cost.** “ $O(\log n)$ time, about three comparisons per pass instead of one, $O(1)$ space.”

The follow-ups

“Now there can be duplicates. What breaks?” The comparison that tells me which half is sorted stops carrying information. Concretely, on `[3, 1, 3, 3, 3]` with low at 0, mid at 2 and high at 4, all three values are 3, and there is no way to tell whether the break is left or right of the middle — the array `[3, 3, 3, 1, 3]` looks identical at those three positions and has the break on the other side. Since neither half can be ruled out, the only safe move is to shrink the range by one from each end and try again, which makes the worst case $O(n)$ — an array of a hundred thousand ones with a single two in it. I

want to be precise that this is a property of the problem and not a weakness in my code: no comparison-based method can distinguish those two arrays without looking at more elements, so $O(n)$ is a lower bound here. In the average case with few duplicates it is still logarithmic, and the one-line guard — `if nums[low] == nums[mid] == nums[high]: low += 1; high -= 1` — is the entire difference between the two solutions. LeetCode 81 also changes the return type to a boolean, because with duplicates “the index” is not well defined.

“Could you find the pivot first and then binary search normally? Which do you prefer?” Yes, and it is often the clearer answer, so I would offer both. Finding the pivot is its own binary search: the minimum is the first index whose value is at most the last element, which is a monotone question, so the boundary template handles it with no branching — compare `nums[mid]` against `nums[high]`, and if it is bigger the break is to the right, otherwise keep the middle. That gives me the index of the minimum, which is also the rotation count. Then I decide which piece the target lives in by comparing it to the array’s ends, and run a plain binary search on that piece — or, more neatly, I search the whole array with index arithmetic, mapping index `i` to `(i + pivot) % n`, which lets me use a completely unmodified binary search on a virtual sorted array. The cost is $2 \log n$ instead of $\log n$, so the same class with twice the passes. I’d use the one-pass version by default because it is a single loop, but I’d say that the two-pass version is easier to prove correct under time pressure, and if I were writing this for a codebase where someone else has to maintain it, that readability would probably win.

“Why is your discard safe? Convince me you’re not throwing away the answer.” Because I never discard on a single comparison — I discard on a range, and the range is a proof. Suppose the left half is sorted, so it runs from `nums[low]` up to `nums[mid]` with nothing out of order in between. If the target is smaller than `nums[low]` or bigger than or equal to `nums[mid]`, then it is outside the closed interval that half spans, and a sorted stretch contains exactly the values inside its own range — so the target is provably not in there, and discarding it loses nothing. If instead the target is inside that interval, the other half is the one I can discard, by the same argument applied to whichever values it holds. The invariant is unchanged from plain binary search: if the target is anywhere, its index is between low and high. What changed is only the evidence I use to shrink that range. And the reason at least one half is always sorted is that there is exactly one break point in the whole array, and one point cannot be in two halves at once.

A model answer

“First, the structure. A rotated sorted array has exactly one break point — one position where a value is followed by a smaller value. `[9, 11, 1, 3, 5, 7]` breaks between 11 and 1. Everywhere else it climbs.

That single break gives me the key fact: whatever midpoint I pick, the break is in one half or the other, never both. So at least one half is properly sorted, always. And I can tell which in one comparison — if `nums[low] ≤ nums[mid]`, the left half climbs steadily, so it’s the sorted one; otherwise the break is in the left, so the right one is sorted.

Once I know which half is sorted, I use its *range* to decide. A sorted stretch running from 240 to 330 contains exactly the values in that interval, so if my target isn’t in that interval it is provably not in that half, and I discard it. That’s the difference from plain binary search: the discard is justified by a range rather than by one comparison.

```
def search_rotated(nums: list[int], target: int) → int:
    low, high = 0, len(nums) - 1
    while low ≤ high:
        mid = (low + high) // 2
        if nums[mid] == target:
            return mid
        if nums[low] ≤ nums[mid]:           # left half sorted
            if nums[low] ≤ target < nums[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:                             # right half sorted
            if nums[mid] < target ≤ nums[high]:
                low = mid + 1
            else:
                high = mid - 1
    return -1
```

Two details I’d point at. The sortedness test is `≤`, not `<`, because when the range shrinks to one element low equals mid and a single element is sorted — that’s the bug people hit, and it only shows up on the last two passes. And `mid` is excluded from both range checks, since I’ve already compared it to the target and it didn’t match.

No special case for an un-rotated array: it takes the left-is-sorted branch every pass and behaves exactly like ordinary binary search.

Cost is $O(\log n)$ — about three comparisons per pass instead of one, so a constant factor of three — and $O(1)$ space. One clarifying question I'd want answered before committing: can the values repeat? If they can, `nums[low] = nums[mid] = nums[high]` carries no information, the only safe move is to shrink both ends by one, and the worst case becomes $O(n)$ — which is a property of the problem, not of the solution.”

9 Recall card

- **One break point, ever.** So at any midpoint the break is in one half only, which means **at least one half is properly sorted** — that single fact is the whole problem.
- `nums[low] ≤ nums[mid]` **picks the sorted half.** `≤`, not `<`: a one-element range has `low = mid` and is trivially sorted. This is the bug everyone writes.
- **Discard on a range, never on one comparison.** Target inside the sorted half's interval → go there; outside → it is provably elsewhere. `mid` is excluded from both intervals.
- **Find-minimum is the clean cousin:** compare `nums[mid]` against `nums[high]` (never `low`) — monotone, so [day 043](#)'s template, no branching. The min's index is also the rotation count.
- **Duplicates → $O(n)$ worst case, unavoidably.** When `nums[low] = nums[mid] = nums[high]`, shrink both ends by one; `[1]*100000 + [2]` is the killer, and no comparison-based method can do better.

1 What this is, and why they ask it

Encapsulation means an object keeps its own data to itself and lets the outside world act on it only through methods it chose to offer. The data is not hidden to be mysterious. It is hidden so that every change to it passes through code the object controls, which means the object can guarantee something is always true about itself — and no caller, anywhere, can break that guarantee.

They ask it with the getter question because that question is a trap, and it is asked on purpose. The weak answer is “so you can add validation later” or “it’s good practice”, both of which suggest the candidate has learnt a rule without the reason. The strong answer is that a getter is usually a failure too, and the real goal is that the object should be asked to *do* things rather than asked for its internals. Encapsulation is also the load-bearing idea under every remaining day of this phase: inheritance, polymorphism and composition are all ways of arranging encapsulated things, and none of them help if every object’s insides are public.

2 The story

Vinod’s medical shop is on the corner opposite the bus stop, and it opens at half past eight.

It is a small place. There is a glass counter at the front with the everyday things behind it — paracetamol, bandage rolls, a basket of throat lozenges — and behind that, four wooden shelves going up to the ceiling, packed tight, floor to top.

You cannot go behind the counter. Nobody can. There is a wooden flap at one end that lifts, and in nineteen years Vinod has lifted it for exactly two people: his brother, and the man who comes to service the cold cabinet.

Customers have asked. A schoolteacher who comes every month once told him, quite kindly, that it would be faster if she just went and got her own. And she was right, on that particular afternoon, in that particular way. Vinod said no anyway, and he could not really explain why at the time.

He can now, because he has watched what happens when the rule slips.

The shelves are not arranged in any way a customer could work out. They are arranged the way he remembers them, which is partly by kind, partly by how fast they move, and partly by where a box happens to fit. He can put his hand on any of six hundred things without looking. One person putting one strip back on the wrong shelf costs him twenty minutes, three days later, with a queue at the counter.

The cold cabinet is worse. Some things must stay cold, and they do not say so on the front of the box in a way that means anything to somebody who does not already know. He has had a boy hand him a box off the wrong shelf that had been sitting out since morning, and the only honest thing to do with it was throw it away.

And then there is the thing that matters most, which is that some medicines he will not hand over without seeing the doctor's slip. If people take their own, that check does not happen. It does not happen *sometimes*, and it does not happen *usually* — it simply stops existing, because the counter was the only place it ever happened.

So there is one way in and it is him. Tell him what you need. He will get it, check what needs checking, and put it in your hand.

3 The idea in plain English

Vinod's shelves are the object's data. The counter is its public interface. The prescription check is an **invariant** — a rule about the object that must always hold — and the reason the flap stays down is that an invariant enforced in one place is enforced, and an invariant enforced at every call site is enforced nowhere.

The invariant is the point

Start with the rule, not with the keyword. Take a bank account:

The balance is never negative.

That sentence is the invariant. Now compare two designs.

```
# Public data. The rule lives at every call site.
class Account:
    def __init__(self, balance: int) → None:
        self.balance = balance

# somewhere in the withdrawal screen
if account.balance ≥ amount:
    account.balance -= amount
```

```
# Encapsulated. The rule lives in one place, and there is no way round it.
class Account:
    def __init__(self, balance: int) → None:
        self._balance = balance

    def withdraw(self, amount: int) → None:
        if amount ≤ 0:
            raise ValueError("amount must be positive")
        if amount > self._balance:
            raise ValueError("insufficient funds")
        self._balance -= amount
```

The first version is not wrong today. It is wrong in eight months, when the third call site — a batch job, a refund path, an admin tool — writes `account.balance -= amount` without the check, and the balance goes negative in production. There is no way to prevent that, because the field is public and Python will happily assign to it.

The second version has one door. You cannot make a negative balance through it. That is the whole of encapsulation, and everything else is mechanics.

Why a getter is usually still a failure

Now the interviewer's question. Suppose you make the field private and add a getter:

```
class Account:
    def get_balance(self) → int:
        return self._balance
```

Have you gained anything? A little: reads are now channelled, so you could log them, or compute the value instead of storing it. But if the caller then writes

```
if account.get_balance() ≥ amount:
    account.set_balance(account.get_balance() - amount)
```

you have rebuilt the public field with extra typing. The rule is back at the call site. The getter plus setter pair is public data wearing a coat.

The real answer to the question is: **do not add the getter — add the behaviour**. The caller does not want the balance. The caller wants to withdraw money. `account.withdraw(amount)` puts the decision inside the object, where the rule lives.

That principle has a name worth knowing: **Tell, Don't Ask**. Tell the object to do something; do not ask it for its data so you can decide on its behalf. In an interview, saying "I'd rather expose `withdraw` than `balance`", because the rule about withdrawing belongs to the account" answers the getter question completely.

Getters are still legitimate when the value is genuinely something the outside world needs to *see* — a balance printed on a statement, an order's total on a receipt. The test is whether the caller is going to read the value and then make a decision the object should have made.

How Python actually does it

Python has no `private` keyword. It has conventions and one mechanism.

```
class Account:
    def __init__(self) → None:
        self.balance = 0          # public: part of the promise
        self._ledger = []         # single underscore: internal, do not touch
        self.__key = "abc"        # double underscore: name-mangled to _Account__key
```

- **One underscore** is a convention. It means “this is not part of the interface; if you use it and it changes, that is on you”. Nothing enforces it. Every linter and every reviewer respects it.
- **Two underscores** triggers **name mangling**: inside the class the attribute is rewritten as `_Account__key`, so a subclass defining its own `__key` does not collide. It is not a security feature — `obj._Account__key` still works — and it is meant for avoiding accidental clashes in inheritance, not for privacy.

Python's own phrase for this is that we are all consenting adults. The convention is the fence; the language does not build a wall. In Java or C++ `private` is enforced by the compiler, which is why the same design intent looks stronger there. **The intent is identical in both languages, and the intent is what the interview is about.**

Properties: changing your mind without changing callers

Sometimes you genuinely want an attribute-looking read, with control behind it:

```
class Account:
    @property
    def balance(self) → int:
        return self._balance          # callers write account.balance, and cannot assign
```

With no setter defined, `account.balance = 500` raises `AttributeError`. So reads look natural and writes are impossible, which is often exactly the shape you want. Properties also let you start with a stored field and later replace it with a computation — `total` as a stored number today, a sum over line items tomorrow — without a single caller changing.

The leak nobody notices: handing out your insides

This is the mistake that survives all the underscores:

```
class Order:
    def __init__(self) → None:
        self._items: list[str] = []

    def get_items(self) → list[str]:
        return self._items          # ← hands out the real list
```

```
order.get_items().append("free laptop")    # no method was called on Order at all
```

The list is a **reference**, not a copy. Returning it hands the caller the object's internals with full write access, and every rule `Order` might have about items — a maximum count, a price recalc — is bypassed silently. There was never a moment to check anything.

Three fixes, cheapest first:

```
def items(self) → tuple[str, ...]:
    return tuple(self._items)      # a copy, and immutable

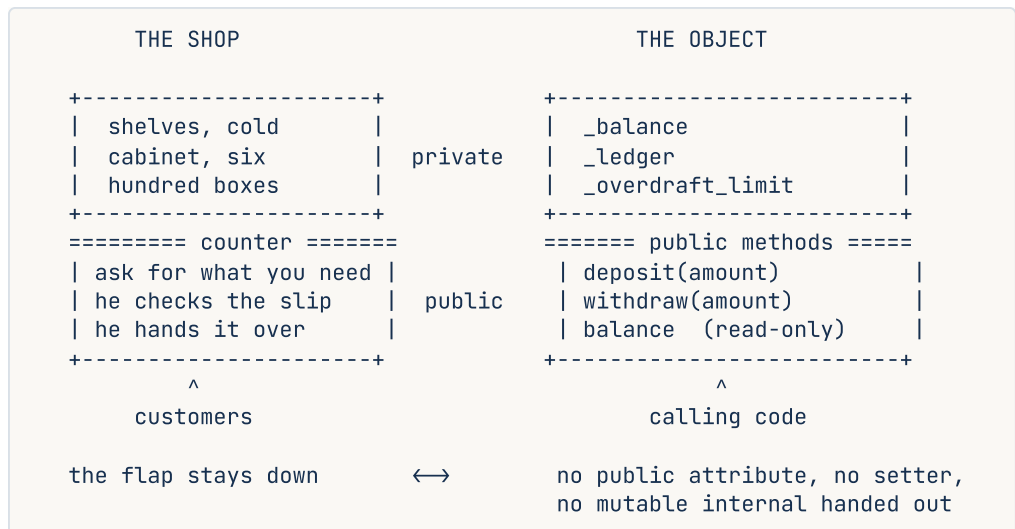
def items(self) → Iterator[str]:
    return iter(self._items)       # read-only walk, no copy

def add_item(self, item: str) → None:
    if len(self._items) ≥ 20:
        raise ValueError("cart is full")
    self._items.append(item)        # better still: no getter, just the behaviour
```

The same trap applies to dictionaries, sets, dates in mutable form, and any object you return that the caller can mutate. **Returning a mutable internal is the same as making the field public**, and it is worth saying that sentence in an interview because most candidates only think about the attribute.

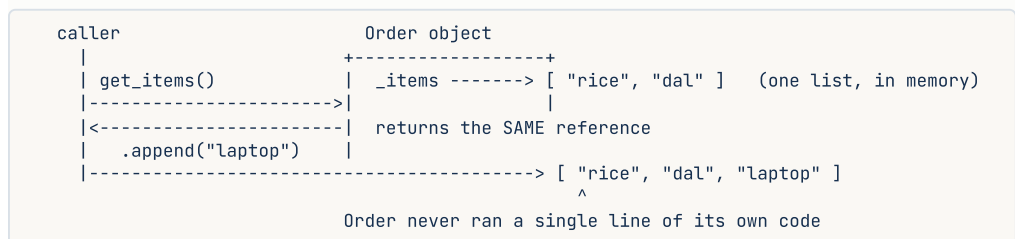
4 The picture

The shop, and the object, side by side:



What to notice: the counter is narrow on purpose. Every rule the shop has — the cold chain, the prescription, the arrangement of the shelves — is enforceable only because there is exactly one way in.

Where the leak happens, drawn:



What to notice: no method on `Order` was called during the mutation. Every check `Order` owns was skipped, and nothing anywhere reports an error. The underscore on `_items` did not help at all, because the reference escaped.

5 How it actually works

What the language gives you, and what it does not

Java / C++	private, protected, public enforced at compile time. Reflection can still break it, deliberately and visibly.
Python	no enforcement. <code>_name</code> is a convention; <code>__name</code> is mangled to <code>__Class__name</code> to avoid subclass collisions, not to hide. <code>@property</code> gives a read that looks like an attribute. <code>__slots__</code> prevents new attributes being added from outside.
JavaScript	<code>#field</code> is genuinely private since ES2022, enforced at runtime.

The design intent is identical everywhere. Only the enforcement differs, and interviewers are testing the intent, so “Python doesn’t have private” is never an answer — it is at most half a sentence before you say what you do instead.

The products you already use

- Python’s `datetime.date` exposes `.year`, `.month` and `.day` as read-only properties, and the object is immutable. You cannot construct 30 February through any door, because the only door is the constructor and it validates. That is encapsulation used to make an invalid object unrepresentable.
- `collections.Counter` and `dict` keep their hash table entirely internal. You can never corrupt the bucket array from outside, which is why the `O(1)` guarantee from [day 021](#) is a guarantee and not a hope.
- Java’s `ArrayList` keeps `size` and the backing array private. Making `size` public would let any caller set it to 100 on a list of three elements, and the next `get` reads uninitialised memory. The class’s entire correctness rests on nobody being able to do that.
- `java.lang.String` and Python’s `str` are immutable, which is encapsulation taken to its limit — you met the consequences on [day 019](#). An immutable object needs no defensive copies, is safe to share between threads, and can be safely hashed.
- Stripe’s SDK does not let you assign `intent.status = "succeeded"`. Status changes only through operations the server authorised, because a client that can write the field can lie about payment.

Designing the interface: the order to do it in

1. **Write the invariants first, as sentences.** “The balance is never negative.” “A spot holds at most one vehicle.” “The total equals the sum of the lines.” Two or three per class.
2. **For each invariant, name every operation that could break it.** Those operations become methods.

3. **Make everything else private.** Not “make everything private then add getters” — start closed, and open only what an operation needs.
4. **Check what you return.** Any mutable object handed out is a hole. Copy it, wrap it, or replace the getter with behaviour.
5. **Re-read the method list and ask: could a caller assemble a broken object out of these?** If yes, an operation is missing a check or an operation should not exist.

What “breaking encapsulation” looks like in a real codebase

```
if order.status == "PENDING" and order.items and order.customer.tier == "GOLD":  
    order.status = "PRIORITY"
```

Three objects’ internals read, one written, and the rule about what makes an order priority now lives in a screen. Find that same rule written slightly differently in two other files and you have the classic bug where an order is priority on one page and not on another. The fix is a method — `order.promote_if_eligible(customer)` — and the reason to say it out loud in an interview is that this is what the failure actually looks like in production. Not a security breach. A quiet disagreement between two files.

6 The numbers

The invariant, counted in call sites

```
"balance never negative", enforced at the call site:  
  withdrawal screen      1  
  ATM path               1  
  scheduled standing order 1  
  admin refund tool      1  
  the data-fix script    1 (the one nobody remembers)  
  ---  
                        5 copies of one rule  
  
after one year and two new features:      7-9 copies  
probability all of them are correct:      the product of each being right  
  
the same rule inside Account.withdraw():  1. Always 1, whatever ships next year.
```

What encapsulation costs at runtime

```
direct attribute read      obj.x          ~20-30 ns
property read             obj.x (@property) ~90-150 ns  -- roughly 4x
method call               obj.get_x()      ~60-80 ns

a request that does 1,000 property reads: ~0.1 ms
the same request's single database round trip: ~1 ms

so the entire encapsulation overhead is ~10% of ONE query.
```

The conclusion to say out loud: **the cost is real, it is nanoseconds, and it is dominated by the first input/output call in the request.** Optimise round trips, not property access.

Defensive copying, priced

```
returning tuple(self._items) on a 20-item list: ~400 ns, 20 pointer copies
returning iter(self._items): ~50 ns, no copy at all
returning self._items: ~0 ns, and a hole in the class

a cart page rendering 50 orders x 20 items:
  copies: 50 x 400 ns = 20 microseconds
```

Twenty microseconds against a page budget of 200 milliseconds. The copy is free in any sense that matters. The exception, and it is a real one: copying a list of a million elements on every access inside a loop is `O(n)` per call — there, return an iterator or expose the behaviour instead.

`__slots__`, from [day 044](#)

```
__slots__ = ("_balance", "_ledger")
- removes the per-instance dict: ~184 bytes → ~64 bytes
- AND blocks obj.blance = 100 (the typo) with an AttributeError
```

The second effect is the encapsulation one and it is underrated: without `__slots__`, a typo in an assignment silently creates a new attribute and the real one never changes.

7 The trade-offs

Hiding against convenience

Every private field is one thing callers cannot do without asking you to add a method. On a small script that is friction with no payoff. *I would not encapsulate a throwaway data-holder that crosses one boundary and dies* — a parsed row on its way into a list is fine as a dataclass with public fields, or as a `NamedTuple` where immutability does the job for free.

Immutability against churn

A frozen object cannot be corrupted, needs no defensive copies and is safe to hash and share. It also means every change allocates. *I would freeze value objects* — `Money`, `DateRange`, `Coordinate` — *and leave entities mutable*, exactly as [day 044](#) put it. Freezing an `Order` that changes state ten times in its life turns every transition into a new object and a new identity, which is worse, not safer.

Encapsulation against frameworks that want your fields

ORMs, serialisers and dataclass-to-JSON converters all want to read every attribute, and most reach past the underscore to do it. *I would not fight the framework at the boundary* — I would keep the rich, encapsulated object in the domain layer and convert to a plain dictionary or a schema object at the edge, so that the framework sees a flat structure and the rules stay in one place. That conversion layer is annoying and it is the price of having rules at all.

Getters against behaviour

I would not add a getter for a value the caller is only going to use to make a decision about my object. That decision belongs to me. I would add a getter for a value the caller genuinely needs to display, report, or hand to something else. The test is the next line of the caller's code: if it is an `if`, the getter is wrong and a method is missing.

The honest sentence

Encapsulation is not about stopping malicious code. It is about making sure that in two years, when someone adds the seventh way to withdraw money, the rule about not going negative is somewhere they cannot avoid.

8 In the interview

How it gets asked

- “*Why make a field private if you’re going to add a getter anyway?*” — the trap, and the answer is that the getter is often also a mistake.
- “*What’s wrong with this class?*” — shown a class with public fields, and expected to name the invariant that nothing protects.
- “*Where do you put the validation?*” — inside a larger design round. The answer is “on the object that owns the data being validated”, and then you point at the method.

- “Python doesn’t have private. So does encapsulation matter here?” — a genuine question that sounds like a gotcha, and the answer is about intent and conventions, plus `@property` and the returned-reference trap.

What to say out loud, in the first ninety seconds

1. **Lead with the invariant, not the keyword.** “The rule here is that the balance is never negative. Encapsulation is how I make that rule impossible to break rather than merely documented.”
2. **Name the failure concretely.** “With a public field, the rule has to be repeated at every call site. Right now there are three; in a year there are seven, and the one somebody forgets is the one that ships.”
3. **Answer the getter trap directly.** “A getter alone doesn’t fix it — `get` then `set` is a public field with extra typing. What I actually want is `withdraw(amount)`, so the decision happens inside the object. Tell, don’t ask.”
4. **Mention the leak.** “And I’d check what the class returns. Handing back the internal list means a caller can append to it without any of my code running — that’s a public field by another route.”
5. **Say what Python gives you.** “Python won’t enforce it: underscore is a convention, double underscore is name mangling for inheritance, and `@property` gives a read-only attribute-shaped access. The intent is the same as `private` in Java; only the enforcement differs.”

The follow-ups

“Give me a concrete bug that a public field causes and a private one prevents.” A cart with a maximum of twenty items. With `cart.items` public, the checkout page appends through the object’s own `add_item` and the limit is respected — but the bulk-reorder feature, written a year later by someone else, does `cart.items.extend(previous_order.items)` because it is one line and it works. Now a cart holds sixty items, the packing system that assumed twenty overflows its label layout, and the failure surfaces three systems downstream with nothing in the logs pointing back at the cart. There is no exception at the moment the rule breaks, which is what makes it expensive: the bug and its symptom are in different services. With the list private and only `add_item` exposed, the bulk path has to call the same method twenty times and gets a clear `ValueError` on the twenty-first, at the right place, in the right stack trace. What I want to draw out is that the value of encapsulation is not that it prevents the write — it is that it moves the error to where somebody can understand it.

“You made the list private, but you still return it from `get_items`. Is that safe?” No, and this is the mistake that survives the underscore. Returning `self._items` hands the caller the same list object my class is holding, so `order.get_items().append(...)` mutates my state directly, without a single line of my code running. Underscore is a nam-

ing convention, not a barrier around the object it names. There are three fixes and they trade off differently. Returning `tuple(self._items)` gives an immutable snapshot and costs a copy — about four hundred nanoseconds on twenty items, which is nothing, but is $O(n)$ if the collection is large and the call is in a loop. Returning `iter(self._items)` gives a read-only walk with no copy, and is the right choice for large collections, though the caller can still be surprised if the underlying list changes mid-iteration. And the best option is usually to delete the getter — if the caller only wants a count, expose `item_count`; if they want to add, expose `add_item`. The general rule I'd state: any mutable object you return is part of your public interface, whether you meant it to be or not. That applies to lists, dicts, sets, and any of your own entities you hand out.

“Python has no `private`. Doesn't that make all of this decoration?” No, and there are two separate reasons. The first is that enforcement was never the point. Encapsulation is a statement about where a rule lives, and a rule that lives in one method is enforced by the fact that all the sensible paths go through that method — the compiler was only ever a reminder. In practice, a leading underscore is respected by every reviewer, every linter, and every IDE's autocomplete, which is most of the value. The second reason is that Python does give you real tools, they are just different ones. `@property` with no setter makes assignment raise `AttributeError` at runtime, which is genuine enforcement. `@dataclass(frozen=True)` makes an object immutable, so the invariant holds by construction and no defensive copy is ever needed. `__slots__` blocks new attributes being created from outside, which catches the typo case. And double-underscore name mangling exists for a narrower purpose that is worth being accurate about — it prevents a subclass accidentally colliding with a base class's attribute, which matters from [tomorrow](#) when inheritance arrives. What I would never say in an interview is “Python doesn't have private, so I use public fields” — the design intent is language-independent, and that is what is being scored.

A model answer

“Let me answer that with the rule rather than the keyword, because the keyword is the part that doesn’t matter.

Say the rule is: an account balance is never negative. If `balance` is public, that rule has to be written at every place that takes money out. Today that’s the withdrawal screen. In a year it’s the withdrawal screen, the standing-order job, the refund tool and a data-fix script — and the one somebody forgets is the one that puts a negative balance in production, with no error at the moment it happens.

So I make the field private and expose `withdraw(amount)`, which checks and then subtracts. Now there is exactly one door and the rule is behind it, and there is no way to write a negative balance through the interface at all.

On your actual question — the getter doesn’t buy me that. If I expose `get_balance` and `set_balance`, the caller writes `if get_balance() ≥ amount: set_balance(get_balance() - amount)` and the rule is back at the call site. That’s a public field with more typing. What I want is not a getter at all: I want the behaviour. Tell the object to withdraw; don’t ask it for its balance and decide on its behalf.

I’d still expose a read-only `balance` if something genuinely needs to display it — a statement, a receipt. In Python that’s `@property` with no setter, so `account.balance` reads naturally and `account.balance = 500` raises `AttributeError`. The test I apply is: what’s the caller’s next line? If it’s an `if`, the getter is wrong and a method is missing.

The other thing I’d check is what the class hands out. If `get_items` returns the internal list, a caller can append to it and none of my code runs — the underscore didn’t protect the object, only the name. So I’d return a tuple, or an iterator for a large collection, or better, expose `add_item` and delete the getter.

And the cost of all this is nanoseconds. A property read is maybe four times a direct attribute read, which is about a hundred nanoseconds, against a millisecond for one database round trip. It’s not a performance conversation.”

9 Recall card

- **Start from the invariant, not the keyword:** “balance never negative”, “a spot holds one vehicle”. Encapsulation makes the rule *impossible to break* instead of merely documented.

- **The failure is call-site multiplication.** One rule at 5-9 call sites, and the forgotten one ships. Inside one method it is 1 place, forever — and the error lands where somebody can read it.
- **The getter answer: don't add the getter, add the behaviour.** `get` + `set` is a public field with more typing. Tell, Don't Ask. A getter is fine only when the caller *displays* the value — if their next line is an `if`, a method is missing.
- **Returning a mutable internal is a public field by another route.** `get_items().append(...)` runs none of your code. Return a tuple, an iterator, or nothing at all.
- **Python:** `_` is convention, `__` is name mangling for subclass collisions, `@property` without a setter is real enforcement, `frozen=True` and `__slots__` help. The intent is identical to Java's `private`; only the enforcement differs, and intent is what is scored.

DSA topic: Search in a rotated sorted array **System design topic:** Encapsulation

Code these, in this order

Do the minimum before the search. Finding the break point is the easier half of the idea, and it makes the harder half obvious.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Find Minimum in Rotated Sorted Array	LeetCode 153 (Medium)	The break point, and comparing against <code>nums[high]</code> rather than <code>nums[low]</code> .
2	Search in Rotated Sorted Array	LeetCode 33 (Medium)	Discarding on a <i>range</i> instead of a single comparison.
3	Search in Rotated Sorted Array II	LeetCode 81 (Medium)	Whether you can say “O(n) worst case” and explain why it is unavoidable.
4	Find Minimum in Rotated Sorted Array II	LeetCode 154 (Hard)	The same duplicate problem on the cleaner cousin — and the one-line guard.

On problem 1, break your own solution first

Write the version that compares `nums[mid]` against `nums[low]` and run it on `[1, 2, 3, 4, 5]`. It returns 5. Say why in one sentence before you fix it: on an un-rotated array every middle is bigger than the first element, so `low` marches to the end. Then write the `nums[high]` version and confirm the same input gives 1.

On problem 2, say the sentence before the code

Out loud: *at least one half is properly sorted, because there is exactly one break point and one point cannot be in two halves*. Then write it. Then run the three inputs that decide correctness — an un-rotated array, a break at the very last position, and a single-element array. If you needed a special case for any of them, delete it and find the real bug.

On problem 3, do not patch — understand

Before writing the duplicate version, hold `[3, 1, 3, 3, 3]` and `[3, 3, 3, 1, 3]` side by side and check what `nums[0]`, `nums[2]` and `nums[4]` say in each. They are identical. That is the proof that no comparison can decide, and it is what you say in the interview rather than “duplicates make it slower”. Then add the one-line guard and time it on `[1] * 100000 + [2]`.

On problem 4, notice what the guard becomes

Here the guard is `high -= 1` alone, not `low += 1; high -= 1`. Work out why: when `nums[mid] == nums[high]`, you cannot rule out the middle, but you can safely discard *one* copy of the value at the right end, because if it were the unique minimum the middle would equal it anyway. This is the hardest reasoning in the phase; if it does not land in ten minutes, write the $O(n)$ scan, say it is $O(n)$, and come back to it.

The which-half drill

For each array and midpoint, say which half is sorted and how you know, in under five seconds each:

1. `[4, 5, 6, 7, 0, 1, 2]`, `mid = 3`.
2. `[6, 7, 0, 1, 2, 4, 5]`, `mid = 3`.
3. `[1, 2, 3, 4, 5, 6, 7]`, `mid = 3`.
4. `[2, 1]`, `mid = 0`.
5. `[5]`, `mid = 0`.

Numbers 3, 4 and 5 are the ones that catch people. Say which comparison operator each depends on.

The discard-proof drill

For each, say whether the discard is justified and give the reason as a range statement:

1. Left half sorted, runs 240 to 330, target 88 — discard which half?
2. Right half sorted, runs 1 to 239, target 300 — discard which half?
3. Left half sorted, runs 4 to 7, target 7 — careful, where is `mid`?
4. Right half sorted, runs 0 to 2, target 0 — careful, where is `mid`?

Cases 3 and 4 are about whether `mid` is inside or outside your interval. Say which boundary is inclusive on each side and why.

The encapsulation audit

Take this class and list every way a caller can break it. There are at least four:

```
class Cart:
    MAX_ITEMS = 20

    def __init__(self):
        self.items = []
        self.total = 0

    def add(self, item, price):
        if len(self.items) < self.MAX_ITEMS:
            self.items.append(item)
            self.total += price
```

1. Name the invariants the class is trying to hold — there are two.
2. Name four distinct ways a caller breaks one of them without any exception being raised.
3. Rewrite it so none of the four is possible, and say which change fixes which.
4. Then answer: should `total` be stored at all, or computed? Give the trade-off in one sentence.

The getter drill

For each, say whether a getter is right, or whether a behaviour method is missing — and if the latter, name the method:

1. `order.get_total()` , used to print a receipt.
2. `order.get_status()` , used as `if order.get_status() == "PENDING": order.set_status("PAID")` .
3. `account.get_balance()` , used to display on a statement.
4. `account.get_balance()` , used as `if account.get_balance() ≥ amount: ...` .
5. `cart.get_items()` , used to render the list in the checkout view.
6. `cart.get_items()` , used as `cart.get_items().append(x)` .

Say the one-line test that decides all six.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Search a rotated sorted array in $O(\log n)$.* The one-break-point structure, the “at least one half is sorted” fact, the range-based discard, the `≤` in the sortedness test, and the duplicates clarifying question.

2. *Why make a field private if you are going to add a getter anyway?* The invariant, the call-site multiplication with a number, and then the real answer — that the getter is often also wrong and the behaviour is what should be exposed.
3. *Your rotated search returns -1 for a value that is in the array. Where is the bug?* Two candidates: a plain `nums[mid] < target` comparison instead of a range check, or `<` instead of `≤` in the sortedness test. Say which input exposes each.
-

Before you move on

- [] I wrote find-minimum comparing against `nums[high]` and can say what breaks with `nums[low]`.
- [] I solved LeetCode 33 with no special case for the un-rotated array.
- [] I can hold up `[3, 1, 3, 3, 3]` and `[3, 3, 3, 1, 3]` and explain why duplicates force $O(n)$.
- [] I answered all five which-half drills, including the one-element case.
- [] I found at least four ways to break the `Cart` class and fixed each one.
- [] I can state the one-line test that decides whether a getter or a method is right.
- [] I answered all three questions above out loud.

Binary search on the answer

DATA STRUCTURES AND ALGORITHMS

Binary search on the answer

You can search a range of possible answers when the array itself is not what is sorted.

SYSTEM DESIGN

Inheritance and its costs

You can use inheritance where it fits and name the three places it goes wrong.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the smallest capacity that lets you ship all packages within d days.*
- *When does inheritance become a problem?*

1 What this is, and why they ask it

Until today, binary search has run over an array. Today it runs over the **answers**. You are asked for the smallest value of something — a capacity, a speed, a size, a number of days — that makes a task possible. You do not search the input. You take the range of every value the answer could possibly be, ask “*does this value work?*” about the middle of that range, and throw away half the candidates.

This is the most valuable idea in the phase and the one that most changes what you can solve. The problems do not look like search problems at all — they read as optimisation, and candidates reach for greedy strategies or dynamic programming and get lost. The tell is a specific sentence shape: *the minimum X such that Y is possible, or the maximum X such that Y still holds*. Once you see that shape, the recipe is fixed and short. LeetCode 875, 1011, 410 and 1482 are all the same problem in different costumes, and at least one of them shows up in interviews at every large product company.

2 The story

Sundar has an onion load to shift from the mandi at Yeshwanthpur to a shop in Malleshwaram, and the shop pulls its shutters down at four.

The load is standing in the yard as twelve stacks, one behind the other, and the man who stacked them that morning did it in the order the lorry unloaded. Sundar cannot pull from the middle — the front stack comes first, then the one behind it, and so on down the line. The stacks are not the same size: some are eight sacks, one is nearly thirty.

It is half past twelve. Each round trip to Malleshwaram and back takes about fifty minutes in that traffic. So he has four trips in him, at the outside. Four.

The question is which tempo to hire. There is a row of them at the stand and they take different loads — the small ones about six hundred kilos, the biggest about two thousand. The bigger the tempo, the more it costs him for the afternoon, and he is not a man who pays for capacity he does not use.

So he stands there and works it out, and the way he works it out is the whole point.

He does not walk the row of tempos asking each driver. He picks a size in the middle — say twelve hundred — and then he does one thing: he walks down his twelve stacks in order and counts. First stack, second, third — that is about eleven hundred, adding the

fourth would go over, so that is trip one. Then he starts again from the fourth. He gets to the end of the row and finds he has used five trips. Five is too many. So twelve hundred is too small, and — this is the part he is sure of without checking — so is every tempo smaller than twelve hundred. All of those are gone.

Now the middle of what is left, between twelve hundred and two thousand. Sixteen hundred. He walks the stacks again, counting. Four trips. It works. So sixteen hundred is enough, and so is anything bigger — but he does not want anything bigger, he wants the smallest that works, so the big ones are gone too.

Between twelve hundred and sixteen hundred, then. Fourteen hundred. Walk, count: four trips. Works. Keep looking below it.

Four or five walks down the row, and he knows the exact smallest size that gets the onions there by four. He hires that one.

The walking is the expensive part — the row is long and he is counting in his head. That is exactly why he does not test every tempo at the stand.

3 The idea in plain English

Sundar's twelve stacks are the array, and it is **not sorted and never will be**. What is sorted — in the only sense binary search needs — is the list of tempo sizes, and the yes-or-no answer attached to each one.

The shape to recognise

Every problem in this family reads the same way:

Find the smallest X such that some task is possible.

Find the largest X such that some condition still holds.

“The smallest tempo that finishes in four trips.” “The slowest eating speed that finishes the bananas in h hours.” “The smallest ship capacity that clears the packages in d days.” “The largest minimum distance between cows.” Same problem, four costumes.

The three things you write down

One: the range of possible answers. Not the array — the answers. For Sundar:

smallest possible tempo	= the biggest single stack	(anything less cannot lift it at all)
largest tempo worth it	= all twelve stacks at once	(one trip; nothing bigger helps)

In code, `lo = max(weights)` and `hi = sum(weights)`. Getting these two wrong is the commonest bug in the whole family, and §7 shows both directions of it.

Two: the yes-or-no question, written as a function that takes a candidate answer and returns `True` or `False`.

```
def works(capacity: int) → bool:
    """Can the stacks be shifted in at most 4 trips with this capacity?"""
```

This is Sundar's walk down the row. It is a **greedy simulation**: load as much as fits, then start a new trip. It costs one full pass over the input.

Three: check that the question is monotone. Say it out loud, in words: *if a tempo of 1400 works, does 1500 also work?* Obviously yes — more room can never mean more trips. So the answers look like:

capacity:	700	900	1100	1300	1400	1500	1700	2000
works?	no	no	no	no	YES	YES	YES	YES

^
the boundary: the smallest capacity that works

That row is monotone, so it is exactly [day 043](#)'s boundary template — and the answer is the first `True`.

Monotonicity is the precondition, and it replaces sortedness. If a bigger capacity could somehow be worse, none of this works. Interviewers ask you to justify it, and the justification is one sentence about the real world, never a proof.

The feasibility check, in detail

For the shipping version — the exact wording in the hub — the check is:

```
def days_needed(weights: list[int], capacity: int) → int:
    days, load = 1, 0
    for w in weights:
        if load + w > capacity:      # this one does not fit; start a new day
            days += 1
            load = 0
        load += w
    return days
```

Start on day one with an empty ship. Walk the packages in order. If the next one does not fit, that day ends and a new one starts. Note `days` starts at 1, not 0 — the first day exists before anything is loaded, and starting at 0 gives an answer one too small on every input.

Then the question is simply:

```
def works(capacity: int) → bool:
    return days_needed(weights, capacity) ≤ d
```

Putting it together

```
lo, hi = max(weights), sum(weights)
while lo < hi:
    mid = (lo + hi) // 2
    if works(mid):
        hi = mid          # mid might be the smallest that works: keep it
    else:
        lo = mid + 1      # mid is too small, and so is everything below it
return lo
```

Six lines, and they are the same six lines as [day 043](#). Only the question changed — from “is `nums[i]` at least the target?” to “can it be done with capacity `mid`?”. That is the entire lesson, and it is why yesterday’s template was worth learning as a template rather than as a function.

Why no `-1` case exists

`hi = sum(weights)` always works — one day, everything on board — so at least one candidate in the range is `True`. There is always an answer, so unlike an array search there is no “not found”. If you find yourself writing a not-found branch here, your `hi` is too small.

For a maximum instead of a minimum

Some problems ask for the *largest* `X` that still works, and the monotone row runs the other way: `YES, YES, YES, no, no`. Two options, and the second is better:

- Mirror the template: on `True`, `lo = mid`; on `False`, `hi = mid - 1`; and use `mid = (lo + hi + 1) // 2` to round *up*, or it hangs when `lo` and `hi` are adjacent.
- Or flip the question — search for the first `False` and subtract one — and keep the template untouched.

Prefer the second. One template that you never modify is worth more than two you have to remember the difference between. If you do write the mirrored version, the ceiling midpoint is not optional, and §7 shows why.

4 The picture

The two things being searched, side by side:


```

THE INPUT -- not sorted, never searched:
stack      0      1      2      3      4      5      6      7      8      9     10     11
weight    [ 8 , 30 , 12 , 5 , 22 , 17 , 9 , 14 , 26 , 3 , 19 , 11 ]

THE ANSWER RANGE -- this is what binary search runs over:
lo = max(weights) = 30                hi = sum(weights) = 176

capacity   30 ..... 60 ..... 100 ..... 140 ..... 176
works(c)?  no          no          YES          YES          YES
                                ^
                                the boundary. Everything left is no,
                                everything right is yes.

```

What to notice: the array is never sorted and never needs to be. The monotone thing is the row of answers to `works(c)`, and that row exists whether or not anyone writes it down.

One evaluation of `works(60)`, drawn as the greedy walk:

```

capacity 60, weights [8, 30, 12, 5, 22, 17, 9, 14, 26, 3, 19, 11]

day 1:  8 + 30 + 12 + 5   = 55      next is 22 → 77 > 60, so day 1 ends
day 2:  22 + 17 + 9      = 48      next is 14 → 62 > 60, so day 2 ends
day 3:  14 + 26 + 3      = 43      next is 19 → 62 > 60, so day 3 ends
day 4:  19 + 11          = 30      the row is finished
                                      → 4 days

if d = 3, works(60) is False. If d = 5, works(60) is True.

```

What to notice: one call to `works` costs a full pass over the array. That is why the total cost is $O(n \times \log(\text{range}))$ and not $O(\log(\text{range}))$ — the comparison is not free any more, and saying so is worth marks.

The whole method as a flow:

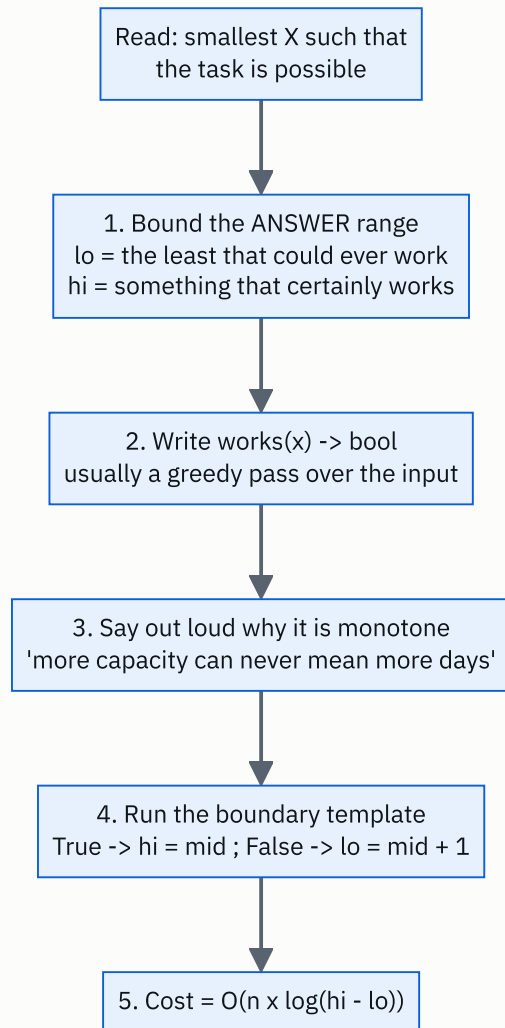


FIGURE 46.1

What to notice: step three is the one candidates skip, and it is the one that is wrong most often. If the question is not monotone, everything below it is nonsense.

5 The code, built step by step

The feasibility check, first

```
def days_needed(weights: list[int], capacity: int) → int:
    days, load = 1, 0
    for w in weights:
        if load + w > capacity:
            days, load = days + 1, 0
        load += w
    return days
```

Write this before the search, always. It is the part with the real logic in it, and it is the part you can test on its own: `days_needed([1,2,3,4,5], 5)` should be 4, and `days_needed([1,2,3,4,5], 15)` should be 1.

The bounds

```
lo, hi = max(weights), sum(weights)
```

`max` because a package heavier than the ship can never be loaded at all, so no smaller capacity is even meaningful. `sum` because everything in one day certainly works, so the answer cannot be larger. Both are $O(n)$ and both are computed once, outside the loop.

The search

```
while lo < hi:
    mid = (lo + hi) // 2
    if days_needed(weights, mid) ≤ d:
        hi = mid
    else:
        lo = mid + 1
return lo
```

Unchanged template. `days_needed(weights, mid) ≤ d` is the question; everything else is [day 043](#).

The complete solution

```
def ship_within_days(weights: list[int], d: int) → int:
    """LeetCode 1011. Smallest capacity that ships all packages, in order, within d days.

    Searches the ANSWER range [max(weights), sum(weights)], not the array.
    works(c) is monotone: more capacity can never require more days.
    """

    def days_needed(capacity: int) → int:
        days, load = 1, 0
        for w in weights:
            if load + w > capacity:
                days, load = days + 1, 0
            load += w
        return days

    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(mid) ≤ d:
            hi = mid
        else:
            lo = mid + 1
    return lo

def min_eating_speed(piles: list[int], h: int) → int:
    """LeetCode 875. Koko eats one pile at a time at k bananas/hour; smallest k finishing in h.

    Same recipe. Note the ceiling division: a part-eaten pile still costs a whole hour.
    """
    def hours_needed(k: int) → int:
        return sum((pile + k - 1) // k for pile in piles)

    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(mid) ≤ h:
            hi = mid
        else:
            lo = mid + 1
    return lo

def split_array_max_sum(nums: list[int], k: int) → int:
    """LeetCode 410. Split nums into k contiguous parts, minimising the largest part sum.

    Identical to shipping: 'capacity' is the largest allowed part sum, 'days' is the part count.
    """
    def parts_needed(limit: int) → int:
        parts, total = 1, 0
        for x in nums:
            if total + x > limit:
                parts, total = parts + 1, 0
            total += x
        return parts

    lo, hi = max(nums), sum(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if parts_needed(mid) ≤ k:
            hi = mid
        else:
            lo = mid + 1
    return lo

if __name__ == "__main__":
    print(ship_within_days([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5))
    print(ship_within_days([3, 2, 2, 4, 1, 4], 3))
    print(ship_within_days([1, 2, 3, 1, 1], 4))
    print(ship_within_days([500], 1))
    print(ship_within_days([1, 1, 1, 1], 4))
    print(min_eating_speed([3, 6, 7, 11], 8))
```

15
6
5
500 ← one package
1 ← one per day
4

```

print(min_eating_speed([30, 11, 23, 4, 20], 5))      # 30 ← h = len(piles)
print(min_eating_speed([30, 11, 23, 4, 20], 6))      # 23

print(split_array_max_sum([7, 2, 5, 10, 8], 2))     # 18
print(split_array_max_sum([1, 2, 3, 4, 5], 2))      # 9

```

Three problems, one recipe, and the search loop is character-for-character identical in all three. Say that out loud in an interview — it is the sentence that shows you have the pattern, not three memorised solutions.

The maximisation form, if you must write it

```

def largest_that_works(lo: int, hi: int, works) → int:
    """Largest x in [lo, hi] with works(x) True, assuming works(lo) is True."""
    while lo < hi:
        mid = (lo + hi + 1) // 2      # ceiling: rounds UP, or this hangs
        if works(mid):
            lo = mid                  # mid works; it might be the largest
        else:
            hi = mid - 1
    return lo

```

The `+ 1` in the midpoint is mandatory. Without it, when `hi = lo + 1` the midpoint is `lo`, and `lo = mid` sets `lo` to itself — an infinite loop, silently. This is the one place in the phase where a ceiling midpoint is required, and it is the reason the recommended route is to search for the first `False` instead and subtract one.

6 What it costs

Time

Two things multiply, and both have to be named:

```

number of binary search passes = log2(hi - lo)
cost of one feasibility check  = O(n)           -- one pass over the array
total                          = O(n x log(hi - lo))

```

That is a different shape from every earlier day, where a comparison was free. Put numbers on it:

```

shipping, n = 50,000 packages, weights up to 500:
    lo = 500, hi = 50,000 x 500 = 25,000,000
    log2(25,000,000 - 500) ~ 25 passes
    25 passes x 50,000 = 1,250,000 operations

by comparison, trying every capacity from 500 upwards:
    up to 25,000,000 candidates x 50,000 = 1.25 x 1012 operations

```

A million operations against a trillion. And note where the logarithm sits: it is over the **range of answers**, not over the array. Doubling the array doubles the work; doubling the range of answers adds one pass.

Space

```
lo, hi, mid, and the check's own two counters → 0(1) extra space
```

The feasibility check must be careful here. If your `works` builds a list of the days, that is $O(n)$ space *per call*; the counting version above allocates nothing.

Why the bounds matter for cost as well as correctness

```
lo = 1,      hi = 10^18 (a lazy upper bound) → 60 passes
lo = max,    hi = sum   (the tight bounds)   → 25 passes
```

Both are correct. The tight bounds are less than half the work, and — more importantly — the tight bounds are what show the interviewer you thought about the problem rather than pasting a template.

The number to have ready

One feasibility check is a full pass, $O(n)$. The search does $\log_2$ of the answer range of them. So $O(n \log(\text{range}))$ — fifty thousand packages over a range of twenty-five million is about twenty-five passes, a bit over a million operations, against a trillion if you tried every capacity.

7 The traps

The near-miss: bounds that are too tight at the bottom

```
lo, hi = 1, sum(weights)          # ← 1, not max(weights)
```

This is not wrong — it is merely wasteful — because `works(c)` is `False` for every `c` below `max(weights)` and the search discards them anyway. The genuinely broken version is the other direction:

```
lo, hi = max(weights), max(weights) * len(weights) // 2    # ← hi too small
print(ship_within_days([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 1))
```

The correct answer is 55 — one day means the whole load in one go. Every candidate in the range answered False, so `lo` marched to the top of a range that does not contain the answer, and the function returned it with total confidence. If `works(hi)` is False, the answer is outside your range and the return value is meaningless. A cheap habit: assert `works(hi)` before the loop while developing.

The near-miss: `days` starting at zero

```
def days_needed(weights, capacity):
    days, load = 0, 0          # ← should be 1
    for w in weights:
        if load + w > capacity:
            days, load = days + 1, 0
        load += w
    return days

print(days_needed([1, 2, 3, 4, 5], 5))    # 3    should be 4
```

3

The first day exists before any package is loaded, so the counter starts at one and increments only when a *new* day begins. Starting at zero undercounts by exactly one on every input, which makes `works` too optimistic, which makes the final answer too small — and it is too small by an amount that depends on the input, so it passes some tests.

The real error: division by zero in the eating-speed check

```
piles = [3, 6, 7, 11]
lo = 0                                     # ← speed 0 is not a sensible candidate
print(sum((p + lo - 1) // lo for p in piles))
```

```
Traceback (most recent call last):
  File "day46.py", line 3, in <module>
    print(sum((p + lo - 1) // lo for p in piles))
              ~~~~~^~~~~
ZeroDivisionError: integer division or modulo by zero
```

`lo` must be 1 for a rate, not 0. The general rule: **the lower bound is the smallest value that is meaningful, not the smallest number**. A capacity of zero carries nothing; a speed of zero never finishes; a group size of zero is not a group.

The near-miss: a question that is not monotone

```
# "Find the smallest k such that exactly k groups can be formed."
```

“Exactly” is the warning word. If `k = 5` works and `k = 6` does not and `k = 7` does, the row of answers is not monotone and binary search will land somewhere arbitrary — with no error. Before writing a line, say the sentence: *if x works, does every value above x also work?* If you cannot answer yes with a reason from the problem itself, this is not the tool.

The same warning applies to costs that are not smooth. “The smallest number of workers such that the total wage bill is under ₹X” is not monotone if adding a worker can reduce overtime and lower the bill.

The real error: the maximisation form without the ceiling midpoint

```
lo, hi = 1, 10
while lo < hi:
    mid = (lo + hi) // 2          # ← floor, in a version that does lo = mid
    if mid ≤ 7:
        lo = mid
    else:
        hi = mid - 1
```

No exception, no output, no end. When `lo = 7` and `hi = 8`, `mid` is 7, the condition holds, `lo = 7` — unchanged — and the loop runs forever. The fix is `mid = (lo + hi + 1) // 2`. The better fix is not to write this form at all: search for the first `False` with the standard template and subtract one.

8 In the interview

How it gets asked

- “Find the least weight capacity of a ship that can carry all packages within d days, in order.” — LeetCode 1011, the exact hub phrasing.
- “Koko eats bananas at k per hour, one pile at a time. What’s the minimum k to finish in h hours?” — LeetCode 875, the friendliest introduction to the pattern.
- “Split the array into k contiguous subarrays, minimising the largest subarray sum.” — LeetCode 410, which reads like dynamic programming and is not.
- “Place c cows in n stalls so the minimum distance between any two is as large as possible.” — the maximisation form, and a classic.

What to say out loud, in the first ninety seconds

1. **Name the shape before anything.** “This is ‘the smallest X such that something is possible’, so I’m going to binary search on the answer rather than on the array. The array itself is never sorted and never needs to be.”

2. **Bound the answer range, with reasons.** *“The capacity can’t be less than the heaviest single package, because that one would never load. And it never needs to exceed the sum of everything, because that’s one day. So the range is max to sum.”*
3. **Write the feasibility check first, out loud.** *“ `days_needed(capacity)` walks the packages in order, adding to the current day until the next one doesn’t fit, then starting a new day. It starts the counter at one, because day one exists before anything is loaded.”*
4. **Justify monotonicity in one sentence.** *“More capacity can never require more days — every packing that fits in a smaller ship also fits in a bigger one. So the yes-answers form one unbroken run at the top, and I’m looking for where it starts.”*
5. **Give the two-part cost.** *“One check is $O(n)$. The search does log of the answer range of them, so $O(n \log(\text{sum} - \text{max}))$. At fifty thousand packages that’s about twenty-five passes.”*

The follow-ups

“How do you know your feasibility check is monotone? Prove it.” For this problem it comes straight from the greedy packing, and I’d argue it rather than assert it. Take any capacity c that finishes in D days, and take c' bigger than c . Run the same greedy loading with c' . At every step the ship has at least as much room as it did with c , so every package that was loaded on day one with c is still loaded on day one with c' , and possibly more. By induction, after each day the c' run has loaded at least as many packages as the c run, so it cannot finish later. So `days_needed` is non-increasing in capacity, which means `days_needed(c) ≤ d` is monotone in exactly the direction I need. The reason I care about stating this is that the whole method is invalid without it, and the family has cousins where it fails — anything with the word “exactly” in it, and anything where a cost can go down as the resource goes up. So I check the property, in words, before I write the loop, every time.

“Your bounds — could you use 1 and 10^{18} instead and save yourself the thought?” It would still be correct here, and I’d say why: every capacity below `max(weights)` returns False and every capacity above `sum(weights)` returns True, so the boundary is in the same place whatever wrapper I put around it. What it costs is passes. Log base two of 10^{18} is about sixty, against about twenty-five for the tight bounds, so I’d more than double the runtime for no reason — and each pass is a full $O(n)$ walk, so that is not free. There is also a correctness risk in the general case: a lazy upper bound can overflow in a fixed-width language, and a lazy lower bound can hit an invalid value — a rate of zero divides by zero, a group size of zero isn’t a group. So my rule is that `lo` is the smallest value that is *meaningful*, and `hi` is a value I can point at and say “this certainly works”. If I cannot name why `hi` works, I do not have an upper bound, I have a guess.

“Now the packages can be shipped in any order, not just the given order. Same approach?” No, and that is a much harder problem, so I would not pretend otherwise. With the order fixed, the greedy check is exact — there is only one way to pack a prefix, so “fill until it doesn’t fit” is optimal. Remove the ordering and the check becomes “can these n weights be partitioned into d groups each summing to at most c ”, which is bin packing, and that is NP-hard. So the binary search skeleton survives — I’d still search the same answer range and still look for the boundary — but the feasibility check stops being exact, and a binary search over an approximate check gives an approximate answer. What I’d actually propose depends on the size: for small n , exact via a subset-based dynamic programme over 2^n states, which is fine up to about twenty packages. For large n , first-fit-decreasing as a heuristic, and I’d say plainly that the result is a good answer rather than the optimum. The important thing is naming that the *check* is where the difficulty moved, since that is the part of the pattern that carries the problem.

A model answer

“The phrasing is ‘the smallest capacity such that it’s possible in d days’, and that shape means binary search on the answer. The array of weights is not sorted and doesn’t need to be — what’s monotone is the yes-or-no answer attached to each candidate capacity.

So three things before I write the loop.

The range. The smallest capacity worth considering is the heaviest single package, because anything lighter can never load it at all. The largest is the sum of all the packages, because that ships everything in one day and nothing bigger helps. Both are $O(n)$ to compute, once.

The check. `days_needed(capacity)` walks the packages in the given order, adding each to the current day’s load, and when the next one doesn’t fit it starts a new day. The counter starts at one, not zero — day one exists before anything is loaded, and starting at zero undercounts on every input.

Monotonicity. More capacity can never mean more days: every packing that fits in a smaller ship still fits in a bigger one, so `days_needed` is non-increasing in capacity. That’s what makes the row of answers no-no-no-yes-yes-yes with exactly one boundary, and it’s the property the whole method depends on.

```
def ship_within_days(weights: list[int], d: int) → int:
    def days_needed(capacity: int) → int:
        days, load = 1, 0
        for w in weights:
            if load + w > capacity:
                days, load = days + 1, 0
            load += w
        return days

    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(mid) ≤ d:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

That’s the same six-line boundary template as ordinary binary search — the only thing that changed is the question.

Cost has two parts and I’d name both: one check is $O(n)$, and the search does $\log$ of the answer range many checks, so $O(n \log(\text{sum} - \text{max}))$. For fifty thousand packages with a range of twenty-five million that’s about twenty-five passes and just over

a million operations — against a trillion if I tried every capacity in turn.

And there's no not-found case: `hi = sum` always works, so a valid answer is guaranteed to be in the range. If I ever needed a not-found branch here, it would mean my upper bound was wrong."

9 Recall card

- **The tell is the sentence shape:** *smallest X such that it is possible / largest X such that it still holds*. The array is not sorted and is never searched — the **answers** are.
- **Three things, in order:** bound the answer range (`lo` = smallest *meaningful*, `hi` = something that certainly works) · write `works(x)` as a greedy pass · **say out loud why it is monotone**. Then it is [day 043](#)'s template, unchanged.
- **Cost is** `O(n × log(range))` — the check is no longer free. Tight bounds (`max .. sum`) halve the passes against lazy ones (`1 .. 1018`), and `lo` must be meaningful or you divide by zero.
- **No not-found case exists.** `works(hi)` is True by construction; if it is not, your `hi` is wrong and the answer returned is a confident lie.
- **"Exactly" kills monotonicity**, and the maximisation form needs `mid = (lo + hi + 1) // 2` or it hangs — so search for the first `False` and subtract one instead. Same family: LC 875, 1011, 410, 1482.

1 What this is, and why they ask it

Inheritance lets one class be declared a specialised kind of another. The child gets everything the parent has — attributes and methods — and may add to it or replace parts of it. `class Car(Vehicle)` says a car is a vehicle, and every place in the program that works with vehicles now works with cars without being told cars exist.

They ask about its problems, not its mechanics, and the wording of the question tells you so. Anyone can define a subclass; the interview is checking whether you know the three specific ways inheritance turns into a liability, because those three are what you will actually meet in a codebase. The question is also a proxy for judgement: a candidate who reaches for a class hierarchy at the first sign of two similar things will build one that is unchangeable in a year, and the interviewer has seen that codebase. Expect the question in every low-level design round, usually phrased as “why not just subclass it?” when you have proposed something else.

2 The story

Sathyan has been secretary of a four-storey building in Kaloor for six years, and about a third of what he does is water.

There is one tank on the roof and one main line coming down the side of the building, and every flat taps off that line. That is the arrangement and it has been since the building went up in 1994. It is a good arrangement, mostly. Nobody had to run their own line from the roof. A new family moves into a flat, they connect to what is already there, and water comes out of the tap on the first evening.

Then three things happened, in that order, and Sathyan can now tell you about all of them at length.

The ground-floor flat became a small clinic. A clinic needs the water filtered and it needs the pressure steady through the afternoon, and the main line gives neither — the pressure on the ground floor swings depending on who is running a washing machine two floors up. There was no way to fix that from the main line. They ran their own pipe from the tank, separately, at their own cost. So the rule that everyone taps the main was simply not true for them any more, and every notice Sathyan puts up about the main line has to carry an exception.

The second thing was the association's decision to fit a narrower valve at the top of the main line, because a family on the third floor was leaving a tap running. Three flats did not notice. The fourth had, two years earlier, put in a small booster of their own, sized for the old flow. With the narrower valve the booster ran dry and burnt out in a fortnight. Nobody had done anything wrong. Nobody who took the decision knew that flat had a booster, and nobody in that flat knew a decision was being taken.

The third thing is the one that still annoys him. The second-floor flat did not tap the main line at all. When they moved in, the first floor was empty and the plumber found it easier to take a branch off the first floor's pipe. It works. But now, if the first floor closes their own valve to fix a leak, the second floor has no water, and the second floor cannot understand why — they are not doing anything, and the tap is dry.

Sathyan's view after six years is not that the main line was a mistake. It is that it works when the flats really do want the same water in the same way, and it becomes a knot the moment one of them does not.

3 The idea in plain English

The main line is the parent class. Each flat is a subclass. The clinic is the **is-a lie**, the narrowed valve is the **fragile base class**, and the second floor branching off the first floor is the **deep hierarchy**. Those are the three places inheritance goes wrong, and the interviewer's question is asking for exactly these three.

What inheritance actually gives you

```
class Vehicle:
    def __init__(self, plate: str) → None:
        self.plate = plate

    def describe(self) → str:
        return f"{type(self).__name__} {self.plate}"

class Car(Vehicle):
    def required_spot(self) → str:
        return "compact"
```

`Car` did not define `__init__` or `describe`, and it has both. It gained them by declaration.

Two distinct benefits are bundled together here, and separating them is most of today's lesson:

- **Reuse** — the child does not rewrite the parent's code.

- **Substitutability** — any code written against `Vehicle` works with a `Car`, with no change.

The second is the one worth having. Reuse alone can be got from a helper function or from holding another object, which is [day 049](#)'s subject. Inheritance is the right tool when callers should be able to treat the child as the parent and never notice.

The rule: is-a, not has-a

Before writing `class B(A)`, say the sentence out loud: “**every B is an A.**” If it is not true without qualification, do not write it.

```
every Car is a Vehicle      → true. Subclass.
every SavingsAccount is an Account → true. Subclass.
every Car has an Engine     → "has a". NOT a subclass; hold one.
every Stack is a List       → false. A stack cannot be indexed in the middle.
```

That last one is not a made-up example. `java.util.Stack` extends `Vector` in the Java standard library, which means a `Stack` has `get(int index)` and `insertElementAt`, so anyone can reach into the middle of a stack. It is documented as a mistake by the people who maintain it, and it cannot be undone, because removing methods from a released class breaks every program that uses them.

The first failure: the is-a lie

The child cannot honour a promise the parent made. Sathyan's clinic.

The textbook case is `Square` and `Rectangle`. Mathematically a square is a rectangle, so the subclass looks obviously right:

```
class Rectangle:
    def __init__(self, w: int, h: int) → None:
        self._w, self._h = w, h
    def set_width(self, w: int) → None:
        self._w = w
    def set_height(self, h: int) → None:
        self._h = h
    def area(self) → int:
        return self._w * self._h

class Square(Rectangle):
    def set_width(self, w: int) → None:
        self._w = self._h = w          # a square must stay square
    def set_height(self, h: int) → None:
        self._w = self._h = h
```

Now this function, written against `Rectangle` long before `Square` existed:

```
def stretch(r: Rectangle) → None:
    r.set_width(5)
    r.set_height(4)
    assert r.area() == 20          # obviously true for a rectangle
```

Pass it a `Square` and the assertion fails: the area is 16. Nothing is buggy in isolation. `Square` is internally consistent and `stretch` is reasonable. The inheritance claim was the lie — a mutable square is **not** substitutable for a mutable rectangle, because a rectangle promises that width and height move independently and a square cannot keep that promise.

This has a name you will meet properly in the SOLID phase from [day 055](#): the **Liskov substitution principle**, which says a subclass must be usable anywhere the parent is, without the caller being able to tell. When it is violated, callers start writing `if isinstance(x, Square)`, and at that moment the inheritance has bought you nothing and cost you a type check in every caller.

The other classic: `class Penguin(Bird)` where `Bird.fly()` exists. The penguin must either raise, or do nothing, or lie, and all three are worse than not having claimed to be that kind of bird.

The second failure: the fragile base class

A change in the parent silently breaks children. Sathyan's narrowed valve.

```
class Collection:
    def add(self, item):
        self._items.append(item)

    def add_all(self, items):
        for item in items:
            self.add(item)          # ← calls its own add()

class CountingCollection(Collection):
    def __init__(self):
        super().__init__()
        self.count = 0
    def add(self, item):
        self.count += 1
        super().add(item)
```

`CountingCollection().add_all([1, 2, 3])` gives a count of 3. Correct. Now the parent's author, who has never heard of `CountingCollection`, optimises:

```
def add_all(self, items):
    self._items.extend(items)    # faster, and no longer calls self.add()
```


The parent's tests all pass. Every subclass that counted on `add_all` routing through `add` is now silently wrong — the count stays 0. Nobody did anything wrong. The parent's *internal* choice about which of its own methods to call had become part of its interface without anyone deciding that it should be.

This is why inheritance is described as the tightest coupling in object-oriented programming: **the child depends not only on what the parent does, but on how it does it**. Encapsulation, from [yesterday](#), protects an object from its callers. It does not protect a parent from its children, and that asymmetry is the whole problem.

The third failure: depth and multiple axes

Sathyan's second floor, tapping off the first floor.

Two things go wrong as hierarchies grow. First, **depth**: with `Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar`, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

Second, and worse, **more than one axis of variation**. Suppose vehicles vary by size (bike, car, truck) and by fuel (petrol, electric). Inheritance has one axis, so you get:

```
6 classes for 3 x 2

PetrolBike  PetrolCar  PetrolTruck
ElectricBike ElectricCar ElectricTruck

add hybrid          → 9 classes
add "with trailer"  → 18 classes
```

The class count is the product of the axes, and shared behaviour ends up duplicated across branches because there is nowhere else to put it. The fix is not a cleverer hierarchy. It is to stop using inheritance for the second axis and hold an object instead — a `Vehicle` that *has* a `FuelSystem` — which is [day 049](#).

Where inheritance is genuinely right

It is not a warning label on the whole idea. Four places it is the correct tool:

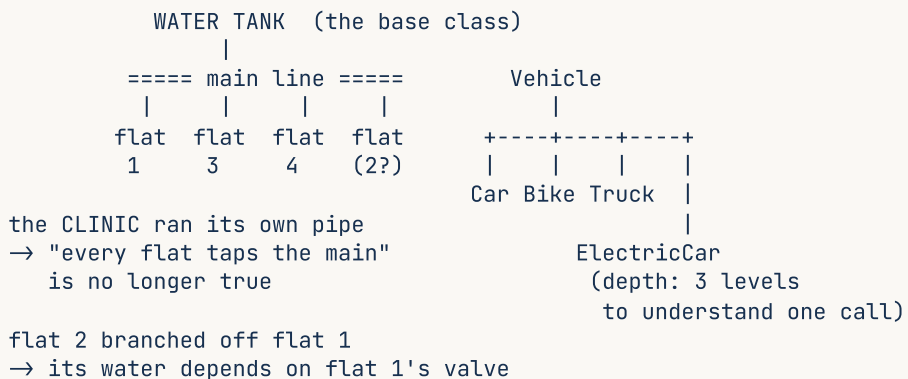
- **Exception hierarchies.** `class PaymentDeclined(PaymentError)` lets a caller catch the family or the specific one. Pure substitutability, no shared state, no fragile base — the parent has almost no implementation to break.
- **Abstract base classes as contracts.** `class PricingStrategy(ABC)` with `@abstractmethod fee()`. The parent has *no* implementation, so there is nothing for a child to depend on the internals of. This is inheritance used for the interface only, and it is the safest form.

- **Framework extension points.** Django's `class OrderView(ListView)`, Python's `class MyHandler(logging.Handler)`. The framework author designed the class to be subclassed and documented which methods you may override.
- **A genuine specialisation with no second axis.** `class SavingsAccount(Account)` where the only difference is interest calculation, and no other dimension is ever going to vary.

Notice what the first two have in common: **the parent has little or no implementation**. That is the practical rule — the less code in the parent, the less there is to be fragile about.

4 The picture

The building, and the hierarchy:



What to notice: the clinic did not break the main line. It broke the *claim* that every flat is served by it. That claim is what code written against the parent relies on.

The three failures, as a decision aid:

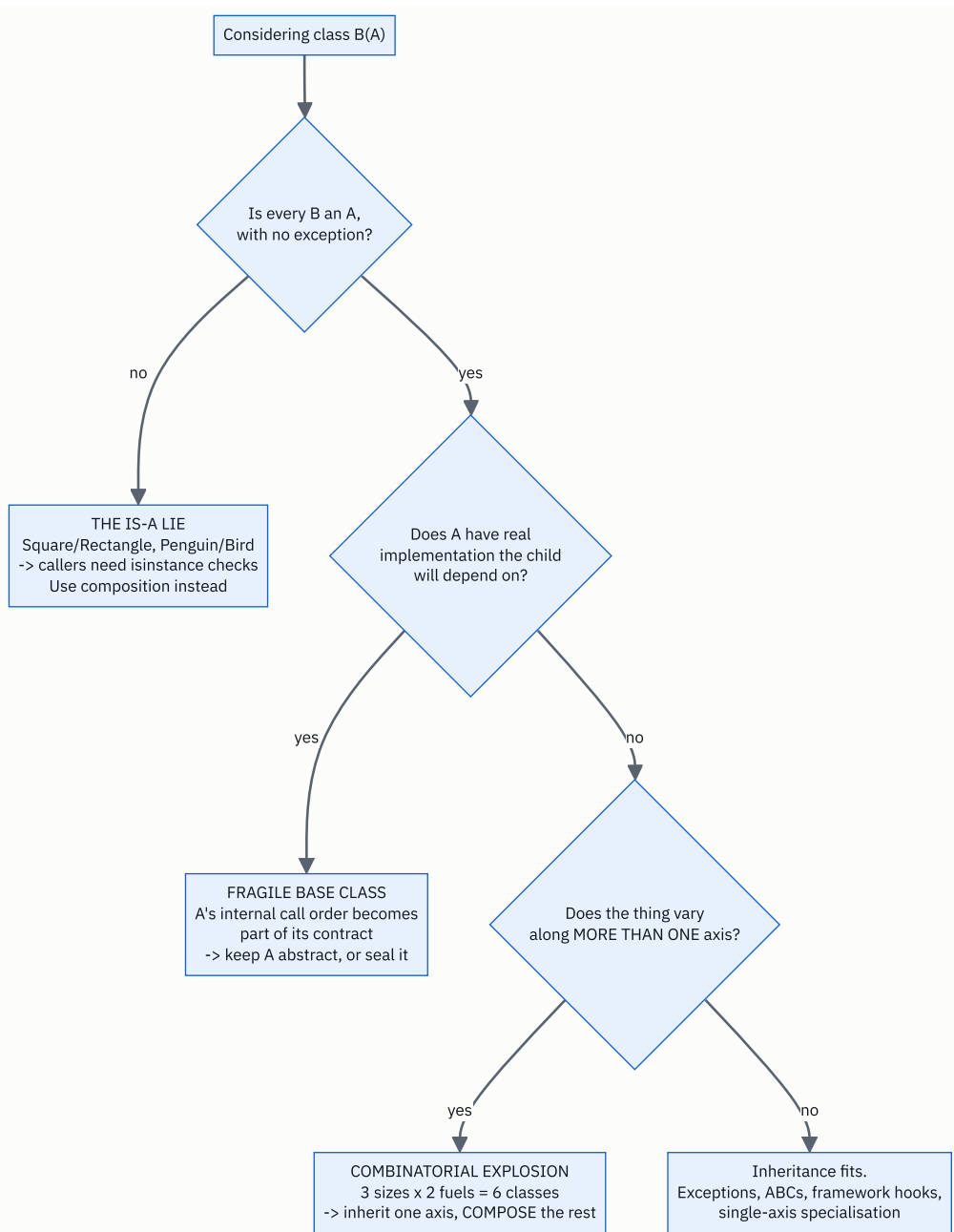


FIGURE 46.2

What to notice: every “no” branch ends at composition. That is not a coincidence, and it is the whole argument of [day 049](#).

5 How it actually works

The lookup, and `super()`

When you call `car.describe()`, Python looks for `describe` on the instance, then on `Car`, then on `Car`'s parents in a fixed order called the **method resolution order** — `Car.__mro__` prints it. The first one found wins. That is why an override works: the child's method is met first.

`super().describe()` does not mean “my parent”. It means “the next class after me in the MRO of the object I am actually running on”, which with multiple inheritance is not the same thing. The rule that keeps this sane: **always call `super().__init__()` in a child's constructor**, and always call it before using anything the parent sets up.

The diamond, and why languages disagree

```
class A:      pass
class B(A):   pass
class C(A):   pass
class D(B, C): pass      # D inherits A twice, through two paths
```

If `B` and `C` both override a method of `A`, which does `D` get? Python answers with a defined linearisation — `D, B, C, A, object` — so it is at least predictable. C++ requires you to say. **Java forbids it outright for classes**, allowing multiple inheritance only of interfaces, precisely because the ambiguity produced years of subtle bugs. Python's mixins work well when each mixin is small, stateless and touches a different method; they become unreadable when three of them all override `save()`.

The famous mistakes, in shipped libraries

- `java.util.Stack` extends `Vector`. A stack that can be indexed. Cannot be fixed; still in the standard library.
- `java.util.Properties` extends `Hashtable`. `Properties` is meant to map strings to strings, but it inherits `put(Object, Object)`, so anyone can store an `Integer` in it and the type guarantee is gone. The class's own documentation warns you not to use its inherited methods.
- **Java's `Date` and `Timestamp`**. `Timestamp` extends `Date`, and their `equals` methods are not symmetric — `date.equals(ts)` and `ts.equals(date)` can disagree. A substitutability violation shipped in the platform.

Naming one of these in an interview is worth more than a paragraph of theory, because it shows the problem is real rather than academic.

The designs that use it well

- **Python's exception tree** — `Exception` → `ArithmeticError` → `ZeroDivisionError`. `except ArithmeticError` catches the family. This works because the parents carry no behaviour.
- `collections.abc` — `Sequence`, `Mapping`, `Iterable` are abstract; you inherit the contract and get a few mixin methods derived from the ones you implement.
- `logging.Handler` — you subclass it and override `emit()`, and the framework documents exactly that method as the extension point.
- **Django's `Model`** — genuinely useful, and also the most common source of deep-MRO pain in Python web codebases, which is worth knowing both halves of.

6 The numbers

The combinatorial cost, counted

```
one axis of variation:    3 vehicle types           = 3 classes
two axes:                 3 types x 2 fuels           = 6 classes
three axes:               3 x 2 x 2 (trailer or not) = 12 classes
four axes:                3 x 2 x 2 x 3 (ownership)  = 36 classes

with composition instead:
    3 vehicle types + 2 fuel systems + 2 trailer options + 3 ownerships
                                = 10 classes, total

36 against 10, and adding a fifth fuel type is 12 new classes against 1.
```

That multiplication is the single most persuasive thing you can say on this topic. It is the reason “favour composition over inheritance” is advice and not taste.

The cost of a change, in files

```
change a method in a base class with 12 subclasses:
    files to READ before you can be confident      : 13
    subclasses that override the method             : maybe 4 -- unaffected
    subclasses that call it via another parent      : 
    method (the fragile-base case)                 : unknown until you read all 12

change a method in a class 5 levels deep:
    files to read to understand one call           : 5
    places the behaviour could be coming from      : 5
```

Runtime cost, so you do not over-weight it

attribute lookup on the class itself	~20 ns	
lookup found 3 levels up the MRO	~35 ns	(Python caches the MRO)
a super() call	~150 ns	
a request doing 10,000 such lookups:	~0.35 ms	
one database round trip:	~1 ms	

Depth costs comprehension, not speed. Say that explicitly — candidates sometimes argue against inheritance on performance grounds, and it is the wrong argument.

How deep is too deep

1 level (a class and its subclasses)	: normal, fine
2 levels	: fine with a reason
3 levels	: justify it
4+	: almost always two axes crammed into one

7 The trade-offs

Inheritance against composition

Inheritance is less code today and tighter coupling forever; composition is more wiring today and independent parts forever. *I would not use inheritance purely to reuse code* — if I want the parent's behaviour but not its identity, I hold an instance of it and call it. The one thing inheritance gives that composition does not is substitutability without any adapter, and that is what I would decide on.

A rich base class against an abstract one

A base class with real implementation saves duplication and creates the fragile-base problem. An abstract base with only signatures cannot be fragile, and pushes the shared code into a helper or a collaborator. *I would not put behaviour in a base class that its subclasses could come to depend on the internal call order of* — and if I do, I would document which methods are safe to override, the way `logging.Handler` documents `emit`.

Deep hierarchies against wide ones

I would not go past two levels without a reason I can say out loud. Three levels means a reader has three files open to understand one call, and it is usually the signature of a second axis of variation that should have been composed.

Multiple inheritance against mixins against interfaces

Mixins are genuinely useful when each is small, stateless and owns a distinct method. *I would not use multiple inheritance where two parents both carry state or both override the same method* — the MRO is defined but nobody reading the code will hold it in their head. Java's decision to allow only interface multiple-inheritance is a reasonable default to imitate even in Python.

The honest sentence

Inheritance is the strongest claim one class can make about another — that the child can stand in for the parent everywhere, forever. Make that claim when it is true and you get substitution for free. Make it because two classes happened to share four lines and you have welded them together for the life of the codebase.

8 In the interview

How it gets asked

- *“When does inheritance become a problem?”* — the direct form, and the answer is three named failures with an example each.
- *“Why not just subclass it?”* — the pushback when you have proposed composition in a design round.
- *“Is a square a rectangle?”* — the trap. The answer depends on mutability, and saying so is the whole answer.
- *“You have three vehicle types and two fuel types. Model it.”* — the combinatorial question, asked as a modelling exercise.

What to say out loud, in the first ninety seconds

1. **Give the test before the opinion.** *“Before I subclass anything I say the sentence: every B is an A, with no exception. If it needs a qualification, it's a has-a and I'll compose instead.”*
2. **Separate the two benefits.** *“Inheritance bundles reuse and substitutability. Reuse I can get from holding an object. Substitutability is the thing only inheritance gives, so that's what I decide on.”*
3. **Name the three failures, briefly.** *“The three ways it goes wrong: the is-a lie, where the child can't keep a promise the parent made; the fragile base class, where the parent's internal call order becomes part of its contract; and multiple axes of variation, where the class count becomes a product.”*

4. **Say where it is right.** *“I’d still use it for exception hierarchies, abstract base classes as contracts, and framework extension points — all cases where the parent has little or no implementation, so there’s nothing fragile to depend on.”*
5. **Then apply it to the actual problem in front of you.**

The follow-ups

“Is a square a rectangle? Should `Square` extend `Rectangle`?” Mathematically yes, in code it depends entirely on whether the objects are mutable, and that is the real answer. If `Rectangle` has `set_width` and `set_height`, it is making a promise: those two move independently. `Square` cannot keep that promise — setting the width has to change the height — so any function written against `Rectangle` can be broken by passing a `Square`. The concrete failure is a function that sets width to 5, sets height to 4, and expects an area of 20; with a square it gets 16, and nothing in either class is buggy in isolation. That is a Liskov substitution violation, and the symptom in a real codebase is `isinstance` checks appearing in callers — at which point the inheritance has bought nothing and cost a type check everywhere. If instead both are immutable value objects with no setters, `Square` extending `Rectangle` is completely fine, because there is no promise to break: you can only construct, never mutate. So my answer is that the question is not about geometry at all — it is about which operations the parent exposes.

“Give me a real bug caused by a fragile base class.” The one I’d describe is the counting-collection case, because it is the shape that appears again and again. A base `Collection` has `add(item)` and `add_all(items)`, and `add_all` is implemented as a loop that calls `self.add`. A subclass overrides `add` to increment a counter, and everything works — `add_all` routes through the override, the count is right. Then someone optimises the base class to use `self._items.extend(items)` instead of the loop. Every test on the base class passes, because nothing about the base class’s behaviour changed. Every subclass that was counting is now silently wrong, and the failure shows up as a number being too low in a report, weeks later, with nothing in the logs. The root cause is that the parent’s choice about which of its own methods to call had become part of its interface without anyone deciding it should be. The lessons I’d draw are: a base class meant for subclassing must document its self-calls as part of the contract, or avoid them entirely; and this is why abstract base classes with no implementation are the safest form of inheritance — there is nothing to depend on.

“So should I never use inheritance?” No, and I would push back on the slogan version of the advice. Inheritance is the right tool exactly when the parent has little or no implementation and the child genuinely substitutes. Exception hierarchies are the cleanest example: `PaymentDeclined` extending `PaymentError` extending `Exception` lets a caller catch the family or the specific case, there is no state in the parents, and there is no way for a change in `PaymentError` to break `PaymentDeclined`. Abstract base classes are

the second: a `PricingStrategy` with one abstract method is a contract, and implementing it is inheritance used purely for substitutability. Framework hooks are the third — when Django or the `logging` module documents a class as designed for subclassing and names the method to override, the fragile-base problem has been handled by the author. The pattern across all three is that the danger lives in the parent's *implementation*, so the less of it there is, the safer the inheritance. The rule I actually use is: inherit interfaces freely, inherit implementation carefully, and never inherit just to avoid typing.

A model answer

“It becomes a problem in three specific ways, and I’d name them with an example each rather than talk about coupling in the abstract.

The first is the is-a lie. Inheritance claims the child can stand in for the parent everywhere, and when that’s not true the callers pay. The standard case is a mutable `Square` extending a mutable `Rectangle`: a function that sets width to 5, then height to 4, then expects an area of 20 gets 16. Neither class is buggy on its own — the claim was wrong. The symptom in a codebase is `isinstance` checks appearing in callers, and at that point the inheritance is costing more than it saved.

The second is the fragile base class. A child depends not just on what the parent does but on how it does it. If a base `add_all` loops calling `self.add`, a subclass that overrides `add` to count works — until someone optimises `add_all` to use `extend`. The base class’s tests all still pass and every subclass is silently wrong. The parent’s internal call order had become part of its contract without anyone deciding that.

The third is more than one axis of variation. Three vehicle types and two fuel types is six classes; add a trailer option and it’s twelve; add ownership models and it’s thirty-six. The count is the product of the axes, and shared behaviour gets duplicated across branches because there’s nowhere else to put it. With composition it’s ten classes total and a new fuel type is one class instead of twelve.

Where I’d still use it: exception hierarchies, abstract base classes as contracts, and framework extension points that were designed to be subclassed. What those have in common is that the parent carries little or no implementation — which is the practical rule. Inherit interfaces freely, inherit implementation carefully, and never inherit purely to reuse code, because reuse I can get by holding an object. Substitutability is the only thing inheritance uniquely gives, so that’s what I decide on.

The test I say out loud before writing `class B(A)`: every B is an A, with no exception. If it needs a qualification, it’s a has-a.”

9 Recall card

- **Say the sentence first: “every B is an A, with no exception.”** If it needs a qualification, it is has-a — hold an object instead. Inheritance bundles reuse and substitutability; only the second is worth the coupling.
- **Failure 1 — the is-a lie.** Mutable `Square` / `Rectangle`: set width 5, height 4, expect area 20, get 16. Symptom: `isinstance` checks in callers. (Liskov, properly on [day 055](#).)
- **Failure 2 — the fragile base class.** The parent’s internal self-calls become its contract; optimising `add_all` from a loop to `extend` silently breaks every counting subclass, with all parent tests green.
- **Failure 3 — multiple axes.** 3 types $\times$ 2 fuels = 6 classes, $\times$ trailer = 12, $\times$ ownership = 36 against 10 by composition. The class count is the *product* of the axes.
- **It is right when the parent has almost no implementation:** exception trees, abstract base classes as contracts, documented framework hooks, single-axis specialisation. Depth costs comprehension, not speed — past two levels, justify it.

DSA topic: Binary search on the answer **System design topic:** Inheritance and its costs

Code these, in this order

For every one of these, **write the three things down before the loop**: the answer range with a reason for each end, the `works(x)` function, and the sentence saying why it is monotone. The search loop is then copied unchanged from the previous problem.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Koko Eating Bananas	LeetCode 875 (Medium)	The friendliest entry: bounds of 1 to max, and ceiling division.
2	Capacity To Ship Packages Within D Days	LeetCode 1011 (Medium)	Bounds of <code>max</code> to <code>sum</code> , and the day counter starting at one.
3	Split Array Largest Sum	LeetCode 410 (Hard)	That a problem which reads like dynamic programming is this instead.
4	Minimum Number of Days to Make m Bouquets	LeetCode 1482 (Medium)	A feasibility check with a run-length rule inside it.

On problem 1, get the bounds right and say why

`lo = 1`, not 0 — a speed of zero never finishes, and in code it raises `ZeroDivisionError: integer division or modulo by zero`. `hi = max(piles)` — at that speed every pile takes exactly one hour, so `len(piles)` hours, which is the fastest anything can be. Prove to yourself that no bigger `hi` helps by checking `hours_needed(max(piles))` equals `hours_needed(10**9)`.

Then get the ceiling right: `(pile + k - 1) // k`, because a part-eaten pile still costs a whole hour. Check it on `pile = 7, k = 3`: three hours, not two.

On problem 2, test the counter

Write `days_needed` on its own and check three things before the search exists: `[1,2,3,4,5]` with capacity 5 gives 4, with capacity 15 gives 1, and with capacity 100 gives 1. Then deliberately change `days` to start at 0 and re-run all three — you get 3, 0 and 0. Say the one-line reason.

On problem 3, notice what changed

Nothing. Rename `capacity` to `limit` and `days` to `parts`, and the code is character-for-character problem 2. Do problem 3 without looking at problem 2, then diff them. If they differ anywhere except the names, one of them is wrong.

On problem 4, the check has structure inside it

The feasibility check is not a running total this time — it counts *runs* of consecutive bloomed flowers of length `k`. Write it separately and test it before searching. Then handle the impossible case: if `m * k > len(bloomDay)` there is no answer at all, and that is the one problem in this set with a genuine not-found branch. Say why the others do not have one.

The three-things drill

For each phrasing, produce the answer range with a reason for each end, the `works` function in one sentence, and the monotonicity sentence — under sixty seconds each, out loud:

1. Smallest ship capacity to clear packages in `d` days.
2. Slowest eating speed to finish piles in `h` hours.
3. Smallest largest-part-sum when splitting into `k` contiguous parts.
4. Largest minimum distance when placing `c` cows in `n` stalls.
5. Smallest number of days until `m` bouquets of `k` adjacent flowers can be made.

Number 4 is the maximisation form. Say which of the two routes you would take and why.

The bounds drill

For each, say whether the bounds are correct, wasteful, or broken — and what the symptom is:

1. `lo, hi = 0, sum(weights)` for ship capacity.
2. `lo, hi = 1, max(piles)` for eating speed.
3. `lo, hi = max(weights), max(weights) * len(weights) // 2` for ship capacity.
4. `lo, hi = 1, 10**18` for eating speed.
5. `lo, hi = min(nums), sum(nums)` for split-array largest sum.

For the broken ones, say what the function returns and why no exception is raised.

The monotonicity drill

For each phrasing, say monotone or not — and if not, why binary search on the answer is the wrong tool:

1. “The smallest capacity such that it takes **at most** d days.”
2. “The smallest k such that **exactly** k groups can be formed.”
3. “The smallest number of workers such that the total wage bill is under ₹ X , where extra workers cut overtime.”
4. “The largest minimum gap between any two chosen positions.”
5. “The smallest window size such that every value appears **at least** twice.”

The hierarchy audit

Take this and answer five questions about it:

```
class Employee:
    def pay(self): ...
    def annual_review(self): ...

class Manager(Employee):
    def approve_leave(self, request): ...

class Contractor(Employee):
    def annual_review(self):
        raise NotImplementedError("contractors have no annual review")
```

1. Which of the three named failures does `Contractor` demonstrate? Give the exact symptom a caller will show.
2. What does a function written as `for e in employees: e.annual_review()` do, and whose fault is it?
3. Rewrite it so the problem cannot exist. There is more than one right answer — give two.
4. Now add a second axis: employees are either on-site or remote, and that changes how expenses are claimed. How many classes does a pure inheritance model need? How many with composition?
5. Which of the four “inheritance is right here” cases from the lesson does none of this resemble, and why?

The is-a drill

For each, say subclass or compose, and give the one-sentence reason:

1. `SavingsAccount` and `Account`.
2. `Stack` and `List`.
3. `Car` and `Engine`.
4. `PaymentDeclined` and `PaymentError`.
5. `ElectricCar` and `Car`, in a system that also varies by size and by ownership model.

6. `Square` and `Rectangle`, where both are immutable.

Number 6 has a different answer from the mutable version. Say what changed.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the smallest capacity that lets you ship all packages within d days.* The sentence shape, the bounds with a reason for each end, the greedy check with its counter starting at one, the monotonicity sentence, and the two-part cost.
 2. *When does inheritance become a problem?* The three named failures with an example each, then the four places it is genuinely right and the property they share.
 3. *Why is your feasibility check monotone? Prove it.* The induction argument for the shipping case — more capacity means at least as many packages loaded each day, so it can never finish later — and then name a phrasing where it fails.
-

Before you move on

- ☐ For all four problems I wrote the range, the check and the monotonicity sentence before the loop.
- ☐ My search loop is character-identical across problems 2 and 3.
- ☐ I can say why there is no not-found case in three of the four, and why problem 4 has one.
- ☐ I sorted all five bounds in the drill and named the symptom of each broken one.
- ☐ I can name the three inheritance failures with an example each, unprompted.
- ☐ I did the hierarchy audit including the two-axis class count, both ways.
- ☐ I answered all three questions above out loud.

Minimise the maximum: the capacity family

DATA STRUCTURES AND ALGORITHMS

Minimise the maximum: the capacity family

You can recognise minimise-the-maximum wording and turn it into a yes-or-no predicate.

SYSTEM DESIGN

Polymorphism

You can explain polymorphism with code, not with a dictionary definition.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Split the array into k parts, minimising the largest part sum.*
- *What is polymorphism? Show me, do not tell me.*

Minimise the maximum: the capacity family

1 What this is, and why they ask it

Yesterday you learnt the machine: bound the answer range, write a feasibility check, run the boundary template. Today you learn to **recognise when to start it**, and you learn its mirror. Half of this family asks you to make the biggest thing as small as possible — *minimise the maximum* — and the other half asks you to make the smallest thing as large as possible — *maximise the minimum*. They are the same machine pointed in opposite directions, and confusing which one you are in is the most common way to lose an otherwise correct answer.

They ask it because the wording is a deliberate disguise. “Split the array into k parts, minimising the largest part sum” reads as an optimisation problem, and the honest first instinct is dynamic programming — which works, at $O(n^2 \cdot k)$, and is far more code. Interviewers use LeetCode 410 as a hard question precisely to see whether you can turn an optimisation phrase into a decision question. Once you can, a whole shelf of problems that look unrelated — splitting arrays, placing cows in stalls, dividing chocolate, distributing products to stores — become one recipe you have already written.

2 The story

Vijaya has invigilated the half-yearly exams at the same school in Erode for fourteen years, and the hall is the same hall every time: one long room with a single row of forty desks bolted to the floor down one wall.

The desks are not evenly spaced. Whoever fixed them worked around two pillars and a doorway, so some are almost touching and there is a long empty stretch in the middle where a bench used to be.

This year she has eight students in her sitting, and one job: get them as far apart as she can.

Not “far apart on average”. What she actually worries about is the closest pair — the two who end up nearest each other are the two who will be able to see something, and it does not matter how spread out the other six are. So the number she is trying to push up is the smallest gap in the room.

She has done this long enough that she does not fiddle with it. She thinks of a gap, tests it, and the test is always the same walk.

Say the gap is six desks. She puts the first student at the very first desk — there is never a reason not to, because starting further in only wastes room. Then she walks along the row until she reaches the first desk that is at least six away from him, and puts the second student there. Then she walks on from there, six more, third student. And so on, taking the earliest desk that is far enough each time, because leaving anyone further along than necessary only makes it harder for whoever comes after.

If she gets all eight seated before the row runs out, six works.

The first time she tried six this year, she got seven students in and ran out of desks. So six is too much — and, she knows without checking, so is seven, and eight, and anything larger. Those are all gone in one go.

So she tried three. All eight seated, easily, with desks to spare. Three works — but then so does two and so does one, and she does not want a gap that works, she wants the largest gap that works. Below three is finished with.

Four: all eight seated. Five: only seven. So the answer is four.

Four walks down the row, and every one of them told her about a whole stretch of gaps at once rather than about a single one.

3 The idea in plain English

Vijaya's row of desks is the input array. The **gap** is the answer she is searching for — not something in the array, a number she is choosing. And her walk is the feasibility check.

The two phrasings, and how to tell them apart

Every problem in this family asks for one of two things:

MINIMISE THE MAXIMUM	"split into k parts, make the LARGEST part as SMALL as possible"
answers run:	no no no YES YES YES
	^ first True = the answer
MAXIMISE THE MINIMUM	"place c cows, make the SMALLEST gap as LARGE as possible"
answers run:	YES YES YES no no no
	^ last True = the answer

The way to tell them apart in one second: **ask which direction "more" helps**. A bigger allowed part sum makes splitting *easier*, so the yes-answers are at the top. A bigger required gap makes seating *harder*, so the yes-answers are at the bottom. Say that sentence out loud before writing anything; it decides which end of the range you are looking for.

Minimise the maximum, in full

The canonical form is LeetCode 410:

Split `nums` into `k` contiguous parts. Minimise the largest part sum.

The trick is to **stop asking for the minimum and start asking a yes-or-no question about a limit**:

Given a limit `L`, can `nums` be split into at most `k` contiguous parts, each summing to at most `L`?

That question has a greedy answer that is exactly yesterday's shipping walk — keep adding to the current part until the next value would exceed `L`, then start a new part — and the greedy is optimal because the parts must be contiguous, so there is only one sensible place for each boundary.

```
def parts_needed(nums: list[int], limit: int) → int:
    parts, total = 1, 0
    for x in nums:
        if total + x > limit:
            parts, total = parts + 1, 0
        total += x
    return parts
```

Then `works(L)` is `parts_needed(nums, L) ≤ k`, and the answer is the first `L` for which it is true.

Range: `lo = max(nums)` — no part can be smaller than the biggest single element, since parts are contiguous and every element must land in one. `hi = sum(nums)` — one part holds everything.

Maximise the minimum, in full

Vijaya's problem is the standard “aggressive cows”, and LeetCode 1552 is its exact form:

Given positions of `n` stalls and `c` cows, place the cows so that the minimum distance between any two is as large as possible.

Again, stop asking for the maximum:

Given a required gap `g`, can all `c` cows be placed with every pair at least `g` apart?

The check is Vijaya's walk. Place the first at the earliest position; then take the earliest position at least `g` beyond the last placement; repeat.

```
def can_place(positions: list[int], cows: int, gap: int) → bool:
    placed, last = 1, positions[0]      # first cow at the first stall, always
    for p in positions[1:]:
        if p - last ≥ gap:
            placed += 1
            last = p
        if placed == cows:
            return True
    return False
```

`positions` must be sorted first — that is an $O(n \log n)$ cost outside the search, and forgetting it is the trap in §7.

The greedy is optimal, and it is worth being able to say why: placing a cow later than the earliest valid stall can never help, because it leaves strictly less room for everyone after. That is an exchange argument, and one sentence of it is what the interviewer wants.

Range: `lo = 1` — the smallest meaningful gap. `hi = positions[-1] - positions[0]` — the whole span, which certainly fails for more than two cows but is a valid upper edge.

Handling the mirrored direction without a second template

The answers now run `YES, YES, no, no`, and the answer is the **last True**. Two routes:

Route one, recommended. Search for the **first False** with the untouched template, then subtract one:

```
first_bad = first_true(lo, hi + 1, lambda g: not can_place(positions, cows, g))
answer = first_bad - 1
```

One template, forever. The `hi + 1` is because the range is half-open and you want `hi` itself to be a candidate.

Route two. Mirror the template, and remember the ceiling midpoint:

```

while lo < hi:
    mid = (lo + hi + 1) // 2      # + 1 is MANDATORY here
    if can_place(positions, cows, mid):
        lo = mid                # mid works; it might be the largest
    else:
        hi = mid - 1
return lo

```

Without the `+ 1`, when `hi = lo + 1` the midpoint is `lo`, and `lo = mid` changes nothing — the loop hangs silently. Both routes are correct; route one costs you nothing to remember.

The translation table

This is the thing to hold. Every one of these is the same machine:

phrasing	the yes-or-no question
minimise the largest part sum, k parts	can it be done in $\leq k$ parts with limit L?
minimise the largest number of products any store must handle	can every store take $\leq L$ products?
minimise the days to finish, given a rate	can it be finished in $\leq d$ days at rate r?
maximise the smallest gap between placed items	can c items be placed with every gap $\geq g$?
maximise the smallest piece of chocolate	can k+1 pieces each be $\geq s$ sweetness?
maximise the number of items you can buy	can n items be bought with budget $\leq B$?

Every right-hand column is a `works(x) → bool` that costs one `O(n)` pass. That is the family.

Why not dynamic programming

LeetCode 410 has a genuine dynamic-programming solution: `dp[i][j]` is the best largest-part-sum for the first `i` elements in `j` parts. It is correct, and it costs `O(n2 k)` time and `O(nk)` space.

```

n = 1,000 elements, k = 50 parts:
DP:           1,000 x 1,000 x 50   = 50,000,000 operations, 50,000 cells
binary search: 1,000 x log2(sum)    = 1,000 x ~30 = 30,000 operations, O(1) space

```

Over a thousand times less work, in about a quarter of the code. Naming the dynamic-programming solution and then rejecting it with those numbers is a strong move in an interview — it shows you chose rather than only knew one thing.

4 The picture

Vijaya's row, with a gap of 4 being tested. Eight students, and these are the desk positions measured along the wall:

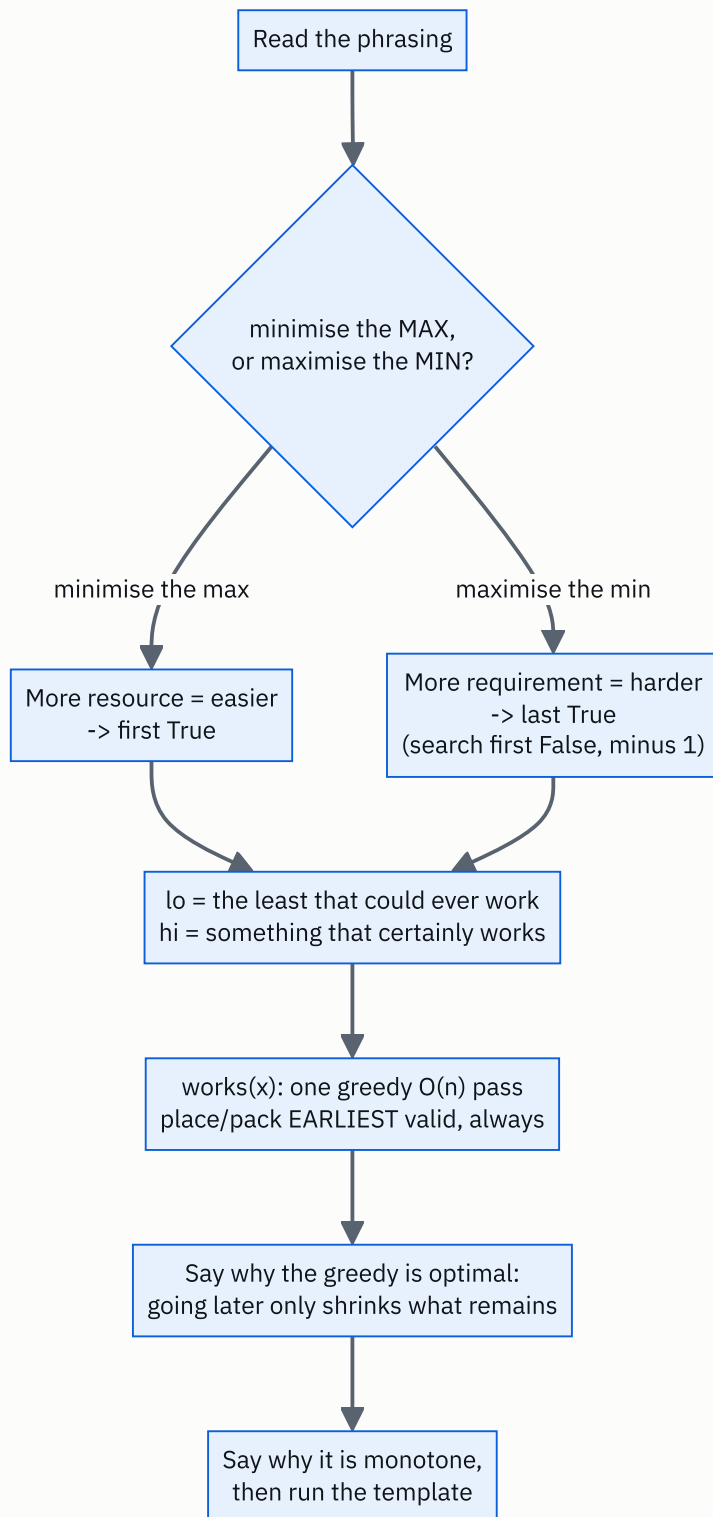


FIGURE 47.1

What to notice: two of the six boxes are sentences you say rather than code you write, and they are the two candidates skip.

5 The code, built step by step

Minimise the maximum: the check

```
def parts_needed(nums: list[int], limit: int) → int:
    parts, total = 1, 0
    for x in nums:
        if total + x > limit:
            parts, total = parts + 1, 0
        total += x
    return parts
```

Yesterday's shipping walk, renamed. `parts` starts at 1 because the first part exists before anything is added to it.

Minimise the maximum: the search

```
lo, hi = max(nums), sum(nums)
while lo < hi:
    mid = (lo + hi) // 2
    if parts_needed(nums, mid) ≤ k:
        hi = mid
    else:
        lo = mid + 1
return lo
```

Unmodified template. `lo = max(nums)` because a single element bigger than the limit could never be placed anywhere.

Maximise the minimum: the check

```
def can_place(positions: list[int], cows: int, gap: int) → bool:
    placed, last = 1, positions[0]
    for p in positions[1:]:
        if p - last ≥ gap:
            placed, last = placed + 1, p
            if placed == cows:
                return True
    return False
```

The early `return True` matters for cost: once all cows are placed there is no reason to keep walking. It does not change the class, and it halves the work on easy candidates.

Maximise the minimum: the search, done the safe way

```
positions.sort()                                # REQUIRED. O(n log n), once.
lo, hi = 1, positions[-1] - positions[0]

# search for the first gap that FAILS, then step back one
first_bad = lo
low, high = lo, hi + 1                          # half-open, so hi itself is a candidate
while low < high:
    mid = (low + high) // 2
    if not can_place(positions, cows, mid):
        high = mid
    else:
        low = mid + 1
first_bad = low
return first_bad - 1
```

One template, pointed at “the first failure”. No ceiling midpoint, no mirrored branches, nothing new to remember.

The complete solution

```
def split_array_largest_sum(nums: list[int], k: int) → int:
    """LeetCode 410. Split nums into k contiguous parts; minimise the largest part sum.

    MINIMISE THE MAXIMUM: more limit makes splitting easier, so the yes-answers are at
    the top and the answer is the FIRST True.
    """
    def parts_needed(limit: int) → int:
        parts, total = 1, 0
        for x in nums:
            if total + x > limit:
                parts, total = parts + 1, 0
            total += x
        return parts

    lo, hi = max(nums), sum(nums)          # least possible / certainly enough
    while lo < hi:
        mid = (lo + hi) // 2
        if parts_needed(mid) ≤ k:
            hi = mid
        else:
            lo = mid + 1
    return lo

def max_min_distance(positions: list[int], cows: int) → int:
    """LeetCode 1552 / 'aggressive cows'. Place cows so the smallest gap is as large as possible.

    MAXIMISE THE MINIMUM: more gap makes placing harder, so the yes-answers are at the
    bottom and the answer is the LAST True -- found as (first False) - 1.
    """
    positions = sorted(positions)          # the search is meaningless without this

    def can_place(gap: int) → bool:
        placed, last = 1, positions[0]     # earliest stall, always: going later never helps
        for p in positions[1:]:
            if p - last ≥ gap:
                placed, last = placed + 1, p
            if placed == cows:
                return True
        return False

    lo, hi = 1, positions[-1] - positions[0]
    low, high = lo, hi + 1                  # half-open so that hi itself is a candidate
    while low < high:
        mid = (low + high) // 2
        if not can_place(mid):              # looking for the FIRST failure
            high = mid
        else:
            low = mid + 1
    return low - 1                          # step back to the last gap that worked

def min_max_products(quantities: list[int], n: int) → int:
    """LeetCode 2064. Distribute products to n stores; minimise the most any store gets.

    Same family again: the check is a sum of ceiling divisions rather than a greedy walk.
    """
    def stores_needed(limit: int) → int:
        return sum((q + limit - 1) // limit for q in quantities)

    lo, hi = 1, max(quantities)
    while lo < hi:
        mid = (lo + hi) // 2
        if stores_needed(mid) ≤ n:
            hi = mid
        else:
            lo = mid + 1
    return lo

if __name__ == "__main__":
```

```

print(split_array_largest_sum([7, 2, 5, 10, 8], 2))      # 18
print(split_array_largest_sum([1, 2, 3, 4, 5], 2))      # 9
print(split_array_largest_sum([1, 4, 4], 3))            # 4
print(split_array_largest_sum([5], 1))                  # 5    ← one part
print(split_array_largest_sum([1, 1, 1, 1], 4))          # 1    ← k = n

print(max_min_distance([1, 2, 4, 8, 9], 3))             # 3
print(max_min_distance([5, 4, 3, 2, 1, 1000000000], 2)) # 999999999
print(max_min_distance([1, 2, 3, 4, 7], 3))             # 3
print(max_min_distance([1, 2], 2))                      # 1    ← two cows, two stalls

print(min_max_products([11, 6], 6))                    # 3
print(min_max_products([15, 10, 10], 7))               # 5
print(min_max_products([100000], 1))                   # 100000

```

Three problems. The search loop in the first and third is character-identical; the second is the same loop with the question negated and one subtraction at the end. That is the point of the day.

6 What it costs

Time, both directions

```

minimise the maximum (LC 410):
    one check      : 0(n)                      one pass over nums
    passes         : log2(sum(nums) - max(nums))
    total          : 0(n x log(sum - max))

maximise the minimum (LC 1552):
    sort           : 0(n log n)                once, outside the search
    one check      : 0(n)
    passes         : log2(last - first)
    total          : 0(n log n + n x log(span))

```

Note where the sort sits. For the placement problems it is a real cost and it can dominate when the coordinate span is small:

```

n = 100,000 positions, span up to 10^9:
sort   : 100,000 x 17      = ~1,700,000 operations
search : 100,000 x 30 passes = 3,000,000 operations
-----
~4,700,000 -- the search dominates, but only just

```

Against the alternatives

```

LC 410, n = 1,000, k = 50:
dynamic programming  0(n^2 k) = 1,000 x 1,000 x 50 = 50,000,000 ops, 50,000 cells
binary search        0(n log(sum)) = 1,000 x 30 = 30,000 ops, 0(1) space

~1,600x less work, and about a quarter of the code.

```

Space

```
lo, hi, mid + the check's two counters    → 0(1) extra
sorting in place (positions.sort())       → 0(1) extra beyond the input
sorted(positions) making a copy           → 0(n)
```

Worth one sentence in an interview: sorting in place mutates the caller's list, which is sometimes rude and sometimes exactly what is wanted. Say which you are doing.

The number to have ready

Bound the answer range, write an $O(n)$ greedy check, run log-of-the-range checks. For a thousand elements split into fifty parts that is about thirty thousand operations against fifty million for the dynamic-programming solution — and the binary search is $O(1)$ space where the DP table is fifty thousand cells.

7 The traps

The near-miss: searching the wrong end

```
# "maximise the minimum gap", written with the minimise-the-maximum template
lo, hi = 1, positions[-1] - positions[0]
while lo < hi:
    mid = (lo + hi) // 2
    if can_place(mid):
        hi = mid
    else:
        lo = mid + 1
return lo

print(wrong([1, 2, 4, 8, 9], 3))    # 1    should be 3
```

1

No error. The code found the *smallest* gap that works, which is always 1, and returned it with confidence. The fix is not a patch — it is asking the one-second question first: **does more of this quantity make the task easier or harder?** Easier means first True. Harder means last True.

The near-miss: forgetting to sort

```
print(max_min_distance_unsorted([5, 1, 9, 2, 8], 3))    # 1    should be 4
```

The greedy walk assumes positions come in increasing order — `p - last` is only a distance if `p` comes after `last`. On unsorted input the differences go negative, the placement count is wrong, and the answer is garbage. Third appearance of the same lesson: **binary search on the answer does not need the input sorted, but the feasibility check often does.** Those are different requirements and it is worth keeping them separate in your head.

The real error: the mirrored template without the ceiling midpoint

```
lo, hi = 1, 10
while lo < hi:
    mid = (lo + hi) // 2          # floor, in a loop that does lo = mid
    if mid ≤ 7:
        lo = mid
    else:
        hi = mid - 1
print(lo)
```

There is no traceback. There is no output. The process sits at 100% of one core until you kill it:

```
^C
Traceback (most recent call last):
  File "day47.py", line 3, in <module>
    while lo < hi:
KeyboardInterrupt
```

When `lo = 7` and `hi = 8`, `mid = 7`, the branch sets `lo = 7`, and nothing has changed. Use `mid = (lo + hi + 1) // 2`, or — better — do not write this form at all.

The near-miss: a greedy check that is not optimal

For the contiguous-split problems the greedy is provably optimal, because the parts must be adjacent and there is only one place each boundary can go. **Remove the contiguity and it collapses.** “Split these numbers into k groups in any arrangement, minimising the largest group sum” is a partitioning problem, and it is NP-hard — the greedy check gives an answer, not the answer, and a binary search over an approximate check gives an approximate result. Read the problem statement for the words “contiguous”, “subarray”, or “in the given order”. If they are absent, stop and say so.

The trap: `lo` set to a meaningless value

```
lo, hi = 0, max(quantities)          # ← a store limit of 0
print(sum((q + lo - 1) // lo for q in quantities))
```

```
Traceback (most recent call last):
  File "day47.py", line 2, in <module>
    print(sum((q + lo - 1) // lo for q in quantities))
           ~~~~~^~~~~
ZeroDivisionError: integer division or modulo by zero
```

Same rule as yesterday, and it keeps mattering: **the lower bound is the smallest meaningful value, not the smallest number**. A limit of zero, a gap of zero, a rate of zero — none of these are candidates, and two of the three raise.

8 In the interview

How it gets asked

- “Split the array into k contiguous subarrays. Minimise the largest subarray sum.” — LeetCode 410, labelled Hard, and one of the best-value questions in this family to have prepared.
- “Place c cows in these stalls so the minimum distance between any two is as large as possible.” — aggressive cows, LeetCode 1552, the mirrored form.
- “You have m products of various quantities and n stores. Minimise the maximum any one store handles.” — LeetCode 2064, with a ceiling-division check instead of a greedy walk.
- “Divide the chocolate into $k + 1$ pieces; maximise the sweetness of the piece you get, which is the smallest.” — LeetCode 1231, and the phrasing is deliberately convoluted.

What to say out loud, in the first ninety seconds

1. **Name the family from the phrasing.** “‘Minimise the largest part’ is a minimise-the-maximum problem, so I’ll binary search on the answer rather than compute it directly.”
2. **Ask the direction question, out loud.** “Does a bigger allowed limit make the task easier or harder? Easier — more room means fewer parts. So the yes-answers are at the top and I want the first True.”
3. **Restate the problem as a decision.** “Instead of ‘what is the minimum largest sum’, I’ll answer ‘given a limit L , can I do it in at most k parts?’ — and then find the smallest L for which the answer is yes.”
4. **Bound the range with reasons.** “ L can’t be below $\max(\text{nums})$, because that element has to sit in some part on its own at worst. And it never needs to exceed $\text{sum}(\text{nums})$, which is one part. So max to sum.”
5. **Justify the greedy in one sentence.** “The parts are contiguous, so the check is: keep adding until the next value would exceed L , then start a new part. That’s optimal because ending a part earlier than forced only makes the remaining work harder.”

6. Name and reject the alternative. *“There’s a dynamic-programming solution at $O(n^2k)$ — for a thousand elements and fifty parts that’s fifty million operations against about thirty thousand here, and it needs an $O(nk)$ table. I’ll take the binary search.”*

The follow-ups

“Prove your greedy check is optimal. Why is filling each part as full as possible right?” By an exchange argument, and it depends on contiguity, which is why I’d point at that word in the problem first. Suppose there is some valid split into at most k parts with every part summing to at most L , and suppose my greedy split uses more parts than it. Look at the first place they differ: my greedy’s first boundary is at some index i , and the other split’s first boundary is at some j , with j before i — because greedy pushes its boundary as far right as it possibly can without exceeding L , so it cannot be earlier. Now take the other split and move its first boundary right to i . That is still valid: the first part still sums to at most L , because greedy verified exactly that, and the second part only got smaller. Repeat for each boundary in turn, and I transform any valid split into my greedy one without ever increasing the number of parts. So greedy uses the fewest parts of any valid split, which is exactly what the check needs to be correct. The place this breaks is when the parts do not have to be contiguous — then there is no “first boundary” to exchange, and the problem becomes NP-hard partitioning.

“How do you handle ‘maximise the minimum’ without writing a second, mirrored template?” By negating the question rather than mirroring the loop. In the maximise-the-minimum direction the row of answers runs yes, yes, yes, no, no, and I want the last yes. That is the same as the first no, minus one. So I keep the standard first-true template completely untouched and pass it `lambda g: not can_place(g)`, search the range as half-open with `hi + 1` so that `hi` itself is a candidate, and subtract one from the result. The alternative — mirroring the loop so that a True sets `lo = mid` — is correct but requires a ceiling midpoint, `(lo + hi + 1) // 2`, and forgetting that gives an infinite loop with no error and no output, which is the worst possible failure in an interview because you cannot debug it by reading a traceback. So my rule is one template, never modified, and everything else expressed as a change of question. That is also what makes this family feel like one problem instead of eight.

“The parts don’t have to be contiguous any more. Does your solution still work?” No, and the honest answer is that the problem changed class rather than difficulty. With contiguity removed, the check becomes “can these n numbers be partitioned into k groups each summing to at most L ”, which is bin packing, and that is NP-hard. The binary search skeleton survives — I’d still search the same answer range, and monotonicity still holds because a bigger L can never need more groups — but the feasibility check is no longer exact, so what comes out is an approximation, not the optimum, and I’d say so plainly rather than let the interviewer discover it. What I’d propose depends on the size.

For small n , say up to about twenty, exact by dynamic programming over subsets. For large n , first-fit-decreasing inside the check, which is known to use at most about $11/9$ of the optimal number of bins plus a constant — so I get a bounded approximation and I can quote the bound. The general lesson I'd draw is that in this family the binary search is the easy half; the feasibility check is where the actual difficulty of the problem lives, and the word “contiguous” is doing an enormous amount of work.

A model answer

“The phrase ‘minimise the largest part sum’ is a minimise-the-maximum problem, and the move is to stop asking for the minimum and ask a yes-or-no question instead: given a limit L , can I split this into at most k contiguous parts with every part summing to at most L ?

First, direction. Does a bigger L make the task easier or harder? Easier — more room per part means fewer parts. So the answers run no, no, no, yes, yes, yes, and the answer I want is the first yes.

The range. L can never be below $\max(\text{nums})$, because in the worst case that element sits alone in a part and the part is at least that big. And L never needs to exceed $\text{sum}(\text{nums})$, which is one part holding everything. So the range is $\max$ to sum , and both ends have a reason.

The check is a greedy pass: walk the array, keep adding to the current part, and when the next value would take it past L , close that part and start a new one. That’s optimal because the parts are contiguous — closing a part earlier than forced only leaves more work for the parts after it. I can give an exchange argument if you want it.

```
def split_array_largest_sum(nums: list[int], k: int) → int:
    def parts_needed(limit: int) → int:
        parts, total = 1, 0
        for x in nums:
            if total + x > limit:
                parts, total = parts + 1, 0
            total += x
        return parts

    lo, hi = max(nums), sum(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if parts_needed(mid) ≤ k:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Cost is $O(n \log(\text{sum} - \max))$: one $O(n)$ check, and about thirty of them for realistic values. Space is $O(1)$.

Two things I’d add. There is a dynamic-programming solution at $O(n^2k)$ time and $O(nk)$ space — for a thousand elements and fifty parts that’s fifty million operations and a fifty-thousand-cell table, against thirty thousand operations and three integers here — so I’d choose this one and say why rather than only knowing one of them.

And the mirror image of this problem — ‘place c items so the smallest gap is as large as possible’ — is the same machine with the answers running the other way. I’d handle that by searching for the first gap that *fails* and subtracting one, so I never have to modify the template.”

9 Recall card

- **One question tells you the direction, in one second:** *does more of this make the task easier or harder?* Easier → minimise-the-maximum → **first True**. Harder → maximise-the-minimum → **last True**.
- **Stop asking for the optimum; ask a decision question about a limit.** “Minimise the largest part” becomes “can it be done in $\leq k$ parts with limit L ?” — then it is [day 046](#)’s machine, unchanged.
- **Never mirror the template — negate the question.** Last True = (first False) - 1, with `hi + 1` for the half-open range. The mirrored form needs `(lo + hi + 1) // 2` and hangs silently without it.
- **The greedy check is optimal only because the parts are contiguous** (exchange argument: pushing a boundary as far right as possible never costs a part). Drop “contiguous” and it is NP-hard bin packing — say so.
- **Placement problems must sort first** — $O(n \log n)$, outside the search. The search needs a monotone *question*; the check often needs a sorted *input*, and those are different requirements. Beats DP by $\sim 1,600\times$ on LC 410 at $O(1)$ space.

1 What this is, and why they ask it

Polymorphism means one call site behaving differently depending on which object is at the other end of it. You write `spot.can_fit(vehicle)` once. There are four kinds of spot, each with its own answer, and the line that calls it never learns which kind it has. The caller sends a message; the object decides what the message means.

They ask you to show it rather than define it because the definition is memorised everywhere and understood in about half those places. “Many forms” is not an answer. What the interviewer wants is the before-and-after: a chain of `if type = ...` branches, and the same code with the branches gone, and a clear statement of what that bought — which is that adding a fifth kind of thing means writing one new class and editing nothing. That is also why this question sits in the middle of design rounds rather than in a quiz: every extensibility follow-up you will be asked in the next five days is answered with polymorphism.

2 The story

Sarojini has been headmistress of a school in Vellore for eleven years, and the thing she is proudest of is not a result.

When she took over, there were nine sections and a fire drill that took nineteen minutes. The way it worked was that she walked. Bell at ten forty, and she started at the far end of the corridor and put her head into each room: *you go down the back stairs, wait at the gate; you go through the hall, you're closest; you two switch off the gas taps before anyone moves*. Nine rooms, one at a time, and the little ones at the far end stood there for eleven minutes before anybody told them anything.

Two things went wrong with it, and the second one is why she changed it.

The first was that it was slow, which everyone could see.

The second was that in her third year a new pre-primary section opened in the annexe, and at the drill in August she did her round of nine rooms and went back to her office, and those twenty-two children sat where they were, because nobody had come to tell them. Not one person had done anything wrong. She had a list of rooms in her head and the list was a year old.

So now there is a bell. One long ring, held for five seconds, at the announced time.

And the point of it — the thing she explains to every new teacher — is that the bell does not say anything. It does not say *use the back stairs*. It says *now*, and each room already knows what *now* means for that room. The little ones line up in twos and hold hands and go with their teacher. The middle school walks itself down to the ground without being led. The science room switches off the burners first, which takes forty seconds, and then goes. The office staff take the register box and the keys. The annexe, whenever a new one opens, is told once, on the day it opens, what it does when the bell goes.

Sarojini does not know, on the day of a drill, how many sections there are. She does not need to. She rings the bell.

Four minutes and ten seconds, last February, with fourteen sections.

3 The idea in plain English

The bell is a method call. Each section is a class with its own version of “what to do now”. And the head teacher’s old round of nine rooms is the `if/elif` chain — the thing polymorphism deletes.

The before, and the after

Here is a parking lot deciding whether a vehicle fits a spot, written the way most people write it first:

```
def can_fit(spot, vehicle) → bool:
    if vehicle.kind == "bike":
        return spot.size in ("bike", "compact", "large")
    elif vehicle.kind == "car":
        return spot.size in ("compact", "large")
    elif vehicle.kind == "truck":
        return spot.size == "large"
    raise ValueError(f"unknown vehicle kind {vehicle.kind}")
```

It works. Now add electric vehicles, which fit any spot that has a charger. You edit this function. Then somebody adds pricing, which needs the same branches; then the display board, which needs them again. The rule about what fits where is now in three places, and the fourth developer will add a fifth kind and update two of them.

Here it is polymorphically:

```

class Vehicle:
    def fits(self, spot: "Spot") → bool:
        raise NotImplementedError

class Bike(Vehicle):
    def fits(self, spot): return spot.size in ("bike", "compact", "large")

class Car(Vehicle):
    def fits(self, spot): return spot.size in ("compact", "large")

class Truck(Vehicle):
    def fits(self, spot): return spot.size == "large"

```

and the caller becomes one line with no branches in it at all:

```

if vehicle.fits(spot):
    ...

```

That line is the whole idea. It does not know how many kinds of vehicle exist. Adding electric vehicles is a new class and no edit to any existing one, exactly like Sarojini telling the annexe once, on the day it opens.

The two names, and the one that matters

Textbooks split polymorphism in two:

- **Compile-time polymorphism, or overloading:** several methods with the same name and different argument types, and the compiler picks. Java has it; **Python does not**. In Python a second `def` with the same name simply replaces the first. Say this if asked — it is a common trip-up, and the Python answers are default arguments, `*args`, or `functools.singledispatch`.
- **Runtime polymorphism, or overriding:** a subclass replaces a parent's method, and which one runs is decided by the object at run time. This is the one interviews mean, and it is the one that removes branches.

Duck typing: Python does not need a common parent

In Java, polymorphism requires a shared type — the objects must implement the same interface or extend the same class. In Python they do not:

```

class SlackNotifier:
    def send(self, message: str) → None: ...

class EmailNotifier:
    def send(self, message: str) → None: ...

for notifier in notifiers:          # no common base class anywhere
    notifier.send("build failed")

```

This is **duck typing**: if it has a `send` method that takes a string, it is a notifier. Python looks the method up on the object at the moment of the call and does not care about its ancestry.

That freedom is real and it has one cost: nothing tells you at write time that a class forgot to implement `send`, so you find out when the call happens. Two ways to get the check back without giving up the freedom:

```
from abc import ABC, abstractmethod

class Notifier(ABC):
    @abstractmethod
    def send(self, message: str) → None: ...
```

An **abstract base class** cannot be instantiated if an abstract method is missing — the error arrives at construction rather than at the call. Or:

```
from typing import Protocol

class Notifier(Protocol):
    def send(self, message: str) → None: ...
```

A **Protocol** is a structural type: any class with a matching `send` satisfies it, with no inheritance at all, and a type checker verifies it before the program runs. Protocol is duck typing that a tool can check, and it is the modern Python answer.

Where the branches are allowed to stay

Polymorphism is not the answer to every `if`. The honest rule:

- **Branching on a type that will grow** — vehicle kinds, payment methods, export formats, spot sizes — is what polymorphism is for.
- **Branching on a value or a state that will not grow** — `if amount > 1000, if status == "CANCELLED"` — is just a condition, and turning it into three classes makes the code worse.

The test: *am I going to add a new branch to this `if` when a new kind of thing appears?* If yes, it should be a method. If the branch list is closed and small — the days of the week, the four suits in a pack — a `match` statement in one place is clearer than four classes.

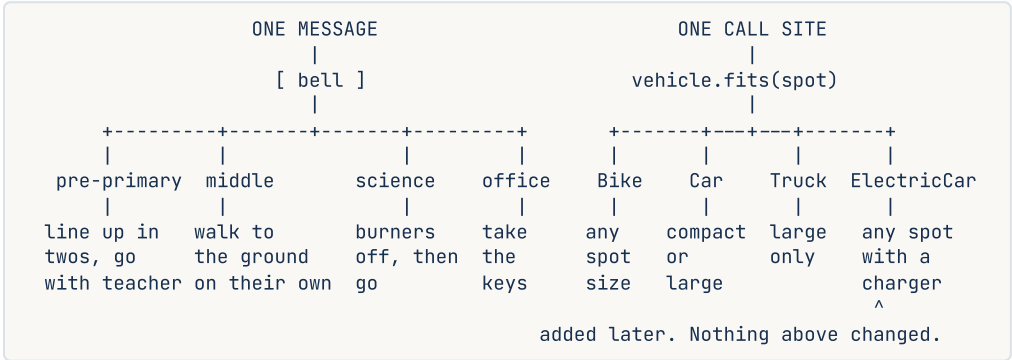
What it costs the reader

There is a genuine downside and interviewers respect you for naming it. With an `if/elif`, all four behaviours are visible in one screen. With polymorphism they are in four files, and answering “what happens for a truck?” means finding the truck class. You

have traded *seeing all cases at once* for *adding a case without touching anything*. That trade is usually right, and it is a trade.

4 The picture

The bell, and the call:



What to notice: the top of both diagrams is one thing. The width at the bottom can grow forever without the top being edited — that is the entire payoff.

The refactor, drawn as what happens when a fifth kind arrives:

WITH BRANCHES	WITH POLYMORPHISM
<pre>can_fit() → edit price_for() → edit display_text() → edit SpotSize enum → edit the report → edit</pre>	<pre>class ElectricCar(Vehicle) → NEW file def fits(self, spot) def price(self, hours) def display_text(self)</pre>
5 edits to WORKING code	register it where vehicles are constructed → 1 line
	1 new file, 1 line changed

What to notice: the units are edits to code that currently works. Each of those five is a chance to break parking for ordinary cars; the one new file cannot break anything, because nothing depends on it yet.

5 How it actually works

The lookup, in Python

When Python evaluates `vehicle.fits(spot)` it looks up `fits` on the object's own dictionary, then on its class, then up the method resolution order, and calls the first one it finds with `vehicle` as the first argument. Nothing is decided in advance. That is why a `Truck` created a millisecond ago runs `Truck.fits` even though the calling code was written before `Truck` existed.

In Java the mechanism is a **virtual method table** — each object points to its class's table of method addresses, and the call is an indirect jump through it. Different machinery, identical consequence, and worth one sentence if the interviewer's language is Java.

The special methods you already use

Python's built-in functions are polymorphism you have been relying on since [day 005](#):

```
len(x)           → x.__len__()
str(x)           → x.__str__()
x + y            → x.__add__(y)
for i in x        → x.__iter__()
x[3]             → x.__getitem__(3)
```

`len` works on a list, a string, a dict, a set and on any class you write that defines `__len__`. The `len` function contains no branches on type at all. That is exactly the shape you are being asked to demonstrate, and pointing at it is a strong answer: *“the standard library's `len` is polymorphic — it calls `__len__` on whatever it is given, and my class joins that set by defining one method.”*

Real designs built on it

- **Python's** `logging.Handler` — `StreamHandler`, `FileHandler`, `RotatingFileHandler`, `SMTPHandler`. `Logger` calls `handler.emit(record)` and knows nothing else. Adding a handler that posts to Slack requires zero changes to the logging module.
- **File-like objects.** `open()`, `io.StringIO`, `gzip.open` and a network socket all support `.read()` and `.write()`, so `json.load(f)` works on all of them. There is no `File` base class enforcing this — it is duck typing, at the scale of an entire ecosystem.
- **Django's storage backends.** `FileSystemStorage` and `S3Storage` both implement `save`, `open` and `url`, so moving uploads to S3 is a settings line.
- **Payment gateways.** `RazorpayGateway`, `StripeGateway`, `PayPalGateway` behind one `charge` method. The checkout flow has no idea which one it has, which is what makes adding a gateway a contained change rather than a release.

- `sorted(items, key=...)` — the comparison behaviour is supplied by the caller. Same idea, arriving as a function instead of an object, and it is the seam you will meet as strategy on [day 058](#).

The refactoring, step by step

Turning a type switch into polymorphism is a mechanical recipe, and being able to run it out loud is worth a lot in a design round.

1. **Find the switch.** Look for `if x.kind ==` or `isinstance` chains. Look for *two* of them on the same set of types — that is the strongest signal, because the duplication is the actual cost.
2. **Name the question the switch is answering.** “Does this fit?” “What does it cost?” That name becomes the method.
3. **Create the base declaration** — an abstract method or a Protocol.
4. **Move each branch body into its own class**, unchanged.
5. **Replace the switch with the call.**
6. **Delete the type tag.** If `vehicle.kind` is no longer read anywhere, remove it. Leaving it means somebody will write a new switch on it next year.

Step six is the one people skip, and it is the one that makes the refactoring stick.

6 The numbers

The cost of adding a kind

```
a system with 3 vehicle types and 4 places that branch on type
    (allocation, pricing, display, the daily report)

add a 4th type, with branches:
    4 files edited, all of them currently working
    each edit is a chance to break the 3 existing types
    the branch somebody forgets fails at run time, in production

add a 4th type, polymorphically:
    1 new class implementing 4 methods
    1 line where vehicles are constructed
    a missing method fails at CONSTRUCTION with an abstract-class error,
    or before running at all if a Protocol and a type checker are used
```

Four edits to working code against one new file. And the failure moves from run time in production to construction time in development, which is the more valuable half of the change.

The branch-count multiplication

```
kinds x places-that-branch = branches to maintain
```

```
3 kinds x 4 places = 12 branches  
5 kinds x 4 places = 20 branches  
5 kinds x 7 places = 35 branches
```

```
polymorphic: 5 classes x 4 methods = 20 methods, but each one lives with its own kind,  
and adding a 6th kind touches 0 of the existing 20.
```

The count is similar; the *coupling* is not. Twenty branches spread over four files must all be edited together. Twenty methods in five files are edited one file at a time.

Runtime cost

```
direct function call           ~40-60 ns  
polymorphic method call (Python) ~60-80 ns  
Java virtual call             ~1-2 ns (often inlined away entirely)  
  
10,000 polymorphic calls in one request: ~0.7 ms  
one database round trip:           ~1 ms
```

The dispatch is free at any scale a web request cares about. If a candidate argues against polymorphism on speed, they are optimising the wrong four orders of magnitude — say so.

When the switch is genuinely cheaper

```
a closed set that will never grow (7 days, 4 suits, 3 HTTP verbs you support):  
  match statement in ONE place : 1 file, ~10 lines  
  a class per case             : 7 files, ~70 lines, and a factory  
  
→ for a closed set, the switch wins on every measure that matters.
```

7 The trade-offs

Seeing all the cases against adding a case cheaply

An `if/elif` shows every behaviour on one screen; polymorphism scatters them across files. *I would not use polymorphism for a set of cases that is closed and small*, because then I am paying the scatter and getting nothing — nobody is ever adding a fifth day to the week. The moment the set can grow, the trade flips, because a growing set means repeated edits to working code.

Polymorphism against a `match` statement

Python 3.10's `match` is genuinely good for closed sets and for dispatching on *shape* rather than on type. *I would not turn three `match` arms into three classes unless the arms are going to multiply* — and I would say the trigger out loud: the second place in the codebase that switches on the same set is the moment to refactor, because that is when the duplication starts costing.

Duck typing against explicit contracts

Duck typing is flexible and gives you no warning when a class forgets a method. *I would not rely on bare duck typing across a module boundary* — inside one file it is fine; across a plugin interface I want a Protocol or an abstract base class, so a missing method is an error at construction or at type check rather than at three in the morning.

Adding behaviour against adding types

This is the trade-off nobody mentions and it is worth knowing. Polymorphism makes adding a **new type** cheap and adding a **new operation** expensive — a new operation means a new method on every existing class. A switch statement is the exact opposite: adding an operation is one new function, adding a type edits everything. *I would choose based on which axis actually grows*. Vehicle kinds grow; the operations on them are fairly stable, so polymorphism wins. If instead the types were fixed and the operations kept multiplying, the honest answer is a visitor-style arrangement or plain functions, and saying that is a strong signal.

The honest sentence

Polymorphism is not about “many forms”. It is a bet that the list of kinds will grow and the list of operations will not. Make that bet where it is true, and adding a kind is a new file; make it where it is false, and every new operation touches every class you own.

8 In the interview

How it gets asked

- “What is *polymorphism*? Show me, don't tell me.” — the direct form, and the answer is a before-and-after with a type switch removed.
- “Refactor this.” — handed a function with an `isinstance` chain in it. Same answer, applied.

- “How would you support a new payment provider?” — polymorphism arriving as a design question, which is how it usually appears in an LLD round.
- “What’s the difference between overloading and overriding?” — the vocabulary check, where the useful extra sentence is that Python has no overloading.

What to say out loud, in the first ninety seconds

1. **Refuse the definition, offer the demonstration.** *“Let me show it with the code rather than define it. Here’s the version with a type switch, and here’s the same thing with the switch gone.”*
2. **Write the switch first.** Four lines of `if vehicle.kind == .` Then say what is wrong with it: *“the rule about what fits where is going to appear in pricing and in the display too, so it’ll be in three places and someone will update two.”*
3. **Write the polymorphic version,** and point at the call site: *“the caller is now one line with no branches, and it doesn’t know how many kinds of vehicle exist.”*
4. **State the payoff in units of edits.** *“Adding electric vehicles is one new class and one line where vehicles are constructed — against four edits to code that currently works.”*
5. **Name the cost, unprompted.** *“What I’ve given up is seeing all four behaviours on one screen. That trade is right when the list of kinds grows, and wrong when it’s closed — I wouldn’t do this for the days of the week.”*

The follow-ups

“Python has no interfaces and no `private`. Does polymorphism even mean anything here?” It means more here, not less, because Python does not require a shared base class at all. If an object has a `send` method taking a string, it works everywhere a notifier is expected — that is duck typing, and the standard library is built on it. File-like objects are the clearest example: `open()`, `io.StringIO`, `gzip.open` and a socket all support `read` and `write`, there is no `File` base class anywhere enforcing that, and `json.load` works with all of them. What Python gives up is the write-time guarantee: nothing tells you a class forgot `send` until the call happens. So I’d say what I actually do about that, because that is the real question. Inside one module, plain duck typing. Across a boundary where other people plug things in, an abstract base class, so that a missing method raises at construction rather than at use — you cannot instantiate an ABC with an unimplemented abstract method. Or, better in modern code, a `typing.Protocol`, which is structural — any class with a matching `send` satisfies it, with no inheritance — and which a type checker verifies before the program runs. Protocol is duck typing that a tool can check, and it is the answer I would give.

“When is an if/else actually better?” When the set of cases is closed and small, and I would give the test rather than a rule. The question is whether I will be adding a branch to this `if` when a new kind of thing appears. Days of the week, the four suits in a pack,

the three HTTP methods a particular endpoint supports — those sets do not grow, and turning them into seven classes plus a factory costs seventy lines to replace ten and gains nothing. Branching on a *value* or a *state* is also usually fine: `if amount > 1000` is a business condition, not a type, and modelling large amounts as a subclass would be strange. The second signal I watch for is duplication rather than size: one switch in one place is a switch, but the *same* switch appearing in a second place is the point where the cost starts, because now the two can disagree. So my trigger for refactoring is not “there are more than three branches”, it is “this is the second function that branches on the same set of types”. And I’d mention the deeper trade-off: polymorphism makes adding types cheap and adding operations expensive, while a switch is the opposite, so the right choice depends on which axis actually grows in this system.

“You’ve replaced four branches with four classes. Isn’t that the same amount of code, just spread out?” Roughly the same number of lines, yes, and a genuinely different coupling, which is what actually costs money. With four branches in four functions — allocation, pricing, display, the report — adding a fifth vehicle type means editing four files that currently work, and all four edits have to land together or the system is inconsistent: a vehicle that can park but cannot be priced. Each of those edits is a chance to break the four existing types, and the one somebody forgets fails at run time in production, with no error at write time. With polymorphism, adding a type is one new file implementing four methods, and it cannot break anything that exists because nothing existing refers to it. If I forget a method, an abstract base class stops me at construction, or a type checker stops me before the program runs at all. So the count of lines is similar; what changed is that the change is *local* and the failure moved from production to development. That is the whole argument, and I’d also concede the other side honestly: I have lost the ability to see all four behaviours on one screen, and if I ever needed to add a *fifth operation* rather than a fifth type, I would now be editing five classes instead of writing one function.

A model answer

“Let me show it rather than define it.

Here’s the version people write first — a parking lot deciding whether a vehicle fits a spot:

```
def can_fit(spot, vehicle) → bool:
    if vehicle.kind == "bike":
        return spot.size in ("bike", "compact", "large")
    elif vehicle.kind == "car":
        return spot.size in ("compact", "large")
    elif vehicle.kind == "truck":
        return spot.size == "large"
```

It works today. The problem is that the same three-way branch is going to appear in pricing, in the display board and in the daily report — so the rule lives in four places, and when someone adds electric vehicles they’ll update three of them.

Polymorphically, each kind owns its own answer:

```
class Bike(Vehicle):
    def fits(self, spot): return spot.size in ("bike", "compact", "large")

class Car(Vehicle):
    def fits(self, spot): return spot.size in ("compact", "large")

class Truck(Vehicle):
    def fits(self, spot): return spot.size == "large"
```

and the caller is one line: `if vehicle.fits(spot):`. That line doesn’t know how many kinds of vehicle exist, and it never has to.

The payoff in units that matter: adding electric vehicles is one new class and one line where vehicles are constructed, against four edits to code that currently works — and each of those four is a chance to break parking for ordinary cars. The failure also moves: if I forget a method, an abstract base class stops me at construction, rather than a missing branch failing in production.

In Python I’d note two things. There’s no overloading — a second `def` with the same name just replaces the first — so when people say ‘compile-time polymorphism’ that’s Java, and the Python answers are default arguments or `functools.singledispatch`. And Python doesn’t need a common base class at all: duck typing means anything with a `fits` method works. I’d still declare a Protocol or an abstract base for anything crossing a module boundary, so a missing method is caught before run time.

And the cost, since there is one: I've given up seeing all four behaviours on one screen. That's the right trade when the list of kinds grows. For a closed set — days of the week, the four suits — I'd keep a single `match` statement, because seven classes and a factory to replace ten lines is worse."

9 Recall card

- **Show it, never define it:** an `if vehicle.kind = ...` chain, then the same thing as `vehicle.fits(spot)` with no branches. The caller never learns how many kinds exist.
- **The payoff in edits:** a new kind is 1 new file + 1 wiring line, against 4 edits to *working* code — and a missing method fails at construction (ABC) or before running (Protocol) instead of in production.
- **Python has no overloading** (a second `def` replaces the first) — only overriding, plus `singledispatch`. And it needs no common base: **duck typing**, made checkable by `typing.Protocol` or enforced at construction by an `ABC`.
- **The refactor is mechanical:** find the switch → name the question it answers → declare the abstract method → move each branch into its class → replace the switch with the call → **delete the type tag**, or someone rebuilds the switch.
- **Keep the `if` when the set is closed** (days, suits) or you are branching on a *value*. The trigger to refactor is the *second* place switching on the same types. Polymorphism makes new **types** cheap and new **operations** expensive — choose by which axis grows.

PRACTICE

Practice — Minimise the maximum: the capacity family

DSA topic: Minimise the maximum: the capacity family **System design topic:** Polymorphism

Code these, in this order

For every problem, **ask the direction question out loud before writing anything**: does more of this quantity make the task easier or harder? Write the answer down. Then the range, the check, and the monotonicity sentence — then the loop, copied unchanged.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Minimized Maximum of Products Distributed to Any Store	LeetCode 2064 (Medium)	The gentlest minimise-the-maximum, with a ceiling-division check.
2	Split Array Largest Sum	LeetCode 410 (Hard)	Whether you reach for binary search instead of dynamic programming.
3	Magnetic Force Between Two Balls	LeetCode 1552 (Medium)	The mirror: maximise the minimum, and the sort you must not forget.
4	Divide Chocolate	LeetCode 1231 (Hard)	The same mirror behind deliberately confusing wording.

On problem 1, get the ceiling right

```
(q + limit - 1) // limit
```

 is the number of stores a quantity of `q` needs at a limit of `limit`. Check it by hand on `q = 11, limit = 3`: four stores, not three. Then confirm `lo = 1` and not 0 by running the check with `limit = 0` and collecting the `ZeroDivisionError: integer division or modulo by zero`.

On problem 2, write the DP first, then throw it away

Spend ten minutes writing the $O(n^2k)$ dynamic-programming solution. Get it right or get it nearly right — it does not matter. Then write the binary search version and time both on a thousand elements with $k = 50$. Say the ratio out loud. Having felt the difference once is what makes you reach for the right tool under pressure, and being able to say “there’s a DP at $O(n^2k)$ and I’m not using it because” is stronger than not knowing it exists.

On problem 3, break it two ways deliberately

First, run it on unsorted input and watch the answer come out wrong with no error. Then run the version that uses the minimise-the-maximum direction — `if can_place(mid): hi = mid` — and watch it return 1 every time. Say which failure each one is: a broken precondition on the check, and a search pointed at the wrong end. They look the same from the outside and they are completely different bugs.

On problem 4, translate the wording first

Before coding, rewrite the problem in your own words as a decision question with a limit in it. It is “can I cut the bar into at least $k + 1$ pieces, every piece summing to at least s ?” — and once that sentence exists the code is problem 3 with a different check. If you cannot produce that sentence, you do not understand the problem yet, and no amount of coding will fix that.

The direction drill

For each, say **easier or harder**, therefore **first True or last True**, in under five seconds:

1. A bigger ship capacity, with a day limit.
2. A bigger required gap between placed items.
3. A faster eating speed, with an hour limit.
4. A bigger allowed part sum, with a part-count limit.
5. A bigger minimum sweetness per piece, with a piece-count target.
6. A bigger budget, maximising items bought.

The translation drill

Turn each optimisation phrase into a `works(x) → bool` sentence, out loud, in one line each:

1. “Minimise the largest part sum over k contiguous parts.”
2. “Maximise the smallest distance between c placed items.”
3. “Minimise the maximum products any of n stores handles.”
4. “Maximise the smallest piece when cutting into $k + 1$ pieces.”
5. “Minimise the number of days to finish, given a fixed daily rate.”

Then say which of the five has a check that needs its input sorted first, and why the others do not.

The greedy-proof drill

Say the exchange argument out loud, in under ninety seconds, for the contiguous-split check: why is filling each part as full as possible optimal? Then answer the harder half — what exactly breaks when the parts no longer have to be contiguous, and what is the honest thing to say to an interviewer at that point?

The hang drill

Write the mirrored maximise template with `mid = (lo + hi) // 2` and run it. Kill it with Ctrl-C and read the traceback. Then fix it with the ceiling midpoint. Then delete both and write the first-False-minus-one version instead, and say why that is the one to keep.

The switch-removal drill

Take this and refactor it out loud in four minutes:

```
def notify(user, message, channel):
    if channel == "email":
        smtp.send(user.email, message)
    elif channel == "sms":
        gateway.post(user.phone, message)
    elif channel == "push":
        fcm.push(user.device_token, message)
    else:
        raise ValueError(channel)
```

1. Run the six-step recipe from the lesson, naming each step as you do it.
2. Name the question the switch is answering — that becomes the method name.
3. Say what step six is and why skipping it means the switch comes back next year.
4. Count the edits to add WhatsApp, both ways.
5. Say the one cost you have accepted by refactoring.
6. Then answer the pushback: *“isn’t that the same amount of code, just spread out?”*

The is-it-polymorphism drill

For each, say whether polymorphism is the right tool, or whether an `if` should stay — with the reason:

1. Four vehicle types, branched on in three separate functions.
2. `if amount > 10000: require_approval()`.
3. Seven days of the week, each with a different opening time.
4. Three payment gateways, with a fourth expected next quarter.
5. `if order.status == "CANCELLED": return`.

6. Two export formats today, and the product manager keeps asking for more.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Split the array into k parts, minimising the largest part sum.* The direction question, the decision restatement, the range with reasons, the greedy check, the exchange argument, and the DP you are choosing not to write, with numbers.
 2. *What is polymorphism? Show me, do not tell me.* Write the switch, say what is wrong with it, write the polymorphic version, count the edits both ways, and name the cost you accepted.
 3. *Maximise the minimum gap. How do you do it without writing a second template?* Negate the question, search the first False with `hi + 1`, subtract one — and say what goes wrong with the mirrored version if the midpoint does not round up.
-

Before you move on

- [] I asked the direction question before writing code on all four problems.
- [] I wrote the DP for LeetCode 410, timed it against the binary search, and can quote the ratio.
- [] I broke the placement problem both ways — unsorted input, and the wrong direction — and can tell the two failures apart.
- [] I made the mirrored template hang, read the traceback, and then chose the version that cannot.
- [] I ran the switch-removal recipe on the notifier, including step six.
- [] I can sort all six cases into “polymorphism” and “leave the `if`” with a reason each.
- [] I answered all three questions above out loud.

Binary search on floats, and the epsilon question

DATA STRUCTURES AND ALGORITHMS

Binary search on floats, and the epsilon question

You can binary search a continuous range and decide when to stop.

SYSTEM DESIGN

Abstraction and interfaces

You can design to an interface so a swap of implementation touches one line.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the square root of a number to six decimal places, without the library call.*
- *How would you make it easy to swap the payment provider later?*

Binary search on floats, and the epsilon question

1 What this is, and why they ask it

Every binary search so far has run over whole numbers, and it stopped because the range eventually held nothing. On a **continuous** range — real numbers rather than integers — that never happens. You can always halve again, forever, and the range never becomes empty. So the halving is unchanged and **the stopping rule is a new decision you have to make and defend.**

They ask it because it is the shortest problem where “correct” stops being a yes-or-no property. A candidate who writes `while lo < hi` on floats produces a program that hangs; a candidate who writes `while hi - lo > 1e-15` produces one that sometimes hangs, depending on the input; and a candidate who says “I’ll run a hundred iterations, because a hundred halvings takes any starting range below what a double can even represent” has answered the actual question. Square root without `math.sqrt` is the standard vehicle — LeetCode 69 in its integer form, and the six-decimal form as the follow-up — and the same stopping decision shows up in every optimisation problem with a real-valued answer.

2 The story

Anitha’s son is fourteen months old and he has his bath at about half past four, before it gets cool.

The bucket is a red plastic one that lives in the corner of the bathroom. There is a hot tap, and there is a drum of water in the other corner that has been standing since morning and is cold.

She starts by knowing two things without testing anything at all. Water straight from the hot tap is too hot for him — she has never had to check that. And water straight from the drum is too cold. So whatever she wants is somewhere between the two, and she does not have to consider anything outside that.

She half fills the bucket from the hot tap and pours in a good mug of cold, then puts her elbow in. Still too warm. So she adds cold again — but only about half as much as last time, because the last pour clearly overshot in the right direction and a full mug

would take her past. Elbow in. Now it is slightly cool. So a small splash back from the hot tap, smaller again than the last correction. Elbow.

Each correction is about half the size of the one before it. That is not something she thinks about. It is just that after the first pour she knows which side she is on, and the amount left to fix is smaller than it was.

The interesting part is where she stops, and it is not where you would guess.

She does not stop when the water is exactly right, because there is no such moment. Her elbow cannot tell one small difference from another. What happens is that after four or five corrections she puts her elbow in and simply cannot tell any more whether it is a shade warm or a shade cool. That is the signal. Not *it is correct* — *I can no longer tell the difference, and neither will he*.

Her mother-in-law, who has done this for four children and eleven grandchildren, does it in four pours. Always four. She does not test at the end. She has done it so many times that she knows four corrections from those two starting points is past the point where anybody's elbow could complain, and she picks the baby up and gets on with it.

3 The idea in plain English

The hot tap and the cold drum are `lo` and `hi`. Each correction is a halving. And the difference between Anitha's elbow-test and her mother-in-law's four pours is exactly the two stopping rules you have to choose between.

What changes, and what does not

The halving is identical to every day this week:

```
lo, hi = 0.0, x
for _ in range(100):
    mid = (lo + hi) / 2          # NOT // -- this is real division now
    if mid * mid < x:
        lo = mid
    else:
        hi = mid
```

Two small differences from the integer version and one large one.

- `/` instead of `//`. Integer division would floor every midpoint and the search would stall.
- No `+ 1` or `- 1` anywhere. Between any two distinct real numbers there is another one, so there is no “next” value to step past. `lo = mid` and `hi = mid` are correct here and would hang on integers.

- And the large one: **the loop condition is no longer about the range being empty**. It cannot become empty.

Why `while lo < hi` hangs

```
lo, hi = 0.0, 2.0
while lo < hi:                # ← never becomes false
    mid = (lo + hi) / 2
    ...
```

Halving a positive gap gives a smaller positive gap. In exact arithmetic this runs forever. In floating point it runs until `lo` and `hi` become adjacent representable numbers and `mid` equals one of them — which happens after about fifty iterations for values near 1, and after a different number for values near 10^{15} . So it either hangs or terminates at an unpredictable point, and neither is something you want to explain in an interview.

Stopping rule one: an epsilon

Epsilon, written `eps`, is the size of the gap you are willing to accept:

```
while hi - lo > 1e-9:
    ...
```

Read it as “stop when the answer is pinned down to within a billionth.” It is Anitha’s elbow.

The problem asks for six decimal places, so `1e-7` would do; `1e-9` gives two spare digits and costs about seven extra iterations, which is nothing.

Two ways to get this wrong, and both are worth knowing:

- **Too large.** `1e-4` and the sixth decimal place is unreliable.
- **Too small.** `1e-18` is below what a 64-bit double can distinguish for numbers around 1, so `hi - lo` never gets there and the loop never ends. This is the failure that hangs.

The absolute-versus-relative trap

An absolute epsilon means different things at different magnitudes:

answer near 1:	<code>1e-9</code> is a billionth of the answer. Excellent.
answer near 10^{15} :	<code>1e-9</code> is smaller than the gap between neighbouring representable doubles at that size. Unreachable → hangs.

A double holds about 15-17 significant decimal digits, so near 10^{15} the smallest distinguishable step is about 0.125. Demanding `hi - lo > 1e-9` there is asking for something the number system cannot express.

The fix is a **relative epsilon** — stop when the gap is small *compared to the value*:

```
while hi - lo > 1e-9 * max(1.0, abs(lo)):
    ...
```

The `max(1.0, ...)` keeps it sane when the answer is near zero, where a relative test would demand infinite precision.

Stopping rule two: a fixed number of iterations

This is the mother-in-law, and it is what to write in an interview:

```
for _ in range(100):
    ...
```

No condition, no epsilon, no hang. Every iteration halves the range, so after `k` iterations the starting range has shrunk by a factor of `2^k`:

```
100 iterations: range / 2^100 = range / 1.27e30
```

Starting from a range of 10^{18} , a hundred halvings leaves an interval of about 10^{-12} , which is far below what a double can distinguish at any magnitude. **The loop cannot fail to converge and cannot hang, on any input, ever.**

The cost is that it always does a hundred iterations even when ten would do. A hundred iterations of a two-line body is a few microseconds. Say the trade out loud: *“I’d rather pay a fixed few microseconds than debug a stopping condition.”*

In practice 50 to 100 is the range people use. Fifty halvings is a factor of 10^{15} , enough for almost anything; a hundred is enough for everything.

Prefer integers when you can

Before reaching for floats at all, ask whether the problem really needs them. LeetCode 69 asks for the integer square root — the floor of the true root — and the whole thing can be done in integers:

```
lo, hi = 0, x
while lo < hi:
    mid = (lo + hi + 1) // 2      # ceiling: this is "last True"
    if mid * mid ≤ x:
        lo = mid
    else:
        hi = mid - 1
return lo
```

Python's integers are exact and unbounded, so `mid * mid` is exact however large `x` is. The float version of the same problem can return 2 for `x = 4` in some languages and 1.9999999999 in others. **If the answer is an integer, search the integers.** It is one of the most reliable pieces of advice in this subject.

The one bound everyone gets wrong

For square root, the tempting range is `[0, x]`. It is wrong for `x < 1`:

```
x = 0.25 → sqrt(0.25) = 0.5, which is BIGGER than x
```

So the upper bound must be `max(1.0, x)`. This is the same lesson as [day 046](#)'s "if `works(hi)` is false, your range is wrong", arriving in a new costume — and it is the input an interviewer will hand you.

4 The picture

The two starting facts, and the halving:

```
lo = 0.0                                     hi = max(1.0, x) = 10.0
|-----|
|               searching for sqrt(10) = 3.16227766...               |
|-----|

pass 1  mid = 5.0      5.0^2 = 25.0  > 10  →  hi = 5.0
|-----|
pass 2  mid = 2.5      2.5^2 = 6.25  < 10  →  lo = 2.5
|-----|
pass 3  mid = 3.75     14.06        > 10  →  hi = 3.75
|-----|
pass 4  mid = 3.125    9.77         < 10  →  lo = 3.125
|-----|
pass 5  mid = 3.4375   11.82        > 10  →  hi = 3.4375
|-----|
...
pass 30                                     lo ~ hi ~ 3.16227766017
```

What to notice: `lo = mid` and `hi = mid`, with no `+ 1` or `- 1` anywhere. On integers that would hang; on reals it is the only correct move, because there is no "next" real number to step to.

Where the two stopping rules stop:

gap after k halvings, starting from a range of 10:

k = 10	gap ~ 0.0098	-- 2 decimal places
k = 20	gap ~ 9.5e-6	-- 5 decimal places
k = 30	gap ~ 9.3e-9	-- 8 decimal places ← an eps of 1e-9 stops near here
k = 50	gap ~ 8.9e-15	-- at the edge of double precision for values near 10
k = 100	gap ~ 7.9e-30	-- far below anything representable. Cannot fail.

asking for eps = 1e-18 with an answer near 10¹⁵:
smallest representable step at 10¹⁵ is ~0.125
hi - lo can NEVER get below 1e-18 → infinite loop

What to notice: the fixed count has no input it can fail on. The epsilon has one, and it is the input where the answer is large.

5 The code, built step by step

The bounds

```
lo, hi = 0.0, max(1.0, x)
```

0.0 because a square root is never negative. `max(1.0, x)` because for `x` between 0 and 1 the root is bigger than `x` itself, and an upper bound that excludes the answer makes the whole search meaningless.

The loop, with a fixed count

```
for _ in range(100):  
    mid = (lo + hi) / 2  
    if mid * mid < x:  
        lo = mid  
    else:  
        hi = mid
```

Two branches, no equality test. Note there is no `if mid * mid == x: return mid` — comparing floats for equality is very nearly never right, and the loop converges on the exact value anyway.

Returning the answer

```
return (lo + hi) / 2
```

The midpoint of the final interval, not `lo` or `hi`. It halves the worst-case error at no cost, and it is one of those small things an interviewer notices.

The complete solution

```
def sqrt_float(x: float, iterations: int = 100) → float:
    """Square root of  $x \geq 0$ , to full double precision, without math.sqrt.

    A fixed iteration count rather than an epsilon: 100 halvings shrink any starting
    range by a factor of  $2^{100}$ , which is below what a double can represent at any
    magnitude. It cannot hang, on any input.
    """
    if x < 0:
        raise ValueError("no real square root of a negative number")
    if x == 0:
        return 0.0

    lo, hi = 0.0, max(1.0, x)          # max(1, x): for  $x < 1$  the root EXCEEDS x
    for _ in range(iterations):
        mid = (lo + hi) / 2
        if mid * mid < x:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2                # the midpoint halves the worst-case error

def sqrt_epsilon(x: float, eps: float = 1e-9) → float:
    """The same search with a RELATIVE epsilon.

    The max(1.0, lo) is what stops it hanging when the answer is large: an absolute
    1e-9 is smaller than the gap between representable doubles above about  $10^7$ .
    """
    if x < 0:
        raise ValueError("no real square root of a negative number")
    lo, hi = 0.0, max(1.0, x)
    while hi - lo > eps * max(1.0, lo):
        mid = (lo + hi) / 2
        if mid * mid < x:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2

def isqrt_int(x: int) → int:
    """LeetCode 69. The FLOOR of the square root, computed entirely in integers.

    'Last True' form: the largest m with  $m*m \leq x$ . Note the ceiling midpoint --
    without the + 1 this hangs when  $hi = lo + 1$ .
    """
    if x < 2:
        return x
    lo, hi = 1, x // 2                  # for  $x \geq 2$  the root is at most  $x // 2$ 
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if mid * mid ≤ x:
            lo = mid
        else:
            hi = mid - 1
    return lo

def nth_root(x: float, n: int, iterations: int = 100) → float:
    """The n-th root of  $x \geq 0$ . Exactly the same search with a different question."""
    lo, hi = 0.0, max(1.0, x)
    for _ in range(iterations):
        mid = (lo + hi) / 2
        if mid ** n < x:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2

if __name__ == "__main__":
    print(f"{sqrt_float(10):.10f}")      # 3.1622776602
```

```

print(f"{sqrt_float(2):.10f}")      # 1.4142135624
print(f"{sqrt_float(0.25):.10f}")  # 0.5000000000 ← the x < 1 case
print(f"{sqrt_float(1e18):.4f}")   # 1000000000.0000
print(sqrt_float(0), sqrt_float(1)) # 0.0 1.0

print(f"{sqrt_epsilon(10):.10f}")  # 3.1622776610 ← eps stopped early: 9 digits, not 16
print(f"{sqrt_epsilon(1e18):.4f}") # 999999999.9073 ← relative eps survives, but loosely

print(isqrt_int(8), isqrt_int(9), isqrt_int(10)) # 2 3 3
print(isqrt_int(0), isqrt_int(1), isqrt_int(2))  # 0 1 1
print(isqrt_int(10**18))                        # 1000000000

print(f"{nth_root(27, 3):.6f}")      # 3.000000
print(f"{nth_root(2, 10):.6f}")     # 1.071773

```

Run it. Three lines are worth watching. `sqrt_float(0.25)` is wrong on any version that uses `hi = x`. `sqrt_epsilon(1e18)` hangs on any version with an *absolute* epsilon and merely loses precision with a relative one. And compare the two `sqrt_epsilon` outputs against `sqrt_float` — the epsilon version stops as soon as its tolerance is met, so it gives about nine good digits where the fixed count gives every digit a double has. That is the second reason to prefer the fixed count: it is not only safer, it is more accurate.

The comparison you should never write

```

if mid * mid == x:      # ← do not
    return mid

```

`0.1 + 0.2 == 0.3` is `False` in Python, and for exactly the same reason `mid * mid` almost never lands on `x` exactly. The test either never fires — costing nothing but noise — or fires accidentally on a value that happens to round right. If you genuinely need to compare floats, compare with a tolerance:

```

from math import isclose
if isclose(mid * mid, x, rel_tol=1e-12):
    ...

```

6 What it costs

Time, with a fixed count

```

100 iterations, each doing one division, one multiplication and one comparison
→ 100 × O(1) = O(1) time, regardless of x

```

It is genuinely constant time, and saying so is worth a moment: the input's *size* does not change the work, because the range is halved by a fixed factor each pass and the count is fixed.

Time, with an epsilon

```
iterations = log2( (hi - lo) / eps )

range = 10,      eps = 1e-9   → log2(10 / 1e-9) = log2(1e10) ~ 34 iterations
range = 10^18,  eps = 1e-9   → log2(1e27)      ~ 90 iterations
range = 10,      eps = 1e-18  →              NEVER (unreachable)
```

So the epsilon version is $O(\log((hi - lo) / \epsilon))$ and the fixed version is $O(1)$ with a larger constant. In practice:

```
34 iterations x ~3 float operations ~ 100 operations ~ 0.1 microseconds
100 iterations x ~3                  ~ 300 operations ~ 0.3 microseconds
```

The difference between a smart stopping rule and a dumb one is two tenths of a microsecond. That is the whole argument for the fixed count.

Space

```
lo, hi, mid: three floats → O(1)
```

Precision, stated properly

```
a 64-bit double holds ~15-17 significant decimal digits
machine epsilon (the smallest e with 1 + e ≠ 1) ~ 2.22e-16

so near 1.0      the smallest distinguishable step is ~2.2e-16
   near 1e6                      ~1.2e-10
   near 1e15                     ~0.125
   near 1e18                     ~128
```

That last line is why an absolute epsilon of $1e-9$ is unreachable for large answers, and it is the number to have ready if an interviewer pushes on precision.

7 The traps

The real error: `while lo < hi` on floats

```
lo, hi = 0.0, 10.0
count = 0
while lo < hi:
    mid = (lo + hi) / 2
    count += 1
    if mid * mid < 10:
        lo = mid
    else:
        hi = mid
print(count)
```

```
^C
Traceback (most recent call last):
  File "day48.py", line 3, in <module>
    while lo < hi:
KeyboardInterrupt
```

Nothing printed, no error, one core at 100%. Halving a positive gap gives a positive gap; the condition is never false. The `+ 1` and `- 1` that made the integer version terminate do not exist on reals, so the termination has to come from somewhere else — the count, or the epsilon.

The near-miss: an absolute epsilon with a large answer

```
def sqrt_absolute(x, eps=1e-9):
    lo, hi = 0.0, max(1.0, x)
    while hi - lo > eps:          # ← absolute
        mid = (lo + hi) / 2
        if mid * mid < x:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2

print(sqrt_absolute(10))          # 3.1622776601683795 -- fine
print(sqrt_absolute(1e18))       # hangs
```

It works on every small test and hangs on the large one. Above about 10^7 the gap between neighbouring doubles exceeds `1e-9`, so `hi - lo` reaches a floor it can never go below. The fix is `eps * max(1.0, lo)` — a relative tolerance — or, better, the fixed count.

The near-miss: `hi = x` when `x < 1`

```
def sqrt_bad_bound(x):
    lo, hi = 0.0, x                # ← should be max(1.0, x)
    for _ in range(100):
        mid = (lo + hi) / 2
        if mid * mid < x:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2

print(sqrt_bad_bound(0.25))        # 0.25    should be 0.5
```

```
0.25
```

`sqrt(0.25)` is 0.5, which is outside `[0, 0.25]`. Every candidate in the range answers “too small”, `lo` climbs to the top of the range, and the function returns the top with total confidence. Same failure as a wrong upper bound on [day 046](#), and it is the input the interviewer will hand you after your first three tests pass.

The near-miss: float equality

```
print(0.1 + 0.2 == 0.3)
print(0.1 + 0.2)
```

```
False
0.30000000000000004
```

`0.1` and `0.2` cannot be written exactly in binary, so their sum is not exactly `0.3`. Any binary search that tries `if mid * mid == x: return mid` is relying on an event that almost never happens. Delete the branch; the loop converges anyway.

The trap: `//` left in by habit

```
mid = (lo + hi) // 2                # ← integer division on floats
```

This does not raise. `(0.0 + 10.0) // 2` is `5.0`, and `(3.0 + 4.0) // 2` is `3.0` — it floors. The search stalls at whole numbers and returns an integer-valued answer for a problem asking for six decimal places. It is a silent single-character bug, and the symptom is an answer that is right to zero decimal places.

The trap that is not really a trap: `mid` computed as `(lo + hi) / 2`

For integers this could overflow in a fixed-width language, and the safe form was `lo + (hi - lo) / 2`. For doubles, `(lo + hi) / 2` can overflow only if both are near the maximum double, around 10^{308} , which does not happen in these problems. Use whichever you like, and mention you know the difference.

8 In the interview

How it gets asked

- “Implement square root without the library function.” — and then, immediately, “to six decimal places”, which is where the interesting half starts.
- “Return the integer square root.” — LeetCode 69, and the right first sentence is “I’ll do this in integers”.
- “When does your loop stop, and why is that the right condition?” — the real question, asked directly.
- As a **component**: any binary-search-on-the-answer problem where the answer is a real number — minimising the maximum distance to a petrol pump, maximising the average of a subarray — brings this stopping decision with it.

What to say out loud, in the first ninety seconds

1. **Ask what kind of answer is wanted.** “Do you want the integer square root, or a real number to a given precision? Those are different problems and I’d write them differently.”
2. **If integer, say so and stay in integers.** “Then I’ll search integers and keep the exactness — `mid * mid ≤ x` with Python’s unbounded ints is exact at any size, and I avoid float rounding entirely.”
3. **State the bounds with the awkward case named.** “Zero to $\max(1, x)$ — not zero to x , because for x below 1 the root is bigger than x and the answer would be outside my range.”
4. **Say what changes on floats.** “No plus-one or minus-one, because between any two reals there’s another one. `lo = mid` and `hi = mid` are correct here, and would hang on integers.”
5. **Announce the stopping rule and defend it.** “I’ll run a fixed hundred iterations rather than test an epsilon. A hundred halvings shrink any starting range by a factor of 2^{100} , which is far below what a double can distinguish at any magnitude — so it can’t hang on any input. An epsilon can, if the answer is large.”

The follow-ups

“Why a fixed iteration count rather than an epsilon? Isn’t that wasteful?” It is a little wasteful and it is a trade I would make every time. The epsilon version stops when `hi - lo` falls below some tolerance, and the failure mode is that on some inputs it never does. A 64-bit double holds about fifteen to seventeen significant digits, so near 10^{15} the smallest gap between two representable numbers is roughly 0.125 — asking for `hi - lo` below $1e-9$ there is asking for a number the format cannot express, and the loop spins forever. That bug passes every small test and fails on the large input, which is the worst kind. A fixed hundred iterations shrinks any starting range by 2^{100} , about 10^{30} , so it converges to full double precision from any starting range and cannot hang on any input at all. The cost is a hundred iterations of a three-operation body — about 0.3 microseconds — where a smart epsilon might use thirty-four. I would rather pay two tenths of a microsecond than own a stopping condition that has an input class it fails on. If I did want the epsilon, I’d make it relative — `eps * max(1.0, lo)` — because that scales with the magnitude of the answer and removes the failure.

“Do it without floating point at all.” For the integer square root that is straightforwardly better, and it is what I would reach for first. The question becomes “the largest m with `m*m ≤ x`”, which is a last-True boundary search over integers, and in Python `m * m` is exact for any size of integer, so there is no rounding to reason about. The one detail is the ceiling midpoint — `(lo + hi + 1) // 2` — because this is the maximise form, and with a floor midpoint it hangs when `hi` is `lo + 1`. For a *fractional* answer without floats, the honest route is fixed-point: decide the precision up front, multiply by 10^6 , and search for the largest integer `m` with `m*m ≤ x * 1012`. That gives six exact decimal places with exact integer arithmetic and no epsilon question at all. It is more code and it is the right answer when the result feeds a financial calculation, where accumulated float error is a real problem — the same reason money is stored as `NUMERIC` rather than `FLOAT`, which was [day 026](#)’s rule.

“What if the function isn’t `mid * mid` — what if it’s something expensive?” Then the cost model changes and the stopping rule becomes a real decision rather than a free one. The search is `O(iterations × cost of one evaluation)`, so if evaluating the predicate means a pass over an array, a hundred iterations is a hundred passes. In that case I would tune the count rather than leave it at a hundred: work out what precision the problem actually needs, take the log base two of the starting range divided by that precision, and use that number with a small margin. For a “minimise the maximum distance between petrol pumps” problem where the answer needs three decimal places and the range is a thousand, that is $\log_2(10^6)$, about twenty iterations, not a hundred — a five-fold saving on real work. I would also say what I would *not* do, which is switch to an epsilon loop to save iterations, because that reintroduces the hang. And if the predicate is smooth and I need many digits, Newton’s method converges quadratically — the number

of correct digits doubles each step, so about five steps instead of fifty — but it needs a derivative and a good starting guess, and it is the wrong answer to a question about binary search unless the interviewer asks for the fastest possible method.

A model answer

“First a clarifying question: do you want the integer square root, or a real number to a given precision? They’re different problems. If it’s the integer floor, I’d do it entirely in integers and avoid floats altogether — Python’s integers are exact at any size, so `mid * mid ≤ x` never lies. Let me do the six-decimal version, since that’s the harder one.

The bounds are zero to `max(1, x)`. Not zero to `x` — for `x` below 1, like 0.25, the root is 0.5, which is bigger than `x`, so a range of `[0, x]` wouldn’t contain the answer and I’d return the top of the range with total confidence.

The halving is the same as any binary search, with one change: no plus-one or minus-one. Between any two reals there’s another one, so there’s no ‘next value’ to step past — `lo = mid` and `hi = mid` are correct on floats and would hang on integers.

Then the interesting decision, which is when to stop. `while lo < hi` never terminates, because halving a positive gap gives a positive gap. So I’ll use a fixed iteration count:

```
def sqrt_float(x: float) → float:
    if x < 0:
        raise ValueError("no real square root of a negative number")
    lo, hi = 0.0, max(1.0, x)
    for _ in range(100):
        mid = (lo + hi) / 2
        if mid * mid < x:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2
```

A hundred halvings shrink any starting range by 2^{100} — about 10^{30} — so it converges to full double precision whatever the input, and it cannot hang. The alternative is an epsilon, and I’d avoid the absolute version specifically: a double has about fifteen significant digits, so near 10^{15} the gap between representable numbers is roughly 0.125, and `while hi - lo > 1e-9` there spins forever. If I wanted an epsilon I’d make it relative — `eps * max(1.0, lo)`.

I return the midpoint of the final interval rather than `lo`, which halves the worst-case error for free. And I’ve deliberately not written `if mid * mid == x: return mid` — float equality almost never fires, and the loop converges without it.

Cost is $O(1)$ — a hundred iterations regardless of input — and $O(1)$ space. If the predicate were expensive rather than a multiplication, I'd size the count from the precision I actually need instead of leaving it at a hundred."

9 Recall card

- **On reals the range never empties**, so `while lo < hi` hangs and there is no `± 1` — `lo = mid` and `hi = mid` are correct. The stopping rule is a decision you must make and defend.
- **Prefer a fixed count**: `for _ in range(100)`. A hundred halvings shrink any range by 2^{100} , below double precision at every magnitude, so it cannot hang on any input. $O(1)$ time, $\sim 0.3 \mu\text{s}$.
- **An absolute epsilon has an input class that hangs**. A double holds ~ 15 - 17 digits, so near 10^{15} the smallest step is ~ 0.125 and `hi - lo > 1e-9` is unreachable. Use `eps * max(1.0, lo)` if you must use one.
- `hi = max(1.0, x)`, **never** `hi = x` — for `x < 1` the root exceeds `x`, and a range missing the answer returns its own edge, confidently. Return `(lo + hi) / 2`, and never test floats with `=`.
- **If the answer is an integer, search integers** — exact, no epsilon question, and remember the ceiling midpoint `(lo + hi + 1) // 2` for the last-True form. Fixed-point ($\times 10^6$) gives exact decimals without floats.

1 What this is, and why they ask it

An **interface** is a list of operations with no implementation behind them — the shape of a capability, written down. **Abstraction** is the act of deciding what that list should be: naming *what* something does and deliberately saying nothing about *how*. A `PaymentGateway` interface says “you can charge and you can refund”; it does not say Razorpay, it does not say HTTP, and it does not say what a card token looks like.

They ask the payment-provider question because it is the cleanest test of whether a candidate can draw a boundary. Everybody knows the words. The differences show up in three specific places: whether the interface is written in *your* vocabulary or the vendor’s, whether the concrete class is chosen inside the code that uses it or handed in from outside, and whether the candidate can say honestly when *not* to do this. It also arrives in real work constantly — swapping a payment provider, moving uploads from disk to S3, replacing one search engine with another — and the difference between a one-line change and a two-week change is a decision somebody made a year earlier.

2 The story

Chandran has run a tea shop near the college gate in Thrissur for twenty-two years. Two stoves going from six in the morning, and he gets through about two gas cylinders a week.

In the monsoon two years ago his usual supplier went to pieces. Deliveries that were promised on Tuesday came on Friday, and twice they did not come at all, and on one of those days he shut at eleven in the morning with forty people still coming.

He changed suppliers on the Saturday. It took him one phone call and about ten minutes when the new man arrived.

Ten minutes, because the neck of a cylinder is the same whoever fills it. The regulator that sits on top is the same fitting, the rubber tube from the regulator to his stove is the same, and the stove does not know or care what is written on the side of the cylinder. He unscrewed one, screwed on the other, checked for a smell, and lit the front burner. Twenty-two years and he has changed supplier four times, and it has never once meant touching the stove.

The shop two doors down is a smarter place, glass front, and they put in an imported cooking range about four years ago. Beautiful machine. It came with its own fitting, its own tube, and a recommendation to use one particular supplier's cylinders, and the man who installed it plumbed it in that way because that is what the box said.

They cannot change supplier. Not easily. The last time they tried, they were quoted a figure for re-fitting the whole line that was more than a month's gas, so they paid the higher rate instead and they are still paying it. The owner told Chandran, quite ruefully, that the range was the best thing he ever bought and the fitting was the worst decision he ever made, and they were the same purchase.

There is one thing Chandran gives up, and he knows about it. The big commercial cylinders that hotels use run at a different setting and would burn hotter than his stoves ever do. His fitting cannot carry that. He decided a long time ago that hotter is not what a tea shop needs — but it is a real thing he cannot have, and it is the price of the fitting that lets him change his mind about everything else in ten minutes.

3 The idea in plain English

The regulator fitting is the interface. Chandran's stove is your application code. The suppliers are the implementations. The imported range is what happens when code depends on a concrete vendor. And the commercial cylinder he cannot use is the honest cost of a standard fitting, which §7 makes explicit.

The shape of it

Three pieces, always:

```
from typing import Protocol

class PaymentGateway(Protocol):
    # 1. the fitting: what, not how
    def charge(self, amount: Money, token: str) → ChargeResult: ...
    def refund(self, charge_id: str, amount: Money) → RefundResult: ...
```

```
class RazorpayGateway:
    # 2. one supplier
    def charge(self, amount: Money, token: str) → ChargeResult:
    ...
    # HTTP, retries, their error codes

class StripeGateway:
    # another supplier
    def charge(self, amount: Money, token: str) → ChargeResult:
    ...
```

```
class Checkout:
    def __init__(self, gateway: PaymentGateway) → None:
        self._gateway = gateway
        # 3. the stove: knows only the fitting
        # handed in, not chosen here

    def pay(self, order: Order, token: str) → None:
        result = self._gateway.charge(order.total, token)
        ...
```

`Checkout` contains the word “Razorpay” exactly zero times. That is the property to check, and it is checkable with `grep`.

The part everyone gets wrong: who chooses

Here is the same code with one thing changed, and it undoes everything:

```
class Checkout:
    def __init__(self) → None:
        self._gateway = RazorpayGateway()
        # ← chosen INSIDE
```

Now `Checkout` names a concrete class, so `Checkout` depends on Razorpay, so swapping the provider means editing `Checkout` — and every test of `Checkout` makes a real network call. The interface bought nothing.

Passing the dependency in from outside is called **dependency injection**, and stripped of the ceremony it means: *the thing that uses a capability does not choose which implementation it gets; whoever builds it does.*

```
# one place in the whole program, at start-up
gateway = StripeGateway(api_key=settings.STRIPE_KEY)
checkout = Checkout(gateway)
# ← the one line that changes
```

That is the answer to the interviewer’s question, and it is one line.

There is a deeper name for the shape, which you will meet properly in the SOLID phase from [day 055](#): **depend on abstractions, not on concretions**. Say the plain version instead — *the code that uses a thing should name the capability, not the vendor.*

The leaky abstraction

This is where good answers separate from great ones. An interface can be present in form and useless in substance:

```
class PaymentGateway(Protocol):
    def charge(self, intent: stripe.PaymentIntent) → stripe.Charge: ...
```

The fitting is bolted to one supplier’s shape. Any other implementation has to construct a Stripe object to satisfy it, and `Checkout` now imports `stripe` to build the argument. The interface exists and the coupling is unchanged.

The rule: the interface speaks your vocabulary, not the vendor's. Define your own small types — `Money`, `ChargeResult`, `PaymentError` — and let each implementation translate at its own edge. The translation is real work and it is exactly the work you are paying for.

The leak has three usual forms, and all three are worth being able to name:

- **Types.** Vendor classes in the signature, as above.
- **Errors.** `except stripe.CardError` appearing in your business logic. Each implementation must catch its own exceptions and raise yours.
- **Semantics.** A method called `capture()` because Stripe has authorise-then-capture, when your application only ever charges once. You have modelled their workflow, not yours.

Interfaces in Python: three ways

```
# 1. duck typing -- nothing declared at all
def pay(gateway, amount): gateway.charge(amount)
```

Works. Nothing checks that `gateway` has `charge` until the call runs.

```
# 2. abstract base class -- inheritance, checked at construction
from abc import ABC, abstractmethod
class PaymentGateway(ABC):
    @abstractmethod
    def charge(self, amount: Money, token: str) → ChargeResult: ...
```

A subclass missing `charge` cannot be instantiated: `TypeError: Can't instantiate abstract class`. Good when you also want to share a little default behaviour, and when you want the failure to be loud.

```
# 3. Protocol -- structural, checked by a type checker, no inheritance
from typing import Protocol
class PaymentGateway(Protocol):
    def charge(self, amount: Money, token: str) → ChargeResult: ...
```

Any class with a matching `charge` satisfies it, with no import and no base class. `mypy` verifies it before the program runs. This is the modern default, and it has one property that matters for third-party code: **you can write a Protocol for a class you do not own**, which an abstract base class cannot do.

Keep the interface small

A `PaymentGateway` with fourteen methods forces every implementation to stub the nine it does not support, and each stub is a lie waiting to be called. Two or three methods per interface is normal; if half the implementers raise `NotImplementedError` on half the methods, the interface is two interfaces. That instinct has a name too — interface segregation — and it arrives on [day 059](#).

The test that stops you over-doing it

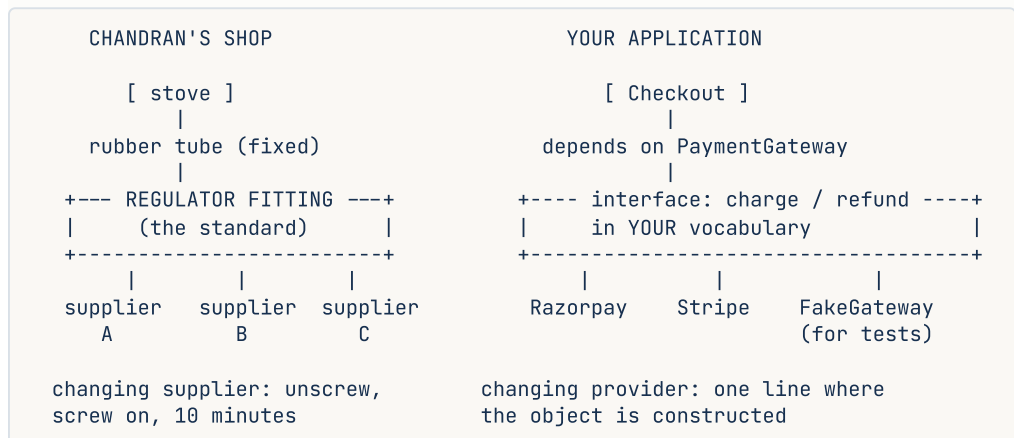
Before writing an interface, ask: **can I name the second implementation?**

<code>PaymentGateway</code>	→ Razorpay and Stripe, both real. Interface.
<code>FileStorage</code>	→ local disk and S3, both real. Interface.
<code>OrderRepository</code>	→ Postgres and... an in-memory fake for tests. Counts, and it is the commonest legitimate "second implementation".
<code>InvoiceNumberer</code>	→ there is one way to number an invoice. No interface.

An in-memory fake for tests is a genuine second implementation, and it is the one people forget. If your only justification is “we might swap it one day”, that is speculation, and [day 043](#)’s warning applies: an interface with one implementation is a layer with no pay-off.

4 The picture

The fitting, and the code:



What to notice: the fitting is in the middle and belongs to *neither* side. It is not the stove’s and it is not the supplier’s. An interface written in the vendor’s types has quietly moved the fitting onto the supplier, which is the imported range two doors down.

Where the dependency points, before and after:

BAD

Checkout -----> RazorpayGateway
(imports it, constructs it)

the arrow goes from your code
to the vendor

→ swapping means editing Checkout
→ testing means a network call

GOOD

Checkout -----> PaymentGateway
 ^
 |
 RazorpayGateway
 StripeGateway
 FakeGateway

the arrows go from the vendors
to your interface

→ swapping means one line at start-up
→ testing means passing a fake

What to notice: the arrows reversed. Before, your code pointed at a vendor; after, three vendors point at a thing you own. That reversal is the whole of “depend on abstractions”.

5 How it actually works

The wiring, in full

```
# gateways/base.py -- yours, and nothing vendor-shaped in it
@dataclass(frozen=True)
class ChargeResult:
    charge_id: str
    succeeded: bool
    failure_reason: str | None = None

class PaymentError(Exception): ...
class CardDeclined(PaymentError): ...

class PaymentGateway(Protocol):
    def charge(self, amount: Money, token: str) → ChargeResult: ...
    def refund(self, charge_id: str, amount: Money) → ChargeResult: ...
```

```
# gateways/razorpay.py -- the translation lives HERE, at the edge
class RazorpayGateway:
    def charge(self, amount: Money, token: str) → ChargeResult:
        try:
            r = self._client.payment.capture(token, int(amount.paise))
        except razorpay.errors.BadRequestError as exc:
            if exc.code == "CARD_DECLINED":
                raise CardDeclined(str(exc)) from exc      # THEIR error → YOURS
            raise PaymentError(str(exc)) from exc
        return ChargeResult(charge_id=r["id"], succeeded=r["status"] == "captured")
```

Three translations happen in that method, and all three are the point: their units (paise as an int), their errors (`BadRequestError` becomes `CardDeclined`), and their vocabulary (`capture` becomes `charge`). Every one of them would otherwise leak into your business

logic.

```
# app.py -- the composition root: the one place that names concrete classes
def build_checkout(settings) → Checkout:
    gateway = {
        "razorpay": lambda: RazorpayGateway(settings.RAZORPAY_KEY),
        "stripe": lambda: StripeGateway(settings.STRIPE_KEY),
        "fake": FakeGateway,
    }[settings.PAYMENT_PROVIDER]()
    return Checkout(gateway)
```

That function is called the **composition root**. Every concrete class in the program is named here and nowhere else, and “swap the provider” becomes a change to one configuration value.

The interfaces you already depend on

- **Python’s DB-API (PEP 249).** `psycopg`, `mysqlclient` and `sqlite3` all expose `connect`, `cursor`, `execute`, `fetchall`. Code written against it moves between databases without changing, and this is exactly why SQLAlchemy can support eight backends.
- **File-like objects.** `open()`, `io.StringIO`, `gzip.open`, a socket — all support `read` and `write`, so `json.load(f)` accepts any of them. No base class enforces it; it is duck typing standardised by convention.
- **Django’s Storage.** `FileSystemStorage` and `S3Boto3Storage` implement `save`, `open`, `url`, `delete`. Moving uploads to S3 is `DEFAULT_FILE_STORAGE = "..."` in settings.
- **logging.Handler.** One `emit` method, and every destination in the ecosystem behind it.
- **JDBC in Java, and Kubernetes’ CSI** for storage drivers — the same idea at the scale of a whole industry: a standard fitting so that vendors compete behind it.

Testing, which is the payoff nobody mentions in the question

```
class FakeGateway:
    def __init__(self) → None:
        self.charges: list[tuple[Money, str]] = []
        self.next_result = ChargeResult("fake_1", succeeded=True)

    def charge(self, amount: Money, token: str) → ChargeResult:
        self.charges.append((amount, token))
        return self.next_result
```

```
def test_declined_card_leaves_order_unpaid():
    gateway = FakeGateway()
    gateway.next_result = ChargeResult("", succeeded=False, failure_reason="declined")
    checkout = Checkout(gateway)
    ...
```

Testing a declined card against a real provider means a sandbox account, a magic card number, network access, and a test that fails when their sandbox is down. With the fake it is two lines and runs in microseconds. **In practice this is the reason most interfaces earn their keep**, and it is the second implementation that always exists.

6 The numbers

The cost of a swap, counted in files

concrete dependency (Checkout constructs RazorpayGateway):	
every file that constructs a Checkout	~6-15 files
every test that mocks the gateway	~20-40 tests
every place that catches razorpay.errors.*	~5-10 files
error-handling semantics scattered	unknown until grep

	~30-60 edits, over 1-2 weeks
interface + injection:	
write StripeGateway implementing 2 methods	1 new file, ~80 lines
change the composition root	1 line

	1 new file, 1 line

That is the number to quote. **Thirty to sixty edits against one new file and one line**, and the first version's edits are all in code that currently works.

The grep test

A one-command health check you can name in an interview:

```
grep -rn "razorpay\\|stripe" src/ --include=*.py | grep -v "src/gateways/"

a healthy codebase:    0 results
a leaky one:           every result is a place the swap will break
```

Run it on the imports too. If `stripe` is imported outside the adapter directory, the abstraction has a hole in it at that line.

Test speed, which compounds

```
40 checkout tests against a provider sandbox:
  ~400 ms each (network round trip + their latency) = 16 seconds
  plus flakiness: ~2% failure rate from their side = 1 failed run in every few

40 checkout tests against FakeGateway:
  ~0.2 ms each = 8 milliseconds
  flakiness: 0

2,000x faster, and deterministic.
```

The cost of building the abstraction

Be honest about this side too:

```
the interface + one adapter + one fake:
  interface + your own types      ~40 lines
  the adapter (translation)       ~80 lines
  the fake                        ~25 lines
  -----
  ~145 lines, half a day

worth it when: a second implementation exists or is near-certain, OR the tests need a fake.
not worth it when: one implementation, no test need, no named second candidate.
```

7 The trade-offs

An abstraction against direct use

Every interface is a layer: one more file to open when reading, one more indirection when debugging. *I would not put an interface in front of something with exactly one implementation and no test need.* The trigger is the second implementation — and an in-memory fake for tests counts as one, which is why so many repositories legitimately get an interface on day one.

A common interface against each provider's best features

This is Chandran's commercial cylinder, and it is the trade-off candidates most often miss. An interface can only expose what all its implementations can do. Stripe has `authorise-then-capture`; Razorpay's flow differs; a fitting that spans both offers neither in full. *I would not force a single interface over two providers whose models genuinely differ* — I would either narrow the interface to the part that really is common and let the rest be provider-specific code behind a feature check, or accept two interfaces. Pretending they are the same is how you get a `capture()` method that means two different things.

A leaky interface against the translation work

Writing your own `Money`, `ChargeResult` and error types is real work, and using the vendor's types is free today. *I would not put a vendor type in an interface signature*, because the moment it is there the interface has stopped being yours and the swap is back to being a two-week job. The translation at the adapter's edge is the price of the fitting.

Hiding against debuggability

A good abstraction hides the provider — including its error messages, its retry behaviour and its rate limits. *I would not swallow provider detail entirely*: I would keep the original exception as the cause (`raise CardDeclined(...) from exc`) and log the provider's request id, so an on-call engineer can still find the transaction in the vendor's dashboard. An abstraction that makes production incidents harder to diagnose has been drawn too tightly.

The honest sentence

An interface is not free and it is not automatically good. It is a bet that this capability will have more than one implementation, and the way to check the bet is to name the second one out loud. If you can, draw the boundary in your own vocabulary and put every vendor word behind it. If you cannot, write the concrete class and wait.

8 In the interview

How it gets asked

- “How would you make it easy to swap the payment provider later?” — the direct form, and the answer ends with “one line at start-up”.
- “We store uploads on disk. How would you move to S3 without a rewrite?” — the same question about files, and Django's `Storage` is the reference answer.
- “How do you test this without calling the real API?” — the same idea approached from testing, which is usually the honest reason the interface exists.
- “What's the difference between abstraction and encapsulation?” — the vocabulary version. Encapsulation hides *data* from callers; abstraction hides *how* behind a named capability.

What to say out loud, in the first ninety seconds

1. **Name the capability, not the vendor.** *“I’d define a `PaymentGateway` interface with two methods — `charge` and `refund` — and my code would depend on that, never on `Razorpay` directly.”*
2. **Say the vocabulary rule immediately.** *“Crucially, the interface is in my vocabulary, not theirs. My own `Money` type, my own `ChargeResult`, my own `PaymentError` hierarchy. No vendor class ever appears in a signature.”*
3. **Say who chooses.** *“`Checkout` doesn’t construct a gateway — it takes one in its constructor. The concrete class is named in exactly one place, at start-up. That’s the line that changes when we swap.”*
4. **Say where the translation lives.** *“Each adapter translates at its own edge: their error codes into my exceptions, their units into my `Money`. That translation is the work I’m paying for, and it’s what stops their model leaking in.”*
5. **Give the number.** *“Concretely: swapping goes from thirty-odd edits across the codebase to one new file and one line.”*
6. **Name the cost, unprompted.** *“The trade is that the interface can only expose what both providers do. If one has `authorise-then-capture` and the other doesn’t, I’d rather narrow the interface than fake a shared model.”*

The follow-ups

“How do you stop the abstraction leaking?” Three specific ways, and I’d check all three because they fail independently. First, types: no vendor class in any signature. If `charge` takes a `stripe.PaymentIntent`, then every other implementation has to construct a Stripe object to satisfy the interface, and my `Checkout` imports `stripe` to build the argument — the interface exists and the coupling is untouched. So I define `Money`, `ChargeResult` and a `PaymentError` hierarchy of my own. Second, errors: `except stripe.CardError` must never appear in business logic. Each adapter catches its provider’s exceptions and re-raises mine, keeping the original as `__cause__` so the stack trace still helps on-call. Third, semantics: I would not add a `capture()` method just because one provider distinguishes `authorise` from `capture` — a method that means two different things behind two adapters is worse than no method. The check I’d actually run is a grep: search the source for the provider’s name outside the adapter directory, and if it returns anything, that line is where the swap will break. A healthy codebase returns nothing.

“Isn’t this over-engineering when you only have one provider?” It can be, and I would apply a test rather than a rule: can I name the second implementation? For a payment gateway I usually can, and not because of vendor risk — the second implementation is the fake I need for tests. Testing a declined card, a timeout and a partial refund against a real sandbox means network access, magic card numbers, and a suite that fails

when their sandbox is down. With a fake it's two lines per case and runs in microseconds — forty tests going from sixteen seconds and flaky to eight milliseconds and deterministic. That alone usually justifies the roughly 145 lines of interface, adapter and fake. Where I would *not* do it is a capability with one implementation and no test need — an invoice numberer, a slug generator. And I'd be honest that if my only argument is “we might swap it one day”, that is speculation, and I'd write the concrete class and extract the interface when the second implementation actually arrives, which is a mechanical refactor rather than a rewrite.

“What if the two providers genuinely work differently — one has authorise-then-capture and the other doesn't?” Then I stop pretending they are the same, because an interface that spans two different models ends up offering neither properly. There are three honest options and I'd pick between them explicitly. One: narrow the interface to what is genuinely common — `charge` and `refund` — and treat authorise-then-capture as a capability that lives outside it, reached through a separate, optional protocol that only some adapters satisfy, with a check before use. Two: widen the interface to the richer model and make the simpler provider's adapter emulate it — `authorise` becomes a no-op that records intent, `capture` does the real charge. That works only if the emulation is honest; if a failure can happen at capture time on one provider and not the other, the emulation is a lie and I would not ship it. Three: accept that these are two interfaces for two different kinds of provider, and let the application choose at a higher level. What decides it is whether my *application* needs the richer model. If checkout only ever charges once, option one is right and the extra capability is not my problem. This is the same trade Chandran makes with his gas fitting: the standard fitting cannot carry the hotter commercial cylinder, and that is a real thing given up in exchange for being able to change supplier in ten minutes.

A model answer

“The goal is that swapping the provider is a configuration change, not a code change, and there are three things that get me there.

First, I define the capability in my own vocabulary. A `PaymentGateway` interface — a Protocol in Python — with two methods: `charge(amount: Money, token: str) → ChargeResult` and `refund`. My own `Money` type, my own `ChargeResult`, my own `PaymentError` and `CardDeclined` exceptions. No vendor class ever appears in a signature, because the moment it does the interface belongs to that vendor rather than to me.

Second, each provider gets an adapter that implements it, and the adapter is where all the translation happens: their units into my `Money`, their error codes into my exceptions, their workflow vocabulary into mine. That translation is the actual work, and doing it at the edge is what stops their model spreading into my business logic. I’d keep the original exception as the cause so the vendor’s request id is still in the trace when something goes wrong at 3am.

Third, and this is where most designs fail — `Checkout` does not construct a gateway. It takes one in its constructor. The concrete class is named in exactly one place, a composition root at start-up, which reads the provider from settings. That’s the one line that changes.

```
class Checkout:
    def __init__(self, gateway: PaymentGateway) → None:
        self._gateway = gateway
```

The check I’d run to prove it worked: grep the source for the provider’s name outside the adapter directory. If that returns nothing, the swap is one file and one line. If it returns anything, every hit is a place the swap will break — and in a codebase where `Checkout` constructs the gateway itself, that’s typically thirty to sixty edits across a week or two.

The reason I’d do this even with one provider is the fake. A `FakeGateway` is the second implementation, and it turns forty checkout tests from sixteen flaky seconds against a sandbox into eight deterministic milliseconds, including cases like a declined card that are genuinely awkward to trigger for real.

And the cost, since there is one: the interface can only offer what all the implementations can do. If one provider has authorise-then-capture and the other doesn't, I'd narrow the interface to what's genuinely common rather than fake a shared model — and if my application ever really needs the richer flow, that's a deliberate decision to make then, not something to paper over now.”

9 Recall card

- **Interface = the capability, in *your* vocabulary.** Your `Money`, your `ChargeResult`, your exceptions. A vendor type in a signature means the interface belongs to the vendor and bought nothing.
- **The user must not choose the implementation.** `Checkout(gateway)`, never `self._gateway = RazorpayGateway()`. Concrete classes are named once, in a composition root — that is the one line that changes.
- **The adapter translates at its own edge:** their units → your types, their errors → your exceptions (`raise CardDeclined(...) from exc`), their workflow words → yours. Three leak types: types, errors, semantics.
- **The test before writing one: can you name the second implementation?** An in-memory **fake for tests** counts, and is usually the real reason — 40 tests from 16 flaky seconds to 8 deterministic milliseconds.
- **Numbers and the check:** swap cost 30-60 edits → 1 file + 1 line; `grep -rn "stripe" src/ --exclude-dir=gateways` should return nothing. **The cost:** an interface offers only what *all* implementations can do — narrow it honestly rather than fake a shared model.

Practice — Binary search on floats, and the epsilon question

DSA topic: Binary search on floats, and the epsilon question **System design topic:** Abstraction and interfaces

Code these, in this order

One rule: **before writing any float loop, say out loud what stops it.** If the answer is not “a fixed count” or “a relative epsilon”, stop and pick one.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Sqrt(x)	LeetCode 69 (Easy)	Whether you notice the answer is an integer and stay in integers.
2	Square root to six decimal places	Standard follow-up to LC 69	The stopping rule, and the <code>max(1.0, x)</code> bound.
3	Pow(x, n)	LeetCode 50 (Medium)	Not binary search on the answer — binary <i>exponentiation</i> . Know the difference.
4	Minimize Max Distance to Gas Station	LeetCode 774 (Hard)	The full float search: an expensive check, so the iteration count is a real decision.

On problem 1, do not touch a float

The answer is an integer, so search integers: the largest `m` with `m * m ≤ x`. Python’s ints are exact at any size, so there is no rounding to reason about. Remember the ceiling midpoint — `(lo + hi + 1) // 2` — because this is the last-True form; write it with the floor midpoint first, watch it hang, kill it, then fix it. Then run it on `10**18` and confirm the answer is exact.

On problem 2, break it three ways before you fix it

Write the working version. Then produce all three failures deliberately and collect the evidence:

- `while lo < hi` — run it, kill it with Ctrl-C, paste the `KeyboardInterrupt`.
- `while hi - lo > 1e-9` with `x = 1e18` — run it, kill it, and say what number makes it unreachable.
- `hi = x` instead of `hi = max(1.0, x)` — run it on `0.25` and read the wrong answer.

Then write the hundred-iteration version and check all four inputs pass.

On problem 3, notice this is a different animal

LeetCode 50 is not binary search on the answer. It is exponentiation by squaring: x^n is $(x^{(n//2)})^2$, with an extra x when n is odd. It is $O(\log n)$ for the same reason binary search is — halving — but nothing is being searched. Being able to say why it is in the same complexity class and a different pattern is the point of including it here. Handle negative n , and say what $x^{(-2^{31})}$ does about overflow.

On problem 4, size the iteration count from the requirement

The check here is an $O(n)$ pass, so a hundred iterations is a hundred passes and the count stops being free. Work out the number: the answer needs about 10^{-6} precision, the range is up to 10^8 , so $\log_2(10^8 / 10^{-6}) = \log_2(10^{14}) \approx 47$. Use 50, not 100, and say the arithmetic out loud. Then say why you would still not switch to an epsilon loop to save the iterations.

The stopping-rule drill

For each, say what happens — converges, hangs, or converges to the wrong precision:

1. `while lo < hi` on floats.
2. `while hi - lo > 1e-9`, answer near 3.
3. `while hi - lo > 1e-9`, answer near 10^{15} .
4. `while hi - lo > 1e-9 * max(1.0, lo)`, answer near 10^{15} .
5. `for _ in range(100)`, answer near 10^{15} .
6. `for _ in range(20)`, answer near 3, six decimals wanted.

The precision drill

Answer from memory, no calculator:

1. How many significant decimal digits does a 64-bit double hold?
2. What is the smallest step between representable doubles near 1.0? Near 10^{15} ?
3. How many halvings does it take to shrink a range of 10 to below 10^{-9} ?
4. By what factor does a hundred halvings shrink any range?
5. Why is `0.1 + 0.2 == 0.3` False?

The integer-first drill

For each, say whether you would search integers or floats, and why:

1. The floor of the square root of `x`.
2. The square root to six decimal places.
3. The smallest ship capacity, weights in whole kilos.
4. The minimum time in seconds, answers to the millisecond.

5. A price, in rupees and paise.

Number 5 has a specific right answer that is neither. Say what it is and why money is not a float.

The swap-the-provider drill

Take this and rework it out loud in five minutes:

```
import stripe

class Checkout:
    def __init__(self):
        self.client = stripe.Client(api_key=settings.STRIPE_KEY)

    def pay(self, order, token):
        try:
            intent = self.client.PaymentIntent.create(
                amount=int(order.total * 100), currency="inr", payment_method=token
            )
        except stripe.error.CardError as e:
            order.mark_failed(e.user_message)
            return
        order.mark_paid(intent.id)
```

1. Name the three leaks — a type leak, an error leak, and a semantics leak. Point at the exact lines.
2. Write the interface. It has two methods and no vendor word in it.
3. Say who constructs the gateway now, and where.
4. Write the `FakeGateway` and the test for a declined card.
5. Count the edits to add Razorpay, before and after.
6. Write the grep command that proves the abstraction holds, and say what a healthy result looks like.

The name-the-second drill

For each, say whether you would write an interface, and name the second implementation — or say there is not one:

1. A payment gateway.
2. A slug generator that turns a title into a URL fragment.
3. A file store for user uploads.
4. An order repository.
5. An invoice numbering scheme.
6. A search index.

Two of these have “an in-memory fake for tests” as their honest second implementation. Say which, and whether that is enough on its own.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the square root of a number to six decimal places, without the library call.* The integer-or-float clarifying question, the `max(1.0, x)` bound with its reason, what changes on floats, the fixed count and why you prefer it, and the input that kills the absolute epsilon.
 2. *How would you make it easy to swap the payment provider later?* The interface in your vocabulary, the injection, the translation at the adapter's edge, the number (30-60 edits against one line), and the cost you accepted.
 3. *When does your loop stop, and why is that the right condition?* The three candidates, what each does on a large answer, and the sentence about a hundred halvings shrinking any range past double precision.
-

Before you move on

- [] I solved LeetCode 69 entirely in integers, with the ceiling midpoint, and it is exact at 10^{18} .
- [] I made a float loop hang three separate ways and collected the evidence for each.
- [] I can state the double-precision numbers — 15-17 digits, $\sim 2.2e-16$ near 1, ~ 0.125 near 10^{15} .
- [] I sized the iteration count for LeetCode 774 from the required precision, with the arithmetic.
- [] I found all three leaks in the `Checkout` class and named each by kind.
- [] I wrote the grep command and can say what a healthy result is.
- [] I answered all three questions above out loud.

Peak finding, and searching data that is structured but not sorted

DATA STRUCTURES AND ALGORITHMS

Peak finding, and searching data that is structured but not sorted

You can binary search using local structure alone, with no global ordering.

SYSTEM DESIGN

Composition over inheritance

You can rewrite an inheritance tree as composition and say what improved.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- Find a peak element in $O(\log n)$.
- Refactor this class hierarchy. Why is your version better?

Peak finding, and searching data that is structured but not sorted

1 What this is, and why they ask it

A **peak** is an element bigger than both its neighbours. Given an array with no order to it at all — values going up and down at random — you can find one in $O(\log n)$. Not the largest element: *a* peak, any peak. The array is not sorted, nothing about it is monotone, and binary search still works, because the only thing binary search has ever needed is a rule that lets you throw half away with a guarantee.

They ask it because it breaks the mental model most candidates have built. Ask “can I binary search this?” and the reflex answer is “only if it’s sorted”, which is wrong and has been wrong since [day 045](#)’s rotated array. LeetCode 162 is the cleanest demonstration: the input is deliberately unsorted, the answer is deliberately not unique, and the $O(\log n)$ requirement is deliberately stated so you cannot scan. Candidates who have understood the invariant get it in three minutes; candidates who memorised a template stare at it. It is also the day the phase’s real lesson lands — **binary search is about discarding halves with proof, not about sortedness**.

2 The story

Kunjappan has four acres of areca on the slope above his house in Wayanad, and in December he goes up before six, when it is still dark and the mist has come down.

The mist there is not like rain. It is a solid white thing that sits on the hill until about half past eight, and on a bad morning he can see perhaps two arm-lengths ahead of him. He has walked that hill for thirty years and he still cannot see it in December.

There is a bund running along the top edge of the plot, a raised earth path about half a kilometre long that the water follows. It is not level. Whoever cut it followed the ground, so it climbs and drops the whole way along — small rises, small dips, one big hump near the middle where a rock is.

What he wants, at half past five in the morning, is a signal on his phone to call his brother in Kannur. He does not need the highest point on the bund. He needs *any* high point, because from the top of any rise on that hill the signal comes and in the dips it does not.

So he does this.

He walks out to about the middle of the bund and puts his foot forward, feeling with his toe. The ground ahead of him rises. That one small fact tells him something much bigger than it looks, and it is the thing he has never had to explain to himself: if the ground goes up from here, then somewhere ahead there is a top. It has to be. The bund cannot go on climbing forever — it ends at the fence. So it must climb for a while and then stop climbing, and where it stops climbing is a rise, and that is all he wants.

Which means the whole half of the bund behind him is finished with. He does not need to walk it, feel it, or think about it again. He has not seen any of it and he does not care.

He goes forward, roughly to the middle of what is left, and feels again. This time the ground drops ahead of him. Same argument in the other direction: if it falls from here going forward, then behind him — between where he stopped last time and here — the ground must have come up and turned over somewhere. There is a top back there. So the stretch in front is finished with instead.

Six or seven of these, each one throwing away half the bund he has not walked, and he is standing on a rise with two bars on the phone.

He has walked maybe eighty steps on a bund of six hundred, and he still could not draw the shape of it if you asked him.

3 The idea in plain English

Kunjappan's bund is the array. The slope under his toe is a comparison between two neighbours. And the sentence he has never had to explain — *it cannot keep climbing forever* — is the entire proof that the method works.

What a peak is

```
nums = [1, 2, 1, 3, 5, 6, 4]
```

An element is a **peak** if it is strictly greater than the element on each side. Here index 1 (value 2) is a peak, because $2 > 1$ and $2 > 1$. Index 5 (value 6) is also a peak, because $6 > 5$ and $6 > 4$. **There are two, and either is an acceptable answer.** That is unusual and it is part of the problem's design: an answer that is not unique is a signal that you are not being asked to search for a specific thing.

The ends need a convention. LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as **negative infinity** — imaginary values smaller than anything real, just off each end. With that convention:

- The first element is a peak if it is bigger than the second. Nothing to its left can beat it.

- The last element is a peak if it is bigger than the second-last.
- **Every array has at least one peak.** A finite list of values has a maximum, and the maximum beats both its neighbours, so it is a peak. So there is no “not found” case, ever.

The problem also guarantees `nums[i]  $\neq$  nums[i + 1]` for all `i` — no two adjacent values are equal. That matters, and §7 shows what happens without it.

The rule that discards half

Stand at `mid` and compare it with its right-hand neighbour only:

```
if nums[mid] < nums[mid + 1]:
    # the ground rises to the right → a peak exists in [mid + 1, hi]
    lo = mid + 1
else:
    # the ground falls to the right → a peak exists in [lo, mid]
    hi = mid
```

Two branches, one comparison, and each one is Kunjappan’s argument.

If `nums[mid] < nums[mid + 1]` — it climbs to the right. Now walk right in your head. Either the values keep climbing all the way to the end, in which case the last element is bigger than its left neighbour and bigger than the imaginary negative infinity on its right, so it is a peak; or somewhere they stop climbing, and the element where they turn over is bigger than both its neighbours, so *it* is a peak. **Either way there is a peak strictly to the right of `mid`.** So everything from `lo` to `mid` can go.

If `nums[mid] > nums[mid + 1]` — it falls to the right. Mirror the argument leftwards from `mid`: either the values keep climbing all the way back to `lo` and beyond, in which case `nums[lo]` beats the negative infinity on its left and is a peak, or they turn over somewhere in between. **Either way there is a peak in `[lo, mid]`, and `mid` itself may be it** — which is why `hi = mid` and not `hi = mid - 1`.

The invariant

Say this out loud in the interview, because it is what the question is testing:

The range `[lo, hi]` always contains at least one peak.

It is true at the start, because the whole array contains one. Each branch preserves it, by the argument above. When `lo = hi` the range holds exactly one element, and that element must be the peak.

Notice how this differs from every previous day. There is no target. There is no monotone question. There is only a claim about the range that stays true — and that is enough.

Why this is [day 043](#)'s template

It is the half-open shape you already know:

```
lo, hi = 0, len(nums) - 1
while lo < hi:
    mid = (lo + hi) // 2
    if nums[mid] < nums[mid + 1]:
        lo = mid + 1
    else:
        hi = mid
return lo
```

`while lo < hi` with `hi = mid` — the pairing from [day 043](#), and the reason it terminates is the same inequality: `mid < hi` always, because `lo < hi`. And `mid + 1` is always a valid index, for the same reason: `mid < hi ≤ len(nums) - 1`, so `mid + 1 ≤ len(nums) - 1`.

Not the maximum — and why that is the point

The function returns *a* peak, not *the* maximum. On `[1, 2, 1, 3, 5, 6, 4]` it may return index 5, and on `[1, 2, 1, 3, 5, 6, 4]` it may return index 1, depending on where the halving lands.

Finding the *maximum* of an unsorted array is $O(n)$ and cannot be beaten — you must look at every element, because any one you skip could be the largest. Finding *a peak* is $O(\log n)$. That gap is the whole lesson: **weakening what you ask for from “the best” to “a local best” changes the complexity class**. Say that sentence; interviewers like it because it is the actual insight.

The family this opens

Once you see that “local structure plus a discard proof” is enough, several problems join up:

- **LeetCode 852, peak in a mountain array.** Guaranteed to rise then fall, so there is exactly one peak. Identical code, and the guarantee only makes the answer unique.
 - **LeetCode 153, minimum in a rotated array**, from [day 045](#). That is a *valley* — the same argument upside-down.
 - **LeetCode 1901, a peak in a 2D grid.** Binary search the columns; in each candidate column take the row-maximum, and compare it left and right. $O(\text{rows} \times \log \text{cols})$.
 - **Ternary search**, for a genuinely unimodal function, which finds the *actual* maximum rather than a local one — worth naming as the neighbouring tool.
-

4 The picture

The bund, and the two arguments:

```
index      0      1      2      3      4      5      6      7      8
value      1      3      2      4      7      9      5      6      2
           ^              ^              ^
           peak(3>1,3>2)    peak(9>7,9>5)  peak(6>5,6>2)

three peaks. ANY of them is a correct answer.
the ends: imagine -inf just off each side.
-inf | 1 3 2 4 7 9 5 6 2 | -inf
```

What to notice: there is no order here at all. The values go up, down, up, up, down, up, down. Nothing is sorted and nothing is monotone.

One step of the search, both branches:

```
lo=0                      mid=4                      hi=8
[ 1      3      2      4 | 7 | 9      5      6      2 ]
                        nums[4]=7 < nums[5]=9 → it RISES to the right

going right from 7: 9, then 5 -- so it turned over at 9.
(and if it had never turned over, the last element would beat -inf and be a peak)
⇒ a peak EXISTS in [5, 8]. Discard [0, 4] without looking at it.

lo = 5
                        [ 9      5      6      2 ]
                        mid=6, nums[6]=5 < nums[7]=6 → rises right

lo = 7
                        [ 6      2 ]
                        mid=7, nums[7]=6 > nums[8]=2 → falls right

hi = 7      lo = hi = 7. Return 7. nums[7] = 6, and 6 > 5 and 6 > 2. A peak.
```

What to notice: the search never looked at indices 0 to 3 and never needed to. The discard was justified by an argument about what *must* exist, not by any value it read.

Why “rises to the right” guarantees a peak to the right:

```
case A: it never stops rising          case B: it stops rising somewhere
mid                                  mid      j
|                                  |      |
7 → 9 → 11 → 14 → 20 → 31 | -inf      7 → 9 → 14 → 20 → 6
                        ^              ^
                        the last element beats -inf      20 > 14 and 20 > 6
                        → it is a peak                    → it is a peak

there is no third case. The array is finite, so it cannot rise forever.
```

What to notice: the whole method rests on “the array is finite”. That is Kunjappan’s *the bund ends at the fence*, and it is the sentence to say when an interviewer asks you to justify the discard.

5 The code, built step by step

The bounds and the loop shape

```
lo, hi = 0, len(nums) - 1
while lo < hi:
    ...
return lo
```

Closed range at both ends this time — `hi` is the last valid index — but the loop condition is strictly `<`, because the branch that keeps `mid` writes `hi = mid`. That is the day 043 pairing: whenever a branch keeps the middle, the loop condition must be strict.

There is no `return -1`. Every array has a peak, so the loop always ends on one.

The single comparison

```
mid = (lo + hi) // 2
if nums[mid] < nums[mid + 1]:
    lo = mid + 1
else:
    hi = mid
```

Compare with the **right** neighbour only. Comparing with both neighbours is the trap in §7 — it is unnecessary, and it introduces an index that can go out of range.

`mid + 1` is always safe here. `mid` is the floor of $(lo + hi) / 2$ and `lo < hi`, so `mid < hi`, so `mid + 1 ≤ hi ≤ len(nums) - 1`.

The complete solution

```
def find_peak_element(nums: list[int]) → int:
    """LeetCode 162. Index of ANY peak: an element strictly greater than both neighbours.

    Treats nums[-1] and nums[n] as negative infinity, so the first and last elements
    can be peaks. Every array has at least one peak, so there is no not-found case.

    Invariant: the range [lo, hi] always contains at least one peak.
    """
    lo, hi = 0, len(nums) - 1

    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < nums[mid + 1]:
            lo = mid + 1
        else:
            hi = mid
    return lo

    # mid < hi, so mid + 1 is always valid
    # it rises right: a peak must exist to the right
    # it falls right: a peak exists at mid or left of it
    # lo == hi, and that element is a peak

def peak_index_in_mountain(arr: list[int]) → int:
    """LeetCode 852. Guaranteed to rise then fall, so the peak is unique.

    Identical code. The guarantee changes the answer's uniqueness, not the method.
    """
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] < arr[mid + 1]:
            lo = mid + 1
        else:
            hi = mid
    return lo

def find_valley(nums: list[int]) → int:
    """The mirror: an element smaller than both neighbours. One comparison flipped."""
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[mid + 1]:
            lo = mid + 1
        else:
            hi = mid
    return lo

    # it FALLS right → a valley exists to the right

def is_peak(nums: list[int], i: int) → bool:
    """Check the answer, using the -inf convention at both ends."""
    left = nums[i - 1] if i > 0 else float("-inf")
    right = nums[i + 1] if i < len(nums) - 1 else float("-inf")
    return nums[i] > left and nums[i] > right

if __name__ == "__main__":
    for nums in (
        [1, 3, 2, 4, 7, 9, 5, 6, 2],
        [1, 2, 1, 3, 5, 6, 4],
        [1, 2, 3, 4, 5],
        [5, 4, 3, 2, 1],
        [7],
        [2, 1],
        [1, 2],
    ):
        i = find_peak_element(nums)
        print(nums, "→ index", i, "value", nums[i], "| is a peak:", is_peak(nums, i))

    print(peak_index_in_mountain([0, 2, 5, 3, 1]))
    print(find_valley([9, 7, 4, 6, 8]))
```

Run it. The five inputs that decide correctness are the strictly rising array, the strictly falling array, the single element, and both two-element arrays. Those cover every place the negative-infinity convention does real work, and a solution that handles all five almost certainly handles everything.

The 2D version, named

```
def find_peak_grid(mat: list[list[int]]) → list[int]:
    """LeetCode 1901. A peak in a grid: greater than its four neighbours.

    Binary search the COLUMNS. In each candidate column, take the row with the largest
    value -- that element already beats its up and down neighbours -- then compare it
    left and right, exactly as in 1D.  O(rows x log cols).
    """
    lo, hi = 0, len(mat[0]) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        best_row = max(range(len(mat)), key=lambda r: mat[r][mid])
        if mat[best_row][mid] < mat[best_row][mid + 1]:
            lo = mid + 1
        else:
            hi = mid
    best_row = max(range(len(mat)), key=lambda r: mat[r][lo])
    return [best_row, lo]
```

The trick worth understanding: taking the column's maximum settles the vertical direction for free, so the two-dimensional problem collapses to the one-dimensional argument. Mention it if asked; do not volunteer it unless there is time.

6 What it costs

Time

each pass: one midpoint, one comparison, one assignment	→ 0(1)
each pass discards at least half the range	→ log ₂ (n) passes
	→ 0(log n)

n = 1,000	10 comparisons
n = 1,000,000	20 comparisons
n = 1,000,000,000	30 comparisons

Against the honest alternative

find A PEAK	: 0(log n)	-- 20 comparisons at a million
find THE MAXIMUM	: 0(n)	-- 1,000,000 comparisons at a million

and the maximum cannot be done faster: any element you do not look at could be the largest, so an adversary can always place the maximum where you did not look.

Fifty thousand times fewer comparisons, bought by asking for something weaker. That trade — a local best instead of a global best — is the sentence to have ready, and it is the reason the problem exists.

Space

```
lo, hi, mid: three integers    → O(1) extra space
```

The 2D version

```
find_peak_grid: log2(cols) passes, each scanning one column of `rows` values
                  → O(rows x log cols)

a 1,000 x 1,000 grid:
  scan everything : 1,000,000 reads
  this           : 1,000 x 10 = 10,000 reads    -- 100x fewer
```

7 The traps

The near-miss: comparing with the left neighbour

The mirror of the correct rule looks equally reasonable and is not:

```
def broken_left(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < nums[mid - 1]:    # ← LEFT neighbour
            hi = mid - 1
        else:
            lo = mid
    return lo

print(broken_left([1, 2]))
```

```
0
```

Index 0 holds 1, and `nums[1]` is 2, so 1 is not a peak. The cause is that at `mid = 0`, `nums[mid - 1]` is `nums[-1]` — Python's *last* element — so the comparison is against a value from the wrong end of the array, silently. Sixth appearance of an index reaching `-1`, and the fifth time it has produced a wrong answer rather than a crash.

On other inputs the same function hangs, because `lo = mid` with a floor midpoint makes no progress when `hi = lo + 1`. **Compare with the right neighbour only** — `mid + 1` is always valid because `lo < hi` forces `mid < hi`, and there is no `mid - 1`

to go wrong.

The real error: `while lo <= hi` with `hi = mid`

```
def broken(nums):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:                                # ← non-strict, with hi = mid
        mid = (lo + hi) // 2
        if nums[mid] < nums[mid + 1]:
            lo = mid + 1
        else:
            hi = mid
    return lo

print(broken([1, 2, 3]))
```

```
Traceback (most recent call last):
  File "day49.py", line 11, in <module>
    print(broken([1, 2, 3]))
    ~~~~~^
  File "day49.py", line 5, in broken
    if nums[mid] < nums[mid + 1]:
        ~~~~~^
IndexError: list index out of range
```

The non-strict condition lets `lo` and `hi` both reach the last index, so `mid` becomes the last index and `mid + 1` is off the end. On other inputs the same bug hangs instead, because `hi = mid` when `lo = hi = mid` changes nothing. [Day 043's](#) pairing rule, and it costs you both failure modes at once: **a branch that keeps the middle requires a strict loop condition.**

The near-miss: `hi = mid - 1`

```
def broken_minus_one(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < nums[mid + 1]:
            lo = mid + 1
        else:
            hi = mid - 1                # ← throws away mid, which may BE the peak
    return lo

print(broken_minus_one([1, 2, 1, 3, 5, 6, 4]))
```


Index 4 holds 5, and its right neighbour is 6, so it is not a peak. When the values fall to the right of `mid`, `mid` itself is a candidate — the turning point may be exactly there — and `- 1` discards it. Note that this version happens to be right on `[5, 4, 3, 2, 1]` and on `[1, 2, 3, 4, 5]`, which is what lets it survive a casual test.

The needless check: comparing both sides for an early return

```
if nums[mid - 1] < nums[mid] > nums[mid + 1]:
    return mid                # "found one early"
```

This one is not wrong — it returns early on a genuine peak and otherwise falls through — but it earns nothing. The one-sided rule already drives the range onto a peak in the same number of passes, and the extra line reintroduces `nums[mid - 1]`, with the `-1` wrap waiting at `mid = 0`. **A correct branch you do not need is still a branch you can get wrong**, and in a whiteboard answer it is two more lines to defend.

The trap: equal adjacent values

LeetCode 162 guarantees `nums[i] ≠ nums[i + 1]`. Remove that guarantee and the method breaks:

```
nums = [1, 2, 2, 2, 1]
```

At `mid = 2`, `nums[2]` is not less than `nums[3]` — they are equal — so the code goes left, and it would go left just as happily from `mid = 1`. The plateau carries no information about which side the peak is on, exactly as [day 045's](#) duplicates did. There is no $O(\log n)$ answer on a flat plateau: an adversary can hide the rise at either end of it, so the worst case is $O(n)$. **Read the constraint, and if it is missing, ask.**

The trap: reading “peak” as “maximum”

```
print(find_peak_element([1, 3, 2, 4, 7, 9, 5, 6, 2]))    # may return 7, not 5
```

Index 7 (value 6) is a peak. Index 5 (value 9) is the maximum. Both are correct answers to the question that was asked, and a solution that insists on returning 5 cannot be $O(\log n)$. If you believe the problem wants the maximum, you have misread it — and if the interviewer really does want the maximum, the honest answer is that it is $O(n)$ and cannot be beaten.

8 In the interview

How it gets asked

- “Find a peak element — one that’s greater than its neighbours. The array isn’t sorted. Do it in $O(\log n)$.” — LeetCode 162, and the “isn’t sorted” is said deliberately to see your reaction.
- “This array increases then decreases. Find the turning point.” — LeetCode 852, the friendlier version, often asked first.
- “Why does binary search work here? There’s no order.” — the real question, and the answer is the discard proof.
- “Now find a peak in a 2D grid.” — LeetCode 1901, the hard follow-up, and naming the column-maximum trick is enough even without writing it.

What to say out loud, in the first ninety seconds

1. **Deal with the sortedness objection before they raise it.** “The array isn’t sorted, and it doesn’t need to be. Binary search needs a rule that lets me discard half with a guarantee — sorted is just the usual way to get one.”
2. **Confirm the conventions.** “Are the ends treated as negative infinity, so the first and last elements can be peaks? And are adjacent values guaranteed distinct? That second one matters — a plateau makes it $O(n)$.”
3. **State that an answer always exists.** “Every array has at least one peak, because the maximum beats both its neighbours. So there’s no not-found case.”
4. **Give the rule and the proof in one breath.** “I compare the middle with its right neighbour only. If it rises to the right, then going right either it climbs to the end — in which case the last element beats the negative infinity beyond it and is a peak — or it turns over somewhere, and the turning point is a peak. Either way a peak exists to the right, so I discard everything up to and including the middle.”
5. **State the invariant.** “The invariant is that `[lo, hi]` always contains at least one peak. When they meet, that element is one.”
6. **Give the cost and the contrast.** “ $O(\log n)$ time, $O(1)$ space. Worth saying: finding the maximum is $O(n)$ and can’t be beaten — asking for a local best instead of a global best is what buys the logarithm.”

The follow-ups

“Prove that going uphill always finds a peak.” The proof is one sentence plus a case split, and it turns entirely on the array being finite. Suppose `nums[mid] < nums[mid+1]` — it rises to the right. Now walk rightwards from `mid+1`. There are exactly two possibilities. Either the values keep increasing all the way to the last index, in which case the last element is greater than its left neighbour and greater than the imaginary negative

infinity just past the end, so the last element is a peak. Or at some index j the sequence stops increasing, meaning `nums[j-1] < nums[j]` and `nums[j] > nums[j+1]`, which makes j a peak by definition. There is no third case, because a finite list cannot increase forever. So a peak exists strictly to the right of `mid`, and discarding `[lo, mid]` cannot lose the only answer. The mirrored argument covers the falling branch, except there `mid` itself might be the turning point, which is exactly why that branch writes `hi = mid` rather than `hi = mid - 1`.

“What if adjacent elements can be equal?” Then the method breaks, and I would say so rather than patch it. On `[1, 2, 2, 2, 1]` at a midpoint inside the plateau, the comparison between the middle and its right neighbour is an equality, which tells me nothing about which side holds a peak — and I can construct two arrays that look identical at every position I have examined but have their peaks on opposite sides. That is the same structure as duplicates in a rotated array from [day 045](#): when a comparison carries no information, no half can be discarded safely. The consequence is that the worst case becomes $O(n)$ — an adversary places the plateau across most of the array — and that is a property of the problem rather than a weakness in my code. In practice I would ask about it in the clarifying questions rather than discover it, because LeetCode 162 states the distinctness guarantee explicitly and its presence is a hint that the setter knew it mattered.

“Now do it on a 2D grid — an element greater than its four neighbours.” The same argument survives, with one idea added, and the cost becomes $O(\text{rows} \times \log \text{cols})$. I binary search over the *columns* rather than the elements. For a candidate column, I scan it and find the row holding its largest value. That element already beats its up and down neighbours, because it is the maximum of its column — so the vertical direction is settled for free, and I am back to a one-dimensional problem about the horizontal direction. Then I compare it with the element to its right: if the right one is bigger, a peak exists in the columns to the right, by exactly the argument from the 1D case applied along that row; otherwise a peak exists at this column or to its left. The column scan is $O(\text{rows})$ and I do $\log(\text{cols})$ of them. On a 1,000 by 1,000 grid that is about ten thousand reads against a million for scanning everything. The step people get wrong is scanning the row instead of the column while searching over columns — that does not settle the perpendicular direction, and the discard is no longer justified.

A model answer

“First thing worth saying: the array isn’t sorted and it doesn’t need to be. Binary search has never actually required sortedness — it requires a rule that lets me throw away half with a guarantee, and sorted is just the most common way to get one. Here the guarantee comes from local structure.

Two clarifications. Are the ends treated as negative infinity, so the first and last elements can be peaks? And are adjacent values guaranteed distinct? That second one matters a lot — a plateau of equal values carries no information about which side a peak is on, and would push this to $O(n)$.

Given both, the key observation is that a peak always exists: the maximum of any finite array beats both its neighbours. So there’s no not-found case.

The rule is one comparison, with the right-hand neighbour only. If `nums[mid] < nums[mid+1]` — it rises to the right — then walking right, either the values climb all the way to the end, in which case the last element beats the negative infinity past it and is a peak, or they turn over somewhere, and the turning point is a peak. There’s no third case, because the array is finite. So a peak definitely exists to the right, and I can discard everything up to and including the middle without looking at it. If it falls to the right, the mirrored argument puts a peak at `mid` or to its left — and `mid` itself might be it, which is why that branch keeps it.

```
def find_peak_element(nums: list[int]) → int:
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < nums[mid + 1]:
            lo = mid + 1      # rises right: a peak must exist to the right
        else:
            hi = mid          # falls right: mid itself may be the peak
    return lo
```

The invariant is that `[lo, hi]` always contains at least one peak. It’s true at the start, both branches preserve it, and when `lo` equals `hi` that single element must be one.

Two details on the loop. `mid + 1` is always in range, because `lo < hi` forces `mid < hi`. And the condition is strictly less-than, because the else branch keeps `mid` — a branch that keeps the middle and a non-strict condition together give an infinite loop.

$O(\log n)$ time, $O(1)$ space. The sentence I’d close on: finding the *maximum* of an unsorted array is $O(n)$ and cannot be beaten, because any element you skip could be the largest. Finding a *peak* is $O(\log n)$. Weakening the question from ‘the global best’

to ‘a local best’ is what buys the logarithm.”

9 Recall card

- **Binary search needs a discard proof, not sortedness.** A peak is any element beating both neighbours; ends count as $-\infty$, so **every array has one** and there is no not-found case.
- **One comparison, right neighbour only:** rises right $\rightarrow$ `lo = mid + 1`; otherwise $\rightarrow$ `hi = mid` (keep `mid`, it may be the peak). Never compare both sides — `nums[mid-1]` at `mid = 0` reads the *last* element silently.
- **The proof is one sentence: the array is finite.** Going uphill either reaches the end (which beats $-\infty$) or turns over (which is a peak). Invariant: `[lo, hi]` always contains a peak.
- **while `lo < hi` with `hi = mid`** — day 043’s pairing. Non-strict `<=` here hangs; `hi = mid - 1` throws away the answer.
- **A peak is not the maximum.** The maximum is $O(n)$ and unbeatable; a *local* best is $O(\log n)$ — the weakening is what buys the logarithm. Equal adjacent values kill it ($O(n)$); 2D is `O(rows * log cols)` by searching columns and taking each column’s max row.

1 What this is, and why they ask it

Composition means an object gets its behaviour by holding other objects and asking them, rather than by inheriting it. A `Vehicle` does not *become* electric by extending `ElectricVehicle`; it *holds* a `FuelSystem`, and that object happens to be an electric one. The relationship is has-a rather than is-a, and the held object can be swapped at run time, tested on its own, and shared by things that are otherwise unrelated.

They ask you to refactor because the slogan — “favour composition over inheritance” — is repeated everywhere and demonstrated rarely. Anyone can say it. What separates candidates is being able to take a concrete hierarchy, name the specific pain in it, produce the composed version, and then say what improved in units that mean something: classes needed, files touched to add a variant, whether the behaviour can be changed after construction. This is also the payoff day for the phase. [Day 046](#) told you the three ways inheritance goes wrong; every one of them ended at the same fix, and this is it.

2 The story

Yashoda makes idli and dosa batter at home in Basavanagudi and supplies four small hotels, about thirty kilos a day, and she has been doing it since her husband’s factory closed in 2011.

The machine she works with is an ordinary mixie base — the motor unit, heavy, with the switch and the three speeds. It has been on the same shelf for nine years.

On top of it she screws whichever jar the job needs. The big jar for grinding, the small chutney jar, and a third dry jar she bought later for masala powders. Same base underneath all three. The same grooves cut into the rim, the same click when it seats.

Two things happened that made her notice the arrangement rather than just use it.

The blade in the big jar went in the second year — she had been running it eight hours a day and it gave up, which is fair. She took the jar to the shop on the main road, they had one, four hundred and something rupees, and she was grinding again the same evening. The motor never left the shelf.

Then in the fourth year she started doing the masala powders, which is a different job entirely — dry, and it needs a tighter, smaller jar. She did not buy a machine for it. She bought a jar.

Her neighbour, who started around the same time, bought one of the all-in-one machines. Good machine, does everything, and the jar and the motor are one sealed unit. When her blade went, the service man came, looked at it, and said the whole head has to be replaced, and the head is most of the price of the machine. She waited eleven days for the part and lost eleven days of orders.

The thing Yashoda says about it, when the subject comes up, is that her machine is not better than her neighbour's. It grinds no finer and it is not quieter. It is just that hers comes apart, and so when one part of it fails, or when the work changes, only that part has to change.

3 The idea in plain English

The motor base is the object. The jars are the components it holds. And the neighbour's sealed unit is the class hierarchy where a variant is welded into the type.

The same thing, both ways

Take vehicles that vary by size and by fuel. With inheritance:

```
class Vehicle: ...
class Car(Vehicle): ...
class Truck(Vehicle): ...

class ElectricCar(Car): ...
class PetrolCar(Car): ...
class ElectricTruck(Truck): ...
class PetrolTruck(Truck): ...
```

Two axes, so the class count is their product: $2 \times 2 = 4$, plus the two abstract parents. Add hybrids and it is 6. Add “with trailer” and it is 12. And the electric refuelling logic is written twice — once in `ElectricCar` and once in `ElectricTruck` — because there is nowhere shared to put it.

With composition:

```

class FuelSystem(Protocol):
    def refuel(self, amount: float) → None: ...
    def range_km(self) → float: ...

class PetrolTank:
    def refuel(self, litres: float) → None: ...
    def range_km(self) → float: ...

class Battery:
    def refuel(self, kwh: float) → None: ...
    def range_km(self) → float: ...

class Vehicle:
    def __init__(self, size: Size, fuel: FuelSystem) → None:
        self._size = size
        self._fuel = fuel                                # HOLDS one, rather than BEING one

    def range_km(self) → float:
        return self._fuel.range_km()                    # delegation: pass the question along

```

Now 2 sizes + 2 fuel systems = 4 classes, and an electric truck is `Vehicle(Size.TRUCK, Battery(...))` — an *object*, not a class. Adding hybrids is one new class, not three. The electric logic exists once.

The two words to have

Delegation is what `range_km` does: the object receives a request and passes it to the component that knows the answer. It is one line, and it is the mechanical heart of composition.

Injection is what the constructor does: the component is handed in from outside rather than chosen inside. That is the same rule as [yesterday's](#) payment gateway, and for the same reason — the moment `Vehicle.__init__` writes `self._fuel = Battery()`, every vehicle is electric and the flexibility is gone.

What you actually gain

Four things, and they are worth listing separately because each is a different kind of win.

One: the class count stops being a product. Sizes plus fuels plus trailers, instead of sizes times fuels times trailers. §6 does the arithmetic.

Two: behaviour can change after construction. A subclass is fixed at the moment the object is made — an `ElectricCar` is an `ElectricCar` forever. A held component can be replaced:

```

vehicle.set_fuel(Battery(kwh=60))    # the same vehicle, converted

```

Whether you want that is a design decision. That you *can* is something inheritance simply cannot offer.

Three: the parts are testable and reusable on their own. `Battery` can be tested with no `Vehicle` anywhere, and the same `Battery` class can be held by a `Scooter`, a `Bus` and a `ForkLift`, which sit in completely different parts of the hierarchy — or in no hierarchy at all.

Four: the coupling is a stated interface, not the whole parent. With inheritance the child depends on everything the parent does, including *how* it does it — the fragile base class from [day 046](#). With composition the vehicle depends on two method signatures and nothing else.

The cost, stated plainly

Composition is not free, and pretending otherwise is how candidates lose credibility.

- **More wiring.** Someone must construct the parts and hand them over. Four objects instead of one `new ElectricCar()`.
- **Delegation boilerplate.** If a vehicle needs to expose eight methods of its fuel system, that is eight one-line forwarding methods. Inheritance gives them free.
- **An extra hop when reading.** `vehicle.range_km()` calls `self._fuel.range_km()`, so understanding one call means opening two files. Inheritance has the same problem in a different direction, but it is a real cost.

When inheritance still wins

The slogan is “favour”, not “always”. Four cases, unchanged from [day 046](#):

- **Exception hierarchies.** `PaymentDeclined(PaymentError)` — no state, no implementation, pure substitutability.
- **Abstract base classes as contracts.** The parent has no implementation to be fragile about.
- **Documented framework extension points.** `logging.Handler`, Django’s `ListView`.
- **A genuine single-axis specialisation** where nothing else will ever vary.

The practical rule: **inherit interfaces freely, compose implementations.**

The middle ground: mixins

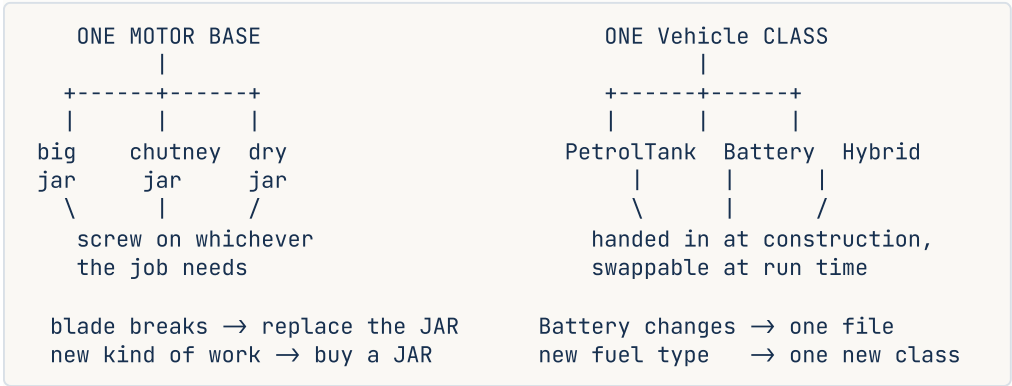
A **mixin** is a small class carrying one piece of behaviour, meant to be inherited alongside others:

```
class TimestampMixin:
    def touch(self) → None:
        self.updated_at = datetime.now(UTC)
```

It sits between the two. It is inheritance, so it has the fragile-base risk; but each mixin is small, stateless and owns a distinct method, so the risk is small too. Mixins work when there are three of them and each touches a different method. They stop working when two of them override the same method and the resolution order decides which wins — at which point nobody reading the code can predict the behaviour, and composition is the honest answer.

4 The picture

The mixie, and the object:



What to notice: the base is unchanged in every scenario. In the sealed-unit design — and in the class hierarchy — a change to one part is a change to the whole thing.

The two designs, counted:

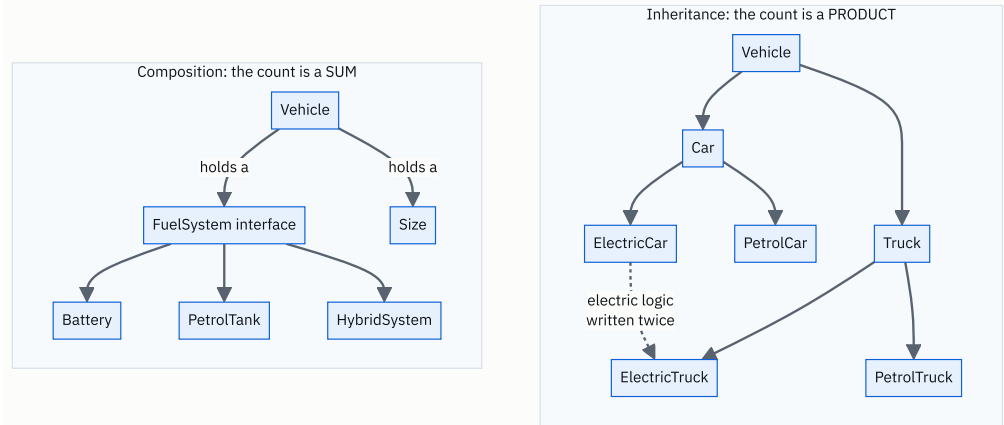


FIGURE 49.1

What to notice: in the top box, `ElectricCar` and `ElectricTruck` share behaviour with no shared home for it, so it is duplicated. In the bottom box `Battery` exists once and anything can hold it — including classes that are not vehicles at all.

5 How it actually works

The refactoring, step by step

This is the recipe to run out loud when handed a hierarchy.

1. **Find the axes.** Write down what actually varies. If the class names read like `AdjectiveNoun` — `ElectricCar`, `PremiumSubscriber`, `UrgentEmailNotification` — the adjective is the second axis and it is the one to extract.
2. **Pick the axis that is not the identity.** A car is a car; being electric is something it *has*. Keep the identity as the class, extract the rest.
3. **Name the extracted thing as a capability, not as a variant.** `FuelSystem`, not `ElectricBits`. The name should describe what the component does for anyone who holds it.
4. **Declare the interface** — a Protocol, or an abstract base class. Two or three methods.
5. **Move each subclass's differing code into a component class**, unchanged.
6. **Give the base class a field and delegate to it**, and take the component in the constructor.
7. **Delete the subclasses.** This is the step people skip, and leaving them means both designs exist at once, which is worse than either.

Real designs built this way

- **Python's `logging`.** A `Logger` holds `Handler`s; each `Handler` holds a `Formatter` and `Filter`s. None of that is inheritance between the pieces — it is objects holding objects, which is why you can put a `JSONFormatter` on a `FileHandler` and a plain one on a `StreamHandler` in the same program.
- **`sorted(items, key=...)`.** The comparison behaviour is handed in as a function. That is composition at the smallest possible scale, and it is why one `sorted` serves every ordering anyone will ever want, rather than there being a `SortedByName` class.
- **Django's `Storage`, and every payment SDK**, from [yesterday](#). The client holds a backend.
- **Go has no inheritance at all.** It has struct embedding and interfaces, and the entire standard library is built by composing small interfaces — `io.Reader`, `io.Writer` — into bigger ones. It is the strongest existence proof that the slogan is practical rather than aspirational.

- **React's component model.** The team's own documentation says to prefer composition to inheritance for reusing UI behaviour, and the framework offers no inheritance mechanism for it.
- **The classic teaching example,** worth knowing by name: a `Duck` with a `FlyBehaviour` and a `QuackBehaviour` held as fields, rather than `FlyingDuck` and `RubberDuck` subclasses. It is from *Head First Design Patterns*, and interviewers of a certain vintage will recognise it.

Delegation, and the boilerplate question

```
class Vehicle:
    def __init__(self, fuel: FuelSystem) → None:
        self._fuel = fuel

    def range_km(self) → float:
        return self._fuel.range_km()

    def refuel(self, amount: float) → None:
        self._fuel.refuel(amount)
```

Two forwarding methods. If there were twelve, the boilerplate would be a real argument against, and there are three answers:

- **Do not forward at all** — expose the component: `vehicle.fuel.range_km()`. Simple, and it leaks the component into every caller, so a change of interface reaches everywhere.
- **Forward only what the outside actually needs.** Usually two or three of the twelve, and this is almost always the right answer.
- **`__getattr__` for automatic forwarding.** It works and it is a bad idea in most codebases: it forwards methods you did not intend, breaks autocompletion, and turns a typo into a confusing runtime error instead of an `AttributeError` at the right place.

Testing, which is where it pays

```
class FakeFuel:
    def range_km(self) → float:
        return 0.0 # simulate an empty tank

def test_vehicle_warns_when_range_is_zero():
    v = Vehicle(Size.CAR, FakeFuel())
    assert v.needs_refuel()
```

With the inheritance version, testing the empty-tank case means constructing a real `PetrolCar` and manipulating its internals until the tank is empty — reaching past encapsulation to set up a test, which is a smell in itself. With composition you hand in a

component that answers however you like. **A design where the parts are separately constructible is a design that is separately testable**, and that is the practical reason experienced people prefer it.

6 The numbers

The class count: product against sum

axes of variation	inheritance	composition
-----	-----	-----
3 sizes	3	3
3 sizes x 2 fuels	6	3 + 2 = 5
3 x 2 x 2 (trailer)	12	3 + 2 + 2 = 7
3 x 2 x 2 x 3 (ownership)	36	3 + 2 + 2 + 3 = 10
adding a 3rd fuel type:	+6 classes	+1 class
adding a 5th axis with 2 values: x2 = 72 classes		+2 classes

Thirty-six against ten, and the growth rates are the real story: one multiplies, the other adds. This single table is the most persuasive thing you can put in front of an interviewer on this subject.

Edits to add a variant

inheritance, adding "hybrid":	
HybridCar	→ new file, duplicating some of ElectricCar
HybridTruck	→ new file, duplicating the same thing again
HybridBike	→ new file, a third copy
any factory / registry that maps names to classes	→ edit

	3 new files (2 duplicated), 1 edit
composition, adding "hybrid":	
HybridSystem implementing FuelSystem	→ 1 new file
the construction site	→ 1 line

	1 new file, 1 line

Duplication, counted

electric-specific logic in the inheritance version:	
ElectricCar.refuel	~25 lines
ElectricTruck.refuel	~25 lines (near-identical)
ElectricBike.refuel	~25 lines (near-identical)

	75 lines, 3 places to fix a bug
composition:	25 lines, 1 place

Run-time cost of the extra hop

direct method call	~60 ns
delegated call (one extra hop)	~120 ns
two levels of delegation	~180 ns
10,000 delegated calls in a request:	~1.2 ms
one database round trip:	~1 ms

Comparable to a single query, at ten thousand calls. Real, and almost never the constraint — say the number rather than dismissing the question.

7 The trade-offs

Flexibility against wiring

Every component must be constructed and passed in, so object creation gets longer and something has to know how to assemble the pieces. *I would not compose a class that has exactly one variant and always will* — a `Ticket` does not need a `TicketNumberingStrategy`. The trigger is a second axis, or a second implementation you can name.

Composition against the readability of one class

An `ElectricCar` class tells you what it is from its name. A `Vehicle` holding a `Battery` tells you at the construction site, which may be in a different file. *I would not scatter a design across six tiny classes when two would do* — small classes are not automatically better, and a component with one method and one implementation is usually a field pretending to be an object.

Delegation boilerplate against exposing the component

Twelve forwarding methods is noise; `vehicle.fuel.range_km()` leaks the component's interface into every caller, so changing `FuelSystem` reaches everywhere. *I would forward only the two or three methods the outside genuinely needs, and expose the component itself only inside the same module.* And I would not reach for `__getattr__` — automatic forwarding breaks tooling and hides mistakes.

Composition against mixins

Mixins give you shared behaviour with no wiring at all, which is genuinely convenient. They are still inheritance, so the fragile-base risk applies. *I would use a mixin only when it is small, stateless, and owns a method no other mixin touches* — and the moment two mixins override the same method and the resolution order decides the winner, I would convert to composition, because nobody reading the code can predict which one runs.

The honest sentence

The point is not that inheritance is bad. It is that inheritance welds a variant into the type, and a type cannot be changed after the object is built or shared with something in a different part of the tree. Composition costs you some wiring and buys you parts that come apart — which is what you want on the day one of them fails, or the work changes.

8 In the interview

How it gets asked

- “Refactor this class hierarchy. Why is your version better?” — the direct form, usually with a four-to-eight-class tree on screen.
- “You have three vehicle types and two fuel types. Model it.” — the same question before the bad version exists, and the trap is to draw six classes.
- “Why not just subclass it?” — the pushback in a design round, when you have proposed a component.
- “When would you still use inheritance?” — the balance check, and answering “never” fails it.

What to say out loud, in the first ninety seconds

1. **Name the axes before touching anything.** “Looking at these class names — `ElectricCar`, `PetrolCar`, `ElectricTruck` — there are two axes here: what the vehicle is, and how it’s powered. Inheritance only has one axis, which is why the count is a product.”
2. **Say which axis is the identity.** “A car is a car. Being electric is something it has, not something it is. So `Car` stays a class and the fuel becomes a component.”
3. **Name the component as a capability.** “I’d call it `FuelSystem`, with `refuel` and `range_km` — named for what it does for whoever holds it, not for which variant it is.”
4. **Give the count.** “That takes six classes to four — three sizes plus two fuel systems instead of three times two. And adding hybrids is one class instead of three.”
5. **Name the duplication you removed.** “Right now the electric refuelling logic is written in `ElectricCar` and again in `ElectricTruck`, because there’s nowhere shared to put it. After this it exists once.”
6. **Concede the cost.** “What I’ve added is wiring — something has to construct the fuel system and hand it in — and one delegating method. That’s the trade.”

The follow-ups

“Give me the concrete win, not the principle.” Four wins, and I would give them as separate things because they are different kinds of benefit. First, the class count stops multiplying: three sizes and two fuel types is five classes composed against six inherited, and by the time there are four axes it is ten against thirty-six — one grows by addition, the other by multiplication. Second, the duplication goes: right now the electric refuelling logic sits in `ElectricCar` and again in `ElectricTruck`, roughly twenty-five lines each, because those two classes are in different branches and there is no shared home for it — so a bug there is fixed in two places or one. Third, behaviour becomes changeable after construction. An `ElectricCar` is an `ElectricCar` for the life of the object; a `Vehicle` holding a `Battery` can be handed a different fuel system, which matters for a fleet system that models conversions and for tests. Fourth, and the one I care most about in practice, the parts become separately testable: to test the empty-tank path with inheritance I have to build a real `PetrolCar` and reach into its internals to drain it, which means breaking encapsulation to set up a test. With composition I hand in a fake fuel system that reports zero range, in two lines.

“When would you still use inheritance?” Four cases, and they share one property. Exception hierarchies — `PaymentDeclined` extending `PaymentError` — because the parents carry no state and no implementation, so there is nothing for a child to depend on the internals of, and catching the family is exactly what substitutability is for. Abstract base classes used as contracts, for the same reason: the parent is signatures only. Framework extension points that were designed to be subclassed and document which method to override, like `logging.Handler` and its `emit`. And a genuine single-axis specialisation where nothing else will ever vary — a `SavingsAccount` that differs from `Account` only in interest calculation. The property those four share is that **the parent has little or no implementation**, which is where the fragile-base problem lives. So the rule I actually use is: inherit interfaces freely, compose implementations, and never inherit purely to avoid retyping code — that is the case where the coupling buys nothing.

“Isn’t composition just more code and more indirection?” Yes to both, and I would rather concede it than argue. There is more wiring, because someone has to construct the parts and hand them over — four objects where there was one constructor call. There is delegation boilerplate, one forwarding line per method the outside needs. And there is an extra hop when reading: `vehicle.range_km()` calls `self._fuel.range_km()`, so understanding one call means opening two files. What I would say back is that inheritance has the same reading cost in a different direction — understanding a method on a class four levels deep also means opening four files, and worse, you cannot tell from the call site which level it came from. And I would keep the boilerplate small deliberately: forward only the two or three methods that actually cross the boundary, rather than mirroring the component’s whole interface, and definitely not with `__getattr__`, which forwards

things you did not intend and breaks tooling. The trade is a few lines of wiring now against a class count that multiplies later, and I would take that, while agreeing it is a trade rather than a free win.

A model answer

“Let me name what varies before I touch the code, because that’s what decides the shape.

These class names are `ElectricCar`, `PetrolCar`, `ElectricTruck`, `PetrolTruck` — an adjective and a noun. That means two axes: what the vehicle is, and how it’s powered. Inheritance has one axis, so the class count is the product of the two, and it’s already six with the parents. Add hybrids and it’s nine. Add trailers and it’s eighteen.

There’s a second symptom already visible: the electric refuelling logic is written in `ElectricCar` and again in `ElectricTruck`, because those two are in different branches and there’s no shared place to put it. About twenty-five lines, duplicated, so a bug there gets fixed once or twice depending on who’s looking.

So: which axis is the identity? A car is a car. Being electric is something it *has*. So the size stays as the class, or even just as a field, and the fuel becomes a component:

```
class FuelSystem(Protocol):
    def refuel(self, amount: float) → None: ...
    def range_km(self) → float: ...

class Vehicle:
    def __init__(self, size: Size, fuel: FuelSystem) → None:
        self._size, self._fuel = size, fuel

    def range_km(self) → float:
        return self._fuel.range_km()
```

I’ve named it `FuelSystem` rather than `ElectricBits` deliberately — it’s a capability, described by what it does for whoever holds it.

Now an electric truck is `Vehicle(Size.TRUCK, Battery(...))` — an object, not a class. The count goes from six classes to four, and by the fourth axis it’s ten instead of thirty-six. Adding hybrids is one new class and one line where vehicles are constructed, instead of three new classes duplicating each other.

Two more things I get. The fuel system can be swapped after construction, which a subclass can never be. And I can test the empty-tank path by handing in a fake that reports zero range, instead of building a real `PetrolCar` and reaching into its internals to drain it.

The cost, honestly: more wiring at construction, one delegating method per thing the outside needs, and an extra hop when reading the code. I’d forward only the two or three methods that actually cross the boundary rather than mirroring the whole

interface.

And I'd still use inheritance in four places — exception hierarchies, abstract base classes as contracts, documented framework hooks, and a genuine single-axis specialisation. What those share is that the parent has almost no implementation, which is where the fragile-base problem lives. Inherit interfaces freely; compose implementations.”

9 Recall card

- **Inheritance welds a variant into the type; composition holds it.** `Vehicle(Size.TRUCK, Battery())` is an *object*, not a class — and it can be swapped after construction, which a subclass never can.
- **The count is the argument:** inheritance multiplies the axes ($3 \times 2 \times 2 \times 3 = 36$), composition adds them ($3 + 2 + 2 + 3 = 10$). A new fuel type is +6 classes against +1.
- **The refactor, in order:** find the axes (adjective-noun class names are the tell) → keep the identity as the class → name the rest as a *capability* (`FuelSystem`, not `ElectricBits`) → declare the interface → move each subclass's code into a component → delegate → **delete the subclasses**.
- **The practical payoff is testing:** hand in a `FakeFuel` returning zero range, instead of building a real object and reaching past encapsulation to set up the case. Separately constructible means separately testable.
- **Concede the cost** (wiring, one forwarding line per exposed method, an extra hop) and **keep inheritance for exception trees, ABCs as contracts, documented framework hooks, and true single-axis specialisation** — all cases where the parent has almost no implementation. *Inherit interfaces, compose implementations.*

PRACTICE

Practice — Peak finding, and searching data that is structured but not sorted

DSA topic: Peak finding, and searching data that is structured but not sorted **System design topic:** Composition over inheritance

Code these, in this order

One rule today: **for every problem, say the invariant and the discard proof out loud before writing a line.** These problems have no target and no monotone question — the proof is all you have, and if you cannot state it you are guessing.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Peak Index in a Mountain Array	LeetCode 852 (Medium)	The rule, with the answer guaranteed unique so nothing can distract you.
2	Find Peak Element	LeetCode 162 (Medium)	The same code with the guarantee removed — and whether you accept a non-unique answer.
3	Find Minimum in Rotated Sorted Array	LeetCode 153 (Medium)	The mirror: a valley. Re-derive it as local structure, not as a rotation trick.
4	Find a Peak Element II	LeetCode 1901 (Medium)	The 2D version, and whether you search columns and take each column's maximum row.

On problem 1, do the proof first

Before typing: *if it rises to the right of the middle, either it climbs to the end — and the last element beats the imaginary negative infinity beyond it — or it turns over somewhere, and the turning point is a peak. Either way a peak exists to the right.* Then write four lines. Then confirm the guarantee that makes the answer unique changes nothing about the code.

On problem 2, resist checking your answer against the maximum

Run it on `[1, 3, 2, 4, 7, 9, 5, 6, 2]`. It returns index 7, value 6 — and 9 is in the array. That is correct. Write the `is_peak` helper from §5 and use it to check your answers, rather than comparing against `max(nums)`. If you find yourself wanting the maximum, say out loud why that is $O(n)$ and cannot be beaten.

On problem 3, do not look at day 045

Re-derive it as a valley: the minimum is the one element smaller than both neighbours, with the wrap-around counted. Compare `nums[mid]` against `nums[hi]`, and say why comparing against `nums[lo]` returns the maximum on an un-rotated array. Then notice

you have written today's code with one comparison flipped.

On problem 4, get the axis right

Search over the **columns**, and inside each candidate column take the row holding that column's largest value. That element already beats its up and down neighbours for free, which is what collapses the problem back to one dimension. Write the version that scans the *row* instead and see it fail — then say precisely why the discard is not justified in that version.

The proof drill

Say each out loud, in under thirty seconds:

1. Why does every array have at least one peak?
2. Why does “it rises to the right” guarantee a peak to the right? Name the two cases and the fact that rules out a third.
3. Why is `hi = mid` and not `hi = mid - 1` in the falling branch?
4. Why is `mid + 1` always a valid index?
5. Why is the loop condition strictly `<`?

The breakage drill

Produce each failure yourself and record what happens — a crash with a named exception, a silent wrong answer, or a hang:

1. `while lo ≤ hi` with `hi = mid`, on `[1, 2, 3]`.
2. `hi = mid - 1` in the falling branch, on `[1, 2, 1, 3, 5, 6, 4]`.
3. Comparing with `nums[mid - 1]` instead of `nums[mid + 1]`, on `[1, 2]`.
4. `[1, 2, 2, 2, 1]` on the correct solution — what does it return, and is it a peak?

Number 4 is not a bug in your code. Say whose problem it is and what you would have asked for.

The sortedness drill

For each, say whether binary search applies, and what the discard proof is:

1. A sorted array, find a target.
2. A rotated sorted array, find a target.
3. An unsorted array, find any peak.
4. An unsorted array, find the maximum.
5. An unsorted array of distinct values, find the third largest.
6. A monotone yes-or-no question over a range of capacities.

Two of these six have no discard proof. Name them and say what the cost is instead.

The hierarchy refactor drill

Take this and rework it out loud in five minutes:

```
class Notification: ...
class EmailNotification(Notification): ...
class SmsNotification(Notification): ...
class UrgentEmailNotification(EmailNotification): ...
class UrgentSmsNotification(SmsNotification): ...
class ScheduledEmailNotification(EmailNotification): ...
class ScheduledSmsNotification(SmsNotification): ...
```

1. Name the axes. How many are there, and what is the tell in the class names?
2. Which axis is the identity and which are things the object *has*?
3. Name each extracted component as a capability, not as a variant.
4. Count the classes now, and after adding a third channel, and after adding a fourth delivery timing.
5. Name the duplication that exists in the current version and where it goes.
6. Write the construction line for an urgent SMS.
7. Say what you gave up.

The count drill

Fill in the table from memory, then check it:

axes	inheritance	composition
3 channels	?	?
3 channels x 3 timings	?	?
3 x 3 x 2 (with attachment)	?	?
adding a 4th channel	? new classes	? new classes

The still-inheritance drill

For each, say compose or inherit, with the one-sentence reason:

1. `CardDeclined` and `PaymentError`.
2. `ElectricCar` and `Car`, in a system that also varies by size.
3. `JsonFormatter` and `logging.Formatter`.
4. `PremiumSubscriber` and `Subscriber`, where the difference is a discount rate.
5. `RetryingHttpClient` and `HttpClient`.
6. `TimestampMixin` on four unrelated model classes.

Say the one property shared by every “inherit” answer.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find a peak element in $O(\log n)$.* Deal with the sortedness objection first, confirm the two conventions, state that an answer always exists, give the rule with its proof in one breath, and close on the peak-versus-maximum contrast.
 2. *Refactor this class hierarchy. Why is your version better?* Name the axes from the class names, pick the identity, name the capability, give the count both ways, name the duplication removed, and concede the cost.
 3. *Why does binary search work here? There is no order.* The general statement — binary search needs a discard proof, not sortedness — then this problem's proof, then two other days where the proof was something else.
-

Before you move on

- [] I stated the discard proof before writing code on all four problems.
- [] I checked my peak answers with `is_peak`, not against `max(nums)`.
- [] I produced all four breakages and can say which is a crash, which is silent, and which hangs.
- [] I can say why finding the maximum is $O(n)$ and cannot be beaten.
- [] I refactored the notification hierarchy and gave the class count for four axes both ways.
- [] I can name the property shared by every case where inheritance is still right.
- [] I answered all three questions above out loud.

Binary search revision and mock round

DATA STRUCTURES AND ALGORITHMS

Binary search revision and mock round

You can solve two unseen binary search problems, including one search-on-answer.

SYSTEM DESIGN

Class diagrams and the UML you will actually draw

You can draw a class diagram on a whiteboard that an interviewer can read.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *Draw the class diagram for what you just described.*

Binary search revision and mock round

1 What this is, and why they ask it

The binary search phase ends today. Eight days, six distinct shapes, and one template that has not changed since [day 043](#). Today is not new material. It is the drill that turns eight lessons into one act you can perform cold: **look at a problem you have never seen, name which of the six it is, and start.**

Naming is a separate skill from solving, and it is the one interviews actually test. A candidate who knows all six templates and cannot tell which one a problem wants will spend twenty minutes on a greedy approach that does not work. The naming takes ten seconds and it decides the next thirty minutes. So today has two halves: the recognition drill, and a mock round run under real conditions — two unseen problems, talking the whole time, no hints.

2 The story

Yesudas has taken the choir at the church in Thrissur for nineteen years, and the Christmas service is on the twenty-fourth.

From September they practise on Wednesdays and it is all stop-and-fix. The altos come in half a beat late on the second piece, so they do that entry eleven times. Somebody is flat on a held note, so they sit on that note. That is what a practice is for and it works.

Then, three weeks out, he changes what happens in the room, and the change is the whole of what he knows about this.

The first change is that he stops correcting. From the first week of December they sing each piece straight through, and whatever goes wrong, they carry on. He does not lift a hand. He told them years ago why: nobody has ever ruined a service by singing a wrong note. What ruins it is that somebody sings a wrong note and stops, and then the two beside her stop, and there are four seconds of nothing in a full church. Recovering while continuing is a different skill from getting it right, and it cannot be practised in a room where you are allowed to stop.

The second change is smaller and he thinks it matters more. He stops telling them what is next.

For three months the order has been on the board. Now he simply stands up and raises his hand, and the first chord is the only clue anyone gets. And the thing he is watching, in those first two seconds, is the row of faces — who has recognised the piece and taken a breath, and who is still looking at the person next to them to work out which one it is.

Because on the night, the pieces will not come in the order anybody rehearsed. The priest will run long, one piece will be dropped, the collection will take four minutes instead of two, and Yesudas will turn round and start something. If they need three seconds to work out what it is, the opening is gone.

Nineteen years, and he has never once had to correct a note in the last week. What he corrects, in the last week, is the two seconds before the singing starts.

3 The idea in plain English

Yesudas’s last three weeks are today. The full run-through without stopping is the mock. And the two seconds before the first note — recognising which piece from one chord — is the naming drill, which is the thing this phase has actually been building toward.

The six chapters, and the tell for each

Every binary search problem you will be handed is one of these. Learn the left column; the right column is the eight days you have already done.

THE TELL IN THE PROBLEM	THE CHAPTER
"sorted array" + "find the index of"	plain search / lower_bound (042, 043)
"first and last" / "how many times does x appear" / "how many are in [a, b]"	two bounds, one char apart (044)
"rotated at an unknown pivot"	one half is sorted; discard on a RANGE, not a comparison (045)
"not sorted" + "a peak" / "the minimum of a rotated array"	local structure + a discard proof; no order needed (049, 045)
"the smallest X such that it is possible" / "minimise the maximum" / "maximise the minimum" / a capacity, speed, rate or days	binary search on the ANSWER bound the range, write works(x), check monotonicity (046, 047)
"to six decimal places" / a real-valued answer	float search, FIXED iteration count, hi = max(1.0, x) (048)

The three exits — problems that look like binary search and are not

Naming when *not* to use it is worth as much as naming when to.

- **“Exactly k groups.”** The word *exactly* usually kills monotonicity — if 5 works, 6 may not, and 7 may. Check by asking “if x works, does every larger x work?” If you cannot answer yes with a reason, stop.
- **Duplicates in a rotated array, or a plateau in a peak search.** When `nums[lo] == nums[mid] == nums[hi]`, no comparison carries information, so no half can be discarded. `O(n)` worst case, and it is a property of the problem, not a weakness in your

code.

- “The maximum of an unsorted array.” $O(n)$ and unbeatable — any element you skip could be the largest. Compare with “a peak”, which is $O(\log n)$. That contrast is the single sharpest thing you can say in this phase.

And one more: **a single query on unsorted data**. Sorting first costs $O(n \log n)$, which is worse than one $O(n)$ scan. Binary search earns its place from the second or third query onward.

The one template, which has not changed

```
def first_true(lo: int, hi: int, question) → int:
    """Smallest i in [lo, hi) with question(i) True, else hi."""
    while lo < hi:
        mid = (lo + hi) // 2
        if question(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Six lines, written on day 043, unmodified since. Everything in the phase is this with a different question:

question(i) = nums[i] ≥ target	→ lower bound (first occurrence, insert position, exists)
question(i) = nums[i] > target	→ upper bound (last occurrence = -1, count = upper - lower)
question(c) = days_needed(c) ≤ d	→ smallest workable capacity
question(g) = not can_place(g)	→ largest workable gap, minus one
question(i) = nums[i] ≤ nums[-1]	→ the minimum of a rotated array
question(i) = nums[i] > nums[i + 1]	→ a peak

Six problems, one loop. If you find yourself writing a second loop shape, stop and ask what question you should have written instead.

The five bugs of the phase

Every failure in eight days has been one of these. Learn them as a pre-flight list, with the input that catches each.

One: mixing the two conventions. Closed `[lo, hi]` goes with `while lo ≤ hi` and `± 1`. Half-open `[lo, hi)` goes with `while lo < hi` and `hi = mid`. Blend them and you get either an infinite loop or an `IndexError`. *Checking input:* a two-element array.

Two: `hi = mid - 1` where `mid` might be the answer. In every boundary search, the middle is a candidate when the question answers True. *Checking input:* an array whose answer is at the position the first midpoint lands on.

Three: discarding on a single comparison instead of on a proof. Plain binary search gets away with `nums[mid] < target` because the array is sorted. Rotated arrays and peaks do not — the discard must be justified by a range or by an argument about what must exist. *Checking input:* `[9, 11, 1, 3, 5, 7]` searching for 9.

Four: a range that does not contain the answer. `hi = x` for the square root of 0.25. An upper capacity that is too small. The function returns the edge of its own range with total confidence and no error. *Checking habit:* assert that `works(hi)` is True before the loop.

Five: `lo` set to a meaningless value. A rate of 0 raises `ZeroDivisionError`; a capacity of 0 carries nothing. The lower bound is the smallest *meaningful* value, not the smallest number. *Checking input:* whatever makes the check divide.

The recognition minute

Before writing anything, sixty seconds, out loud:

1. **Which chapter?** Name it from the tells above.
2. **Is the precondition true?** Sorted — on which key? Monotone — say the sentence. Distinct adjacent values?
3. **What is the range?** Indices, or answers? Both ends, with a reason each.
4. **What is the question?** Write `works(x)` as one English sentence before it is code.
5. **What does the return value mean?** An index, a position, a value — and what must the caller check before using it?

Say those five out loud and the code takes ninety seconds, because every decision is already made.

4 The picture

The phase, as a decision you run in ten seconds:

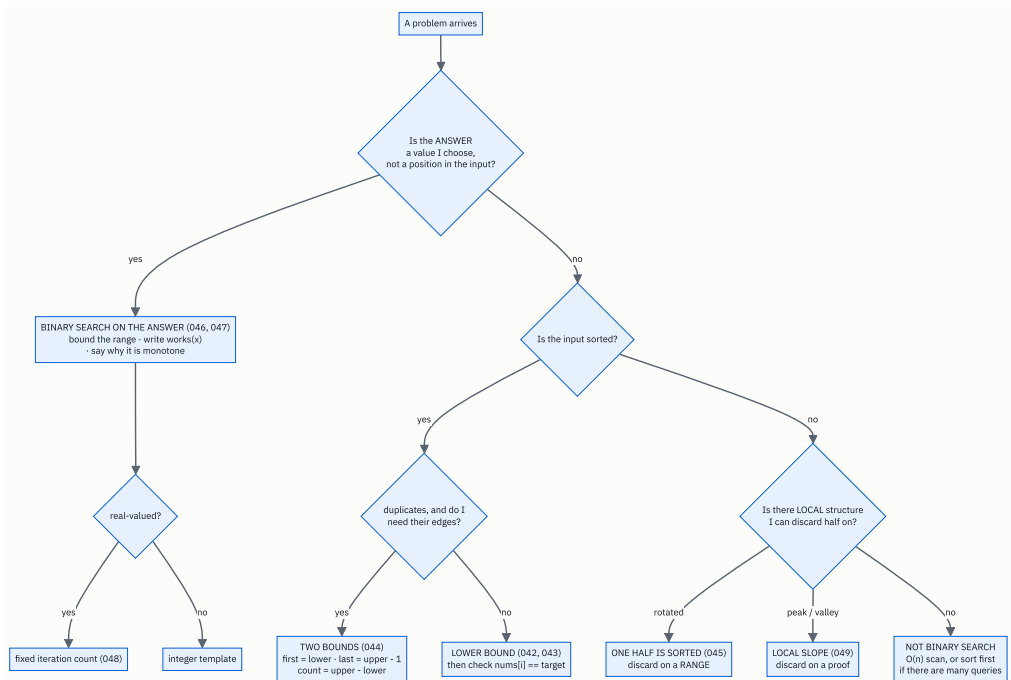


FIGURE 50.1

What to notice: the very first question is not “is it sorted?” It is “what is the answer?” That one reordering is what stops candidates missing the search-on-answer family, which is where the hardest questions live.

The costs of the whole phase, on one card:

tool	time	space
plain search / lower bound	$O(\log n)$	$O(1)$
two bounds (first + last + count)	$2 \log n = O(\log n)$	$O(1)$
rotated, distinct values	$O(\log n)$, ~3 cmp/pass	$O(1)$
rotated, with duplicates	$O(n)$ worst case	$O(1)$
peak (1D)	$O(\log n)$	$O(1)$
peak (2D, $r \times c$)	$O(r \times \log c)$	$O(1)$
search on the answer	$O(n \times \log(\text{range}))$	$O(1)$
float search	$O(1)$ -- fixed 100 passes	$O(1)$
sorting first, if the input is not sorted	$O(n \log n)$	$O(n)$ or $O(1)$ in place

What to notice: every row is $O(1)$ space. If your solution to a binary search problem allocates, something has gone wrong — the one legitimate exception is sorting a copy of the input.

5 The code, built step by step

The pre-flight, written as code

Before the mock, put this in your head. It is not a function you will write in an interview; it is the shape of the sixty seconds.

```
# 1. chapter?      answer-space → works(x) | sorted → bounds | local structure → proof
# 2. precondition? sorted on WHICH key? monotone (say why)? adjacent values distinct?
# 3. range?        lo = smallest MEANINGFUL, hi = certainly-works (or len(nums))
# 4. question?     one English sentence, then code
# 5. return means? index / position / value -- and what must the caller check?
```

The three loops, side by side

Everything in the phase is one of these three shapes. Know which you are writing and why.

```
# A. boundary, half-open. The default. Use it unless you have a reason not to.
while lo < hi:
    mid = (lo + hi) // 2
    if question(mid):
        hi = mid
    else:
        lo = mid + 1
return lo
```

```
# B. closed range, three-way. For "find this exact value and return -1 if absent",
#    and for the rotated search where mid must be compared to the target directly.
while lo ≤ hi:
    mid = (lo + hi) // 2
    if nums[mid] == target:
        return mid
    if nums[mid] < target:
        lo = mid + 1
    else:
        hi = mid - 1
return -1
```

```
# C. floats. No condition at all -- a fixed count.
for _ in range(100):
    mid = (lo + hi) / 2
    if predicate(mid):
        lo = mid
    else:
        hi = mid
return (lo + hi) / 2
```

A is the one to reach for. B exists because rotated search genuinely needs the equality branch. C exists because a continuous range never empties.

The phase, in one runnable file

```
from bisect import bisect_left, bisect_right

# ----- chapter 1: sorted, find a target -----
def search(nums: list[int], target: int) → int:
    """Index of target, or -1. Day 042/043."""
    lo, hi = 0, len(nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] ≥ target:
            hi = mid
        else:
            lo = mid + 1
    return lo if lo < len(nums) and nums[lo] == target else -1

# ----- chapter 2: duplicates, find both edges -----
def search_range(nums: list[int], target: int) → list[int]:
    """First and last index of target, or [-1, -1]. Day 044."""
    first = bisect_left(nums, target)
    if first == len(nums) or nums[first] ≠ target:
        return [-1, -1]
    return [first, bisect_right(nums, target) - 1]

# ----- chapter 3: rotated, distinct values -----
def search_rotated(nums: list[int], target: int) → int:
    """Day 045. At least one half is sorted; discard on that half's RANGE."""
    lo, hi = 0, len(nums) - 1
    while lo ≤ hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] ≤ nums[mid]:           # ≤ : a one-element range is sorted
            if nums[lo] ≤ target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:
            if nums[mid] < target ≤ nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1

# ----- chapter 4: local structure, no order -----
def find_peak(nums: list[int]) → int:
    """Day 049. Compare with the RIGHT neighbour only; the array being finite is the proof."""
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < nums[mid + 1]:
            lo = mid + 1
        else:
            hi = mid
    return lo

def find_min_rotated(nums: list[int]) → int:
    """Day 045. A valley. Compare against nums[hi], never nums[lo]."""
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]:
            lo = mid + 1
        else:
            hi = mid
    return nums[lo]
```

```

# ----- chapter 5: search the ANSWER -----
def ship_within_days(weights: list[int], d: int) → int:
    """Day 046. Bound the answer range, write works(x), say why it is monotone."""
    def days_needed(capacity: int) → int:
        days, load = 1, 0
        for w in weights:
            if load + w > capacity:
                days, load = days + 1, 0
            load += w
        return days

    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(mid) ≤ d:
            hi = mid
        else:
            lo = mid + 1
    return lo

def max_min_gap(positions: list[int], count: int) → int:
    """Day 047. Maximise the minimum: search the first FAILURE, then step back one."""
    positions = sorted(positions)

    def can_place(gap: int) → bool:
        placed, last = 1, positions[0]
        for p in positions[1:]:
            if p - last ≥ gap:
                placed, last = placed + 1, p
            if placed == count:
                return True
        return False

    lo, hi = 1, positions[-1] - positions[0] + 1    # + 1: half-open, hi is a candidate
    while lo < hi:
        mid = (lo + hi) // 2
        if not can_place(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo - 1

# ----- chapter 6: real-valued answers -----
def sqrt_float(x: float) → float:
    """Day 048. A fixed iteration count. hi = max(1.0, x), never x."""
    if x < 0:
        raise ValueError("no real square root of a negative number")
    lo, hi = 0.0, max(1.0, x)
    for _ in range(100):
        mid = (lo + hi) / 2
        if mid * mid < x:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2

if __name__ == "__main__":
    print(search([3, 9, 14, 23, 31], 23), search([3, 9, 14], 10))    # 3 -1
    print(search_range([2, 4, 4, 4, 7], 4), search_range([2, 4], 5))    # [1, 3] [-1, -1]
    print(search_rotated([9, 11, 1, 3, 5, 7], 9))    # 0
    print(search_rotated([1, 3, 5, 7, 9], 7))    # 3
    print(find_peak([1, 3, 2, 4, 7, 9, 5, 6, 2]))    # 7
    print(find_min_rotated([9, 11, 1, 3, 5, 7]), find_min_rotated([1, 3]))    # 1 1
    print(ship_within_days([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5))    # 15
    print(max_min_gap([1, 2, 4, 8, 9], 3))    # 3
    print(f"{sqrt_float(10):.6f}", f"{sqrt_float(0.25):.6f}")    # 3.162278 0.500000

```


Six chapters, about a hundred lines, and one loop shape doing almost all of it. Read it once, then close it and write `first_true` from memory — that is the only thing in the file you must be able to produce cold.

6 What it costs

The whole phase, priced

Say these without pausing. Interviewers ask for the number, not the letter.

```
n = 1,000,000 elements:

plain search           20 comparisons
first + last          40 comparisons
rotated (distinct)    20 passes x ~3 = 60 comparisons
rotated (duplicates)  up to 1,000,000 -- the worst case is the point
peak                  20 comparisons
search on answer, range 25M
                      25 passes x 1,000,000 = 25,000,000 operations
                      (the CHECK is O(n): the comparison is no longer free)
float, fixed count    100 passes, whatever the input
```

The two comparisons worth having ready

```
find A PEAK      O(log n)      20 comparisons at a million
find THE MAXIMUM O(n)          1,000,000 -- and it cannot be beaten
                                   → weakening "global best" to "local best" buys the logarithm

LC 410 by binary search  O(n log(sum)) = 1,000 x 30      = 30,000 ops, O(1) space
LC 410 by dynamic prog.  O(n^2 k)      = 1,000 x 1,000 x 50 = 50,000,000 ops, 50,000 cells
                                   → ~1,600x, and a quarter of the code
```

Where the logarithm sits, which people get wrong

```
searching an ARRAY:  log2(n)      -- doubling the array adds one pass
searching the ANSWER: log2(hi - lo) -- doubling the array adds NOTHING to the pass count,
                                   but doubles the cost of every check
```

That distinction is a good sentence to have: *“the log is over the answer range, not the input, so the input size multiplies rather than adds.”*

Space, and the one thing that breaks it

```
every tool in the phase:  O(1) extra space
the exception:           sorting an unsorted input first -- O(n log n) time,
                           and O(n) space unless you sort in place
```

7 The traps

The five-bug pre-flight, with the input that catches each

Run this list before you say “done” on any binary search, in an interview or out of it.

1. convention mixed → two-element array, e.g. [2, 1] or [1, 2]
2. $hi = mid - 1$ on a branch that keeps mid → an array whose answer sits where the first midpoint lands, e.g. `find_peak` on [1, 2, 1, 3, 5, 6, 4]
3. discard on one comparison, not a proof → [9, 11, 1, 3, 5, 7] searching for 9
4. range misses the answer → `sqrt(0.25)`; a capacity upper bound smaller than `sum(weights)`
5. lo is meaningless → a rate of 0, a capacity of 0

The real error: the convention blend

```
lo, hi = 0, len(nums)          # half-open bound
while lo ≤ hi:                 # closed-range condition
    mid = (lo + hi) // 2
    if nums[mid] ≥ target:
        hi = mid
    else:
        lo = mid + 1
```

```
Traceback (most recent call last):
  File "day50.py", line 5, in <module>
    if nums[mid] ≥ target:
    ~~~~^~~~~~
IndexError: list index out of range
```

`hi` starts at `len(nums)` and the non-strict condition lets `mid` reach it. On other inputs the same blend hangs instead, because `hi = mid` at `lo == hi` changes nothing. **Pick a convention per function and never mix within one.**

The near-miss that survives testing: an upper bound that is too small

```
def broken_ship(weights, d):
    lo, hi = max(weights), max(weights) * len(weights) // 2    # hi too small
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(weights, mid) ≤ d:
            hi = mid
        else:
            lo = mid + 1
    return lo

print(broken_ship([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 1))
```

The correct answer is 55. Every candidate in the range answered False, `lo` marched to the top, and the function returned the edge of its own range. **Assert `works(hi)` before the loop while you develop.** It costs one line and it converts this failure from silent to loud.

The near-miss: a question that is not monotone

```
# "the smallest k such that EXACTLY k groups can be formed"
```

No code needed to see the problem. If 5 works, 6 fails and 7 works, the row of answers is not monotone, the halving lands somewhere arbitrary, and there is no error. **Say the monotonicity sentence out loud before you write the loop, every single time: if x works, does every larger x also work, and why?**

The trap: sorting you forgot

```
print(max_min_gap([5, 1, 9, 2, 8], 3))    # meaningless if positions is not sorted first
```

The feasibility check computes `p - last` as a distance, which is only a distance if the positions come in order. Binary search on the answer does not need the input sorted; the *check* often does. Those are different requirements and keeping them separate in your head is worth doing.

The trap: reaching for a scan because “it isn’t sorted”

The reflex that this phase exists to remove. Rotated arrays are not sorted. Peak finding has no order at all. Capacity problems never sort anything. **Binary search needs a rule for discarding half with a guarantee — sortedness is one way to get one, and it is not the only way.**

8 In the interview

How it gets asked

Today is a mock, so the framing is the round itself:

- “Two problems, no hints. Talk as you go.” — forty-five minutes, roughly twenty each, and the talking is scored.
- “Here’s the first one.” — usually a medium from chapters 1 to 4.
- “And here’s the second.” — usually a search-on-answer, because that is where candidates separate.
- “You’ve got five minutes left.” — the pressure question, and the right answer is to say what you are choosing to finish.

The mock protocol

Run it exactly like this. Standing, out loud, nothing open, a timer running.

```
minutes 0-1    the recognition minute: chapter, precondition, range, question, return meaning
minutes 1-3    say the invariant, and the cost you expect, BEFORE any code
minutes 3-12   write it, narrating each line as a sentence
minutes 12-15  run the pre-flight five, and the two-element and empty inputs
minutes 15-20  the follow-up you know is coming (duplicates / updates / a bigger n)
```

The most common failure in a mock is silence between minutes 3 and 12. If you find yourself thinking without speaking, say what you are weighing — *“I’m deciding whether the middle can be the answer, which decides between `hi = mid` and `hi = mid - 1`”* — because that sentence is worth as much as the code.

What to say out loud, in the first ninety seconds

1. **Name the chapter.** *“This is ‘the smallest X such that something is possible’, so I’ll binary search the answer rather than the array.”*
2. **Check the precondition and say it.** *“More capacity can never need more days, so the question is monotone. That’s what the whole method rests on.”*
3. **State the range with a reason each end.** *“Not below the heaviest package — it’d never load. Not above the total — that’s one day.”*
4. **State the invariant.** *“Below `lo` everything answers no; from `hi` up, everything answers yes.”*
5. **Predict the cost.** *“The check is $O(n)$ and I’ll do about twenty-five of them, so $O(n \log \text{range})$.”*
6. **Then write it**, narrating.

The follow-ups

“How do you decide, in ten seconds, whether a problem is binary search at all?” I ask one question first, and it is not “is it sorted?” — it is “what is the answer?” If the answer is a *position in the input*, then I need the input to have structure I can discard on: sorted, rotated, or a local slope like a peak. If the answer is a *value I get to choose* — a capacity, a rate, a gap, a number of days — then the input’s order is irrelevant and I am searching the answer range instead, which is where the harder problems live. Getting that order right is what stops people missing the search-on-answer family, because those problems read as optimisation and the input is usually unsorted, so “is it sorted?” gives a misleading no. After that I check the precondition for whichever branch I am in: sortedness and on which key, or monotonicity said as a sentence about the real world. If neither holds, I say so and price the scan instead — $O(n)$, and if there are many queries, sort once at $O(n \log n)$ and amortise.

“Your search-on-answer solution is $O(n \log \text{range})$. Isn’t the log over the input for a real binary search?” No, and the distinction is worth being precise about because it changes how the solution scales. In an array search the log is over n , so doubling the input adds one comparison and the comparison itself is free. In a search on the answer the log is over the *range of candidate answers*, and each evaluation is a full pass over the input. So doubling the input adds nothing to the number of passes and doubles the cost of every pass — the input size multiplies rather than adds. Concretely, fifty thousand packages over a capacity range of twenty-five million is about twenty-five passes of fifty thousand operations, so a bit over a million, against a trillion if I tried every capacity. It also tells me where to spend effort if it is too slow: tightening the bounds saves passes, but making the check cheaper saves more, because it multiplies.

“You’ve got five minutes left and the second problem isn’t finished. What do you do?” I say so out loud and choose, rather than typing faster and hoping. With five minutes I would finish the *feasibility check* completely and correctly, because that is where the actual difficulty of a search-on-answer problem lives — the search loop around it is six lines I can write in ninety seconds and have written identically five times this week. So I would say: “the loop is the standard boundary template, and I’ll write it last; let me make the check right first, because that’s the part that’s specific to this problem.” Then, in the final minute, I would state the things I know I have not verified — the two-element case, whether the upper bound is provably feasible — rather than let the interviewer discover them. Naming your own gaps is worth real marks, because the alternative reading is that you did not see them.

A model answer

“Let me take the recognition minute before I write anything, because on this family the naming decides everything after it.

The phrasing is ‘the smallest capacity such that all packages ship within d days’. That’s ‘the smallest X such that something is possible’, so the answer is a value I choose, not a position in the input — which means I’m binary searching the answer range, and the fact that the weights aren’t sorted is irrelevant.

Precondition: is it monotone? More capacity can never require more days — every packing that fits a smaller ship also fits a bigger one — so the answers run no, no, no, yes, yes, yes with exactly one boundary. That’s the property the whole method rests on, and I check it in words before I trust it.

Range, with a reason at each end. Not below `max(weights)`, because a package heavier than the ship never loads. Not above `sum(weights)`, because that’s everything in one day. Both $O(n)$, computed once.

The question: `days_needed(capacity)` walks the packages in order, adds to the current day, and starts a new day when the next one doesn’t fit. The counter starts at one, because day one exists before anything is loaded.

Invariant: everything below `lo` answers no, everything from `hi` up answers yes, so `lo` ends on the boundary. Cost: the check is $O(n)$ and I’ll run about twenty-five of them, so $O(n \log \text{range})$ — and $O(1)$ space.

```
lo, hi = max(weights), sum(weights)
while lo < hi:
    mid = (lo + hi) // 2
    if days_needed(mid) ≤ d:
        hi = mid
    else:
        lo = mid + 1
return lo
```

That’s the same six lines as every other search this week; only the question changed.

Before I say I’m done, the pre-flight: the convention is half-open so the condition is strict and there’s no minus one anywhere; the True branch keeps `mid` because `mid` might be the smallest capacity that works; the upper bound provably works, so there’s no not-found case; and the lower bound is meaningful rather than zero. I’d test a single package, all-equal weights, and d equal to the number of packages.

And the follow-up I'd expect: if packages could ship in any order, the greedy check stops being exact — that's bin packing, NP-hard — so the search skeleton survives but the answer becomes an approximation, and I'd say that rather than let it pass."

9 Recall card

- **Ask “what is the answer?” before “is it sorted?”** A *position* → the input needs discardable structure (sorted / rotated / local slope). A *value you choose* → search the answer range, and the input's order is irrelevant.
- **Six chapters, one loop.** `first_true` from [day 043](#) — half-open, strict `<`, `hi = mid` / `lo = mid + 1` — with a different question each time. Writing a second loop shape means you should have written a different question.
- **The five bugs, with their inputs:** convention blend (two-element array) · `hi = mid - 1` where `mid` is a candidate · discarding on one comparison instead of a proof · a range that misses the answer (`assert works(hi)`) · a meaningless `lo` (rate 0 → `ZeroDivisionError`).
- **The three exits:** “exactly `k`” kills monotonicity · duplicates/plateaus make it $O(n)$, unavoidably · the *maximum* of an unsorted array is $O(n)$ and unbeatable, where a *peak* is $O(\log n)$.
- **Say the numbers, not the letters:** a million → 20 comparisons; two bounds → 40; rotated with duplicates → up to a million; search-on-answer → $O(n \times \log \text{range})$, the log over the *answer* range, so input size multiplies rather than adds. Every tool in the phase is $O(1)$ space.

Class diagrams and the UML you will actually draw

1 What this is, and why they ask it

A **class diagram** is a picture of a design: boxes for classes, lines for the relationships between them, and small numbers saying how many of each. **UML** — the Unified Modeling Language — is the standard it comes from, and it has fourteen diagram types and a specification running to hundreds of pages. You need roughly five symbols from it, and today is about which five and how to put them on a board in four minutes.

They ask because the diagram is the shared object in the room. Everything you said in the first fifteen minutes of an object-oriented design round lived in the air; the diagram makes it something you and the interviewer can both point at, and the follow-up questions get asked *about the picture*. A candidate who draws twelve unlabelled boxes joined by identical arrows has made the conversation harder rather than easier. A candidate who draws seven boxes with the important fields, three kinds of line used consistently, and multiplicities on every relationship has, in four minutes, made the rest of the round easy for both of them.

2 The story

Ouseph supervises small building sites around Aluva — two-storey houses, mostly — and the part of the job he cares most about happens on one morning, before any wall goes up.

The ground floor slab gets poured, and then it sits for a week. On the morning the masons are due to start, Ouseph gets there early with a lump of chalk and draws the whole ground floor on the concrete. Full size. The kitchen wall, the passage, where the stairs land, the line of the front veranda.

It takes him about forty minutes and it looks rough. He does not measure everything; he paces most of it. He does not draw the plug points or the tile pattern or the window frames. He draws walls, doors and the staircase, and nothing else.

Then he stands there with the owner and the head mason, and they walk about on it.

What happens on that hour is the reason for the chalk. The owner walks from the chalk front door to the chalk kitchen and says the passage feels narrow, and it is narrow, and they move a line by nine inches. The mason stands at the foot of the chalk stairs,

looks up at nothing, and says the landing will foul the window above, which nobody had noticed. Last year an owner stood in her chalk kitchen and realised the door was on the wrong side entirely, and they changed it before a single brick was laid.

None of those three would have come out of talking. They came out of standing in it.

And then the bricks go down and the chalk disappears under the first course, that same afternoon. Nobody photographs it. Nobody keeps it. Ouseph has never once gone back to look at an old chalk drawing, because by the time the walls are up, the walls are the drawing.

He is quite clear about what would ruin it, too. If he tried to chalk everything — every socket, every shelf, the cupboard depths — it would take two days, the floor would be a mess of lines nobody could read, and the owner would not walk about on it. The drawing works because it leaves nearly everything out.

3 The idea in plain English

Ouseph's chalk outline is the class diagram. The hour of walking about on it is the design conversation. The bricks covering it up is the code, which becomes the real record. And the two-day version with every socket on it is a fully specified UML model — technically more complete, and worse at the one job it had.

The five symbols you need

Everything you will draw in an interview uses these and nothing else.

One: the class box. Three compartments — name, fields, methods:

+-----+	
ParkingSpot	← name, always
+-----+	
- size: SpotSize	← the fields that MATTER, 3-5 of them
- vehicle: Vehicle None	
+-----+	
+ can_fit(v): bool	← the methods that carry rules, 2-4
+ occupy(v)	
+ release()	
+-----+	

- means private, + means public. That is the whole of visibility notation you need. Leave out constructors, getters and anything that does not carry a decision — the diagram is for the conversation, and `get_size()` has never started a conversation.

Two: association — a plain line, meaning “these two know about each other”. Add an arrowhead if only one direction knows:

```
Ticket -----> ParkingSpot      a ticket knows its spot; the spot does not know the ticket
```

Three: inheritance — a line with a **hollow triangle** pointing at the parent:

```
Car -----▷ Vehicle
Bike -----▷ Vehicle
```

Four: implements an interface — the same triangle, but a **dashed** line:

```
HourlyPricing - - -▷ PricingStrategy
```

Five: multiplicity — small numbers at each end of a line, saying how many:

```
ParkingLot 1 ----- 1..* Level      one lot has one or more levels
Level      1 ----- 0..* ParkingSpot one level has any number of spots
```

The four you need: `1`, `0..1`, `1..*`, `0..*`. **Put a multiplicity on every line.** It is the cheapest information in the diagram and it is the thing candidates leave out most.

The diamond, and why not to worry about it

UML has two more line types. A **hollow diamond** at the container end means *aggregation* — the parts can exist without the whole. A **filled diamond** means *composition* — the parts die with the whole, so `Order ◆— OrderLine` because a line item has no meaning without its order.

Learn what they mean; do not spend interview time deciding between them. No interviewer has ever marked a candidate down for drawing a plain line where a hollow diamond belonged. Several have marked candidates down for spending ninety seconds visibly agonising over it. **If the distinction genuinely matters to your design — because it decides a cascade delete — say it in a sentence instead of encoding it in a symbol.**

What to leave out

This list is longer than the list of what to include, and that is the point.

- **Getters, setters and constructors.** They fill the box and say nothing.
- **Every field.** Three to five that matter. `created_at` is not one of them.
- **Type details.** `list[ParkingSpot]` is fine; `Optional[Sequence[ParkingSpot]]` is noise on a board.
- **The other thirteen UML diagram types.** Component, deployment, state, activity, package — interesting, and not what “draw the class diagram” means.

- Stereotypes, notes, constraint boxes, `«interface»` markers if you have already used the dashed line.
- Anything you already said out loud and nobody questioned.

Seven boxes, not fifteen

There is a real limit and it comes from what a person can hold at once: about seven items, plus or minus two. A diagram of fifteen boxes is not read; it is scanned and abandoned.

If your design genuinely has fifteen classes, **draw two diagrams**: the core five or six, and then a second showing one subsystem in detail. Say what you are doing — *“let me draw the core first, then zoom into pricing”* — and the interviewer will follow you happily.

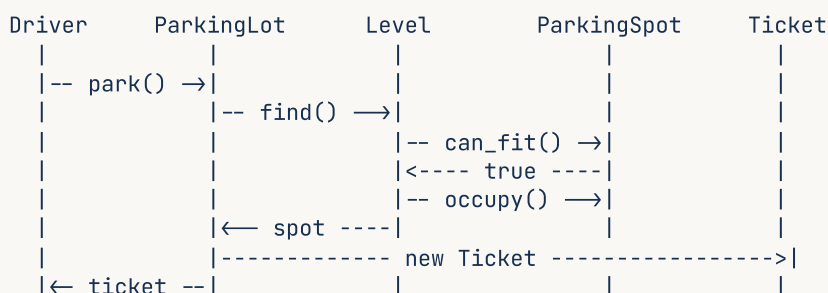
The order to draw in

This matters more than the symbols, because a board fills up and you cannot undo.

1. **Put the two or three central classes in the middle first**, spread wide apart. Leave far more space than feels necessary.
2. **Draw the containment spine** — lot, level, spot — top to bottom or left to right, with multiplicities as you go.
3. **Hang the inheritance groups off to one side**. Vehicles in one corner, spot types in another. Do not let hierarchies cut across the middle.
4. **Add the interfaces last**, with dashed triangles, usually at an edge.
5. **Fill in fields and methods only for the boxes the conversation is actually about**. Some boxes will have a name and nothing else, and that is correct rather than lazy.

The sequence diagram, in one paragraph

Occasionally the question is not “what are the classes” but “walk me through what happens when a car arrives”. That is a **sequence diagram**: participants as columns, time running down, and each message an arrow from one column to the next.



Five participants maximum, one flow per diagram, and only draw one when the *interaction order* is the interesting thing. A class diagram shows structure; a sequence diagram shows a story. Knowing which the question wants is most of the skill.

4 The picture

The parking lot, drawn the way it should look on a board at minute twenty:

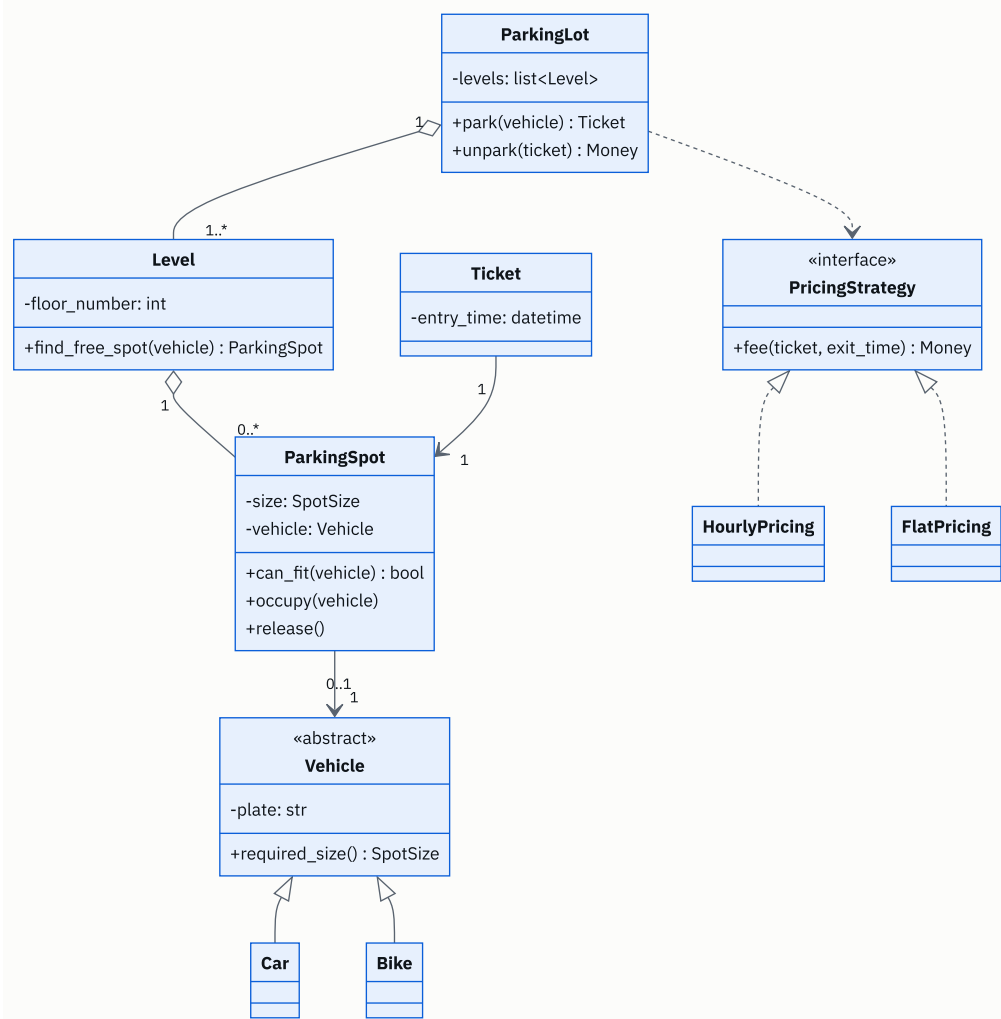
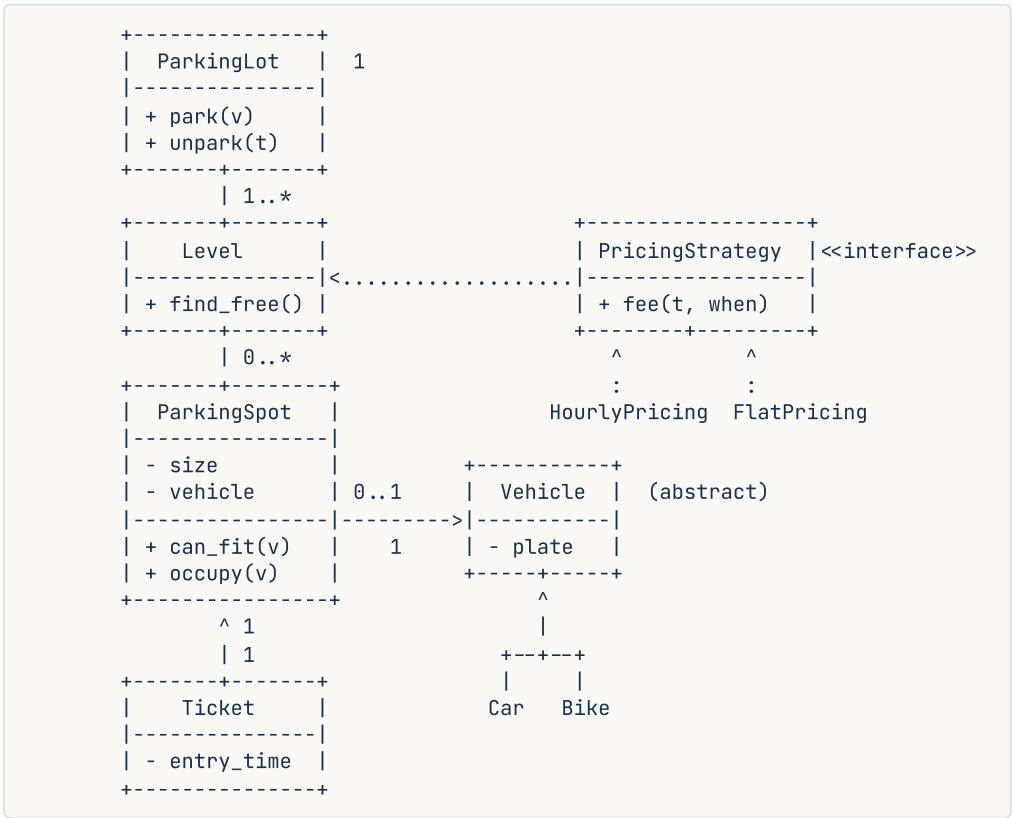


FIGURE 50.2

What to notice: nine boxes, and only four of them carry fields and methods. The three that matter to the conversation — the lot, the spot and the pricing interface — are detailed; **Car** and **Bike** are names only, because everything about them was said out

loud. Every relationship line carries a multiplicity except the inheritance ones, which never do.

The same thing as chalk on a whiteboard, which is what you will actually draw:



What to notice: the containment spine runs straight down the left, the inheritance groups sit off to the sides, and the interface is at the right edge. Nothing crosses anything. That layout is not decoration — it is what makes the diagram readable from where the interviewer is sitting.

5 How it actually works

What people really do with these

Being honest about this is worth marks, because interviewers know it.

- **In an interview**, a class diagram is the artefact of the round. It is drawn once, discussed, and thrown away.

- **In a design document or a pull request**, a small diagram of the five classes you are adding is genuinely useful, and Mermaid means it lives in the repository next to the code.
- **In onboarding**, a diagram of the core domain saves a new engineer a week.
- **Almost nowhere** is a full UML model maintained in step with the code. Round-trip tools that generate code from diagrams and back had their moment and it passed. Diagrams that are not regenerated go stale, and a stale diagram is worse than none because people trust it.

The conclusion is Ouseph's: **draw the diagram for the conversation it enables, not as a record**. The code is the record.

Mermaid, which is the version you will type

Every diagram in this course is Mermaid, and it is the practical answer for a design doc:

```
classDiagram
    class Order {
        -id: int
        -status: OrderStatus
        +cancel()
        +total() Money
    }
    Order "1" *-- "1..*" OrderLine      %% filled diamond: lines die with the order
    Order "0..*" --> "1" Customer      %% plain association with an arrow
    Order ..> PricingStrategy          %% dashed: depends on / uses
    Notification <|-- EmailNotification %% hollow triangle: inheritance
    Sendable <.. EmailNotification      %% dashed triangle: implements
```

The five arrow forms to remember: `<|--` inheritance, `<..` implements, `*--` composition, `o--` aggregation, `-->` association. Quotes carry multiplicities. That is enough for every diagram you will draw for years.

GitHub, GitLab and most documentation tools render Mermaid in Markdown, which is what makes it worth learning over a drawing tool: the diagram is reviewable in a pull request and diffs like text.

The tools, named

- **A whiteboard**, or the interviewer's shared drawing tool — Excalidraw and tldraw are the common ones in remote rounds. Practise on whichever your interviews use; drawing a straight line with a mouse is a skill and it is worth ten minutes of practice.
- **Mermaid**, for anything living in a repository.
- **PlantUML**, the older text-to-diagram tool. More expressive, uglier output, still common in enterprise codebases.
- **draw.io** / **diagrams.net**, for a diagram somebody will edit later by hand.

How to talk while drawing

The diagram is not the answer; it is the accompaniment. Narrate as you draw, and the narration is what is scored:

"I'll put ParkingLot at the top — it's the entry point and it coordinates, but it owns no rules itself. Under it, Level, one-to-many. Under that, ParkingSpot, one-to-many again — and I'm putting `can_fit` on the spot rather than on the lot, because the spot is the thing that knows its own size. Off to the right, PricingStrategy as an interface with two implementations, because pricing is the requirement most likely to change."

Three sentences, and every one of them is a design decision with a reason. Silence while drawing is the commonest way to waste four minutes.

6 The numbers

What fits, and what is read

```
readable on a whiteboard from where the interviewer sits: 6-9 boxes
the working-memory limit (7 +/- 2):                        ~7 items
a diagram of 15 boxes:                                     scanned, not read

so: 15 classes → TWO diagrams (core, then one subsystem), announced out loud
```

Time, on the board

```
one class box with 3 fields and 3 methods  ~30 seconds to draw legibly
7 boxes                                     ~3.5 minutes
relationship lines and multiplicities       ~1 minute
-----
                                           ~4.5 minutes

the OOD round's budget for the diagram     3-5 minutes, from a 45-minute round
```

Which is why detail is the enemy. Adding `created_at`, `updated_at` and two getters to each of seven boxes is another two minutes and adds nothing anyone will ask about.

Detail per box, budgeted

```
fields shown per class      : 3-5      (the ones a rule depends on)
methods shown per class     : 2-4      (the ones carrying decisions)
boxes with NO detail        : 30-50%   -- and that is correct, not lazy

a box with 12 fields and 9 methods:
    ~90 seconds to draw, and it is the class that should have been two classes
```

That last line is worth saying out loud if it happens: a box that will not fit is usually a design problem showing up as a drawing problem.

The symbols, counted

UML 2.5 specification:	~800 pages, 14 diagram types
symbols used in a real interview: 5	(box, association, inheritance, implements, multiplicity)
symbols worth knowing but not agonising over:	2 (hollow / filled diamond)

7 The trade-offs

Detail against readability

Every field you add makes the diagram more complete and less useful. *I would not draw a field the conversation does not depend on* — the diagram is a conversation aid, and a complete one is Ouseph's two-day chalk drawing that nobody walks on. If the interviewer wants a field, they will ask, and answering is faster than pre-empting.

Precision against speed

You could get every diamond right, mark every navigability arrow, and label every association with a role name. *I would not spend interview time on aggregation-versus-composition* — I would draw a plain line and, if the distinction matters, say “these die with the order” in a sentence. The exception is a design document that will be read without you present; there, precision earns its cost because nobody can ask.

A class diagram against a sequence diagram

A class diagram shows structure and is silent about time. A sequence diagram shows one flow and is silent about structure. *I would not draw a sequence diagram unless the ordering of the interaction is the interesting part* — for “design a parking lot” it is not, and for “walk me through what happens on a payment retry” it very much is. Drawing the wrong one wastes four minutes and answers a question nobody asked.

A maintained diagram against no diagram

I would not commit a large diagram to a repository unless someone owns regenerating it, because a stale diagram is worse than no diagram — people trust it and it lies. A small Mermaid diagram of five classes next to the code it describes usually survives, because it is small enough that whoever changes the code will fix it. A forty-box model will not.

The honest sentence

The diagram's job is to let two people point at the same thing and disagree productively. Judge it by whether the next five minutes of conversation got easier — not by whether it is complete, and certainly not by whether the diamonds are right.

8 In the interview

How it gets asked

- *“Draw the class diagram for what you just described.”* — the direct form, usually at minute twenty, after you have talked through the classes.
- *“Can you show me how these fit together?”* — the same request, phrased as if it were optional. It is not.
- *“Walk me through what happens when a car arrives.”* — a sequence diagram question wearing plain clothes, and drawing a class diagram in response answers something else.
- *“What’s the difference between aggregation and composition?”* — the vocabulary check. Answer in one sentence and add that you would not spend board time on it.

What to say out loud, in the first ninety seconds

1. **Say the plan before the first box.** *“I’ll draw the core structure first — lot, level, spot, vehicle — then hang the pricing interface off the side. About four minutes.”*
2. **Place the central class and say why it is central.** *“ParkingLot at the top: it coordinates and owns no rules of its own.”*
3. **Draw the spine with multiplicities as you go.** *“One lot, one-to-many levels. One level, zero-to-many spots.”*
4. **Narrate one design decision while drawing it.** *“`can_fit` goes on the spot, not on the lot, because the spot is what knows its own size — if I put it in the lot it becomes a type-switch I’d have to edit for every new vehicle.”*
5. **Do the interfaces last and say why they exist.** *“PricingStrategy as an interface, dashed triangle, two implementations — because pricing is the requirement most likely to change, and I can name the second implementation.”*
6. **Say what you left out.** *“I’ve left out getters and audit fields deliberately, and payment is out of scope — tell me if you want either.”*

The follow-ups

“What’s the difference between aggregation and composition, and does it matter here?” Aggregation is a whole-part relationship where the parts can outlive the whole — a hollow diamond at the container end. Composition is one where they cannot — a filled diamond — so if the whole is deleted, the parts are deleted with it. In this design, `Order` to `OrderLine` is composition, because a line item has no meaning without its order and would be cascade-deleted with it, and `ParkingLot` to `Level` is arguably aggregation, because a level is a physical thing that would survive the software object being destroyed. Does it matter? It matters in exactly one place, which is the cascade-delete decision, and that is a database `ON DELETE` clause rather than a drawing symbol — which was [day 026](#)’s point about choosing `ON DELETE` deliberately. So my honest answer is that I know the distinction, I will draw a plain line under time pressure, and if the life-cycle matters I will say “these die with the order” in a sentence, which is clearer to everyone in the room than a diamond that half the readers will not decode.

“Your diagram has fifteen boxes and I can’t read it. What do you do?” I would say that is my mistake and split it, and I would say what I am splitting on rather than just erasing things. The limit is what a person can hold at once — about seven items — so fifteen boxes is not a diagram, it is a wall. So: rub out everything except the five or six classes that carry the main flow, redraw those with room around them, and then say “let me zoom into pricing separately”, and draw the second diagram beside it. That is two readable pictures instead of one unreadable one, and the narration — announcing the split before doing it — is what stops it looking like panic. The other thing I would take from it is that a diagram that will not fit is usually telling me something about the design: if fifteen classes are all mutually connected, the boundaries are probably wrong, and I would rather notice that on the board than three weeks into building it.

“Do you actually use these at work, or is this an interview exercise?” Both, in different sizes, and I think being straight about that matters. In an interview the diagram is the artefact of the conversation — it gets drawn, argued over and wiped, and its whole value was the argument. At work I use small ones: five or six classes in a Mermaid block in a design document or a pull request description, describing the thing I am adding, which reviewers can read in twenty seconds and which diffs like text in code review. What I have never seen work is a full UML model kept in step with the code. Those go stale within a release, and a stale diagram is worse than none because people trust it and it lies. The tools that generated code from diagrams and back had a period of popularity and it passed. So my rule is that the code is the record, and a diagram is a temporary shared surface for a conversation — which is exactly why I draw it rough, leave the getters out, and do not agonise over the diamonds.

A model answer

“Let me draw the core first and hang the variable parts off the side. Four minutes or so, and I’ll talk while I draw.

ParkingLot goes at the top, because it’s the entry point and it coordinates — it owns no rules itself, which is deliberate. Below it, Level, one-to-many. Below that, ParkingSpot, zero-to-many. I’m putting multiplicities on every line because they’re the cheapest information here and they’re what people leave out.

ParkingSpot gets real detail, because it’s where the interesting decision is: it holds `size` and an optional `vehicle`, and exposes `can_fit`, `occupy` and `release`. `can_fit` is on the spot rather than on the lot because the spot is the thing that knows its own size — put it in the lot and it becomes a type-switch I have to edit every time a vehicle type is added.

Off to the right, Vehicle as an abstract class with Car and Bike under it, hollow triangles. Those two boxes get names only — everything about them was in what I said, and drawing their fields would just fill the board.

On the other side, PricingStrategy as an interface, dashed triangles from HourlyPricing and FlatPricing, and a dashed dependency from ParkingLot to it. That’s there because pricing is the requirement most likely to change, and I can name the second implementation — which is my test for whether an interface earns its place.

Ticket down at the bottom, pointing at the spot it was issued for, with the entry time on it.

Things I’ve left out on purpose: getters, constructors, audit fields, and the payment flow. And I’ve drawn plain lines rather than agonising over aggregation versus composition — the only place that distinction matters here is whether tickets are deleted with the lot, and if that’s interesting I’d rather say it in a sentence than encode it in a diamond half the room won’t decode.

Nine boxes, four of them detailed. If you want, the next thing I’d draw is a sequence diagram for the arrival flow — but only if the ordering of the interaction is what you want to dig into, because this picture is already telling you the structure.”

9 Recall card

- **Five symbols is the whole kit:** the three-compartment box (`+` public, `-` private), a plain **association** line, a **hollow triangle** for inheritance, a **dashed** triangle for implementations, and a **multiplicity** on every line (`1`, `0..1`, `1..*`, `0..*`).

- **6-9 boxes, 30-50% of them detail-free.** Fifteen boxes is a wall — split into “the core” plus one subsystem, and announce the split. A box that will not fit is usually two classes.
- **Draw in order:** central class first with space around it → the containment spine with multiplicities → inheritance groups off to the sides → interfaces at an edge → fields and methods only where the conversation is.
- **Leave out getters, constructors, audit fields, full types, and the diamond argument.** Say “these die with the order” in a sentence rather than encoding it — nobody has ever been marked down for a plain line.
- **Narrate every box with its reason** (“`can_fit` on the spot, because the spot knows its own size”). The diagram enables the conversation; **the code is the record** — a stale diagram is worse than none. Class diagram = structure; sequence diagram = one flow, ≤5 participants, only when order is the point.

DSA topic: Binary search revision and mock round **System design topic:** Class diagrams and the UML you will actually draw

Code these, in this order

A mock, so the protocol rules: **twenty-five minutes a problem, standing, talking the whole time, nothing open — and Yesudas’s two seconds first.** Name the chapter and the tell before you solve anything, and score yourself on the naming separately from the code.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Find First and Last Position of Element in Sorted Array	LeetCode 34 (Medium)	The warm-up: two bounds, and whether find-then-walk still tempts you.
2	Search in Rotated Sorted Array	LeetCode 33 (Medium)	Discarding on a range, and the $\leq$ in the sortedness test, cold.
3	Koko Eating Bananas	LeetCode 875 (Medium)	Whether the answer-space instinct fires without the word “capacity” in it.
4	Split Array Largest Sum	LeetCode 410 (Hard)	The phase’s summit, re-derived a week later, with the DP named and rejected.

On problem 1, respect the warm-up

Two template calls, one character apart, and the guard in the right order. If it takes more than ten minutes, the phase needs re-reading rather than more problems. Then run `[8] * 200000` with target 8 and confirm you did not write a walk.

On problem 2, say the sentence before the code

At least one half is properly sorted, because there is exactly one break point and one point cannot be in two halves. Then write it. Then run the un-rotated array, the break-at-the-end array, and the single-element array. A special case for any of those means the real bug is still there.

On problem 3, notice there is no capacity in sight

The word “capacity” does not appear and the shape is identical. The tell is “the minimum k such that it can be finished in h hours”. Bound it at 1 to `max(piles)` — say why the lower bound is not 0 and what it raises. Ceiling division: `(pile + k - 1) // k`, checked by hand on `pile = 7, k = 3`.

On problem 4, name the DP and reject it with numbers

Before writing the search, say: “there’s a dynamic-programming solution at $O(n^2k)$ — a thousand elements and fifty parts is fifty million operations and a fifty-thousand-cell table, against about thirty thousand operations and three integers here.” Then write the binary search. Then diff it against your LeetCode 1011 solution from day 046 — if they differ anywhere except the names, one of them is wrong.

The two-second drill

Ten phrasings. Name the chapter and the tell in under ten seconds each, out loud, no code:

1. Answer 100,000 range-sum queries on a fixed array.
2. Find the index of a target in a sorted array with duplicates.
3. Find the minimum in an array that was sorted and then rotated.
4. Find any element greater than both its neighbours.
5. Find the smallest divisor such that the sum of the divisions is at most a threshold.
6. Find the largest minimum distance when placing c items in n stalls.
7. Find the median of two sorted arrays.
8. Find the maximum of an unsorted array.
9. Find the smallest k such that exactly k groups can be formed.
10. Find the cube root to six decimal places.

Numbers 1, 8 and 9 are not binary search. Say what each of them is instead, and why.

The pre-flight drill

Say the five bugs of the phase from memory, each with the input that catches it:

1. The convention blend — which two things must never be mixed, and what are the two symptoms?
2. `hi = mid - 1` — when is it right and when does it throw away the answer?
3. Discarding on one comparison — which day introduced the need for a proof, and what is the input?
4. A range that misses the answer — name two problems where it happens, and the one-line habit that catches it.
5. A meaningless `lo` — what does it raise, and what is the general rule?

The one-loop drill

Write `first_true` from memory, then express each of these as a question passed to it — one line each, no new loops:

1. The first index whose value is at least the target.
2. The last index holding the target.
3. How many values lie in `[a, b]`.
4. The smallest capacity shipping everything in `d` days.
5. The largest gap at which `c` items can still be placed.
6. The minimum of a rotated array.
7. A peak.

The cost drill

Answer in under five seconds each, with numbers not letters:

1. Comparisons for a plain search at a million.
2. Comparisons for first-and-last at a million.
3. Worst case for a rotated search with duplicates at a million.
4. Operations for a search-on-answer with $n = 50,000$ and a range of 25 million.
5. Peak versus maximum on an unsorted array of a million.
6. Space, for every tool in the phase.

The four-minute diagram drill

Set a timer for four minutes and draw the class diagram for a **library** — the model you built on [day 044](#) — in whatever tool you like, talking the whole time. Then score it:

- [] Six to nine boxes, no more.
- [] A multiplicity on every relationship line.
- [] Hollow triangle for inheritance, dashed for implements, used consistently.
- [] Between a third and a half of the boxes have no fields or methods at all.
- [] No getters, no constructors, no `created_at`.
- [] Nothing crosses anything: spine down the middle, hierarchies at the sides, interface at an edge.
- [] I said a design decision out loud for at least three of the boxes.
- [] I said what I left out, unprompted.

Then draw the same thing again in Mermaid and check it renders.

The wrong-diagram drill

For each request, say which diagram it wants — class, sequence, or neither — and why:

1. “Show me how these fit together.”
2. “Walk me through what happens when a book is returned late.”

3. “What are your classes?”
4. “How does the fee get calculated, and who calls whom?”
5. “Draw the deployment.”

The symbol drill

From memory, and then check:

1. The five Mermaid arrow forms and what each means.
 2. The four multiplicities you actually use.
 3. What a filled diamond means, what a hollow one means, and the one sentence you would say instead of drawing either.
 4. How many boxes are readable from where the interviewer sits, and what you do at fifteen.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two problems, no hints, talk as you go.* Run the recognition minute out loud on problem 4 as if it were unseen: chapter, precondition, range, question, return meaning — then the invariant and the predicted cost, before any code.
 2. *Draw the class diagram for what you just described.* The plan sentence, the central class with its reason, the spine with multiplicities, one design decision narrated, the interface with its second implementation, and what you left out.
 3. *How do you decide in ten seconds whether a problem is binary search at all?* The first question is not “is it sorted?” — it is “what is the answer?” Say both branches, the precondition for each, and the three exits.
-

Before you move on

- [] I did all four problems standing, talking, with the naming scored separately.
- [] I named all ten phrasings in the two-second drill, including the three that are not binary search.
- [] I can recite the five pre-flight bugs with their checking inputs.
- [] I wrote `first_true` from memory and expressed all seven problems as questions to it.
- [] I can give every cost in the phase as a number, not a letter.

- [] I drew the library diagram in four minutes and it passed all eight checks.
- [] I can produce the five Mermaid arrow forms and the four multiplicities from memory.
- [] I answered all three questions above out loud.

The nine sections, and what each is for

Every lesson in this book, on both tracks, carries the same nine sections in the same order. This is what each one is for, and how to read it.

1. What this is, and why they ask it

The topic in plain words, and the reason an interviewer spends time on it. This section exists so you never learn something without knowing what it buys you.

2. The story

A scene with a person in it, told in ordinary language, with no technical words at all. Two hundred to four hundred words about matching socks, or a canteen queue, or laying tables.

This is the most important section for a beginner and the easiest to skip. The story gives you a place to put the idea before you have the vocabulary for it. Later, when you are asked the question under pressure, the story is what you will reach for first.

3. The idea in plain English

The story, translated. Every term is defined the first time it appears, and defined terms are set in **bold** so you can find them again. Appendix D collects them all.

4. The picture

A diagram. Arrays, memory and pointers are drawn as ASCII boxes with indices above and values below, because adjacency matters and a box drawing keeps it. Trees, graphs, architectures and request flows are drawn as proper figures.

Every diagram is followed by a line telling you what to notice in it.

5. The code, built step by step

Fragments of ten lines or fewer, each one followed by prose explaining it. Then the complete solution in one block, ready to run.

The full answer is always here. This book does not withhold solutions.

6. What it costs

The arithmetic, done where you can see it.

On the DSA track that means counting the loop iterations out loud: not “this is quadratic” but “the inner loop runs sixty times for each of sixty items, so about three thousand comparisons”. On the system design track it means doing the multiplication: not “that is a lot of storage” but $50\text{M} \times 20 \times 2\text{KB} = 2\text{TB}$.

7. The traps

The mistakes people actually make, with the real error text printed exactly as it appears on screen. When your own program breaks this way, you will recognise it.

8. In the interview

The point of the whole document.

Real interviewer phrasings. A script for the first ninety seconds, because that is when candidates either take control of the question or lose it. The three follow-ups that usually come next. And a model answer, written out in full, so you have something to measure yours against.

9. Recall card

The two or three sentences that are the entire lesson. Say them aloud, from memory, in full sentences.

If you can do that, you have finished the lesson. If you cannot, you have read it, which is a different thing.

Recall cards

Section 9 of every lesson in this volume, collected. Cover the card, say it aloud, then check. Cards are grouped by the phase they belong to, on both tracks.

Binary search

Day 042 · dsa — Binary search: the idea and the invariant

- **The invariant is the lesson:** *if the target is anywhere, its index is in `nums[low..high]`.* Every line exists to keep it true; an empty space is a conclusion, not a guess.
- **Four details, all off-by-one:** `high = len(nums) - 1`, `while low ≤ high`, `low = middle + 1`, `high = middle - 1`. Drop the `+1` and it hangs; use `<` and it misses the last element.
- **$O(\log n)$ time, $O(1)$ space** — count halvings out loud: 1,000 → 10, a million → 20, a billion → 30. Every doubling costs one more comparison.
- **Sorted is a precondition, not a hint.** On unsorted input it returns `-1` for values that are there, with no error. Ask before you write.
- **`bisect_left` is the library answer** and returns an *insertion point*, not a found flag — check `nums[i] == target` after. Tomorrow: one template that removes the off-by-ones for good.

Day 043 · dsa — Writing binary search without off-by-one bugs

- **One template, forever:** half-open `[lo, hi)`, `hi = len(nums)`, `while lo < hi`, `mid` in the middle, `question(mid) → hi = mid` else `lo = mid + 1`. Return `lo`.
- **The asymmetry is the lesson:** True keeps the middle (`hi = mid`), False steps past it (`lo = mid + 1`). `- 1` belongs to closed ranges only; blending the two conventions is the bug.
- **Invariant:** below `lo` all False, from `hi` up all True — so `lo` is the first True, and `len(nums)` is the honest answer for “no True at all”.
- **Change one character, get five problems:** `≥` is lower bound (first occurrence, insert position, exists); `>` is upper bound (last occurrence = upper - 1, count = upper - lower). `bisect_left` / `bisect_right` are these two.
- **It needs a monotone question, not a sorted array** — and it returns a *position*, so check `i < len(nums)` before reading `nums[i]`. Termination: `mid < hi` always, so `hi = mid` shrinks.

Day 044 · dsa — First and last occurrence

- **Equal values in a sorted array form one contiguous run** — two indices describe it, so run [yesterday's template](#) twice, one character apart.
- **first = lower_bound ($\geq$ target); last = upper_bound ($>$ target) - 1.** And `nums[lower:upper]` is the run — the slice, not the minus one, is the idea.
- **The guard is two halves, in order:** `first < len(nums)` (or it raises) and `nums[first] == target` (or it lies). Check before computing `last`.
- **Never find-then-walk.** That is $O(\log n + k)$ with k unbounded — a million identical values makes it a linear scan, and that is the input you will be given.
- **Free from the same two calls:** `count = upper - lower` (zero needs no special case); values in `[a, b] = upper(b) - lower(a)`. `bisect_left` / `bisect_right` in production.

Day 045 · dsa — Search in a rotated sorted array

- **One break point, ever.** So at any midpoint the break is in one half only, which means **at least one half is properly sorted** — that single fact is the whole problem.
- `nums[low] ≤ nums[mid]` **picks the sorted half.** `≤`, not `<`: a one-element range has `low == mid` and is trivially sorted. This is the bug everyone writes.
- **Discard on a range, never on one comparison.** Target inside the sorted half's interval → go there; outside → it is provably elsewhere. `mid` is excluded from both intervals.
- **Find-minimum is the clean cousin:** compare `nums[mid]` against `nums[high]` (never `low`) — monotone, so [day 043's](#) template, no branching. The min's index is also the rotation count.
- **Duplicates → $O(n)$ worst case, unavoidably.** When `nums[low] == nums[mid] == nums[high]`, shrink both ends by one; `[1]*100000 + [2]` is the killer, and no comparison-based method can do better.

Day 046 · dsa — Binary search on the answer

- **The tell is the sentence shape:** *smallest X such that it is possible / largest X such that it still holds*. The array is not sorted and is never searched — the **answers** are.
- **Three things, in order:** bound the answer range (`lo` = smallest *meaningful*, `hi` = something that certainly works) · write `works(x)` as a greedy pass · **say out loud why it is monotone**. Then it is [day 043's](#) template, unchanged.
- **Cost is $O(n \times \log(\text{range}))$** — the check is no longer free. Tight bounds (`max .. sum`) halve the passes against lazy ones (`1 .. 1018`), and `lo` must be meaningful or you divide by zero.

- **No not-found case exists.** `works(hi)` is True by construction; if it is not, your `hi` is wrong and the answer returned is a confident lie.
- **“Exactly” kills monotonicity**, and the maximisation form needs `mid = (lo + hi + 1) // 2` or it hangs — so search for the first `False` and subtract one instead. Same family: LC 875, 1011, 410, 1482.

Day 047 · dsa — Minimise the maximum: the capacity family

- **One question tells you the direction, in one second:** *does more of this make the task easier or harder?* Easier → minimise-the-maximum → **first True**. Harder → maximise-the-minimum → **last True**.
- **Stop asking for the optimum; ask a decision question about a limit.** “Minimise the largest part” becomes “can it be done in $\leq k$ parts with limit L ?” — then it is [day 046](#)’s machine, unchanged.
- **Never mirror the template — negate the question.** Last True = (first False) - 1, with `hi + 1` for the half-open range. The mirrored form needs `(lo + hi + 1) // 2` and hangs silently without it.
- **The greedy check is optimal only because the parts are contiguous** (exchange argument: pushing a boundary as far right as possible never costs a part). Drop “contiguous” and it is NP-hard bin packing — say so.
- **Placement problems must sort first** — $O(n \log n)$, outside the search. The search needs a monotone *question*; the check often needs a sorted *input*, and those are different requirements. Beats DP by $\sim 1,600\times$ on LC 410 at $O(1)$ space.

Day 048 · dsa — Binary search on floats, and the epsilon question

- **On reals the range never empties**, so `while lo < hi` hangs and there is no `± 1` — `lo = mid` and `hi = mid` are correct. The stopping rule is a decision you must make and defend.
- **Prefer a fixed count:** `for _ in range(100)`. A hundred halvings shrink any range by 2^{100} , below double precision at every magnitude, so it cannot hang on any input. $O(1)$ time, $\sim 0.3 \mu\text{s}$.
- **An absolute epsilon has an input class that hangs.** A double holds ~ 15 -17 digits, so near 10^{15} the smallest step is ~ 0.125 and `hi - lo > 1e-9` is unreachable. Use `eps * max(1.0, lo)` if you must use one.
- **`hi = max(1.0, x)`, never `hi = x`** — for `x < 1` the root exceeds `x`, and a range missing the answer returns its own edge, confidently. Return `(lo + hi) / 2`, and never test floats with `=`.

- **If the answer is an integer, search integers** — exact, no epsilon question, and remember the ceiling midpoint `(lo + hi + 1) // 2` for the last-True form. Fixed-point ($\times 10^6$) gives exact decimals without floats.

Day 049 · dsa — Peak finding, and searching data that is structured but not sorted

- **Binary search needs a discard proof, not sortedness.** A peak is any element beating both neighbours; ends count as $-\infty$, so **every array has one** and there is no not-found case.
- **One comparison, right neighbour only:** rises right $\rightarrow$ `lo = mid + 1`; otherwise $\rightarrow$ `hi = mid` (keep `mid`, it may be the peak). Never compare both sides — `nums[mid-1]` at `mid = 0` reads the *last* element silently.
- **The proof is one sentence: the array is finite.** Going uphill either reaches the end (which beats $-\infty$) or turns over (which is a peak). Invariant: `[lo, hi]` always contains a peak.
- `while lo < hi with hi = mid` — day 043's pairing. Non-strict `≤` here hangs; `hi = mid - 1` throws away the answer.
- **A peak is not the maximum.** The maximum is $O(n)$ and unbeatable; a *local* best is $O(\log n)$ — the weakening is what buys the logarithm. Equal adjacent values kill it ($O(n)$); 2D is `O(rows × log cols)` by searching columns and taking each column's max row.

Day 050 · dsa — Binary search revision and mock round

- **Ask “what is the answer?” before “is it sorted?”** A *position* $\rightarrow$ the input needs discardable structure (sorted / rotated / local slope). A *value you choose* $\rightarrow$ search the answer range, and the input's order is irrelevant.
- **Six chapters, one loop.** `first_true` from [day 043](#) — half-open, strict `<`, `hi = mid` / `lo = mid + 1` — with a different question each time. Writing a second loop shape means you should have written a different question.
- **The five bugs, with their inputs:** convention blend (two-element array) · `hi = mid - 1` where `mid` is a candidate · discarding on one comparison instead of a proof · a range that misses the answer (`assert works(hi)`) · a meaningless `lo` (rate 0 $\rightarrow$ `ZeroDivisionError`).
- **The three exits:** “exactly k” kills monotonicity · duplicates/plateaus make it $O(n)$, unavoidably · the *maximum* of an unsorted array is $O(n)$ and unbeatable, where a *peak* is $O(\log n)$.
- **Say the numbers, not the letters:** a million $\rightarrow$ 20 comparisons; two bounds $\rightarrow$ 40; rotated with duplicates $\rightarrow$ up to a million; search-on-answer $\rightarrow O(n \times \log \text{range})$, the log over the *answer* range, so input size multiplies rather than adds. Every tool in the

phase is $O(1)$ space.

Databases from zero

Day 027 · system-design — SQL you must know for interviews

- **SQL is declarative:** say what you want, the planner picks the route. Six clauses cover almost everything.
- **Execution order is** `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT` — which is why `WHERE` cannot see an alias and `ORDER BY` can.
- **`WHERE` filters rows, `HAVING` filters groups.** Put every condition you can in `WHERE`.
- **`COUNT(*)` counts rows, `COUNT(col)` counts non-nulls**, and aggregates skip `NULL`. Use `IS NULL`, never `= NULL`.
- **Always `ORDER BY` before `LIMIT`, and add a tie-breaker.** Run `EXPLAIN ANALYZE` before optimising anything.

Day 028 · system-design — Joins, drawn

- **The only difference is what happens to unmatched rows.** Inner drops them; left keeps the left side; right mirrors; full keeps both.
- **Row counts come from matches, not table sizes.** 3 left rows $\times$ 2 right rows on the same key = 6.
- **`COUNT(o.id)`, not `COUNT(*)`, after a left join** — the unmatched row is still a row.
- **Right-table conditions go in `ON`, not `WHERE`** — in `WHERE` they turn a left join into an inner join.
- **`NOT EXISTS` over `NOT IN`** — one `NULL` in the subquery makes `NOT IN` return nothing. And index every join column.

Day 029 · system-design — Normalisation and when to break it

- **Normalisation = each fact in exactly one place.** 3NF: every non-key column depends on the key, the whole key, and nothing but the key.
- **It prevents three anomalies:** update, insert, delete. Name all three.
- **Denormalise by the read-to-write ratio.** Below $\sim 10:1$, don't. At 265:1, a stored count saves four cores and costs 3.5 writes a second.
- **A snapshot is not a copy.** `order_items.unit_price` is what was charged — it *should* differ from today's price, and it never drifts.
- **Every real copy needs a reconciliation job that reports what it fixed.** No job, no copy — keep the join instead.

Day 030 · system-design — Indexes: how a database finds a row fast

- **An index is a sorted B-tree kept alongside the table.** Fanout ~ 500 , so **three levels covers ten million rows** — a lookup is 3–4 page reads, not a 200 MB scan.
- **Sortedness buys ranges**, `BETWEEN`, prefix `LIKE` and free `ORDER BY` — often the bigger win.
- **Left-prefix rule:** an index on `(A, B)` serves `A` and `A+B`, never `B` alone.
- **Every write updates every index.** Eight indexes = nine writes per insert, and 10–25% storage each.
- **Measure first.** `EXPLAIN ANALYZE`, and check for a function on the column, an `N+1`, or deep `OFFSET` before adding anything. On a live table, `CREATE INDEX CONCURRENTLY`.

Day 031 · system-design — B-trees and why indexes are shaped that way

- **The unit of cost on disk is the page, not the comparison.** A page read is ~ 0.1 ms and fetches 8 KB whatever you asked for.
- **So make each node one page and fill it.** Fanout ~ 500 , so $\log_{500}(10M) = 3$ levels against $\log_2(10M) = 24$.
- **It does not save comparisons** — 27 against 24 — **it saves page reads**, 3 against 24.
- **B+ tree: pointers in the leaves, leaves linked in order.** That is what makes ranges and `ORDER BY` cheap.
- **Sequential keys fill the rightmost leaf and stay cached; random keys scatter and split.** B-trees favour reads; **LSM trees** favour writes.

Day 032 · system-design — Query plans and the slow query

- **The plan is only as good as the estimates.** Compare **estimated rows against actual rows** first — a bad estimate poisons every decision above it, and the fix is `ANALYZE`.
- **Read plans inside out.** `cost` is an estimate; the `actual` line is truth; **actual time is per loop**, so `loops=61` multiplies.
- **Look for:** `Seq Scan` with high `Rows Removed by Filter`, `loops > 1` on a big node, and anything saying `Disk`.
- **Two causes never appear in the plan:** an `ORM N+1`, and deep `OFFSET`. Check the app log and `pg_stat_statements`.
- **Fix in order of cost:** rewrite the query, then add an index, then change the schema. And pick the query by **total** time, not mean.

Day 033 · system-design — Transactions and ACID

- **A transaction is one indivisible unit.** `BEGIN` ... `COMMIT`, and all of it or none of it.
- **Atomicity and Durability** both come from the **write-ahead log**: append the intent, `fsync`, then report success.
- **Consistency** is the constraints you declared — the odd letter out, since you define it and the database enforces it.
- **Isolation** is the interesting one, because it is the only one with **levels** — that is [day 034](#).
- **Keep transactions short.** No external calls inside one; batch bulk writes in thousands, not one at a time and not a million.

Day 034 · system-design — Isolation levels and the anomalies they allow

- **Three read anomalies, in strength order:** dirty read (uncommitted data), non-repeatable read (same row, two answers), phantom (new rows appear in a repeated search). Plus the write one: the lost update.
- **The levels forbid them cumulatively:** read committed kills dirty; repeatable read kills non-repeatable; serialisable kills phantoms — and write skew.
- **Postgres: default read committed; repeatable read = one snapshot per transaction** and stops phantoms too; serialisable = SSI, aborts losers — **retry loop required**.
- **Write skew:** two snapshots, an invariant across rows, writes to *different* rows — no conflict fires. Fix with `FOR UPDATE` on what you checked, or serialisable.
- **Choose per transaction:** default + SQL arithmetic for writes, repeatable read for reports, serialisable (retried) for check-then-act. Say what you are accepting.

Day 035 · system-design — Locking and deadlocks

- **Blocking is a queue; deadlock is a cycle.** T1 holds A wants B, T2 holds B wants A — nobody can ever move. Queues drain; cycles need a victim.
- **The database resolves it:** after `deadlock_timeout` (1 s), find the cycle in the waits-for graph, abort one transaction — `ERROR: deadlock detected` — and the app retries from `BEGIN`.
- **Prevention rule number one: lock in a consistent order** — lowest id first, everywhere; `WHERE id IN (...) ORDER BY id FOR UPDATE` puts the discipline in the query.
- `FOR UPDATE SKIP LOCKED` turns a table into a job queue — workers claim different rows, a crashed worker's job frees itself.
- **Optimistic locking (version column) has no deadlocks** — right when contention is rare; a hot row wants pessimistic locks or database-side arithmetic instead.

Day 036 · system-design — NoSQL: what it actually means

- **NoSQL = four models, one decision:** store data in the shape you will read it. Arranged for serving; relational is arranged for questions.
- **The trade, both directions:** buy designed access paths + horizontal scaling by key; sell ad-hoc queries, cross-record transactions, single-copy facts.
- **“Faster” needs an object.** Faster at the shaped path (assembly moved to write time); slower or silent elsewhere. Speed belongs to a query against an arrangement.
- **The scaling claim, stated honestly:** no joins → split by key → one machine per request. It matters above what one node holds (~tens of thousands of TPS) — most systems never get there.
- **Families and homes:** Redis/DynamoDB key-value, MongoDB document, Cassandra wide-column, Neo4j graph — and Postgres JSONB quietly covers many “need Mongo” cases. Default relational; move access paths, not systems.

Day 037 · system-design — Key-value stores

- **One promise:** GET by exact key, O(1), any size — and it never looks inside the value. Every future question must be a key you wrote at write time.
- **Redis = RAM shelf:** sub-ms, atomic `INCR`, TTL, single-threaded; durability optional (RDB gap / AOF ≈ 1 s) — nothing irreplaceable lives only there.
- **DynamoDB = durable, partitioned by hashed key** (+ optional sort key), single-digit ms at any scale, **priced per request** — chatty designs pay linearly.
- **Homes: sessions, carts, counters, rate limiters, flags** — fetched whole, by identity, often expiring. Sizing: count $\times$ size against RAM.
- **It sits beside the system of record, not instead of it** — Postgres owns the truth, the shelf absorbs every-request reads, and every copy has an owner and a reconciliation path.

Day 038 · system-design — Document databases

- **A document = one entity + its belongings, read and written as one unit; atomicity ends at the document’s edge.** Model so everyday writes never cross it.
- **Embed or reference, by three questions:** read together? change-rate $\times$ copies? unbounded growth? Bounded + read-together + own-able → embed; shared or growing → reference.
- **The rename answer:** truth in `users`, embedded display names fanned out by an idempotent background job — every copy has an owner and a reconciliation path, or the copy is not made.

- **Ceilings:** 16 MB hard cap; reads haul the whole document far sooner. Run size $\times$ growth on every array at design time.
- **Trees tolerate documents; webs want tables.** And nested data alone justifies JSONB, not a second database.

Day 039 · system-design — Wide-column and time-series stores

- **Wide-column = two-part key:** partition key $\rightarrow$ which machine and bucket (hashed); clustering columns $\rightarrow$ sort order inside. The designed query reads one sorted slice; everything else fans out to the whole cluster.
- **One table per query, written at write time** — no joins, no ad-hoc filters; a new question is a new table, not a new index. The promise: *I already know my queries.*
- **Writes scale because of the LSM** — append, flush, background-merge (day 031), every node accepts writes: millions/s per cluster. Price: read merges and compaction forever.
- **Key stress-test: unbounded partitions (bucket by time) and hot partitions (spread the key)** — both are landing-rule decisions, unfixable by tuning later.
- **Time series = the model + compression + retention:** delta encoding $\sim 50\times$; raw briefly, downsampled aggregates long, chunks dropped whole. TimescaleDB beside an app; Prometheus for infra; InfluxDB/Cassandra at cluster scale.

Day 040 · system-design — Choosing SQL or NoSQL in an interview

- **Never answer the binary — answer per access path.** Patterns $\rightarrow$ invariants $\rightarrow$ numbers $\rightarrow$ settledness, and a product name only at the end.
- **Invariants pick the core:** cross-entity transactions and constraints = relational. Webs want tables; trees tolerate anything (day 038).
- **Numbers carve the exceptions:** one node $\approx 10\text{--}50\text{k TPS}$ / terabytes; 1B events/day $\approx 11.6\text{k/s}$ — inside it. Sessions $\rightarrow$ Redis (037); firehose with fixed queries $\rightarrow$ wide-column/TS (039).
- **Every carve-out is a copy with an owner (029) and an operational bill** — default to the store you already run; write trigger numbers instead of paying for scale early.
- **The winning close:** say what evidence would change your mind. The procedure is the answer; the diagram is just today's output.

Day 041 · system-design — Connection pools, ORMs, and the N+1 query

- **N+1 = one query for the list + one per item**, emitted by lazy loading in a loop. Signature: identical statements differing only in an id. Invisible to EXPLAIN — day 032's warning.

- **Latency is paid per conversation turn:** 101×1 ms sequential beats the database at nothing; 2–3 eager queries carry the same data. Fix: declare the read before the loop — join for to-one (`select_related`), batched `IN` for to-many (`prefetch_related`).
- **Guard rails, not vigilance:** query counts visible in dev, `assertNumQueries` in CI, `pg_stat_statements` by call count in production.
- **Pool always:** opening = TCP + TLS + auth + a Postgres process (~5–10 ms, ~7 MB). Size near `cores × 2` — bigger pools are queue positions, not throughput.
- **Fleet arithmetic:** instances × pool vs `max_connections` → PgBouncer multiplexes (transaction mode trades away session state). Most “outgrew Postgres” stories are conversation-shape stories.

Day 042 · system-design — Database revision and interview questions

- **Run the six questions in order, every time:** access paths → numbers → invariants → schema → failure modes → the trigger that would change your mind. Never name a product first.
- **Schema order is fixed:** table per thing, surrogate key, foreign key on the many side, `NOT NULL / NUMERIC / TIMESTAMPTZ`, index every foreign key, 3NF then denormalise only by read-to-write ratio with a reconciliation job.
- **The concurrency answer is one sentence:** do the arithmetic in the `UPDATE` with a `WHERE` guard; use `FOR UPDATE` for check-then-act on one row; use serialisable plus a retry loop when the invariant spans rows.
- **Diagnose in cost order:** `EXPLAIN ANALYZE` (estimated vs actual rows) → `N+1` and deep `OFFSET`, which never appear in the plan → composite index → partition. Most answers are at step two.
- **Numbers to carry:** index lookup 3-4 pages ≈ 0.4 ms; one node ≈ 10 -50k simple reads/s and tens of terabytes; 1B events/day = 11.6k/s; pool $\approx \text{cores} \times 2$. A second database moves one access path, never the system.

Object-oriented design

Day 043 · system-design — Why interviews ask object-oriented design at all

- **The round tests one thing: can somebody else change this code in a year.** Score comes from scoping, from where behaviour lives, from surviving the extension question, and from narrating all three.
- **It is not schema design, not a patterns quiz, not HLD.** No tables, no replica counts, no pattern names before the problem has forced the shape.

- **Spend the 45 minutes: 5 scope · 5 classes · 10 relationships · 15 code one flow · 7 extension · 3 gaps.** Say the plan at minute one. Aim for 6-10 classes and one flow that actually runs.
- **Behaviour goes next to the data it needs.** `spot.can_fit(vehicle)`, never a type-switch in the coordinator. The anaemic model — fields plus a service class — is objects in name only.
- **The extensibility test is “name the second implementation”.** If you can, make an interface; if you cannot, do not. Measure a design in edits to *existing* logic: 4 edits versus 1 line is the whole argument.

Day 044 · system-design — Classes and objects

- **Class = the mould, object = one thing from it.** One copy of the code, one set of state per object; `idol.sell()` is literally `Idol.sell(idol)`, which is why `self` exists.
- **Behaviour lives with the data it needs.** `member.borrow()`, `loan.late_fee(date)` — never a rule written at four call sites. Fields plus a `Service` class is the **anaemic model**, and it is the commonest way to lose this round.
- **Nouns → classes, verbs → methods, then add the noun that was not in the paragraph** — `Loan`, `Ticket`, `Reservation`. That missing class is where the interesting rules live.
- **One responsibility, said in one sentence with no “and”.** If the sentence needs an “and”, it is two classes; a 200-line class or a 20-line method almost always is.
- **is is identity, = is equality.** Define `__eq__` on an entity’s unchanging identity, always define `__hash__` with it, never hash a mutable field. Freeze value objects (`Money`, `date`); leave entities mutable.

Day 045 · system-design — Encapsulation

- **Start from the invariant, not the keyword:** “balance never negative”, “a spot holds one vehicle”. Encapsulation makes the rule *impossible to break* instead of merely documented.
- **The failure is call-site multiplication.** One rule at 5-9 call sites, and the forgotten one ships. Inside one method it is 1 place, forever — and the error lands where somebody can read it.
- **The getter answer: don’t add the getter, add the behaviour.** `get` + `set` is a public field with more typing. Tell, Don’t Ask. A getter is fine only when the caller *displays* the value — if their next line is an `if`, a method is missing.

- **Returning a mutable internal is a public field by another route.** `get_items().append(...)` runs none of your code. Return a tuple, an iterator, or nothing at all.
- **Python: `_` is convention, `__` is name mangling for subclass collisions, `@property` without a setter is real enforcement, `frozen=True` and `__slots__` help.** The intent is identical to Java's `private`; only the enforcement differs, and intent is what is scored.

Day 046 · system-design – Inheritance and its costs

- **Say the sentence first: “every B is an A, with no exception.”** If it needs a qualification, it is has-a — hold an object instead. Inheritance bundles reuse and substitutability; only the second is worth the coupling.
- **Failure 1 — the is-a lie.** Mutable `Square / Rectangle`: set width 5, height 4, expect area 20, get 16. Symptom: `isinstance` checks in callers. (Liskov, properly on [day 055](#).)
- **Failure 2 — the fragile base class.** The parent's internal self-calls become its contract; optimising `add_all` from a loop to `extend` silently breaks every counting subclass, with all parent tests green.
- **Failure 3 — multiple axes.** 3 types × 2 fuels = 6 classes, × trailer = 12, × ownership = 36 against 10 by composition. The class count is the *product* of the axes.
- **It is right when the parent has almost no implementation:** exception trees, abstract base classes as contracts, documented framework hooks, single-axis specialisation. Depth costs comprehension, not speed — past two levels, justify it.

Day 047 · system-design – Polymorphism

- **Show it, never define it:** an `if vehicle.kind == ...` chain, then the same thing as `vehicle.fits(spot)` with no branches. The caller never learns how many kinds exist.
- **The payoff in edits:** a new kind is 1 new file + 1 wiring line, against 4 edits to *working* code — and a missing method fails at construction (ABC) or before running (Protocol) instead of in production.
- **Python has no overloading** (a second `def` replaces the first) — only overriding, plus `singledispatch` . And it needs no common base: **duck typing**, made checkable by `typing.Protocol` or enforced at construction by an `ABC`.
- **The refactor is mechanical:** find the switch → name the question it answers → declare the abstract method → move each branch into its class → replace the switch with the call → **delete the type tag**, or someone rebuilds the switch.

- **Keep the `if` when the set is closed** (days, suits) or you are branching on a *value*. The trigger to refactor is the *second* place switching on the same types. Polymorphism makes new **types** cheap and new **operations** expensive — choose by which axis grows.

Day 048 · system-design — Abstraction and interfaces

- **Interface = the capability, in *your* vocabulary.** Your `Money`, your `ChargeResult`, your exceptions. A vendor type in a signature means the interface belongs to the vendor and bought nothing.
- **The user must not choose the implementation.** `Checkout(gateway)`, never `self._gateway = RazorpayGateway()`. Concrete classes are named once, in a composition root — that is the one line that changes.
- **The adapter translates at its own edge:** their units → your types, their errors → your exceptions (`raise CardDeclined(...)` from `exc`), their workflow words → yours. Three leak types: types, errors, semantics.
- **The test before writing one: can you name the second implementation?** An in-memory **fake for tests** counts, and is usually the real reason — 40 tests from 16 flaky seconds to 8 deterministic milliseconds.
- **Numbers and the check:** swap cost 30-60 edits → 1 file + 1 line; `grep -rn "stripe" src/ --exclude-dir=gateways` should return nothing. **The cost:** an interface offers only what *all* implementations can do — narrow it honestly rather than fake a shared model.

Day 049 · system-design — Composition over inheritance

- **Inheritance welds a variant into the type; composition holds it.** `Vehicle(Size.TRUCK, Battery())` is an *object*, not a class — and it can be swapped after construction, which a subclass never can.
- **The count is the argument:** inheritance multiplies the axes ($3 \times 2 \times 2 \times 3 = 36$), composition adds them ($3 + 2 + 2 + 3 = 10$). A new fuel type is +6 classes against +1.
- **The refactor, in order:** find the axes (adjective-noun class names are the tell) → keep the identity as the class → name the rest as a *capability* (`FuelSystem`, not `ElectricBits`) → declare the interface → move each subclass's code into a component → delegate → **delete the subclasses**.
- **The practical payoff is testing:** hand in a `FakeFuel` returning zero range, instead of building a real object and reaching past encapsulation to set up the case. Separately constructible means separately testable.

- **Concede the cost** (wiring, one forwarding line per exposed method, an extra hop) and **keep inheritance for exception trees, ABCs as contracts, documented framework hooks, and true single-axis specialisation** — all cases where the parent has almost no implementation. *Inherit interfaces, compose implementations.*

Day 050 · system-design — Class diagrams and the UML you will actually draw

- **Five symbols is the whole kit:** the three-compartment box (`+` public, `-` private), a plain **association** line, a **hollow triangle** for inheritance, a **dashed** triangle for implementations, and a **multiplicity** on every line (`1` , `0..1` , `1..*` , `0..*`).
- **6-9 boxes, 30-50% of them detail-free.** Fifteen boxes is a wall — split into “the core” plus one subsystem, and announce the split. A box that will not fit is usually two classes.
- **Draw in order:** central class first with space around it → the containment spine with multiplicities → inheritance groups off to the sides → interfaces at an edge → fields and methods only where the conversation is.
- **Leave out getters, constructors, audit fields, full types, and the diamond argument.** Say “these die with the order” in a sentence rather than encoding it — nobody has ever been marked down for a plain line.
- **Narrate every box with its reason** (“`can_fit` on the spot, because the spot knows its own size”). The diagram enables the conversation; **the code is the record** — a stale diagram is worse than none. Class diagram = structure; sequence diagram = one flow, ≤ 5 participants, only when order is the point.

Prefix sums

Day 037 · dsa — Prefix sums: answering range queries instantly

- **prefix[i] = sum of the first i elements** — $n + 1$ entries, one per *boundary*, sentinel 0 in front. `accumulate(nums, initial=0)`.
- **Range sum = after the end minus before the start:** `prefix[j + 1] - prefix[i]` for inclusive `[i, j]`. Say the sentence, not the indices.
- **The trade: $O(n)$ build and space buys $O(1)$ per query** — worth it when queries are many and the array holds still; one update invalidates everything to its right.
- **Without the sentinel:** ranges from 0 read `prefix[-1]` — the last element, silently. The sentinel and the formula are a set; adopt both or neither.
- **Negatives are fine here** — which windows could not say. Tomorrow: prefix + hash map counts subarrays summing to k, negatives included.

Day 038 · dsa — Subarray sum equals K: prefix plus hash map

- A subarray ending here sums to $k \Leftrightarrow$ some earlier prefix equals `running - k`. Count earlier prefixes with a map; never re-sum.
- `seen[0] = 1` **first, always** — the prefix before the array. Without it, every left-edge subarray is silently missed.
- **Ask, then record.** `count += seen[running - k]` before `seen[running] += 1` — `k = 0` is the input that checks the order.
- **Count-map for “how many”; first-index map (never overwritten, sentinel `{0: -1}`) for “longest”.** Recast $0 \rightarrow -1$ gives equal-0s-and-1s; remainders give divisible-by- k .
- **$O(n)$ time, $O(n)$ space — the space is the price of negatives.** All positive + at-most $\rightarrow$ yesterday’s $O(1)$ window instead. Ask “can values be negative?” before choosing.

Day 039 · dsa — Difference arrays: range updates, cheaply

- **Range update = two boundary marks:** `diff[l] += amount`, `diff[r + 1] -= amount` — $O(1)$ each, any width. Settle once with a running total; the marks compose by plain addition.
- **It is day 037 mirrored:** prefix sums buy $O(1)$ range *reads*; difference arrays buy $O(1)$ range *writes*. Each settles once; mixed read/write workloads go to Fenwick/segment trees, $O(\log n)$.
- **Size `length + 1`** — the step-down for a range ending at the last element needs its slot, and the settle drops it. No guards.
- **Ask the boundary question every time:** inclusive end $\rightarrow$ minus at `r + 1` (flights, bookings); exclusive end $\rightarrow$ minus at `r` (car pooling — off *at* the stop). Last meal, or last night?
- **Marks are promises, not values** — no reads before the settle. Huge coordinates $\rightarrow$ same marks in a sorted map, $O(u \log u)$, range-independent.

Day 040 · dsa — 2D prefix sums and inclusion-exclusion

- **prefix[r][c] = sum of first r rows $\times$ first c columns** — table one bigger each way, zero frame top and left; build with `cell + above + left - overlap`.
- **Query = big – strip above – strip left + corner:** `p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1]`. Derive from the picture; never trust memorised signs.
- **The law: removed twice $\rightarrow$ add back once; added twice $\rightarrow$ subtract once.** Inclusion-exclusion — the same answer for “divisible by 3 or 5” with no matrix in sight.

- **Costs:** $O(\text{rows} \times \text{cols})$ build, $O(1)$ per rectangle — 1.4M ops for 100k queries where re-summing needs 10^{11} . Updates break the deal → rebuild-if-rare or 2D Fenwick.
- **Two build traps:** `[[0]*(c+1)]*(r+1)` aliases rows (day 016); no zero frame → `prefix[-1]` wraps silently (day 037, squared). Comprehension + frame, always.

Day 041 · dsa — Prefix sums revision and mock round

- **Name the chapter before solving — it is a separate act.** The tells: many queries / exactly / negatives / range updates / rectangle — and the tool names itself.
- **The border question is “can values be negative?”** All-positive at-most stays with windows at $O(1)$ space; negatives or “exactly” cross to prefix-plus-map at $O(n)$.
- **Pre-flight, every time:** sentinel · ask-then-record · boundary (r vs $r + 1$) · map contract (count vs frozen first-index) · any index that can reach -1 .
- **Two transfer lessons:** products = prefix idea minus the inverse (no division — restructure as two sweeps); multiple-of- k = equal remainders, map bounded by k .
- **The escalation names, with costs:** rebuild-if-rare; Fenwick tree $O(\log n)$ point-update; segment tree + lazy for range-update-range-query. Name, price, stop.

Two pointers and sliding window

Day 027 · dsa — Two pointers: the idea

- **Two indices, one rule, and an argument for why each move is safe.** That argument is the question.
- **Opposite ends needs sorted input.** Too small → the left value can never work; too big → the right value can never work.
- **Each move deletes a whole row or column of the pair table** — that is why $O(n^2)$ becomes $O(n)$.
- **Three shapes:** opposite ends (sorted, pairs), same direction (compaction), fast and slow (cycles).
- **Unsorted and need original indices** → hash map, $O(n)$ time, $O(n)$ space. Sorted → two pointers, $O(1)$ space.

Day 028 · dsa — Opposite ends: pair sums on a sorted array

- **The invariant:** any valid pair lies between `left` and `right`. Each move discards only pairs already proved impossible — so meeting with nothing found is a **proof**, not a guess.

- **Sortedness is used once:** `nums[right]` is the largest remaining partner. That is the whole dependency.
- **On a hit, move both indices**, then skip repeats of both values. Otherwise you emit the same pair twice.
- **3Sum = sort, fix one, two-pointer the rest.** $O(n^2)$, three duplicate guards in three places.
- `left < right` for pairs; `left ≤ right` when filling positions, as in sorted squares, where you write from the back.

Day 029 · dsa — Same direction: the read pointer and the write pointer

- **Both indices move forward.** `read` visits everything; `write` advances only on a keep. The gap is what you discarded.
- **The invariant:** `write ≤ read`, so overwriting is always safe — and `items[0:write]` is finished and correct.
- **Compare against `items[write - 1]`, not `items[read - 1]`** — what you kept, not what you saw. That makes `k`-duplicates a one-character change.
- **Copy is stable; swap is not** for the discarded group. `[3,1,2,4] → [2,4,3,1]`.
- **Sort Colors: on a 2, `mid` does not advance** — the value from `high` is unexamined. On a 0 it does, because the value from `low` is a known 1.

Day 030 · dsa — Fast and slow pointers

- **Slow moves 1, fast moves 2.** When fast ends, slow is at the **middle**. If they ever meet, there is a **cycle**.
- **Why they must meet:** once both are in the loop the gap shrinks by **exactly one** per step, so it cannot skip zero. That is why fast moves 2, not 3.
- **Cycle start:** reset one to the beginning, then move **both at speed 1**. They meet at the entry.
- `while fast and fast.next` — that order, or you dereference `None` on an even-length list.
- **An array where every value is a valid index is a path.** A repeated value makes a loop, and the loop entry is the duplicate — $O(n)$ time, $O(1)$ space, no modification.

Day 031 · dsa — Fixed-size sliding window

- **Consecutive windows overlap in `k - 1` elements.** Do not recompute — **add the entering, subtract the leaving**. $O(n \cdot k)$ becomes $O(n)$.

- When `i` enters, `i - k` leaves, and the window is then `nums[i-k+1 .. i]`. Say that range out loud to check yourself.
- Build the first window before the loop; start the loop at `k`; start `best` from that window, not from 0.
- Sums, counts and sets slide; `max` does not — subtraction needs an inverse. `max` needs a monotonic deque of indices, still $O(n)$.
- With a `Counter`, delete keys at zero — a zero entry makes every equality comparison fail.

Day 032 · dsa — Variable-size sliding window

- Right edge always advances. Left edge advances only when the rule says so. Neither ever goes back — so $2n$ moves, $O(n)$.
- Minimise → shrink while VALID, record INSIDE the loop. Maximise → shrink while INVALID, record AFTER. Swapping them gives plausible nonsense.
- The condition must be monotonic. Negative numbers break sum-windows — that needs prefix sums plus a deque instead. Ask first.
- Shape B: shrink before adding, or the character you just added triggers the shrink.
- With a count map, `del` at zero — here it breaks the loop condition, not just a comparison.

Day 033 · dsa — Window plus hash map: the longest-substring family

- The window carries a map, and the condition is a property of the map: `len(count)`, `count[x]`, the maximum value, or a comparison against a needed map.
- `del` at zero — mandatory whenever `len(count)` is in the condition, or the shrink loop never ends.
- Maximise: shrink while invalid, record after. Six problems, one skeleton, one line different.
- `left` must never move backwards. Guard the jump with `max(left, last[ch] + 1)`, or shrink one step at a time and the bug cannot exist.
- $O(n)$ time, $O(k)$ space where `k` is the alphabet. When there are only two kinds of value, the map collapses to one integer.

Day 034 · dsa — At-most-K, and the exactly-K trick

- Counting window: `total += right - left + 1` — after the shrink, every start from `left` to `right` gives a valid subarray ending here.

- **Why it works: shortening never hurts** — removing front elements cannot add distinct values. Monotonicity is the licence to count this way.
- `exactly(k) = at_most(k) - at_most(k - 1)` — at-most-($k-1$) nests inside at-most- k , so the difference is exactly- k . Two linear passes, still $O(n)$.
- **Guard the inner call: negative k returns 0** — else `at_most(-1)` walks off the array (`IndexError`), and product-less-than- k needs `k ≤ 1 → 0` for the same reason.
- **“How many” → accumulate; “longest” → max.** Say which before typing. And sums with negatives break the trick — that is prefix sums, [day 038](#).

Day 035 · dsa — Choosing between two pointers and a window, under pressure

- **Route before solving, out loud.** Q1 contiguous? Q2 fixed k ? Q3 best or count? Q4 monotonic? Q5 which pointer shape? Under a minute, every problem.
- **Words route wrong; structure routes right.** “Longest” on a subsequence is no window; “indices” plus sorting destroys the answer — hash map.
- **Two leaves lead out of the family:** negatives in a sum → prefix sums ([day 037](#)); “subsequence” → not this corridor at all.
- **Overlapping source and destination** → **write from the free end** — merge sorted arrays runs backwards, and `i ≥ 0` guards the wrap to `nums1[-1]`.
- **Everything here is $O(n)$.** The differentiator is the first minute, not the asymptotics — one minute of routing against twenty minutes of misroute.

Day 036 · dsa — Two pointers revision and mock round

- **The invariant is the answer.** Seven tools, seven sentences — say the sentence before the code, every time; a loop you cannot defend scores as a guess.
- **Container: discard the shorter line** — it caps every pair it could still form at smaller width. Moving the taller line returns 8 where 49 exists.
- **Counting has two directions.** At-most survives shortening → add `right - left + 1`. At-least survives lengthening → shrink while valid, add `left`. The wrong one computes the complement.
- **Pressure eats, in order: the routing minute, the edge cases, the narration.** Counter each with a ritual — one sentence before code, five inputs by voice, stuck-narration instead of silence.
- **Everything here is $O(n)$ and the interviewer knows it.** The score lives in the sentences between the lines of code.

APPENDIX C

The complete 180-day map

All one hundred and eighty days, both tracks, in order. Days in **bold** are in this volume.

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
027	Two pointers: the idea	SQL you must know for interviews
028	Opposite ends: pair sums on a sorted array	Joins, drawn
029	Same direction: the read pointer and the write pointer	Normalisation and when to break it
030	Fast and slow pointers	Indexes: how a database finds a row fast
031	Fixed-size sliding window	B-trees and why indexes are shaped that way
032	Variable-size sliding window	Query plans and the slow query
033	Window plus hash map: the longest-substring family	Transactions and ACID
034	At-most-K, and the exactly-K trick	Isolation levels and the anomalies they allow
035	Choosing between two pointers and a window, under pressure	Locking and deadlocks
036	Two pointers revision and mock round	NoSQL: what it actually means
037	Prefix sums: answering range queries instantly	Key-value stores
038	Subarray sum equals K: prefix plus hash map	Document databases
039	Difference arrays: range updates, cheaply	Wide-column and time-series stores
040	2D prefix sums and inclusion-exclusion	Choosing SQL or NoSQL in an interview
041	Prefix sums revision and mock round	Connection pools, ORMs, and the $N + 1$ query
042	Binary search: the idea and the invariant	Database revision and interview questions
043	Writing binary search without off-by-one bugs	Why interviews ask object-oriented design at all
044	First and last occurrence	Classes and objects
045	Search in a rotated sorted array	Encapsulation
046	Binary search on the answer	Inheritance and its costs
047	Minimise the maximum: the capacity family	Polymorphism
048	Binary search on floats, and the epsilon question	Abstraction and interfaces
049	Peak finding, and searching data that is structured but not sorted	Composition over inheritance

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
050	Binary search revision and mock round	Class diagrams and the UML you will actually draw

Glossary

Every term this book defines, with the day it first appears. Definitions are the sentence that introduced the term.

4

4 kb [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

A

a replicated log [Day 119](#) — Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around a replicated log: an ordered, numbered list of entries, identical on every machine.

abstract class [Day 052](#) — Definition. An abstract class is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in.

acid [Day 025](#) — When Postgres says COMMIT succeeded, the data is on disk, and that promise is the D in ACID — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [\[day 033\]\(../day-033-window-with-a-map/README.md\)](#).

adaptee [Day 068](#) — The adaptee — the washing machine, which has a different shape and cannot be changed.

adapter [Day 059](#) — An adapter is an implementation that translates to a specific technology (PostgresOrderRepository).

add a tie-breaker [Day 027](#) — And add a tie-breaker — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

algorithm [Day 076](#) — A strategy is an algorithm plugged into a caller; a command is a request with its receiver and arguments already bound, so it can be stored, queued and undone.

always comparable [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

amortised [Day 005](#) — The word for that is amortised: `list.append` is $O(1)$ amortised, meaning any single append might be expensive but a long run of them averages out to constant time.

anaemic model [Day 044](#) — A design where classes hold only data and all the rules live in some `IdolService` is called an anaemic model — objects in name only — and it is the most common way candidates lose this round.

and a direction [Day 079](#) — It carries a floor and a direction — "I am on 4 and I want to go up".

anti-diagonals [Day 016](#) — To collect the down-left diagonals — usually called the anti-diagonals — group cells by the value of $r + c$.

anti-sniping [Day 095](#) — Starting the count again when somebody speaks at the last moment is anti-sniping: extending the deadline whenever a bid arrives near the end.

approximate [Day 102](#) — Also: Redis's LRU is approximate — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access.

array [Day 015](#) — Padma's shelf is an array — the fixed row of numbered boxes you met on [day 009](../day-009-what-an-array-is/README.md).

artefact [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

assignment rule [Day 108](#) — "Walk forwards until you reach a point" is the assignment rule: a key belongs to the first node clockwise.

at-least-once [Day 129](#) — Two: you gain duplicates. Queues deliver at-least-once: if the consumer does not acknowledge in time, the message comes back.

atomic [Day 008](#) — The fix is to make check-and-act atomic — indivisible, so that no other thread can act in the middle of it.

atomicity [Day 025](#) — That is atomicity — the A in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

availability [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *every* non-failing node answers.

aws ecs [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

B

backend [Day 002](#) — The backend is the code that never leaves the kitchen.

bearer token [Day 019](#) — The token is a bearer token: whoever bears it is treated as the user.

best case [Day 002](#) — The best case is 1, the worst case is n.

binary tree [Day 098](#) — A binary tree is a tree where every node has at most two children, called left and right.

blast radius [Day 111](#) — The eleven days revealing water, diesel and medicine is blast radius: what actually stops.

block [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

blocked [Day 007](#) — When the barber stood there holding a head while the boy fetched hair colour, he was blocked — occupying a chair while doing no work.

both [Day 030](#) — Once both are inside the loop, look at the gap between them, measured the way round the loop from fast to slow.

boundary [Day 118](#) — The unifying idea worth stating: *two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.*

bounded context [Day 051](#) — The name for the boundary between those worlds is a bounded context: each context gets its own model of "customer", and they are linked by an id rather than merged.

bounded optimism [Day 116](#) — Spending up to two thousand is bounded optimism — act early where the downside is small.

breadth-first [Day 100](#) — The other family is breadth-first — one whole level at a time — which needs a queue instead of a stack and is [tomorrow](../day-101-bfs-level-order/README.md).

break point [Day 045](#) — The result has one break point — one place where a value is followed by a smaller value.

broken links [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

bucket by time [Day 157](#) — The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so bucket by time: the partition key becomes (channel_id, month).

bucket counts [Day 171](#) — The fix is histograms. Each machine exports bucket counts — "how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms" — and you sum the buckets across machines and compute the percentile from the sum. That is exactly what Prometheus's `histogram_quantile` does, and it is why counters and histograms are the metric types that aggregate and...

bulkhead [Day 111](#) — A bulkhead is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

burst size [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

C

cache [Day 003](#) — Saving it in his phone is caching. A cache is a local copy of an answer, kept so that you do not have to ask again.

cache it [Day 107](#) — Fix: you cannot split a single key across shards, so cache it — the photocopy — or replicate it to every shard, or give it its own shard.

cache stampede [Day 090](#) — And the first cold morning is a cache stampede.

candidates [Day 093](#) — The trays are the candidates — the values you are allowed to pick from.

capacity [Day 005](#) — A list keeps a capacity — how many slots it has room for — which is bigger than its length, the number of slots actually used.

cdn [Day 070](#) — CDN — a geographically distributed caching proxy.

certificate [Day 006](#) — When your browser connects to `google.com`, the machine sends back a certificate — a small document containing the name it claims (`google.com`), a public key, an expiry date, and a signature from a Certificate Authority.

chaining [Day 060](#) — The standard fix is chaining: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket.

channels [Day 092](#) — The group message, the direct message and the phone call are channels — the different ways a message can physically reach a person.

checks permission [Day 011](#) — The kernel checks the arguments and checks permission — this is Balan saying no.

chunks [Day 160](#) — A file is not stored as a file. It is split into chunks — fixed or variable-size blocks, typically around 4 MB — and each chunk is identified by the hash of its contents. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

circular buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position $7 + 1$ is position 0.

class [Day 043](#) — A class is the description; an object is one thing made from that description.

client [Day 068](#) — The client — the wall socket, which will only accept one shape.

collapse key [Day 141](#) — Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a collapse key: a new notification with the same key replaces any undelivered one.

collapsing writes [Day 102](#) — Its real advantage is collapsing writes — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

collection [Day 016](#) — A collection is the set of things: `/bags`.

column [Day 026](#) — A column is one attribute of it, with a type that constrains what can go there.

compact [Day 011](#) — The price is that you now need a separate way to know which slots are real, and you must eventually compact: one pass that closes all the holes at once.

comparator [Day 058](#) — And the two uncles are the one case that needs a comparator — a rule about a pair, not a fact about an item.

comparison [Day 052](#) — A comparison is one question: **is this one bigger than that one?** A swap is exchanging two values so each sits where the other was.

complete [Day 114](#) — Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree complete — any other position leaves a gap and destroys the array layout.

component [Day 069](#) — The tumblers are the component — the real object that does the actual job.

composite [Day 029](#) — This only ever applies when the primary key is composite — two or more columns together.

composite key [Day 026](#) — Its primary key is usually the two columns together — a composite key — which also enforces that the same pair cannot appear twice.

composition [Day 071](#) — Strategy uses composition — the varying part is an object passed in.

composition root [Day 053](#) — The composition root is the one place in the program where the real concrete classes are named and wired together — usually `main()` or a startup module.

connection [Day 041](#) — Under all of this, every query needs a connection — and opening one is genuinely expensive: TCP, then TLS ([[day 006](#)](../day-006-python-strings-dicts-sets/README.md)'s handshake), authentication, and in Postgres a whole operating-system process per connection ([[day 008](#)](../day-008-reading-a-problem/README.md)'s processes, ~5–10 MB each).

connection-oriented [Day 004](#) — This is why TCP is called connection-oriented — there is a real, agreed-upon connection with state on both sides.

consequence [Day 109](#) — The lorries are the consequence — the number that changes a decision, which is the only number that actually mattered.

consistency [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that **every** non-failing node answers.

constraints [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

constructor [Day 044](#) — `__init__` is the constructor: the code that runs when a new object is made, whose job is to leave the object in a valid state.

container [Day 013](#) — A container is an ordinary process — the same kind of process you met on [[day 008](#)](../day-008-reading-a-problem/README.md) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume.

container image [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

containerd [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

context [Day 073](#) — The object — the context — holds one of those classes and forwards every request to it.

coordinator [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

correlation id [Day 091](#) — So every record carries a correlation id — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

cost [Day 032](#) — It enumerates the plausible combinations, assigns each a cost — an abstract number, not milliseconds — and picks the smallest.

count map [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

counter [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

cri-o [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

critical path [Day 134](#) — The number of levels is the critical path — the minimum time to finish everything if you had unlimited workers.

cycle [Day 125](#) — Cyclic or acyclic is the third. A cycle is a path that starts and ends at the same vertex without reusing an edge.

D

dangerous [Day 104](#) — Compact, and dangerous: any statement that is not deterministic gives a different result on the follower.

dashed [Day 050](#) — Four: implements an interface — the same triangle, but a dashed line:

dau [Day 097](#) — DAU — daily active users.

deep-copied [Day 067](#) — The cooker and the dabbas were deep-copied — genuinely new things that happen to look the same.

deleted [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

delta of delta [Day 137](#) — So you store the difference of the differences — the delta of delta — which is almost always zero, and zero costs one bit.

dependency [Day 053](#) — A dependency is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.

depth [Day 046](#) — First, depth: with Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

depth-first [Day 100](#) — Going depth-first means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

deque [Day 031](#) — The fix for max is to keep a deque — a double-ended queue, from [day 074](../day-074-deques-and-window-max/README.md) — holding indices, arranged so that their values are decreasing from front to back.

diminishing returns [Day 135](#) — But with diminishing returns: the eighth mention adds much less than the second.

direct addressing [Day 066](#) — The wall of numbered holes is direct addressing: because the keys are the numbers 1 to 120, you do not need to hash anything.

domain name [Day 003](#) — "Anjali Sharma" is the domain name. A domain name is a human-readable label for a machine or a group of machines: google.com, leetcode.com, api.github.com.

draw two diagrams [Day 050](#) — If your design genuinely has fifteen classes, draw two diagrams: the core five or six, and then a second showing one subsystem in detail.

duck typing [Day 047](#) — This is duck typing: if it has a send method that takes a string, it is a notifier.

dummy node [Day 080](#) — Sister Mary is a dummy node — sometimes called a sentinel head.

E

earlier [Day 118](#) — The safe pattern: the leader treats its lease as expiring earlier than the grantor does, so there is a gap in which nobody believes they are leader.

edge cases [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

edge compute [Day 103](#) — There is a real modern exception, and mentioning it is a good signal: edge compute — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin.

effect [Day 113](#) — Say it that way: "exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key." Systems that advertise exactly-once are doing precisely this.

elastic scaling [Day 100](#) — The locum starting with no handover is elastic scaling: add a machine and it is immediately useful.

election timeout [Day 118](#) — "By half past five" is the election timeout — the guess that the leader is gone.

empty [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

etl [Day 139](#) — ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce.

event object [Day 072](#) — In practice you push a small, immutable event object — a plain record describing what happened, like `OrderPlaced(order_id, user_id, total, items)`.

eventual consistency [Day 115](#) — At the bottom is eventual consistency: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile.

exactly [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

exactly correct [Day 089](#) — This is exactly correct — no boundary effect, no approximation — and it is the version you already wrote on [\[day 077\]\(../day-077-stacks-queues-revision/README.md\)](#) as a deque of timestamps.

exactly one [Day 171](#) — XOR is exactly one — which is the same as "different", and that is the property the whole of the next XOR day rests on.

exclusive lock [Day 035](#) — When a transaction updates a row, the database gives it an exclusive lock on that row: nobody else may change the row until the transaction commits or rolls back.

expiry [Day 090](#) — The dates are expiry — a completely separate mechanism.

F

facts [Day 082](#) — So: a FinePolicy. The loan holds the facts — who, what, when it went out, when it was due, when it came back.

false negative [Day 124](#) — A false negative is failing to notice a machine that really has died.

false positive [Day 124](#) — A false positive is declaring a healthy machine dead.

fanout [Day 030](#) — The critical property is the fanout: because a page is 8 KB and an entry is small, one node holds hundreds of entries.

fee [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day."

fifo [Day 090](#) — FIFO — evict the oldest inserted.

file [Day 093](#) — A file is a thing that is not a box: a plate, a lamp.

filled diamond [Day 050](#) — A filled diamond means *composition* — the parts die with the whole, so `Order` ♦ — `OrderLine` because a line item has no meaning without its order.

finite state machine [Day 073](#) — This is a finite state machine — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which.

float [Day 081](#) — Devi's second tin is a float: coins reserved so tomorrow can start.

four-direction move [Day 096](#) — "Each one immediately beside the one before" is the four-direction move: up, down, left, right.

fragile [Day 021](#) — It is also fragile: it silently breaks on capital letters, spaces, digits and anything non-English, and §7 shows exactly how silently.

frontend [Day 002](#) — The frontend is the code that crosses the counter.

G

galloping mode [Day 057](#) — When merging, if one run keeps winning, Timsort switches to galloping mode — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time.

gap [Day 047](#) — The gap is the answer she is searching for — not something in the array, a number she is choosing.

generator expression [Day 020](#) — That is a generator expression — it produces each item as join asks for it.

greedy simulation [Day 046](#) — It is a greedy simulation: load as much as fits, then start a new trip.

guest operating system [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

H

hash map [Day 066](#) — Kiran's phone is a hash map: a general system that can look up anything by any key, at the price of storing the keys and computing where they go.

hash ring [Day 108](#) — The eleven-kilometre loop is the hash ring — the space of all hash values, joined end to end.

hateoas [Day 016](#) — And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: HATEOAS — Hypermedia As The Engine Of Application State.

header interface [Day 058](#) — A header interface is named after the implementation and lists everything it can do: Repository, Machine, UserService.

heap [Day 117](#) — The six letters on the table are the heap: one candidate per list.

heapsort [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

heartbeat [Day 124](#) — The whistle is a heartbeat. A heartbeat is a small, regular message that says "I am alive".

hidden spof [Day 111](#) — The shared approach road is a hidden SPOF — two components that are not independent.

hiding mechanism [Day 052](#) — Abstraction is about hiding mechanism: the caller says "charge this card" and does not learn which payment provider ran it.

highest node [Day 104](#) — Every path has exactly one highest node — the point where it turns around.

historical snapshot [Day 029](#) — | Copy | Instead of | Why | |---|---|---| | posts.comment_count | COUNT(*) on comments | counting on every read is unaffordable at a high read ratio | | orders.customer_name | joining to customers | a historical snapshot: the name at the time of the order | | order_items.unit_price | joining to products | the price *then*, which must...

hollow triangle [Day 050](#) — Three: inheritance — a line with a hollow triangle pointing at the parent:

hook [Day 075](#) — award is a hook: the base class implements it as **doing nothing**, so a subclass may override it and most do not.

horizontal scaling [Day 098](#) — Horizontal scaling means putting more machines beside it and dividing the work.

hypervisor [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

I

image [Day 013](#) — An image is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code.

implicit graph [Day 130](#) — This is an implicit graph — the edges exist by rule rather than by record — and you met the idea on [\[day 125\]\(../day-125-what-a-graph-is/README.md\)](#).

in parallel [Day 111](#) — Components in parallel — any one will do — multiply their **failure** probabilities, which is a completely different shape.

in series [Day 111](#) — Components in series — every one is needed — multiply their availabilities, so the total is always worse than the worst one.

in-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

index [Day 025](#) — A database keeps an index — a separate structure, usually a B-tree, that maps a value to the location of the matching rows.

indexed heap [Day 114](#) — Hence the indexed heap: a hash map from value to index, updated on every swap.

inheritance [Day 071](#) — Template Method uses inheritance — an abstract base with the skeleton and subclasses filling in the holes.

inorder [Day 110](#) — The standing order is inorder: left subtree, then root, then right subtree.

input [Day 008](#) — In a problem: the input — its type, its size, and what is in it.

integrity [Day 006](#) — TLS also guarantees integrity: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection.

intent plus multiplicity [Day 070](#) — The honest distinction is intent plus multiplicity: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

interface [Day 052](#) — An interface is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

invariant [Day 028](#) — That is called an invariant — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

is possible [Day 046](#) — $>$ **Find the smallest X such that some task is possible.** $>$ **Find the largest X such that some condition still holds.**

item [Day 016](#) — An item is one of them: `/bags/213`.

iterator [Day 075](#) — Iterator is the pattern where "go through this collection one item at a time" is separated from "this collection".

J

json [Day 015](#) — JSON — JavaScript Object Notation — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [\[day 007\]\(../day-007-space-complexity/README.md\)](#); it looks like `{"id": 42, "name": "Zoya"}`.

jump [Day 094](#) — A snake and a ladder are both a jump: a pair (from, to).

jwt [Day 020](#) — a session id — an opaque reference, meaningless by itself, that the server looks up; - a JWT — a self-contained signed token that carries the facts inside it, so no lookup is needed; - OAuth 2.0 and OpenID Connect — a protocol for getting one of the above from *somebody else*, so a user can log in with Google without your service ever...

K

k closest points [Day 116](#) — k closest points — the same shape with a computed key.

k most frequent [Day 116](#) — k most frequent — count first, then top-k on the counts.

k-th largest specifically [Day 116](#) — k-th largest specifically — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

key [Day 023](#) — The key is as much of the design as the algorithm.

key property [Day 108](#) — Adding the temple point disturbing only sixty houses is the key property: adding a node moves only the keys between it and its predecessor.

L

lag [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

largest [Day 055](#) — the 1st smallest is at position 0, so the k-th smallest is at position k - 1; - the 1st largest is at position n - 1, so the k-th largest is at position n - k.

latency [Day 089](#) — The cost is latency — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

lazy [Day 117](#) — Merging sorted streams — `heapq.merge(*iterables)` does exactly this and is lazy, so it works on inputs larger than memory.

lazy deletion [Day 114](#) — Python has neither. The idiomatic workaround is lazy deletion: push a new entry and mark the old one stale, skipping stale entries as they surface.

lazy loading [Day 041](#) — That is lazy loading: the ingredient bought at the moment it is needed.

lazy refill [Day 089](#) — The lazy refill is the implementation trick: no timer, no background job.

leader [Day 105](#) — Counter two is the leader: writes go there, and reads from it are always current.

leaf [Day 098](#) — He is a leaf — an item with no children.

lease [Day 118](#) — A lease is leadership with an expiry: you are the leader until time T unless you renew.

ledger entries [Day 085](#) — An expense produces a set of ledger entries: the payer is owed, each participant owes their share.

left [Day 040](#) — Remove the strip to the left: `prefix[r2 + 1][c1]`.

lfu [Day 090](#) — LFU — evict the least frequently used.

linear scan [Day 062](#) — Scrolling the whole list of names is a linear scan: to answer one question you touch everything.

linearizability [Day 114](#) — C — Consistency. Specifically linearizability: every read sees the most recent write, and the system behaves as if there were one copy of the data.

list [Day 001](#) — heap is a list: a row of values kept in order, like the socks piled on the bed.

list of lists [Day 016](#) — A matrix is a list of lists: the outer list holds the rows, and each row is itself a list holding that row's values.

load balancer [Day 098](#) — Sending students to the right counter is a load balancer.

load factor [Day 060](#) — The number that controls this is the load factor: the number of items divided by the number of buckets.

load shedding [Day 007](#) — This idea has a name — load shedding — and it is one of the things that separates a system that degrades from one that collapses.

log-structured merge tree [Day 031](#) — A log-structured merge tree — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade.

logical cohesion [Day 061](#) — Anjali's first kitchen is logical cohesion — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind.

logical time [Day 113](#) — The replacement is logical time — Lamport clocks and vector clocks — which order events by *causality* rather than by wall time.

loop [Day 001](#) — A loop is a statement that says, "do the following again and again".

loop body [Day 002](#) — A loop body is the indented block underneath the loop header — the part that runs again and again.

low cohesion [Day 061](#) — The kitchen arranged by size is low cohesion — things used together kept apart, so every job takes trips.

lru [Day 090](#) — LRU — evict the least recently used.

M

manifest [Day 158](#) — Adaptive bitrate streaming. The client downloads a manifest — an HLS .m3u8 or DASH .mpd — listing every rendition and every segment.

many times [Day 149](#) — Virtual nodes fix that. Each physical machine is placed on the circle many times — 100 to 256 positions each, by hashing machine-1#0, machine-1#1, and so on.

many-to-many [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

matrix [Day 016](#) — A matrix is a rectangle of values addressed by two numbers instead of one: which row, then which column.

mau [Day 097](#) — MAU — monthly active users.

max-heap [Day 116](#) — The mirror rule: to find the k smallest, keep a max-heap of size k — in Python, a min-heap of negated values.

method resolution order [Day 052](#) — The Python answer. The method resolution order — `D._mro_` — is computed by a rule called C3 linearisation, and `super()` follows that order rather than jumping straight to a parent.

milliseconds [Day 009](#) — About 10 milliseconds — a hundred times slower again than SSD, and a hundred thousand times slower than RAM.

min-heap [Day 118](#) — Each half is a heap, and they face each other: a max-heap holding the lower half so its largest is at the top, and a min-heap holding the upper half so its smallest is at the top — the two elements at the boundary are exactly the ones you need. And the only real work is keeping the two halves the same size, which is two lines and is...

minimum spanning forest [Day 139](#) — Both algorithms then produce a minimum spanning forest — one tree per component — and you detect it by counting: fewer than $V - 1$ edges taken means the graph was disconnected.

mixin [Day 049](#) — A mixin is a small class carrying one piece of behaviour, meant to be inherited alongside others:

monotone [Day 043](#) — A yes-or-no question with that shape is called monotone: once the answer flips from no to yes, it stays yes.

monotonic deque [Day 074](#) — The good solution keeps a small monotonic deque of "elements that could still be the maximum" and is $O(n)$ overall.

N

name mangling [Day 045](#) — Two underscores triggers name mangling: inside the class the attribute is rewritten as `_Account_key`, so a subclass defining its own `_key` does not collide.

namespace [Day 013](#) — A namespace is a kernel feature that gives a process its own private version of one part of the system.

negative [Day 035](#) — A sum over values that can be negative is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [\[day 037\]\(../day-037-prefix-sums/README.md\)](#).

negative infinity [Day 049](#) — LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as negative infinity — imaginary values smaller than anything real, just off each end.

negatives [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

nested loop [Day 001](#) — A nested loop is a loop inside another loop.

network partition [Day 114](#) — The exchange fault is a network partition — the nodes are alive but cannot talk.

never equal [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

never forgets [Day 090](#) — Needs a third structure and a `min_frequency` counter to stay $O(1)$, and it never forgets: an entry that was hot last month holds a huge count and will not leave.

never persisted [Day 157](#) — Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and never persisted — a typing event that arrives late is worse than one that is dropped.

no children [Day 121](#) — It has no children — nothing continues past it.

node [Day 125](#) — The word node means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code.

not a map [Day 067](#) — | not a map — heap or sorted array |

O

$o(1)$ [Day 079](#) — tail — a reference to the last node, which makes append $O(1)$ instead of $O(n)$.

object [Day 043](#) — An object is data and the behaviour that belongs to that data, kept together in one place.

observers [Day 072](#) — The objects on the list are the observers.

occupied [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

offset [Day 130](#) — Each viewer's position is an offset. An offset is just a number — the position in the log that a particular reader has got to.

on first touch [Day 041](#) — Each `post.author` is a `*different table*`, not yet loaded — so the ORM, helpfully and silently, fetches it on first touch.

one-shot [Day 075](#) — A list can be walked many times; a generator is one-shot — walk it twice and the second walk is empty, which is a bug people hit constantly.

opens [Day 126](#) — When failures exceed a threshold, it opens: from then on, calls to that dependency fail instantly without being attempted.

orchestrator [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

origin [Day 101](#) — The storeroom is the origin — the source of truth.

origin pull [Day 103](#) — Sending the finished pages down the wire is origin pull: the edge fetches from the origin on demand.

orphan rows [Day 026](#) — Without it you get orphan rows — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

orphans [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

out-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

overloading [Day 047](#) — Compile-time polymorphism, or overloading: several methods with the same name and different argument types, and the compiler picks.

overriding [Day 047](#) — Runtime polymorphism, or overriding: a subclass replaces a parent's method, and which one runs is decided by the object at run time.

P

page [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

parallel edges [Day 132](#) — And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — parallel edges — then A—B twice genuinely *is* a cycle, and skipping "the parent vertex" wrongly ignores it.

parameters [Day 015](#) — In an API those are the parameters — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

parent [Day 132](#) — The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its parent — and the check becomes "seen, and not my parent".

partial failure [Day 113](#) — The one that matters most is partial failure: a call over a network has a third outcome that does not exist locally, *"I do not know"*, and almost every difficult problem in the rest of this phase is a consequence of it.

participants [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

partition key [Day 039](#) — A wide-column store — Cassandra is the name to know — organises data by two keys: a partition key that decides which machine (and which bucket on it) a row belongs to, and clustering columns that keep rows **sorted inside** that bucket.

partitions [Day 130](#) — It is split into partitions — independent logs that together make up the topic.

path [Day 005](#) — A request has four parts: a method saying what you want done, a path saying what you want it done to, headers carrying facts about the request itself, and an optional body carrying the content.

port [Day 003](#) — A port is a second number that says which program on that machine you want, the way a flat number says which door inside the building.

prefix sums [Day 037](#) — His noted meter readings are the prefix sums — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

preorder [Day 110](#) — The calling order is preorder: root, then left subtree, then right subtree.

priority queue [Day 139](#) — Finding "the cheapest edge leaving the tree" efficiently means a priority queue — which makes Prim's structurally almost identical to [Dijkstra's](../day-136-dijkstra/README.md).

procedure [Day 062](#) — It is a procedure: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first.

process [Day 008](#) — A process is a running program with its own private memory.

program [Day 001](#) — A program is a sequence of statements.

protecting state [Day 052](#) — Definition. Encapsulation is about protecting state: the balance cannot go negative because the only way to change it runs a check.

Q

qps [Day 097](#) — QPS stands for *queries per second* — the number of requests arriving at your system each second.

queue [Day 127](#) — The queue holds the frontier in order. A queue is a line where things come out in the order they went in — first in, first out — which you met on [day 73](../day-073-queues/README.md).

quorum [Day 118](#) — "Three of the four must agree" is a quorum — the majority from [day 117](../day-117-merge-k-sorted/README.md).

R

race condition [Day 008](#) — A race condition is when the result depends on the exact timing of two things running at once.

ram [Day 009](#) — The suitcase on the almirah — main memory, RAM. RAM — random access memory — is where your running program's data lives.

rank [Day 138](#) — Union by rank is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

rebalancing [Day 107](#) — Moving twenty bundles is rebalancing — moving whole logical shards, not rehashing keys.

rebuilding the tree [Day 109](#) — Repacking the whole cart is rebuilding the tree — $O(n)$.

recursion [Day 053](#) — That is recursion: a function whose body calls the same function on a smaller piece of the problem.

references [Day 009](#) — A Python list stores references — memory addresses of objects that live elsewhere:

refill rate [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

register [Day 009](#) — The chair beside the bed — registers. A register is a tiny store inside the processor itself, holding one value.

relative epsilon [Day 048](#) — The fix is a relative epsilon — stop when the gap is small *compared to the value*:

removal [Day 118](#) — The same two heaps, plus removal, which a heap cannot do.

replica [Day 136](#) — A replica is a copy of a shard on another node, serving reads and taking over if the primary fails.

replica divergence [Day 115](#) — Different people knowing different things is replica divergence — and none of them is wrong.

request-aware routing [Day 099](#) — The joint-family card going to counter one is request-aware routing — some requests are much more expensive than others.

resolver [Day 021](#) — Each field in the schema has a resolver — a function that knows how to produce that field.

resources [Day 017](#) — Before writing a single path, list the resources — the things the feature is about.

returned [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day. ^{*}

ring buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position 7 + 1 is position 0.

role interface [Day 058](#) — A role interface is named after what a client needs: UserReader, Printer, PriceQuoter.

root [Day 093](#) — The root is the loft itself, written /.

rotation [Day 109](#) — The fix is a rotation: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order.

routine [Day 109](#) — Counting paces and multiplying is the routine — the same sequence every time, said out loud.

row [Day 026](#) — A row is one of those things.

run [Day 044](#) — So the copies of a value form one run: a contiguous stretch, described completely by its two ends.

S

same remainder [Day 038](#) — A stretch is divisible by k when the prefixes at its two ends leave the same remainder — so carry running % k and count matching remainders.

schema [Day 021](#) — A GraphQL API is defined by a schema: every type, every field, and every field's type, written down and strongly typed.

scoping [Day 096](#) — The two or three narrowing questions are scoping — deciding out loud what you are **not** building.

script [Day 077](#) — Today is the script — the fixed order of moves you run on every one of these prompts for the next twenty days.

seam [Day 053](#) — A seam is a place where you can change behaviour without editing the class.

search space [Day 042](#) — The stretch `nums[low..high]` is called the search space — the part of the array you have not ruled out yet.

selectivity [Day 032](#) — From those numbers the planner estimates selectivity: what fraction of rows a condition keeps.

sentinel [Day 037](#) — That extra entry is called a sentinel: a value placed at the boundary so that the boundary needs no special treatment.

separator [Day 019](#) — The string before the `.join` is the separator — `"".join` glues with nothing between, `",".join` puts a comma and a space between each pair.

sequence diagram [Day 050](#) — That is a sequence diagram: participants as columns, time running down, and each message an arrow from one column to the next.

serialisable [Day 033](#) — The strongest form is serialisable: the result is as if the transactions had run one after another, in some order.

service credits [Day 173](#) — An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always service credits — money back off the next bill — rather than damages.

session [Day 020](#) — A session is a record the server keeps about a logged-in user.

session guarantees [Day 115](#) — And the interesting rungs are in the middle, particularly causal consistency and the four session guarantees, because they are what users actually notice and they are much cheaper than the top.

set [Day 006](#) — A set stores values and answers "is this in here?" in a single step, no matter how many it holds.

shallow-copied [Day 067](#) — The milk and the newspaper were shallow-copied — the new kitchen got a *reference* to the same arrangement, and changing it from either end changed it for both.

shape [Day 003](#) — It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the shape of the growth.

shard [Day 106](#) — Each room is a shard — a machine holding a subset of the rows.

shared lock [Day 035](#) — A shared lock is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other.

shared store [Day 100](#) — The shared cabinet at the front desk is a shared store — a database, or Redis.

singleton [Day 064](#) — Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it.

singly linked list [Day 076](#) — A singly linked list — where each node knows only the *next* one — cannot do that.

slice [Day 019](#) — `s[1:4]` is a slice — a new string made of part of the old one.

smallest [Day 046](#) — `>` *Find the smallest X such that some task is possible.* `>` *Find the largest X such that some condition still holds.*

snapshot [Day 034](#) — The transaction takes a snapshot at its first read and every statement in it sees that snapshot, however long it runs.

snipe [Day 095](#) — A snipe is a bid placed in the last second, leaving nobody time to respond.

sorted region [Day 052](#) — And the sorted region is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in *which end* the region grows from and *how* the next value gets there.

sorting everything [Day 116](#) — The nephew laying out the whole load is sorting everything — correct, and far more work than asked for.

splits [Day 031](#) — If the leaf is full, it splits: half the keys move to a new page, and a key is pushed up to the parent.

spof [Day 111](#) — The bridge is an SPOF — one component, and everything depends on it.

squares [Day 003](#) — | Example | |---|---|---| | $O(1)$ | constant | does not change | reading items[5] | | $O(\log n)$ | logarithmic | goes up by one step | halving until you reach 1 | | $O(n)$ | linear | doubles | one loop over the list | | $O(n \log n)$ | linearithmic | slightly more than doubles | sorting | | $O(n^2)$ | quadratic | goes up four times | every...

stable [Day 051](#) — Python's sort is Timsort: $O(n \log n)$ worst case, stable, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to $O(n)$.

stack [Day 075](#) — The fridge is a stack: last in, first out.

stack frame [Day 068](#) — When a function calls another function, the machine pushes a stack frame — the local variables and the place to return to — onto a stack.

stack overflow [Day 068](#) — And the bad Sunday is a stack overflow: too many unfinished things, and the one at the bottom is forgotten.

state [Day 143](#) — And the state is the thing to get right. The state is what identifies a subproblem — the arguments to the recursive function.

statements [Day 001](#) — A program is a sequence of statements.

status [Day 086](#) — It has a status — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

still one machine [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

stonith [Day 118](#) — STONITH — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.

strategy [Day 071](#) — Strategy turns an algorithm into a value.

strictly shorter [Day 072](#) — `left[i]` — the index of the nearest bar to the left that is strictly shorter.

structural [Day 058](#) — In Python, `typing.Protocol` is structural — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing (`[day 052](../day-052-quadratic-sorts/README.md)`).

subarray [Day 024](#) — There is a third word you will meet: a subarray is the same thing as a substring, for arrays.

subject [Day 072](#) — Observer is the pattern where one object — the subject — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call.

subscription list [Day 072](#) — The group itself is the subscription list, and the one message is the notification.

substring [Day 155](#) — First the definitions. A substring is contiguous — "aba" is a substring of "xabay", but "aa" is not.

subtree [Day 122](#) — If you get to the end, the node you are standing on is the root of a subtree — the whole set of words that begin with what the user typed.

surge multiplier especially [Day 088](#) — The surge multiplier especially is frozen.

swap [Day 052](#) — A comparison is one question: `*is this one bigger than that one?*` A swap is exchanging two values so each sits where the other was.

system call [Day 011](#) — The mechanism for asking is a system call, and the boundary it crosses is the most important line in a computer.

T

table [Day 026](#) — A table holds one kind of thing.

tail call [Day 088](#) — A tail call is a recursive call whose result is returned directly, with nothing left to do afterwards:

tail-recursive [Day 090](#) — This version is also tail-recursive — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free.

tcp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

template [Day 092](#) — The saved wording with the date and the hours swapped in is a template — a fixed piece of text with holes in it, filled in per message.

test double [Day 053](#) — A test double is a stand-in you pass in during a test.

the return address [Day 088](#) — the arguments it was called with, - the local variables it creates, - the return address — the exact point in the caller to resume at, - and a link to the frame below it.

the running median [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — `[day 116](../day-116-top-k/README.md)` - merging k sorted lists — `[day 117](../day-117-merge-k-sorted/README.md)` - the running median — `[day 118](../day-118-two-heaps/README.md)`, with two heaps -...

third table [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

thread [Day 008](#) — A thread is one line of execution inside a process, and all the threads in a process share that memory.

three distinct orderings [Day 092](#) — = 6 decision paths — but only three distinct orderings: [1,1,2], [1,2,1], [2,1,1].

three-way partition [Day 054](#) — The fix is a three-way partition: split into *less than*, *equal to*, and *greater than*, and recurse only into the outer two.

tight coupling [Day 061](#) — The balcony light on the reading lamp's switch is tight coupling — two things wired together that have no reason to be, so you cannot change one without changing the other.

time-to-live [Day 037](#) — Bhaskar's clear-out is time-to-live: attach an expiry to a key and the store deletes it, automatically.

tiny constraint [Day 097](#) — A tiny constraint: $n \leq 8$ means factorial, $n \leq 20$ means 2^n , target ≤ 500 with small candidates means combination sum.

toast [Day 134](#) — And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — TOAST — in a side table, compressed, and only reads them when the column is selected.

token [Day 019](#) — The server verifies them and issues a token — a session id or a signed token.

tombstone [Day 011](#) — There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a tombstone — and databases and hash tables use it constantly.

top-k problems [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

total order [Day 123](#) — It gives you a total order — sort by the number, break ties by machine name — that never contradicts causality.

transitions itself [Day 071](#) — In State, the object transitions itself: a Draft order moves itself to Submitted, and the states know about each other.

transitive dependency [Day 029](#) — This is a transitive dependency: $id \rightarrow department_id \rightarrow department_name$.

tree [Day 040](#) — A domain of independent records — a tree — tolerates anything.

ttl [Day 003](#) — The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a TTL — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again.

tuple [Day 005](#) — A tuple is written with round brackets — (3, 7, 2) — and is a list you cannot change.

two [Day 087](#) — In fib, each call makes two — so the work explodes.

two implementations [Day 079](#) — And, as always, two implementations: NearestSuitable as above, and ZonedDispatcher where each car owns a band of floors.

U

udp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

under-fetching [Day 021](#) — And from [day 015](../day-015-the-write-pointer/README.md)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — under-fetching, usually called the N + 1 problem when it happens in a list.

union filesystem [Day 013](#) — The stack of layers is a union filesystem — several directories presented as one merged tree.

upper bound [Day 044](#) — The upper bound is the first index where `nums[i] > 4`, which is index 4 — one past the run.

uri [Day 016](#) — In REST the thing is called a resource, and its address is a URI — the path in the URL.

url [Day 001](#) — A URL is an address for something on the internet.

V

value object [Day 051](#) — A value object is defined entirely by its values.

version [Day 117](#) — And the second half, which people forget: you must be able to tell which value is newest. The overlapping replica has the new value, but the others may return an old one, so every record carries a version — a counter, a timestamp, or a vector clock — and the reader takes the highest.

vertex [Day 125](#) — The places are the vertices. A vertex is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid.

vertical scaling [Day 098](#) — Vertical scaling means replacing the machine with a bigger one — more cores, more memory, faster disk.

victim [Day 035](#) — It watches who waits for whom, and when it finds a circle, it picks a victim — one transaction in the cycle — and kills it with an error.

virtual address space [Day 011](#) — Memory. Each process gets its own virtual address space — its own set of addresses that the OS maps onto real physical memory.

virtual machine [Day 013](#) — A virtual machine is an emulated computer.

volatile [Day 009](#) — It is also volatile: switch the power off and it is gone.

W

weight [Day 122](#) — To do that in code, every word needs a weight — a number saying how often it is chosen.

which class [Day 076](#) — Factory vs Builder — a factory decides which class; a builder assembles one complicated object step by step.

which is hours [Day 166](#) — The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, which is hours — and there is a separate fast path for trending terms, which is the third idea below.

within [Day 131](#) — Consumer group — Kafka's answer, and it is genuinely both: within a group, consumers compete like a queue; between groups, each gets everything like a topic.

worst case [Day 002](#) — The best case is 1, the worst case is n.

write skew [Day 034](#) — It appears in backend interviews at every product company, usually right after ACID, and the follow-up about write skew is where senior candidates are separated from the table-reciters.

write-around with invalidation [Day 102](#) — Wiping a line when the price moves is write-around with invalidation: update the truth, delete the cache entry, and let the next reader repopulate it.

write-through [Day 102](#) — Changing the board at four fifteen is write-through: update the cache when you update the truth.

Y

yes **Day 093** — | Problem | Recurse on | Duplicates in input | Extra rule | |---|---|---| | Combinations (LC 77) | $i + 1$ | no | stop at size k | | Combination Sum (LC 39) | i — reuse allowed | no | stop when remaining $= 0$ | | Combination Sum II (LC 40) | $i + 1$ — each used once | yes | sort, and skip when $i > \text{start}$ and $c[i] == c[i-1]$ | |...

References and further reading

Further reading, arranged by the part of the book it belongs to. Everything listed is a primary source, an official specification, or reference material that is free to read. None of this book is copied from any of them. They are here for the reader who wants the long version, the formal proof, or the original paper.

FOR THE WHOLE BOOK

Python Software Foundation. *The Python Language Reference and Standard Library.* <https://docs.python.org/3/> [PSF Licence, free to read]

The authority for every language behaviour this book relies on.

Python Wiki. *TimeComplexity - cost of the built-in container operations.* <https://wiki.python.org/moin/TimeComplexity> [Free to read]

The table behind almost every complexity claim about list, dict and set.

David Galles, University of San Francisco. *Data Structure Visualizations.* <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html> [Free to use]

Step-through animations for most structures in Parts II to XV.

Steven Halim and others. *VisuAlgo - visualising data structures and algorithms.* <https://visualgo.net/en> [Free for self-study]

Useful when a diagram in this book is not enough and you want to press play.

MIT OpenCourseWare. *6.006 Introduction to Algorithms.* <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Full lecture videos and notes. The formal treatment this book deliberately avoids.

LeetCode. *Problem set.* <https://leetcode.com/problemset/> [Free tier]

Where every problem named in the practice sheets is solved.

DATA STRUCTURES AND ALGORITHMS, BY PART

PART IV · TWO POINTERS AND SLIDING WINDOW (DAYS 27–36)

Wikipedia. *Dutch national flag problem.* https://en.wikipedia.org/wiki/Dutch_national_flag_problem [CC BY-SA]

Three pointers walking one array. The purest statement of the pattern.

Wikipedia. *Maximum subarray problem.* https://en.wikipedia.org/wiki/Maximum_subarray_problem [CC BY-SA]

Kadane's scan, which is the window argument in its shortest form.

Python Software Foundation. *collections.deque.* <https://docs.python.org/3/library/collections.html#collections.deque> [PSF Licence]

The structure behind monotonic-window problems.

PART V · PREFIX SUMS (DAYS 37–41)

Python Software Foundation. *itertools.accumulate.* <https://docs.python.org/3/library/itertools.html#itertools.accumulate> [PSF Licence]

The one-line prefix sum you would actually write in an interview.

Wikipedia. *Prefix sum.* https://en.wikipedia.org/wiki/Prefix_sum [CC BY-SA]

Includes the parallel version, which is a good follow-up answer.

Wikipedia. *Summed-area table.* https://en.wikipedia.org/wiki/Summed-area_table [CC BY-SA]

The 2D prefix sum of Day 40, under the name computer vision gave it.

PART VI · BINARY SEARCH (DAYS 42–50)

Python Software Foundation. *bisect - array bisection algorithm.* <https://docs.python.org/3/library/bisect.html> [PSF Licence]

bisect_left and bisect_right are the library form of Days 43 and 44.

Wikipedia. *Binary search algorithm.* https://en.wikipedia.org/wiki/Binary_search [CC BY-SA]

The section on implementation issues covers the midpoint overflow bug.

SYSTEM DESIGN, BY PHASE

DATABASES FROM ZERO (DAYS 25–42)

PostgreSQL Global Development Group. *PostgreSQL documentation.* <https://www.postgresql.org/docs/current/> [PostgreSQL Licence]

The chapters on indexes, transactions and explain are the ones to read.

PostgreSQL Global Development Group. *Transaction Isolation.* <https://www.postgresql.org/docs/current/transaction-iso.html> [PostgreSQL Licence]

Read-committed against repeatable-read against serialisable, with examples.

Wikipedia. *Database normalization.* https://en.wikipedia.org/wiki/Database_normalization [CC BY-SA]

First through third normal form, which is as far as interviews go.

Wikipedia. *B-tree.* <https://en.wikipedia.org/wiki/B-tree> [CC BY-SA]

Why a database index is not a hash map.

MongoDB. *MongoDB Manual - data modelling.* <https://www.mongodb.com/docs/manual/data-modeling/> [Free to read]

The document-model side of the SQL against NoSQL question.

OBJECT-ORIENTED DESIGN (DAYS 43–54)

Python Software Foundation. *Classes tutorial.* <https://docs.python.org/3/tutorial/classes.html> [PSF Licence]

Inheritance and the method resolution order, from the source.

Python Software Foundation. *abc - abstract base classes.* <https://docs.python.org/3/library/abc.html> [PSF Licence]

How to express an interface in Python, which comes up in every LLD round.

Wikipedia. *Object-oriented programming.* https://en.wikipedia.org/wiki/Object-oriented_programming [CC BY-SA]

Encapsulation, inheritance, polymorphism, with the criticisms included.

Colophon: how this book was made

This book was typeset from the course sources with free software only. Every typeface is published under the SIL Open Font Licence, every library under a permissive licence, and the page engine is the open-source Chromium print pipeline. Licence labels below describe the linked project, not this book.

TYPE, TOOLS AND LIBRARIES

SIL International, after Matthew Carter. *Charis SIL*. <https://software.sil.org/charis/> [SIL Open Font Licence 1.1]

The text face. An extension of Bitstream Charter, which Carter drew in 1987 to stay readable on coarse output. It is the standard free choice for technical books.

Bold Monday for IBM. *IBM Plex Sans and Plex Sans Condensed*. <https://github.com/IBM/plex> [SIL Open Font Licence 1.1]

Headings, labels, tables, folios and the covers.

JetBrains. *JetBrains Mono*. <https://github.com/JetBrains/JetBrainsMono> [SIL Open Font Licence 1.1]

All code, output and ASCII diagrams.

Google. *Noto Serif Devanagari*. <https://github.com/notofonts/devanagari> [SIL Open Font Licence 1.1]

The Devanagari on the covers and title page.

Knut Sveidqvist and contributors. *Mermaid*. <https://github.com/mermaid-js/mermaid> [MIT Licence]

Renders every flow chart, sequence diagram and tree.

Georg Brandl and contributors. *Pygments*. <https://pygments.org/> [BSD 2-Clause]

Colours the code. The palette used here was written for this book.

Waylan Limberg and contributors. *Python-Markdown*. <https://python-markdown.github.io/> [BSD 3-Clause]

Turns the lesson sources into the pages you are reading.

Martin Thoma and contributors. *pypdf*. <https://github.com/py-pdf/pypdf> [BSD 3-Clause]

Joins the covers, front matter and body, and writes the bookmarks.

The Chromium Authors. *Chromium print-to-PDF, driven by Puppeteer*. <https://pptr.dev/> [Apache 2.0 / BSD 3-Clause]

The typesetting engine. No proprietary software was used to make this book.

HOW A PAGE WAS MADE

The lesson sources are Markdown. Fenced blocks are lifted out before the Markdown is parsed, so diagrams keep every space they have and code reaches the colouring pass untouched. Each monospace block is measured, and its type size chosen so the widest line fits the column rather than falling off the page.

The result is rendered in a browser engine at 6.75 by 9.5 inches and printed to PDF. The body is printed first so that the contents and the index can be built from the page numbers the printing actually produced, then the covers and front matter are printed and the three are joined.

Problem index

Every problem named in a practice sheet, in the order the book reaches it, with its source and the chapter that sets it. Problem statements are not reproduced anywhere in this book. Solve them where they live.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
027	Two Sum II — Input Array Is Sorted	LeetCode 167 (Medium)	The base case, and whether you notice it wants 1-indexed positions.
027	Container With Most Water	LeetCode 11 (Medium)	A *different* elimination argument. Moving the taller line is the wrong move, and you should be able to say why in one sentence.
027	3Sum	LeetCode 15 (Medium)	That it is not three pointers. Fix one, two-pointer the rest — and the duplicate-skipping is most of the work.
027	Trapping Rain Water	LeetCode 42 (Hard)	The two-pointer version, after you have written the prefix-max one. The argument here is genuinely subtle and worth the hour.
028	Two Sum II — Input Array Is Sorted	LeetCode 167 (Medium)	The skeleton, the completeness argument, and that the output is 1-indexed.
028	Squares of a Sorted Array	LeetCode 977 (Easy)	Writing from the back, and $\text{left} \leq \text{right}$ rather than $<$. The $O(n)$ answer versus the $O(n \log n)$ one-liner.
028	3Sum	LeetCode 15 (Medium)	Fix one, two-pointer the rest — and three duplicate guards in three different places.
028	3Sum Closest	LeetCode 16 (Medium)	The same skeleton tracking a best-so-far. Note that you can no longer stop early on a mismatch.
029	Remove Duplicates from Sorted Array	LeetCode 26 (Easy)	Compare-against-what-you-kept. $\text{nums}[\text{write} - 1]$, and the empty guard that silently returns 1 without it.
029	Remove Duplicates from Sorted Array II	LeetCode 80 (Medium)	The same, at $\text{write} - 2$. Should be a one-character change. If it is a rewrite, problem 1 compared against the wrong thing.
029	Sort Colors	LeetCode 75 (Medium)	Partition, three pointers. The whole question is why mid advances on a 0 and not on a 2.
029	Backspace String Compare	LeetCode 844 (Easy)	Walk from the back. Easy with $O(n)$ space; the question is the $O(1)$ version.
030	Middle of the Linked List	LeetCode 876 (Easy)	Two speeds, one pass. The condition order fast and fast.next, and which of the two middles.
030	Linked List Cycle	LeetCode 141 (Easy)	Detection. Give the hash-set answer with its space cost, then Floyd's.
030	Happy Number	LeetCode 202 (Easy)	The path is a function on integers. Note what the meeting value tells you.
030	Find the Duplicate Number	LeetCode 287 (Medium)	The path is $i \rightarrow \text{nums}[i]$. The three constraints rule out every obvious approach, and the loop entry *is* the answer.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
031	Maximum Average Subarray I	LeetCode 643 (Easy)	The core slide, and dividing once at the end rather than inside the loop.
031	Contains Duplicate II	LeetCode 219 (Easy)	The window is $k + 1$ wide, so the element leaving is at $i - k - 1$. Off by one from every other problem here, on purpose.
031	Find All Anagrams in a String	LeetCode 438 (Medium)	Window plus a Counter, and the del at zero without which every comparison fails after the first zero.
031	Sliding Window Maximum	LeetCode 239 (Hard)	The monotonic deque. Easy problem, one sentence away, and a completely different technique — because max has no inverse.
032	Minimum Size Subarray Sum	LeetCode 209 (Medium)	Shape A. Record inside the shrink loop. And whether you ask about negative values.
032	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	Shape B. Shrink before adding, record after.
032	Longest Repeating Character Replacement	LeetCode 424 (Medium)	Shape B with a subtle validity test, and an optimisation whose correctness argument is the real question.
032	Minimum Window Substring	LeetCode 76 (Hard)	Shape A with a count map and a missing counter. The hardest window problem asked routinely.
033	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	The skeleton itself, and whether left can ever move backwards.
033	Fruit Into Baskets	LeetCode 904 (Medium)	Recognising "at most 2 distinct" through a story about fruit.
033	Max Consecutive Ones III	LeetCode 1004 (Medium)	Noticing the map collapses to a single counter when the alphabet is size two.
033	Longest Repeating Character Replacement	LeetCode 424 (Medium)	The <code>window_len - max_freq > k</code> condition, and defending the <code>max_freq</code> optimisation.
034	Subarray Product Less Than K	LeetCode 713 (Medium)	The counting line <code>total += right - left + 1</code> , no subtraction needed — and the <code>k <= 1</code> guard.
034	Binary Subarrays With Sum	LeetCode 930 (Medium)	The subtraction, the collapsed map, and whether <code>goal = 0</code> crashes you.
034	Count Number of Nice Subarrays	LeetCode 1248 (Medium)	Seeing "exactly k odd numbers" as the same problem after one observation.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
034	Subarrays with K Different Integers	LeetCode 992 (Hard)	The full trick with a real map — a hard problem that is two tested calls and a minus sign.
035	Merge Sorted Array	LeetCode 88 (Easy)	Routing to pointers, then the real lesson: overlapping destination → write from the back.
035	Backspace String Compare	LeetCode 844 (Easy)	First route (stack) versus the follow-up route (two pointers, backwards, $O(1)$ space).
035	Longest Subarray of 1's After Deleting One Element	LeetCode 1493 (Medium)	Window, maximise shape, collapsed map — and the right - left twist for the forced deletion.
035	Maximum Sum of Distinct Subarrays of Length K	LeetCode 2461 (Medium)	Hearing "length k" over "distinct": fixed window carrying a sum and a map together.
036	Container With Most Water	LeetCode 11 (Medium)	Whether you can *produce* the discard argument — moving the taller line can never help.
036	Number of Substrings Containing All Three Characters	LeetCode 1358 (Medium)	Re-deriving the counting line for an at-least condition instead of pasting yesterday's.
036	3Sum Closest	LeetCode 16 (Medium)	Opposite ends inside a fixed outer loop, unseen variant — tracking nearest instead of equal.
036	Minimum Window Substring	LeetCode 76 (Hard)	The phase's summit, revisited cold: shape A with a need map and a missing counter.
037	Running Sum of 1d Array	LeetCode 1480 (Easy)	The build loop itself — $\text{prefix}[i + 1] = \text{prefix}[i] + \text{nums}[i]$ with nothing else in the way.
037	Range Sum Query — Immutable	LeetCode 303 (Easy)	The sentinel and the formula as a set; the constructor/method split.
037	Find Pivot Index	LeetCode 724 (Easy)	The running-total form — no stored array, check before adding, empty left side at index 0.
037	Left and Right Sum Differences	LeetCode 2574 (Easy)	Two directions of the same idea; whether your boundaries stay honest when one runs backwards.
038	Subarray Sum Equals K	LeetCode 560 (Medium)	The machine itself: $\text{seen}[0] = 1$, ask-then-record, and why the window is illegal.
038	Subarray Sums Divisible by K	LeetCode 974 (Medium)	The remainder costume — same remainder at both ends — and Python's friendly %.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
038	Contiguous Array	LeetCode 525 (Medium)	The longest variant: recast $0 \rightarrow -1$, first-index map, sentinel $\{0: -1\}$, never overwrite.
038	Maximum Size Subarray Sum Equals k	LeetCode 325 (Medium)	Problem 1's quantity with problem 3's map — proof you know which contract is which.
039	Range Addition	LeetCode 370 (Medium, premium — or code it from §5)	The pure pattern: two marks, extra slot, one settle.
039	Corporate Flight Bookings	LeetCode 1109 (Medium)	1-indexed inclusive ends — deriving $\text{diff}[\text{first}-1]$ and $\text{diff}[\text{last}]$ instead of memorising.
039	Car Pooling	LeetCode 1094 (Medium)	The exclusive end — off $*\text{at}*$ the stop — and the settle doubling as a capacity check.
039	Number of Zero-Filled Subarrays	LeetCode 2348 (Medium)	A cooldown problem: no difference array at all — proof you route by structure, not by phase.
040	Range Sum Query 2D — Immutable	LeetCode 304 (Medium)	The build and the four-term query, with the zero frame doing the edge work.
040	Matrix Block Sum	LeetCode 1314 (Medium)	A loop of $O(1)$ queries with edge clamping — and keeping the two index worlds separate.
040	Count Square Submatrices with All Ones	LeetCode 1277 (Medium)	A ringer: it looks 2D-prefix-shaped and wants a different recurrence — route by structure.
040	Number of Submatrices That Sum to Target	LeetCode 1074 (Hard)	The composition: fix row pairs, collapse columns, run day 038's map on the 1D remainder.
041	Find the Highest Altitude	LeetCode 1732 (Easy)	The warm-up: running prefix in one variable, sentinel 0 as the starting altitude.
041	Product of Array Except Self	LeetCode 238 (Medium)	Whether the prefix instinct survives a change of operation — and why division is banned.
041	Continuous Subarray Sum	LeetCode 523 (Medium)	The remainder disguise, the frozen first-index map, and the length-2 rule.
041	Range Sum Query 2D — Immutable	LeetCode 304 (Medium)	The phase's four-corner summit, re-derived cold a day later.
042	Binary Search	LeetCode 704 (Easy)	The template itself: $\leq$, middle + 1, middle - 1, and -1 as a conclusion.
042	Search Insert Position	LeetCode 35 (Easy)	Whether you noticed where low ends up when the target is absent.
042	Guess Number Higher or Lower	LeetCode 374 (Easy)	Binary search with no array at all — the space is a range of numbers.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
042	Valid Perfect Square	LeetCode 367 (Easy)	The first sight of searching an answer range instead of data.
043	Search Insert Position	LeetCode 35 (Easy)	The lower bound with nothing added — and that "one past the end" is a real answer.
043	First Bad Version	LeetCode 278 (Easy)	The template with no array at all; the question is an API call.
043	Find Smallest Letter Greater Than Target	LeetCode 744 (Easy)	The upper bound, plus a wrap-around that the template does not solve for you.
043	Find First and Last Position of Element in Sorted Array	LeetCode 34 (Medium)	Both bounds, and whether you resist the find-then-walk shortcut.
044	Find First and Last Position of Element in Sorted Array	LeetCode 34 (Medium)	The two bounds, and whether you resist find-then-walk.
044	Kth Missing Positive Number	LeetCode 1539 (Easy)	A boundary question where the array is not what you search on.
044	Find K Closest Elements	LeetCode 658 (Medium)	The bound as a *starting point* for a window, not as the answer itself.
045	Find Minimum in Rotated Sorted Array	LeetCode 153 (Medium)	The break point, and comparing against nums[high] rather than nums[low].
045	Search in Rotated Sorted Array	LeetCode 33 (Medium)	Discarding on a *range* instead of a single comparison.
045	Search in Rotated Sorted Array II	LeetCode 81 (Medium)	Whether you can say "O(n) worst case" and explain why it is unavoidable.
045	Find Minimum in Rotated Sorted Array II	LeetCode 154 (Hard)	The same duplicate problem on the cleaner cousin — and the one-line guard.
046	Koko Eating Bananas	LeetCode 875 (Medium)	The friendliest entry: bounds of 1 to max, and ceiling division.
046	Capacity To Ship Packages Within D Days	LeetCode 1011 (Medium)	Bounds of max to sum, and the day counter starting at one.
046	Split Array Largest Sum	LeetCode 410 (Hard)	That a problem which reads like dynamic programming is this instead.
046	Minimum Number of Days to Make m Bouquets	LeetCode 1482 (Medium)	A feasibility check with a run-length rule inside it.
047	Minimized Maximum of Products Distributed to Any Store	LeetCode 2064 (Medium)	The gentlest minimise-the-maximum, with a ceiling-division check.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
047	Split Array Largest Sum	LeetCode 410 (Hard)	Whether you reach for binary search instead of dynamic programming.
047	Magnetic Force Between Two Balls	LeetCode 1552 (Medium)	The mirror: maximise the minimum, and the sort you must not forget.
047	Divide Chocolate	LeetCode 1231 (Hard)	The same mirror behind deliberately confusing wording.
048	Sqrt(x)	LeetCode 69 (Easy)	Whether you notice the answer is an integer and stay in integers.
048	Square root to six decimal places	Standard follow-up to LC 69	The stopping rule, and the $\max(1.0, x)$ bound.
048	Pow(x, n)	LeetCode 50 (Medium)	Not binary search on the answer — binary *exponentiation*. Know the difference.
048	Minimize Max Distance to Gas Station	LeetCode 774 (Hard)	The full float search: an expensive check, so the iteration count is a real decision.
049	Peak Index in a Mountain Array	LeetCode 852 (Medium)	The rule, with the answer guaranteed unique so nothing can distract you.
049	Find Peak Element	LeetCode 162 (Medium)	The same code with the guarantee removed — and whether you accept a non-unique answer.
049	Find Minimum in Rotated Sorted Array	LeetCode 153 (Medium)	The mirror: a valley. Re-derive it as local structure, not as a rotation trick.
049	Find a Peak Element II	LeetCode 1901 (Medium)	The 2D version, and whether you search columns and take each column's maximum row.
050	Find First and Last Position of Element in Sorted Array	LeetCode 34 (Medium)	The warm-up: two bounds, and whether find-then-walk still tempts you.
050	Search in Rotated Sorted Array	LeetCode 33 (Medium)	Discarding on a range, and the $\leq$ in the sortedness test, cold.
050	Koko Eating Bananas	LeetCode 875 (Medium)	Whether the answer-space instinct fires without the word "capacity" in it.
050	Split Array Largest Sum	LeetCode 410 (Hard)	The phase's summit, re-derived a week later, with the DP named and rejected.

H

Index

Page numbers in **bold** are where the term is defined. Terms are those the book defines; the glossary in Appendix D gives each one a sentence.

#

4 kb, 186, 385, 386, 393, 411, 831, 832, 842

A

a replicated log, 831
abstract class, 724, 800, 831
acid, 240, 260, 262, 267, 269, 270, 273, 275, 276, ...
adaptee, 831
adapter, 659, 728, 729, 730, 731, 732, 733, 734, 738, ...
add a tie-breaker, 22, 24, 32, 813, 831
algorithm, 9, 16, 40, 51, 55, 64, 66, 70, 76, ...
always comparable, 831, 841
amortised, 831
anaemic model, 556, 561, **583**, 589, 594, 598, 819, 831
and a direction, 831
anti-diagonals, 831
anti-sniping, 831
approximate, 831
array, 1, 4, 5, 6, 9, 10, 12, 13, 15, ...
artefact, 832, 834
assignment rule, 832
at-least-once, 832
atomic, 261, 265, 267, 303, 358, 360, 362, 366, 384, ...
atomicity, 261, 412, 831, 832
availability, 832, 834, 839, 846
aws ecs, 832, 842

B

backend, 140, 219, 295, 326, 692, 727, 760, 832, 848
bearer token, 832
best case, 16, 17, 37, 56, 146, 222, 832, 848
binary tree, 161, 181, 186, 832
blast radius, 832
block, 3, 4, 40, 45, 47, 145, 146, 147, 149, ...
blocked, 330, 832
both, 4, 5, 6, 10, 12, 16, 24, 31, **122**, ...
boundary, 372, 375, 412, 448, 537, 538, 540, 542, 544, ...
bounded context, 832
bounded optimism, 832
breadth-first, 832

break point, 601, 611, 612, 613, 614, 629, 802, 810, 832, ...
broken links, 832, 842
bucket by time, 440, 445, 817, 832
bucket counts, 833
bulkhead, 833
burst size, 833, 843

C

cache, 148, 183, 185, 187, 188, 192, 224, 269, 357, ...
cache it, 833
cache stampede, 833
candidates, 28, 68, 110, 150, 188, 229, 270, 332, 557, ...
capacity, 301, 331, 385, 428, 429, 495, 505, 633, 636, ...
cdn, 833, 839, 846
certificate, 833
chaining, 833
channels, 771, 833
checks permission, 833
chunks, 268, 438, 439, 440, 441, 444, 445, 446, 449, ...
circular buffer, 833, 844
class, 27, 37, 86, 95, 118, 211, 358, 374, **535**, ...
client, 726, 727, 737, 771, 833, 840, 844
collapse key, 833
collapsing writes, 833
collection, 167, 170, 209, 250, 287, 319, 348, 357, 401, ...
column, 7, 8, 12, 13, 15, 17, 18, 21, 22, ...
compact, 8, 183, 559, 651, 688, 689, 691, 698, 833, ...
comparator, 834
comparison, 65, 66, 90, 131, 165, 172, 180, 186, 197, ...
complete, 11, 20, 23, 24, 29, 31, 49, 63, 65, ...
component, 553, 763, 834, 837, 838, 841, 845
composite, 100, 142, 149, 834
composite key, 100, 834
composition, 365, 476, 656, 658, 659, 727, 728, 739, 755, ...
composition root, 727, 728, 834
connection, 268, **479**, 491, 493, 494, 496, 497, 500, 503, ...
connection-oriented, 834
consequence, 834, 842, 844
consistency, 262, 831, 832, 834, 836, 839, 845

constraints, 133, 834, 836, 854
constructor, 831, 834, 856
container, 9, 345, 832, 834, 835, 850, 854, 856
container image, 832, 834
containerd, 834, 835
context, 832, 834
coordinator, 267, 269, 272, 561, 819, 834, 842
correlation id, 834
cost, 12, 14, 15, 16, 25, 28, 33, 34, 200, ...
count map, 164, 203, 207, 209, 216, 218, 234, 255, 282, ...
counter, 3, 27, 35, 95, 98, 109, 164, 165, 169, ...
cri-o, 834, 835
critical path, 835
cycle, 5, 8, 18, 34, 122, 124, 128, 131, 132, ...

D

dangerous, 62, 835
dashed, 791, 792, 795, 798, 800, 804, 822, 835
dau, 425, 835
deep-copied, 835
deleted, 81, 83, 295, 317, 319, 413, 421, 439, 445, ...
delta of delta, 835
dependency, 100, 101, 728, 835, 842, 847
depth, 9, 11, 14, 16, 27, 34, 58, 141, 654, ...
depth-first, 835
deque, 165, 166, 167, 168, 170, 172, 174, 175, 176, ...
diminishing returns, 835
direct addressing, 835
domain name, 835
draw two diagrams, 835
duck typing, 689, 695, 724, 835
dummy node, 835

E

earlier, 94, 138, 177, 205, 215, 247, 256, 258, 280, ...
edge cases, 56, 218, 344, 353, 553, 577, 601, 827, 834, ...
edge compute, 836
effect, 133, 148, 180, 260, 267, 270, 301, 302, 305, ...
elastic scaling, 836
election timeout, 836
empty, 9, 42, 45, 55, 56, 62, 74, 85, 92, ...
etl, 98, 174, 358, 363, 380, 384, 388, 411, 462, ...
event object, 836
eventual consistency, 836
exactly, 1, 3, 4, 5, 12, 17, 28, 30, 396, ...
exactly correct, 836
exactly one, 5, 12, 17, 51, 55, 56, 66, 77, 84, ...
exclusive lock, 331, 836
expiry, 382, 388, 389, 392, 833, 836, 839, 847

F

facts, 84, 111, 113, 276, 354, 355, 360, 361, 363, ...
false negative, 836
false positive, 836
fanout, 144, 148, 183, 186, 188, 836
fee, 102, 108, 110, 227, 287, 321, 337, 354, 581, ...
fifo, 836
file, 183, 186, 188, 190, 191, 271, 290, 321, 332, ...
filled diamond, 795, 797, 836
finite state machine, 837
float, 167, 170, 202, 206, 207, 209, 210, 214, 217, ...
four-direction move, 837
fragile, 653, 656, 658, 837
frontend, 837

G

galloping mode, 837
gap, 5, 7, 8, 12, 25, 56, 64, 70, 669, ...
generator expression, 837
greedy simulation, 837
guest operating system, 837, 838

H

hash map, 1, 6, 12, 13, 15, 16, 18, 35, 54, ...
hash ring, 837
hateoas, 837
header interface, 837
heap, 19, 27, 109, 113, 146, 149, 152, 172, 176, ...
heapsort, 837
heartbeat, 837
hidden spof, 837
hiding mechanism, 837
highest node, 837
historical snapshot, 837
hollow triangle, 795, 838
hook, 656, 662, 663, 768, 820, 822, 838
horizontal scaling, 359, 838
hypervisor, 837, 838

I

image, 5, 8, 11, 15, 17, 58, 452, 511, 605, ...
implicit graph, 838
in parallel, 838
in series, 838
in-degree, 838, 842
index, 7, 12, 14, 16, 17, 19, 25, 26, 27, ...
indexed heap, 838
inheritance, 633, 650, 651, 654, 656, 659, 660, 724, 739, ...
inorder, 838, 844
input, 5, 6, 8, 11, 12, 13, 16, 18, 33, ...

integrity, 838
intent plus multiplicity, 838
interface, 552, 554, 556, 557, 621, 660, 704, 721, 724, ...
invariant, 81, 117, 199, 299, 345, 365, 467, 469, 483, ...
is possible, 639, 775, 838, 845
item, 61, 63, 65, 66, 68, 69, 70, 71, 76, ...
iterator, 619, 623, 627, 628, 820, 838

J

json, 147, 356, 358, 362, 363, 408, 411, 412, 414, ...
jump, 124, 135, 137, 138, 145, 204, 213, 231, 235, ...
jwt, 839

K

k closest points, 839
k most frequent, 839
k-th largest specifically, 839
key, 24, 30, 31, 55, 58, 61, 63, 64, 65, ...
key property, 839

L

lag, 8, 10, 31, 71, 79, 82, 84, 88, 93, ...
largest, 4, 5, 11, 13, 15, 16, 17, 37, 42, ...
latency, 188, 190, 227, 230, 266, 268, 269, 270, 273, ...
lazy, 41, 433, 489, 490, 491, 492, 493, 495, 497, ...
lazy deletion, 839
lazy loading, 493, 839
lazy refill, 839
leader, 358, 384, 438, 444, 836, 839, 846
leaf, 144, 145, 182, 183, 184, 185, 187, 189, 190, ...
lease, 312, 328, 329, 335, 494, 495, 496, 503, 554, ...
ledger entries, 839
left, 3, 4, 5, 7, 8, 9, 11, 12, 452, ...
lfu, 839
linear scan, 839
linearizability, 839
list, 3, 5, 9, 10, 11, 12, 17, 34, 46, ...
list of lists, 840
load balancer, 840
load factor, 840
load shedding, 840
log-structured merge tree, 840
logical cohesion, 840
logical time, 840
loop, 2, 3, 4, 5, 6, 7, 8, 12, 14, ...
loop body, 349, 840
low cohesion, 840
lru, 384, 831, 840

M

manifest, 840

many times, 211, 215, 237, 397, 406, 492, 546, 567, 577, ...
many-to-many, 61, 69, 522, 840, 847
matrix, 451, 454, 455, 457, 458, 840, 857
mau, 840
max-heap, 172, 176, 840
method resolution order, 840, 851
milliseconds, 729, 834, 840
min-heap, 840
minimum spanning forest, 841
mixin, 459, 657, 658, 660, 758, 759, 763, 771, 841
monotone, 533, 545, 595, 606, 639, 641, 644, 675, 778, ...
monotonic deque, 165, 841, 855

N

name mangling, 618, 625, 626, 628, 820, 841
namespace, 841
negative, 50, 133, 170, 207, 210, 212, 235, 289, 315, ...
negative infinity, 746, 841
negatives, 50, 235, 315, 322, 345, 401, 402, 482, 483, ...
nested loop, 2, 12, 51, 65, 66, 146, 223, 224, 226, ...
network partition, 841
never equal, 93, 831, 841
never forgets, 841
never persisted, 841
no children, 839, 841
node, 34, 124, 133, 136, 138, 141, 146, 155, 156, ...
not a map, 243, 841

O

o(1), 4, 6, 9, 11, 12, 13, 16, 17, 18, ...
object, 19, 90, 128, 131, 134, 328, 363, 378, 535, ...
observers, 841
occupied, 835, 836, 841
offset, 22, 27, 30, 140, 147, 149, 150, 151, 153, ...
on first touch, 492, 497, 841
one-shot, 842
opens, 84, 179, 323, 370, 408, 493, 507, 615, 688, ...
orchestrator, 832, 842
origin, 11, 12, 13, 322, 836, 842, 850
origin pull, 842
orphan rows, 842
orphans, 832, 842
out-degree, 838, 842
overloading, 842
overriding, 842

P

page, 20, 26, 31, 63, 65, 66, 71, 110, 112, ...
parallel edges, 842
parameters, 842

parent, 338, 658, 689, 838, 840, 842, 845
partial failure, 842
participants, 834, 842, 844
partition key, 437, 448, 832, 842
partitions, 842
path, 106, 111, 122, 123, 124, 125, 126, 127, 131, ...
port, 3, 17, 22, 24, 26, 28, 31, 35, 58, ...
prefix sums, 315, 368, 369, 370, 391, 451, 452, 475, 479, ...
preorder, 843
priority queue, 843
procedure, 102, 224, 476, 843
process, 262, 264, 330, 494, 496, 526, 834, 841, 843, ...
program, 673, 679, 723, 834, 842, 843, 846, 851, 858, ...
protecting state, 843

Q

qps, 843
queue, 106, 165, 219, 223, 265, 312, 313, 327, 329, ...
quorum, 843

R

race condition, 843
ram, 3, 38, 77, 82, 94, 120, 127, 137, 142, ...
rank, 25, 147, 215, 333, 494, 843
rebalancing, 843
rebuilding the tree, 843
recursion, 843
references, 105, 159, 588, 832, 842, 843, 850
refill rate, 833, 843
register, 381, 382, 385, 388, 556, 688, 691, 843
relative epsilon, 711, 843
removal, 213, 217, 702, 703, 843
replica, 523, 831, 833, 843, 844, 848
replica divergence, 844
request-aware routing, 844
resolver, 844
resources, 263, 844, 847
returned, 22, 51, 229, 232, 234, 322, 575, 583, 586, ...
ring buffer, 833, 844
role interface, 844
root, 144, 145, 182, 184, 185, 190, 221, 224, 377, ...
rotation, 601, 610, 844, 859
routine, 234, 844, 855
row, 3, 4, 5, 7, 8, 13, 15, 17, 18, ...
run, 14, 16, 19, 20, 21, 22, 28, 29, 567, ...

S

same remainder, 398, 400, 401, 419, 484, 485, 489, 844, 856
schema, 65, 78, 98, 104, 108, 109, 110, 112, 119, ...
scoping, 844

script, 106, 111, 316, 323, 342, 344, 345, 351, 364, ...
seam, 79, 408, 409, 410, 411, 559, 564, 591, 693, ...
search space, 508, 509, 510, 513, 517, 532, 844
selectivity, 143, 149, 844
sentinel, 375, 376, 377, 419, 455, 459, 484, 835, 844, ...
separator, 185, 844
sequence diagram, 792, 797, 844, 852
serialisable, 298, 299, 300, 301, 302, 844, 851
service credits, 844
session, 225, 383, 385, 388, 465, 467, 468, 469, 470, ...
session guarantees, 845
set, 3, 6, 9, 20, 22, 27, 30, 44, 45, ...
shallow-copied, 845
shape, 2, 5, 6, 7, 9, 10, 15, 18, 19, ...
shard, 65, 188, 412, 438, 496, 499, 520, 564, 833, ...
shared lock, 327, 845
shared store, 845
singleton, 552, 564, 845
singly linked list, 845
slice, 172, 196, 281, 436, 438, 439, 443, 445, 570, ...
smallest, 4, 7, 199, 205, 210, 540, 542, 572, 633, ...
snapshot, 104, 105, 298, 299, 301, 837, 845
snipe, 845
sorted region, 845
sorting everything, 845
splits, 182, 183, 185, 188, 189, 191, 361, 363, 377, ...
spof, 837, 845
squares, 40, 43, 44, 45, 48, 50, 51, 53, 55, ...
stable, 24, 29, 31, 82, 88, 92, 93, 94, 97, ...
stack, 80, 321, 337, 351, 436, 437, 438, 440, 442, ...
stack frame, 514, 845
stack overflow, 845
state, 3, 8, 14, 15, 16, 19, 33, 37, 46, ...
statements, 147, 223, 224, 227, 229, 230, 231, 232, 233, ...
status, 20, 23, 24, 29, 31, 36, 37, 63, 65, ...
still one machine, 839, 846
stonith, 846
strategy, 219, 230, 554, 556, 557, 558, 559, 560, 586, ...
strictly shorter, 846
structural, 724, 843, 846
subarray, 161, 199, 206, 209, 210, 212, 214, 217, 235, ...
subject, 846
subscription list, 846
substring, 1, 206, 207, 240, 241, 252, 275, 315, 346, ...
subtree, 838, 843, 846
surge multiplier especially, 846
swap, 22, 79, 82, 83, 85, 86, 88, 90, 92, ...
system call, 846

T

table, 5, 7, 13, 15, 17, 18, 19, 21, 23, ...

tail call, 846
tail-recursive, 846
tcp, 493, 494, 500, 526, 818, 834, 846, 847
template, 535, 539, 540, 542, 543, 544, 562, 568, 571, ...
test double, 846
the return address, 846
the running median, 837, 846, 847
third table, 522, 840, 847
thread, 384, 389, 524, 559, 560, 584, 621, 816, 832, ...
three distinct orderings, 847
three-way partition, 847
tight coupling, 847
time-to-live, 382, 847
tiny constraint, 847
toast, 847
token, 312, 366, 702, 721, 722, 723, 724, 726, 727, ...
tombstone, 847
top-k problems, 837, 846, 847
total order, 847
transitions itself, 847
transitive dependency, 847
tree, 26, 64, 70, 140, 141, 144, 147, 148, 461, ...
ttl, 44, 98, 188, 242, 293, 327, 352, 358, 372, ...
tuple, 27, 49, 131, 587, 619, 623, 626, 627, 628, ...
two, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...
two implementations, 847

U

udp, 846, 847
under-fetching, 847

union filesystem, 848
upper bound, 292, 539, 541, 546, 548, 549, 562, 568, 569, ...
uri, 408, 436, 707, 846, 848, 852
url, 557, 560, 692, 727, 737, 791, 793, 794, 800, ...

V

value object, 587, 592, 594, 624, 661, 819, 848
version, 86, 88, 90, 92, 108, 133, 167, 171, 235, ...
vertex, 835, 841, 842, 848
vertical scaling, 848
victim, 331, 848
virtual address space, 848
virtual machine, 832, 842, 848
volatile, 848

W

weight, 3, 4, 79, 267, 303, 323, 379, 440, 441, ...
which class, 584, 586, 597, 848
which is hours, 578, 848
within, 17, 62, 132, 137, 138, 142, 143, 145, 147, ...
worst case, 181, 182, 402, 430, 461, 596, 607, 611, 614, ...
write skew, 295, 297, 299, 300, 302, 303, 305, 310, 526, ...
write-around with invalidation, 848
write-through, 848

Y

yes, 6, 40, 46, 74, 79, 111, 142, 143, 145, ...

Two subjects, every day, for one hundred and eighty days.

Most preparation teaches algorithms for three months, then starts again from nothing on system design. Interviews do not work that way. A single loop will ask you to reverse a linked list in one room and design a rate limiter in the next.

So this book teaches both, every day, for one hundred and eighty days. Written for someone starting from zero - not a graduate, not somebody brushing up - and finishing able to answer either kind of question out loud, to a stranger, without notes.

- ◆ One hundred and eighty chapters. Two lessons and a practice sheet in every one.
- ◆ The same nine sections every time, so you learn one reading rhythm and keep it.
- ◆ Every lesson opens with a story about a person, told without a single technical word.
- ◆ Every solution shown in full, built in fragments and then given complete.
- ◆ The arithmetic done out loud, and error messages printed exactly as they appear.
- ◆ Real interviewer phrasings, an opening script, the follow-ups, and a model answer.